Python 101: A Function-by-Function Code Walkthrough

Data preparation, demand estimation, and dynamic pricing

November 6, 2026

Contents

Python 101: learning to read this research code **10**
1. A program is a sequence of instructions . . . 10
2. Values, types, and containers . . . 11
3. Functions: inputs, work, and outputs . . . 11
4. Branches, loops, and scope . . . 12
5. Arrays: many numbers with one operation . . . 12
6. Masks and comparisons . . . 13
7. pandas tables and Polars query expressions . . . 13
8. Strings, regular expressions, and paths . . . 14
9. Numerical optimization and symbolic algebra are different . . . 14
10. Modules, entry points, and tests . . . 14
11. A tiny trace you can do on paper . . . 15

Worked reading lessons: unpacking the intimidating lines **16**
Lesson 1: a table pipeline is a sequence of ordinary calls . . . 16
Lesson 2: masks express a sentence one cell at a time . . . 17
Lesson 3: from named coefficients to a utility matrix . . . 17
Lesson 4: softmax, log probabilities, and zero counts . . . 17
Lesson 5: one household must keep its latent type across weeks . . . 18
Lesson 6: understanding the gradient's einsum without memorizing it . . . 18
Lesson 7: first-stage least squares does not mean matrix inversion . . . 19
Lesson 8: symbolic expressions are not hidden numerical functions . . . 19
Lesson 9: closures and the expanding price bracket . . . 19
Lesson 10: practice before editing the real scripts . . . 20

Current empirical workflow: separate switching and habit **21**
Lesson 11: two coefficients mean two independently estimated effects . . . 21
Lesson 12: a population distribution is not an estimation confidence interval . . . 21
Lesson 13: recover both household effects, but rank only by switching . . . 22
Lesson 14: three focal flavors, one inside-versus-outside calculation each . . . 23
Lesson 15: read backward induction in the order it is solved . . . 23
Lesson 16: reconstruct deleted inputs without estimating demand again . . . 24

data_code: source walkthrough **25**
data_code/cleaning_merging/master_dat.py . . . 25
Module setup and direct-execution blocks . . . 25
data_code/cleaning_merging/panel_concat.py . . . 33
Module setup and direct-execution blocks . . . 33
data_code/cleaning_merging/panel_convert.py . . . 35
Module setup and direct-execution blocks . . . 35
data_code/cleaning_merging/panel_dat.py . . . 40
Module setup and direct-execution blocks . . . 40
data_code/cleaning_merging/panel_merge.py . . . 40

Module setup and direct-execution blocks . . . 40
data_code/cleaning_merging/panel_week.py . . . 45
Module setup and direct-execution blocks . . . 45
data_code/cleaning_merging/retail_concat.py . . . 46
Module setup and direct-execution blocks . . . 46
data_code/cleaning_merging/retail_convert.py . . . 49
Module setup and direct-execution blocks . . . 49
data_code/cleaning_merging/retail_dat.py . . . 55
Module setup and direct-execution blocks . . . 55
data_code/cleaning_merging/retail_merge.py . . . 55
Module setup and direct-execution blocks . . . 56
data_code/recover_flavors.py . . . 61
Module setup and direct-execution blocks . . . 61
main . . . 62
data_code/summ_stats.py . . . 63
Module setup and direct-execution blocks . . . 63
normalize upcs . . . 64
normalized upc expr . . . 65
valid flavor labels . . . 66
scan purchases . . . 67
single person purchases . . . 70
repeated yogurt households . . . 71
household summary . . . 72
as lazy . . . 75
partition purchases . . . 76
household batches . . . 78
save table . . . 79
demographic tables . . . 80
scan reference . . . 83
recover flavor reference . . . 83
flavor reference . . . 87
yogurt weeks . . . 91
flavor summaries . . . 93
make hhi figures . . . 99
make figures . . . 101
run . . . 104
main . . . 112

est_code: source walkthrough **116**
est_code/bootstrap_se.py . . . 116
Module setup and direct-execution blocks . . . 116
resample households . . . 116
bootstrap standard errors . . . 117
est_code/est.py . . . 121
Module setup and direct-execution blocks . . . 121
normalize upcs . . . 123
normalized upc expr . . . 124
read flavors . . . 125
week dates . . . 131
prepare data . . . 132
add stocks . . . 143
make arrays . . . 145
make draws . . . 148
parameter names . . . 149
mixture components . . . 152
mixture loglike score . . . 153

history utility . . . 155
household loglike . . . 156
demand objective gradient . . . 159
optimize demand . . . 163
optimize demand.objective . . . 167
optimize demand.progress . . . 168
plot gamma distribution . . . 169
plot history distributions . . . 173
fit model . . . 174
fit model.estimate . . . 179
main . . . 180
est_code/est_gamma.py . . . 184
Module setup and direct-execution blocks . . . 184
est_code/est_het_gamma.py . . . 185
Module setup and direct-execution blocks . . . 185
est_code/est_het_history.py . . . 185
Module setup and direct-execution blocks . . . 185
est_code/est_het_naive.py . . . 186
Module setup and direct-execution blocks . . . 186
est_code/est_mixture_gamma.py . . . 186
Module setup and direct-execution blocks . . . 186
est_code/est_naive.py . . . 187
Module setup and direct-execution blocks . . . 187
est_code/est_no_price_control.py . . . 188
Module setup and direct-execution blocks . . . 188
est_code/est_separate_history.py . . . 188
Module setup and direct-execution blocks . . . 188
est_code/hessian_se.py . . . 189
Module setup and direct-execution blocks . . . 189
observed covariance . . . 189
hessian standard errors . . . 192
est_code/history_distributions.py . . . 193
Module setup and direct-execution blocks . . . 193
export history distributions . . . 194
est_code/price_control.py . . . 198
Module setup and direct-execution blocks . . . 199
check dma . . . 199
other dma instruments . . . 200
absorb effects . . . 202
add price controls . . . 204
est_code/regenerate_weekly_inputs.py . . . 207
Module setup and direct-execution blocks . . . 207
regenerate . . . 208
main . . . 214
est_code/wtp.py . . . 214
Module setup and direct-execution blocks . . . 215
implied wtp . . . 215
implied wtp.add . . . 217

supply_code: source walkthrough **219**
supply_code/comparative_statics/___init___.py . . . 219
Module setup and direct-execution blocks . . . 219
supply_code/comparative_statics/alpha_p1.py . . . 219
Module setup and direct-execution blocks . . . 219
supply_code/comparative_statics/alpha_p2.py . . . 219
Module setup and direct-execution blocks . . . 220

supply_code/comparative_statics/beta_p1.py . . . 220
Module setup and direct-execution blocks . . . 220
supply_code/comparative_statics/beta_p1gamma.py . . . 220
Module setup and direct-execution blocks . . . 221
supply_code/comparative_statics/beta_p1p2.py . . . 221
Module setup and direct-execution blocks . . . 221
supply_code/comparative_statics/beta_p2.py . . . 221
Module setup and direct-execution blocks . . . 221
supply_code/comparative_statics/beta_profit.py . . . 222
Module setup and direct-execution blocks . . . 222
supply_code/comparative_statics/canonical.py . . . 222
Module setup and direct-execution blocks . . . 222
price2 derivative . . . 224
value2 derivative . . . 225
price1 derivative . . . 225
total derivative . . . 226
terminal total derivative . . . 227
profit derivative . . . 228
supply_code/comparative_statics/delta_p1.py . . . 229
Module setup and direct-execution blocks . . . 229
supply_code/comparative_statics/delta_p2.py . . . 230
Module setup and direct-execution blocks . . . 230
supply_code/comparative_statics/gamma2_p1.py . . . 230
Module setup and direct-execution blocks . . . 230
supply_code/comparative_statics/gamma2_p2.py . . . 230
Module setup and direct-execution blocks . . . 230
supply_code/comparative_statics/gamma_p1.py . . . 231
Module setup and direct-execution blocks . . . 231
supply_code/comparative_statics/gamma_p1beta.py . . . 231
Module setup and direct-execution blocks . . . 231
supply_code/comparative_statics/gamma_p1p2.py . . . 232
Module setup and direct-execution blocks . . . 232
supply_code/comparative_statics/gamma_p2.py . . . 232
Module setup and direct-execution blocks . . . 232
supply_code/comparative_statics/gamma_p2beta.py . . . 232
Module setup and direct-execution blocks . . . 232
supply_code/comparative_statics/gamma_profit.py . . . 233
Module setup and direct-execution blocks . . . 233
supply_code/comparative_statics_symbolic.py . . . 233
Module setup and direct-execution blocks . . . 233
supply_code/export/___init___.py . . . 234
Module setup and direct-execution blocks . . . 234
supply_code/export/latex_export.py . . . 234
Module setup and direct-execution blocks . . . 234
display equation . . . 234
supply_code/export/pretty_print.py . . . 235
Module setup and direct-execution blocks . . . 235
show . . . 235
supply_code/make_comparative_static_figures.py . . . 236
Module setup and direct-execution blocks . . . 236
state outcomes . . . 237
plot state outcomes . . . 238
plot state shares . . . 240
save . . . 241
main . . . 242
supply_code/make_latex_document.py . . . 249

Module setup and direct-execution blocks . . . 249
main . . . 250
supply_code/model/___init___.py . . . 257
Module setup and direct-execution blocks . . . 257
supply_code/model/assumptions.py . . . 257
Module setup and direct-execution blocks . . . 257
supply_code/model/numerical.py . . . 258
Module setup and direct-execution blocks . . . 258
inside share . . . 259
optimal price . . . 259
evaluate . . . 261
solve . . . 263
supply_code/model/profit.py . . . 264
Module setup and direct-execution blocks . . . 265
supply_code/model/shares.py . . . 265
Module setup and direct-execution blocks . . . 265
supply_code/model/symbols.py . . . 266
Module setup and direct-execution blocks . . . 266
supply_code/model/utility.py . . . 267
Module setup and direct-execution blocks . . . 267
signed state . . . 268
raw utilities . . . 268
supply_code/pricing/___init___.py . . . 269
Module setup and direct-execution blocks . . . 269
supply_code/pricing/period1_foc.py . . . 269
Module setup and direct-execution blocks . . . 269
supply_code/pricing/period1_price.py . . . 270
Module setup and direct-execution blocks . . . 270
supply_code/pricing/period2_foc.py . . . 270
Module setup and direct-execution blocks . . . 270
supply_code/pricing/period2_price.py . . . 271
Module setup and direct-execution blocks . . . 271
supply_code/run_all.py . . . 271
Module setup and direct-execution blocks . . . 271
supply_code/structural_switching_reference.py . . . 272
Module setup and direct-execution blocks . . . 272
expit safe . . . 273
inside share . . . 274
p2 opt . . . 275
evaluate . . . 276
verify . . . 278
plot series . . . 280
main . . . 281

pricing_exercises: source walkthrough **286**
pricing_exercises/run_empirical_targeting.py . . . 286
Module setup and direct-execution blocks . . . 286
expit . . . 287
logsumexp . . . 288
plotting . . . 289
persistent draws . . . 289
posterior . . . 290
compositions . . . 294
forecast context . . . 295
digest . . . 297
targeted coupons . . . 298

run households . . . 306
make targeting figures . . . 313
self test . . . 319
main . . . 328
pricing_exercises/run_pricing_exercises.py . . . 331
Module setup and direct-execution blocks . . . 331
main . . . 331
pricing_exercises/run_split_targeting.py . . . 333
Module setup and direct-execution blocks . . . 333
expit . . . 334
logsumexp . . . 335
plotting . . . 336
persistent draws . . . 336
posterior . . . 337
compositions . . . 341
forecast context . . . 342
digest . . . 344
targeted coupons . . . 345
run households . . . 353
make targeting figures . . . 361
self test . . . 367
main . . . 368
pricing_exercises/run_supply_exercises.py . . . 372
Module setup and direct-execution blocks . . . 372
expit . . . 373
golden maximum . . . 374
scalar maximum . . . 375
static price . . . 377
static price.foc . . . 379
supply solution . . . 379
supply solution.share1 . . . 384
supply solution.adaptive value . . . 385
supply solution.committed profile . . . 385
supply solution.committed profile.objective . . . 386
plotting . . . 387
outcome tables . . . 388
make figures . . . 391
self test . . . 395
main . . . 396

Current readable demand workflow (final_simple) 401
../final_simple/demand/estimation.py . . . 401
Module setup and direct-execution blocks . . . 401
check dma . . . 404
other dma instruments . . . 406
absorb effects . . . 408
add price controls . . . 410
normalize upcs . . . 413
normalized upc expr . . . 414
read flavors . . . 415
week dates . . . 421
prepare data . . . 422
add stocks . . . 433
make arrays . . . 436
make draws . . . 439
parameter names . . . 440

mixture components . . . 442
mixture loglike score . . . 443
history utility . . . 446
household loglike . . . 447
demand objective gradient . . . 450
optimize demand . . . 454
optimize demand.objective . . . 458
optimize demand.progress . . . 459
resample households . . . 460
bootstrap standard errors . . . 461
observed covariance . . . 465
hessian standard errors . . . 467
implied wtp . . . 468
implied wtp.add . . . 471
plot gamma distribution . . . 471
export history distributions . . . 476
plot history distributions . . . 480
write run notes . . . 481
fit model . . . 481
fit model.estimate . . . 487
read command line . . . 487
run estimation . . . 490
../final_simple/demand/split_pricing.py . . . 492
Module setup and direct-execution blocks . . . 492
expit . . . 494
logsumexp . . . 495
plotting . . . 496
persistent draws . . . 497
posterior . . . 498
compositions . . . 502
forecast context . . . 502
digest . . . 505
targeted coupons . . . 506
run households . . . 515
make targeting figures . . . 523
self test . . . 529
read command line . . . 530
run pricing . . . 531
../final_simple/demand/regenerate_weekly_inputs.py . . . 534
Module setup and direct-execution blocks . . . 535
regenerate . . . 536
main . . . 541
../final_simple/demand/history_distributions.py . . . 542
Module setup and direct-execution blocks . . . 542
main . . . 543

Tests and verification: worked calling examples **546**
data_code/test_summ_stats_memory.py . . . 546
Module setup and direct-execution blocks . . . 546
MemorySummaryChecks.test single person records precede repeat week rule . . . 547
MemorySummaryChecks.test batches preserve global statistics . . . 548
MemorySummaryChecks.test demographics match previous fieldwise sort . . . 552
MemorySummaryChecks.test recovery preserves existing labels and uses earliest year . . . 554
est_code/test_est.py . . . 555
Module setup and direct-execution blocks . . . 556
EstimationChecks.frame . . . 557

EstimationChecks.test history excludes current counts and accumulates all occasions . . . 557
EstimationChecks.test xi sign and unlimited history . . . 558
EstimationChecks.test parquet pipeline and fit . . . 560
EstimationChecks.test parquet pipeline and fit.append . . . 564
EstimationChecks.test unavailable flavor is excluded from logit denominator . . . 565
EstimationChecks.test persistent household integration . . . 567
EstimationChecks.test separate history nests combined index . . . 569
EstimationChecks.test no control ignores residuals but keeps price . . . 570
EstimationChecks.test analytic gradient . . . 572
EstimationChecks.test mixture combines household likelihoods and nests normal . . . 573
EstimationChecks.test mixture fit outputs . . . 575
EstimationChecks.test four model specifications . . . 577
EstimationChecks.test mixed naive integrates only alpha . . . 578
EstimationChecks.test zero heterogeneity nests homogeneous model . . . 580
est_code/test_hessian_se.py . . . 581
Module setup and direct-execution blocks . . . 581
HessianChecks.test correlated information and household scaling . . . 581
HessianChecks.test boundary and indefinite information withheld . . . 583
est_code/test_price_control.py . . . 584
Module setup and direct-execution blocks . . . 584
synthetic panel . . . 585
PriceControlChecks.test excludes consumer dma even when exposure store elsewhere . . . 587
PriceControlChecks.test absorption matches explicit dummy ols . . . 589
PriceControlChecks.test first stage residuals orthogonal and in utility . . . 590
PriceControlChecks.test first stage uses only covered flavor prices . . . 592
PriceControlChecks.test resampling preserves histories and relabels duplicates . . . 594
PriceControlChecks.test bootstrap refits first stage and reports empirical sd . . . 595
PriceControlChecks.test bootstrap refits first stage and reports empirical sd.estimate . . . 597
PriceControlChecks.test controlled demand optimizer runs . . . 598
est_code/test_regenerate_weekly_inputs.py . . . 599
Module setup and direct-execution blocks . . . 599
RecoveryTests.check recovery . . . 600
RecoveryTests.test recovery preserves parameters . . . 602
RecoveryTests.test mismatch preserves existing export . . . 602
est_code/test_summary_stats.py . . . 603
Module setup and direct-execution blocks . . . 603
SummaryChecks.test demographics and mixed baskets . . . 604
est_code/test_wtp.py . . . 607
Module setup and direct-execution blocks . . . 607
WTPChecks.table . . . 608
WTPChecks.test fixed alpha and signed history . . . 608
WTPChecks.test mixture moments include between component variation . . . 609
WTPChecks.test separate history wtp has no fixed weight . . . 610
WTPChecks.test beta averages ratios . . . 611
supply_code/checks/___init___.py . . . 612
Module setup and direct-execution blocks . . . 612
supply_code/checks/check_alpha.py . . . 612
Module setup and direct-execution blocks . . . 612
supply_code/checks/check_beta.py . . . 613
Module setup and direct-execution blocks . . . 613
supply_code/checks/check_delta.py . . . 613
Module setup and direct-execution blocks . . . 613
supply_code/checks/check_focs.py . . . 614
Module setup and direct-execution blocks . . . 614
supply_code/checks/check_gamma.py . . . 614
Module setup and direct-execution blocks . . . 614

supply_code/checks/check_numerical.py . . . 615
Module setup and direct-execution blocks . . . 615
supply_code/checks/check_pricing_derivatives.py . . . 615
Module setup and direct-execution blocks . . . 615
supply_code/checks/test_symmetric.py . . . 616
Module setup and direct-execution blocks . . . 616
SymbolicChecks.equal . . . 617
SymbolicChecks.test raw normalization and both states . . . 618
SymbolicChecks.test focs from actual profit . . . 619
SymbolicChecks.test envelope and implicit gamma derivatives . . . 620
SymbolicChecks.test all parameter implicit derivatives . . . 621
NumericalChecks.test reference on entire benchmark grid . . . 622
NumericalChecks.test independent reoptimization and derivatives . . . 623
NumericalChecks.test independent reoptimization and derivatives.negative profit . . . 625
NumericalChecks.test all parameter and mixed derivatives numerically . . . 626
NumericalChecks.test all parameter and mixed derivatives numerically.arguments at . . . 628
NumericalChecks.test zero switching static case and symmetry . . . 629
pricing_exercises/test_empirical_timing.py . . . 630
Module setup and direct-execution blocks . . . 630
EmpiricalTimingTests.test first observation becomes t0 . . . 631
EmpiricalTimingTests.test p1 h0 then terminal p2 h1 matches scalar enumeration . . . 634
pricing_exercises/test_split_targeting.py . . . 637
Module setup and direct-execution blocks . . . 637
EmpiricalTimingTests.test first observation becomes t0 . . . 638
EmpiricalTimingTests.test p1 h0 then terminal p2 h1 matches scalar enumeration . . . 641
SplitPosteriorTests.fixture . . . 645
SplitPosteriorTests.test posterior matches scalar and estimator . . . 646
SplitPosteriorTests.test all focal exports and integrated types . . . 650
../final_simple/checks/test_split_pricing.py . . . 652
Module setup and direct-execution blocks . . . 652
EmpiricalTimingTests.test first observation becomes t0 . . . 653
EmpiricalTimingTests.test p1 h0 then terminal p2 h1 matches scalar enumeration . . . 656
SplitPosteriorTests.fixture . . . 660
SplitPosteriorTests.test posterior matches scalar and estimator . . . 661
SplitPosteriorTests.test three focal cases both type treatments . . . 665
../final_simple/checks/test_regenerate_weekly_inputs.py . . . 666
Module setup and direct-execution blocks . . . 666
RecoveryTests.check recovery . . . 667
RecoveryTests.test recovery preserves parameters . . . 669
RecoveryTests.test mismatch preserves existing export . . . 670

Coverage, rebuilding, and next steps **671**

Python 101: learning to read this research code

This is a separate companion to the economic and pipeline guides. Those guides explain the models. This book teaches you to read their implementation, beginning with Python punctuation and ending with each function and script in `data_code`, `est_code`, `supply_code`, and `pricing_exercises`, together with the current readable demand workflow in `final_simple/demand`. You do not need to understand all the economics before learning what a line of Python does.

The book contains the actual source, source line references, function-specific explanations, statement-by-statement reading notes, alternatives, and small examples. Nested helper functions and test methods are included. Scripts with no functions receive the same treatment at module level. Empty package files are identified explicitly. Source excerpts are a snapshot of the working tree at build time; they are not edited versions of the executable research code.

Current edition, November 6, 2026. For the revised empirical exercise, read worked lessons 11–16 after the foundations, then the chapters for `../final_simple/demand/estimation.py` and `split_pricing.py`. The split model estimates independent novelty and own-stock effects; novelty defines reporting groups while both effects enter demand. All three flavors are priced separately as the focal inside product. The regeneration and distribution-export chapters explain how to recover the deleted weekly file on the server using the saved fit. The original combined-gamma examples remain useful comparisons.

Read the foundations first, then the small `make_draws` and `signed_state` functions. After that, try `add_stocks`, `resample_households`, and `optimal_price`. Return to the long preparation functions after you can recognize loops, columns, masks, and returned values. The table of contents and PDF bookmarks let you find any function by file and name. Appendix tests are useful worked examples of how to call the code without the private datasets.

1. A program is a sequence of instructions

Python executes statements in order. An assignment evaluates its right-hand side, then binds its result to the name on its left:

```
packages = 3
price = 1.50
spending = packages * price
```

`spending` is now 4.5. The equals sign is an instruction to store a result; it is not a mathematical claim to solve. `spending == 4.5` instead asks a question and produces the Boolean value `True`.

Names such as `weekly` and `frame` are ordinary variable names chosen by the author. They have no special meaning to Python. A name can later refer to a different object. `frame = frame.copy()` first copies the old table and then rebinds `frame` to the new one. Assigning `other = frame` does not copy the table; it gives the same object another name.

`#` begins a comment outside a string. Python ignores the rest of that line. Triple-quoted text at the start of a module, class, or function is a docstring: it is documentation retrievable with `help`, not a hidden calculation.

2. Values, types, and containers

Example	Type	How this code uses it
`3`	Integer, `int`	Counts, loop limits, seeds
`3.0`, `.7`, `1e-6`	Floating-point number, `float`	Prices, coefficients, tolerances
`"berry"`	String, `str`	Column labels, model names, paths
`True`, `False`	Boolean, `bool`	Settings and validation
`None`	The special absence value	Optional input not supplied
`["berry", "plain"]`	List	Ordered, changeable sequence
`("j", "0")`	Tuple	Ordered collection, often used for fixed choices
`{"price": 1.5}`	Dictionary, `dict`	Named values looked up by key
`{"upc", "flavor"}`	Set	Unordered distinct labels and membership checks

Square brackets have several jobs. In `[1, 2]` they create a list. In `items[0]` they select an item. In `frame["price"]` they select a named table column. Read the object to the left before deciding what the brackets mean.

Python indexes start at zero. For `names = ["other", "berry", "plain"]`, `names[0]` is other and `names[2]` is plain. `names[:2]` selects positions zero and one; slice stops are excluded. `names[1:]` starts at berry and goes to the end. A NumPy slice such as `prices[:, j]` means every row of column j.

`dict(zip(names, theta))` pairs labels with values. If the labels are `["beta", "gamma"]` and values `[2.0, -.5]`, the result is `{"beta": 2.0, "gamma": -.5}`. Ordinary `zip` stops at the shorter input; the surrounding code is responsible for consistent lengths. `p["gamma"]` requires the key; `p.get("gamma", 0)` supplies zero if it is absent.

`a, b = (10, 20)` unpacks two returned values. `_` is a conventional name for an ignored value; Python does not magically discard it. In `fit, _, _ = ...`, the author keeps the fit and does not use the other returned objects.

3. Functions: inputs, work, and outputs

```python
def package_cost(quantity, price=1.0):
    total = quantity * price
    return total

answer = package_cost(3, price=1.5)
```

`def` defines a function. The name is `package_cost`; parentheses contain its parameters. The colon introduces the body. Indentation determines which statements belong to that body. Defining the function does not yet execute its body. Calling it with parentheses does.

In the call, 3 is a positional argument and `price=1.5` is a keyword argument. The `=` inside a call names an argument; it is not assignment to a variable outside the function. `price=1.0` in the definition supplies a default when the caller omits that argument. `return` ends this call and gives its value back to the caller. Printing 4.5 displays text; returning 4.5 lets another calculation use the number. A function that falls off its end returns `None`.

`def solve(*, initial_state="0", **parameters)` contains two more pieces of syntax: bare `*` makes following arguments keyword-only; `**parameters` collects additional named arguments into a dictionary. In a call, `evaluate(gamma, state, **parameters)` expands such a dictionary back into named arguments. `*command` in a list expands a sequence into its elements.

Functions defined inside another function are nested functions. They can read names from the surrounding call. In `optimize_demand`, the nested `objective` remembers `data`, `draws`, and `args`; SciPy only has to pass the changing parameter vector. This is a closure. It avoids global variables, whose contents could accidentally affect another fit.

`lambda p: p * p` is a compact function returning the square of p. It is handy for a one-expression root equation. A named `def` is clearer if the calculation needs several steps, comments, or reuse. A function object can itself be passed as an argument: `brentq(f, lo, hi)` asks a solver to call f repeatedly.

4. Branches, loops, and scope

```
if quantity <= 0:
    raise ValueError("Quantity must be positive")
else:
    print("Usable quantity")
```

`if` evaluates a condition, `elif` tries another condition when the previous one was false, and `else` handles the remaining case. `raise` stops normal execution by creating an exception. Rejecting unusable counts is preferable to quietly turning them into valid-looking zeros.

`for j, name in enumerate(FLAVORS):` repeats a block once for each flavor, providing both its numeric position and its label. It is equivalent to looping over positions and looking up labels, but makes the correspondence explicit. `range(3)` produces 0, 1, 2. A `while` loop instead keeps going while its condition is true; the price solver uses it because it does not know in advance how many bracket expansions are needed.

`["price_" + f for f in FLAVORS]` is a list comprehension. A slower-to-write but equally valid spelling is:

```
columns = []
for flavor in FLAVORS:
    columns.append("price_" + flavor)
```

An optional `if` at the end filters elements. `{key: value for ...}` builds a dictionary. `(value for ...)` is a generator expression: it yields values as needed rather than first making a complete list. `next(generator, None)` takes the first yielded value, or returns `None` if no match exists.

`try` executes code that might fail. A matching `except` handles named errors. The bootstrap records identification failures rather than accepting them as estimates. Catching every possible exception would also hide programming bugs, so it catches specific expected failure types.

5. Arrays: many numbers with one operation

NumPy arrays are rectangular numerical containers. Their `shape` is a tuple of axis lengths. A three-week, three-flavor table has shape `(3, 3)`. Axis zero indexes rows; axis one indexes columns. Consequently, `.sum(axis=1)` adds across flavors for each week, while `.sum(axis=0)` adds across weeks for each flavor.

```
import numpy as np
counts = np.array([[2., 0., 1.], [0., 4., 0.]])
totals = counts.sum(axis=1)             # [3., 4.]
shares = counts / totals[:, None]
```

`totals[:, None]` changes a shape `(2,)` vector into a `(2, 1)` column. NumPy then broadcasts each row total across its three columns. Without that extra axis, `(2,3) / (2,)` cannot broadcast: the trailing dimensions 3 and 2 conflict. Broadcasting is controlled shape matching, not a guess at intended economics.

Similarly, adding a three-element beta vector to a W-by-3 utility matrix adds the flavor intercepts to every row. Array multiplication `*` is elementwise. Matrix multiplication `@` sums products over the matching inner dimension. For one-dimensional vectors it is the dot product, e.g. `[1,2] @ [3,4] = 11`. `**2` squares a value; `^` is not Python's exponentiation operator.

`np.zeros((W,3))` allocates an array of zeros. `np.zeros_like(prices)` matches another array's shape and dtype. Preallocating avoids repeatedly growing an array inside a long loop. `np.column_stack` joins columns; `np.vstack` stacks rows. `np.bincount(hh, weights=row_ll)` sums contributions by nonnegative integer household label, avoiding a slower Python loop over each household.

`np.nan` represents a missing floating-point value; `np.inf` represents positive infinity. NaN is not equal to itself, so use `np.isnan` or table missing-value methods rather than `x == np.nan`. `np.isfinite` rejects both NaN and infinities.

6. Masks and comparisons

```
covered = np.isfinite(prices) & (prices > 0)
clean_prices = np.where(covered, prices, 0.)
```

A mask is an array of True/False values. `&` combines array conditions with elementwise AND; `|` is elementwise OR; `~` reverses Booleans. Put each comparison in parentheses because Python operator precedence otherwise changes the grouping. `and`, `or`, and `not` instead combine scalar truth values and short-circuit; they generally cannot combine an entire pandas Series unambiguously.

`.any()` asks whether at least one entry is true; `.all()` asks whether every entry is true. `available.any(axis=1)` asks separately for each row. A useful translation is: `((counts == 0) | available).all(axis=1)` means "every flavor was either unpurchased or available." It rejects a purchased flavor without a price.

`np.where` selects values element by element. It does not lazily evaluate its two alternatives: both argument expressions are evaluated before the call. Use safe denominators or explicit indexing when the unused branch would raise or perform an invalid calculation. A zero price placeholder is not a free good when the corresponding alternative is subsequently masked out of utility.

7. pandas tables and Polars query expressions

A pandas `DataFrame` is a table with named columns and row labels; a `Series` is a single labeled column. `frame["price"]` selects one Series; `frame[["price", "quantity"]]` selects a two-column DataFrame. `frame.loc[mask, "price"]` selects specified rows and one column. `frame.iloc[0]` selects by numeric row position. `.copy()` makes an independent table before modifications, avoiding accidental changes to the original.

`.groupby(keys)["quantity"].sum()` splits rows by common keys and sums within each group. `.transform("nunique")` broadcasts a group's distinct count back to every original row. In contrast, an ordinary group aggregation returns one row per group. `.unstack` and `.pivot` turn flavor labels into columns; `.reindex(columns=FLAVORS)` makes their ordering explicit for numerical arrays.

A merge is a lookup using matching key columns. `how="left"` keeps left rows, `inner` keeps matches, `semi` keeps only matching left rows without adding right columns, and `anti` keeps unmatched left rows. A duplicate right key can multiply rows. `validate="many_to_one"` checks the right side is unique; it does not force an invalid table to become unique. Deduplication and validation are different decisions.

Polars uses `pl.col("price")` to build a column expression. Think of it as a recipe, not yet a column of numbers. `pl.lit(0)` is a literal-valued expression; `.alias("price")` names an expression's output. In `pl.when(condition).then(a) .otherwise(b)`, each row chooses an outcome. `.with_columns` adds or replaces columns, while `.select` projects only the requested columns/expressions.

`pl.scan_parquet(path)` builds a lazy plan. `.filter`, `.select`, and `.join` extend that plan. `.collect()` executes it and materializes a table in memory; `.sink_parquet(path)` executes it while writing output. This allows early projection/filtering and streaming execution. It does not guarantee tiny memory: joins, sorts, and exact distinct counts still need state. `.to_pandas()` converts to an eager pandas table, appropriate for the later NumPy computations.

Method chaining means "take the result so far, then call the next method":

```
cleaned = frame.dropna().sort_values("week").reset_index(drop=True)
```

The code often places the chain inside parentheses so it can span physical lines. Leading dots on later lines continue the same expression; they are not new statements. Nested function calls evaluate their arguments before the outer call. Starting with the innermost parentheses is often the easiest reading strategy.

8. Strings, regular expressions, and paths

`"price_" + flavor` concatenates text. `f"Start {s + 1}"` inserts an expression into a formatted string. `:.4f` means four digits after the decimal point, `:,` adds thousands separators, and `:.1%` displays a fraction as a percentage. `flush=True` asks printed progress to reach the log promptly instead of waiting in a buffer.

Regexes describe string patterns. In `r"[0-9]+(?:\.0+)?"`, `[0-9]+` means one or more digits, `(?:...)` groups without capturing, `\.` is a literal dot, and `?` makes the decimal-zero suffix optional. The raw-string prefix r keeps Python from processing backslashes first. `^` and `$` anchor start and end. The UPC normalization removes only a numeric `.0` suffix, not every decimal point or arbitrary letters. String handling avoids rounding a large UPC through a floating-point number.

`Path("/scratch") / "Data"` joins filesystem components. The slash is overloaded by `Path`; here it does not divide numbers. `.exists()` checks presence, `.mkdir(parents=True, exist_ok=True)` creates needed directories, and `.resolve()` normalizes a path for comparisons. A `with TemporaryDirectory() as tmp:` block cleans up the temporary directory on exit, including ordinary exception exits. `with path.open("w") as file:` similarly ensures the file is closed.

9. Numerical optimization and symbolic algebra are different

`minimize(objective, initial, ...)` repeatedly calls a numerical function, changing an array of parameter values to lower its returned loss. `jac=True` means this objective supplies both the number and its derivative vector. `brentq(f, lo, hi)` finds a zero of a continuous scalar function with a sign-changing bracket; it does not minimize the function. A root of a profit derivative is a candidate optimum, whose optimality needs the argument in the economic guide.

`sp.symbols("alpha beta")` creates symbolic mathematical placeholders. It does not create estimated numerical values. `sp.diff(expr, alpha)` differentiates algebra; `.subs({alpha: -1})` substitutes; `sp.simplify` tries to reduce an expression. `sp.lambdify(arguments, expression, "numpy")` turns that symbolic formula into a callable numerical function with an explicit argument order.

A crucial trap: SymPy treats distinct symbols as independent unless dependence is encoded. If `s` is a separate symbol, differentiating `s*p` with respect to p gives s, not `s + p*ds/dp`. The supply code's total-derivative helper manually adds that chain-rule contribution for equilibrium shares and prices.

For likelihood calculations, `logsumexp` computes the logarithm of a sum of exponentials stably. `np.log(np.exp(x).sum())` can overflow even when its final mathematical answer is finite. The optimizer would then receive unusable values. The analytic gradient is exact for the fixed simulated draws, while the simulation integral itself still approximates the population integral.

10. Modules, entry points, and tests

`import numpy as np` imports a library and gives it a short local name. `from price_control import check_dma` imports one object from a neighboring module. Imports can execute a module's top-level code. That is why the legacy cleaning scripts should not be imported merely for inspection: they perform data work at module level.

```
if __name__ == "__main__":
    main()
```

When Python runs a file directly, its `__name__` is `"__main__"`, so this block calls its entry point. When another file imports it, the guard is false and the guarded work is skipped. ArgumentParser translates command-line text into attributes such as `args.draws`; `type=int` converts text to integers and `action="store_true"` creates a Boolean flag.

A class groups behavior. `class Checks(unittest.TestCase):` inherits the testing tools. A method is a function attached to a class; `self` refers to the current instance and is supplied automatically in calls such as `self.frame()`. `self.assertEqual(actual, expected)` fails the test if the values differ. Floating-point results need tolerances: `.1 + .2` is generally not represented exactly as .3. `assert_allclose` compares arrays element by element with stated

absolute/relative tolerances. A passing test confirms its constructed case, not every possible input or the truth of an economic assumption.

11. A tiny trace you can do on paper

```
stock = np.zeros(3)
counts = np.array([0., 2., 0.])
share = counts / counts.sum()
before = stock.copy()
stock = stock + share
```

Initially stock is `[0,0,0]`. Counts sum to 2, so share is `[0,1,0]`. `before` preserves zero history; stock becomes `[0,1,0]`. The order explains why the current purchase does not enter its own pre-choice state. Reversing the last two statements would change the statistical model, even though both versions are valid Python.

Try these questions before looking back: what does `np.arange(4)` return? Why is `frame[["price"]]` different from `frame["price"]`? When does a nested objective run: when it is defined or when SciPy calls it? Does `new = old` copy a table? Does `.all(axis=1)` summarize rows or columns? The answers are 0–3, table versus Series, when called, no, and one result per row after combining columns. These are the building blocks used repeatedly below.

Worked reading lessons: unpacking the intimidating lines

The file-by-file chapters below are a reference. These short lessons demonstrate how to slow down and reason through a dense expression. The expanded versions are teaching equivalents, not suggested edits to the production code.

Lesson 1: a table pipeline is a sequence of ordinary calls

The estimator contains this expression:

```
weekly = (lines.groupby(keys + ["flavor"])["quantity"].sum()
          .unstack("flavor", fill_value=0)
          .reindex(columns=FLAVORS, fill_value=0).reset_index())
```

Read left to right after identifying the innermost list expression:

1. `keys + ["flavor"]` concatenates the household/week key list with one more column label. The plus sign joins lists; it is not adding column values.
2. `lines.groupby(...)` splits purchase rows into household/week/flavor groups.
3. `["quantity"]` selects the quantity Series from that grouped object.
4. `.sum()` adds package counts within each group. Two separate berry purchases in the same household-week become one berry total.
5. `.unstack("flavor", fill_value=0)` moves the flavor part of the grouped index into columns. An unobserved flavor within an existing week gets zero count.
6. `.reindex(columns=FLAVORS, fill_value=0)` fixes the columns to other, berry, plain, in that order, adding any completely absent flavor with zeros.
7. `.reset_index()` turns household/week index levels into ordinary columns.
8. `weekly = ...` names the resulting table only after the whole chain completes.

You could write intermediate objects instead:

```
group_columns = keys + ["flavor"]
grouped = lines.groupby(group_columns)["quantity"]
totals = grouped.sum()
wide = totals.unstack("flavor", fill_value=0)
ordered = wide.reindex(columns=FLAVORS, fill_value=0)
weekly = ordered.reset_index()
```

Both forms express the same intended pandas operations. Intermediates are useful while learning or debugging because you can inspect each type and shape. Chaining keeps a completed transformation compact once its stages are familiar. It is not intrinsically more statistically correct, and it is not automatically lazy just because it is written across several lines.

For a household with berry purchases 2 and 1 and plain purchase 4 in one week, the final count columns are `(other=0, berry=3, plain=4)`. Index order becomes important only when those columns are converted to position-based arrays.

Lesson 2: masks express a sentence one cell at a time

```
complete = available.any(axis=1) & ((counts == 0) | available).all(axis=1)
```

Suppose one week has counts `[0,2,1]` and availability `[True,True,False]`. The zero-count comparison gives `[True,False,False]`. OR with availability gives `[True,True,False]`: the plain purchase has no price. `.all(axis=1)` therefore gives False for that row. Although `.any(axis=1)` confirms a nonempty menu, AND combines it with False, so the week is rejected.

If plain count were zero, the OR vector would be all True, so the missing plain price would be acceptable: plain can be excluded from the choice set. This distinguishes "unpriced and unpurchased" from "unpriced but purchased."

The expression could be expanded into a row loop and an inner flavor loop:

```
complete_rows = []
for row_counts, row_available in zip(counts, available):
    nonempty_menu = any(row_available)
    every_purchase_covered = all(
        q == 0 or offered
        for q, offered in zip(row_counts, row_available)
    )
    complete_rows.append(nonempty_menu and every_purchase_covered)
```

The loop uses scalar `or` and `and` because q and offered are individual values. The production expression uses `|` and `&` because its comparisons concern whole arrays. Vectorization moves the element loops into NumPy, but the logical rule remains the same. Parentheses are necessary: do not rely on English-like operator precedence in an array expression.

Lesson 3: from named coefficients to a utility matrix

```
p = dict(zip(parameter_names(model, data["outside"], data["control"]), theta))
beta = np.array([0.0, p["beta_berry"], p["beta_plain"]])
```

The first call obtains parameter labels for this particular model. `zip` pairs each with the coefficient in the same position; `dict` makes the lookup mapping. For a naive three-parameter model, `[.4, -.2, 0.]` corresponds to berry intercept .4, plain intercept -.2, and mean log-minus-alpha zero. It does not mean alpha is zero: the later transform is `-exp(0) = -1`.

With no gamma or price-control effect and one week's prices `[1,1.2,1.5]`, the utility calculation is

```
beta = np.array([0., .4, -.2])
alpha = -1.
prices = np.array([[1., 1.2, 1.5]])
utility = beta + alpha * prices
# utility is [[-1., -.8, -1.7]]
```

Beta has shape `(3,)`, prices `(1,3)`, and the output `(1,3)`. Each flavor's intercept is added to that flavor's price term. Alpha's negative sign means higher prices lower utility. The other-flavor intercept is fixed at zero as a normalization; that does not imply other flavor has zero total utility.

Why not use positions such as `theta[0]` everywhere? That is possible, but a different model inserts/removes coefficients and can change positions. A single ordered label helper plus dictionary lookup makes the interpretation visible. The numeric values remain in an array for efficient optimization.

Lesson 4: softmax, log probabilities, and zero counts

```
logprob = utility - logsumexp(utility, axis=1, keepdims=True)
row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
```

For logit, the logarithm of a probability is utility minus the log of the sum of exponentiated utilities. `axis=1` performs that denominator calculation across alternatives within each week. `keepdims=True` preserves a W-by-1 column so it can be subtracted from the W-by-J utility matrix.

An unavailable alternative has utility minus infinity and probability zero. Its log probability is minus infinity. A zero purchase count should contribute zero to the log likelihood, but floating point computes `0 * -inf` as NaN. The `where` replaces that log probability with zero **where count is zero** before multiplying, protecting the valid zero contribution.

For counts `[0,2,1]`, the row contribution is two times berry log probability plus one times plain log probability. This is why changing an actual package count changes the likelihood's weight; it is not just a single flavor label. Subtracting raw `np.exp(utility).sum()` rather than its log would be the wrong formula, and computing exp then log naively can overflow. `logsumexp` handles the same mathematics in a stable numerical form.

Lesson 5: one household must keep its latent type across weeks

Assume two possible types, equally weighted. Type A gives the household's chosen flavor probability .9 in each of two weeks; type B gives .1 in each. Persistent types imply the history probability

```
likelihood = (.9 * .9 + .1 * .1) / 2
# .41
```

Averaging probabilities independently each week instead gives

```
wrong_history = ((.9 + .1) / 2) * ((.9 + .1) / 2)
# .25
```

The difference is not rounding: it is whether the household can redraw its type between weeks. The estimator first adds weekly log likelihoods within household for each draw, then integrates those full-history likelihoods. `np.bincount` performs the within-household sum. A list accumulates one household vector per draw. The final `logsumexp(..., axis=0) - log(R)` integrates the draw dimension while preserving the household dimension.

This also explains why the gradient uses posterior draw weights. A type assigning higher probability to the entire observed history receives greater weight in that household's score. Those weights are calculation intermediates, not new estimated household coefficients written by the program.

Lesson 6: understanding the gradient's einsum without memorizing it

```
gradient = -np.einsum("rh,rhp->p", weights, np.asarray(scores)) / data["n_households"]
```

The quoted letters label axes; they are not Python variable names. `r` is draw, `h` household, and `p` parameter. Weights has shape `(R,H)`, scores `(R,H,P)`. Multiply matching draw/household entries, sum over r and h because they do not appear after the arrow, and retain p. The resulting vector has length P.

Here is the same sum written in a teaching loop:

```
gradient = np.zeros(P)
for parameter in range(P):
    for draw in range(R):
        for household in range(H):
            gradient[parameter] += (
                weights[draw, household] * scores[draw, household, parameter]
            )
gradient = -gradient / H
```

The negative sign converts a log-likelihood score to a negative-log-likelihood gradient. Division by H matches the mean loss returned to the optimizer. The compact NumPy form avoids Python executing three nested numerical loops, while the teaching loop makes the axis meaning explicit. In this example P/R/H are dimension labels you would obtain from the actual arrays, not existing global constants in the source.

Lesson 7: first-stage least squares does not mean matrix inversion

```
z_partial = z - controls @ np.linalg.lstsq(controls, z, rcond=None)[0]
```

Starting inside, `lstsq(controls,z,...)` fits z on the control columns. Its return contains coefficients plus diagnostics, so `[0]` extracts the coefficient vector. Matrix multiplication produces fitted z, then subtraction leaves the part not linearly explained by those controls. `rcond=None` requests NumPy's default singular-value cutoff convention for that calculation.

For a three-row example, let controls be a column of ones and z `[1,2,6]`. Least squares estimates the mean 3, fitted values `[3,3,3]`, and residual `[-2,-1,3]`. The production controls are Xi after fixed-effect absorption, but the projection arithmetic has the same structure.

The alternative formula involving an inverse of `X.T @ X` requires nonsingular cross-products and can be numerically less stable. `lstsq` handles rank deficiency using its numerical decomposition. It cannot create excluded-instrument variation where none exists; the following sum-of-squares check rejects that case.

Lesson 8: symbolic expressions are not hidden numerical functions

```
alpha, beta, gamma = sp.symbols("alpha beta gamma", real=True)
price = sp.Symbol("price", real=True)
index = beta + alpha * price + gamma
direct = sp.diff(index, gamma)
```

`direct` is the symbolic number 1. No household data, optimizer, or probability is involved. If a separate symbol s is introduced, SymPy does not know that s depends on price unless the dependence is encoded. This is why the supply code implements explicit total-derivative rules.

`sp.lambdify((alpha,beta,gamma,price), index, "numpy")` builds a callable expecting four numbers in that order. A call with `(-1,2,.5,1)` gives 1.5. Passing beta where alpha belongs gives a different value without necessarily raising an error. Named dictionaries at output do not remove the need to keep this positional bridge correct. Symbolic algebra improves formula reuse, while numerical tests at asymmetric states catch argument-order mistakes.

Lesson 9: closures and the expanding price bracket

```
f = lambda p: alpha*(p-effective_cost) + 1 + np.exp(
    np.clip(intercept + alpha*p, -745, 700)
)
lo = effective_cost
width = max(2., 2/abs(alpha))
hi = lo + width
while f(hi) > 0:
    width *= 2
    hi = lo + width
```

Only p is written as a lambda parameter; alpha, effective cost, and intercept come from the enclosing call. Evaluating `f(hi)` substitutes the current hi without changing those fixed inputs. Under the maintained negative-alpha domain, f starts positive at lo and eventually becomes negative as p increases. Doubling width searches for a reliable sign-changing bracket rather than guessing a universal maximum price.

Once bracketed, Brent's root method solves f(p)=0. It is solving the transformed FOC, not trying to make the price itself zero. A loop that forgot to update hi could never change its condition; a loop with `width += 2` would expand more slowly for far-away roots. The mathematical guide establishes why this unique FOC root is the optimum for the current unconstrained logit problem.

Lesson 10: practice before editing the real scripts

For each exercise, predict the result before checking the answer:

1. For model `naive`, outside False, control True, what parameter names are returned? Answer: beta_berry, beta_plain, mu_log_minus_alpha, price_control.
2. If a household is selected twice by bootstrap, why change its ID twice? Answer: the two sampled histories must remain separate likelihood contributions.
3. What does `.copy()` protect when changing bootstrap IDs? Answer: the original household subtable that may be selected again in another draw.
4. Is a lazy query just a synonym for a small table? Answer: no; it is a plan, and executing it can still need large intermediate state.
5. In `return fitted, results, bounds`, why do the commas matter? Answer: they construct a tuple the caller can unpack; there are three outputs.
6. Why is saving `.tex` not the same as saving `.pdf`? Answer: the former is source text requiring a TeX engine to typeset it.
7. Can a zero outside count justify adding missing purchase weeks? Answer: no; the implemented sample contains only observed purchase weeks.
8. Does a first-stage F statistic establish exclusion? Answer: no; it describes conditional instrument relevance, while exclusion is an economic assumption.

You can learn most of the syntax by running tiny independent arrays and tables. Read the legacy scripts as text before executing them: several contain top-level filesystem writes and replacements. The following chapters reproduce that source without executing it. The numbered listings are fragments for reading; a fragment beginning inside a function is not necessarily runnable on its own.

Current empirical workflow: separate switching and habit

These lessons update the earlier combined-gamma examples. Begin with the readable files under `Code/final_simple/demand/`; the source chapters later in this book include both these files and their original `final_draft` counterparts. The old model remains a comparison, rather than the current empirical specification.

Lesson 11: two coefficients mean two independently estimated effects

The original combined index was

$$\Xi_{ijt} = (1-\lambda)N_{ijt} - \lambda S_{ijt}, \qquad V_{ijt} = \beta_j - a_i p_{ijt} + \gamma_i \Xi_{ijt} + \rho c f_{ijt}.$$

It forced novelty and own-stock coefficients to move together. The split model instead estimates

$$V_{ijt} = b_0 + \beta_j - a_i p_{ijt} + b_{Ni} N_{ijt} + b_{Si} S_{ijt} + \rho c f_{ijt}.$$

`N` is recent other-flavor purchase shares, `S` is accumulated own-flavor shares, and `a` is positive price sensitivity. A positive novelty coefficient rewards switching away from recently purchased rivals. Positive own-stock effects mean reinforcement; negative ones mean satiation. The name habit does not constrain its estimated sign. These are purchase histories, not observed consumption.

For example, let berry's N be .75 and S be 2, with b_N=.4 and b_S=-.1. The history part of berry utility is $.4.75\text{-}.12{=}.1$. You do not multiply these coefficients by .3 and -.7 again. Those weights belong to the older combined-index model.

In `final_simple/demand/estimation.py`, choose:

```
model = "heterogeneous_history"
capacity = 7
```

The estimator already supports this model. Updating distribution exports does not require changing its likelihood or re-estimating a saved fit. Capacity 7 adds constructed nonpurchase counts; it is a maintained participation convention, not observed purchase-free weeks. Capacity 0 remains available for conditional flavor demand, but the present pricing solver requires positive integer capacity.

Lesson 12: a population distribution is not an estimation confidence interval

The split population model uses independent normal novelty/stock effects and independent lognormal price sensitivity:

```
b_n = mean_novelty + sd_novelty * draws[:, 1]
b_s = mean_own_stock + sd_own_stock * draws[:, 2]
a = np.exp(mu_log_minus_alpha + sd_log_minus_alpha * draws[:, 0])
```

Every row of `draws` defines one persistent type across all weeks. The normal SDs describe variation across consumers, not uncertainty about the estimated population means. Hessian or bootstrap SEs answer the latter question.

For the previously saved controlled split fit, the exported novelty mean/SD are about -.4646/.8466 and own-stock mean/SD about -.01703/.09117. These are existing fit values, not results of a new optimization or new pricing run. Their fitted normal sign shares are about 70.84% and 57.41% below zero respectively. A negative novelty coefficient reverses the recent-other-flavor incentive; it need not imply positive cumulative reinforcement when the two effects are estimated separately.

`export_history_distributions` evaluates each normal's PDF and CDF, stores quantiles, and draws both marginal densities. With SD zero, it records a point mass instead of evaluating a normal density with zero denominator. It writes:

- `history_distribution_summary.csv`: parameters, sign shares and quantiles.
- `history_distribution_grid.csv`: density/CDF values for plotting.
- `history_distributions.pdf` and `.png`: the two population marginals.

The readable saved-fit entry point is `demand/history_distributions.py`. Existing plots are copied under `final_simple/demand/fitted_distributions/`. Their source fits and interpretation are recorded there. No household table is needed merely to export these marginal distributions from saved coefficients.

Lesson 13: recover both household effects, but rank only by switching

A household's posterior type probabilities use its complete history:

$$\ell_{id} = \sum_{t,k} n_{ikt} \log P_{ikt}(a_d, b_{Nd}, b_{Sd}), \qquad w_{id} = \frac{\exp \ell_{id}}{\sum_m \exp \ell_{im}}.$$

The numerical implementation subtracts log-sum-exp rather than exponentiating large log likelihoods directly. Add weeks before integrating draws; averaging weekly probabilities would redraw the person's type every week.

The split pricing script returns a D-by-2 array `history`. Column zero is b_N; column one is b_S. For one household:

```
weights = np.array([.75, .25])
history = np.array([[.4, -.1], [-.2, .3]])
mean_type = weights @ history        # [.25, 0.]
```

The posterior mean switching coefficient is .25 and mean stock effect is zero. The code ranks households by the first mean only, assigns bottom/top quarters, and keeps both means in utility. It does not set stock effects to zero just because stock is absent from the ranking variable. Posterior coefficient uncertainty can be correlated even when population integration draws are independent.

The default firm treats posterior means as known fixed types. The optional posterior-integrated exercise retains the joint draw weights and updates them after the first counterfactual basket. A terminal coupon must optimize revenue averaged over remaining unobserved types; optimizing separately for each true draw would give the firm information it does not have.

Type estimates use the full observed sample, but the first observed basket alone initializes counterfactual history. This is a retrospective exercise with known estimated types, not a prospective holdout forecast. Comparing novelty quartiles does not isolate novelty causally: their habits, prices and initial states differ.

Lesson 14: three focal flavors, one inside-versus-outside calculation each

For focal berry, combine other and plain as the outside alternative. Include constructed nonpurchase when present. If utilities are V_o,V_b,V_p,0, then

$$I_{-b} = \log\{e^{V_o} + e^{V_p} + 1\}, \qquad P_b = \text{logistic}(V_b - I_{-b}).$$

This equals berry's original multinomial-logit probability exactly. The outside utility is a log-sum-exp, not the arithmetic mean of rival utilities. With all four utilities zero, each original alternative has probability 1/4; the composite outside has utility log(3), so the binary formula also gives 1/4, not 1/2.

`focal_flavor = "all"` repeats the seller exercise for other, berry and plain. Only that run's focal product earns revenue and changes price. These are three separate counterfactuals with fixed competitors, not three firms solving a joint equilibrium. The solver internally uses coordinate one for the priced product. `run_households` swaps the focal product's counts, prices, histories, controls and intercept into that coordinate together. Household posterior types are recovered before that swap under the original fitted labels.

Combining rivals in a current choice probability does not erase their histories. Other and plain baskets can produce different future stocks and inclusive values. Keep their identities in basket integration. The seller observes the full basket before the terminal coupon; observing only focal sales would require additional beliefs about which rival flavors were bought.

Lesson 15: read backward induction in the order it is solved

There are three choice dates but two pricing decisions:

1. The first observed basket establishes initial h0 at model t=0.
2. The firm chooses p1(h0); a basket is realized and establishes h1.
3. It observes h1 and chooses terminal p2(h1).

For each basket n and each known estimated type, the terminal value is

$$W_2(n) = \max_{d_2} \; p(d_2) KP_{f,2}(n, d_2), \qquad p(d) = (1-d)p_{\text{posted}}.$$

The initial action solves

$$\max_{d_1} \; p(d_1) KP_{f,1}(d_1) + \delta \sum_n M(n \mid d_1) W_2(n).$$

Read `targeted_coupons` in `final_simple/demand/split_pricing.py`: the outer loop visits one household. STEP A calculates terminal outcomes for all coupons and baskets. STEP B calculates first-action basket probabilities and chooses each basket's best terminal coupon. STEP C picks the best first coupon. The terminal maximization happens before averaging over baskets. Both focal-purchase and no-focal-purchase continuation branches count, including all-nonpurchase baskets.

For basket shares x, update S2=S1+x and N2=sum(x)-x. All-nonpurchase means x=0, leaving stock unchanged and setting next novelty to zero. Capacity 7 yields 120 possible baskets. Updating only at expected shares would generally yield a different value function because logit demand and optimized prices are nonlinear.

The known-type, observed-state timing matches the supply baseline. Empirical binary aggregation is exact at a given full state, but it does not reduce the whole split-stock model to the theory's symmetric +gamma/-gamma one-lag model. The empirical solver also restricts prices to coupons from 0% through 100%, while theory allows unrestricted prices and introductory subsidies. Both new split pricing and theoretical supply default to delta=.92. With zero cost, net revenue is profit. Consumers follow current-period demand without strategically waiting.

Lesson 16: reconstruct deleted inputs without estimating demand again

The weekly export was deleted; regenerate it on the server where its underlying sample lives. `demand/regenerate_weekly_inputs.py` has three conceptual stages:

```
frame = prepare_data(saved_settings)
frame, first_stage = add_price_controls(frame)  # for a controlled fit
# Compare regenerated counts and the fixed-parameter likelihood with fit.json.
# Write weekly_inputs.csv only after both checks pass.
```

This schematic omits the implementation's guards and temporary-file publication. It does not call `optimize_demand`. Reconstructing price controls is necessary to recover the same regressors; it does not replace the saved demand estimates or standard errors. The procedure restores original years, sample selection, history weight, capacity, draw seed/count and model from the saved fit. Moved sample or flavor-reference paths can be overridden explicitly. A mismatch stops rather than silently pairing old coefficients with a different weekly sample.

The server job is `Code/final_simple/server/run_split_pricing.sh`. Its numbered steps regenerate the controlled fit's table, export both marginal distributions from controlled and uncontrolled fits, then run all three focal cases using the controlled fit. Sync code and install the listed requirements in the server environment before submitting:

```
sbatch Code/final_simple/server/run_split_pricing.sh
```

This command is an instruction for the server, not something the book builder executes. Reading or rebuilding this book parses source text only. Local validation used generated inputs with CSV reads blocked; full empirical pricing has not been run locally. The saved marginal distribution plots are not new pricing results.

Begin reading pricing outputs at the root `focal_flavor_comparison.csv` and `run_notes.txt`, then open each focal subfolder's posterior summaries, household policies, group/aggregate tables and figures. `focal_units` always means the selected run's product. The manifest maps rival state coordinates back to flavor names. Keep fit records beside outputs so these consistency checks remain possible.

data_code: source walkthrough

data_code/cleaning_merging/master_dat.py

This matches household purchases to retail quotes. The four-element key list is reused for projection, deduplication, and joins, reducing accidental differences between stages. The filtered retail yogurt table is collected once into memory; panel purchases are scanned and joined year by year to bound panel-side work. An inner join means a price match is a sample-selection condition.

Yearly results are staged as Parquet and scanned together into the final sample. `os.remove` deletes the generated temporary files and `os.rmdir` removes their directory after consolidation. These are real filesystem mutations. Final diagnostic queries return scalar counts or small tables. `next(..., None)` picks supported household/price names rather than assuming a suffix. Building all years eagerly in one pandas merge would be shorter but much more memory-hungry. Resolve its retail input path against the retail merge's actual output before running a fresh pipeline.

Module setup and direct-execution blocks

Source lines 1–7:

```
1 import glob
2 import os
3 import polars as pl
4
5 input_dir = "/scratch/dtm63837/Kilts_Panel/Data"
6 final_out = os.path.join(input_dir, "yogurt_estimation_sample.parquet")
7 temp_dir = os.path.join(input_dir, "temp_years")
```

Read this block:

- Line 1: import `glob`. An as-name is a convenient local alias, not another library.
- Line 2: import `os`. An as-name is a convenient local alias, not another library.
- Line 3: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 5, `input_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 6, `final_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 7, `temp_dir`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 8–16:

```
8 os.makedirs(temp_dir, exist_ok=True)
9
10 composite_keys = ["panel_year", "week_end", "upc", "store_code_uc"]
11
12 # 1. Load RMS with explicit column projection (ONLY load what you need)
```

```
13 print("[STEP 1/3] Materializing unique Yogurt RMS keys...", flush=True)
14
15 # Define exact RMS columns needed (join keys + filter col + price/movement cols)
16 rms_needed_cols = composite_keys + ["product_module_code", "price"]
```

Read this block:

- Line 8: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 10, `composite_keys`: Set this name to the list `['panel_year', 'week_end', 'upc', 'store_code_uc']`.
- Line 13: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 16, `rms_needed_cols`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 18–31:

```
18 rms_yogurt = (
19     pl.scan_parquet(os.path.join(input_dir, "merged_rms.parquet"))
20     # Select ONLY required columns up front (drops ~85% of memory loading)
21     .select([c for c in rms_needed_cols])
22     # Filter immediately
23     .filter(
24         pl.col("product_module_code").is_in([3603, 3612])
25     )  # Update to your yogurt code
26     # Deduplicate on composite keys
27     .unique(subset=composite_keys, keep="first")
28     .collect()
29 )
30
31 print(f"--> Yogurt RMS loaded successfully: {len(rms_yogurt):,} rows.", flush=True)
```

Read this block:

- Line 18, `rms_yogurt`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 31: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 34–39:

```
34 panel_years = list(range(2006, 2016))
35
36 print(
37     f"[STEP 2/3] Processing panel year-by-year across {len(panel_years)} years...",
38     flush=True,
39 )
```

Read this block:

- Line 34, `panel_years`: Set this name to the result of calling `list(...)`; the arguments and preceding method chain are shown above.
- Line 36: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `list`: Materialize iterable entries as an ordered Python list.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 41–61:

```
41 for idx, yr in enumerate(panel_years, 1):
42     yr_out = os.path.join(temp_dir, f"yogurt_{yr}.parquet")
43     print(
44         f"  --> [{idx}/{len(panel_years)}] Joining & streaming Year {yr}...",
45         flush=True,
46     )
47
48     # Scan HMS
49     hms_yr = pl.scan_parquet(os.path.join(input_dir, "merged_panel.parquet")).filter(
50         pl.col("panel_year") == yr
51     )
52
53     # Filter in-memory RMS to current year
54     rms_yr = rms_yogurt.filter(pl.col("panel_year") == yr)
55
56     # Stream inner join
57     hms_yr.join(
58         rms_yr.lazy(), on=composite_keys, how="inner", suffix="_rms"
59     ).sink_parquet(yr_out)
60
61     print(f"      [DONE] Saved Year {yr} chunk.", flush=True)
```

Read this block:

- Line 41: for each entry yielded by `enumerate(panel_years, 1)`, bind/unpack it into `(idx, yr)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 42, `yr_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 43: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49, `hms_yr`: Set this name to the result of calling `.filter(...)`; the arguments and preceding method chain are shown above.
- Line 54, `rms_yr`: Set this name to the result of calling `rms_yogurt.filter(...)`; the arguments and preceding method chain are shown above.
- Line 57: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 61: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.

Source lines 64–68:

```
print(
    "\n[STEP 3/3] Combining yearly files into final estimation sample...",
    flush=True,
)
pl.scan_parquet(os.path.join(temp_dir, "yogurt_*.parquet")).sink_parquet(final_out)
```

Read this block:

- Line 64: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 68: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 71–72:

```
for f in glob.glob(os.path.join(temp_dir, "yogurt_*.parquet")):
    os.remove(f)
```

Read this block:

- Line 71: for each entry yielded by `glob.glob(os.path.join(temp_dir, 'yogurt_*.parquet'))`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 72: call `os.remove` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- `.remove`: Delete the named file; in these scripts it cleans generated temporary year chunks.

Source lines 73–74:

```
if os.path.exists(temp_dir):
    os.rmdir(temp_dir)
```

Read this block:

- Line 73: test `os.path.exists(temp_dir)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 74: call `os.rmdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.rmdir`: Remove an empty directory after its temporary files have been deleted.

Source lines 76–79:

```
print(
    f"\n[SUCCESS] Saved 10-year Yogurt estimation sample to: {final_out}",
    flush=True,
)
```

Read this block:

- Line 76: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 84–91:

```
84 print("\n" + "=" * 70, flush=True)
85 print("RUNNING ESTIMATION SAMPLE AUDIT DIAGNOSTICS", flush=True)
86 print("=" * 70, flush=True)
87
88 # Scan final dataset lazily for memory-efficient auditing
89 sample_scan = pl.scan_parquet(final_out)
90 schema = sample_scan.collect_schema()
91 col_names = schema.names()
```

Read this block:

- Line 84: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 85: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 86: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 89, `sample_scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 90, `schema`: Set this name to the result of calling `sample_scan.collect_schema(...)`; the arguments and preceding method chain are shown above.
- Line 91, `col_names`: Set this name to the result of calling `schema.names(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.

Source lines 94–104:

```
94 total_rows = sample_scan.select(pl.len()).collect().item()
95 unique_upcs = sample_scan.select(pl.col("upc").n_unique()).collect().item()
96 unique_stores = sample_scan.select(pl.col("store_code_uc").n_unique()).collect().item()
97
98 # Check household column dynamically depending on HMS schema
99 hh_col = next((c for c in ["household_code", "hh_id"] if c in col_names), None)
100 unique_hhs = (
101     sample_scan.select(pl.col(hh_col).n_unique()).collect().item() if hh_col else "N/A"
102 )
103
104 print("\n[1] RECORD COUNTS & ENTITY COVERAGE:", flush=True)
```

Read this block:

- Line 94, `total_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 95, `unique_upcs`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 96, `unique_stores`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 99, `hh_col`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 100, `unique_hhs`: Set this name to the true-branch value if `hh_col` holds, otherwise the else-branch value.
- Line 104: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.
- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 105–116:

```
105 print(f"    - Total Estimation Rows:        {total_rows:,}", flush=True)
106 print(
107     f"    - Unique Households Matched:    {unique_hhs:,}"
108     if isinstance(unique_hhs, int)
109     else f"    - Unique Households Matched:    {unique_hhs}",
110     flush=True,
111 )
112 print(f"    - Unique Yogurt UPCs Matched:   {unique_upcs:,}", flush=True)
113 print(f"    - Unique Retail Stores Matched: {unique_stores:,}", flush=True)
114
115 # --- DIAGNOSTIC 2: Join Key Integrity & Null Check ---
116 print("\n[2] JOIN KEY INTEGRITY (NULL CHECK):", flush=True)
```

Read this block:

- Line 105: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 106: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 112: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 113: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 116: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 117–121:

```
117 null_counts = (
118     sample_scan.select([pl.col(k).null_count().alias(k) for k in composite_keys])
119     .collect()
120     .to_dict(as_series=False)
121 )
```

Read this block:

- Line 117, `null_counts`: Set this name to the result of calling `.to_dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.

Source lines 123–125:

```
123 for key, count in null_counts.items():
124     null_pct = (count[0] / total_rows) * 100 if total_rows > 0 else 0
```

```
125     print(f"    - Nulls in '{key}': {count[0]:,} ({null_pct:.2f}%)", flush=True)
```

Read this block:

- Line 123: for each entry yielded by `null_counts.items()`, bind/unpack it into `(key, count)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 124, `null_pct`: Set this name to the true-branch value if `total_rows > 0` holds, otherwise the else-branch value.
- Line 125: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 128–134:

```
128 print("\n[3] PANEL YEAR DISTRIBUTION:", flush=True)
129 year_counts = (
130     sample_scan.group_by("panel_year")
131     .agg(pl.len().alias("count"))
132     .sort("panel_year")
133     .collect()
134 )
```

Read this block:

- Line 128: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 129, `year_counts`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 136–140:

```
136 for row in year_counts.iter_rows(named=True):
137     print(
138         f"    - Year {row['panel_year']}: {row['count']:,} observations",
139         flush=True,
140     )
```

Read this block:

- Line 136: for each entry yielded by `year_counts.iter_rows(named=True)`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 137: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 143–143:

```
143 price_col = next((c for c in ["price", "price_rms"] if c in col_names), None)
```

Read this block:

- Line 143, `price_col`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Source lines 144–178:

```
144 if price_col:
145     price_stats = (
146         sample_scan.select(
147             [
148                 pl.col(price_col).min().alias("min_price"),
149                 pl.col(price_col).mean().alias("mean_price"),
150                 pl.col(price_col).median().alias("median_price"),
151                 pl.col(price_col).max().alias("max_price"),
152                 pl.col(price_col).null_count().alias("null_price_count"),
153                 (pl.col(price_col) <= 0).sum().alias("zero_price_count"),
154             ]
155         )
156         .collect()
157         .to_dict(as_series=False)
158     )
159 
160     print(
161         f"\n[4] PRICE COLUMN DIAGNOSTICS ('{price_col}'):",
162         flush=True,
163     )
164     print(f"    - Min Price:    ${price_stats['min_price'][0]:.2f}", flush=True)
165     print(f"    - Mean Price:   ${price_stats['mean_price'][0]:.2f}", flush=True)
166     print(
167         f"    - Median Price: ${price_stats['median_price'][0]:.2f}",
168         flush=True,
169     )
170     print(f"    - Max Price:    ${price_stats['max_price'][0]:.2f}", flush=True)
171     print(
172         f"    - Null Prices:  {price_stats['null_price_count'][0]:,}",
173         flush=True,
174     )
175     print(
176         f"    - Zero/Negative:{price_stats['zero_price_count'][0]:,}",
177         flush=True,
178     )
```

Read this block:

- Line 144: test `price_col`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 145, `price_stats`: Set this name to the result of calling `.to_dict(...)`; the arguments and preceding method chain are shown above.
- Line 160: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 164: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 165: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 166: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 170: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 171: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 175: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.median`: Compute the middle-value statistic rather than a mean; global medians cannot generally be averaged across batches.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 181–188:

```
181 file_size_mb = os.path.getsize(final_out) / (1024**2)
182
183 print("\n[5] OUTPUT FILE SUMMARY:", flush=True)
184 print(f"    - Total Schema Columns: {len(col_names)}", flush=True)
185 print(f"    - File Size on Disk:    {file_size_mb:.2f} MB", flush=True)
186 print(f"    - Output Location:      {final_out}", flush=True)
187
188 print("=" * 70, flush=True)
```

Read this block:

- Line 181, `file_size_mb`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 183: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 184: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 185: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 186: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 188: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.getsize`: Read filesystem byte size, later divided by powers of 1024 for MiB/GiB display.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 189–189:

```
189 print("AUDIT COMPLETE — Yogurt estimation sample verified.", flush=True)
```

Read this block:

- Line 189: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

data__code/cleaning__merging/panel__concat.py

This stacks annual cleaned panel tables into combined files. `concat_list` is an ordinary list of three table names. In the loop, an f-string builds a wildcard such as `cleaned_trips_*.parquet`. A lazy scan over those files followed by `sink_parquet` writes one combined table. It does not add rows by matching keys: concatenation stacks records, whereas a join combines columns through matches. There is no deduplication or explicit year reconciliation here. A stale matching file is therefore part of the input unless the directory is audited.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 import os
2 import polars as pl
```

Read this block:

- Line 1: import `os`. An as-name is a convenient local alias, not another library.
- Line 2: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 6–8:

```
6 base_out_dir = "/scratch/dtm63837/Kilts_Panel/Data"
7 final_dir = os.path.join(base_out_dir, "combined")
8 os.makedirs(final_dir, exist_ok=True)
```

Read this block:

- Line 6, `base_out_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 7, `final_dir`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 8: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 12–12:

```
12 concat_list = ["panelists", "purchases", "trips"]
```

Read this block:

- Line 12, `concat_list`: Set this name to the list `['panelists', 'purchases', 'trips']`.

Source lines 13–20:

```
13 for table in concat_list:
14     file_pattern = os.path.join(base_out_dir, f"cleaned_{table}_*.parquet")
15     concat_out = os.path.join(final_dir, f"{table}.parquet")
16 
17     print(f"Combining {table} files...")
18 
19     (pl.scan_parquet(file_pattern).sink_parquet(concat_out))
20     print(f"Saved: {concat_out}")
```

Read this block:

- Line 13: for each entry yielded by `concat_list`, bind/unpack it into `table` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 14, `file_pattern`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 15, `concat_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 17: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 19: call `pl.scan_parquet(file_pattern).sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 20: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

data_code/cleaning_merging/panel_convert.py

This script runs at module level: importing it can perform data work. First the imports make filesystem and Polars tools available; path strings select raw/server locations. Integer switches `annual` and `master` are truth-tested to decide which conversion loops run. `range(2006, 2016)` includes 2006 through 2015, excluding the upper endpoint. Dictionaries translate source table names to output stems. Nested loops construct one TSV/Parquet pair per year/table, and `exists` prevents trying to read missing files.

After conversion, wildcard scans discover Parquet files. `columns_to_keep` is a dictionary of table-specific projection lists, sometimes containing Polars alias expressions. `split("_")[0]` extracts a table prefix before selecting its projection. Scanning and sinking keep large tables out of ordinary Python lists; eagerly reading every annual table would need much more memory. Lowercasing column names standardizes downstream joins but does not reconcile every schema difference or verify raw-codebook meaning. No CLI parser protects the hard-coded paths; inspect them before running.

Module setup and direct-execution blocks

Source lines 2–4:

```
2 import glob
3 import os
4 import polars as pl
```

Read this block:

- Line 2: import `glob`. An as-name is a convenient local alias, not another library.
- Line 3: import `os`. An as-name is a convenient local alias, not another library.
- Line 4: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 8–14:

```
8 annual = 0
9 master = 0
10
11 years = range(2006, 2016)
12 base_raw_dir = "/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/nielsen_extracts/HMS"
13 base_out_dir = "/scratch/dtm63837/Kilts_Panel/Data"
14 os.makedirs(base_out_dir, exist_ok=True)
```

Read this block:

- Line 8, `annual`: Set this name to the literal value `0`.
- Line 9, `master`: Set this name to the literal value `0`.
- Line 11, `years`: Set this name to the result of calling `range(...)`; the arguments and preceding method chain are shown above.
- Line 12, `base_raw_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/`.
- Line 13, `base_out_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 14: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 16–21:

```
16 base_mapping = {
17     "panelists": "panelists",
18     "purchases": "purchases",
19     "trips": "trips",
20 }
21 master_mappings = {"products": "products", "retailers": "retailers"}
```

Read this block:

- Line 16, `base_mapping`: Set this name to a dictionary with named entries `panelists`, `purchases`, `trips`.
- Line 21, `master_mappings`: Set this name to a dictionary with named entries `products`, `retailers`.

Source lines 24–46:

```
24 if annual:
25     for y in years:
26         print(f"\n--- Converting year {y} ---")
27 
28         for base_tsv, out_name in base_mapping.items():
29             tsv_filename = f"{base_tsv}_{y}.tsv"
30             tsv_path = os.path.join(base_raw_dir, str(y), "Annual_Files", tsv_filename)
31             parquet_out = os.path.join(base_out_dir, f"{out_name}_{y}.parquet")
32 
33             if os.path.exists(tsv_path):
34                 (
35                     pl.scan_csv(
36                         tsv_path,
37                         separator="\t",
38                         quote_char=None,
39                         infer_schema_length=10000,
40                     )
41                     .rename(str.lower)
42                     .sink_parquet(parquet_out)
43                 )
44                 print(f"Converted: {tsv_filename} -> {out_name}_{y}.parquet")
45             else:
46                 print(f"WARNING: File not found {tsv_path}")
```

Read this block:

- Line 24: test `annual`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 25: for each entry yielded by `years`, bind/unpack it into `y` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 26: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 28: for each entry yielded by `base_mapping.items()`, bind/unpack it into `(base_tsv, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 29, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 30, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 31, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 33: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 34: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 44: call `print` with the displayed arguments. No assignment stores its return here; its documented side

effect or assertion is what matters.
- Line 46: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `str`: Convert to text for paths, messages, or serialization.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 47–64:

```
47 if master:
48     for base_tsv, out_name in master_mappings.items():
49         tsv_filename = f"{base_tsv}.tsv"
50         tsv_path = os.path.join(base_raw_dir, "Master_Files/Latest", tsv_filename)
51         parquet_out = os.path.join(base_out_dir, f"{out_name}.parquet")
52 
53         if os.path.exists(tsv_path):
54             (
55                 pl.scan_csv(
56                     tsv_path,
57                     separator="\t",
58                     quote_char=None,
59                     infer_schema_length=10000,
60                     encoding="utf8-lossy",
61                 )
62                 .rename(str.lower)
63                 .sink_parquet(parquet_out)
64             )
```

Read this block:

- Line 47: test `master`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 48: for each entry yielded by `master_mappings.items()`, bind/unpack it into `(base_tsv, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 49, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 50, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 51, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 53: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 54: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 68–68:

```
68 parquet_files = glob.glob(os.path.join(base_out_dir, "*.parquet"))
```

Read this block:

- Line 68, `parquet_files`: Set this name to the result of calling `glob.glob(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.

Source lines 70–74:

```
70 for filepath in sorted(parquet_files):
71     filename = os.path.basename(filepath)
72     cols = pl.scan_parquet(filepath).collect_schema().names()
73     print(f"\n--- {filename} ({len(cols)} columns) ---")
74     print(cols)
```

Read this block:

- Line 70: for each entry yielded by `sorted(parquet_files)`, bind/unpack it into `filepath` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 71, `filename`: Set this name to the result of calling `os.path.basename(...)`; the arguments and preceding method chain are shown above.
- Line 72, `cols`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 73: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 74: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `len`: Count elements/rows rather than summing their numerical values.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 76–116:

```
76 columns_to_keep = {
77     "panelists": [
78         pl.col("household_cd").alias("household_code"),
79         "panel_year",
80         "household_income",
81         "household_size",
82         "male_head_age",
83         "female_head_age",
84         "male_head_employment",
85         "female_head_employment",
86         "race",
87         "hispanic_origin",
88         "dma_cd",
89         "dma_name",
90     ],
91     "purchases": [
92         "trip_code_uc",
93         "upc",
94         "quantity",
95         "total_price_paid",
96         "deal_flag_uc",
97     ],
```

```
    "trips": [
        "trip_code_uc",
        "household_code",
        "purchase_date",
        "store_code_uc",
        "panel_year",
    ],
    "products": [
        "upc",
        "product_module_code",
        "product_module_descr",
        "product_group_code",
        "product_group_descr",
        "brand_code_uc",
        "brand_descr",
        "size1_amount",
        "size1_units",
    ],
}
```

Read this block:

- Line 76, `columns_to_keep`: Set this name to a dictionary with named entries `panelists`, `purchases`, `trips`, `products`.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 118–132:

```
for filepath in glob.glob(os.path.join(base_out_dir, "*.parquet")):
    filename = os.path.basename(filepath)

    # Remove .parquet extension first
    clean_name = os.path.splitext(filename)[0]  # e.g., "retailers" or "trips_2011"

    # Split by underscore if present to get the table name
    table_type = clean_name.split("_")[0]  # e.g., "retailers", "trips", "panelists"

    if table_type in columns_to_keep:
        target_cols = columns_to_keep[table_type]
        cleaned_path = os.path.join(base_out_dir, f"cleaned_{filename}")

        (pl.scan_parquet(filepath).select(target_cols).sink_parquet(cleaned_path))
        print(f"Cleaned {filename} -> saved to {cleaned_path}")
```

Read this block:

- Line 118: for each entry yielded by `glob.glob(os.path.join(base_out_dir, '*.parquet'))`, bind/unpack it into `filepath` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 119, `filename`: Set this name to the result of calling `os.path.basename(...)`; the arguments and preceding method chain are shown above.
- Line 122, `clean_name`: Set this name to the entry/slice selected by `os.path.splitext(filename)[0]`; brackets select from the object to their left.
- Line 125, `table_type`: Set this name to the entry/slice selected by `clean_name.split('_')[0]`; brackets select from the object to their left.
- Line 127: test `table_type in columns_to_keep`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 128, `target_cols`: Set this name to the entry/slice selected by `columns_to_keep[table_type]`; brackets select from the object to their left.

- Line 129, `cleaned_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 131: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 132: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

data_code/cleaning_merging/panel_dat.py

This file is empty in the source snapshot: it contains no function or executable data-processing block to explain.

Module setup and direct-execution blocks

There are no statements in this file.

data_code/cleaning_merging/panel_merge.py

Purchase lines anchor the merge. Join-key casts align integer dtypes before an inner trip join, then left joins attach annual household and UPC product metadata. Method chaining expresses a sequence of relational operations whose result is sunk to `merged_panel.parquet`. An inner join drops unmatched purchase lines; a left join retains them but may add missing metadata. Duplicate right keys can multiply rows in either case.

The second half scans the output and prints diagnostics. `collect().item()` extracts a scalar from a one-cell query result; `to_dict(as_series=False)` makes a dictionary of Python lists for null-count reporting. `count[0]` picks its only row. File byte size divided by `1024**3` expresses GiB. The empty-output check raises, but a nonempty result is not proof of correct joins. Explicit key validation would be a stronger alternative to relying only on total row ratios.

Module setup and direct-execution blocks

Source lines 1–9:

```
1 import os
2 import polars as pl
3
4 base_dir = "/scratch/dtm63837/Kilts_Panel/Data"
5 final_out = os.path.join(base_dir, "merged_panel.parquet")
6
7 ## === Load Data === ##
8 print("Loading lazy frames...")
9 panelists = pl.scan_parquet(os.path.join(base_dir, "combined/panelists.parquet"))
```

Read this block:

- Line 1: import `os`. An as-name is a convenient local alias, not another library.
- Line 2: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 4, `base_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 5, `final_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 8: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 9, `panelists`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 10–12:

```
10 purchases = pl.scan_parquet(os.path.join(base_dir, "combined/purchases.parquet"))
11 trips = pl.scan_parquet(os.path.join(base_dir, "combined/trips.parquet"))
12 products = pl.scan_parquet(os.path.join(base_dir, "cleaned_products.parquet"))
```

Read this block:

- Line 10, `purchases`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 11, `trips`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 12, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.

Source lines 16–30:

```
16 purchases = purchases.with_columns(
17     [pl.col("trip_code_uc").cast(pl.Int64), pl.col("upc").cast(pl.Int64)]
18 )
19
20 trips = trips.with_columns(
21     [
22         pl.col("trip_code_uc").cast(pl.Int64),
23         pl.col("household_code").cast(pl.Int64),
24         pl.col("panel_year").cast(pl.Int32),
25     ]
26 )
27
28 panelists = panelists.with_columns(
29     [pl.col("household_code").cast(pl.Int64), pl.col("panel_year").cast(pl.Int32)]
30 )
```

Read this block:

- Line 16, `purchases`: Set this name to the result of calling `purchases.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 20, `trips`: Set this name to the result of calling `trips.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 28, `panelists`: Set this name to the result of calling `panelists.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 32–44:

```
32 products = products.with_columns([pl.col("upc").cast(pl.Int64)])
33
34 ## === Merge === ##
35 merged = (
36     purchases.join(trips, on="trip_code_uc", how="inner")
37     .join(panelists, on=["household_code", "panel_year"], how="left")
38     .join(products, on="upc", how="left")
```

```
)

print("Merging purchases, trips, panelists, and products...")
print("Streaming to disk (this will take a few minutes)...")
merged.sink_parquet(final_out)
print(f"\n[SUCCESS] Successfully saved merged file to: {final_out}")
```

Read this block:

- Line 32, `products`: Set this name to the result of calling `products.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 35, `merged`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 41: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `merged.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 44: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 47–56:

```
print("\n" + "=" * 50)
print("RUNNING DIAGNOSTICS ON MERGED DATASET")
print("=" * 50)

# 1. Scan output dataset lazily
merged_scan = pl.scan_parquet(final_out)

# 2. Total Row Count Check
total_rows = merged_scan.select(pl.len()).collect().item()
print(f"\n[1] Total Row Count: {total_rows:,}")
```

Read this block:

- Line 47: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 48: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52, `merged_scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 55, `total_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 56: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 58–59:

```
58 if total_rows == 0:
59     raise ValueError("CRITICAL ERROR: Merged dataset contains 0 rows! Check join keys.")
```

Read this block:

- Line 58: test `total_rows == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 59: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 62–69:

```
62 purchases_rows = purchases.select(pl.len()).collect().item()
63 pct_retained = (total_rows / purchases_rows) * 100
64 print(f"    - Anchor purchases rows: {purchases_rows:,}")
65 print(f"    - Retained post-join: {pct_retained:.2f}%")
66
67 # 4. Null / NA Check across key columns
68 print("\n[2] Null Value Diagnostics (Key Columns):")
69 key_cols = ["trip_code_uc", "household_code", "upc", "panel_year"]
```

Read this block:

- Line 62, `purchases_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 63, `pct_retained`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 64: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 68: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 69, `key_cols`: Set this name to the list `['trip_code_uc', 'household_code', 'upc', 'panel_year']`.

Source lines 70–76:

```
70 existing_keys = [col for col in key_cols if col in merged_scan.collect_schema().names()]
71
72 null_counts = (
73     merged_scan.select([pl.col(c).null_count().alias(c) for c in existing_keys])
74     .collect()
75     .to_dict(as_series=False)
76 )
```

Read this block:

- Line 70, `existing_keys`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 72, `null_counts`: Set this name to the result of calling `.to_dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.

- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.

Source lines 78–80:

```
78 for col, count in null_counts.items():
79     null_pct = (count[0] / total_rows) * 100
80     print(f"    - {col}: {count[0]:,} nulls ({null_pct:.2f}%)")
```

Read this block:

- Line 78: for each entry yielded by `null_counts.items()`, bind/unpack it into `(col, count)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 79, `null_pct`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 80: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 83–89:

```
83 file_size_gb = os.path.getsize(final_out) / (1024**3)
84 schema_cols = merged_scan.collect_schema().names()
85
86 print("\n[3] Output File Summary:")
87 print(f"    - Total Columns: {len(schema_cols)}")
88 print(f"    - Disk Size: {file_size_gb:.2f} GB")
89 print(f"    - Columns: {schema_cols}")
```

Read this block:

- Line 83, `file_size_gb`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 84, `schema_cols`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 86: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 87: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 89: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.getsize`: Read filesystem byte size, later divided by powers of 1024 for MiB/GiB display.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 90–91:

```
90 print("=" * 50)
91 print("DIAGNOSTICS COMPLETE - File is verified and ready.")
```

Read this block:

- Line 90: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

data_code/cleaning_merging/panel_week.py

This adds a Saturday-ending integer key to the merged panel. The nested expression parses purchase-date strings, extracts weekdays, computes a duration, adds it, formats `%Y%m%d`, and casts to Int64. Read inside out. With Polars' Monday=1 through Sunday=7 convention, `6-(weekday % 7)` gives Monday five days, Saturday zero, and Sunday six days. It agrees with the downstream modulo-seven formula over those weekday values.

The query writes a separate temporary file before `os.replace` replaces the original. Writing directly to the source path while scanning it could corrupt the read; staging then replacement avoids that pattern. It still overwrites the original file after success, so this is not an inspection-only script.

Module setup and direct-execution blocks

Source lines 1–8:

```
1 import os
2 import polars as pl
3
4 input_dir = "/scratch/dtm63837/Kilts_Panel/Data"
5 panel_path = os.path.join(input_dir, "merged_panel.parquet")
6 panel_temp = os.path.join(input_dir, "merged_panel_updated.parquet")
7
8 print("Pre-computing week_end on merged_panel.parquet...")
```

Read this block:

- Line 1: import `os`. An as-name is a convenient local alias, not another library.
- Line 2: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 4, `input_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data'`.
- Line 5, `panel_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 6, `panel_temp`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 8: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 10–24:

```
10 (
11     pl.scan_parquet(panel_path)
12     .with_columns(
13         # Convert "YYYY-MM-DD" to Date, compute Saturday week-ending, format to YYYYMMDD Int64
14         week_end=(
15             pl.col("purchase_date").str.to_date()
16             + pl.duration(
17                 days=(6 - (pl.col("purchase_date").str.to_date().dt.weekday() % 7))
18             )
19         )
20         .dt.strftime("%Y%m%d")
21         .cast(pl.Int64)
```

```
    )
    .sink_parquet(panel_temp)
)
```

Read this block:

- Line 10: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.strftime`: Format a parsed date according to directives, e.g. %Y%m%d for an eight-digit week-ending key.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 27–28:

```
os.replace(panel_temp, panel_path)
print("Updated merged_panel.parquet successfully with Int64 'week_end'!")
```

Read this block:

- Line 27: call `os.replace` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 28: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

data__code/cleaning__merging/retail__concat.py

This scans annual retail outputs, harmonizes selected dtypes, and vertically concatenates tables. `glob.glob` returns matching filenames; `sorted` supplies deterministic order. A missing match list triggers `continue`, skipping just that target rather than the whole script. The list comprehension creates casts only for columns actually present in a particular schema.

`how="vertical_relaxed"` allows compatible dtype coercion when stacking frames; it is not permission to ignore arbitrary incompatible schemas. Explicit casts for flavor/size codes and join IDs reduce annual type conflicts. Keeping codes as strings can preserve nonnumeric categories that an integer cast would reject.

Module setup and direct-execution blocks

Source lines 1–7:

```
import glob
import os
import polars as pl
```

```
4
5 base_out_dir = "/scratch/dtm63837/Kilts_Panel/Data/RMS"
6 final_dir = os.path.join(base_out_dir, "combined")
7 os.makedirs(final_dir, exist_ok=True)
```

Read this block:

- Line 1: import `glob`. An as-name is a convenient local alias, not another library.
- Line 2: import `os`. An as-name is a convenient local alias, not another library.
- Line 3: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 5, `base_out_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data/RMS'`.
- Line 6, `final_dir`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 7: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 9–15:

```
9 concat_targets = {
10     "stores": "cleaned_stores_*.parquet",
11     "products_extra": "cleaned_products_extra_*.parquet",
12     "3612": "cleaned_3612_*.parquet",
13     "3603": "cleaned_3603_*.parquet",
14     "rms": "rms_*.parquet",
15 }
```

Read this block:

- Line 9, `concat_targets`: Set this name to a dictionary with named entries `stores`, `products_extra`, `3612`, `3603`, `rms`.

Source lines 18–24:

```
18 CAST_SCHEMA = {
19     "flavor_code": pl.String,
20     "size1_code_uc": pl.String,
21     "upc": pl.Int64,
22     "store_code_uc": pl.Int64,
23     "panel_year": pl.Int32,
24 }
```

Read this block:

- Line 18, `CAST_SCHEMA`: Set this name to a dictionary with named entries `flavor_code`, `size1_code_uc`, `upc`, `store_code_uc`, `panel_year`.

Source lines 26–57:

```
26 for table, pattern in concat_targets.items():
27     file_pattern = os.path.join(base_out_dir, pattern)
28     matching_files = sorted(glob.glob(file_pattern))
29
30     if not matching_files:
31         print(f"WARNING: No files found matching {file_pattern}. Skipping {table}...")
32         continue
33
34     concat_out = os.path.join(final_dir, f"{table}.parquet")
35     print(f"Combining {len(matching_files)} files for '{table}'...")
36
37     # Load files individually and align column types to avoid SchemaError
```

```
38      lazy_frames = []
39      for f in matching_files:
40          lf = pl.scan_parquet(f)
41
42          # Cast any problematic columns present in this table schema
43          schema_names = lf.collect_schema().names()
44          cast_exprs = [
45              pl.col(col).cast(target_type)
46              for col, target_type in CAST_SCHEMA.items()
47              if col in schema_names
48          ]
49
50          if cast_exprs:
51              lf = lf.with_columns(cast_exprs)
52
53          lazy_frames.append(lf)
54
55      # Concatenate using relaxed type resolution
56      (pl.concat(lazy_frames, how="vertical_relaxed").sink_parquet(concat_out))
57      print(f"Saved: {concat_out}")
```

Read this block:

- Line 26: for each entry yielded by `concat_targets.items()`, bind/unpack it into `(table, pattern)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 27, `file_pattern`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 28, `matching_files`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.
- Line 30: test `not matching_files`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 31: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 32: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 34, `concat_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 35: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38, `lazy_frames`: Set this name to the list `[]`.
- Line 39: for each entry yielded by `matching_files`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 40, `lf`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 43, `schema_names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 44, `cast_exprs`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 50: test `cast_exprs`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 51, `lf`: Set this name to the result of calling `lf.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 53: call `lazy_frames.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 57: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `len`: Count elements/rows rather than summing their numerical values.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

data_code/cleaning_merging/retail_convert.py

This is the retail analogue of panel conversion. Multiple mapping dictionaries separate annual metadata, movement modules, and master files. All are ordinary Python loops around lazy TSV reads and Parquet writes. `quote_char=None` disables CSV-style quote interpretation for TSV inputs. Schema inference inspects a prefix, so later inconsistent values may still need attention.

Cleaning keys are sorted by descending length, and a boundary-aware prefix test distinguishes `products_extra` from `products`. This is a concrete reason for the `sorted(..., key=len, reverse=True)` choice: testing the shorter prefix first could select the wrong projection. `continue` skips already cleaned files; `break` stops the key-search loop after a match. A `try/except` around schema inspection reports unreadable files without aborting all inspection.

Module setup and direct-execution blocks

Source lines 1–7:

```
1 import glob
2 import os
3 import polars as pl
4
5 input_dir = "/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/RMS"
6 output_dir = "/scratch/dtm63837/Kilts_Panel/Data/RMS"
7 os.makedirs(output_dir, exist_ok=True)
```

Read this block:

- Line 1: import `glob`. An as-name is a convenient local alias, not another library.
- Line 2: import `os`. An as-name is a convenient local alias, not another library.
- Line 3: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 5, `input_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/nielsen_extracts/nielsen_extracts/`.
- Line 6, `output_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data/RMS'`.
- Line 7: call `os.makedirs` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 9–23:

```
9  annual = 1
10 master = 1
11
12 years = range(2006, 2016)
13
14 annual_mapping = {
15     "products_extra": "products_extra",
16     "rms_versions": "rms",
17     "stores": "stores",
18 }
19 movement_mapping = {
20     "3603": "3603",
21     "3612": "3612",
22 }
23 master_mapping = {"brand_variations": "brand_variations", "products": "products"}
```

Read this block:

- Line 9, `annual`: Set this name to the literal value `1`.
- Line 10, `master`: Set this name to the literal value `1`.
- Line 12, `years`: Set this name to the result of calling `range(...)`; the arguments and preceding method chain are shown above.
- Line 14, `annual_mapping`: Set this name to a dictionary with named entries `products_extra`, `rms_versions`, `stores`.
- Line 19, `movement_mapping`: Set this name to a dictionary with named entries `3603`, `3612`.
- Line 23, `master_mapping`: Set this name to a dictionary with named entries `brand_variations`, `products`.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 25–66:

```
25 if annual:
26     for y in years:
27         for base_mapping, out_name in annual_mapping.items():
28             tsv_filename = f"{base_mapping}_{y}.tsv"
29             tsv_path = os.path.join(input_dir, str(y), "Annual_Files", tsv_filename)
30             parquet_out = os.path.join(output_dir, f"{out_name}_{y}.parquet")
31
32             if os.path.exists(tsv_path):
33                 (
34                     pl.scan_csv(
35                         tsv_path,
36                         separator="\t",
37                         quote_char=None,
38                         infer_schema_length=10000,
39                     )
40                     .rename(str.lower)
41                     .sink_parquet(parquet_out)
42                 )
43                 print(f"Converted: {tsv_filename} -> {out_name}_{y}.parquet")
44             else:
45                 print(f"FAILED: File not found {tsv_filename}")
46         for base_mapping, out_name in movement_mapping.items():
47             tsv_filename = f"{base_mapping}_{y}.tsv"
48             tsv_path = os.path.join(
49                 input_dir, str(y), f"Movement_Files/2510_{y}", tsv_filename
50             )
51             parquet_out = os.path.join(output_dir, f"{out_name}_{y}.parquet")
52
53             if os.path.exists(tsv_path):
```

```
54                (
55                    pl.scan_csv(
56                        tsv_path,
57                        separator="\t",
58                        quote_char=None,
59                        infer_schema_length=10000,
60                    )
61                    .rename(str.lower)
62                    .sink_parquet(parquet_out)
63                )
64                print(f"Converted: {tsv_filename} -> {out_name}_{y}.parquet")
65            else:
66                print(f"FAILED: File not found {tsv_filename}")
```

Read this block:

- Line 25: test `annual`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26: for each entry yielded by `years`, bind/unpack it into `y` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 27: for each entry yielded by `annual_mapping.items()`, bind/unpack it into `(base_mapping, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 28, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 29, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 30, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 32: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 33: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 45: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 46: for each entry yielded by `movement_mapping.items()`, bind/unpack it into `(base_mapping, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 47, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 48, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 51, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 53: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 54: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 66: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.

- `str`: Convert to text for paths, messages, or serialization.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 68–90:

```
68 if master:
69     for base_mapping, out_name in master_mapping.items():
70         tsv_filename = f"{base_mapping}.tsv"
71         tsv_path = os.path.join(
72             input_dir, "Master_Files_2006-2020/Latest", tsv_filename
73         )
74         parquet_out = os.path.join(output_dir, f"{out_name}.parquet")
75 
76         if os.path.exists(tsv_path):
77             (
78                 pl.scan_csv(
79                     tsv_path,
80                     separator="\t",
81                     quote_char=None,
82                     infer_schema_length=10000,
83                     encoding="utf8-lossy",
84                 )
85                 .rename(str.lower)
86                 .sink_parquet(parquet_out)
87             )
88             print(f"Converted: {tsv_filename} -> {out_name}.parquet")
89         else:
90             print(f"FAILED: File not found {tsv_filename}")
```

Read this block:

- Line 68: test `master`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 69: for each entry yielded by `master_mapping.items()`, bind/unpack it into `(base_mapping, out_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 70, `tsv_filename`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 71, `tsv_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 74, `parquet_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 76: test `os.path.exists(tsv_path)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 77: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 93–93:

```
93 parquet_files = glob.glob(os.path.join(output_dir, "*.parquet"))
```

Read this block:

- Line 93, `parquet_files`: Set this name to the result of calling `glob.glob(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.

Source lines 95–108:

```
95 for filepath in sorted(parquet_files):
96     filename = os.path.basename(filepath)
97 
98     # Check if file is empty before reading
99     if os.path.getsize(filepath) == 0:
100         print(f"\n--- {filename} is EMPTY (0 bytes), skipping ---")
101         continue
102 
103     try:
104         cols = pl.scan_parquet(filepath).collect_schema().names()
105         print(f"\n--- {filename} ({len(cols)} columns) ---")
106         print(cols)
107     except Exception as e:
108         print(f"\n--- Could not read {filename} (corrupted or partial file): {e} ---")
```

Read this block:

- Line 95: for each entry yielded by `sorted(parquet_files)`, bind/unpack it into `filepath` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 96, `filename`: Set this name to the result of calling `os.path.basename(...)`; the arguments and preceding method chain are shown above.
- Line 99: test `os.path.getsize(filepath) == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 100: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 101: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 103: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 104, `cols`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 105: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 106: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 107: catch `Exception` from the preceding try; the optional as-name holds the error object for logging.
- Line 108: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.getsize`: Read filesystem byte size, later divided by powers of 1024 for MiB/GiB display.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `len`: Count elements/rows rather than summing their numerical values.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 110–138:

```
110 columns_to_keep = {
111     "3603": ["store_code_uc", "upc", "week_end", "price"],
112     "3612": ["store_code_uc", "upc", "week_end", "price"],
113     "products": [
114         "upc",
115         "upc_ver_uc",
116         "product_module_code",
117         "brand_code_uc",
118         "brand_descr",
119         "size1_code_uc",
120         "size1_amount",
121         "size1_units",
122     ],
123     "products_extra": [
124         "upc",
125         "upc_ver_uc",
126         "panel_year",
127         "flavor_code",
128         "flavor_descr",
129     ],
130     "stores": [
131         "store_code_uc",
132         "year",  # Keep as string 'year' during initial extraction
133         "retailer_code",
134         "channel_code",
135         "dma_code",
136         "dma_descr",
137     ],
138 }
```

Read this block:

- Line 110, `columns_to_keep`: Set this name to a dictionary with named entries `3603`, `3612`, `products`, `products_extra`, `stores`.

Source lines 141–141:

```
141 table_keys_by_length = sorted(columns_to_keep.keys(), key=len, reverse=True)
```

Read this block:

- Line 141, `table_keys_by_length`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.

Source lines 143–167:

```
143 for filepath in glob.glob(os.path.join(output_dir, "*.parquet")):
144     filename = os.path.basename(filepath)
145 
146     # Skip files that have already been cleaned
147     if filename.startswith("cleaned_"):
148         continue
149 
150     # Remove extension -> "products_extra_2011"
151     clean_name = os.path.splitext(filename)[0]
152 
153     # Match against dictionary keys taking exact boundaries into account
154     matched_table = None
155     for key in table_keys_by_length:
156         if clean_name == key or clean_name.startswith(f"{key}_"):
157             matched_table = key
158             break
159 
```

```
160     if matched_table:
161         target_cols = columns_to_keep[matched_table]
162         cleaned_path = os.path.join(output_dir, f"cleaned_{filename}")
163 
164         (pl.scan_parquet(filepath).select(target_cols).sink_parquet(cleaned_path))
165         print(
166             f"Cleaned {filename} (Matched: {matched_table}) -> saved to cleaned_{filename}"
167         )
```

Read this block:

- Line 143: for each entry yielded by `glob.glob(os.path.join(output_dir, '*.parquet'))`, bind/unpack it into `filepath` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 144, `filename`: Set this name to the result of calling `os.path.basename(...)`; the arguments and preceding method chain are shown above.
- Line 147: test `filename.startswith('cleaned_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 148: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 151, `clean_name`: Set this name to the entry/slice selected by `os.path.splitext(filename)[0]`; brackets select from the object to their left.
- Line 154, `matched_table`: Set this name to the literal value `None`.
- Line 155: for each entry yielded by `table_keys_by_length`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 156: test `clean_name == key or clean_name.startswith(f'{key}_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 157, `matched_table`: Set this name to the object currently named `key`; plain assignment does not copy it.
- Line 158: exit the nearest enclosing loop immediately, rather than merely skipping one iteration.
- Line 160: test `matched_table`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 161, `target_cols`: Set this name to the entry/slice selected by `columns_to_keep[matched_table]`; brackets select from the object to their left.
- Line 162, `cleaned_path`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 164: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 165: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

data__code/cleaning__merging/retail__dat.py

This file is empty in the source snapshot: it contains no function or executable data-processing block to explain.

Module setup and direct-execution blocks

There are no statements in this file.

data__code/cleaning__merging/retail__merge.py

This concatenates movement modules and uses them as the retail anchor. Integer floor division by 10000 extracts a calendar year from a YYYYMMDD integer. Several metadata tables are deduplicated before left joins to avoid

multiplying movement rows. `suffix` names colliding non-key columns, while composite `on=[...]` lists specify exact matching keys.

First-row deduplication is a policy, not an inference of the correct UPC version. Extra-product sorting prioritizes nonnull flavor descriptions before retaining a UPC/year row. The diagnostics compare anchor and merged row counts and inspect nonnull metadata. A conditional expression chooses which brand field is available for a match diagnostic, so that diagnostic may inspect a source key rather than a successful dictionary match. Output is under `Data/RMS/`, unlike the current master script's `Data/merged_rms.parquet` input path.

Module setup and direct-execution blocks

Source lines 1–7:

```
1 import os
2 import polars as pl
3
4 base_dir = "/scratch/dtm63837/Kilts_Panel/Data/RMS"
5 final_out = os.path.join(base_dir, "merged_rms.parquet")
6
7 print("Loading lazy frames...")
```

Read this block:

- Line 1: import `os`. An as-name is a convenient local alias, not another library.
- Line 2: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 4, `base_dir`: Set this name to the literal text `'/scratch/dtm63837/Kilts_Panel/Data/RMS'`.
- Line 5, `final_out`: Set this name to the result of calling `os.path.join(...)`; the arguments and preceding method chain are shown above.
- Line 7: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 10–22:

```
10 movements = pl.concat(
11     [
12         pl.scan_parquet(os.path.join(base_dir, "combined/3612.parquet")),
13         pl.scan_parquet(os.path.join(base_dir, "combined/3603.parquet")),
14     ]
15 ).with_columns(
16     [
17         pl.col("upc").cast(pl.Int64),
18         pl.col("store_code_uc").cast(pl.Int64),
19         # Integer division converts YYYYMMDD (e.g., 20120107) directly to YYYY (2012)
20         (pl.col("week_end") // 10000).cast(pl.Int32).alias("panel_year"),
21     ]
22 )
```

Read this block:

- Line 10, `movements`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 25–36:

```
25 rms = (
26     pl.scan_parquet(os.path.join(base_dir, "combined/rms.parquet"))
27     .with_columns([pl.col("upc").cast(pl.Int64), pl.col("upc_ver_uc").cast(pl.Int32)])
28     .unique(subset=["upc"], keep="first")
29 )
30 
31 # 3. Products table (deduplicated on exact UPC + Version Code composite key)
32 products = (
33     pl.scan_parquet(os.path.join(base_dir, "cleaned_products.parquet"))
34     .with_columns([pl.col("upc").cast(pl.Int64), pl.col("upc_ver_uc").cast(pl.Int32)])
35     .unique(subset=["upc", "upc_ver_uc"], keep="first")
36 )
```

Read this block:

- Line 25, `rms`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 32, `products`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

Source lines 39–49:

```
39 stores = (
40     pl.scan_parquet(os.path.join(base_dir, "combined/stores.parquet"))
41     .rename({"year": "panel_year"})  # Safely renames 'year' to 'panel_year'
42     .with_columns(
43         [
44             pl.col("store_code_uc").cast(pl.Int64),
45             pl.col("panel_year").cast(pl.Int32),
46         ]
47     )
48     .unique(subset=["store_code_uc", "panel_year"], keep="first")
49 )
```

Read this block:

- Line 39, `stores`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.

Source lines 52–64:

```
52 products_extra = (
53     pl.scan_parquet(os.path.join(base_dir, "combined/products_extra.parquet"))
54     .with_columns([pl.col("upc").cast(pl.Int64), pl.col("panel_year").cast(pl.Int32)])
55     .sort("flavor_descr", descending=False, nulls_last=True)
56     .unique(subset=["upc", "panel_year"], keep="first")
57 )
58 
59 # 6. Brand Variations (deduplicated on brand code)
60 brand_var = (
61     pl.scan_parquet(os.path.join(base_dir, "brand_variations.parquet"))
```

```
    .with_columns(pl.col("brand_code_uc").cast(pl.Int64))
    .unique(subset=["brand_code_uc"], keep="first")
)
```

Read this block:

- Line 52, `products_extra`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 60, `brand_var`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.

Source lines 67–81:

```
merged = (
    movements
    # Step A: Attach upc_ver_uc from RMS
    .join(rms, on="upc", how="left", suffix="_rms")
    # Step B: Match exact version in products
    .join(products, on=["upc", "upc_ver_uc"], how="left", suffix="_prod")
    # Step C: Match store metadata by store
    .join(stores, on=["store_code_uc", "panel_year"], how="left", suffix="_str")
    # Step D: Match extra annual metadata
    .join(products_extra, on=["upc", "panel_year"], how="left", suffix="_xtra")
    # Step E: Match master brand dictionary
    .join(brand_var, on="brand_code_uc", how="left", suffix="_brand")
)

print("Streaming merged RMS dataset to disk...")
```

Read this block:

- Line 67, `merged`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 81: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 82–90:

```
merged.sink_parquet(final_out)
print(f"Successfully saved: {final_out}")

## === Run Post-Merge Audit === ##
print("\n" + "=" * 60)
print("RUNNING POST-MERGE AUDIT: RMS DATASET")
print("=" * 60)

merged_scan = pl.scan_parquet(final_out)
```

Read this block:

- Line 82: call `merged.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 83: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 86: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 87: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 88: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90, `merged_scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 93–99:

```
93 total_merged_rows = merged_scan.select(pl.len()).collect().item()
94 anchor_rows = movements.select(pl.len()).collect().item()
95 pct_retained = (total_merged_rows / anchor_rows) * 100
96
97 print(f"\n[1] Row Count & Integrity Audit:")
98 print(f"    - Movements Anchor Rows: {anchor_rows:,}")
99 print(f"    - Merged File Rows:     {total_merged_rows:,}")
```

Read this block:

- Line 93, `total_merged_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 94, `anchor_rows`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 95, `pct_retained`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 97: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 98: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 100–100:

```
100 print(f"    - Row Ratio:           {pct_retained:.2f}%")
```

Read this block:

- Line 100: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 102–105:

```
102 if total_merged_rows != anchor_rows:
103     print("    ⚠  WARNING: Row count discrepancy detected! Check join keys.")
104 else:
105     print("    ✓ PERFECT MATCH: Exact 1:1 row retention (0 rows added or dropped).")
```

Read this block:

- Line 102: test `total_merged_rows != anchor_rows`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 103: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 108–116:

```
108 print("\n[2] Dimension Table Match Rates:")
109 match_targets = {
110     "products (brand_descr)": "brand_descr",
111     "stores (retailer_code)": "retailer_code",
112     "products_extra (flavor_descr)": "flavor_descr",
113     "brand_var": "brand_code_uc_brand"
114     if "brand_code_uc_brand" in merged_scan.collect_schema().names()
115     else "brand_code_uc",
116 }
```

Read this block:

- Line 108: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 109, `match_targets`: Set this name to a dictionary with named entries `products (brand_descr)`, `stores (retailer_code)`, `products_extra (flavor_descr)`, `brand_var`.

Syntax and calls used here:

- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 118–127:

```
118 for dim_name, col_name in match_targets.items():
119     if col_name in merged_scan.collect_schema().names():
120         matched_count = (
121             merged_scan.filter(pl.col(col_name).is_not_null())
122             .select(pl.len())
123             .collect()
124             .item()
125         )
126         match_pct = (matched_count / total_merged_rows) * 100
127         print(f"    - {dim_name:<30}: {matched_count:,} matched ({match_pct:.2f}%)")
```

Read this block:

- Line 118: for each entry yielded by `match_targets.items()`, bind/unpack it into `(dim_name, col_name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 119: test `col_name in merged_scan.collect_schema().names()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 120, `matched_count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 126, `match_pct`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 127: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.

Source lines 130–136:

```
130 file_size_gb = os.path.getsize(final_out) / (1024**3)
131 schema_cols = merged_scan.collect_schema().names()
132
133 print("\n[3] Output File Summary:")
134 print(f"    - Total Columns: {len(schema_cols)}")
135 print(f"    - File Size:     {file_size_gb:.2f} GB")
136 print(f"    - Output Path:   {final_out}")
```

Read this block:

- Line 130, `file_size_gb`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 131, `schema_cols`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 133: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 134: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 135: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 136: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.getsize`: Read filesystem byte size, later divided by powers of 1024 for MiB/GiB display.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 137–138:

```
137 print("=" * 60)
138 print("AUDIT COMPLETE.")
```

Read this block:

- Line 137: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 138: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

data_code/recover_flavors.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Supplement the flavor CSV from annual RMS product metadata on the server."""
2 import argparse
3 from pathlib import Path
4
```

```
5
6 from summ_stats import DATA_DIR, FLAVOR_FILE, recover_flavor_reference as recover
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 3: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: from `summ_stats`, import `DATA_DIR, FLAVOR_FILE, recover_flavor_reference as recover` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 20–21:

```
20 if __name__ == "__main__":
21     main()
```

Read this block:

- Line 20: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 21: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

main

Original function: `main`. Source begins at line 11.

This small command-line wrapper imports the existing recovery function under the shorter local name `recover`. `as recover` changes only the imported name, not the implementation. Three Path arguments specify the original CSV, annual products, and output file; `recover(...)` does the actual work. Reusing that function avoids a second recovery algorithm drifting away from the summary pipeline. The guarded entry point lets tests import the module without running the recovery job.

Definition and stated intent:

Source lines 11–11:

```
11 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 12–17:

```
12 parser = argparse.ArgumentParser(description=__doc__)
13 parser.add_argument("--flavors", type=Path, default=FLAVOR_FILE)
14 parser.add_argument("--products", type=Path, default=DATA_DIR / "RMS/combined/products_extra.parquet")
15 parser.add_argument("--output", type=Path, default=DATA_DIR / "recovered_flavor_reference.csv")
16 args = parser.parse_args()
17 recover(args.flavors, args.products, args.output)
```

Read this block:

- Line 12, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.

- Line 13: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 14: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 15: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 16, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.
- Line 17: call `recover` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.
- `recover`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

data_code/summ_stats.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–12:

```
1  """Unweighted single-person summaries for the full panel and matched yogurt sample.
2
3  Run on the server: python3 summ_stats.py --help
4  No data are loaded when this file is imported.
5  """
6
7  from pathlib import Path
8  import argparse
9  import tempfile
10 import time
11
12 import numpy as np
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 9: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 10: import `time`. An as-name is a convenient local alias, not another library.
- Line 12: import `numpy as np`. An as-name is a convenient local alias, not another library.

Source lines 13–22:

```
13 import pandas as pd
14 import polars as pl
```

```
15
16 DATA_DIR = Path("/scratch/dtm63837/Kilts_Panel/Data")
17 # Set this to the flavor-reference CSV's full path on the Slurm server.
18 FLAVOR_FILE = (
19     DATA_DIR
20     / "../nielsen_extracts/nielsen_extracts/RMS/Reference_Documentation/2006-
       ↪ 2020_Documentation/Latest_Flavor_2010.csv"
21 )
22 FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 13: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 14: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 16, `DATA_DIR`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 18, `FLAVOR_FILE`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 22, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 27–30:

```
27 RACE_LABELS = {}
28 INCOME_LABELS = {}
29 HISPANIC_LABELS = {}
30 AGE_LABELS = {}
```

Read this block:

- Line 27, `RACE_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 28, `INCOME_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 29, `HISPANIC_LABELS`: Set this name to a dictionary from the displayed keys and values.
- Line 30, `AGE_LABELS`: Set this name to a dictionary from the displayed keys and values.

Source lines 669–670:

```
669 if __name__ == "__main__":
670     main()
```

Read this block:

- Line 669: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 670: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

normalize upcs

Original function: `normalize_upcs`. Source begins at line 33.

This is the pandas normalization helper also used in estimation: strip whitespace, recognize numeric-looking identifiers, remove `.0` and leading zeros, and retain nonnumeric strings. Its argument is a Series and its return is a Series. The `where` condition retains original text on nonnumeric rows. The reason for string processing rather

than floating-point conversion is identifier precision. See the estimation function's worked examples; identical naming here does not mean Python automatically shares their definitions across modules.

Definition and stated intent:

Source lines 33–34:

```
33 def normalize_upcs(values):
34     """Canonical numeric UPC keys without floating-point conversion."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 35–37:

```
35 text = values.astype("string").str.strip()
36 numeric = text.str.fullmatch(r"[0-9]+(?:\.0+)?", na=False)
37 canonical = text.str.replace(r"\.0+$", "", regex=True).str.lstrip("0")
```

Read this block:

- Line 35, `text`: Set this name to the result of calling `.strip(...)`; the arguments and preceding method chain are shown above.
- Line 36, `numeric`: Set this name to the result of calling `text.str.fullmatch(...)`; the arguments and preceding method chain are shown above.
- Line 37, `canonical`: Set this name to the result of calling `.lstrip(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fullmatch`: Require the entire string to match a regex; contains would allow a partial match inside unrelated text.
- `.lstrip`: Remove any leading occurrences of the specified character set, such as zeros in numeric UPC text.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 38–38:

```
38 return text.where(~numeric, canonical.replace("", "0"))
```

Read this block:

- Line 38: return the result of calling `text.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

normalized upc expr

Original function: `normalized_upc_expr`. Source begins at line 41.

This constructs the Polars-expression version of UPC normalization for a lazy pipeline. It returns a recipe named `upc`, not a table. Nested conditions replace an all-zero numeric identifier with `"0"`, preserving nonnumeric text. The function avoids repeating a long expression at every join boundary. Using matching canonical keys on both sides prevents `"001"` and `"1"` from mistakenly appearing unmatched.

Definition and stated intent:

Source lines 41–41:

```
41 def normalized_upc_expr():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 42–44:

```
42 text = pl.col("upc").cast(pl.String).str.strip_chars()
43 numeric = text.str.contains(r"^[0-9]+(?:\.0+)?$")
44 canonical = text.str.replace(r"\.0+$", "").str.strip_chars_start("0")
```

Read this block:

- Line 42, `text`: Set this name to the result of calling `.strip_chars(...)`; the arguments and preceding method chain are shown above.
- Line 43, `numeric`: Set this name to the result of calling `text.str.contains(...)`; the arguments and preceding method chain are shown above.
- Line 44, `canonical`: Set this name to the result of calling `.strip_chars_start(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.strip_chars_start`: Trim characters from the start of each Polars string; it does not remove matching characters inside it.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 45–47:

```
45 return pl.when(numeric).then(
46     pl.when(canonical == "").then(pl.lit("0")).otherwise(canonical)
47 ).otherwise(text).alias("upc")
```

Read this block:

- Line 45: return the result of calling `.alias(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

valid flavor labels

Original function: `valid_flavor_labels`. Source begins at line 50.

Input is a pandas table with `flavor_descr`; output is a Boolean Series. Missing text becomes an empty string, is cast to string, stripped, and lowercased. `.isin` checks a list of unusable sentinel strings, then `~` negates the result. Thus `" N/A "` is invalid while `"Strawberry"` is valid. Normalizing once before comparison is clearer than writing separate conditions for every case and whitespace variant. This helper exists alongside directly written lazy filters; it is not automatically applied to every flavor-processing path.

Definition and stated intent:

Source lines 50–50:

```
50 def valid_flavor_labels(frame):
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 51–51:

```
51 labels = frame["flavor_descr"].fillna("").astype(str).str.strip().str.lower()
```

Read this block:

- Line 51, `labels`: Set this name to the result of calling `.lower(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 52–52:

```
52 return ~labels.isin(["", "na", "n/a", "nan", "null", "none"])
```

Read this block:

- Line 52: return the negated or sign-adjusted expression `~labels.isin(['', 'na', 'n/a', 'nan', 'null', 'none'])`. This ends the call.

Syntax and calls used here:

- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

scan purchases

Original function: `scan_purchases`. Source begins at line 55.

This builds and returns a lazy, projected purchase query. It validates required column labels and maps optional source labels to a common schema. `aliases.items()` supplies output name plus candidate source names. `next(..., None)` picks the first available candidate; otherwise `pl.lit(None, dtype=pl.String)` makes an explicitly typed missing column. A consistent schema simplifies later summaries.

The comprehension builds expressions without reading rows. Dates, UPCs, quantities, and prices get consistent representations. Head-age zero codes become null, then `pl.coalesce` picks the first nonmissing head-age category, preferring female then male. This is a priority rule, not a mean. Eagerly loading all columns first would use more memory; dropping rows because an optional demographic is absent would change the summary population unnecessarily.

Definition and stated intent:

Source lines 55–56:

```
55 def scan_purchases(path, first_year, last_year):
56     """Project needed columns before loading; accept the names in current code."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `first_year`, `last_year`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 57–67:

```
scan = pl.scan_parquet(path)
names = scan.collect_schema().names()
required = [
    "household_code",
    "panel_year",
    "purchase_date",
    "trip_code_uc",
    "upc",
    "quantity",
]
missing = set(required) - set(names)
```

Read this block:

- Line 57, `scan`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 58, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 59, `required`: Set this name to a list containing 6 displayed entries.
- Line 67, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 68–69:

```
if missing:
    raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

Read this block:

- Line 68: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 69: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 70–79:

```
aliases = {
    "race": ["race"],
    "income": ["household_income", "income"],
    "hispanic_origin": ["hispanic_origin", "hispanic"],
    "household_size": ["household_size"],
    "female_head_age": ["female_head_age"],
    "male_head_age": ["male_head_age"],
    "dma_cd": ["dma_cd"],
}
expressions = [pl.col(c) for c in required]
```

Read this block:

- Line 70, `aliases`: Set this name to a dictionary with named entries `race`, `income`, `hispanic_origin`, `household_size`, `female_head_age`, `male_head_age`, `dma_cd`.
- Line 79, `expressions`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 80–85:

```
80 for target, choices in aliases.items():
81     source = next((c for c in choices if c in names), None)
82     expression = (
83         pl.col(source).cast(pl.String) if source else pl.lit(None, dtype=pl.String)
84     )
85     expressions.append(expression.alias(target))
```

Read this block:

- Line 80: for each entry yielded by `aliases.items()`, bind/unpack it into `(target, choices)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 81, `source`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.
- Line 82, `expression`: Set this name to the true-branch value if `source` holds, otherwise the else-branch value.
- Line 85: call `expressions.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 87–101:

```
87 price_column = next((c for c in ["price", "price_rms"] if c in names), None)
88 expressions.append(
89     pl.col(price_column).cast(pl.Float64).alias("price")
90     if price_column
91     else pl.lit(None, dtype=pl.Float64).alias("price")
92 )
93 purchases = (
94     scan.select(expressions)
95     .filter(pl.col("panel_year").is_between(first_year, last_year))
96     .with_columns(
97         pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date(),
98         normalized_upc_expr(),
99         pl.col("quantity").cast(pl.Float64),
100     )
101 )
```

Read this block:

- Line 87, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

- Line 88: call `expressions.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 93, `purchases`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.

Source lines 104–110:

```
104 age_columns = ["female_head_age", "male_head_age"]
105 purchases = purchases.with_columns(
106     [
107         pl.when(pl.col(c) == "0").then(None).otherwise(pl.col(c)).alias(c)
108         for c in age_columns
109     ]
110 )
```

Read this block:

- Line 104, `age_columns`: Set this name to the list `['female_head_age', 'male_head_age']`.
- Line 105, `purchases`: Set this name to the result of calling `purchases.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.

Source lines 111–111:

```
111 return purchases.with_columns(pl.coalesce(age_columns).alias("head_age"))
```

Read this block:

- Line 111: return the result of calling `purchases.with_columns(...)`; the arguments and preceding method chain are shown above. This ends the call.

single person purchases

Original function: `single_person_purchases`. Source begins at line 114.

Filters purchase records to household size one before purchase weeks or histories are counted. This is a record-level sample restriction: filtering households after constructing their histories would retain excluded purchases in the stocks.

Definition and stated intent:

Source lines 114–115:

```
114 def single_person_purchases(scan):
115     """Match estimation: retain size-one purchase records before histories/weeks."""
```

`def` creates this callable; the parentheses name its inputs (`scan`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 116–117:

```
116 if "household_size" not in scan.collect_schema().names():
117     raise ValueError("Single-person summaries require household_size.")
```

Read this block:

- Line 116: test `'household_size' not in scan.collect_schema().names()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 117: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.

Source lines 118–118:

```
118 return scan.filter(pl.col("household_size").cast(pl.Float64, strict=False) == 1)
```

Read this block:

- Line 118: return the result of calling `scan.filter(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

repeated yogurt households

Original function: `repeated_yogurt_households`. Source begins at line 121.

This keeps household histories with at least two distinct purchase weeks after the caller's year/price filters. `week` is a Polars expression using modulo arithmetic to reach Saturday. The query obtains each household's minimum and maximum week: unequal endpoints prove at least two distinct weeks without retaining a full distinct-week set.

Only eligible household IDs are collected. A semi join filters the original lazy purchase table while preserving its columns and all eligible purchase rows. Joining an expanded eligibility table could multiply rows; the semi join expresses membership only. Multiple trips in the same week do not satisfy this rule. Estimation later repeats a similar restriction after additional exclusions.

Definition and stated intent:

Source lines 121–126:

```
121 def repeated_yogurt_households(scan):
122     """Keep households with two distinct Saturday-ending purchase weeks.
123
124     Apply after year selection; multiple lines/trips in one week count once.
125     Summaries precede the estimation-only menu and IV exclusions.
126     """
```

`def` creates this callable; the parentheses name its inputs (`scan`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 127–137:

```
127 week = pl.col("purchase_date") + pl.duration(
128     days=(6 - pl.col("purchase_date").dt.weekday()) % 7
129 )
130 ids = (
131     scan.group_by("household_code")
132     .agg(week.min().alias("first_week"), week.max().alias("last_week"))
133     .filter(pl.col("first_week") != pl.col("last_week"))
134     .select("household_code")
135 )
136 # Materialize only household IDs so the repeated scan cannot cache all purchases.
137 ids = ids.collect(engine="streaming")
```

Read this block:

- Line 127, `week`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 130, `ids`: Set this name to the result of calling `.select(...)`; the arguments and preceding method chain are shown above.
- Line 137, `ids`: Set this name to the result of calling `ids.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.

Source lines 138–138:

```
138 return scan.join(ids.lazy(), on="household_code", how="semi")
```

Read this block:

- Line 138: return the result of calling `scan.join(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.

household summary

Original function: `household_summary`. Source begins at line 141.

Input is a lazy purchase table; output is a lazy one-row-per-household query. The `expressions` list defines exact purchase totals, distinct counts, and observed years. Demographic history follows a separate path: select a narrow

table, deduplicate, stably sort, and take each field's first nonmissing value. Aggregating the full purchase table separately preserves exact trip/UPC/line counts while avoiding repeated wide demographic sorts.

`group_by(...).agg([...])` evaluates each listed expression per group. A validated one-to-one join reunites totals and demographic values. Later `with_columns` steps preserve original race, apply the Hispanic override, and calculate approximate weeks/trip rate. The second step can refer to the new `approx_observed_weeks` created in the first. Counting only observed years avoids filling panel gaps but still assumes full-year exposure within those years.

Definition and stated intent:

Source lines 141–142:

```
141 def household_summary(scan):
142     """One row per household, so frequent shoppers do not dominate demographics."""
```

`def` creates this callable; the parentheses name its inputs (`scan`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 143–152:

```
143 demographics = [
144     "race",
145     "income",
146     "hispanic_origin",
147     "household_size",
148     "head_age",
149     "female_head_age",
150     "male_head_age",
151     "dma_cd",
152 ]
```

Read this block:

- Line 143, `demographics`: Set this name to a list containing 8 displayed entries.

Source lines 154–159:

```
154 scan = scan.with_columns(
155     [
156         pl.when(pl.col(c) == "0").then(None).otherwise(pl.col(c)).alias(c)
157         for c in ["female_head_age", "male_head_age"]
158     ]
159 )
```

Read this block:

- Line 154, `scan`: Set this name to the result of calling `scan.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 160–174:

```
160 expressions = [
161     pl.col("panel_year").n_unique().alias("observed_years"),
```

```
    pl.col("panel_year").min().alias("first_year"),
    pl.col("panel_year").max().alias("last_year"),
    pl.col("trip_code_uc").n_unique().alias("trips"),
    pl.col("purchase_date").n_unique().alias("shopping_days"),
    pl.col("upc").n_unique().alias("distinct_upcs"),
    pl.len().alias("purchase_lines"),
    pl.col("quantity").sum().alias("packages"),
]
# Keep exact purchase counts separate from the much narrower demographic
# history. Sort that history once for all fields, preserving input order
# for conflicting values on the same date. Removing identical rows cannot
# change the first nonmissing value.
totals = scan.group_by("household_code").agg(expressions)
```

Read this block:

- Line 160, `expressions`: Set this name to a list containing 8 displayed entries.
- Line 174, `totals`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.
- `.min`: Take the smallest value, or a per-group/axis minimum when used on a table expression.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 175–181:

```
history = (scan.select(["household_code", "panel_year", "purchase_date"] + demographics)
           .unique(maintain_order=True)
           .sort(["household_code", "panel_year", "purchase_date"], maintain_order=True))
values = history.group_by("household_code").agg([
    pl.col(c).drop_nulls().first().alias(c) for c in demographics
])
households = totals.join(values, on="household_code", how="left", validate="1:1")
```

Read this block:

- Line 175, `history`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 178, `values`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 181, `households`: Set this name to the result of calling `totals.join(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.drop_nulls`: Remove null values from the Polars expression before selecting a first observed value.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 182–189:

```
182 return households.with_columns(
183     pl.col("race").alias("race_original"),
184     pl.when(pl.col("hispanic_origin") == "1").then(pl.lit("Hispanic"))
185     .otherwise(pl.col("race")).alias("race"),
186     (pl.col("observed_years") * 365.25 / 7).alias("approx_observed_weeks"),
187 ).with_columns(
188     (pl.col("trips") / pl.col("approx_observed_weeks")).alias("trips_per_observed_week")
189 )
```

Read this block:

- Line 182: return the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

as lazy

Original function: `as_lazy`. Source begins at line 192.

This small adapter always returns a Polars LazyFrame. `isinstance` checks the runtime type. An existing LazyFrame is returned unchanged, a pandas table is converted through `pl.from_pandas(...).lazy()`, and the remaining expected Polars DataFrame receives `.lazy()`. This lets callers supply convenient small eager tables in tests while production uses lazy scans. It is not a universal converter: an arbitrary unsupported object without `.lazy()` will still fail. Avoiding a copy/conversion when already lazy preserves the existing query plan.

Definition and stated intent:

Source lines 192–192:

```
192 def as_lazy(frame):
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 193–194:

```
193 if isinstance(frame, pl.LazyFrame):
194     return frame
```

Read this block:

- Line 193: test `isinstance(frame, pl.LazyFrame)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 194: return the object currently named `frame`; plain assignment does not copy it. This ends the call.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 195–196:

```
195 if isinstance(frame, pd.DataFrame):
196     return pl.from_pandas(frame).lazy()
```

Read this block:

- Line 195: test `isinstance(frame, pd.DataFrame)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 196: return the result of calling `.lazy(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.from_pandas`: Convert a pandas object into a Polars table; calling lazy afterward makes a query plan from it.

Source lines 197–197:

```
197 return frame.lazy()
```

Read this block:

- Line 197: return the result of calling `frame.lazy(...)`; the arguments and preceding method chain are shown above. This ends the call.

partition purchases

Original function: `partition_purchases`. Source begins at line 200.

This executes a lazy scan once to write physical household-hash partitions, then returns a list of lazy scans over those files. `hasattr(pl, "PartitionBy")` checks API availability before starting. `(hash(seed=0) % batches)` maps every row with the same household ID to the same bucket. This is deterministic routing, not random sample selection.

`sink_parquet(pl.PartitionBy(...))` writes by bucket. The list comprehension includes only existing bucket directories, important because small examples can leave buckets empty. `sorted(directory.glob(...))` makes input order predictable. `hive_partitioning=False` prevents folder names from automatically adding a partition column that was intentionally excluded. The alternative of filtering/recollecting the original source for every bucket repeats large scans; physical partitions trade temporary disk space for reusable reads.

Definition and stated intent:

Source lines 200–201:

```
200 def partition_purchases(scan, work, prefix, batches):
201     """Read the projected source once, writing household hash partitions."""
```

`def` creates this callable; the parentheses name its inputs (`scan`, `work`, `prefix`, `batches`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 202–203:

```
202 if not hasattr(pl, "PartitionBy"):
203     raise RuntimeError("Summary partitioning requires Polars >= 2.0; install est_code/requirements.txt.")
```

Read this block:

- Line 202: test `not hasattr(pl, 'PartitionBy')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 203: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `hasattr`: Check that an object exposes a named attribute before using an optional/version-dependent API.
- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 204–210:

```
204 directory = work / f"{prefix}_purchases"
205 began = time.monotonic()
206 print(f"{prefix}: partitioning purchases once into {batches} batches", flush=True)
207 (scan.with_columns((pl.col("household_code").hash(seed=0) % batches).alias("_batch"))
208  .sink_parquet(pl.PartitionBy(directory, key="_batch", include_key=False), engine="streaming"))
209 parts = [pl.scan_parquet(sorted(directory.glob(f"_batch={batch}/*.parquet")), hive_partitioning=False)
210          for batch in range(batches) if (directory / f"_batch={batch}").exists()]
```

Read this block:

- Line 204, `directory`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 205, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 206: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 207: call `.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 209, `parts`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.

Syntax and calls used here:

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.glob`: Find filesystem paths matching a wildcard pattern, which may include stale files unless the folder is controlled.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 211–212:

```
211 if not parts:
212     raise ValueError(f"Empty {prefix} sample in selected years.")
```

Read this block:

- Line 211: test `not parts`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 212: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 213–213:

```
213 print(f"{prefix}: partitioning completed in {time.monotonic() - began:.1f}s", flush=True)
```

Read this block:

- Line 213: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 214–214:

```
214 return parts
```

Read this block:

- Line 214: return the object currently named `parts`; plain assignment does not copy it. This ends the call.

household batches

Original function: `household_batches`. Source begins at line 217.

This accepts an optional list of already written partitions. The conditional expression reuses them if supplied; otherwise it creates them once. `enumerate` gives each partition an output filename suffix, `household_summary` runs on it, and `sink_parquet` writes its reduced result. The final wildcard scan returns one logical table of all household outputs without first concatenating them eagerly in Python. Households never cross partition boundaries, so exact household summaries can be computed independently and then stacked.

Definition and stated intent:

Source lines 217–218:

```
217 def household_batches(scan, work, prefix, batches, partitions=None):
218     # Each physical partition is read on its own; no full-source rescan per batch.
```

`def` creates this callable; the parentheses name its inputs (`scan`, `work`, `prefix`, `batches`, `partitions`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 219–219:

```
219 partitions = partitions if partitions is not None else partition_purchases(scan, work, prefix, batches)
```

Read this block:

- Line 219, `partitions`: Set this name to the true-branch value if `partitions is not None` holds, otherwise the else-branch value.

Syntax and calls used here:

- `partition_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 220–224:

```
220 for batch, subset in enumerate(partitions):
221     began = time.monotonic()
222     print(f"{prefix}: household batch {batch + 1}/{len(partitions)}", flush=True)
223     household_summary(subset).sink_parquet(work / f"{prefix}_{batch}.parquet", engine="streaming")
224     print(f"{prefix}: household batch {batch + 1} completed in {time.monotonic() - began:.1f}s", flush=True)
```

Read this block:

- Line 220: for each entry yielded by `enumerate(partitions)`, bind/unpack it into `(batch, subset)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 221, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 222: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 223: call `household_summary(subset).sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 224: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `household_summary`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 225–225:

```
225 return pl.scan_parquet(work / f"{prefix}_*.parquet")
```

Read this block:

- Line 225: return the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

save table

Original function: `save_table`. Source begins at line 228.

This writes both a machine-readable CSV and an escaped LaTeX table and prints a readable version. Polars inputs are collected and converted to pandas because the chosen table formatting/export methods belong to pandas. A pandas input already has those methods, so conversion is skipped.

`lambda value: f"{value:.2f}"` formats display numbers without changing the stored numerical data. `index=False` suppresses an unwanted row-number column. `escape=True` protects LaTeX syntax characters in labels. CSV retains reusable data whereas LaTeX supplies paper-ready formatting; either output alone would serve only one of those purposes. This function's useful result is its side effects, so no explicit return is required.

Definition and stated intent:

Source lines 228–229:

```
228 def save_table(frame, output, name, caption):
229     """Keep machine-readable CSV beside the paper-ready LaTeX table."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `output`, `name`, `caption`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 230–231:

```
230 if isinstance(frame, (pl.LazyFrame, pl.DataFrame)):
231     frame = as_lazy(frame).collect(engine="streaming").to_pandas()
```

Read this block:

- Line 230: test `isinstance(frame, (pl.LazyFrame, pl.DataFrame))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 231, `frame`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `as_lazy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 232–237:

```
232 print(f"\n{caption} [{name}]", flush=True)
233 print(frame.to_string(index=False, float_format=lambda value: f"{value:.2f}"), flush=True)
234 frame.to_csv(output / f"{name}.csv", index=False)
235 (output / f"{name}.tex").write_text(
236     frame.to_latex(index=False, escape=True, float_format="%.2f", caption=caption)
237 )
```

Read this block:

- Line 232: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 233: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 234: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 235: call `(output / f'{name}.tex').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.to_latex`: Serialize a pandas table as LaTeX source, which needs a separate TeX compilation to become PDF.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

demographic tables

Original function: `demographic_tables`. Source begins at line 240.

Input is a one-row-per-household table, an output directory, and filename prefix. The initial count is its denominator. The loop builds one categorical table per demographic using null-to-`Missing` replacement, grouped counts, optional codebook mappings, and percentages of all households. `.map(labels).fillna(code)` keeps source codes visible when a verified label is absent.

The `specs` list pairs human-readable metric names with Polars aggregation expressions. Enumerated string aliases let all metrics be computed in a single query and then zipped back to names. `rows.extend(generator)` appends race percentages to the overview. Equal household weighting is achieved by summarizing households first; directly counting purchase rows would overweight frequent shoppers. Numeric means of income/head-age codes remain means of categories, not economic levels.

Definition and stated intent:

Source lines 240–240:

```
240 def demographic_tables(households, output, prefix):
```

`def` creates this callable; the parentheses name its inputs (`households`, `output`, `prefix`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 241–243:

```
241 households = as_lazy(households)
242 count = households.select(pl.len()).collect(engine="streaming").item()
243 race_table = None
```

Read this block:

- Line 241, `households`: Set this name to the result of calling `as_lazy(...)`; the arguments and preceding method chain are shown above.
- Line 242, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 243, `race_table`: Set this name to the literal value `None`.

Syntax and calls used here:

- `as_lazy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 244–256:

```
244 for column, labels in [("race", RACE_LABELS), ("hispanic_origin", HISPANIC_LABELS),
245                        ("income", INCOME_LABELS), ("household_size", {}),
246                        ("head_age", AGE_LABELS), ("female_head_age", AGE_LABELS),
247                        ("male_head_age", AGE_LABELS)]:
248     table = (households.select(pl.col(column).cast(pl.String).fill_null("Missing").alias("code"))
249              .group_by("code").len(name="households").sort("code")
250              .collect(engine="streaming").to_pandas())
251     table["category"] = table["code"].map(labels).fillna(table["code"])
252     table["percent_all_households"] = 100 * table["households"] / count
253     save_table(table, output, f"{prefix}_{column}",
254                f"{prefix.replace('_', ' ').title()}: {column.replace('_', ' ')} (unweighted)")
255     if column == "race":
256         race_table = table
```

Read this block:

- Line 244: for each entry yielded by `[('race', RACE_LABELS), ('hispanic_origin', HISPANIC_LABELS), ...]`, bind/unpack it into `(column, labels)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 248, `table`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 251, `table['category']`: Store the result of calling `.fillna(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 252, `table['percent_all_households']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 253: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 255: test `column == 'race'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 256, `race_table`: Set this name to the object currently named `table`; plain assignment does not copy it.

Syntax and calls used here:

- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 257–271:

```
257 specs = [
258     ("Households", pl.len()),
259     ("Mean observed panel years", pl.col("observed_years").mean()),
260     ("Mean trips per household", pl.col("trips").mean()),
261     ("Mean trips per approximate observed week", pl.col("trips_per_observed_week").mean()),
262     ("Median trips per approximate observed week", pl.col("trips_per_observed_week").median()),
263     ("Mean distinct UPCs per household", pl.col("distinct_upcs").mean()),
264     ("Mean purchase lines per household", pl.col("purchase_lines").mean()),
265     ("Mean packages per household", pl.col("packages").mean()),
266     ("Mean head age (source category code)", pl.col("head_age").cast(pl.Float64, strict=False).mean()),
267     ("Households with numeric head age", pl.col("head_age").cast(pl.Float64, strict=False).count()),
268     ("Mean household income (source category code)", pl.col("income").cast(pl.Float64, strict=False).mean()),
269     ("Households with numeric income", pl.col("income").cast(pl.Float64, strict=False).count()),
270 ]
271 values = households.select([expr.alias(str(i)) for i, (_, expr) in
    ↪  enumerate(specs)]).collect(engine="streaming").row(0)
```

Read this block:

- Line 257, `specs`: Set this name to a list containing 12 displayed entries.
- Line 271, `values`: Set this name to the result of calling `.row(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.median`: Compute the middle-value statistic rather than a mean; global medians cannot generally be averaged across batches.
- `str`: Convert to text for paths, messages, or serialization.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 272–276:

```
272 rows = [(name, value) for (name, _), value in zip(specs, values)]
273 rows.extend((f"Race: {row.category} (% of all households)", row.percent_all_households)
274             for row in race_table.itertuples())
275 save_table(pd.DataFrame(rows, columns=["metric", "value"]), output, f"{prefix}_overview",
276           f"{prefix.replace('_', ' ').title()}: Household purchase summary (unweighted)")
```

Read this block:

- Line 272, `rows`: Set this name to a list computed by the displayed for-expression.
- Line 273: call `rows.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 275: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

scan reference

Original function: `scan_reference`. Source begins at line 279.

This wraps `pl.scan_csv` with consistent parsing choices. `infer_schema=False` keeps reference fields as strings instead of guessing that UPCs are numbers. `null_values=[...]` explicitly recognizes several missing-value spellings. The return is lazy, so calling the function sets up a CSV read without loading the entire table. Centralizing these options prevents different recovery and classification paths from interpreting the same file differently.

Definition and stated intent:

Source lines 279–279:

```
279 def scan_reference(path):
```

`def` creates this callable; the parentheses name its inputs (`path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 280–281:

```
280 return pl.scan_csv(path, infer_schema=False,
281                    null_values=["NA", "N/A", "NaN", "nan", "NULL", "null", "None", "<NA>"])
```

Read this block:

- Line 280: return the result of calling `pl.scan_csv(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.

recover flavor reference

Original function: `recover_flavor_reference`. Source begins at line 284.

This supplements, rather than overwrites, the descriptor reference. Both input tables are lazy; an anti join excludes product UPCs already present in the CSV. The filter removes missing/placeholder candidate descriptors. If annual metadata exists, each UPC's candidate columns take earliest-year values; otherwise first row deduplication is used. The output is a new combined reference plus a separate CSV of just recovered rows.

`.with_name(output.stem + ...)` constructs a sibling output filename. Comparing resolved paths rejects accidental overwrite of the original reference. After writing additions, `scan_reference` rereads them under the same schema conventions before diagonal relaxed concatenation allows unequal optional columns. The return is the output

Path, enabling `run` to use it for subsequent classification. It does not repair blank descriptors on UPCs already in the original reference.

Definition and stated intent:

Source lines 284–284:

```
284 def recover_flavor_reference(reference_path, products_path, output):
```

`def` creates this callable; the parentheses name its inputs (`reference_path`, `products_path`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 285–285:

```
285 reference = scan_reference(reference_path).with_columns(normalized_upc_expr())
```

Read this block:

- Line 285, `reference`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `scan_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 286–287:

```
286 if not {"upc", "flavor_descr"}.issubset(reference.collect_schema().names()):
287     raise ValueError("Reference needs upc and flavor_descr columns.")
```

Read this block:

- Line 286: test `not {'upc', 'flavor_descr'}.issubset(reference.collect_schema().names())`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 287: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.

Source lines 288–289:

```
288 products = pl.scan_parquet(products_path)
289 names = products.collect_schema().names()
```

Read this block:

- Line 288, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 289, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.

Source lines 290–291:

```
290 if not {"upc", "flavor_descr"}.issubset(names):
291     raise ValueError(f"{products_path}: needs upc and flavor_descr columns.")
```

Read this block:

- Line 290: test `not {'upc', 'flavor_descr'}.issubset(names)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 291: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 292–299:

```
292 columns = [c for c in ["upc", "flavor_descr", "panel_year", "product_descr"] if c in names]
293 products = (products.select(columns).with_columns(normalized_upc_expr())
294             .join(reference.select("upc").unique(), on="upc", how="anti"))
295 products = products.filter(
296     pl.col("upc").is_not_null() & pl.col("flavor_descr").is_not_null()
297     & ~pl.col("flavor_descr").cast(pl.String).str.strip_chars().str.to_lowercase()
298     .is_in(["", "na", "n/a", "nan", "null", "none"])
299 )
```

Read this block:

- Line 292, `columns`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 293, `products`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 295, `products`: Set this name to the result of calling `products.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 300–306:

```
300 if "panel_year" in names:
301     recovered = products.group_by("upc").agg([
302         pl.col(c).sort_by("panel_year", nulls_last=True).first().alias(c)
303         for c in columns if c != "upc"
304     ])
305 else:
```

```
306     recovered = products.unique(subset="upc", keep="first", maintain_order=True)
```

Read this block:

- Line 300: test `'panel_year' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 301, `recovered`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 306, `recovered`: Set this name to the result of calling `products.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.sort_by`: Order values within an expression/group by another column before selecting a first value.

Source lines 307–308:

```
307 output = Path(output)
308 output.parent.mkdir(parents=True, exist_ok=True)
```

Read this block:

- Line 307, `output`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 308: call `output.parent.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 309–310:

```
309 if output.resolve() == Path(reference_path).resolve():
310     raise ValueError("Choose a separate output file to preserve the original reference.")
```

Read this block:

- Line 309: test `output.resolve() == Path(reference_path).resolve()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 310: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.

Source lines 311–316:

```
311 recovered_path = output.with_name(output.stem + "_recovered_rows.csv")
312 recovered.sink_csv(recovered_path, engine="streaming")
313 additions = scan_reference(recovered_path)
314 pl.concat([reference, additions], how="diagonal_relaxed").sink_csv(output, engine="streaming")
315 count = additions.select(pl.len()).collect(engine="streaming").item()
316 print(f"Recovered labels for {count:,} additional UPCs. Existing entries retained. Saved {output}", flush=True)
```

Read this block:

- Line 311, `recovered_path`: Set this name to the result of calling `output.with_name(...)`; the arguments and preceding method chain are shown above.
- Line 312: call `recovered.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 313, `additions`: Set this name to the result of calling `scan_reference(...)`; the arguments and preceding method chain are shown above.
- Line 314: call `.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 315, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 316: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.with_name`: Construct a sibling Path with a replacement filename, preserving its parent directory.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 317–317:

```
317 return output
```

Read this block:

- Line 317: return the object currently named `output`; plain assignment does not copy it. This ends the call.

flavor reference

Original function: `flavor_reference`. Source begins at line 320.

This resolves one label per normalized UPC and returns an eager Polars mapping. `detailed` selects two different expression recipes. Detailed labels use trimmed lowercase descriptors with whitespace collapsed; grouped labels use ordered plain/berry text matching and an other fallback. `.replace` maps specific whole values; regex `.str.replace_all` instead matches patterns within text.

Before deduplicating, the function counts UPCs assigned multiple distinct labels and can save their rows for review. `unique(..., keep="first", maintain_order=True)` implements first-reference-row precedence reproducibly. This is a transparent rule but not a semantic resolution of conflicting descriptions. Returning only UPC and final flavor gives downstream joins a small unique lookup table.

Definition and stated intent:

Source lines 320–321:

```
320 def flavor_reference(path, detailed=False, output=None):
321     """Use first reference row per normalized UPC; keep missing descriptors."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `detailed`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 322–324:

```
322 frame = scan_reference(path)
323 names = frame.collect_schema().names()
324 missing = {"upc", "flavor_descr"} - set(names)
```

Read this block:

- Line 322, `frame`: Set this name to the result of calling `scan_reference(...)`; the arguments and preceding method chain are shown above.
- Line 323, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 324, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `scan_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 325–326:

```
325 if missing:
326     raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

Read this block:

- Line 325: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 326: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 327–327:

```
327 frame = frame.with_columns(normalized_upc_expr())
```

Read this block:

- Line 327, `frame`: Set this name to the result of calling `frame.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 328–329:

```
328 if frame.select((pl.col("upc").is_null() | (pl.col("upc") == "")).any()).collect(engine="streaming").item():
329     raise ValueError(f"{path}: flavor reference contains missing UPCs.")
```

Read this block:

- Line 328: test `frame.select((pl.col('upc').is_null() | (pl.col('upc') == '')).any()).collect...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 329: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.

Source lines 330–330:

```
330 text = pl.col("flavor_descr").fill_null("")
```

Read this block:

- Line 330, `text`: Set this name to the result of calling `.fill_null(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.

Source lines 331–332:

```
331 if "product_descr" in names:
332     text = text + pl.lit(" ") + pl.col("product_descr").fill_null("")
```

Read this block:

- Line 331: test `'product_descr' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 332, `text`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

Source lines 333–333:

```
333 text = text.str.to_lowercase()
```

Read this block:

- Line 333, `text`: Set this name to the result of calling `text.str.to_lowercase(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.

Source lines 334–340:

```
334 if detailed:
335     flavor = (pl.col("flavor_descr").fill_null("").str.to_lowercase().str.strip_chars()
336               .str.replace_all(r"\s+", " ").replace({"": "unknown", "unflavored": "plain"}))
337 else:
338     flavor = (pl.when(text.str.contains("plain|unflavored")).then(pl.lit("plain"))
339               .when(text.str.contains("berry|straw|blue|rasp|black|cran|cherry|wildberry"))
340               .then(pl.lit("berry")).otherwise(pl.lit("other")))
```

Read this block:

- Line 334: test `detailed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 335, `flavor`: Set this name to the result of calling `.replace(...)`; the arguments and preceding method chain are shown above.
- Line 338, `flavor`: Set this name to the result of calling `.otherwise(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.replace_all`: Replace every regex match within each string, such as collapsing repeated whitespace.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.

Source lines 341–343:

```
341 frame = frame.with_columns(flavor.alias("flavor"))
342 conflicting = frame.group_by("upc").agg(pl.col("flavor").n_unique().alias("n")).filter(pl.col("n") > 1)
343 count = conflicting.select(pl.len()).collect(engine="streaming").item()
```

Read this block:

- Line 341, `frame`: Set this name to the result of calling `frame.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 342, `conflicting`: Set this name to the result of calling `.filter(...)`; the arguments and preceding method chain are shown above.
- Line 343, `count`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.

Source lines 344–348:

```
344 if count:
345     print(f"{count:,} UPCs have conflicting flavor classifications; keeping the first CSV row.", flush=True)
346     if output is not None:
347         name = "detailed_flavor_conflicts.csv" if detailed else "aggregated_flavor_conflicts.csv"
348         frame.join(conflicting.select("upc"), on="upc", how="semi").sink_csv(output / name, engine="streaming")
```

Read this block:

- Line 344: test `count`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 345: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 346: test `output is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 347, `name`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 348: call `.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 349–350:

```
349 blanks = frame.select((pl.col("flavor_descr").fill_null("").str.strip_chars() ==
    ↪  "").sum()).collect(engine="streaming").item()
350 print(f"Flavor-reference rows with blank descriptors: {blanks:,}", flush=True)
```

Read this block:

- Line 349, `blanks`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 350: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 351–351:

```
351 return frame.select("upc", "flavor").unique(subset="upc", keep="first",
    ↪  maintain_order=True).collect(engine="streaming")
```

Read this block:

- Line 351: return the result of calling `.collect(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

yogurt weeks

Original function: `yogurt_weeks`. Source begins at line 354.

This attaches flavor labels, audits matched purchase lines, and returns a lazy weekly quantity query plus an eager small audit table. A left join preserves unmatched purchases long enough to count/report them. Unmatched UPCs are exported and cause a ValueError, preventing an apparently successful summary of a silently reduced sample. Nonpositive/missing quantities also stop the run.

The Saturday expression adds a row-dependent duration to purchase date. Grouping by household/week/flavor and summing quantity keeps only observed combinations. The alternative dense pivot would create a column for every detailed flavor and potentially a great many empty cells. Price problems are audited here; the main matched-sample caller has already applied its positive-price filter.

Definition and stated intent:

Source lines 354–355:

```
354 def yogurt_weeks(scan, reference, flavors=FLAVORS, output=None):
355     """Keep sparse household/week/flavor counts rather than a dense pivot."""
```

`def` creates this callable; the parentheses name its inputs (`scan`, `reference`, `flavors`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 356–362:

```
356 joined = scan.join(reference.lazy(), on="upc", how="left")
357 audit = joined.select(
358     pl.len().alias("purchase_lines"),
359     pl.col("flavor").is_null().sum().alias("unmatched_flavor_lines"),
360     ((~pl.col("price").is_finite()) | (pl.col("price") <=
    ↪  0)).fill_null(True).sum().alias("missing_or_nonpositive_quoted_prices"),
361     ((~pl.col("quantity").is_finite()) | (pl.col("quantity") <=
    ↪  0)).fill_null(True).sum().alias("invalid_quantity_lines"),
362 ).collect(engine="streaming")
```

Read this block:

- Line 356, `joined`: Set this name to the result of calling `scan.join(...)`; the arguments and preceding method chain are shown above.
- Line 357, `audit`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 363–368:

```
363 if audit["unmatched_flavor_lines"][0]:
364     unmatched = (joined.filter(pl.col("flavor").is_null()).group_by("upc")
365                  .agg(pl.len().alias("purchase_lines"), pl.col("quantity").sum().alias("packages")))
366     if output is not None:
367         unmatched.sink_csv(output / "unmatched_purchase_upcs.csv", engine="streaming")
368     raise ValueError("Purchased UPCs still lack flavor labels; see unmatched_purchase_upcs.csv.")
```

Read this block:

- Line 363: test `audit['unmatched_flavor_lines'][0]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 364, `unmatched`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 366: test `output is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 367: call `unmatched.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 368: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.

Source lines 369–370:

```
369 if audit["invalid_quantity_lines"][0]:
370     raise ValueError("Nonpositive or missing quantities; audit the matched sample first.")
```

Read this block:

- Line 369: test `audit['invalid_quantity_lines'][0]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 370: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 371–373:

```
371 week = pl.col("purchase_date") + pl.duration(days=(6 - pl.col("purchase_date").dt.weekday()) % 7)
372 grouped = (joined.with_columns(week.alias("week"))
373            .group_by(["household_code", "week", "flavor"]).agg(pl.col("quantity").sum()))
```

Read this block:

- Line 371, `week`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 372, `grouped`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.

Source lines 374–374:

```
374 return grouped, audit
```

Read this block:

- Line 374: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

flavor summaries

Original function: `flavor_summaries`. Source begins at line 377.

This is the main descriptive-statistics engine. It returns pandas share and transition tables for plotting, while writing much richer outputs. First it sums quantities by flavor and computes package shares; a rounding filter removes rows whose displayed share is zero but leaves the denominator unchanged. Next it aggregates household/flavor totals to number of flavors, squared quantities, and largest quantity, producing HHI and largest-flavor share.

The basket query groups again by household/week. `pl.when(pl.len() == 1)` gives a single-flavor label only to unmixed baskets. `.shift().over("household_code")` looks at the previous basket within the household, not across household borders. Date differences identify adjacent weeks. A cumulative sum of spell-start flags creates run identifiers; mixed baskets and gaps break the runs.

The expensive basket query is checkpointed once in a temporary directory so transitions, spells, CSV exports, and global medians can reuse it. Transition counts pivot into a small matrix; dividing by row sums produces probabilities. Zero denominators become NaN. Spells are averaged as spells, not as households. Finally a small metric table records basket frequencies and eligible-pair switching. This sequence uses several aggregation units intentionally; replacing them all with one grand groupby would obscure their different denominators.

Definition and stated intent:

Source lines 377–377:

```
377 def flavor_summaries(weekly, output, flavors=FLAVORS):
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `output`, `flavors`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 378–379:

```
378 weekly = as_lazy(weekly)
379 totals = weekly.group_by("flavor").agg(pl.col("quantity").sum().alias("packages")).collect(engine="streaming")
```

Read this block:

- Line 378, `weekly`: Set this name to the result of calling `as_lazy(...)`; the arguments and preceding method chain are shown above.
- Line 379, `totals`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `as_lazy`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 380–381:

```
380 if flavors != FLAVORS:
381     flavors = sorted(totals["flavor"].to_list())
```

Read this block:

- Line 380: test `flavors != FLAVORS`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 381, `flavors`: Set this name to the result of calling `sorted(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.

Source lines 383–391:

```
383 shares = (pl.DataFrame({"flavor": flavors}).join(totals, on="flavor", how="left")
384           .with_columns(pl.col("packages").fill_null(0))
385           .with_columns((100 * pl.col("packages") / pl.col("packages").sum()).alias("percent_packages")))
386 # Match the table's two-decimal display; keep the original denominator.
387 shares = shares.filter(pl.col("percent_packages").round(2) > 0).sort(
388     ["percent_packages", "flavor"], descending=[True, False]
389 )
```

```
save_table(shares, output, "yogurt_flavor_shares", "Flavor shares of purchased packages")
counts = weekly.group_by(["household_code", "flavor"]).agg(pl.col("quantity").sum())
```

Read this block:

- Line 383, `shares`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 387, `shares`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 390: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 391, `counts`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.round`: Round numeric values for a check/display rule; rounding quantities is compared to the original before acceptance.
- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 392–401:

```
households = (counts.group_by("household_code").agg(
    pl.len().alias("flavors_bought"), pl.col("quantity").sum().alias("packages"),
    pl.col("quantity").pow(2).sum().alias("squared_packages"),
    pl.col("quantity").max().alias("largest_packages"))
    .with_columns((pl.col("squared_packages") / pl.col("packages").pow(2)).alias("flavor_HHI"),
                  (pl.col("largest_packages") / pl.col("packages")).alias("largest_flavor_share"))
    .select("household_code", "flavors_bought", "flavor_HHI", "largest_flavor_share"))
hh_path = output / "yogurt_household_flavors.csv"
households.sink_csv(hh_path, engine="streaming")
hh_metrics = pl.scan_csv(hh_path).select(pl.len(), pl.col("flavors_bought").mean(),
 ↪  pl.col("flavor_HHI").mean()).collect(engine="streaming").row(0)
```

Read this block:

- Line 392, `households`: Set this name to the result of calling `.select(...)`; the arguments and preceding method chain are shown above.
- Line 399, `hh_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 400: call `households.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 401, `hh_metrics`: Set this name to the result of calling `.row(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.pow`: Raise each expression value to a power, such as squaring quantities for concentration calculations.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.scan_csv`: Build a lazy delimited-text read plan; parser options determine types and missing values.

Source lines 402–411:

```
402 baskets = (weekly.group_by(["household_code", "week"]).agg(
403     pl.col("quantity").sum().alias("packages"), pl.len().alias("flavors_in_basket"),
404     pl.when(pl.len() == 1).then(pl.col("flavor").first()).otherwise(None).alias("single_flavor"))
405     .sort(["household_code", "week"])
406     .with_columns(pl.col("single_flavor").shift().over("household_code").alias("previous_flavor"),
407                   (pl.col("week").diff().over("household_code").dt.total_days() / 7).alias("gap_weeks"))
408     .with_columns((pl.col("single_flavor").is_not_null() & pl.col("previous_flavor").is_not_null()
409                    & (pl.col("gap_weeks") == 1)).fill_null(False).alias("eligible"))
410     .with_columns((~(pl.col("eligible") & (pl.col("single_flavor") == pl.col("previous_flavor")))).fill_null(False))
411                   .cast(pl.UInt64).cum_sum().alias("spell")))
```

Read this block:

- Line 402, `baskets`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.cum_sum`: Accumulate running totals, used to turn spell-start flags into spell IDs.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.over`: Evaluate a Polars expression within the specified group/window, preserving row-level results.
- `.total_days`: Convert a duration expression into its elapsed number of days before comparing weekly gaps.
- `.shift`: Move values by one row by default; over(household) prevents the shift crossing household boundaries.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 413–446:

```
413 with tempfile.TemporaryDirectory(prefix="baskets-", dir=output) as tmp:
414     checkpoint = Path(tmp) / "baskets.parquet"
415     baskets.sink_parquet(checkpoint, engine="streaming")
416     baskets = pl.scan_parquet(checkpoint)
417     baskets.sink_csv(output / "yogurt_weekly_baskets.csv", engine="streaming")
418     weekly.sink_csv(output / "yogurt_weekly_flavor_quantities.csv", engine="streaming")
419     transitions = (baskets.filter(pl.col("eligible")).group_by(["previous_flavor", "single_flavor"])
420                    .len().collect(engine="streaming").to_pandas())
421     if transitions.empty:
422         matrix = pd.DataFrame(0, index=flavors, columns=flavors)
423     else:
424         matrix = transitions.pivot(index="previous_flavor", columns="single_flavor",
    ↪ values="len").fillna(0).reindex(index=flavors, columns=flavors, fill_value=0)
425     matrix.index.name = "previous_flavor"
426     matrix.columns.name = "single_flavor"
427     matrix.to_csv(output / "yogurt_transition_counts.csv")
428     probabilities = matrix.div(matrix.sum(axis=1).replace(0, np.nan), axis=0)
429     probabilities.to_csv(output / "yogurt_transition_probabilities.csv")
430     print("\nYogurt transition counts", flush=True)
431     print(matrix.to_string(), flush=True)
```

```
    print("\nYogurt transition probabilities", flush=True)
    print(probabilities.to_string(float_format=lambda v: f"{v:.2f}"), flush=True)
    spells = (baskets.filter(pl.col("single_flavor").is_not_null()).group_by(["household_code", "spell"])
              .agg(pl.col("single_flavor").first().alias("flavor"), pl.len().alias("weeks")))
    spells_path = Path(tmp) / "spells.parquet"
    spells.sink_parquet(spells_path, engine="streaming")
    spells = pl.scan_parquet(spells_path)
    spells.sink_csv(output / "yogurt_single_flavor_spells.csv", engine="streaming")
    spell_mean = spells.select(pl.col("weeks").mean()).collect(engine="streaming").item()
    bm = baskets.select(
        pl.len().alias("weeks"), (100 * (pl.col("flavors_in_basket") > 1).mean()).alias("mixed"),
        (100 * (pl.col("packages") > 7).mean()).alias("above_seven"), pl.col("eligible").sum().alias("pairs"),
        (100 * (pl.col("single_flavor") !=
    pl.col("previous_flavor")).filter(pl.col("eligible")).mean()).alias("switch_rate"),
        pl.col("gap_weeks").median().alias("gap"),
    ).collect(engine="streaming").row(0, named=True)
```

Read this block:

- Line 413: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 414, `checkpoint`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 415: call `baskets.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 416, `baskets`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 417: call `baskets.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 418: call `weekly.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 419, `transitions`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 421: test `transitions.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 422, `matrix`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 424, `matrix`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 425, `matrix.index.name`: Set this name to the literal text `'previous_flavor'`.
- Line 426, `matrix.columns.name`: Set this name to the literal text `'single_flavor'`.
- Line 427: call `matrix.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 428, `probabilities`: Set this name to the result of calling `matrix.div(...)`; the arguments and preceding method chain are shown above.
- Line 429: call `probabilities.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 430: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 431: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 432: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 433: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 434, `spells`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.

- Line 436, `spells_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 437: call `spells.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 438, `spells`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 439: call `spells.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 440, `spell_mean`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 441, `bm`: Set this name to the result of calling `.row(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.div`: Divide a pandas object with explicit alignment options; axis=0 aligns the denominator by rows.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.
- `.median`: Compute the middle-value statistic rather than a mean; global medians cannot generally be averaged across batches.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 447–454:

```
447 rows = [
448     ("Matched households", hh_metrics[0]), ("Observed yogurt purchase weeks", bm["weeks"]),
449     ("Mean flavors bought per household", hh_metrics[1]), ("Mean household flavor HHI (0 to 1)", hh_metrics[2]),
450     ("Mixed-flavor weeks (%)", bm["mixed"]), ("Weeks above seven packages (%)", bm["above_seven"]),
451     ("Eligible adjacent single-flavor week pairs", bm["pairs"]), ("Switch rate in eligible pairs (%)",
    ↪  bm["switch_rate"]),
452     ("Mean single-flavor spell length (consecutive calendar weeks)", spell_mean), ("Median gap between purchase
    ↪  weeks", bm["gap"]),
453 ]
454 save_table(pd.DataFrame(rows, columns=["metric", "value"]), output, "yogurt_dynamics", "Matched yogurt sample:
    ↪  purchase dynamics")
```

Read this block:

- Line 447, `rows`: Set this name to a list containing 10 displayed entries.

- Line 454: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 455–455:

```
455 return shares.to_pandas(), probabilities
```

Read this block:

- Line 455: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

make hhi figures

Original function: `make_hhi_figures`. Source begins at line 458.

This reads just the saved `flavor_HHI` column and writes histogram and CDF PDFs. `usecols` avoids reading irrelevant household fields. `.dropna().to_numpy()` leaves a numeric vector. If it is empty, a bare return avoids dividing by zero. The histogram weights each household by `100 / n`, so heights are percentages. `linspace(0,1,41)` defines 40 bins.

The CDF uses sorted observations and cumulative percentages. `np.r_` concatenates an initial zero with the vectors to draw the starting segment. `where="post"` creates a right-continuous step plot. A loop over two `kind` strings shares styling/export code without confusing their different calculations. It reuses saved household summaries instead of rerunning a large private-data pipeline.

Definition and stated intent:

Source lines 458–459:

```
458 def make_hhi_figures(output):
459     """Equal-household distributions using all flavor labels, without top-20 truncation."""
```

`def` creates this callable; the parentheses name its inputs (`output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 460–464:

```
460 import matplotlib
461 matplotlib.use("Agg")
462 import matplotlib.pyplot as plt
463
464 values = pd.read_csv(output / "yogurt_household_flavors.csv",
    ↪  usecols=["flavor_HHI"])["flavor_HHI"].dropna().to_numpy()
```

Read this block:

- Line 460: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 461: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 462: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 464, `values`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.dropna`: Remove pandas missing observations before the following calculation.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 465–466:

```
465 if not len(values):
466     return
```

Read this block:

- Line 465: test `not len(values)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 466: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 467–479:

```
467 for kind in ["histogram", "cdf"]:
468     fig, ax = plt.subplots(figsize=(7, 5))
469     if kind == "histogram":
470         ax.hist(values, bins=np.linspace(0, 1, 41), weights=np.full(len(values), 100 / len(values)),
    ↪  color="steelblue")
471         ax.set_ylabel("Households (%)")
472     else:
473         ax.step(np.r_[0., np.sort(values)], np.r_[0., 100 * np.arange(1, len(values) + 1) / len(values)],
    ↪  where="post")
474         ax.set_ylabel("Cumulative households (%)")
475     ax.set(xlim=(0, 1), xlabel="Household flavor HHI (quantity shares)", title="Yogurt flavor concentration")
476     fig.tight_layout()
477     fig.savefig(output / f"flavor_hhi_{kind}.pdf")
478     plt.close(fig)
479     print(f"Saved HHI figure: {output / f'flavor_hhi_{kind}.pdf'}", flush=True)
```

Read this block:

- Line 467: for each entry yielded by `['histogram', 'cdf']`, bind/unpack it into `kind` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 468, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 469: test `kind == 'histogram'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 470: call `ax.hist` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 471: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 473: call `ax.step` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 474: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 475: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 476: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 477: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 478: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 479: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.hist`: Bin observed values, with optional weights controlling whether heights are counts, percentages, or another unit.
- `.set_ylabel`: Set the vertical-axis label, whose unit should match the plotted values.
- `.step`: Draw an empirical step curve; where='post' extends each value to the right.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

make figures

Original function: `make_figures`. Source begins at line 482.

Inputs are already computed flavor-share and transition tables. Sorting then `head(20)` selects a readable subset for plots, and `reindex` aligns matrix rows and columns with the displayed flavor order. It does not recompute transition denominators within that subset, so a plotted row need not sum to one.

`fig, ax = plt.subplots` separates a Figure container from its plotting Axes. `ax.bar` draws shares; `ax.imshow` colors transition cells. Two nested index loops place formatted text inside each cell. Tick rotation improves long-label readability. `tight_layout`, `savefig`, and `close` finish each output. Plotting inside the data aggregation itself would make it harder to skip figures or recreate them from existing summary tables.

Definition and stated intent:

Source lines 482–483:

```
482 def make_figures(shares, transitions, output):
483     # Limit figure dimensions; all flavors remain in the saved tables.
```

`def` creates this callable; the parentheses name its inputs (`shares`, `transitions`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 484–491:

```
484 shares = shares.sort_values("packages", ascending=False).head(20)
485 flavors = shares["flavor"].tolist()
486 transitions = transitions.reindex(index=flavors, columns=flavors)
487 # Import plotting only if requested (use --no-figures for tables alone).
488 import matplotlib
489 
490 matplotlib.use("Agg")
491 import matplotlib.pyplot as plt
```

Read this block:

- Line 484, `shares`: Set this name to the result of calling `.head(...)`; the arguments and preceding method chain are shown above.
- Line 485, `flavors`: Set this name to the result of calling `shares['flavor'].tolist(...)`; the arguments and preceding method chain are shown above.
- Line 486, `transitions`: Set this name to the result of calling `transitions.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 488: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 490: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 491: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 493–498:

```
493 fig, ax = plt.subplots(figsize=(max(6, len(flavors) * 0.35), 5))
494 ax.bar(shares["flavor"], shares["percent_packages"], color="steelblue")
495 ax.tick_params(axis="x", labelrotation=90)
496 ax.set(ylabel="Share of purchased packages (%)", title="Yogurt flavor shares (up to 20 most purchased flavors)")
497 fig.tight_layout()
498 fig.savefig(output / "flavor_shares.pdf")
```

Read this block:

- Line 493, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 494: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 495: call `ax.tick_params` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 496: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 497: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 498: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `len`: Count elements/rows rather than summing their numerical values.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.
- `.tick_params`: Adjust axis tick appearance, for example rotating long category labels.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 499–512:

```
499 plt.close(fig)
500 side = max(6, len(flavors) * 0.4)
501 fig, ax = plt.subplots(figsize=(side, side))
502 image = ax.imshow(transitions.to_numpy(), vmin=0, vmax=1, cmap="Blues")
503 ax.set(
504     xticks=range(len(flavors)),
505     yticks=range(len(flavors)),
506     xticklabels=flavors,
507     yticklabels=flavors,
508     xlabel="Current flavor",
509     ylabel="Previous flavor",
510     title="Adjacent single-flavor purchase weeks",
511 )
512 ax.tick_params(axis="x", labelrotation=90)
```

Read this block:

- Line 499: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 500, `side`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 501, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 502, `image`: Set this name to the result of calling `ax.imshow(...)`; the arguments and preceding method chain are shown above.
- Line 503: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 512: call `ax.tick_params` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.
- `.imshow`: Display a numerical matrix as colored cells; row/column label alignment remains the caller's responsibility.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 513–522:

```
513 for i in range(len(flavors)):
514     for j in range(len(flavors)):
515         value = transitions.iloc[i, j]
516         ax.text(
517             j,
518             i,
519             f"{value:.2f}" if pd.notna(value) else "N/A",
520             ha="center",
521             va="center",
522         )
```

Read this block:

- Line 513: for each entry yielded by `range(len(flavors))`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 514: for each entry yielded by `range(len(flavors))`, bind/unpack it into `j` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 515, `value`: Set this name to the entry/slice selected by `transitions.iloc[i, j]`; brackets select from the object to their left.
- Line 516: call `ax.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 523–526:

```
523 fig.colorbar(image, ax=ax)
524 fig.tight_layout()
525 fig.savefig(output / "flavor_transitions.pdf")
```

```
526 plt.close(fig)
```

Read this block:

- Line 523: call `fig.colorbar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 524: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 525: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 526: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.colorbar`: Add a legend relating matrix colors to numerical values.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

run

Original function: `run`. Source begins at line 529.

This orchestrates the summary workflow and writes final artifacts. `sources` and `source_partitions` are dictionaries keyed by sample prefix, keeping full panel and matched yogurt objects separate. It optionally supplements flavor metadata, then prepares each population, writes reusable hash partitions, computes household/demographic summaries, and audits annual coverage.

The second loop runs detailed then grouped flavor taxonomies. Partition-level weekly outputs remain sparse, while small audit tables are summed. The last reference therefore holds the grouped mapping, which is saved for estimation. The final export intentionally reopens the original purchase file, rather than the demographic projection, to preserve store/DMA and other fields. A semi join keeps eligible households and a left join attaches the resolved flavor.

`prepared.drop("flavor")` avoids two competing label columns when reprocessing an already labeled input. The resolved-path check prevents writing over the original source. This orchestration keeps the population restrictions explicit; running the estimator on an arbitrary intermediate table can produce another sample even if its filename looks similar.

Definition and stated intent:

Source lines 529–529:

```
529 def run(args, output, work):
```

`def` creates this callable; the parentheses name its inputs (`args`, `output`, `work`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 530–531:

```
530 products_extra = args.products_extra or args.data_dir / "RMS/combined/products_extra.parquet"
531 flavor_path = args.flavors
```

Read this block:

- Line 530, `products_extra`: Set this name to the Boolean condition `args.products_extra or args.data_dir / 'RMS/combined/products_...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 531, `flavor_path`: Set this name to the attribute `args.flavors` of the object before the dot.

Source lines 532–539:

```
532 if products_extra.exists():
533     recovered_path = output / "recovered_flavor_reference.csv"
534     if recovered_path.resolve() == flavor_path.resolve():
535         recovered_path = output / "supplemented_flavor_reference.csv"
536     recover_flavor_reference(flavor_path, products_extra, recovered_path)
537     flavor_path = recovered_path
538 else:
539     print(f"Flavor recovery source not found: {products_extra}; using the CSV reference.", flush=True)
```

Read this block:

- Line 532: test `products_extra.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 533, `recovered_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 534: test `recovered_path.resolve() == flavor_path.resolve()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 535, `recovered_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 536: call `recover_flavor_reference` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 537, `flavor_path`: Set this name to the object currently named `recovered_path`; plain assignment does not copy it.
- Line 539: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `recover_flavor_reference`: Call this project helper/callable with the displayed arguments; its own walk-through gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 540–543:

```
540 sources = {}
541 prepared_eligible = None
542 print("Summary sample: single-person purchase records (household_size == 1).", flush=True)
543 source_partitions = {}
```

Read this block:

- Line 540, `sources`: Set this name to a dictionary from the displayed keys and values.
- Line 541, `prepared_eligible`: Set this name to the literal value `None`.
- Line 542: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 543, `source_partitions`: Set this name to a dictionary from the displayed keys and values.

Source lines 544–581:

```
544 for prefix, filename in [("full_panel", "merged_panel.parquet"), ("matched_yogurt",
    ↪ "yogurt_estimation_sample.parquet")]:
545     source = scan_purchases(args.data_dir / filename, args.first_year, args.last_year)
546     if "household_size" not in pl.scan_parquet(args.data_dir / filename).collect_schema().names():
547         raise ValueError(f"{filename}: single-person summaries require household_size.")
548     if prefix == "matched_yogurt":
549         # Preserve all-household retail coverage in the prepared estimation
```

```
550         # input. est.py restricts demand records while retaining these quotes.
551         covered_all = source.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
552         prepared_eligible = (repeated_yogurt_households(covered_all)
553                              .select("household_code").unique().collect(engine="streaming"))
554     source = single_person_purchases(source)
555     if prefix == "matched_yogurt":
556         covered = (pl.col("price").is_finite() & (pl.col("price") > 0)).fill_null(False)
557         audit = source.select(pl.len().alias("purchase_lines_before_price_filter"),
558                               (~covered).sum().alias("purchase_lines_without_price_coverage"))
559         save_table(audit, output, "yogurt_price_coverage", "Yogurt matched-price coverage")
560         source = source.filter(covered)
561         source = repeated_yogurt_households(source)
562     parts = partition_purchases(source, work, prefix, args.household_batches)
563     source_partitions[prefix] = parts
564     source = pl.concat(parts)
565     sources[prefix] = source
566     households = household_batches(source, work, prefix, args.household_batches, partitions=parts)
567     if households.select(pl.len()).collect(engine="streaming").item() == 0:
568         raise ValueError(f"Empty {prefix} sample in selected years.")
569     demographic_tables(households, output, prefix)
570     households.sink_csv(output / f"{prefix}_households.csv", engine="streaming")
571     # Exact distinct-trip counts for one year at a time bound aggregation state.
572     years = source.select("panel_year").unique().sort("panel_year").collect(engine="streaming")["panel_year"]
573     coverage_rows = []
574     for year in years:
575         coverage_rows.append(source.filter(pl.col("panel_year") == year).select(
576             pl.lit(year).alias("panel_year"),
577             pl.col("household_code").n_unique().alias("households"),
578             pl.col("trip_code_uc").n_unique().alias("trips"), pl.len().alias("purchase_lines"),
579             pl.col("quantity").sum().alias("packages")).collect(engine="streaming"))
580     coverage = pl.concat(coverage_rows)
581     save_table(coverage, output, f"{prefix}_year_coverage", "Sample coverage by year")
```

Read this block:

- Line 544: for each entry yielded by `[('full_panel', 'merged_panel.parquet'), ('matched_yogurt', 'y...`, bind/unpack it into `(prefix, filename)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 545, `source`: Set this name to the result of calling `scan_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 546: test `'household_size' not in pl.scan_parquet(args.data_dir / filename).collect_sch...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 547: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 548: test `prefix == 'matched_yogurt'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 551, `covered_all`: Set this name to the result of calling `source.filter(...)`; the arguments and preceding method chain are shown above.
- Line 552, `prepared_eligible`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 554, `source`: Set this name to the result of calling `single_person_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 555: test `prefix == 'matched_yogurt'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 556, `covered`: Set this name to the result of calling `.fill_null(...)`; the arguments and preceding method chain are shown above.
- Line 557, `audit`: Set this name to the result of calling `source.select(...)`; the arguments and preceding method chain are shown above.
- Line 559: call `save_table` with the displayed arguments. No assignment stores its return here; its docu-

mented side effect or assertion is what matters.
- Line 560, `source`: Set this name to the result of calling `source.filter(...)`; the arguments and preceding method chain are shown above.
- Line 561, `source`: Set this name to the result of calling `repeated_yogurt_households(...)`; the arguments and preceding method chain are shown above.
- Line 562, `parts`: Set this name to the result of calling `partition_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 563, `source_partitions[prefix]`: Store the object currently named `parts`; plain assignment does not copy it in the selected entry/column.
- Line 564, `source`: Set this name to the result of calling `pl.concat(...)`; the arguments and preceding method chain are shown above.
- Line 565, `sources[prefix]`: Store the object currently named `source`; plain assignment does not copy it in the selected entry/column.
- Line 566, `households`: Set this name to the result of calling `household_batches(...)`; the arguments and preceding method chain are shown above.
- Line 567: test `households.select(pl.len()).collect(engine='streaming').item() == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 568: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 569: call `demographic_tables` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 570: call `households.sink_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 572, `years`: Set this name to the entry/slice selected by `source.select('panel_year').unique().sort('panel_year').c...`; brackets select from the object to their left.
- Line 573, `coverage_rows`: Set this name to the list `[]`.
- Line 574: for each entry yielded by `years`, bind/unpack it into `year` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 575: call `coverage_rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 580, `coverage`: Set this name to the result of calling `pl.concat(...)`; the arguments and preceding method chain are shown above.
- Line 581: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `scan_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `single_person_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `partition_purchases`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_batches`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `demographic_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.sink_csv`: Execute/write the query result as CSV; unlike select/filter, this is a filesystem side effect.
- `save_table`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `repeated_yogurt_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.
- `.n_unique`: Count distinct Polars values within the relevant aggregation scope.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 582–606:

```
582 for detailed in [True, False]:
583     folder = output if detailed else output / "aggregated_flavors"
584     folder.mkdir(parents=True, exist_ok=True)
585     reference = flavor_reference(flavor_path, detailed=detailed, output=output)
586     flavors = sorted(reference["flavor"].unique().to_list()) if detailed else FLAVORS
587     if detailed:
588         reference.write_csv(output / "detailed_flavor_reference.csv")
589     # Process household batches independently, keeping weekly baskets long.
590     audit_total = None
591     tag = "detailed" if detailed else "grouped"
592     parts = source_partitions["matched_yogurt"]
593     for batch, subset in enumerate(parts):
594         began = time.monotonic()
595         print(f"{tag}: yogurt week batch {batch + 1}/{len(parts)}", flush=True)
596         weekly, audit = yogurt_weeks(subset, reference, flavors, output=folder)
597         weekly.sink_parquet(work / f"{tag}_weeks_{batch}.parquet", engine="streaming")
598         print(f"{tag}: yogurt week batch {batch + 1} completed in {time.monotonic() - began:.1f}s", flush=True)
599         audit_total = audit if audit_total is None else pl.concat([audit_total, audit]).select(pl.all().sum())
600     if detailed:
601         save_table(audit_total, output, "yogurt_sample_audit", "Yogurt sample audit")
602     weekly = pl.scan_parquet(work / f"{tag}_weeks_*.parquet")
603     shares, transitions = flavor_summaries(weekly, folder, flavors)
604     if not args.no_figures:
605         make_hhi_figures(folder)
606         make_figures(shares, transitions, folder)
```

Read this block:

- Line 582: for each entry yielded by `[True, False]`, bind/unpack it into `detailed` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 583, `folder`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 584: call `folder.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 585, `reference`: Set this name to the result of calling `flavor_reference(...)`; the arguments and preceding method chain are shown above.
- Line 586, `flavors`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 587: test `detailed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 588: call `reference.write_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 590, `audit_total`: Set this name to the literal value `None`.
- Line 591, `tag`: Set this name to the true-branch value if `detailed` holds, otherwise the else-branch value.
- Line 592, `parts`: Set this name to the entry/slice selected by `source_partitions['matched_yogurt']`; brackets select from the object to their left.
- Line 593: for each entry yielded by `enumerate(parts)`, bind/unpack it into `(batch, subset)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 594, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 595: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 596, `(weekly, audit)`: Unpack the result of calling `yogurt_weeks(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 597: call `weekly.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 598: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 599, `audit_total`: Set this name to the true-branch value if `audit_total is None` holds, otherwise the else-branch value.
- Line 600: test `detailed`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 601: call `save_table` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 602, `weekly`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 603, `(shares, transitions)`: Unpack the result of calling `flavor_summaries(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 604: test `not args.no_figures`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 605: call `make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 606: call `make_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `flavor_reference`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `flavor_summaries`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- `.write_csv`: Write an eager Polars table to CSV; it does not return a newly analyzed dataset.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `yogurt_weeks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.sink_parquet`: Execute/write a lazy result to Parquet instead of first keeping its whole result in a Python table.
- `make_hhi_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another

expression.

Source lines 609–616:

```
609 reference.write_csv(output / "estimation_flavor_reference.csv")
610 print(f"Saved estimation flavor mapping to {output / 'estimation_flavor_reference.csv'}", flush=True)
611 # Retain every original purchase column needed downstream, including store
612 # and DMA fields that demographic projections do not otherwise need.
613 eligible = prepared_eligible
614 prepared = pl.scan_parquet(args.data_dir / "yogurt_estimation_sample.parquet")
615 original_names = prepared.collect_schema().names()
616 price_column = next((c for c in ["price", "price_rms"] if c in original_names), None)
```

Read this block:

- Line 609: call `reference.write_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 610: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 613, `eligible`: Set this name to the object currently named `prepared_eligible`; plain assignment does not copy it.
- Line 614, `prepared`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 615, `original_names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 616, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 617–618:

```
617 if price_column is None:
618     raise ValueError("Yogurt purchase table needs price or price_rms for matched-price coverage.")
```

Read this block:

- Line 617: test `price_column is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 618: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 619–620:

```
619 prepared = prepared.filter(pl.col(price_column).cast(pl.Float64).is_finite()
620                            & (pl.col(price_column).cast(pl.Float64) > 0))
```

Read this block:

- Line 619, `prepared`: Set this name to the result of calling `prepared.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.

Source lines 621–622:

```
621 if "flavor" in original_names:
622     prepared = prepared.drop("flavor")
```

Read this block:

- Line 621: test `'flavor' in original_names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 622, `prepared`: Set this name to the result of calling `prepared.drop(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.

Source lines 623–628:

```
prepared = (prepared.filter(pl.col("panel_year").is_between(args.first_year, args.last_year))
            .with_columns(normalized_upc_expr(),
                          pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date())
            .join(eligible.lazy(), on="household_code", how="semi")
            .join(reference.lazy(), on="upc", how="left"))
sample_path = output / "yogurt_estimation_sample.parquet"
```

Read this block:

- Line 623, `prepared`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 628, `sample_path`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.

Source lines 629–630:

```
if sample_path.resolve() == (args.data_dir / "yogurt_estimation_sample.parquet").resolve():
    raise ValueError("Choose an output folder separate from the original data directory.")
```

Read this block:

- Line 629: test `sample_path.resolve() == (args.data_dir / 'yogurt_estimation_sample.parquet')...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 630: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 631–633:

```
prepared.sink_parquet(sample_path, engine="streaming")
print(f"Saved full prepared yogurt purchase table to {sample_path}", flush=True)
print(f"Saved summaries to {output}", flush=True)
```

Read this block:

- Line 631: call `prepared.sink_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 632: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 633: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 636.

This parses summary options and chooses ordinary pipeline work or HHI-only work. `args.output or default_path` uses a fallback when the optional output is None. The HHI-only branch checks its saved input, makes graphs, optionally repeats for grouped summaries, then returns before raw-data setup. It is a true early exit, not merely a plotting flag.

Ordinary runs create the output directory and enter a TemporaryDirectory context there, passing its Path to `run`. Context management ensures intermediate partitions are cleaned on ordinary completion or exceptions. Keeping temporary work under output makes disk requirements predictable. `--no-figures` and `--hhi-only` are rejected together because their requested actions conflict.

Definition and stated intent:

Source lines 636–636:

```
636 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 637–642:

```
637 parser = argparse.ArgumentParser(description=__doc__)
638 parser.add_argument("--data-dir", type=Path, default=DATA_DIR)
639 parser.add_argument("--flavors", type=Path, default=FLAVOR_FILE)
640 parser.add_argument("--products-extra", type=Path)
641 parser.add_argument("--output", type=Path)
642 parser.add_argument("--first-year", type=int, default=2006)
```

Read this block:

- Line 637, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 638: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 639: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 640: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 641: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 642: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.

Source lines 643–649:

```
643 parser.add_argument("--last-year", type=int, default=2015)
644 parser.add_argument("--household-batches", type=int, default=16,
645                     help="Household disk partitions; more batches reduce aggregation memory (default 16).")
646 parser.add_argument("--no-figures", action="store_true")
647 parser.add_argument("--hhi-only", action="store_true",
648                     help="Plot HHI from existing yogurt_household_flavors.csv in --output; skip raw data and all
                     ↪  summaries.")
```

```
649 args = parser.parse_args()
```

Read this block:

- Line 643: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 644: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 646: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 647: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 649, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 650–651:

```
650 if args.hhi_only and args.no_figures:
651     parser.error("--hhi-only cannot be combined with --no-figures")
```

Read this block:

- Line 650: test `args.hhi_only and args.no_figures`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 651: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 652–653:

```
652 if args.household_batches < 1:
653     parser.error("--household-batches must be positive")
```

Read this block:

- Line 652: test `args.household_batches < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 653: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 654–654:

```
654 output = args.output or args.data_dir / "summary_outputs"
```

Read this block:

- Line 654, `output`: Set this name to the Boolean condition `args.output or args.data_dir / 'summary_outputs'`; it can be a scalar or a row/array mask depending on its inputs.

Source lines 655–663:

```
655 if args.hhi_only:
656     source = output / "yogurt_household_flavors.csv"
657     if not source.exists():
658         parser.error(f"HHI-only needs the saved detailed household table: {source}")
659     make_hhi_figures(output)
```

```
660     grouped = output / "aggregated_flavors"
661     if (grouped / "yogurt_household_flavors.csv").exists():
662         make_hhi_figures(grouped)
663     return
```

Read this block:

- Line 655: test `args.hhi_only`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 656, `source`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 657: test `not source.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 658: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 659: call `make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 660, `grouped`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 661: test `(grouped / 'yogurt_household_flavors.csv').exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 662: call `make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 663: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `make_hhi_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 664–664:

```
664 output.mkdir(parents=True, exist_ok=True)
```

Read this block:

- Line 664: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 665–666:

```
665 with tempfile.TemporaryDirectory(prefix="summary-work-", dir=output) as tmp:
666     run(args, output, Path(tmp))
```

Read this block:

- Line 665: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 666: call `run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.

- `run`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

est_code: source walkthrough

est_code/bootstrap_se.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–5:

```
"""Household bootstrap for the two-stage demand estimator."""
import numpy as np
import pandas as pd

from price_control import add_price_controls
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 5: from `price_control`, import `add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.

resample households

Original function: `resample_households`. Source begins at line 8.

Input is the weekly frame and a NumPy random Generator. Output is a concatenated bootstrap sample with the same number of household draws as the original. The comprehension stores whole household subtables. `rng.integers(0, H, size=H)` draws indices with replacement; its upper endpoint H is excluded.

`enumerate(selected)` supplies a fresh ID and selected old-group position. `.copy()` is essential before changing that ID so the original history is not modified. If selection is `[2,2,0]`, two copies of old household 2 get new IDs 0 and 1, keeping their likelihoods separate. Concatenating with `ignore_index=True` gives clean row labels. Resampling individual weeks would break persistent household preferences and within-household dependence.

Definition and stated intent:

Source lines 8–13:

```
def resample_households(frame, rng):
    """Keep whole histories; repeated households get distinct bootstrap IDs.

    Stocks/Xi stay attached to the original histories. Rebuilding stocks after
    dropping incomplete price weeks would incorrectly erase observed purchases.
    """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `rng`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 14–16:

```
14 groups = [group for _, group in frame.groupby("household_code", sort=True)]
15 selected = rng.integers(0, len(groups), size=len(groups))
16 samples = []
```

Read this block:

- Line 14, `groups`: Set this name to a list computed by the displayed for-expression.
- Line 15, `selected`: Set this name to the result of calling `rng.integers(...)`; the arguments and preceding method chain are shown above.
- Line 16, `samples`: Set this name to the list `[]`.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.integers`: Draw integer positions with replacement by default; the high endpoint is excluded.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 17–20:

```
17 for new_id, old_id in enumerate(selected):
18     group = groups[old_id].copy()
19     group["household_code"] = new_id
20     samples.append(group)
```

Read this block:

- Line 17: for each entry yielded by `enumerate(selected)`, bind/unpack it into `(new_id, old_id)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 18, `group`: Set this name to the result of calling `groups[old_id].copy(...)`; the arguments and preceding method chain are shown above.
- Line 19, `group['household_code']`: Store the object currently named `new_id`; plain assignment does not copy it in the selected entry/column.
- Line 20: call `samples.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 21–21:

```
21 return pd.concat(samples, ignore_index=True)
```

Read this block:

- Line 21: return the result of calling `pd.concat(...)`; the arguments and preceding method chain are shown above. This ends the call.

bootstrap standard errors

Original function: `bootstrap_standard_errors`. Source begins at line 24.

The `estimate` argument is a callable, not an estimate array. Each repetition resamples households, refits enabled price controls, calls `estimate(sample, initial)`, and records its result. `record.update` adds named fields to an existing dictionary. Accepted vectors go to `values`; all attempts go to `records`, so failure does not disappear from the audit trail.

The `try/except` catches expected data/linear-algebra problems. Every attempt checkpoints the records as CSV. After the loop, the threshold requires at least 20 usable fits and at least 80% success. Returning `(None, report)` explicitly signals inference was withheld. Otherwise, `std(ddof=1)` uses sample rather than population SD, `quantile` forms empirical percentile intervals, and `np.cov(..., rowvar=False)` treats columns as parameters. Analytic Hessian SEs would be cheaper but would require handling the estimated first-stage residuals and household dependence; this implementation uses a two-stage cluster bootstrap.

Definition and stated intent:

Source lines 24–30:

```
24 def bootstrap_standard_errors(frame, args, estimate, initial, names):
25     """Refit price controls AND demand in every household-cluster replication.
26 
27     estimate(frame, initial) returns a scipy optimizer result. All replications
28     use the same integration draws as the main fit. Retail instruments stay
29     fixed: this is inference conditional on the observed retail price universe.
30     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `args`, `estimate`, `initial`, `names`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 31–33:

```
31 rng = np.random.default_rng(args.seed + 10000)
32 records = []
33 values = []
```

Read this block:

- Line 31, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 32, `records`: Set this name to the list `[]`.
- Line 33, `values`: Set this name to the list `[]`.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.

Source lines 34–53:

```
34 for replication in range(args.bootstrap_reps):
35     record = {"replication": replication + 1, "converged": False}
36     try:
37         sample = resample_households(frame, rng)
38         if not args.no_price_control:
39             sample, _ = add_price_controls(sample)
40         fit = estimate(sample, initial)
41         record.update(converged=bool(fit.success), objective=float(fit.fun),
42                       message=str(fit.message))
43         record.update(dict(zip(names, fit.x)))
44         if fit.success and np.isfinite(fit.x).all() and np.isfinite(fit.fun):
45             values.append(fit.x)
46     except (ValueError, np.linalg.LinAlgError) as error:
47         # An unidentifiable resample is recorded, never silently accepted.
48         record["message"] = str(error)
49     records.append(record)
50     # Checkpoint after each replication in case a long cluster job ends.
```

```
51      pd.DataFrame(records).to_csv(args.output / "bootstrap_draws.csv", index=False)
52      print(f"Bootstrap {replication+1}/{args.bootstrap_reps}: "
53            f"{len(values)} usable fits", flush=True)
```

Read this block:

- Line 34: for each entry yielded by `range(args.bootstrap_reps)`, bind/unpack it into `replication` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 35, `record`: Set this name to a dictionary with named entries `replication`, `converged`.
- Line 36: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 37, `sample`: Set this name to the result of calling `resample_households(...)`; the arguments and preceding method chain are shown above.
- Line 38: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 39, `(sample, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 40, `fit`: Set this name to the result of calling `estimate(...)`; the arguments and preceding method chain are shown above.
- Line 41: call `record.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `record.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 44: test `fit.success and np.isfinite(fit.x).all() and np.isfinite(fit.fun)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 45: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 46: catch `(ValueError, np.linalg.LinAlgError)` from the preceding try; the optional as-name holds the error object for logging.
- Line 48, `record['message']`: Store the result of calling `str(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 49: call `records.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51: call `pd.DataFrame(records).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `resample_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `estimate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `str`: Convert to text for paths, messages, or serialization.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 54–58:

```
valid = len(values)
report = dict(method=("household bootstrap" if args.no_price_control
                      else "household bootstrap, refitting both stages"),
              requested=args.bootstrap_reps, usable=valid,
              conditional_on="retail instruments and fixed integration draws")
```

Read this block:

- Line 54, `valid`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 55, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 60–62:

```
if valid < max(20, int(np.ceil(.8 * args.bootstrap_reps))):
    report["status"] = "insufficient converged replications; no SEs reported"
    return None, report
```

Read this block:

- Line 60: test `valid < max(20, int(np.ceil(0.8 * args.bootstrap_reps)))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 61, `report['status']`: Store the literal text `'insufficient converged replications; no SEs reported'` in the selected entry/column.
- Line 62: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.ceil`: Round upward, so the minimum usable-bootstrap threshold meets the requested success fraction.

Source lines 63–70:

```
estimates = np.asarray(values)
table = pd.DataFrame({"parameter": names,
                      "std_error": estimates.std(axis=0, ddof=1),
                      "ci_lower": np.quantile(estimates, .025, axis=0),
                      "ci_upper": np.quantile(estimates, .975, axis=0)})
covariance = pd.DataFrame(np.cov(estimates, rowvar=False), index=names, columns=names)
covariance.to_csv(args.output / "covariance.csv")
report["status"] = "computed; percentile 95% intervals"
```

Read this block:

- Line 63, `estimates`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 64, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.

- Line 68, `covariance`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 69: call `covariance.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 70, `report['status']`: Store the literal text `'computed; percentile 95% intervals'` in the selected entry/column.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `.quantile`: Find empirical quantiles, supplying percentile bootstrap endpoints rather than normal-Wald formulas.
- `.cov`: Compute covariance; rowvar=False means columns represent parameters rather than treating rows as variables.

Source lines 71–71:

```
71 return table, report
```

Read this block:

- Line 71: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

est_code/est.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–12:

```
1 """Weekly yogurt demand: fixed or mixed logit, with or without gamma.
2
3 Read README.md first. Run `python3 est.py --help` for server paths and options.
4 The raw data are not required for importing or testing the likelihood.
5 """
6
7 from pathlib import Path
8 import argparse
9 import json
10 import time
11
12 import numpy as np
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 9: import `json`. An as-name is a convenient local alias, not another library.
- Line 10: import `time`. An as-name is a convenient local alias, not another library.
- Line 12: import `numpy as np`. An as-name is a convenient local alias, not another library.

Source lines 13–19:

```
13 import pandas as pd
14 import polars as pl
15 from scipy.optimize import minimize
16 from scipy.special import logsumexp
17
18 from price_control import check_dma, add_price_controls
19 from bootstrap_se import bootstrap_standard_errors
```

Read this block:

- Line 13: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 14: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 15: from `scipy.optimize`, import `minimize` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: from `scipy.special`, import `logsumexp` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 18: from `price_control`, import `check_dma, add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 19: from `bootstrap_se`, import `bootstrap_standard_errors` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 20–26:

```
20 from hessian_se import hessian_standard_errors
21 from wtp import implied_wtp
22
23 FLAVORS = ["other", "berry", "plain"]
24 DATA_DIR = Path("/scratch/dtm63837/Kilts_Panel/Data")
25 # Set this to the flavor-reference CSV on the Slurm server.
26 FLAVOR_FILE = DATA_DIR / "summary_outputs/estimation_flavor_reference.csv"
```

Read this block:

- Line 20: from `hessian_se`, import `hessian_standard_errors` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 21: from `wtp`, import `implied_wtp` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 23, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.
- Line 24, `DATA_DIR`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 26, `FLAVOR_FILE`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 824–825:

```
824 if __name__ == "__main__":
825     main()
```

Read this block:

- Line 824: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 825: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

normalize upcs

Original function: `normalize_upcs`. Source begins at line 29.

Job and contract. Accept a pandas Series of UPC-like values and return a Series of canonical string identifiers. `values.astype("string")` selects pandas' nullable string dtype; `.str` applies a string operation to every element. `.strip()` removes surrounding whitespace. `numeric` is a Boolean Series, not a single true/false answer.

`fullmatch` requires the entire string to fit the numeric pattern. Replacement removes a trailing decimal-zero suffix, then `lstrip("0")` removes leading zeros. The last `where` keeps the original text where `~numeric` is true and uses the canonical version elsewhere. Thus `"00123.0"` becomes `"123"`, `"000"` becomes `"0"`, and `"A001"` stays `"A001"`. The inner `.replace("", "0")` fixes the empty string produced when all digits were zero.

Why this choice? Converting to `float` then `int` looks shorter but can round long identifiers and fails on nonnumeric UPCs. A Python loop would work but duplicates the same string operations per row and loses convenient missing-value handling. `where` is easy to misread: its true branch retains the receiver.

Definition and stated intent:

Source lines 29–30:

```
29 def normalize_upcs(values):
30     """Canonical numeric UPC keys without floating-point conversion."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 31–33:

```
31 text = values.astype("string").str.strip()
32 numeric = text.str.fullmatch(r"[0-9]+(?:\.0+)?", na=False)
33 canonical = text.str.replace(r"\.0+$", "", regex=True).str.lstrip("0")
```

Read this block:

- Line 31, `text`: Set this name to the result of calling `.strip(...)`; the arguments and preceding method chain are shown above.
- Line 32, `numeric`: Set this name to the result of calling `text.str.fullmatch(...)`; the arguments and preceding method chain are shown above.
- Line 33, `canonical`: Set this name to the result of calling `.lstrip(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fullmatch`: Require the entire string to match a regex; contains would allow a partial match inside unrelated text.
- `.lstrip`: Remove any leading occurrences of the specified character set, such as zeros in numeric UPC text.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 34–34:

```
34 return text.where(~numeric, canonical.replace("", "0"))
```

Read this block:

- Line 34: return the result of calling `text.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

normalized upc expr

Original function: `normalized_upc_expr`. Source begins at line 37.

This returns a **Polars expression**, not a loaded Series. Calling it constructs a reusable recipe for the `upc` column. `pl.col` identifies that column; `cast(pl.String)` standardizes its type. The local names `text`, `numeric`, and `canonical` hold expression objects. Their calculations run later when the surrounding lazy query is executed.

Nested `when/then/otherwise` first decides whether a UPC is numeric, then whether stripping zeros would leave an empty identifier. `pl.lit("0")` means literal text, not a column named zero. `.alias("upc")` ensures `with_columns` replaces the intended column. This version avoids collecting an entire lazy table just to normalize one field. A pandas conversion would forfeit the lazy query's memory and projection advantages.

Definition and stated intent:

Source lines 37–37:

```
37 def normalized_upc_expr():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 38–40:

```
38 text = pl.col("upc").cast(pl.String).str.strip_chars()
39 numeric = text.str.contains(r"^[0-9]+(?:\.0+)?$")
40 canonical = text.str.replace(r"\.0+$", "").str.strip_chars_start("0")
```

Read this block:

- Line 38, `text`: Set this name to the result of calling `.strip_chars(...)`; the arguments and preceding method chain are shown above.
- Line 39, `numeric`: Set this name to the result of calling `text.str.contains(...)`; the arguments and preceding method chain are shown above.
- Line 40, `canonical`: Set this name to the result of calling `.strip_chars_start(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.strip_chars_start`: Trim characters from the start of each Polars string; it does not remove matching characters inside it.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 41–46:

```
41 return (
42     pl.when(numeric)
43     .then(pl.when(canonical == "").then(pl.lit("0")).otherwise(canonical))
```

```
    .otherwise(text)
    .alias("upc")
)
```

Read this block:

- Line 41: return the result of calling `.alias(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

read flavors

Original function: `read_flavors`. Source begins at line 49.

Input `path` is a CSV path; optional `products_extra` is a Parquet path. Output is a two-column pandas table with one row per UPC. This function has an early return: if the CSV already contains resolved `upc,flavor`, validation succeeds and the rest of the recovery/classification code is never executed.

`{"upc", "flavor"}.issubset(frame.columns)` asks whether both required labels exist. `missing = required - set(frame.columns)` is set subtraction. Validation uses `.any()` for "at least one invalid row" and `.all()` for "every label is allowed." `if products_extra is not None` distinguishes an omitted optional argument from a supplied path that happens not to exist.

The metadata path scans only useful columns and anti-joins existing reference UPCs. It appends absent UPCs, rather than overwriting existing classifications. `[... for c in columns if c != "upc"]` builds one aggregation expression per non-key field. `np.select` picks the first matching condition, so plain wins when both plain and berry text occur. A generic substring loop could be easier to write at first, but would make precedence and vectorized missing-value handling less explicit. `drop_duplicates(..., keep="first")` is a reconciliation rule; it does not prove the retained label is correct.

Definition and stated intent:

Source lines 49–50:

```
def read_flavors(path, products_extra=None):
    """Use the same plain-first classification as data_code/summ_stats.py."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `products_extra`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 51–51:

```
frame = pd.read_csv(path, dtype={"upc": str})
```

Read this block:

- Line 51, `frame`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 52–66:

```
52 if {"upc", "flavor"}.issubset(frame.columns):
53     # Consume the exact mapping saved by summ_stats; no recovery or relabeling.
54     frame["upc"] = normalize_upcs(frame["upc"])
55     if (
56         frame["upc"].isna().any()
57         or frame["upc"].eq("").any()
58         or frame["upc"].duplicated().any()
59     ):
60         raise ValueError(f"{path}: saved mapping needs unique, nonmissing UPCs.")
61     if not frame["flavor"].isin(FLAVORS).all():
62         raise ValueError(f"{path}: saved mapping must use other, berry, plain.")
63     print(
64         f"Loaded {len(frame):,} resolved UPC flavor labels from {path}.", flush=True
65     )
66     return frame[["upc", "flavor"]]
```

Read this block:

- Line 52: test `{'upc', 'flavor'}.issubset(frame.columns)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 54, `frame['upc']`: Store the result of calling `normalize_upcs(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 55: test `frame['upc'].isna().any() or frame['upc'].eq('').any() or frame['upc'].duplic...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 60: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 61: test `not frame['flavor'].isin(FLAVORS).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 62: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 63: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 66: return the entry/slice selected by `frame[['upc', 'flavor']]`; brackets select from the object to their left. This ends the call.

Syntax and calls used here:

- `normalize_upcs`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 67–67:

```
67 missing = {"upc", "flavor_descr"} - set(frame.columns)
```

Read this block:

- Line 67, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subex-

pressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 68–69:

```
68 if missing:
69     raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

Read this block:

- Line 68: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 69: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.

Source lines 70–70:

```
70 frame["upc"] = normalize_upcs(frame["upc"])
```

Read this block:

- Line 70, `frame['upc']`: Store the result of calling `normalize_upcs(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 71–72:

```
71 if frame["upc"].isna().any() or frame["upc"].eq("").any():
72     raise ValueError(f"{path}: flavor reference contains missing UPCs.")
```

Read this block:

- Line 71: test `frame['upc'].isna().any() or frame['upc'].eq('').any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 72: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 73–114:

```
73 if products_extra is not None:
74     products_extra = Path(products_extra)
75     if products_extra.exists():
76         products = pl.scan_parquet(products_extra)
77         names = products.collect_schema().names()
78         if not {"upc", "flavor_descr"}.issubset(names):
79             raise ValueError(
80                 f"{products_extra}: needs upc and flavor_descr columns."
81             )
82         columns = [
83             c
84             for c in ["upc", "flavor_descr", "product_descr", "panel_year"]
85             if c in names
86         ]
87         products = (
88             products.select(columns)
89             .with_columns(normalized_upc_expr())
90             .join(
91                 pl.DataFrame(
92                     {"upc": frame["upc"].dropna().unique().tolist()}
93                 ).lazy(),
```

```
                on="upc",
                how="anti",
            )
        )
        products = products.filter(
            pl.col("upc").is_not_null()
            & pl.col("flavor_descr").is_not_null()
            & ~pl.col("flavor_descr")
            .cast(pl.String)
            .str.strip_chars()
            .str.to_lowercase()
            .is_in(["", "na", "n/a", "nan", "null", "none"])
        )
        if "panel_year" in names:
            products = products.group_by("upc").agg(
                [
                    pl.col(c)
                    .sort_by("panel_year", nulls_last=True)
                    .first()
                    .alias(c)
                    for c in columns
```

Source lines 115–132:

```
                if c != "upc"
            ]
        )
    else:
        products = products.unique(
            subset="upc", keep="first", maintain_order=True
        )
    recovered = products.collect(engine="streaming").to_pandas()
    frame = pd.concat([frame, recovered], ignore_index=True)
    print(
        f"Recovered flavor labels for {len(recovered):,} additional UPCs from {products_extra}.",
        flush=True,
    )
else:
    print(
        f"Flavor recovery source not found: {products_extra}; using CSV reference only.",
        flush=True,
    )
```

Read this block:

- Line 73: test `products_extra is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 74, `products_extra`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 75: test `products_extra.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 76, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 77, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 78: test `not {'upc', 'flavor_descr'}.issubset(names)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 79: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 82, `columns`: Set this name to a list computed by the displayed for-expression, retaining only entries

passing its if filter.
- Line 87, `products`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 98, `products`: Set this name to the result of calling `products.filter(...)`; the arguments and preceding method chain are shown above.
- Line 107: test `'panel_year' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 108, `products`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 119, `products`: Set this name to the result of calling `products.unique(...)`; the arguments and preceding method chain are shown above.
- Line 122, `recovered`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 123, `frame`: Set this name to the result of calling `pd.concat(...)`; the arguments and preceding method chain are shown above.
- Line 124: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 129: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.sort_by`: Order values within an expression/group by another column before selecting a first value.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.dropna`: Remove pandas missing observations before the following calculation.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 133–133:

```
133 text = frame["flavor_descr"].fillna("")
```

Read this block:

- Line 133, `text`: Set this name to the result of calling `frame['flavor_descr'].fillna(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 134–135:

```
134 if "product_descr" in frame.columns:
135     text = text + " " + frame["product_descr"].fillna("")
```

Read this block:

- Line 134: test `'product_descr' in frame.columns`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 135, `text`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 136–146:

```
136 text = text.str.lower()
137 frame["flavor"] = np.select(
138     [
139         text.str.contains("plain|unflavored"),
140         text.str.contains("berry|straw|blue|rasp|black|cran|cherry|wildberry"),
141     ],
142     ["plain", "berry"],
143     default="other",
144 )
145 # Match summ_stats: retain missing labels and use the first CSV row per UPC.
146 conflicts = (frame.groupby("upc")["flavor"].nunique() > 1).sum()
```

Read this block:

- Line 136, `text`: Set this name to the result of calling `text.str.lower(...)`; the arguments and preceding method chain are shown above.
- Line 137, `frame['flavor']`: Store the result of calling `np.select(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 146, `conflicts`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.

Source lines 147–152:

```
147 if conflicts:
148     print(
149         f"{conflicts:,} UPCs have conflicting flavor classifications; "
150         "keeping the first flavor in CSV row order.",
151         flush=True,
152     )
```

Read this block:

- Line 147: test `conflicts`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 148: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 153–153:

```
153 return frame[["upc", "flavor"]].drop_duplicates(subset="upc", keep="first")
```

Read this block:

- Line 153: return the result of calling `.drop_duplicates(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.

week dates

Original function: `week_dates`. Source begins at line 156.

`pd.to_datetime` interprets strings with an explicit `%Y%m%d` format: four-digit year, two-digit month, two-digit day. The regex removes a trailing `.0` from integer dates that were stored as float-like text. `errors="raise"` rejects a malformed date instead of silently creating a missing date.

For example, `20100109` should mean January 9, 2010. Passing that integer directly to a general timestamp constructor can instead mean nanoseconds from an epoch. Explicit parsing prevents that mistake. This helper exists in the module, but the current main preparation path parses purchase-date strings with Polars directly; a function's presence does not mean the entry point uses it.

Definition and stated intent:

Source lines 156–157:

```
156 def week_dates(values):
157     """Parse YYYYMMDD safely (integer dates are NOT nanoseconds)."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 158–162:

```
158 return pd.to_datetime(
159     values.astype(str).str.replace(r"\.0$", "", regex=True),
160     format="%Y%m%d",
161     errors="raise",
162 )
```

Read this block:

- Line 158: return the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

prepare data

Original function: `prepare_data`. Source begins at line 165.

This is the data assembly function. `args` is an argparse Namespace: a small object whose attributes hold command-line settings. It returns a pandas table with one row per usable household-week. Its work crosses a deliberate boundary: large input filtering and quote aggregation use lazy Polars, while the smaller household choice table becomes pandas for subsequent array calculations.

Read it in phases. First it scans and filters years, resolves labels, chooses the price field, and projects columns. `getattr(args, name, default)` provides compatibility when a manually constructed Namespace lacks an optional attribute. `next((c for c in candidates if c in names), None)` chooses the first supported price field. It is a small loop written as a generator, not a special price API.

Next it collects selected households, checks quantities, sums counts, and chooses an exposure store. Sorting by descending quantity followed by ascending store ID then keeping the first household-week row implements the tie-break exactly. `groupby(...).sum().unstack(...)` turns long flavor counts into three columns; `reindex` fixes column order even if one flavor is absent from an intermediate table. Merge `validate` arguments catch unintended duplicate keys.

The quote section continues to use full `base`, not the household-limited `selected`. It deduplicates quotes and checks conflicts before constructing menus. The DMA section computes sums and counts and subtracts own-DMA totals. This algebra avoids looping over every possible outside DMA for every household. The same pooled IV is copied into all three flavor-named IV columns.

Finally it constructs histories **before** menu/IV likelihood exclusions, builds Boolean availability masks, filters unusable rows, reapplies the two-week rule, and adds outside counts. `complete &= iv_complete` combines masks in place; it is not a comparison. `transform("nunique")` produces a row-aligned count so the household restriction can be applied without a second merge. Moving `add_stocks` below these exclusions is a plausible-looking alternative that changes the information used to build history.

Definition and stated intent:

Source lines 165–170:

```
165 def prepare_data(args):
166     """Use only the merged purchase table, including its existing prices.
167 
168     The IV pools all flavors: the same-week average observed price outside the
169     consumer's DMA. Repeated purchases do not duplicate store/UPC price quotes.
170     """
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 171–172:

```
171 base = pl.scan_parquet(args.sample)
172 names = base.collect_schema().names()
```

Read this block:

- Line 171, `base`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.

- Line 172, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.

Source lines 173–174:

```
173 if "household_size" not in names:
174     raise ValueError("Estimation requires household_size to restrict purchases to single-person households.")
```

Read this block:

- Line 173: test `'household_size' not in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 174: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 175–175:

```
175 base = base.filter(pl.col("panel_year").is_between(args.first_year, args.last_year))
```

Read this block:

- Line 175, `base`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 176–181:

```
176 if "flavor" not in names:
177     products_extra = getattr(args, "products_extra", None) or DATA_DIR / "RMS/combined/products_extra.parquet"
178     flavor = read_flavors(args.flavors, products_extra)
179     base = base.with_columns(normalized_upc_expr()).join(pl.from_pandas(flavor).lazy(), on="upc", how="left")
180 else:
181     base = base.with_columns(normalized_upc_expr())
```

Read this block:

- Line 176: test `'flavor' not in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 177, `products_extra`: Set this name to the Boolean condition `getattr(args, 'products_extra', None) or DATA_DIR / 'RMS/combi...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 178, `flavor`: Set this name to the result of calling `read_flavors(...)`; the arguments and preceding method chain are shown above.
- Line 179, `base`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 181, `base`: Set this name to the result of calling `base.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `read_flavors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `getattr`: Retrieve an attribute by its string name, optionally supplying a default if it is absent.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.from_pandas`: Convert a pandas object into a Polars table; calling lazy afterward makes a query plan from it.

Source lines 182–182:

```
182 price_column = next((c for c in ["price", "price_rms"] if c in names), None)
```

Read this block:

- Line 182, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 183–184:

```
183 if price_column is None:
184     raise ValueError("Merged purchase table needs a price or price_rms column.")
```

Read this block:

- Line 183: test `price_column is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 184: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 185–187:

```
185 consumer_dma = getattr(args, "household_dma_column", "dma_cd")
186 columns = ["household_code", "household_size", "panel_year", "purchase_date", "trip_code_uc", "upc",
187            "store_code_uc", "quantity", "flavor", pl.col(price_column).cast(pl.Float64).alias("price")]
```

Read this block:

- Line 185, `consumer_dma`: Set this name to the result of calling `getattr(...)`; the arguments and preceding method chain are shown above.
- Line 186, `columns`: Set this name to a list containing 10 displayed entries.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.

Source lines 188–189:

```
188 if not args.no_price_control:
189     columns.append(pl.col(consumer_dma).alias("dma_cd"))
```

Read this block:

- Line 188: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 189: call `columns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 190–197:

```
190 base = base.select(columns).with_columns(
191     pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date())
192 base = base.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
193 base = base.with_columns((pl.col("purchase_date") + pl.duration(
194     days=(6 - pl.col("purchase_date").dt.weekday()) % 7)).alias("week"))
195 # Keep the full merged table for prices/IVs, even in a household-limited run.
196 # Restrict demand histories before weekly aggregation; retain all quotes for menus/IVs.
197 selected = base.filter(pl.col("household_size").cast(pl.Float64, strict=False) == 1)
```

Read this block:

- Line 190, `base`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 192, `base`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.
- Line 193, `base`: Set this name to the result of calling `base.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 197, `selected`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 198–201:

```
198 if args.max_households:
199     ids = (selected.select("household_code").unique().sort("household_code")
200            .limit(args.max_households).collect(engine="streaming")["household_code"])
201     selected = selected.filter(pl.col("household_code").is_in(ids.implode()))
```

Read this block:

- Line 198: test `args.max_households`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 199, `ids`: Set this name to the entry/slice selected by `selected.select('household_code').unique().sort('househol...`; brackets select from the object to their left.
- Line 201, `selected`: Set this name to the result of calling `selected.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

Source lines 202–203:

```
202 print(f"Loading merged purchase table: {args.sample}", flush=True)
203 lines = selected.collect(engine="streaming").to_pandas()
```

Read this block:

- Line 202: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 203, `lines`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.

Source lines 204–205:

```
204 if lines.empty:
205     raise ValueError("No purchase lines in selected sample.")
```

Read this block:

- Line 204: test `lines.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 205: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 206–207:

```
206 if lines["flavor"].isna().any() or not lines["flavor"].isin(FLAVORS).all():
207     raise ValueError("Merged purchase table has missing or invalid flavor labels.")
```

Read this block:

- Line 206: test `lines['flavor'].isna().any() or not lines['flavor'].isin(FLAVORS).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 207: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.

Source lines 208–208:

```
208 q = lines["quantity"].to_numpy(float)
```

Read this block:

- Line 208, `q`: Set this name to the result of calling `lines['quantity'].to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.

Source lines 209–210:

```
209 if not np.isfinite(q).all() or (q <= 0).any() or not np.allclose(q, np.round(q)):
210     raise ValueError("Quantity must be positive integer package counts.")
```

Read this block:

- Line 209: test `not np.isfinite(q).all() or (q <= 0).any() or (not np.allclose(q, np.round(q)))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 210: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.
- `.round`: Round numeric values for a check/display rule; rounding quantities is compared to the original before acceptance.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 211–217:

```
211 lines["week"] = pd.to_datetime(lines["week"])
212 keys = ["household_code", "week"]
213 weekly = (lines.groupby(keys + ["flavor"])["quantity"].sum().unstack("flavor", fill_value=0)
214           .reindex(columns=FLAVORS, fill_value=0).reset_index())
215 stores = lines.groupby(keys + ["store_code_uc"], as_index=False)["quantity"].sum()
216 stores = stores.sort_values(keys + ["quantity", "store_code_uc"], ascending=[True, True, False, True])
217 stores = stores.drop_duplicates(keys)[keys + ["store_code_uc"]]
```

Read this block:

- Line 211, `lines['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 212, `keys`: Set this name to the list `['household_code', 'week']`.
- Line 213, `weekly`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 215, `stores`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 216, `stores`: Set this name to the result of calling `stores.sort_values(...)`; the arguments and preceding method chain are shown above.
- Line 217, `stores`: Set this name to the entry/slice selected by `stores.drop_duplicates(keys)[keys + ['store_code_uc']]`; brackets select from the object to their left.

Syntax and calls used here:

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.unstack`: Move an index level into columns, such as turning long flavor counts into wide weekly outcomes.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.

Source lines 218–218:

```
218 weekly = weekly.merge(stores, on=keys, validate="one_to_one")
```

Read this block:

- Line 218, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 219–223:

```
219 if not args.no_price_control:
220     lines["dma_cd"] = check_dma(lines["dma_cd"], "consumer")
221     if (lines.groupby(keys)["dma_cd"].nunique() > 1).any():
222         raise ValueError("Multiple consumer DMAs within a household-week.")
223     weekly = weekly.merge(lines[keys + ["dma_cd"]].drop_duplicates(), on=keys, validate="one_to_one")
```

Read this block:

- Line 219: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 220, `lines['dma_cd']`: Store the result of calling `check_dma(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 221: test `(lines.groupby(keys)['dma_cd'].nunique() > 1).any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 222: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 223, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `check_dma`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.

Source lines 225–226:

```
225 quote_keys = ["store_code_uc", "week", "upc"]
226 price_columns = quote_keys + ["flavor", "price"]
```

Read this block:

- Line 225, `quote_keys`: Set this name to the list `['store_code_uc', 'week', 'upc']`.
- Line 226, `price_columns`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 227–228:

```
227 if not args.no_price_control:
228     price_columns.append("dma_cd")
```

Read this block:

- Line 227: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 228: call `price_columns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 229–235:

```
229 quotes = (base.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
230           .select(price_columns).unique())
231 # A store/week/UPC should have one price and one flavor. Consumer DMA may
232 # differ when consumers from different markets visit the same store.
233 local = quotes.select(quote_keys + ["flavor", "price"]).unique()
234 conflicting = (local.group_by(quote_keys).len().filter(pl.col("len") > 1)
235                .limit(1).collect(engine="streaming"))
```

Read this block:

- Line 229, `quotes`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 233, `local`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 234, `conflicting`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 236–237:

```
236 if len(conflicting):
237     raise ValueError("Conflicting merged prices or flavors for the same store/week/UPC.")
```

Read this block:

- Line 236: test `len(conflicting)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 237: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 238–244:

```
238 menus = (local.group_by(["store_code_uc", "week", "flavor"])
239          .agg(pl.col("price").mean()).collect(engine="streaming").to_pandas())
240 menus["week"] = pd.to_datetime(menus["week"])
241 menus = (menus.pivot(index=["store_code_uc", "week"], columns="flavor", values="price")
242          .reindex(columns=FLAVORS).rename(columns={f: "price_" + f for f in FLAVORS}).reset_index())
243 weekly = weekly.merge(menus, on=["store_code_uc", "week"], how="left", validate="many_to_one")
244 print("Using merged prices directly; no separate RMS data are loaded.", flush=True)
```

Read this block:

- Line 238, `menus`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 240, `menus['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 241, `menus`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 243, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 244: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 245–269:

```
245 if not args.no_price_control:
246     invalid_dma = quotes.select(
247         ((pl.col("dma_cd").cast(pl.Float64, strict=False).is_null())
248          | (pl.col("dma_cd").cast(pl.Float64, strict=False) <= 0)
249          | (pl.col("dma_cd").cast(pl.Float64, strict=False) != pl.col("dma_cd").cast(pl.Float64,
              ↪  strict=False).floor()))
250         .fill_null(True).any()).collect(engine="streaming").item()
251     if invalid_dma:
252         raise ValueError("Merged price records contain invalid dma_cd values for the other-DMA IV.")
253     # Pool across ALL flavors. Deduplicate quote repeats within each DMA.
254     by_dma = (quotes.with_columns(pl.col("dma_cd").cast(pl.Float64).cast(pl.Int64))
255               .group_by(["week", "dma_cd"]).agg(
256                   pl.col("price").sum().alias("dma_sum"), pl.len().alias("dma_count"))
257               .collect(engine="streaming").to_pandas())
258     by_dma["week"] = pd.to_datetime(by_dma["week"])
259     totals = (by_dma.groupby("week", as_index=False)[["dma_sum", "dma_count"]].sum()
260               .rename(columns={"dma_sum": "total_sum", "dma_count": "total_count"}))
261     weekly = weekly.merge(totals, on="week", how="left", validate="many_to_one")
262     weekly = weekly.merge(by_dma, on=["week", "dma_cd"], how="left", validate="many_to_one")
263     denominator = weekly["total_count"] - weekly["dma_count"].fillna(0)
264     weekly["iv_overall"] = ((weekly["total_sum"] - weekly["dma_sum"].fillna(0))
265                             / denominator.where(denominator > 0))
266     for f in FLAVORS:
267         weekly["iv_" + f] = weekly["iv_overall"]
268     weekly = weekly.drop(columns=["total_sum", "total_count", "dma_sum", "dma_count"])
269     print("IV: overall same-week mean price outside consumer dma_cd, pooled across flavors.", flush=True)
```

Read this block:

- Line 245: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 246, `invalid_dma`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 251: test `invalid_dma`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 252: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 254, `by_dma`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 258, `by_dma['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 259, `totals`: Set this name to the result of calling `.rename(...)`; the arguments and preceding method chain are shown above.
- Line 261, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 262, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 263, `denominator`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 264, `weekly['iv_overall']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 266: for each entry yielded by `FLAVORS`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 267, `weekly['iv_' + f]`: Store the entry/slice selected by `weekly['iv_overall']`; brackets select from the object to their left in the selected entry/column.
- Line 268, `weekly`: Set this name to the result of calling `weekly.drop(...)`; the arguments and preceding method chain are shown above.
- Line 269: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.floor`: Take the greatest integer no larger than each value; equality with it tests whether a positive code is integral.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.

Source lines 270–274:

```
270 weekly = add_stocks(weekly.sort_values(keys).reset_index(drop=True), args.habit_weight)
271 price_cols = ["price_" + f for f in FLAVORS]
272 available = np.isfinite(weekly[price_cols].to_numpy(float)) & (weekly[price_cols].to_numpy(float) > 0)
273 counts = weekly[FLAVORS].to_numpy(float)
274 complete = available.any(axis=1) & ((counts == 0) | available).all(axis=1)
```

Read this block:

- Line 270, `weekly`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 271, `price_cols`: Set this name to a list computed by the displayed for-expression.
- Line 272, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 273, `counts`: Set this name to the result of calling `weekly[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 274, `complete`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 275–276:

```
275 for j, f in enumerate(FLAVORS):
276     weekly["available_" + f] = available[:, j]
```

Read this block:

- Line 275: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, f)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 276, `weekly['available_' + f]`: Store the entry/slice selected by `available[:, j]`; brackets select from the object to their left in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 277–277:

```
277 print(f"Weeks excluded for purchased flavors without a covered price: {(~complete).sum():,}", flush=True)
```

Read this block:

- Line 277: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 278–282:

```
278 if not args.no_price_control:
279     iv_cols = ["iv_" + f for f in FLAVORS]
280     iv_complete = np.isfinite(weekly[iv_cols]).all(axis=1)
281     print(f"Additional weeks excluded for missing other-DMA IVs: {(complete & ~iv_complete).sum():,}", flush=True)
282     complete &= iv_complete
```

Read this block:

- Line 278: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 279, `iv_cols`: Set this name to a list computed by the displayed for-expression.
- Line 280, `iv_complete`: Set this name to the result of calling `.all(...)`; the arguments and preceding method chain are shown above.
- Line 281: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 282: update `complete` using `&=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 283–287:

```
283 weekly = weekly.loc[complete].copy()
284 repeated = weekly.groupby("household_code")["week"].transform("nunique") >= 2
285 print(f"Households excluded for fewer than two usable weeks: {weekly.loc[~repeated,
    ↪  'household_code'].nunique():,}", flush=True)
286 weekly = weekly.loc[repeated].reset_index(drop=True)
287 weekly["outside"] = np.maximum(args.capacity - weekly[FLAVORS].sum(axis=1), 0)
```

Read this block:

- Line 283, `weekly`: Set this name to the result of calling `weekly.loc[complete].copy(...)`; the arguments and preceding method chain are shown above.
- Line 284, `repeated`: Set this name to the Boolean condition `weekly.groupby('household_code')['week'].transform('nunique') ...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 285: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 286, `weekly`: Set this name to the result of calling `weekly.loc[repeated].reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 287, `weekly['outside']`: Store the result of calling `np.maximum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.transform`: Compute a pandas group result broadcast back to original rows, unlike a reduced one-row-per-group aggregation.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.

Source lines 288–289:

```
288 if weekly.empty:
289     raise ValueError("No households with at least two usable purchase weeks.")
```

Read this block:

- Line 288: test `weekly.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 289: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 290–290:

```
290 return weekly
```

Read this block:

- Line 290: return the object currently named `weekly`; plain assignment does not copy it. This ends the call.

add stocks

Original function: `add_stocks`. Source begins at line 293.

Input is a weekly pandas table; output is a sorted copy with stock, novelty, and Xi columns. The outer loop resets history for each household. `for _, group` ignores the household label but keeps its subtable. The inner `iterrows()` loop supplies a row index and a Series containing that row's values.

`stocks[index] = stock` stores pre-choice history into one output row. The seven-day condition tests genuine adjacent calendar weeks. Then counts are normalized into shares, stock is updated, and the current date becomes the previous date. A gap resets recent novelty by leaving its preallocated zeros, but does not reset cumulative stock. A loop is reasonable here because state at the next observation depends on the preceding update; vectorization requires careful shifted cumulative sums and calendar-gap checks.

The final `enumerate` loop writes one named column per flavor. `stocks[:, j]` extracts all rows of the jth flavor. With `habit_weight=.7`, current Xi uses .3 times novelty minus .7 times own stock. The separately preallocated `stocks` matrix means historical output rows do not change when `stock` is reassigned. Try a one-household, two-week example before reading the likelihood.

Definition and stated intent:

Source lines 293–304:

```
293 def add_stocks(weekly, habit_weight=0.7):
294     """All past observed occasions, delta=1; no rolling window or decay.
295 
296     S_jt = sum_{s<t} share_js
297     N_jt = sum_{k != j} share_k,t-1  (immediately preceding calendar week)
298     Xi_jt = (1-habit_weight)*N_jt - habit_weight*S_jt
299 
300     Positive gamma rewards recent other-flavor consumption and penalizes own
301     fatigue. Negative gamma represents habit. A purchase occasion is an observed
302     household purchase week. Current purchases enter history only AFTER Xi is
303     recorded. The first observed occasion starts with zero history.
304     """
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `habit_weight`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 305–309:

```
result = (
    weekly.sort_values(["household_code", "week"]).reset_index(drop=True).copy()
)
stocks = np.zeros((len(result), 3))
novelty = np.zeros_like(stocks)
```

Read this block:

- Line 305, `result`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.
- Line 308, `stocks`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 309, `novelty`: Set this name to the result of calling `np.zeros_like(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 310–321:

```
for _, group in result.groupby("household_code", sort=False):
    stock = np.zeros(3)
    previous_share = np.zeros(3)
    previous_week = None
    for index, row in group.iterrows():
        stocks[index] = stock
        if previous_week is not None and (row["week"] - previous_week).days == 7:
            novelty[index] = previous_share.sum() - previous_share
        counts = row[FLAVORS].to_numpy(float)
        previous_share = counts / counts.sum()
        stock = stock + previous_share
        previous_week = row["week"]
```

Read this block:

- Line 310: for each entry yielded by `result.groupby('household_code', sort=False)`, bind/unpack it into `(_, group)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 311, `stock`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 312, `previous_share`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 313, `previous_week`: Set this name to the literal value `None`.
- Line 314: for each entry yielded by `group.iterrows()`, bind/unpack it into `(index, row)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 315, `stocks[index]`: Store the object currently named `stock`; plain assignment does not copy it in the selected entry/column.
- Line 316: test `previous_week is not None and (row['week'] - previous_week).days == 7`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 317, `novelty[index]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 318, `counts`: Set this name to the result of calling `row[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 319, `previous_share`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 320, `stock`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 321, `previous_week`: Set this name to the entry/slice selected by `row['week']`; brackets select from the object to their left.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 322–327:

```
322 for j, name in enumerate(FLAVORS):
323     result["stock_" + name] = stocks[:, j]
324     result["novelty_" + name] = novelty[:, j]
325     result["xi_" + name] = (1 - habit_weight) * novelty[
326         :, j
327     ] - habit_weight * stocks[:, j]
```

Read this block:

- Line 322: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 323, `result['stock_' + name]`: Store the entry/slice selected by `stocks[:, j]`; brackets select from the object to their left in the selected entry/column.
- Line 324, `result['novelty_' + name]`: Store the entry/slice selected by `novelty[:, j]`; brackets select from the object to their left in the selected entry/column.
- Line 325, `result['xi_' + name]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 328–328:

```
328 return result
```

Read this block:

- Line 328: return the object currently named `result`; plain assignment does not copy it. This ends the call.

make arrays

Original function: `make_arrays`. Source begins at line 331.

This converts a labeled table into a dictionary of NumPy arrays. `pd.factorize` returns both dense integer household codes and their original labels: household IDs `[10,10,30]` become codes `[0,0,1]`. Dense codes let `bincount` sum efficiently. The expression `FLAVORS + (["outside"] if capacity else [])` builds either three or four outcome columns without changing the inside-price array's three columns.

The function validates counts/Xi, checks menu consistency, and inserts zero placeholders for missing prices. The separate availability mask ensures those zeros are not interpreted as offered free goods. `all(c in frame for c in cf_cols)` asks whether every residual column exists. If controls are absent, `zeros_like` supplies the right-shaped

residual array. The returned dictionary deliberately keeps flags such as `outside` and `control` beside arrays so the likelihood can use one interface for several models. Passing raw tables directly into every optimization step would repeatedly perform label lookup and conversion.

Definition and stated intent:

Source lines 331–332:

```
331 def make_arrays(frame, capacity):
332     """Use household codes to sum log likelihoods before integrating types."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `capacity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 333–339:

```
333 hh, labels = pd.factorize(frame["household_code"], sort=True)
334 counts = frame[FLAVORS + (["outside"] if capacity else [])].to_numpy(float)
335 prices = frame[["price_" + f for f in FLAVORS]].to_numpy(float)
336 xi = frame[["xi_" + f for f in FLAVORS]].to_numpy(float)
337 history = {key: frame[[prefix + f for f in FLAVORS]].to_numpy(float)
338             for key, prefix in [("novelty", "novelty_"), ("stock", "stock_")]
339             if all(prefix + f in frame for f in FLAVORS)}
```

Read this block:

- Line 333, `(hh, labels)`: Unpack the result of calling `pd.factorize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 334, `counts`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 335, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 336, `xi`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 337, `history`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.

Syntax and calls used here:

- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 340–341:

```
340 if any(not np.isfinite(value).all() for value in history.values()):
341     raise ValueError("Nonfinite separate history inputs.")
```

Read this block:

- Line 340: test `any((not np.isfinite(value).all() for value in history.values()))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 341: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 342–342:

```
342 available = np.isfinite(prices) & (prices > 0)
```

Read this block:

- Line 342, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 343–344:

```
343 if not all(np.isfinite(x).all() for x in [counts, xi]):
344     raise ValueError("Nonfinite estimation inputs.")
```

Read this block:

- Line 343: test `not all((np.isfinite(x).all() for x in [counts, xi]))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 344: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 345–346:

```
345 if ((counts[:, :3] > 0) & ~available).any() or not available.any(axis=1).all():
346     raise ValueError("Purchased flavors must have covered prices and a nonempty choice set.")
```

Read this block:

- Line 345: test `((counts[:, :3] > 0) & ~available).any() or not available.any(axis=1).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 346: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 348–351:

```
348 prices = np.where(available, prices, 0.)
349 cf_cols = ["cf_" + f for f in FLAVORS]
350 control = all(c in frame for c in cf_cols)
351 cf = frame[cf_cols].to_numpy(float) if control else np.zeros_like(prices)
```

Read this block:

- Line 348, `prices`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 349, `cf_cols`: Set this name to a list computed by the displayed for-expression.
- Line 350, `control`: Set this name to the result of calling `all(...)`; the arguments and preceding method chain are shown above.
- Line 351, `cf`: Set this name to the true-branch value if `control` holds, otherwise the else-branch value.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 352–353:

```
352 if not np.isfinite(cf).all():
353     raise ValueError("Nonfinite price-control residuals.")
```

Read this block:

- Line 352: test `not np.isfinite(cf).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 353: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 354–366:

```
354 return dict(
355     counts=counts,
356     prices=prices,
357     available=available,
358     xi=xi,
359     **history,
360     cf=cf,
361     control=control,
362     hh=hh,
363     labels=labels,
364     n_households=len(labels),
365     outside=bool(capacity),
366 )
```

Read this block:

- Line 354: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `len`: Count elements/rows rather than summing their numerical values.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.

make draws

Original function: `make_draws`. Source begins at line 369.

Inputs are an even draw count, an integer seed, and an optional model name. Output has shape `(number,3)` for heterogeneous_history and `(number,2)` otherwise. `number % 2` is the remainder after division by two: zero for even numbers, one for odd. `number // 2` is integer division. The generator draws half the rows, then `vstack([z, -z])` appends their negatives. With two draws, an initial row `[.4,-1.2]` gives a second row `[-.4,1.2]`.

The seed makes repeated calls reproducible, and a local Generator avoids changing NumPy's global random state. Reusing these fixed antithetic nodes reduces noise when comparing candidate parameter vectors. Drawing afresh inside each objective call would make the optimizer chase random changes as well as parameter effects. More draws still cost more computation.

Split-model update. heterogeneous_history uses D-by-3 draws with alpha, novelty and own-stock coordinates; other models keep D-by-2 draws. The draw dimension and seed must match the saved fit when reconstructing likelihoods.

Definition and stated intent:

Source lines 369–370:

```
369 def make_draws(number, seed, model=None):
370     """Fixed antithetic normal draws, reused throughout optimization."""
```

`def` creates this callable; the parentheses name its inputs (`number`, `seed`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 371–372:

```
371 if number < 2 or number % 2:
372     raise ValueError("Use an even number of draws, at least two.")
```

Read this block:

- Line 371: test `number < 2 or number % 2`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 372: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 373–373:

```
373 z = np.random.default_rng(seed).standard_normal((number // 2, 3 if model == "heterogeneous_history" else 2))
```

Read this block:

- Line 373, `z`: Set this name to the result of calling `.standard_normal(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.standard_normal`: Draw independent standard-normal values in the requested shape, before coefficient transformations.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- The // operator is floor division, used here for integer counts or calendar-year extraction.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 374–374:

```
374 return np.vstack([z, -z])
```

Read this block:

- Line 374: return the result of calling `np.vstack(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.vstack`: Stack input arrays vertically, adding rows while preserving their common column layout.

parameter names

Original function: `parameter_names`. Source begins at line 377.

This returns the ordered labels defining the parameter vector. `names += [...]` extends a list; it does not numerically add its string entries. Membership tests such as `model in ("mixed_naive", "heterogeneous")` let related specifications share branches. The default `control=False` omits a residual coefficient unless the caller explicitly enables it.

For naive conditional demand without controls, the returned labels are two betas and mean log-minus-alpha. Heterogeneous demand adds mean gamma and both SDs; the optional outside constant is prepended and price control appended. Keeping this logic in one helper is safer than independently remembering vector positions in optimization, utility, and output. The order is part of the API: zipping a reordered list with old coefficients silently changes their meaning.

Split layouts. separate_history adds mean_novelty and mean_own_stock plus random price sensitivity; heterogeneous_history also adds sd_novelty and sd_own_stock. This helper is also used when checking the saved likelihood during weekly-export regeneration.

Definition and stated intent:

Source lines 377–377:

```
377 def parameter_names(model, outside, control=False):
```

`def` creates this callable; the parentheses name its inputs (`model`, `outside`, `control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 378–379:

```
378 names = (["inside_constant"] if outside else []) + ["beta_berry", "beta_plain"]
379 names += ["mu_log_minus_alpha"]
```

Read this block:

- Line 378, `names`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 379: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 380–381:

```
380 if model in ("homogeneous", "heterogeneous"):
381     names += ["mean_gamma"]
```

Read this block:

- Line 380: test `model in ('homogeneous', 'heterogeneous')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 381: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 382–383:

```
382 if model in ("mixed_naive", "heterogeneous"):
383     names += ["sd_log_minus_alpha"]
```

Read this block:

- Line 382: test `model in ('mixed_naive', 'heterogeneous')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 383: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 384–385:

```
384 if model == "heterogeneous":
385     names += ["sd_gamma"]
```

Read this block:

- Line 384: test `model == 'heterogeneous'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 385: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 386–388:

```
386 if model == "mixture":
387     names += ["mean_gamma", "sd_log_minus_alpha", "sd_gamma",
388               "gamma_mean_gap", "sd_gamma_2", "mixture_weight"]
```

Read this block:

- Line 386: test `model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 387: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 389–390:

```
389 if model in ("separate_history", "heterogeneous_history"):
390     names += ["mean_novelty", "mean_own_stock", "sd_log_minus_alpha"]
```

Read this block:

- Line 389: test `model in ('separate_history', 'heterogeneous_history')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 390: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 391–392:

```
391 if model == "heterogeneous_history":
392     names += ["sd_novelty", "sd_own_stock"]
```

Read this block:

- Line 391: test `model == 'heterogeneous_history'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 392: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 393–394:

```
393 if control:
394     names += ["price_control"]
```

Read this block:

- Line 393: test `control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 394: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 395–395:

```
395 return names
```

Read this block:

- Line 395: return the object currently named `names`; plain assignment does not copy it. This ends the call.

mixture components

Original function: `mixture_components`. Source begins at line 398.

Unpacks a parameter vector into two ordered normal-gamma components and a common price distribution. Adding a nonnegative gap orders component means and prevents label switching; it does not force either component to have positive gamma.

Definition and stated intent:

Source lines 398–399:

```
398 def mixture_components(theta, data):
399     """Ordered component means prevent label switching; alpha is shared."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 400–405:

```
400 names = parameter_names("mixture", data["outside"], data["control"])
401 p = dict(zip(names, theta))
402 normal_names = parameter_names("heterogeneous", data["outside"], data["control"])
403 first = np.array([p[n] for n in normal_names])
404 second = first.copy()
405 second[normal_names.index("mean_gamma")] += p["gamma_mean_gap"]
```

Read this block:

- Line 400, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 401, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 402, `normal_names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 403, `first`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 404, `second`: Set this name to the result of calling `first.copy(...)`; the arguments and preceding method chain are shown above.
- Line 405: update `second[normal_names.index('mean_gamma')]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.

Source lines 406–406:

```
406 second[normal_names.index("sd_gamma")] = p["sd_gamma_2"]
```

Read this block:

- Line 406, `second[normal_names.index('sd_gamma')]`: Store the entry/slice selected by `p['sd_gamma_2']`; brackets select from the object to their left in the selected entry/column.

Source lines 407–407:

```
407 return names, normal_names, first, second, p["mixture_weight"]
```

Read this block:

- Line 407: return a tuple of 5 results. The caller can unpack them; return ends the call, unlike print.

mixture loglike score

Original function: `mixture_loglike_score`. Source begins at line 410.

Combines complete household likelihoods across persistent mixture components, using stable log-sum-exp. Mixing each week separately would change the model by allowing household membership to change over time. The score combines component derivatives with posterior membership weights.

Definition and stated intent:

Source lines 410–411:

```
410 def mixture_loglike_score(theta, data, draws, score=False):
411     """Mix whole household histories, never individual weeks."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `score`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 412–412:

```
412 names, normal_names, first, second, weight = mixture_components(theta, data)
```

Read this block:

- Line 412, `(names, normal_names, first, second, weight)`: Unpack the result of calling `mixture_components(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `mixture_components`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 413–418:

```
413 if score:
414     ll1, g1 = demand_objective_gradient(first, data, draws, "heterogeneous", household=True)
415     ll2, g2 = demand_objective_gradient(second, data, draws, "heterogeneous", household=True)
416 else:
417     ll1 = household_loglike(first, data, draws, "heterogeneous")
418     ll2 = household_loglike(second, data, draws, "heterogeneous")
```

Read this block:

- Line 413: test `score`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 414, `(ll1, g1)`: Unpack the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 415, `(ll2, g2)`: Unpack the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 417, `ll1`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 418, `ll2`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 419–420:

```
terms = np.array([np.log(weight) + ll1, np.log1p(-weight) + ll2])
ll = logsumexp(terms, axis=0)
```

Read this block:

- Line 419, `terms`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 420, `ll`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.

Source lines 421–422:

```
if not score:
    return ll
```

Read this block:

- Line 421: test `not score`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 422: return the object currently named `ll`; plain assignment does not copy it. This ends the call.

Source lines 423–424:

```
responsibility = np.exp(terms - ll)
gradient = np.zeros((data["n_households"], len(names)))
```

Read this block:

- Line 423, `responsibility`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 424, `gradient`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 425–430:

```
for k, name in enumerate(normal_names):
    if name == "sd_gamma":
        gradient[:, names.index(name)] = responsibility[0] * g1[:, k]
        gradient[:, names.index("sd_gamma_2")] = responsibility[1] * g2[:, k]
    else:
        gradient[:, names.index(name)] = responsibility[0] * g1[:, k] + responsibility[1] * g2[:, k]
```

Read this block:

- Line 425: for each entry yielded by `enumerate(normal_names)`, bind/unpack it into `(k, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 426: test `name == 'sd_gamma'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 427, `gradient[:, names.index(name)]`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 428, `gradient[:, names.index('sd_gamma_2')]`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 430, `gradient[:, names.index(name)]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 431–432:

```
431 gradient[:, names.index("gamma_mean_gap")] = responsibility[1] * g2[:, normal_names.index("mean_gamma")]
432 gradient[:, names.index("mixture_weight")] = responsibility[0] / weight - responsibility[1] / (1 - weight)
```

Read this block:

- Line 431, `gradient[:, names.index('gamma_mean_gap')]`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 432, `gradient[:, names.index('mixture_weight')]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Source lines 433–433:

```
433 return -ll.mean(), -gradient.mean(axis=0)
```

Read this block:

- Line 433: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

history utility

Original function: `history_utility`. Source begins at line 436.

Chooses the history formula from the model name. Split models use b_N*N*+*b*_*S*S directly, with distinct draw coordinates for the two random coefficients. Combined-gamma models use gamma*Xi. The habit-weight parameter belongs to Xi and does not multiply the independently estimated split coefficients.

Definition and stated intent:

Source lines 436–436:

```
436 def history_utility(p, data, z, model):
```

`def` creates this callable; the parentheses name its inputs (`p`, `data`, `z`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 437–444:

```
437 if model in ("separate_history", "heterogeneous_history"):
438     novelty = p["mean_novelty"] + p.get("sd_novelty", 0.) * z[1]
439     own_stock = p["mean_own_stock"]
440     if model == "heterogeneous_history":
441         if len(z) < 3:
442             raise ValueError("heterogeneous_history requires three-dimensional draws.")
443         own_stock += p["sd_own_stock"] * z[2]
```

```
444     return novelty * data["novelty"] + own_stock * data["stock"]
```

Read this block:

- Line 437: test `model in ('separate_history', 'heterogeneous_history')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 438, `novelty`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 439, `own_stock`: Set this name to the entry/slice selected by `p['mean_own_stock']`; brackets select from the object to their left.
- Line 440: test `model == 'heterogeneous_history'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 441: test `len(z) < 3`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 442: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 443: update `own_stock` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 444: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `len`: Count elements/rows rather than summing their numerical values.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 445–445:

```
445 gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
```

Read this block:

- Line 445, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 446–446:

```
446 return gamma * data["xi"]
```

Read this block:

- Line 446: return multiplication of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

household loglike

Original function: `household_loglike`. Source begins at line 449.

Input `theta` is a one-dimensional parameter array, `data` is the dictionary from `make_arrays`, and `draws` is a draw-by-two matrix for combined models or draw-by-three for split heterogeneity. The return is one log likelihood per household, not a scalar. `dict(zip(...))` attaches coefficient names so the rest of the function need not remember their numeric positions.

For each latent-type draw, negative exponential constructs alpha and an affine normal transformation constructs gamma. Broadcasting makes beta and these scalar coefficients operate across every row/flavor. Unavailable utility is set to minus infinity; an optional zero outside column is appended. `logprob = utility - logsumexp(..., axis=1, keepdims=True)` normalizes each row. The retained column dimension makes subtraction broadcast across that row.

The count-weighted row sum uses `where(counts > 0, logprob, 0)` to avoid zero times minus infinity. `bincount` adds all weeks of the same household. Only after this aggregation does the final `logsumexp` integrate draws. This order implements persistent preferences. Averaging weekly probabilities first would be simpler computationally but would estimate a different preference process.

Current split utility. history_utility supplies b_NN+b_SS, with separate means and normal draw coordinates. Gamma*Xi is retained only for combined models. In a mixture model, the separate component helper mixes complete household likelihoods. Neither split coefficient receives the old fixed lambda weight.

Definition and stated intent:

Source lines 449–455:

```
449 def household_loglike(theta, data, draws, model):
450     """L_i = mean_r exp(sum_t sum_j count_itj * log P_itj,r).
451 
452     Gamma is normal and common across flavors; alpha = -exp(mu + sd*z). Beta never varies by draw.
453     Process one draw at a time to avoid a weeks x draws x products array.
454     The multinomial combinatorial constant is omitted (independent of theta).
455     """
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 456–457:

```
456 if model == "mixture":
457     return mixture_loglike_score(theta, data, draws)
```

Read this block:

- Line 456: test `model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 457: return the result of calling `mixture_loglike_score(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `mixture_loglike_score`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 458–464:

```
458 p = dict(zip(parameter_names(model, data["outside"], data["control"]), theta))
459 beta = np.array([0.0, p["beta_berry"], p["beta_plain"]])
460 beta += p.get("inside_constant", 0.0)
461 loglikes = []
462 active_draws = (
463     draws if model in ("mixed_naive", "heterogeneous", "separate_history", "heterogeneous_history") else
    ↪  np.zeros((1, 2))
464 )
```

Read this block:

- Line 458, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 459, `beta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 460: update `beta` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 461, `loglikes`: Set this name to the list `[]`.
- Line 462, `active_draws`: Set this name to the true-branch value if `model in ('mixed_naive', 'heterogeneous', 'sepa...` holds, otherwise the else-branch value.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 465–483:

```
465 for z in active_draws:
466     alpha = -np.exp(
467         p["mu_log_minus_alpha"] + p.get("sd_log_minus_alpha", 0.0) * z[0]
468     )
469     gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
470     utility = (
471         beta
472         + alpha * data["prices"]
473         + history_utility(p, data, z, model)
474         + p.get("price_control", 0.0) * data["cf"]
475     )
476     utility = np.where(data["available"], utility, -np.inf)
477     if data["outside"]:
478         utility = np.column_stack([utility, np.zeros(len(utility))])
479     logprob = utility - logsumexp(utility, axis=1, keepdims=True)
480     row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
481     loglikes.append(
482         np.bincount(data["hh"], weights=row_ll, minlength=data["n_households"])
483     )
```

Read this block:

- Line 465: for each entry yielded by `active_draws`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 466, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(p['mu_log_minus_alpha'] + p.get('sd_log_minus_alp...`.
- Line 469, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 470, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 476, `utility`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 477: test `data['outside']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 478, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 479, `logprob`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 480, `row_ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 481: call `loglikes.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere;

np.where instead takes explicit true/false branches.

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `history_utility`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 484–484:

```
484 return logsumexp(loglikes, axis=0) - np.log(len(active_draws))
```

Read this block:

- Line 484: return subtraction of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

demand objective gradient

Original function: `demand_objective_gradient`. Source begins at line 487.

This returns a tuple `(mean_negative_log_likelihood, gradient)`, which SciPy requests when `jac=True`. The utility and likelihood steps mirror `household_loglike`; the additional work computes derivatives in the same draw loop. `total[:, None]` expands a weekly count vector to a column so it can multiply the matrix of choice probabilities. `residual` is observed count minus predicted count for the three inside alternatives.

The `columns` dictionary holds one weekly score vector per coefficient name. For example, the berry intercept takes column 1 of residual, while the alpha location score uses the price-weighted residual multiplied by alpha itself because differentiating `-exp(mu + sd*z)` contributes alpha. Each score is aggregated to households, and `column_stack` places parameters on a final axis.

`np.einsum("rh,rhp->p", weights, scores)` means multiply posterior weight at draw r/household h by its parameter-p score, then sum r and h, retaining p. A nested loop over r, h, p would express the same sum more verbosely and slowly. The minus sign and household-count division match minimization of average NLL. This is an analytic derivative of the finite-draw objective, not automatic differentiation or a finite-difference estimate.

Definition and stated intent:

Source lines 487–488:

```
487 def demand_objective_gradient(theta, data, draws, model, household=False):
488     """Exact simulated likelihood score, weighted by household type posteriors."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `model`, `household`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 489–490:

```
489 if model == "mixture":
490     return mixture_loglike_score(theta, data, draws, score=True)
```

Read this block:

- Line 489: test `model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 490: return the result of calling `mixture_loglike_score(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `mixture_loglike_score`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 491–495:

```
491 names = parameter_names(model, data["outside"], data["control"])
492 p = dict(zip(names, theta))
493 active = draws if model in ("mixed_naive", "heterogeneous", "separate_history", "heterogeneous_history") else
    ↪  np.zeros((1, 2))
494 likelihoods, scores = [], []
495 total = data["counts"].sum(axis=1)
```

Read this block:

- Line 491, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 492, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 493, `active`: Set this name to the true-branch value if `model in ('mixed_naive', 'heterogeneous', 'sepa...` holds, otherwise the else-branch value.
- Line 494, `(likelihoods, scores)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 495, `total`: Set this name to the result of calling `data['counts'].sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 496–522:

```
496 for z in active:
497     alpha = -np.exp(p["mu_log_minus_alpha"] + p.get("sd_log_minus_alpha", 0.) * z[0])
498     gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
499     utility = (np.array([0., p["beta_berry"], p["beta_plain"]]) + p.get("inside_constant", 0.)
500                + alpha * data["prices"] + history_utility(p, data, z, model) + p.get("price_control", 0.) *
                    ↪  data["cf"])
501     utility = np.where(data["available"], utility, -np.inf)
502     if data["outside"]:
503         utility = np.column_stack([utility, np.zeros(len(utility))])
504     logprob = utility - logsumexp(utility, axis=1, keepdims=True)
505     row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
506     likelihoods.append(np.bincount(data["hh"], weights=row_ll, minlength=data["n_households"]))
507     residual = data["counts"][:, :3] - total[:, None] * np.exp(logprob[:, :3])
508     price_score = (residual * data["prices"]).sum(axis=1) * alpha
509     columns = {"inside_constant": residual.sum(axis=1), "beta_berry": residual[:, 1],
510                "beta_plain": residual[:, 2], "mu_log_minus_alpha": price_score,
511                "sd_log_minus_alpha": price_score * z[0],
```

```
512                 "price_control": (residual * data["cf"]).sum(axis=1)}
513     columns["mean_gamma"] = (residual * data["xi"]).sum(axis=1)
514     columns["sd_gamma"] = columns["mean_gamma"] * z[1]
515     if model in ("separate_history", "heterogeneous_history"):
516         columns["mean_novelty"] = (residual * data["novelty"]).sum(axis=1)
517         columns["mean_own_stock"] = (residual * data["stock"]).sum(axis=1)
518         columns["sd_novelty"] = columns["mean_novelty"] * z[1]
519         if model == "heterogeneous_history":
520             columns["sd_own_stock"] = columns["mean_own_stock"] * z[2]
521     scores.append(np.column_stack([np.bincount(data["hh"], weights=columns[n],
522                                                minlength=data["n_households"]) for n in names]))
```

Read this block:

- Line 496: for each entry yielded by `active`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 497, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(p['mu_log_minus_alpha'] + p.get('sd_log_minus_alp...`.
- Line 498, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 499, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 501, `utility`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 502: test `data['outside']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 503, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 504, `logprob`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 505, `row_ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 506: call `likelihoods.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 507, `residual`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 508, `price_score`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 509, `columns`: Set this name to a dictionary with named entries `inside_constant`, `beta_berry`, `beta_plain`, `mu_log_minus_alpha`, `sd_log_minus_alpha`, `price_control`.
- Line 513, `columns['mean_gamma']`: Store the result of calling `(residual * data['xi']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 514, `columns['sd_gamma']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 515: test `model in ('separate_history', 'heterogeneous_history')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 516, `columns['mean_novelty']`: Store the result of calling `(residual * data['novelty']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 517, `columns['mean_own_stock']`: Store the result of calling `(residual * data['stock']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 518, `columns['sd_novelty']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 519: test `model == 'heterogeneous_history'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 520, `columns['sd_own_stock']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 521: call `scores.append` with the displayed arguments. No assignment stores its return here; its

documented side effect or assertion is what matters.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `history_utility`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `len`: Count elements/rows rather than summing their numerical values.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 523–526:

```
523 likelihoods = np.asarray(likelihoods)
524 normalizer = logsumexp(likelihoods, axis=0)
525 weights = np.exp(likelihoods - normalizer)
526 household_scores = np.einsum("rh,rhp->hp", weights, np.asarray(scores))
```

Read this block:

- Line 523, `likelihoods`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 524, `normalizer`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 525, `weights`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 526, `household_scores`: Set this name to the result of calling `np.einsum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.einsum`: Perform an explicitly labeled sum of products; rh,rhp->p sums draws/households and retains the parameter axis.

Source lines 527–528:

```
527 if household:
528     return normalizer - np.log(len(active)), household_scores
```

Read this block:

- Line 527: test `household`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 528: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

Source lines 529–529:

```
529 gradient = -household_scores.mean(axis=0)
```

Read this block:

- Line 529, `gradient`: Set this name to the negated or sign-adjusted expression `-household_scores.mean(axis=0)`.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 530–530:

```
530 return -(normalizer - np.log(len(active))).mean(), gradient
```

Read this block:

- Line 530: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

optimize demand

Original function: `optimize_demand`. Source begins at line 533.

This sets up and runs multiple bounded local searches and returns the selected fit, every starting solution, and the bounds. Initial SDs are .3 because zero can be stationary under symmetric integration. Bounds are pairs `(lower, upper)` in the same order as parameter names. The optional initial vector is copied before perturbation, preventing caller-owned coefficients from being changed.

Each start uses `minimize(..., method="L-BFGS-B", jac=True)`. The callback reports progress every ten iterations, while completed starts are saved to JSON. `np.clip` keeps perturbed starts feasible. `min(results, key=lambda r: r.fun)` uses each result's loss as its comparison key; it selects the smallest loss, not necessarily a converged fit. An alternative is to select only converged results, but that is not the rule this code implements. Multistart search probes local solutions and does not establish a global optimum.

Definition and stated intent:

Source lines 533–533:

```
533 def optimize_demand(data, args, initial=None, number_starts=None, quiet=False):
```

`def` creates this callable; the parentheses name its inputs (`data`, `args`, `initial`, `number_starts`, `quiet`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 534–537:

```
534 draws = make_draws(args.draws, args.seed, args.model)
535 names = parameter_names(args.model, data["outside"], data["control"])
536 start = np.zeros(len(names))
537 bounds = []
```

Read this block:

- Line 534, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 535, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 536, `start`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 537, `bounds`: Set this name to the list `[]`.

Syntax and calls used here:

- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 538–551:

```
538 for k, name in enumerate(names):
539     if name.startswith("sd_"):
540         start[k] = 0.3  # Zero SD is a stationary point: start away from zero.
541         bounds.append((0.0, 3.0))
542     elif name == "gamma_mean_gap":
543         start[k] = 0.4
544         bounds.append((0.0, 40.0))
545     elif name == "mixture_weight":
546         start[k] = 0.5
547         bounds.append((0.001, 0.999))
548     elif name == "mu_log_minus_alpha":
549         bounds.append((-6.0, 3.0))
550     else:
551         bounds.append((-20.0, 20.0))
```

Read this block:

- Line 538: for each entry yielded by `enumerate(names)`, bind/unpack it into `(k, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 539: test `name.startswith('sd_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 540, `start[k]`: Store the literal value `0.3` in the selected entry/column.
- Line 541: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 542: test `name == 'gamma_mean_gap'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 543, `start[k]`: Store the literal value `0.4` in the selected entry/column.
- Line 544: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 545: test `name == 'mixture_weight'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 546, `start[k]`: Store the literal value `0.5` in the selected entry/column.
- Line 547: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 548: test `name == 'mu_log_minus_alpha'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 549: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 551: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 552–553:

```
552 if args.model == "mixture":
553     start[names.index("mean_gamma")] = -0.2
```

Read this block:

- Line 552: test `args.model == 'mixture'`. Execute the indented branch only when it is true; an else/elif

branch handles its alternative.
- Line 553, `start[names.index('mean_gamma')]`: Store the negated or sign-adjusted expression `-0.2` in the selected entry/column.

Source lines 554–555:

```
554 if initial is not None:
555     start = np.asarray(initial).copy()
```

Read this block:

- Line 554: test `initial is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 555, `start`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 556–557:

```
556 rng = np.random.default_rng(args.seed + 1)
557 results = []
```

Read this block:

- Line 556, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 557, `results`: Set this name to the list `[]`.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.

Source lines 559–560:

```
559 def objective(theta):
560     return demand_objective_gradient(theta, data, draws, args.model)
```

Read this block:

- Line 559: define nested callable `objective`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 562–596:

```
562 for s in range(args.starts if number_starts is None else number_starts):
563     initial = start.copy()
564     if s:
565         initial += rng.normal(0, 0.5, len(start))
566         initial = np.clip(initial, np.array(bounds)[:, 0], np.array(bounds)[:, 1])
567     began = time.monotonic()
568     iterations = [0]
569 
570     def progress(theta):
571         iterations[0] += 1
572         if not quiet and iterations[0] % 10 == 0:
573             print(f"Start {s + 1}: iteration {iterations[0]}, elapsed {time.monotonic() - began:.0f}s", flush=True)
574 
575     fit = minimize(
576         objective,
```

```
        initial,
        method="L-BFGS-B",
        jac=True,
        callback=progress,
        bounds=bounds,
        options={"maxiter": args.maxiter, "ftol": 1e-10, "gtol": 1e-6},
    )
    results.append(fit)
    if not quiet:
        args.output.mkdir(parents=True, exist_ok=True)
        (args.output / "starts_checkpoint.json").write_text(json.dumps([
            dict(theta=r.x.tolist(), nll=float(r.fun * data["n_households"]),
                 converged=bool(r.success), iterations=int(r.nit), evaluations=int(r.nfev))
            for r in results], indent=2))
    if not quiet:
        print(
            f"Start {s + 1}: NLL={fit.fun * data['n_households']:.4f}; "
            f"converged={fit.success}",
            flush=True,
        )
```

Read this block:

- Line 562: for each entry yielded by `range(args.starts if number_starts is None else number_starts)`, bind/unpack it into `s` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 563, `initial`: Set this name to the result of calling `start.copy(...)`; the arguments and preceding method chain are shown above.
- Line 564: test `s`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 565: update `initial` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 566, `initial`: Set this name to the result of calling `np.clip(...)`; the arguments and preceding method chain are shown above.
- Line 567, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 568, `iterations`: Set this name to the list `[0]`.
- Line 570: define nested callable `progress`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 575, `fit`: Set this name to the result of calling `minimize(...)`; the arguments and preceding method chain are shown above.
- Line 584: call `results.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 585: test `not quiet`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 586: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 587: call `(args.output / 'starts_checkpoint.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 591: test `not quiet`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 592: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `minimize`: Run iterative numerical minimization; jac=True expects the objective to return value plus gradient.
- `.normal`: Draw normal random values with the supplied location/scale, used for starts or synthetic data.

- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 598–598:

```
598 best = min(results, key=lambda r: r.fun)
```

Read this block:

- Line 598, `best`: Set this name to the result of calling `min(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 599–599:

```
599 return best, results, bounds
```

Read this block:

- Line 599: return a tuple of 3 results. The caller can unpack them; return ends the call, unlike print.

optimize demand.objective

Original function: `optimize_demand.objective`. Source begins at line 559.

This small nested adapter has one explicit input, `theta`. It forwards theta and the enclosing call's data/draws/model to the actual objective/gradient function. SciPy expects a callable accepting the changing candidate vector; the closure supplies the fixed objects. A global variable for data could work but would make concurrent or repeated fits harder to reason about. Returning the whole pair is intentional because the surrounding minimizer sets `jac=True`.

Definition and stated intent:

Source lines 559–559:

```
559 def objective(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 560–560:

```
560 return demand_objective_gradient(theta, data, draws, args.model)
```

Read this block:

- Line 560: return the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

optimize demand.progress

Original function: `optimize_demand.progress`. Source begins at line 570.

SciPy calls this nested callback after iterations, passing the current vector. The body increments `iterations[0]`, checks a remainder modulo ten, and prints elapsed time unless quiet mode is active. The callback does not return an objective value and does not choose the next parameter vector.

The enclosing variable is a one-element list so the nested function can mutate its element without rebinding the outer name. An ordinary integer counter with assignment would require `nonlocal iterations`. That alternative is often clearer; the existing list technique works because objects can be mutated through a captured reference. `time.monotonic` measures elapsed duration without wall-clock adjustments changing the result.

Definition and stated intent:

Source lines 570–570:

```
570 def progress(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 571–571:

```
571 iterations[0] += 1
```

Read this block:

- Line 571: update `iterations[0]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 572–573:

```
572 if not quiet and iterations[0] % 10 == 0:
573     print(f"Start {s + 1}: iteration {iterations[0]}, elapsed {time.monotonic() - began:.0f}s", flush=True)
```

Read this block:

- Line 572: test `not quiet and iterations[0] % 10 == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 573: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.

- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

plot gamma distribution

Original function: `plot_gamma_distribution`. Source begins at line 602.

Input is an estimate table, output directory, and convergence flag. This is a side-effect function: it saves a figure and implicitly returns `None`. `set_index("parameter")["estimate"]` creates a name-indexed Series for coefficient lookup. The early bare `return` skips naive models with no gamma.

For positive SD, `linspace` makes a smooth horizontal grid, `norm.pdf` computes density, and `fill_between(..., where=x < 0)` shades the negative region. For zero SD a continuous density would divide by zero or misrepresent a point mass, so the `else` branch draws a vertical line. `norm.cdf(0, ...)` computes the implied population habit share. `Agg` supports file rendering without a GUI; `plt.close(fig)` releases the figure. This is a distribution of latent types, not an uncertainty interval for the mean.

Definition and stated intent:

Source lines 602–603:

```
602 def plot_gamma_distribution(table, output, converged=True):
603     """Plot the fitted population distribution, not uncertainty in its mean."""
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 604–604:

```
604 estimates = table.set_index("parameter")["estimate"]
```

Read this block:

- Line 604, `estimates`: Set this name to the entry/slice selected by `table.set_index('parameter')['estimate']`; brackets select from the object to their left.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

Source lines 605–606:

```
605 if "mean_gamma" not in estimates:
606     return  # Naive models have no gamma parameter.
```

Read this block:

- Line 605: test `'mean_gamma' not in estimates`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 606: bare return ends this call immediately with None. Later statements in the function are skipped.

Source lines 607–613:

```
607 import matplotlib
608 matplotlib.use("Agg")
609 import matplotlib.pyplot as plt
610 from scipy.stats import norm
611
612 mean = float(estimates["mean_gamma"])
613 sd = float(estimates.get("sd_gamma", 0.))
```

Read this block:

- Line 607: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 608: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 609: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 610: from `scipy.stats`, import `norm` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 612, `mean`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 613, `sd`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 614–614:

```
614 fig, ax = plt.subplots(figsize=(7, 5))
```

Read this block:

- Line 614, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 615–650:

```
615 if "mixture_weight" in estimates:
616     weight = float(estimates["mixture_weight"])
617     mean2 = mean + float(estimates["gamma_mean_gap"])
618     sd2 = float(estimates["sd_gamma_2"])
619     x = np.linspace(min(mean - 4 * sd, mean2 - 4 * sd2) - .01,
620                     max(mean + 4 * sd, mean2 + 4 * sd2) + .01, 1001)
621     density = np.zeros_like(x)
622     negative = 0.
623     for mu, sigma, share in [(mean, sd, weight), (mean2, sd2, 1 - weight)]:
624         if sigma > 0:
625             component = share * norm.pdf(x, loc=mu, scale=sigma)
626             density += component
627             ax.plot(x, component, linestyle=":", label=f"Component (weight {share:.1%})")
628             negative += share * norm.cdf(0, loc=mu, scale=sigma)
629         else:
630             ax.axvline(mu, linestyle=":", label=f"Point mass (weight {share:.1%})")
631             negative += share * (mu < 0)
632     ax.plot(x, density, color="steelblue", label="Mixture continuous density")
633     ax.set_ylabel("Population density")
634     ax.legend()
635     description = f"Component means = {mean:.4g}, {mean2:.4g}; P(gamma < 0) = {negative:.1%}"
636 elif sd > 0:
637     x = np.linspace(mean - 4 * sd, mean + 4 * sd, 801)
638     density = norm.pdf(x, loc=mean, scale=sd)
639     ax.plot(x, density, color="steelblue")
640     ax.fill_between(x, density, where=x < 0, color="steelblue", alpha=.25)
641     ax.set_ylabel("Population density")
```

```
    negative = norm.cdf(0, loc=mean, scale=sd)
    description = f"Mean = {mean:.4g}; SD = {sd:.4g}; P(gamma < 0) = {negative:.1%}"
else:
    ax.axvline(mean, color="steelblue", linewidth=2, label="Point mass (SD = 0)")
    ax.set_xlim(mean - 1, mean + 1)
    ax.set_yticks([])
    ax.set_ylabel("Point mass")
    ax.legend()
    description = f"All households: gamma = {mean:.4g} (SD = 0)"
```

Read this block:

- Line 615: test `'mixture_weight' in estimates`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 616, `weight`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 617, `mean2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 618, `sd2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 619, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 621, `density`: Set this name to the result of calling `np.zeros_like(...)`; the arguments and preceding method chain are shown above.
- Line 622, `negative`: Set this name to the literal value `0.0`.
- Line 623: for each entry yielded by `[(mean, sd, weight), (mean2, sd2, 1 - weight)]`, bind/unpack it into `(mu, sigma, share)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 624: test `sigma > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 625, `component`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 626: update `density` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 627: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 628: update `negative` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 630: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 631: update `negative` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 632: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 633: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 634: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 635, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 636: test `sd > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 637, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 638, `density`: Set this name to the result of calling `norm.pdf(...)`; the arguments and preceding method chain are shown above.
- Line 639: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 640: call `ax.fill_between` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 641: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 642, `negative`: Set this name to the result of calling `norm.cdf(...)`; the arguments and preceding method chain are shown above.
- Line 643, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 645: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 646: call `ax.set_xlim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 647: call `ax.set_yticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 648: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 649: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 650, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.set_ylabel`: Set the vertical-axis label, whose unit should match the plotted values.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.pdf`: Evaluate a probability density (here a normal distribution), not a PDF document file.
- `.fill_between`: Shade the area between curves/zero, optionally only where a Boolean condition is true.
- `.cdf`: Evaluate cumulative probability below a threshold, used for the population share with negative gamma.
- `.axvline`: Add a vertical reference line, such as zero gamma or a point-mass type.
- `.set_xlim`: Set the visible x-axis limits; this changes presentation rather than the calculated results.
- `.set_yticks`: Choose which vertical tick marks to show, including an empty list for a point-mass display.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 651–652:

```
651 ax.axvline(0, color="gray", linestyle="--", linewidth=1)
652 title = "Fitted population gamma distribution"
```

Read this block:

- Line 651: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 652, `title`: Set this name to the literal text `'Fitted population gamma distribution'`.

Source lines 653–654:

```
653 if not converged or table["at_bound"].any():
654     title += " (provisional fit)"
```

Read this block:

- Line 653: test `not converged or table['at_bound'].any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 654: update `title` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 655–659:

```
655 ax.set(xlabel="Gamma (common across flavors)", title=title)
656 fig.text(.5, .02, description, ha="center")
657 fig.tight_layout(rect=(0, .06, 1, 1))
658 fig.savefig(output / "gamma_distribution.pdf")
659 plt.close(fig)
```

Read this block:

- Line 655: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 656: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 657: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 658: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 659: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.

plot history distributions

Original function: `plot_history_distributions`. Source begins at line 662.

Delegates to the population-distribution export routine immediately after a split fit. Its side effects are tables and plots, not new coefficient estimates. The readable estimator keeps the full export calculation in the same file; the original estimator imports that helper.

Definition and stated intent:

Source lines 662–663:

```
662 def plot_history_distributions(table, output, converged=True):
663     """Export fitted marginal population distributions after every split fit."""
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 664–665:

```
664 from history_distributions import export_history_distributions
665 export_history_distributions(table, output, converged)
```

Read this block:

- Line 664: from `history_distributions`, import `export_history_distributions` into this module's namespace. Importing can execute the imported module's top-level setup.

- Line 665: call `export_history_distributions` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `export_history_distributions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

fit model

Original function: `fit_model`. Source begins at line 668.

This coordinates point fitting, reporting, optional inference, and saved files. It turns `best.x` into a labeled table. The boundary list comprehension pairs each coefficient with its bounds and tests distance below `1e-4`; flags do not automatically change the estimate. Saving point estimates before bootstrap protects the useful main result if a long inference job is interrupted.

The nested `estimate` callback supplies demand fitting to the generic bootstrap. The bootstrap runs only after a converged main fit and with the original weekly frame available. Missing inference columns are explicitly filled with NaN so an absent SE cannot be mistaken for zero. `vars(args).items()` enumerates settings; Path objects become strings so JSON serialization can encode them. The returned `best` is still a SciPy result object, while CSV/JSON provide portable outputs.

Distribution outputs. After the main split fit, both history marginals are exported as parameter/quantile summaries, density/CDF grids and PDF/PNG plots. Their SDs are population variation; SEs in estimates.csv describe sampling uncertainty. The saved fit can regenerate these plots without optimization.

Definition and stated intent:

Source lines 668–669:

```
668 def fit_model(data, args, frame=None):
669     """Estimate demand, compute requested inference, and save readable outputs."""
```

`def` creates this callable; the parentheses name its inputs (`data`, `args`, `frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 670–677:

```
670 best, results, bounds = optimize_demand(data, args)
671 names = parameter_names(args.model, data["outside"], data["control"])
672 table = pd.DataFrame({"parameter": names, "estimate": best.x})
673 table["at_bound"] = [
674     min(abs(v - lo), abs(v - hi)) < 1e-4 for v, (lo, hi) in zip(best.x, bounds)
675 ]
676 args.output.mkdir(parents=True, exist_ok=True)
677 table.to_csv(args.output / "point_estimates.csv", index=False)
```

Read this block:

- Line 670, `(best, results, bounds)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 671, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 672, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 673, `table['at_bound']`: Store a list computed by the displayed for-expression in the selected entry/column.
- Line 676: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 677: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

Source lines 678–683:

```
678 plot_gamma_distribution(table, args.output, converged=best.success)
679 plot_history_distributions(table, args.output, converged=best.success)
680 wtp = implied_wtp(table, getattr(args, "habit_weight", .7))
681 wtp["provisional_fit"] = not best.success or bool(table["at_bound"].any())
682 wtp.to_csv(args.output / "implied_wtp.csv", index=False)
683 inference = {"method": args.se_method, "status": "disabled"}
```

Read this block:

- Line 678: call `plot_gamma_distribution` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 679: call `plot_history_distributions` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 680, `wtp`: Set this name to the result of calling `implied_wtp(...)`; the arguments and preceding method chain are shown above.
- Line 681, `wtp['provisional_fit']`: Store the Boolean condition `not best.success or bool(table['at_bound'].any())`; it can be a scalar or a row/array mask depending on its inputs in the selected entry/column.
- Line 682: call `wtp.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 683, `inference`: Set this name to a dictionary with named entries `method`, `status`.

Syntax and calls used here:

- `plot_gamma_distribution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `plot_history_distributions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `implied_wtp`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `getattr`: Retrieve an attribute by its string name, optionally supplying a default if it is absent.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 684–714:

```
684 if args.se_method == "bootstrap":
685     if not best.success:
686         inference["status"] = "main fit not converged; bootstrap not run"
687     elif frame is None:
688         raise ValueError("Bootstrap requires the weekly input frame.")
689     else:
```

```

        def estimate(sample, initial):
            bootstrap_data = make_arrays(sample, args.capacity)
            fitted, _, _ = optimize_demand(
                bootstrap_data, args, initial, args.bootstrap_starts, quiet=True
            )
            return fitted

        standard_errors, inference = bootstrap_standard_errors(
            frame, args, estimate, best.x, names
        )
        if standard_errors is not None:
            table = table.merge(
                standard_errors, on="parameter", validate="one_to_one"
            )
elif args.se_method == "hessian":
    if not best.success:
        inference["status"] = "main fit not converged; Hessian inference not run"
    else:
        draws = make_draws(args.draws, args.seed, args.model)
        gradient = lambda theta: demand_objective_gradient(theta, data, draws, args.model)[1]
        standard_errors, inference = hessian_standard_errors(
            best.x, gradient, data["n_households"], bounds, names, args.output)
        if standard_errors is not None:
            table = table.merge(standard_errors, on="parameter", validate="one_to_one")
```

Read this block:

- Line 684: test `args.se_method == 'bootstrap'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 685: test `not best.success`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 686, `inference['status']`: Store the literal text `'main fit not converged; bootstrap not run'` in the selected entry/column.
- Line 687: test `frame is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 688: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 691: define nested callable `estimate`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 698, `(standard_errors, inference)`: Unpack the result of calling `bootstrap_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 701: test `standard_errors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 702, `table`: Set this name to the result of calling `table.merge(...)`; the arguments and preceding method chain are shown above.
- Line 705: test `args.se_method == 'hessian'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 706: test `not best.success`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 707, `inference['status']`: Store the literal text `'main fit not converged; Hessian inference not run'` in the selected entry/column.
- Line 709, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 710, `gradient`: Set this name to a one-expression callable, with input `theta` and returned expression `demand_objective_gradient(theta, data, draws, args.m...`.
- Line 711, `(standard_errors, inference)`: Unpack the result of calling `hessian_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.

- Line 713: test `standard_errors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 714, `table`: Set this name to the result of calling `table.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `bootstrap_standard_errors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `hessian_standard_errors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.
- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 716–718:

```
716 for column in ["std_error", "ci_lower", "ci_upper"]:
717     if column not in table:
718         table[column] = np.nan
```

Read this block:

- Line 716: for each entry yielded by `['std_error', 'ci_lower', 'ci_upper']`, bind/unpack it into `column` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 717: test `column not in table`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 718, `table[column]`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.

Source lines 719–719:

```
719 table.to_csv(args.output / "estimates.csv", index=False)
```

Read this block:

- Line 719: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 720–746:

```
720 summary = dict(
721     model=args.model,
722     price_control=data["control"],
723     history_specification="separate" if args.model in ("separate_history", "heterogeneous_history") else
    ↪ "combined_or_none",
724     sample_restriction="household_size == 1 on each purchase record",
725     households=data["n_households"],
726     weeks=len(data["hh"]),
727     capacity=args.capacity,
728     inference=inference,
729     converged=bool(best.success),
730     message=str(best.message),
731     nll=float(best.fun * data["n_households"]),
732     parameters=len(names),
```

```
    aic=float(2 * best.fun * data["n_households"] + 2 * len(names)),
    bic=float(2 * best.fun * data["n_households"] + np.log(data["n_households"]) * len(names)),
    settings={
        k: str(v) if isinstance(v, Path) else v for k, v in vars(args).items()
    },
    starts=[
        dict(
            nll=float(r.fun * data["n_households"]),
            converged=bool(r.success),
            theta=r.x.tolist(),
        )
        for r in results
    ],
)
```

Read this block:

- Line 720, `summary`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `len`: Count elements/rows rather than summing their numerical values.
- `str`: Convert to text for paths, messages, or serialization.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 747–749:

```
(args.output / "fit.json").write_text(json.dumps(summary, indent=2))
print("Inference: " + inference["status"], flush=True)
print(table.to_string(index=False))
```

Read this block:

- Line 747: call `(args.output / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 748: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 749: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 750–751:

```
if not best.success or table["at_bound"].any():
    print("Provisional fit: inspect convergence/bounds before interpretation.")
```

Read this block:

- Line 750: test `not best.success or table['at_bound'].any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 751: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 752–752:

```
752 return best
```

Read this block:

- Line 752: return the object currently named `best`; plain assignment does not copy it. This ends the call.

fit model.estimate

Original function: `fit_model.estimate`. Source begins at line 691.

This nested function accepts one resampled weekly table and a starting coefficient vector. It rebuilds numerical arrays, runs demand optimization with bootstrap start count, and returns only the selected fit. `fitted, _, _` discards other optimizer outputs. It captures capacity and other settings from outer `args`. Passing this callback avoids importing `est.py` back into `bootstrap_se.py`, which could create a circular module dependency. Price controls are reestimated by the bootstrap before this function is called, not inside this callback.

Definition and stated intent:

Source lines 691–691:

```
691 def estimate(sample, initial):
```

`def` creates this callable; the parentheses name its inputs (`sample`, `initial`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 692–695:

```
692 bootstrap_data = make_arrays(sample, args.capacity)
693 fitted, _, _ = optimize_demand(
694     bootstrap_data, args, initial, args.bootstrap_starts, quiet=True
695 )
```

Read this block:

- Line 692, `bootstrap_data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 693, `(fitted, _, _)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 696–696:

```
696 return fitted
```

Read this block:

- Line 696: return the object currently named `fitted`; plain assignment does not copy it. This ends the call.

main

Original function: `main`. Source begins at line 755.

This is the command-line entry point. `default_model` lets small wrapper scripts reuse the entire interface while selecting a different default model. Each `add_argument` describes a setting, conversion, default, or permitted choices. The parser turns a flag such as `--no-price-control` into the attribute `args.no_price_control`; hyphens become underscores in attribute names.

Validation runs before expensive data loading. `parser.error` prints usage and exits for invalid settings. After parsing, execution is preparation, optional first stages, weekly input export, array construction, and fitting. This division lets tests construct a small `SimpleNamespace` and call internal functions without pretending to run a shell command. Repeating this code in all four entry scripts would allow defaults and validations to drift apart.

Definition and stated intent:

Source lines 755–755:

```
755 def main(default_model="heterogeneous", default_no_price_control=False):
```

`def` creates this callable; the parentheses name its inputs (`default_model`, `default_no_price_control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 756–768:

```
756 parser = argparse.ArgumentParser(description=__doc__)
757 parser.add_argument(
758     "--sample",
759     type=Path,
760     default=DATA_DIR / "summary_outputs/yogurt_estimation_sample.parquet",
761 )
762 parser.add_argument("--flavors", type=Path, default=FLAVOR_FILE)
763 parser.add_argument(
764     "--products-extra",
765     type=Path,
766     help="Annual flavor metadata; defaults to RMS/combined/products_extra.parquet beside the RMS data directory.",
767 )
768 parser.add_argument("--output", type=Path, default=Path("estimation_output"))
```

Read this block:

- Line 756, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 757: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 762: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 763: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 768: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 769–782:

```
769 parser.add_argument(
770     "--model",
771     choices=["naive", "homogeneous", "mixed_naive", "heterogeneous", "mixture", "separate_history",
    ↪ "heterogeneous_history"],
```

```
    default=default_model,
)
parser.add_argument("--first-year", type=int, default=2006)
parser.add_argument("--last-year", type=int, default=2015)
parser.add_argument("--max-households", type=int)
parser.add_argument(
    "--capacity",
    type=int,
    default=0,
    help="0: conditional flavor demand; 7: outside=max(7-Q,0)",
)
```

Read this block:

- Line 769: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 774: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 775: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 776: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 777: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 783–795:

```
parser.add_argument(
    "--no-price-control",
    action="store_true",
    default=default_no_price_control,
    help="Disable the price control using the overall other-DMA mean-price IV",
)
parser.add_argument("--household-dma-column", default="dma_cd")
parser.add_argument(
    "--se-method", choices=["hessian", "bootstrap", "none"], default="hessian"
)
parser.add_argument("--bootstrap-reps", type=int, default=49)
parser.add_argument("--bootstrap-starts", type=int, default=2)
parser.add_argument("--habit-weight", type=float, default=0.7)
```

Read this block:

- Line 783: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 789: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 790: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 793: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 794: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 795: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 796–800:

```
parser.add_argument("--draws", type=int, default=128)
parser.add_argument("--starts", type=int, default=3)
parser.add_argument("--maxiter", type=int, default=600)
```

```
799 parser.add_argument("--seed", type=int, default=306)
800 args = parser.parse_args()
```

Read this block:

- Line 796: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 797: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 798: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 799: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 800, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 801–802:

```
801 if not 0 <= args.habit_weight <= 1 or args.capacity < 0 or args.starts < 1:
802     parser.error("Check habit weight, capacity, and starts.")
```

Read this block:

- Line 801: test `not 0 <= args.habit_weight <= 1 or args.capacity < 0 or args.starts < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 802: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 803–806:

```
803 if args.first_year > args.last_year or (
804     args.max_households is not None and args.max_households < 1
805 ):
806     parser.error("Check years and household limit.")
```

Read this block:

- Line 803: test `args.first_year > args.last_year or (args.max_households is not None and args....`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 806: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 807–808:

```
807 if args.draws < 2 or args.draws % 2 or args.maxiter < 1:
808     parser.error("Use an even draw count >= 2 and maxiter >= 1.")
```

Read this block:

- Line 807: test `args.draws < 2 or args.draws % 2 or args.maxiter < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 808: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 809–812:

```
809 if args.se_method == "bootstrap" and (
810     args.bootstrap_reps < 20 or args.bootstrap_starts < 1
811 ):
812     parser.error("Bootstrap requires at least 20 replications and one start.")
```

Read this block:

- Line 809: test `args.se_method == 'bootstrap' and (args.bootstrap_reps < 20 or args.bootstrap...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 812: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 813–813:

```
813 frame = prepare_data(args)
```

Read this block:

- Line 813, `frame`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `prepare_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 814–818:

```
814 if not args.no_price_control:
815     frame, first_stage = add_price_controls(frame)
816     args.output.mkdir(parents=True, exist_ok=True)
817     first_stage.to_csv(args.output / "first_stage.csv", index=False)
818     print(first_stage.to_string(index=False), flush=True)
```

Read this block:

- Line 814: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 815, `(frame, first_stage)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 816: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 817: call `first_stage.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 818: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 819–821:

```
819 args.output.mkdir(parents=True, exist_ok=True)
820 frame.to_csv(args.output / "weekly_inputs.csv", index=False)
821 fit_model(make_arrays(frame, args.capacity), args, frame)
```

Read this block:

- Line 819: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 820: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 821: call `fit_model` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `fit_model`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est_code/est_gamma.py

This wrapper selects the `homogeneous` default, meaning common alpha and common unrestricted gamma. It imports and calls the shared `main`; it does not define a function locally. The name of a script is not itself a Python function or an instruction to fit a different formula unless its actual call says so.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Common beta, alpha, and flavor-specific gamma. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("homogeneous")
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est_code/est_het_gamma.py

This wrapper supplies the `heterogeneous` default to shared `main`. Both alpha and gamma vary by persistent type. Parentheses call the imported function; writing just `main` would retrieve the function object without executing it.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Common beta; heterogeneous gamma AND alpha. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("heterogeneous")
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est_code/est_het_history.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Independent normal novelty and own-stock coefficients; random alpha. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("heterogeneous_history", default_no_price_control=False)
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est__code/est__het__naive.py

This guarded wrapper supplies the `mixed_naive` default. The common estimator then includes price heterogeneity while excluding gamma. The small file is a convenient command name and shares all data, optimizer, and inference options with the main script.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Common beta, heterogeneous alpha, and no gamma. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("mixed_naive")
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est__code/est__mixture__gamma.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Two-normal gamma mixture; shared lognormal alpha. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("mixture")
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est__code/est__naive.py

This wrapper imports `main` and calls it with the `naive` default only under the entry-point guard. The wrapper contains no new estimation algorithm. `--model` can still override the supplied default because the common parser processes the user's options. Reusing one main function avoids four copies of data preparation and CLI validation.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Common beta/alpha and no history effect. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("naive")
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est_code/est_no_price_control.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Original heterogeneous gamma model without the price control function. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("heterogeneous", default_no_price_control=True)
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est_code/est_separate_history.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Separate common novelty and own-stock coefficients; random alpha. See README.md."""
2 from est import main
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `est`, import `main` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     main("separate_history", default_no_price_control=False)
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

est_code/hessian_se.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–4:

```
1 """Observed-information inference, conditional on first-stage fits."""
2 import numpy as np
3 import pandas as pd
4 from scipy.stats import norm
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 4: from `scipy.stats`, import `norm` into this module's namespace. Importing can execute the imported module's top-level setup.

observed covariance

Original function: `observed_covariance`. Source begins at line 7.

Central-differences the analytic mean-household NLL gradient to form observed information, checks bounds and eigenvalues, and divides its inverse by the household count. Missing household scaling would give incorrect standard errors. Nonpositive or ill-conditioned information is rejected instead of inverted blindly.

Definition and stated intent:

Source lines 7–8:

```
7 def observed_covariance(theta, gradient, households, bounds):
8     """Differentiate the mean negative-log-likelihood gradient, then scale by I."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `gradient`, `households`, `bounds`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 9–11:

```
9 theta = np.asarray(theta, dtype=float)
10 steps = np.cbrt(np.finfo(float).eps) * np.maximum(1., np.abs(theta))
11 lower, upper = np.asarray(bounds, dtype=float).T
```

Read this block:

- Line 9, `theta`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 10, `steps`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 11, `(lower, upper)`: Unpack the attribute `np.asarray(bounds, dtype=float).T` of the object before the dot into these targets in order.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 12–13:

```
12 if np.any(np.minimum(theta - lower, upper - theta) <= np.maximum(steps, 1e-4)):
13     raise ValueError("parameter at or near bounds; normal inference withheld")
```

Read this block:

- Line 12: test `np.any(np.minimum(theta - lower, upper - theta) <= np.maximum(steps, 0.0001))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 13: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 14–14:

```
14 hessian = np.empty((len(theta), len(theta)))
```

Read this block:

- Line 14, `hessian`: Set this name to the result of calling `np.empty(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 15–18:

```
15 for j, step in enumerate(steps):
16     delta = np.zeros(len(theta))
17     delta[j] = step
18     hessian[:, j] = (gradient(theta + delta) - gradient(theta - delta)) / (2 * step)
```

Read this block:

- Line 15: for each entry yielded by `enumerate(steps)`, bind/unpack it into `(j, step)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 16, `delta`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 17, `delta[j]`: Store the object currently named `step`; plain assignment does not copy it in the selected entry/column.
- Line 18, `hessian[:, j]`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 19–19:

```
19 hessian = (hessian + hessian.T) / 2
```

Read this block:

- Line 19, `hessian`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 20–21:

```
20 if not np.isfinite(hessian).all():
21     raise ValueError("nonfinite observed information; SEs withheld")
```

Read this block:

- Line 20: test `not np.isfinite(hessian).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 21: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 22–22:

```
22 eigenvalues = np.linalg.eigvalsh(hessian)
```

Read this block:

- Line 22, `eigenvalues`: Set this name to the result of calling `np.linalg.eigvalsh(...)`; the arguments and preceding method chain are shown above.

Source lines 23–24:

```
23 if eigenvalues[0] <= max(0., eigenvalues[-1] * 1e-10):
24     raise ValueError("observed information not positive definite or ill-conditioned; SEs withheld")
```

Read this block:

- Line 23: test `eigenvalues[0] <= max(0.0, eigenvalues[-1] * 1e-10)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 24: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.

Source lines 25–25:

```
25 return np.linalg.solve(hessian, np.eye(len(theta))) / households
```

Read this block:

- Line 25: return division of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

hessian standard errors

Original function: `hessian_standard_errors`. Source begins at line 28.

Converts an accepted observed-information covariance into parameter SEs and normal intervals, writes the covariance, and returns inference metadata. These are conditional model-based SEs; they do not include price-first-stage uncertainty or provide a robust sandwich covariance.

Definition and stated intent:

Source lines 28–28:

```
28 def hessian_standard_errors(theta, gradient, households, bounds, names, output):
```

`def` creates this callable; the parentheses name its inputs (`theta`, `gradient`, `households`, `bounds`, `names`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 29–31:

```
29 report = dict(method="observed-information Hessian",
30               conditional_on="estimated price controls, retail instruments, and fixed integration draws",
31               first_stage_uncertainty="not included", robust=False)
```

Read this block:

- Line 29, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 32–36:

```
32 try:
33     covariance = observed_covariance(theta, gradient, households, bounds)
34 except ValueError as error:
35     report["status"] = str(error)
36     return None, report
```

Read this block:

- Line 32: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 33, `covariance`: Set this name to the result of calling `observed_covariance(...)`; the arguments and preceding method chain are shown above.
- Line 34: catch `ValueError` from the preceding try; the optional as-name holds the error object for logging.
- Line 35, `report['status']`: Store the result of calling `str(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 36: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `observed_covariance`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 37–42:

```
37 pd.DataFrame(covariance, index=names, columns=names).to_csv(output / "covariance.csv")
38 se = np.sqrt(np.diag(covariance))
39 critical = norm.ppf(.975)
40 table = pd.DataFrame(dict(parameter=names, std_error=se,
41                           ci_lower=theta - critical * se, ci_upper=theta + critical * se))
42 report["status"] = "computed; normal 95% intervals; first-stage uncertainty omitted"
```

Read this block:

- Line 37: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38, `se`: Set this name to the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above.
- Line 39, `critical`: Set this name to the result of calling `norm.ppf(...)`; the arguments and preceding method chain are shown above.
- Line 40, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 42, `report['status']`: Store the literal text `'computed; normal 95% intervals; first-stage uncertainty omitted'` in the selected entry/column.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

Source lines 43–43:

```
43 return table, report
```

Read this block:

- Line 43: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

est__code/history__distributions.py

Saved-fit distribution exporter. Read this when you want the two estimated population marginals without rerunning optimization. It reads fit metadata and coefficients, not household histories. The zero-SD case is a point mass. The reusable function is called automatically after a split fit.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Export fitted population history distributions, including from saved fits."""
2 from pathlib import Path
3 import os
4 import tempfile
5 from statistics import NormalDist
6 import argparse
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `os`. An as-name is a convenient local alias, not another library.
- Line 4: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 5: from `statistics`, import `NormalDist` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: import `argparse`. An as-name is a convenient local alias, not another library.

Source lines 7–9:

```
7 import json
8 import numpy as np
9 import pandas as pd
```

Read this block:

- Line 7: import `json`. An as-name is a convenient local alias, not another library.
- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 55–64:

```
55 if __name__ == '__main__':
56     parser = argparse.ArgumentParser(description=__doc__)
57     parser.add_argument('--fit-dir', required=True, type=Path)
58     parser.add_argument('--output', type=Path)
59     args = parser.parse_args()
60     fit = json.loads((args.fit_dir/'fit.json').read_text())
61     if fit['model'] not in ['separate_history', 'heterogeneous_history']:
62         parser.error('Requires a split-history fit')
63     export_history_distributions(pd.read_csv(args.fit_dir/'point_estimates.csv'),
64                                  args.output or args.fit_dir, fit.get('converged', False))
```

Read this block:

- Line 55: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 56, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 57: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 58: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 59, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.
- Line 60, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 61: test `fit['model'] not in ['separate_history', 'heterogeneous_history']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 62: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 63: call `export_history_distributions` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.
- `export_history_distributions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

export history distributions

Original function: `export_history_distributions`. Source begins at line 12.

Inputs and outputs. The coefficient table supplies mean_novelty, mean_own_stock and their SDs; output is a directory. For each marginal, the function writes a summary of mean, SD, sign share and quantiles, a grid of densities/CDFs, and PDF/PNG plots. These describe population heterogeneity, not confidence intervals or household posterior means.

`NormalDist(mean, sd)` represents a fitted normal distribution; `.cdf(0)` gives its share below zero and `.inv_cdf(.25)` gives its lower quartile. A zero SD is handled as a point mass instead of dividing by zero. A negative own-stock coefficient means satiation, while a positive one means reinforcement. Convergence/boundary status marks provisional exports. This calculation recovers the already estimated distribution without optimizing demand again.

Definition and stated intent:

Source lines 12–12:

```
12 def export_history_distributions(table, output, converged=True):
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 13–13:

```
13 p = table.set_index('parameter').estimate
```

Read this block:

- Line 13, `p`: Set this name to the attribute `table.set_index('parameter').estimate` of the object before the dot.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

Source lines 14–15:

```
14 if 'mean_novelty' not in p:
15     return
```

Read this block:

- Line 14: test `'mean_novelty' not in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 15: bare return ends this call immediately with None. Later statements in the function are skipped.

Source lines 16–21:

```
16 output = Path(output)
17 output.mkdir(parents=True, exist_ok=True)
18 provisional = not converged or table.get('at_bound', pd.Series(False)).astype(str).str.lower().eq('true').any()
19 summaries, curves = [], []
20 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
21 import matplotlib
```

Read this block:

- Line 16, `output`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 17: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 18, `provisional`: Set this name to the Boolean condition `not converged or table.get('at_bound', pd.Series(False)).astyp...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 19, `(summaries, curves)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 20: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 21: import `matplotlib`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 22–24:

```
22 matplotlib.use('Agg')
23 import matplotlib.pyplot as plt
24 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

Read this block:

- Line 22: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 23: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 24, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 25–45:

```
25 for ax, term, label in zip(axes, ['novelty', 'own_stock'], ['Switching / novelty', 'Own-stock habit']):
26     mean, sd = float(p['mean_'+term]), float(p.get('sd_'+term, 0.))
27     if not np.isfinite([mean, sd]).all() or sd < 0:
28         raise ValueError('Invalid distribution parameters')
29     normal = NormalDist(mean, sd) if sd > 0 else None
30     row = dict(coefficient=term, mean=mean, sd=sd, point_mass=sd == 0,
31                share_below_zero=normal.cdf(0) if normal else float(mean < 0), provisional=bool(provisional))
32     for q in [.01, .05, .25, .5, .75, .95, .99]:
33         row[f'q{int(100*q):02d}'] = normal.inv_cdf(q) if normal else mean
34     summaries.append(row)
35     if normal:
36         x = np.linspace(mean-4*sd, mean+4*sd, 801)
37         density = np.exp(-.5*((x-mean)/sd)**2)/(sd*np.sqrt(2*np.pi))
38         ax.plot(x, density)
39         curves.extend(dict(coefficient=term, value=float(v), density=float(d), cdf=normal.cdf(float(v)),
    ↪ point_mass=False)
40                       for v, d in zip(x, density))
41     else:
42         ax.axvline(mean, label='Point mass')
43         curves.append(dict(coefficient=term, value=mean, density=np.nan, cdf=1., point_mass=True))
44     ax.axvline(0, color='gray', ls='--')
45     ax.set(title=f'{label}\nMean={mean:.4g}; SD={sd:.4g}', xlabel='Utility coefficient', ylabel='Population
    ↪ density')
```

Read this block:

- Line 25: for each entry yielded by `zip(axes, ['novelty', 'own_stock'], ['Switching / novelty', 'O...`, bind/unpack it into `(ax, term, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 26, `(mean, sd)`: Unpack a tuple containing 2 displayed entries into these targets in order.

- Line 27: test `not np.isfinite([mean, sd]).all() or sd < 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 28: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 29, `normal`: Set this name to the true-branch value if `sd > 0` holds, otherwise the else-branch value.
- Line 30, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 32: for each entry yielded by `[0.01, 0.05, 0.25, 0.5, 0.75, 0.95, 0.99]`, bind/unpack it into `q` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 33, `row[f'q{int(100 * q):02d}']`: Store the true-branch value if `normal` holds, otherwise the else-branch value in the selected entry/column.
- Line 34: call `summaries.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 35: test `normal`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 36, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 37, `density`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 38: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39: call `curves.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `curves.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 44: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 45: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.axvline`: Add a vertical reference line, such as zero gamma or a point-mass type.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `NormalDist`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.cdf`: Evaluate cumulative probability below a threshold, used for the population share with negative gamma.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another

expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 46–49:

```
46 pd.DataFrame(summaries).to_csv(output/'history_distribution_summary.csv', index=False)
47 pd.DataFrame(curves).to_csv(output/'history_distribution_grid.csv', index=False)
48 fig.suptitle('Fitted population history distributions'+(' (provisional)' if provisional else ''))
49 fig.tight_layout()
```

Read this block:

- Line 46: call `pd.DataFrame(summaries).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 47: call `pd.DataFrame(curves).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 48: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 50–51:

```
50 for extension in ['pdf', 'png']:
51     fig.savefig(output/f'history_distributions.{extension}')
```

Read this block:

- Line 50: for each entry yielded by `['pdf', 'png']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 51: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 52–52:

```
52 plt.close(fig)
```

Read this block:

- Line 52: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.

est_code/price_control.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–7:

```
"""Other-DMA instruments and the price first stage. See README.md."""
import numpy as np
import pandas as pd

from scipy.stats import t as student_t

FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 5: from `scipy.stats`, import `t as student_t` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 7, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

check dma

Original function: `check_dma`. Source begins at line 10.

This converts a pandas Series to integer DMA IDs, rejecting missing, nonnumeric, nonpositive, and fractional entries. `pd.to_numeric(errors="coerce")` turns bad strings into missing numbers so one Boolean mask can find all invalid rows. `codes == np.floor(codes)` checks integrality without rounding fractional inputs into a valid ID. Only after validation does `astype("int64")` convert the dtype. `description` makes an error identify the consumer or retail source. Quietly replacing an invalid DMA with zero would contaminate own-market exclusion.

Definition and stated intent:

Source lines 10–11:

```
def check_dma(values, description):
    """Normalize Nielsen's numeric DMA identifiers across panel and RMS files."""
```

`def` creates this callable; the parentheses name its inputs (`values`, `description`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 12–13:

```
codes = pd.to_numeric(values, errors="coerce")
valid = codes.notna() & (codes > 0) & (codes == np.floor(codes))
```

Read this block:

- Line 12, `codes`: Set this name to the result of calling `pd.to_numeric(...)`; the arguments and preceding method chain are shown above.
- Line 13, `valid`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.floor`: Take the greatest integer no larger than each value; equality with it tests whether a positive code is integral.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 14–15:

```
14 if not valid.all():
15     raise ValueError(f"Missing/invalid {description} DMA codes: {(~valid).sum():,}")
```

Read this block:

- Line 14: test `not valid.all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 15: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 16–16:

```
16 return codes.astype("int64")
```

Read this block:

- Line 16: return the result of calling `codes.astype(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

other dma instruments

Original function: `other_dma_instruments`. Source begins at line 19.

This retained helper accepts three pandas tables and returns weekly rows with UPC-matched flavor IVs. It is **not** the pooled instrument construction used by current `est.py`. It first averages retail-DMA prices outside the consumer's DMA per UPC/week, then averages across the exposure store's UPC basket.

Renaming the own-DMA table's columns makes its keys match the merge. `present` is a Boolean Series; converting it to integers subtracts one from the DMA count only when that own-market quote exists. The grouped `count` ignores missing values whereas `size` counts all rows. Comparing them detects an incompletely instrumented basket and marks the whole flavor IV missing, rather than quietly averaging only a subset. Several joins replace a much slower nested lookup loop.

Definition and stated intent:

Source lines 19–25:

```
19 def other_dma_instruments(weekly, exposure_upcs, dma_prices):
20     """Match each exposure-store UPC to its same-week price in OTHER DMAs.
21
22     dma_prices has ONE mean package price per UPC/week/retail DMA. First average
23     these means over all DMAs except the CONSUMER's DMA, then average over the
24     exposure store's UPCs in a flavor. This matches the local price's UPC basket.
25     """
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `exposure_upcs`, `dma_prices`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 26–33:

```
dma_prices = dma_prices.copy()
dma_prices["dma_code"] = check_dma(dma_prices["dma_code"], "retail")
pk = ["upc", "week"]
totals = (dma_prices.groupby(pk)["dma_price"].agg(["sum", "count"])
          .rename(columns={"sum": "sum_dma", "count": "n_dma"}).reset_index())
exposure = weekly[["store_code_uc", "week", "dma_cd"]].drop_duplicates()
exposure = exposure.merge(exposure_upcs, on=["store_code_uc", "week"],
                          how="left", validate="many_to_many")
```

Read this block:

- Line 26, `dma_prices`: Set this name to the result of calling `dma_prices.copy(...)`; the arguments and preceding method chain are shown above.
- Line 27, `dma_prices['dma_code']`: Store the result of calling `check_dma(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 28, `pk`: Set this name to the list `['upc', 'week']`.
- Line 29, `totals`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 31, `exposure`: Set this name to the result of calling `.drop_duplicates(...)`; the arguments and preceding method chain are shown above.
- Line 32, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `check_dma`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.
- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 34–40:

```
exposure = exposure.merge(totals, on=pk, how="left", validate="many_to_one")
own = dma_prices.rename(columns={"dma_code": "dma_cd", "dma_price": "own_dma_price"})
exposure = exposure.merge(own, on=pk + ["dma_cd"], how="left", validate="many_to_one")
present = exposure["own_dma_price"].notna()
denominator = exposure["n_dma"] - present.astype(int)
exposure["iv"] = ((exposure["sum_dma"] - exposure["own_dma_price"].fillna(0))
                  / denominator.where(denominator > 0))
```

Read this block:

- Line 34, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.
- Line 35, `own`: Set this name to the result of calling `dma_prices.rename(...)`; the arguments and preceding method chain are shown above.
- Line 36, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.

- Line 37, `present`: Set this name to the result of calling `exposure['own_dma_price'].notna(...)`; the arguments and preceding method chain are shown above.
- Line 38, `denominator`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 39, `exposure['iv']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 42–46:

```
42 keys = ["store_code_uc", "week", "dma_cd", "flavor"]
43 grouped = exposure.groupby(keys)["iv"].agg(["mean", "count", "size"])
44 grouped.loc[grouped["count"] != grouped["size"], "mean"] = np.nan
45 result = (grouped["mean"].unstack("flavor").reindex(columns=FLAVORS)
46           .rename(columns={f: "iv_" + f for f in FLAVORS}).reset_index())
```

Read this block:

- Line 42, `keys`: Set this name to the list `['store_code_uc', 'week', 'dma_cd', 'flavor']`.
- Line 43, `grouped`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 44, `grouped.loc[grouped['count'] != grouped['size'], 'mean']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 45, `result`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.unstack`: Move an index level into columns, such as turning long flavor counts into wide weekly outcomes.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 47–48:

```
47 return weekly.merge(result, on=["store_code_uc", "week", "dma_cd"], how="left",
48                     validate="many_to_one")
```

Read this block:

- Line 47: return the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above. This ends the call.

absorb effects

Original function: `absorb_effects`. Source begins at line 51.

Input `values` is a numerical matrix; `groups` lists row-aligned group labels such as week and DMA. Output is a copy with group means removed. Factorization converts arbitrary labels to dense integers. `np.add.at(means, code, residual)` accumulates rows into group sums even when indices repeat; ordinary fancy-index `means[code] += residual` may not accumulate repeats as intended.

`counts[:, None]` makes group counts a column so every variable is divided by its group size. The function cycles week and DMA demeaning until the biggest absolute change falls below tolerance, then returns. A maximum iteration limit raises on failure. A huge explicit dummy matrix is a valid alternative used in tests, but its memory grows with the number of fixed-effect groups. One pass is insufficient when group cells are unbalanced.

Definition and stated intent:

Source lines 51–56:

```
51 def absorb_effects(values, groups, tolerance=1e-10, maxiter=2000):
52     """Remove week and DMA means without a huge matrix of dummy variables.
53 
54     Repeated demeaning is needed because DMA/week cells are unbalanced.
55     Each household-week exposure has equal weight in the first stage.
56     """
```

`def` creates this callable; the parentheses name its inputs (`values`, `groups`, `tolerance`, `maxiter`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 57–58:

```
57 residual = np.asarray(values, dtype=float).copy()
58 codes = [pd.factorize(group, sort=True)[0] for group in groups]
```

Read this block:

- Line 57, `residual`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.
- Line 58, `codes`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 59–68:

```
59 for _ in range(maxiter):
60     previous = residual.copy()
61     for code in codes:
62         counts = np.bincount(code)
63         means = np.zeros((len(counts), residual.shape[1]))
64         np.add.at(means, code, residual)
65         means /= counts[:, None]
66         residual -= means[code]
67     if np.max(np.abs(residual - previous)) < tolerance:
68         return residual
```

Read this block:

- Line 59: for each entry yielded by `range(maxiter)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 60, `previous`: Set this name to the result of calling `residual.copy(...)`; the arguments and preceding method chain are shown above.
- Line 61: for each entry yielded by `codes`, bind/unpack it into `code` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 62, `counts`: Set this name to the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above.

- Line 63, `means`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 64: call `np.add.at` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65: update `means` using `/=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 66: update `residual` using `-=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 67: test `np.max(np.abs(residual - previous)) < tolerance`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 68: return the object currently named `residual`; plain assignment does not copy it. This ends the call.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.add.at`: Accumulate in place at possibly repeated indices; this avoids the lost-repeat behavior of buffered fancy-index updates.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 69–69:

```
69 raise ValueError("First-stage fixed-effect demeaning did not converge.")
```

Read this block:

- Line 69: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

add price controls

Original function: `add_price_controls`. Source begins at line 72.

This returns `(augmented_frame, diagnostic_table)`. For each flavor it selects covered prices, absorbs week/DMA effects jointly from price, instrument, and Xi, then partials Xi out of y and z. `within[:, 0]`, `[:, 1]`, `[:, 2:]` split those roles by known column order. `lstsq(...)[0]` keeps only the coefficient vector from the least-squares result tuple.

Dot products compute the IV slope after partialling. A small residual instrument sum of squares raises instead of dividing by an almost zero denominator. `np.linalg.lstsq` tolerates redundant controls more gracefully than explicitly inverting their cross-product matrix. The residual is assigned only to covered rows; unavailable alternatives retain zero placeholders. Cluster score sums, partial R-squared, and residual SD describe first-stage relevance and fit. They do not prove instrument validity. Reusing this function in bootstrap replications keeps the two-stage estimator consistent across samples.

Definition and stated intent:

Source lines 72–77:

```
72 def add_price_controls(frame):
73     """OLS separately by flavor: price ~ IV + all three Xi + week FE + DMA FE.
74 
75     Refit this function in every household bootstrap sample. Return residuals
76     in DOLLARS (no arbitrary rescaling) for the choice-model control function.
```

```
    """
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 78–79:

```
result = frame.copy()
reports = []
```

Read this block:

- Line 78, `result`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 79, `reports`: Set this name to the list `[]`.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.

Source lines 80–119:

```
for flavor in FLAVORS:
    columns = ["price_" + flavor, "iv_" + flavor] + ["xi_" + f for f in FLAVORS]
    covered = np.isfinite(result["price_" + flavor]) & (result["price_" + flavor] > 0)
    sample = result.loc[covered]
    if sample.empty:
        raise ValueError(f"No covered prices for the {flavor} first stage.")
    hh, households = pd.factorize(sample["household_code"], sort=True)
    values = sample[columns].to_numpy(float)
    if not np.isfinite(values).all():
        raise ValueError(f"Nonfinite first-stage inputs for {flavor}.")
    within = absorb_effects(values, [sample["week"], sample["dma_cd"]])
    y, z, controls = within[:, 0], within[:, 1], within[:, 2:]
    # Partial out Xi before estimating the excluded instrument's coefficient.
    # This tolerates redundant Xi columns, but NOT a redundant instrument.
    z_partial = z - controls @ np.linalg.lstsq(controls, z, rcond=None)[0]
    y_partial = y - controls @ np.linalg.lstsq(controls, y, rcond=None)[0]
    z_ss = z_partial @ z_partial
    if z_ss <= 1e-12 * max(1., values[:, 1] @ values[:, 1]):
        raise ValueError(f"No excluded-instrument variation after controls for {flavor}.")
    pi = (z_partial @ y_partial) / z_ss
    control_beta = np.linalg.lstsq(controls, y - pi*z, rcond=None)[0]
    residual = y - pi*z - controls @ control_beta
    if np.std(residual) < 1e-10:
        raise ValueError(f"First-stage residual for {flavor} is essentially zero.")
    result["cf_" + flavor] = 0.
    result.loc[covered, "cf_" + flavor] = residual
    # Household-clustered partial-IV t/F diagnostic (not a weak-IV-robust test).
    scores = np.bincount(hh, weights=z_partial * residual, minlength=len(households))
    pi_se = np.sqrt((scores @ scores) / z_ss**2)
    if len(households) > 1:
        pi_se *= np.sqrt(len(households) / (len(households) - 1))
    f_stat = float((pi / pi_se)**2) if pi_se > 0 else np.nan
    restricted_ss = y_partial @ y_partial
    partial_r2 = 1 - (residual @ residual) / restricted_ss if restricted_ss > 0 else np.nan
    reports.append(dict(flavor=flavor, iv_coefficient=float(pi),
                        iv_cluster_se=float(pi_se), iv_cluster_F=f_stat,
                        iv_pvalue=float(2*student_t.sf(np.sqrt(f_stat), len(households)-1))
                        if len(households) > 1 else np.nan,
                        partial_R2=float(partial_r2), residual_sd=float(np.std(residual)),
                        observations=len(sample), households=len(households)))
```

Read this block:

- Line 80: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 81, `columns`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 82, `covered`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 83, `sample`: Set this name to the entry/slice selected by `result.loc[covered]`; brackets select from the object to their left.
- Line 84: test `sample.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 85: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 86, `(hh, households)`: Unpack the result of calling `pd.factorize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 87, `values`: Set this name to the result of calling `sample[columns].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 88: test `not np.isfinite(values).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 89: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 90, `within`: Set this name to the result of calling `absorb_effects(...)`; the arguments and preceding method chain are shown above.
- Line 91, `(y, z, controls)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 94, `z_partial`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 95, `y_partial`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 96, `z_ss`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 97: test `z_ss <= 1e-12 * max(1.0, values[:, 1] @ values[:, 1])`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 98: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 99, `pi`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 100, `control_beta`: Set this name to the entry/slice selected by `np.linalg.lstsq(controls, y - pi * z, rcond=None)[0]`; brackets select from the object to their left.
- Line 101, `residual`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 102: test `np.std(residual) < 1e-10`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 103: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 104, `result['cf_' + flavor]`: Store the literal value `0.0` in the selected entry/column.
- Line 105, `result.loc[covered, 'cf_' + flavor]`: Store the object currently named `residual`; plain assignment does not copy it in the selected entry/column.
- Line 107, `scores`: Set this name to the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above.
- Line 108, `pi_se`: Set this name to the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above.
- Line 109: test `len(households) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 110: update `pi_se` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

- Line 111, `f_stat`: Set this name to the true-branch value if `pi_se > 0` holds, otherwise the else-branch value.
- Line 112, `restricted_ss`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 113, `partial_r2`: Set this name to the true-branch value if `restricted_ss > 0` holds, otherwise the else-branch value.
- Line 114: call `reports.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `absorb_effects`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.lstsq`: Solve least squares without explicitly inverting a cross-product matrix; [0] keeps its coefficient vector.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `len`: Count elements/rows rather than summing their numerical values.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.sf`: Evaluate survival probability above a threshold, useful for a two-sided t diagnostic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 120–120:

```
120 return result, pd.DataFrame(reports)
```

Read this block:

- Line 120: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

est_code/regenerate_weekly_inputs.py

Server-only workflow for the deleted household export. Saved settings reconstruct the original sample/history/menu table and optional first stages. Exact household/week totals and likelihood agreement are required before writing weekly_inputs.csv. Demand estimates and inference are preserved.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Regenerate weekly inputs on the server without refitting saved demand parameters."""
import argparse
import json
from pathlib import Path
import tempfile
from types import SimpleNamespace
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 3: import `json`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 6: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 8–12:

```
import numpy as np
import pandas as pd

from est import prepare_data, make_arrays, make_draws, parameter_names, household_loglike
from price_control import add_price_controls
```

Read this block:

- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 11: from `est`, import `prepare_data, make_arrays, make_draws, parameter_names, household_loglike` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: from `price_control`, import `add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 77–78:

```
if __name__ == '__main__':
    main()
```

Read this block:

- Line 77: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 78: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

regenerate

Original function: `regenerate`. Source begins at line 15.

Job and contract. Rebuild the deleted weekly_inputs.csv on the server using the existing fit directory, optionally overriding moved sample/reference paths. The function reads saved settings, calls the same sample/history/menu preparation, and reconstructs price controls. It never calls the demand optimizer or replaces fitted parameters and inference.

`SimpleNamespace(**settings)` turns dictionary keys into ordinary attributes. Converting saved path strings back to Path objects restores the input contract. The reconstructed table must match household/week counts and the saved integrated likelihood, using the exact parameter order, draw count and seed. A mismatch raises before publishing a CSV.

`NamedTemporaryFile` creates a private intermediate beside the destination; `temporary.replace(target)` publishes only a verified complete export. The finally block cleans up an unfinished intermediate. The JSON regeneration record stores diagnostics, not household rows. Run this entry point on the server; reading the textbook does not execute it.

Definition and stated intent:

Source lines 15–15:

```
15 def regenerate(fit_dir, sample=None, flavors=None):
```

`def` creates this callable; the parentheses name its inputs (`fit_dir`, `sample`, `flavors`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 16–21:

```
16 fit_dir = Path(fit_dir)
17 fit = json.loads((fit_dir / 'fit.json').read_text())
18 settings = dict(fit['settings'])
19 required = ['sample', 'first_year', 'last_year', 'max_households', 'capacity',
20             'no_price_control', 'household_dma_column', 'habit_weight', 'draws', 'seed']
21 missing = set(required)-settings.keys()
```

Read this block:

- Line 16, `fit_dir`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 17, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 18, `settings`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 19, `required`: Set this name to a list containing 10 displayed entries.
- Line 21, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 22–23:

```
22 if missing:
23     raise ValueError(f'Saved fit lacks reconstruction settings: {sorted(missing)}')
```

Read this block:

- Line 22: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 23: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 24–25:

```
24 settings['model'] = fit['model']
25 settings['output'] = fit_dir
```

Read this block:

- Line 24, `settings['model']`: Store the entry/slice selected by `fit['model']`; brackets select from the object to their left in the selected entry/column.
- Line 25, `settings['output']`: Store the object currently named `fit_dir`; plain assignment does not copy it in the selected entry/column.

Source lines 26–30:

```
26 for key in ['sample', 'flavors', 'products_extra']:
27     if settings.get(key) is not None:
28         settings[key] = Path(settings[key])
29     else:
30         settings[key] = None
```

Read this block:

- Line 26: for each entry yielded by `['sample', 'flavors', 'products_extra']`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 27: test `settings.get(key) is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 28, `settings[key]`: Store the result of calling `Path(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 30, `settings[key]`: Store the literal value `None` in the selected entry/column.

Syntax and calls used here:

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 31–32:

```
31 if sample is not None:
32     settings['sample'] = Path(sample)
```

Read this block:

- Line 31: test `sample is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 32, `settings['sample']`: Store the result of calling `Path(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 33–34:

```
33 if flavors is not None:
34     settings['flavors'] = Path(flavors)
```

Read this block:

- Line 33: test `flavors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 34, `settings['flavors']`: Store the result of calling `Path(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 35–36:

```
35 args = SimpleNamespace(**settings)
36 frame = prepare_data(args)
```

Read this block:

- Line 35, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 36, `frame`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `prepare_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 37–39:

```
37 if not args.no_price_control:
38     # Reconstruct the original first stage; preserve saved inference artifacts.
39     frame, _ = add_price_controls(frame)
```

Read this block:

- Line 37: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 39, `(frame, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 40–40:

```
40 data = make_arrays(frame, args.capacity)
```

Read this block:

- Line 40, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 41–42:

```
41 if len(frame) != fit['weeks'] or data['n_households'] != fit['households']:
42     raise ValueError('Regenerated household/week totals do not match the saved fit; no CSV written.')
```

Read this block:

- Line 41: test `len(frame) != fit['weeks'] or data['n_households'] != fit['households']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 42: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 43–48:

```
43 estimates = pd.read_csv(fit_dir / 'point_estimates.csv')
44 parameters = estimates.set_index('parameter').estimate
45 names = parameter_names(args.model, data['outside'], data['control'])
46 theta = np.array([parameters[name] for name in names])
```

```
draws = make_draws(args.draws, args.seed, args.model)
nll = -float(household_loglike(theta, data, draws, args.model).sum())
```

Read this block:

- Line 43, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 44, `parameters`: Set this name to the attribute `estimates.set_index('parameter').estimate` of the object before the dot.
- Line 45, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 46, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 47, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 48, `nll`: Set this name to the negated or sign-adjusted expression `-float(household_loglike(theta, data, draws, args.model)....`.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 49–50:

```
if not np.isfinite(nll) or not np.isclose(nll, fit['nll'], rtol=1e-9, atol=1e-4):
    raise ValueError(f'Regenerated likelihood {nll} differs from saved {fit["nll"]}; no CSV written.')
```

Read this block:

- Line 49: test `not np.isfinite(nll) or not np.isclose(nll, fit['nll'], rtol=1e-09, atol=0.0001)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 50: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 51–51:

```
target = fit_dir / 'weekly_inputs.csv'
```

Read this block:

- Line 51, `target`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 53–54:

```
53 with tempfile.NamedTemporaryFile(dir=fit_dir, prefix='.weekly_inputs_', suffix='.csv', delete=False) as handle:
54     temporary = Path(handle.name)
```

Read this block:

- Line 53: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 54, `temporary`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.

Source lines 55–59:

```
55 try:
56     frame.to_csv(temporary, index=False)
57     temporary.replace(target)
58 finally:
59     temporary.unlink(missing_ok=True)
```

Read this block:

- Line 55: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 56: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 57: call `temporary.replace` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 59: call `temporary.unlink` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 60–64:

```
60 report = dict(households=data['n_households'], weeks=len(frame), reconstructed_nll=nll,
61               saved_nll=fit['nll'], model=args.model, sample=str(args.sample),
62               demand_refitted=False, price_control_refitted=not args.no_price_control)
63 (fit_dir / 'weekly_inputs_regeneration.json').write_text(json.dumps(report, indent=2)+'\n')
64 print(f'Regenerated {target}; household/week totals and saved likelihood verified.', flush=True)
```

Read this block:

- Line 60, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 63: call `.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 65–65:

```
65 return report
```

Read this block:

- Line 65: return the object currently named `report`; plain assignment does not copy it. This ends the call.

main

Original function: `main`. Source begins at line 68.

Parses a fit directory and optional moved input paths, then calls server-side reconstruction. Missing required paths stop before processing purchases. No optimizer is invoked.

Definition and stated intent:

Source lines 68–68:

```
68 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 69–74:

```
69 parser = argparse.ArgumentParser(description=__doc__)
70 parser.add_argument('--fit-dir', type=Path, required=True)
71 parser.add_argument('--sample', type=Path, help='Override saved prepared-sample path if moved.')
72 parser.add_argument('--flavors', type=Path, help='Override saved flavor-reference path if moved.')
73 args = parser.parse_args()
74 regenerate(args.fit_dir, args.sample, args.flavors)
```

Read this block:

- Line 69, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 70: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 71: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 72: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 73, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.
- Line 74: call `regenerate` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.
- `regenerate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est_code/wtp.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own

input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 """Population WTP moments under independent gamma and lognormal -alpha."""
2 import numpy as np
3 import pandas as pd
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `pandas as pd`. An as-name is a convenient local alias, not another library.

implied wtp

Original function: `implied_wtp`. Source begins at line 6.

Computes population moments of coefficient/positive price sensitivity. Independence permits multiplying the coefficient moments by inverse-lognormal moments. Averaging these household ratios differs from dividing a mean coefficient by mean price sensitivity. Split history rows use their own coefficients without lambda weights.

Definition and stated intent:

Source lines 6–7:

```
6 def implied_wtp(estimates, habit_weight=.7):
7     """Return population moments, not sampling standard errors, in price units."""
```

`def` creates this callable; the parentheses name its inputs (`estimates`, `habit_weight`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 8–12:

```
8 p = dict(zip(estimates.parameter, estimates.estimate))
9 mu, sigma = p['mu_log_minus_alpha'], p.get('sd_log_minus_alpha', 0.)
10 inverse_mean = np.exp(-mu + sigma**2 / 2)
11 inverse_second = np.exp(-2 * mu + 2 * sigma**2)
12 rows = []
```

Read this block:

- Line 8, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 9, `(mu, sigma)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 10, `inverse_mean`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 11, `inverse_second`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 12, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 14–18:

```
14 def add(term, mean, second, interpretation):
15     wtp_mean = mean * inverse_mean
16     variance = max(0., second * inverse_second - wtp_mean**2)
17     rows.append(dict(term=term, mean_wtp=wtp_mean, population_sd=np.sqrt(variance),
18                      interpretation=interpretation))
```

Read this block:

- Line 14: define nested callable `add`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 20–24:

```
20 for name in ['inside_constant', 'beta_berry', 'beta_plain']:
21     if name in p:
22         interpretation = ('Other flavor vs outside, at zero history and control residual'
23                           if name == 'inside_constant' else 'Flavor premium relative to other')
24         add(name, p[name], p[name]**2, interpretation)
```

Read this block:

- Line 20: for each entry yielded by `['inside_constant', 'beta_berry', 'beta_plain']`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 21: test `name in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 22, `interpretation`: Set this name to the true-branch value if `name == 'inside_constant'` holds, otherwise the else-branch value.
- Line 24: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `add`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 25–39:

```
25 if 'mean_gamma' in p:
26     mean, sd = p['mean_gamma'], p.get('sd_gamma', 0.)
27     second = mean**2 + sd**2
28     if 'mixture_weight' in p:
29         weight = p['mixture_weight']
30         mean2 = mean + p['gamma_mean_gap']
31         second2 = mean2**2 + p['sd_gamma_2']**2
32         add('gamma_component_1', mean, second, 'Per unit Xi, conditional on component 1')
33         add('gamma_component_2', mean2, second2, 'Per unit Xi, conditional on component 2')
34         mean, second = weight * mean + (1 - weight) * mean2, weight * second + (1 - weight) * second2
35     add('gamma', mean, second, 'Per unit increase in history index Xi')
36     add('gamma_novelty', (1 - habit_weight) * mean, (1 - habit_weight)**2 * second,
37         'Per unit novelty N, holding own-flavor stock fixed')
38     add('gamma_own_stock', -habit_weight * mean, habit_weight**2 * second,
39         'Per unit own-flavor stock S, holding novelty fixed')
```

Read this block:

- Line 25: test `'mean_gamma' in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26, `(mean, sd)`: Unpack a tuple containing 2 displayed entries into these targets in order.

- Line 27, `second`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 28: test `'mixture_weight' in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 29, `weight`: Set this name to the entry/slice selected by `p['mixture_weight']`; brackets select from the object to their left.
- Line 30, `mean2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 31, `second2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 32: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 34, `(mean, second)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 35: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 40–44:

```
40 for term in ["novelty", "own_stock"]:
41     if "mean_" + term in p:
42         mean, sd = p["mean_" + term], p.get("sd_" + term, 0.)
43         add(term, mean, mean**2 + sd**2,
44             "Per unit " + term + ", holding the other history measure fixed")
```

Read this block:

- Line 40: for each entry yielded by `['novelty', 'own_stock']`, bind/unpack it into `term` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 41: test `'mean_' + term in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 42, `(mean, sd)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 43: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 45–45:

```
45 return pd.DataFrame(rows)
```

Read this block:

- Line 45: return the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above. This ends the call.

implied wtp.add

Original function: `implied_wtp.add`. Source begins at line 14.

This nested helper adds one named WTP row from coefficient first and second moments. It uses variables captured from the enclosing WTP call, converts coefficient moments into ratio moments, and appends to the enclosing output list. Its side effect is a new row; it does not refit coefficients.

Definition and stated intent:

Source lines 14–14:

```
14 def add(term, mean, second, interpretation):
```

`def` creates this callable; the parentheses name its inputs (`term`, `mean`, `second`, `interpretation`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 15–18:

```
15 wtp_mean = mean * inverse_mean
16 variance = max(0., second * inverse_second - wtp_mean**2)
17 rows.append(dict(term=term, mean_wtp=wtp_mean, population_sd=np.sqrt(variance),
18                  interpretation=interpretation))
```

Read this block:

- Line 15, `wtp_mean`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 16, `variance`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 17: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

supply_code: source walkthrough

supply_code/comparative_statics/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

There are no statements in this file.

supply_code/comparative_statics/alpha_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import alpha
2 from comparative_statics.canonical import price1_derivative
3 dp1_dalpha = price1_derivative(alpha)
```

Read this block:

- Line 1: from `model.symbols`, import `alpha` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp1_dalpha`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/alpha_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import alpha
2 from comparative_statics.canonical import price2_derivative
3 dp2_dalpha = price2_derivative(alpha)
```

Read this block:

- Line 1: from `model.symbols`, import `alpha` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price2_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp2_dalpha`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/beta_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import beta
2 from comparative_statics.canonical import price1_derivative
3 dp1_dbeta = price1_derivative(beta)
```

Read this block:

- Line 1: from `model.symbols`, import `beta` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp1_dbeta`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/beta_p1gamma.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.gamma_p1beta import dp1dbeta_dgamma_clean
```

Read this block:

- Line 1: from `comparative_statics.gamma_p1beta`, import `dp1dbeta_dgamma_clean` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/beta_p1p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import *
2 from comparative_statics.canonical import price1_derivative, price2_derivative
3 ratio_derivative = (p2*price1_derivative(beta)-p1*price2_derivative(beta))/p2**2
```

Read this block:

- Line 1: from `model.symbols`, import $*$ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative, price2_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `ratio_derivative`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

supply_code/comparative_statics/beta_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import beta
2 from comparative_statics.canonical import price2_derivative
3 dp2_dbeta = price2_derivative(beta)
```

Read this block:

- Line 1: from `model.symbols`, import `beta` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price2_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp2_dbeta`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/beta_profit.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dPi_dbeta
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `dPi_dbeta` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/canonical.py

Functions provide generic derivative rules; module-level assignments call them to define named gamma, beta, curvature, and ratio expressions. The quotient-rule ratio is created once with generic terminal symbols, then `.subs({...})` specializes it to matched-inside, matched-outside, and acquisition histories. This keeps three ratio definitions tied to one algebraic rule instead of three independent hand-written implementations.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 """All derivatives follow one implicit FOC and the terminal envelope."""
2 import sympy as sp
3 from model.symbols import *
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 3: from `model.symbols`, import `*` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 33–38:

```
33 dp1_dgamma = price1_derivative(gamma)
34 dp2_dgamma = price2_derivative(gamma)
35 dVj_dgamma, dV0_dgamma = value2_derivative(gamma,u,-1,p2j), value2_derivative(gamma,v,1,p20)
36 ds_dgamma = sp.factor(sj1*s01*(q+alpha*dp1_dgamma))
37 du_dgamma, dv_dgamma = -u*(1-u)**2, v*(1-v)**2
```

```
38 d2p1_dgamma2 = total_derivative(dp1_dgamma,gamma)
```

Read this block:

- Line 33, `dp1_dgamma`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 34, `dp2_dgamma`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 35, `(dVj_dgamma, dV0_dgamma)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 36, `ds_dgamma`: Set this name to the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above.
- Line 37, `(du_dgamma, dv_dgamma)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 38, `d2p1_dgamma2`: Set this name to the result of calling `total_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.factor`: Factor common algebraic pieces to make a symbolic derivative easier to inspect.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `total_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 53–58:

```
53 d2p2_dgamma2 = terminal_total_derivative(dp2_dgamma,gamma)
54 ds2_dgamma = sp.factor(sj2*s02*(q2+alpha*dp2_dgamma))
55 dp2dbeta_dgamma = terminal_total_derivative(price2_derivative(beta),gamma)
56 dPi_dgamma = profit_derivative(gamma)
57 dPi_dbeta = profit_derivative(beta)
58 ratio_derivative = (p2*dp1_dgamma-p1*dp2_dgamma)/p2**2
```

Read this block:

- Line 53, `d2p2_dgamma2`: Set this name to the result of calling `terminal_total_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 54, `ds2_dgamma`: Set this name to the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above.
- Line 55, `dp2dbeta_dgamma`: Set this name to the result of calling `terminal_total_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 56, `dPi_dgamma`: Set this name to the result of calling `profit_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 57, `dPi_dbeta`: Set this name to the result of calling `profit_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 58, `ratio_derivative`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `terminal_total_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `profit_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 59–61:

```
matched_j = ratio_derivative.subs({q:-1,p2:p2j,sj2:u,q2:-1})
matched_0 = ratio_derivative.subs({q:1,p2:p20,sj2:v,q2:1})
induce_harvest = ratio_derivative.subs({q:1,p2:p2j,sj2:u,q2:-1})
```

Read this block:

- Line 59, `matched_j`: Set this name to the result of calling `ratio_derivative.subs(...)`; the arguments and preceding method chain are shown above.
- Line 60, `matched_0`: Set this name to the result of calling `ratio_derivative.subs(...)`; the arguments and preceding method chain are shown above.
- Line 61, `induce_harvest`: Set this name to the result of calling `ratio_derivative.subs(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.subs`: Substitute symbols/expressions in symbolic algebra; this does not by itself solve an equilibrium.

price2 derivative

Original function: `price2_derivative`. Source begins at line 6.

This returns a **symbolic expression** for the terminal optimal-price derivative with respect to symbolic parameter theta. Default arguments `sj2`, `q2`, and `p2` are shared symbols; callers substitute a particular branch by passing u/-1/p2j or v/+1/p20. `sp.diff(beta, theta)` is one when theta is beta and zero for an independent parameter, so one generic expression covers several comparative statics.

`direct` is the fixed-price derivative of the utility index. The returned formula already incorporates implicit share/price feedback; `simplify` reduces algebra. Differentiating the markup while holding its share fixed would omit that feedback. Unlike numerical finite differences, symbolic derivatives do not need a step size, though they still depend on a correct symbolic model.

Definition and stated intent:

Source lines 6–6:

```
def price2_derivative(theta, share=sj2, history=q2, price=p2):
```

`def` creates this callable; the parentheses name its inputs (`theta`, `share`, `history`, `price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 7–7:

```
direct = sp.diff(beta, theta) + price*sp.diff(alpha, theta) + history*sp.diff(gamma, theta)
```

Read this block:

- Line 7, `direct`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 8–8:

```
return sp.simplify((1-share)*sp.diff(c, theta) + sp.diff(alpha, theta)/alpha**2 - share*direct/alpha)
```

Read this block:

- Line 8: return the result of calling `sp.simplify(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

value2 derivative

Original function: `value2_derivative`. Source begins at line 11.

This computes the symbolic envelope derivative of a reoptimized continuation value. It uses the same fixed-price index derivative as terminal pricing, then returns `-share*direct/alpha - share*diff(c,theta)`. There is no explicit derivative of optimal price because the terminal FOC cancels that first-order effect. Omitting price responses here is justified by the envelope theorem, not a general Python or SymPy rule. Substituting fully expanded optimal prices would be harder algebra with the same result under the maintained optimum conditions.

Definition and stated intent:

Source lines 11–11:

```
11 def value2_derivative(theta, share, history, price):
```

`def` creates this callable; the parentheses name its inputs (`theta`, `share`, `history`, `price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 12–12:

```
12 direct = sp.diff(beta, theta) + price*sp.diff(alpha, theta) + history*sp.diff(gamma, theta)
```

Read this block:

- Line 12, `direct`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 13–13:

```
13 return -share*direct/alpha - share*sp.diff(c, theta)
```

Read this block:

- Line 13: return subtraction of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

price1 derivative

Original function: `price1_derivative`. Source begins at line 16.

This forms the symbolic first-period price derivative. It builds its direct utility effect, obtains the two terminal envelope derivatives, subtracts them, and inserts the result into the implicit first-price formula. `sp.diff(delta, theta)` includes a direct discount-factor change when theta is delta; for gamma that term is zero but the continuation values still change.

The two helper calls use the correct opposite signed terminal histories. Maintaining a single generic function avoids separate alpha/beta/delta/gamma formulas disagreeing about those branches. `sp.simplify` does algebraic cleanup; it does not decide whether a term is positive under the economic domain.

Definition and stated intent:

Source lines 16–16:

```
16 def price1_derivative(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 17–18:

```
17 direct = sp.diff(beta, theta) + p1*sp.diff(alpha, theta) + q*sp.diff(gamma, theta)
18 dDV = value2_derivative(theta,u,-1,p2j)-value2_derivative(theta,v,1,p20)
```

Read this block:

- Line 17, `direct`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 18, `dDV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 19–20:

```
19 return sp.simplify(s01*sp.diff(c,theta)+sp.diff(alpha,theta)/alpha**2
20                  -sj1*direct/alpha-s01*(sp.diff(delta,theta)*(Vj-V0)+delta*dDV))
```

Read this block:

- Line 19: return the result of calling `sp.simplify(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

total derivative

Original function: `total_derivative`. Source begins at line 23.

Input is a symbolic expression and differentiation parameter. Output is its total derivative along the solved equilibrium. The first assignments compute price derivatives, then logistic chain rules compute derivatives of the three shares. A dictionary maps each endogenous symbol to its derivative.

The return is explicit partial differentiation plus `sum(diff(expr,k)*val for k,val in substitutions.items())`. This is the chain rule over every mapped endogenous object. The dictionary is a derivative map, not values actually substituted with `.subs` in this function. `sp.factor` collects common algebraic factors. Plain `sp.diff(expr,theta)` would treat shares, prices, and values as independent symbols and miss their indirect changes.

Definition and stated intent:

Source lines 23–23:

```
23 def total_derivative(expr, theta):
```

`def` creates this callable; the parentheses name its inputs (`expr`, `theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 24–30:

```
24 dp = price1_derivative(theta)
25 dpj, dp0 = price2_derivative(theta,u,-1,p2j), price2_derivative(theta,v,1,p20)
26 ds = sj1*s01*(sp.diff(beta,theta)+p1*sp.diff(alpha,theta)+q*sp.diff(gamma,theta)+alpha*dp)
27 du = u*(1-u)*(sp.diff(beta,theta)+p2j*sp.diff(alpha,theta)-sp.diff(gamma,theta)+alpha*dpj)
28 dv = v*(1-v)*(sp.diff(beta,theta)+p20*sp.diff(alpha,theta)+sp.diff(gamma,theta)+alpha*dp0)
29 substitutions = {p1:dp, p2j:dpj, p20:dp0, sj1:ds, u:du, v:dv,
30                  Vj:value2_derivative(theta,u,-1,p2j), V0:value2_derivative(theta,v,1,p20)}
```

Read this block:

- Line 24, `dp`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 25, `(dpj, dp0)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 26, `ds`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 27, `du`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 28, `dv`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 29, `substitutions`: Set this name to a dictionary from the displayed keys and values.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 31–31:

```
31 return sp.factor(sp.diff(expr,theta)+sum(sp.diff(expr,k)*val for k,val in substitutions.items()))
```

Read this block:

- Line 31: return the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.factor`: Factor common algebraic pieces to make a symbolic derivative easier to inspect.
- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

terminal total derivative

Original function: `terminal_total_derivative`. Source begins at line 39.

This is the smaller one-branch version of the total derivative. It calculates terminal price response, computes terminal share response including repricing, and adds explicit theta, price, and share chain-rule terms. Because there is only one terminal branch, a short direct expression is clearer than building the full period-1 derivative dictionary. It is used for terminal curvature and mixed derivatives; a second plain symbolic partial derivative would hold the share fixed and give the wrong comparative static.

Definition and stated intent:

Source lines 39–39:

```
39 def terminal_total_derivative(expr,theta):
```

`def` creates this callable; the parentheses name its inputs (`expr`, `theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 40–41:

```
40 dp = price2_derivative(theta)
41 ds = sj2*s02*(sp.diff(beta,theta)+p2*sp.diff(alpha,theta)+q2*sp.diff(gamma,theta)+alpha*dp)
```

Read this block:

- Line 40, `dp`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 41, `ds`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 42–42:

```
42 return sp.factor(sp.diff(expr,theta)+sp.diff(expr,p2)*dp+sp.diff(expr,sj2)*ds)
```

Read this block:

- Line 42: return the result of calling `sp.factor(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.factor`: Factor common algebraic pieces to make a symbolic derivative easier to inspect.

profit derivative

Original function: `profit_derivative`. Source begins at line 45.

This returns the optimized two-period profit derivative with respect to theta. `partial` combines direct current demand/cost effects, direct discounting, and both branch envelope effects. `.subs(p1, markup_expression)` inserts the first-period optimal pricing equation, then `simplify` reduces the result.

The code does not explicitly multiply by dp1/dtheta because the first-period FOC makes that contribution zero at the optimum. Replacing the entire objective with current revenue before differentiating would lose continuation effects. This helper is reusable for gamma and beta, while relying on the same fixed-type, two-branch, interior optimum as the rest of the model.

Definition and stated intent:

Source lines 45–45:

```
45 def profit_derivative(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 46–49:

```
46 direct = sp.diff(beta,theta)+p1*sp.diff(alpha,theta)+q*sp.diff(gamma,theta)
47 partial = (direct*sj1*s01*(p1-c+delta*(Vj-V0))-sj1*sp.diff(c,theta)
48            +sp.diff(delta,theta)*(sj1*Vj+s01*V0)
49            +delta*(sj1*value2_derivative(theta,u,-1,p2j)+s01*value2_derivative(theta,v,1,p20)))
```

Read this block:

- Line 46, `direct`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 47, `partial`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.
- `value2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 50–50:

```
50 return sp.simplify(partial.subs(p1,c-1/(alpha*s01)-delta*(Vj-V0)))
```

Read this block:

- Line 50: return the result of calling `sp.simplify(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.
- `.subs`: Substitute symbols/expressions in symbolic algebra; this does not by itself solve an equilibrium.

supply_code/comparative_statics/delta_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import delta
2 from comparative_statics.canonical import price1_derivative
3 dp1_ddelta = price1_derivative(delta)
```

Read this block:

- Line 1: from `model.symbols`, import `delta` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp1_ddelta`: Set this name to the result of calling `price1_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/delta_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 from model.symbols import delta
2 from comparative_statics.canonical import price2_derivative
3 dp2_ddelta = price2_derivative(delta)
```

Read this block:

- Line 1: from `model.symbols`, import `delta` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price2_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp2_ddelta`: Set this name to the result of calling `price2_derivative(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `price2_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/gamma2_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import d2p1_dgamma2
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `d2p1_dgamma2` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/gamma2_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import d2p2_dgamma2
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `d2p2_dgamma2` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/gamma_p1.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dp1_dgamma, ds_dgamma as dsj1_dgamma_total
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `dp1_dgamma, ds_dgamma as dsj1_dgamma_total` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/gamma_p1beta.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–4:

```
1 from model.symbols import beta, gamma
2 from comparative_statics.canonical import price1_derivative, total_derivative
3 dp1dbeta_dgamma_raw = total_derivative(price1_derivative(beta), gamma)
4 dp1dbeta_dgamma_clean = dp1dbeta_dgamma_raw
```

Read this block:

- Line 1: from `model.symbols`, import `beta, gamma` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2: from `comparative_statics.canonical`, import `price1_derivative, total_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `dp1dbeta_dgamma_raw`: Set this name to the result of calling `total_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 4, `dp1dbeta_dgamma_clean`: Set this name to the object currently named `dp1dbeta_dgamma_raw`; plain assignment does not copy it.

Syntax and calls used here:

- `total_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `price1_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

supply_code/comparative_statics/gamma_p1p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import ratio_derivative, matched_j, matched_0, induce_harvest
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `ratio_derivative, matched_j, matched_0, induce_harvest` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/gamma_p2.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dp2_dgamma, ds2_dgamma as dsj2_dgamma_total
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `dp2_dgamma, ds2_dgamma as dsj2_dgamma_total` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/gamma_p2beta.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dp2dbeta_dgamma
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `dp2dbeta_dgamma` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics/gamma_profit.py

This small module gives an existing canonical derivative a convenient public name, or combines canonical derivatives into the displayed expression. It does not fit a model. The literal imports and assignment below show whether it calls a generic helper or aliases an already derived expression. Reusing the canonical implementation avoids maintaining separate competing versions of the same comparative static.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 from comparative_statics.canonical import dPi_dgamma
```

Read this block:

- Line 1: from `comparative_statics.canonical`, import `dPi_dgamma` into this module's namespace. Importing can execute the imported module's top-level setup.

supply_code/comparative_statics_symbolic.py

This convenience script imports canonical expressions and, only when run directly, iterates over `(label, expression)` pairs and prints each. Tuple unpacking connects a human-readable label to its actual symbolic object. It does not compute independent derivative formulas. Reusing named expressions keeps terminal display consistent with production numeric evaluation.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Print verified canonical gamma derivatives."""
2 from comparative_statics.canonical import *
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `comparative_statics.canonical`, import $*$ into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 3–6:

```
3 if __name__ == "__main__":
4     for label,expr in [("dp1/dgamma",dp1_dgamma),("d2p1/dgamma2",d2p1_dgamma2),
5                        ("dprofit/dgamma",dPi_dgamma),("ratio derivative",ratio_derivative)]:
6         print(label, "=", expr)
```

Read this block:

- Line 3: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 4: for each entry yielded by `[('dp1/dgamma', dp1_dgamma), ('d2p1/dgamma2', d2p1_dgamma2), (...,` bind/unpack it into `(label, expr)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 6: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

supply_code/export/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

There are no statements in this file.

supply_code/export/latex_export.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 """Export the model's actual SymPy expressions as display equations."""
2 import sympy as sp
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `sympy as sp`. An as-name is a convenient local alias, not another library.

display equation

Original function: `display_equation`. Source begins at line 5.

This returns a LaTeX display-equation string. `lhs` is already a LaTeX label; `sp.latex(expression)` serializes the actual symbolic right-hand side. `'\\[\n'` contains a backslash followed by a bracket and a newline; Python string escapes are why source contains doubled backslashes. Concatenation places the equation between display delimiters. Deriving the right side from SymPy avoids manually copying a formula that later changes in the code. The function returns text and does not itself invoke a TeX compiler.

Definition and stated intent:

Source lines 5–6:

```
5 def display_equation(lhs, expression):
6     """Keep the mathematical right-hand side sourced directly from SymPy."""
```

`def` creates this callable; the parentheses name its inputs (`lhs`, `expression`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 7–7:

```
7 return '\\[\n' + lhs + ' = ' + sp.latex(expression) + '\n\\]\n'
```

Read this block:

- Line 7: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.latex`: Serialize symbolic algebra as LaTeX source rather than recomputing the mathematical formula manually.

supply_code/export/pretty_print.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–1:

```
1 import sympy as sp
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.

show

Original function: `show`. Source begins at line 4.

This is a terminal-output helper returning None. Blank `print()` creates a new line; `"=" * 70` repeats a string seventy times, whereas multiplication of numeric values has a different meaning. `sp.pprint` renders an expression in readable mathematical form in the terminal. It is useful for inspection, while `sp.latex` is the appropriate alternative for document source.

Definition and stated intent:

Source lines 4–4:

```
4 def show(title, expr):
```

`def` creates this callable; the parentheses name its inputs (`title`, `expr`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 5–9:

```
5 print()
6 print("=" * 70)
7 print(title)
8 print("=" * 70)
9 sp.pprint(expr)
```

Read this block:

- Line 5: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 6: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 7: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 8: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 9: call `sp.pprint` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.pprint`: Pretty-print symbolic mathematics to the terminal for human inspection.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

supply__code/make__comparative__static__figures.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Generate only the requested symmetric-switching comparative-static figures."""
from pathlib import Path
import csv
import argparse
import numpy as np
import matplotlib
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `csv`. An as-name is a convenient local alias, not another library.
- Line 4: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 5: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 6: import `matplotlib`. An as-name is a convenient local alias, not another library.

Source lines 7–13:

```
matplotlib.use('Agg')
import matplotlib.pyplot as plt
PARAMETERS = dict(beta=2., alpha=-1., delta=.92, c=0.)

ROOT=Path(__file__).resolve().parent
OUT=ROOT/'writeup_math_graphs'/'figures'
GAMMAS=np.linspace(-3,3,601)
```

Read this block:

- Line 7: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 8: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 9, `PARAMETERS`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 11, `ROOT`: Set this name to the attribute `Path(__file__).resolve().parent` of the object before the dot.
- Line 12, `OUT`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 13, `GAMMAS`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 14–15:

```
14 LABELS={'j':r'Initially inside ($x_0=j$)','0':r'Initially outside ($x_0=0$)'}
15 STATES=[('0','0'),('0','j'),('j','j'),('j','0')]
```

Read this block:

- Line 14, `LABELS`: Set this name to a dictionary with named entries `j`, `0`.
- Line 15, `STATES`: Set this name to a list containing 4 displayed entries.

Source lines 156–157:

```
156 if __name__=='__main__':
157     main()
```

Read this block:

- Line 156: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 157: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

state outcomes

Original function: `state_outcomes`. Source begins at line 18.

Conditions on each initial/first-choice history, retains terminal demand expectation, and produces histories with their path weights. This separates conditional economic mechanisms from ex ante comparisons.

Definition and stated intent:

Source lines 18–19:

```
18 def state_outcomes(curves):
19     """Condition on (x0, x1); terminal demand remains integrated out."""
```

`def` creates this callable; the parentheses name its inputs (`curves`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 20–20:

```
20 rows=[]
```

Read this block:

- Line 20, `rows`: Set this name to the list `[]`.

Source lines 21–35:

```
21 for initial, realized in STATES:
22     for r in curves[initial]:
23         p2=r['p2j' if realized=='j' else 'p20']
24         dp2=r['dp2j' if realized=='j' else 'dp20']
25         value=r['V2j' if realized=='j' else 'V20']
26         rows.append(dict(
27             gamma=r['gamma'], x0=initial, x1=realized,
28             probability=r['s'] if realized=='j' else 1-r['s'],
29             share1=r['s'], outside_share1=1-r['s'],
30             share2=r['u' if realized=='j' else 'v'],
31             outside_share2=1-r['u' if realized=='j' else 'v'],
32             price_ratio=r['p1']/p2,
33             price_ratio_derivative=(r['dp1']*p2-r['p1']*dp2)/p2**2,
```

```
34            realized_profit=(r['p1']-PARAMETERS['c'])*(realized=='j')
35                            +PARAMETERS['delta']*value))
```

Read this block:

- Line 21: for each entry yielded by `STATES`, bind/unpack it into `(initial, realized)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 22: for each entry yielded by `curves[initial]`, bind/unpack it into `r` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 23, `p2`: Set this name to the entry/slice selected by `r['p2j' if realized == 'j' else 'p20']`; brackets select from the object to their left.
- Line 24, `dp2`: Set this name to the entry/slice selected by `r['dp2j' if realized == 'j' else 'dp20']`; brackets select from the object to their left.
- Line 25, `value`: Set this name to the entry/slice selected by `r['V2j' if realized == 'j' else 'V20']`; brackets select from the object to their left.
- Line 26: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 36–36:

```
36 return rows
```

Read this block:

- Line 36: return the object currently named `rows`; plain assignment does not copy it. This ends the call.

plot state outcomes

Original function: `plot_state_outcomes`. Source begins at line 39.

Plots conditional prices, ratios or profits across gamma for the history-specific outcomes. Histories that are rare under a policy should not be confused with an equally weighted population average.

Definition and stated intent:

Source lines 39–39:

```
39 def plot_state_outcomes(rows):
```

`def` creates this callable; the parentheses name its inputs (`rows`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 40–45:

```
40 specs=[('all_state_price_ratios','price_ratio',r'$p_1^*(x_0)/p_2^*(x_1)$',
41         'Price ratios by realized history'),
42        ('all_state_price_ratio_derivatives','price_ratio_derivative',r'$dR_{x_0,x_1}/d\gamma$',
43         'Price-ratio derivatives by realized history'),
44        ('all_state_realized_profit','realized_profit',r'$\Pi^*(x_0,x_1)$',
45         'Profit conditional on realized first-period choice')]
```

Read this block:

- Line 40, `specs`: Set this name to a list containing 3 displayed entries.

Source lines 46–59:

```
for name,key,ylabel,title in specs:
    fig,ax=plt.subplots(figsize=(8,5))
    for (initial,realized),style in zip(STATES,['-','--','-.',':']):
        group=[r for r in rows if (r['x0'],r['x1'])==(initial,realized)]
        ax.plot([r['gamma'] for r in group],[r[key] for r in group],
                ls=style,lw=2,label=rf'$(x_0,x_1)=({initial},{realized})$')
    ax.axhline(0,color='gray',lw=.7)
    ax.set(xlabel=r'$\gamma$',ylabel=ylabel,title=title)
    ax.legend();ax.grid(alpha=.2)
    if key=='realized_profit':
        fig.text(.5,.01,'First-period margin if purchased + discounted expected terminal profit',
                 ha='center',fontsize=9)
        fig.tight_layout(rect=(0,.04,1,1))
    save(fig,name)
```

Read this block:

- Line 46: for each entry yielded by `specs`, bind/unpack it into `(name, key, ylabel, title)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 47, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 48: for each entry yielded by `zip(STATES, ['-', '--', '-.', ':'])`, bind/unpack it into `((initial, realized), style)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 49, `group`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 50: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 53: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 54: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 54: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 55: test `key == 'realized_profit'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 56: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 58: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 59: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.grid`: Add visual grid lines without changing underlying numerical results.
- `save`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.text`: Place a text label on the axes/Figure at the supplied coordinates.

- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

plot state shares

Original function: `plot_state_shares`. Source begins at line 62.

Plots how policy-induced probabilities allocate households across realized histories. These probability weights explain why an ex ante profit comparison can differ from comparing conditional histories alone.

Definition and stated intent:

Source lines 62–62:

```
62 def plot_state_shares(rows):
```

`def` creates this callable; the parentheses name its inputs (`rows`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 63–63:

```
63 fig,axes=plt.subplots(2,2,figsize=(11,8),sharex=True,sharey=True)
```

Read this block:

- Line 63, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 64–74:

```
64 for ax,key,title in zip(axes.flat,
65         ['share1','share2','outside_share1','outside_share2'],
66         ['Period 1: inside share','Period 2: inside share',
67          'Period 1: outside share','Period 2: outside share']):
68     for (initial,realized),style in zip(STATES,['-','--','-.',':']):
69         group=[r for r in rows if (r['x0'],r['x1'])==(initial,realized)]
70         ax.plot([r['gamma'] for r in group],[r[key] for r in group],
71                 ls=style,lw=2,label=rf'$({initial},{realized})$')
72     ax.set(xlabel=r'$\gamma$',ylabel='Share',title=title,ylim=(0,1))
73     ax.grid(alpha=.2)
74     ax.legend(title=r'$(x_0,x_1)$',fontsize=8,ncol=2)
```

Read this block:

- Line 64: for each entry yielded by `zip(axes.flat, ['share1', 'share2', 'outside_share1', 'outside...`, bind/unpack it into `(ax, key, title)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 68: for each entry yielded by `zip(STATES, ['-', '--', '-.', ':'])`, bind/unpack it into `((initial, realized), style)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 69, `group`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 70: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 72: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 73: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 74: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.grid`: Add visual grid lines without changing underlying numerical results.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 75–75:

```
75 save(fig,'all_state_shares')
```

Read this block:

- Line 75: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `save`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

save

Original function: `save`. Source begins at line 78.

This formatting/output helper takes a Matplotlib Figure and filename stem. `tight_layout` adjusts spacing; a two-element suffix loop saves both PDF and PNG; `plt.close` frees the Figure. `OUT / f"{name}.{suffix}"` constructs each Path. Shared export code prevents one figure accidentally lacking its vector PDF version. DPI matters chiefly for raster PNG content; it does not make an ordinary vector line in a PDF more mathematically precise.

Definition and stated intent:

Source lines 78–78:

```
78 def save(fig,name):
```

`def` creates this callable; the parentheses name its inputs (`fig`, `name`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 79–79:

```
79 fig.tight_layout()
```

Read this block:

- Line 79: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 80–81:

```
80 for suffix in ['pdf','png']:
81     fig.savefig(OUT/f'{name}.{suffix}',dpi=160)
```

Read this block:

- Line 80: for each entry yielded by `['pdf', 'png']`, bind/unpack it into `suffix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 81: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 82–82:

```
82 plt.close(fig)
```

Read this block:

- Line 82: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 85.

This generates all benchmark rows from the production `evaluate` function, writes a solution CSV, and plots six requested figures. The dictionary comprehension creates a list of gamma evaluations for each initial-state key. `csv.DictWriter` uses dictionary keys as field names, so output records retain branch meanings. Plot `specs` is a data list describing several similar figures; the loop executes the common plotting logic.

Separate blocks handle acquisition ratios and the two-panel price-level figure. Terminal prices can be read from either initial-state collection because their realized history determines them. A final generator computes the largest absolute FOC residual and raises if it exceeds tolerance. Plotting algebra typed anew here would risk drifting from the derivative expressions; numerical values instead come from the common symbolic-to-numerical path.

Definition and stated intent:

Source lines 85–85:

```
85 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 86–90:

```
86 parser=argparse.ArgumentParser(description=__doc__)
87 parser.add_argument('--from-solutions',action='store_true',
88                     help='Use the saved benchmark solutions instead of solving again.')
```

```
89 args=parser.parse_args()
90 OUT.mkdir(parents=True,exist_ok=True)
```

Read this block:

- Line 86, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 87: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 89, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.
- Line 90: call `OUT.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 91–100:

```
91 if args.from_solutions:
92     with (OUT/'solutions.csv').open(newline='') as file:
93         saved=[{k:v if k=='initial_state' else float(v) for k,v in row.items()}
94                for row in csv.DictReader(file)]
95     curves={state:[r for r in saved if r['initial_state']==state] for state in ['j','0']}
96     for group in curves.values():
97         np.testing.assert_allclose([r['gamma'] for r in group],GAMMAS,atol=1e-12)
98 else:
99     from model.numerical import evaluate
100    curves={state:[evaluate(float(g),state,**PARAMETERS) for g in GAMMAS] for state in ['j','0']}
```

Read this block:

- Line 91: test `args.from_solutions`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 92: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 93, `saved`: Set this name to a list computed by the displayed for-expression.
- Line 95, `curves`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 96: for each entry yielded by `curves.values()`, bind/unpack it into `group` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 97: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99: from `model.numerical`, import `evaluate` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 100, `curves`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.

Syntax and calls used here:

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 101–101:

```
101 outcomes=state_outcomes(curves)
```

Read this block:

- Line 101, `outcomes`: Set this name to the result of calling `state_outcomes(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `state_outcomes`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 102–105:

```
102 with (OUT/'all_state_outcomes.csv').open('w',newline='') as file:
103     writer=csv.DictWriter(file,fieldnames=list(outcomes[0]))
104     writer.writeheader()
105     writer.writerows(outcomes)
```

Read this block:

- Line 102: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 103, `writer`: Set this name to the result of calling `csv.DictWriter(...)`; the arguments and preceding method chain are shown above.
- Line 104: call `writer.writeheader` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105: call `writer.writerows` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.DictWriter`: Create a CSV writer mapping dictionary keys to named columns in an explicit field order.
- `.writeheader`: Write the CSV's named column header before writing data rows.
- `.writerows`: Write a sequence of dictionaries/rows using the configured CSV schema.
- `list`: Materialize iterable entries as an ordered Python list.

Source lines 107–111:

```
107 for initial in ['0','j']:
108     groups={a:[r for r in outcomes if r['x0']==initial and r['x1']==a] for a in ['0','j']}
109     expected=[sum(groups[a][i]['probability']*groups[a][i]['realized_profit']
110                   for a in ['0','j']) for i in range(len(GAMMAS))]
111     np.testing.assert_allclose(expected,[r['profit'] for r in curves[initial]],atol=1e-12)
```

Read this block:

- Line 107: for each entry yielded by `['0', 'j']`, bind/unpack it into `initial` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 108, `groups`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 109, `expected`: Set this name to a list computed by the displayed for-expression.
- Line 111: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `len`: Count elements/rows rather than summing their numerical values.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 112–112:

```
112 plot_state_outcomes(outcomes)
```

Read this block:

- Line 112: call `plot_state_outcomes` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `plot_state_outcomes`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 113–116:

```
113 for r in outcomes:
114     for period in [1,2]:
115         assert 0<=r[f'share{period}']<=1
116         np.testing.assert_allclose(r[f'share{period}']+r[f'outside_share{period}'],1)
```

Read this block:

- Line 113: for each entry yielded by `outcomes`, bind/unpack it into `r` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 114: for each entry yielded by `[1, 2]`, bind/unpack it into `period` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 115: require `0 <= r[f'share{period}'] <= 1` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 116: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 117–117:

```
117 plot_state_shares(outcomes)
```

Read this block:

- Line 117: call `plot_state_shares` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `plot_state_shares`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 118–122:

```
118 with (OUT/'solutions.csv').open('w',newline='') as file:
119     writer=csv.DictWriter(file,fieldnames=list(curves['j'][0]))
120     writer.writeheader()
121     for state in ['j','0']:
122         writer.writerows(curves[state])
```

Read this block:

- Line 118: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 119, `writer`: Set this name to the result of calling `csv.DictWriter(...)`; the arguments and preceding method chain are shown above.
- Line 120: call `writer.writeheader` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 121: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 122: call `writer.writerows` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 123–126:

```
123 specs=[('profit_by_gamma','profit',r'$\Pi^*$','Optimized two-period profit'),
124        ('gamma_profit_by_gamma','dprofit',r'$d\Pi^*/d\gamma$','Optimized profit derivative'),
125        ('gamma_p1_by_gamma','dp1',r'$dp_1^*/d\gamma$','Optimal period-1 price derivative'),
126        ('matched_price_ratio_derivative','d_matched_ratio',r'$dR_a/d\gamma$','State-matched price-ratio
    ↪ derivatives')]
```

Read this block:

- Line 123, `specs`: Set this name to a list containing 4 displayed entries.

Source lines 127–134:

```
127 for name,key,ylabel,title in specs:
128     fig,ax=plt.subplots(figsize=(7.2,4.5))
129     for state in ['j','0']:
130         ax.plot(GAMMAS,[r[key] for r in curves[state]],label=LABELS[state],lw=2)
131     ax.axhline(0,color='gray',lw=.7)
132     ax.set(xlabel=r'$\gamma$',ylabel=ylabel,title=title)
133     ax.legend();ax.grid(alpha=.2)
134     save(fig,name)
```

Read this block:

- Line 127: for each entry yielded by `specs`, bind/unpack it into `(name, key, ylabel, title)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 128, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 129: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 130: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 131: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 132: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 134: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.grid`: Add visual grid lines without changing underlying numerical results.
- `save`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.

Source lines 135–140:

```
135 fig,ax=plt.subplots(figsize=(7.2,4.5))
136 ax.plot(GAMMAS,[r['d_induce_ratio'] for r in curves['0']],lw=2,
137         label=r'$R_{0\to j}=p_1^*(0)/p_2^*(j)$')
138 ax.axhline(0,color='gray',lw=.7)
139 ax.set(xlabel=r'$\gamma$',ylabel=r'$dR_{0\to j}/d\gamma$',title='Induce-then-harvest price-ratio derivative')
140 ax.legend();ax.grid(alpha=.2);save(fig,'induce_harvest_price_ratio_derivative')
```

Read this block:

- Line 135, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 136: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 138: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 139: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 140: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 140: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 140–141:

```
140 ax.legend();ax.grid(alpha=.2);save(fig,'induce_harvest_price_ratio_derivative')
141 fig,axes=plt.subplots(1,2,figsize=(10,4.2))
```

Read this block:

- Line 140: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 141, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 142–143:

```
142 for state in ['j','0']:
143     axes[0].plot(GAMMAS,[r['p1'] for r in curves[state]],label=LABELS[state],lw=2)
```

Read this block:

- Line 142: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 143: call `axes[0].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 144–145:

```
144 for state,key,label in [('j','p2j',r'Previous inside ($x_1=j$)'),('0','p20',r'Previous outside ($x_1=0$)')]:
145     axes[1].plot(GAMMAS,[r[key] for r in curves['j']],label=label,lw=2)
```

Read this block:

- Line 144: for each entry yielded by `[('j', 'p2j', 'Previous inside ($x_1=j$)'), ('0', 'p20', 'Prev...`, bind/unpack it into `(state, key, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 145: call `axes[1].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 146–148:

```
146 for ax,title,ylabel in zip(axes,['Optimal period-1 prices','Optimal period-2 prices'],[r'$p_1^*$',r'$p_2^*$']):
147     ax.axhline(0,color='gray',lw=.7);ax.set(xlabel=r'$\gamma$',ylabel=ylabel,title=title)
148     ax.legend(fontsize=8);ax.grid(alpha=.2)
```

Read this block:

- Line 146: for each entry yielded by `zip(axes, ['Optimal period-1 prices', 'Optimal period-2 prices...`, bind/unpack it into `(ax, title, ylabel)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 147: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 147: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 148: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 148: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 149–150:

```
149 save(fig,'optimal_prices_by_gamma')
150 maximum=max(abs(r[k]) for state in curves for r in curves[state] for k in ['foc1','foc2j','foc20'])
```

Read this block:

- Line 149: call `save` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 150, `maximum`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 151–152:

```
151 if maximum>1e-9:
152     raise AssertionError(f'FOC residual {maximum}')
```

Read this block:

- Line 151: test `maximum > 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 152: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 153–153:

```
153 print(f'Saved ten figures, 1,202 solved observations, and {len(outcomes):,} conditional outcomes; max FOC
    ↪  residual={maximum:.3g}')
```

Read this block:

- Line 153: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

supply_code/make_latex_document.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Generate a verified symmetric-switching appendix with six selected figures."""
2 from pathlib import Path
3 import sympy as sp
4 from model.symbols import *
5 from model.utility import U1,U2
6 from model.profit import Pi
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 4: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: from `model.utility`, import `U1, U2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: from `model.profit`, import `Pi` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–12:

```
7 from pricing.period1_foc import foc1_collected
8 from pricing.period2_foc import foc2
9 from pricing.period1_price import p1_star
10 from pricing.period2_price import p2_star
11 from comparative_statics import canonical as C
12 from model.numerical import evaluate
```

Read this block:

- Line 7: from `pricing.period1_foc`, import `foc1_collected` into this module's namespace. Importing can execute the imported module's top-level setup.

- Line 8: from `pricing.period2_foc`, import `foc2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 9: from `pricing.period1_price`, import `p1_star` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: from `pricing.period2_price`, import `p2_star` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 11: from `comparative_statics`, import `canonical as C` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: from `model.numerical`, import `evaluate` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 13–15:

```
13 from export.latex_export import display_equation
14
15 ROOT=Path(__file__).resolve().parent/'writeup_math_graphs'
```

Read this block:

- Line 13: from `export.latex_export`, import `display_equation` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 15, `ROOT`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 276–277:

```
276 if __name__=='__main__':
277     main()
```

Read this block:

- Line 276: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 277: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

main

Original function: `main`. Source begins at line 18.

This writes a research appendix as text. The opening loop pairs LaTeX left-hand side strings with actual SymPy expressions and delegates equation formatting. The very long raw triple-quoted `document` string is document **data** inside the Python program: its LaTeX commands are interpreted later by a TeX engine, not executed as Python statements. A raw string preserves the many backslashes.

Later loops append benchmark rows and figure sections. An f-string inserts computed numbers with three decimals; adjacent raw-string fragments preserve LaTeX row-ending backslashes. The figure-existence check stops on missing assets instead of generating a broken paper. `write_text` creates `.tex`, not a PDF; compilation is a separate step. Repeated string `+=` is acceptable for this small document; accumulating a list and `"".join(parts)` would scale better for very large text assembly.

Definition and stated intent:

Source lines 18–18:

```
18 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 19–21:

```
19 ROOT.mkdir(exist_ok=True)
20 equations='\\section{Symbolically derived equations}\n'
21 equations += r'Here $q_2=q(x_1)$, $s_2=s_2(x_1)$ and $p_2=p_2(x_1)$ describe a fixed terminal history. Price
    ↪  equations and comparative-static right-hand sides use equilibrium shares; the profit and FOCs use shares at the
    ↪  candidate prices.'+'\n'
```

Read this block:

- Line 19: call `ROOT.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 20, `equations`: Set this name to the literal text `'\\section{Symbolically derived equations}\n'`.
- Line 21: update `equations` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 22–28:

```
22 for lhs,expression in [(r'v_1',U1),(r'v_2',U2),(r'\Pi',Pi),
23                        (r'F_1',foc1_collected),(r'F_2',foc2),
24                        (r'p_1^*',p1_star),(r'p_2^*',p2_star),
25                        (r'\frac{dp_1^*}{d\gamma}',C.dp1_dgamma),
26                        (r'\frac{dp_2^*}{d\gamma}',C.dp2_dgamma),
27                        (r'\frac{d\Pi^*}{d\gamma}',C.dPi_dgamma)]:
28     equations+=display_equation(lhs,expression)
```

Read this block:

- Line 22: for each entry yielded by `[('v_1', U1), ('v_2', U2), ('\\Pi', Pi), ('F_1', foc1_collecte...`, bind/unpack it into `(lhs, expression)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 28: update `equations` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `display_equation`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 29–29:

```
29 (ROOT/'sympy_equations.tex').write_text(equations)
```

Read this block:

- Line 29: call `(ROOT / 'sympy_equations.tex').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.write_text`: Write a string into a text file, replacing an existing file at that path.

Source lines 30–71:

```
    document=r'''\documentclass[11pt]{article}
\usepackage[margin=1in]{geometry}
\usepackage{amsmath,amssymb,amsthm,graphicx,booktabs,array}
\usepackage[hidelinks]{hyperref}
\newtheorem{proposition}{Proposition}
\newcommand{\dd}{\mathrm{d}}
\setlength{\parindent}{0pt}
\setlength{\parskip}{5pt}
\title{Two-period pricing with symmetric switching preferences}
\author{}\date{October 30, 2026}
\begin{document}\maketitle
\section{Model, histories, and normalization}
One firm sells one inside product $j$ against outside option $0$ in periods
$t=1,2$. Type $r$ is fixed and observed by the firm, which prices separately for
that type. All formulas below are conditional on $r$; its subscript and the
fixed product index are suppressed. There is no pooling of heterogeneous types.
The initial realized choice is $x_0\in\{j,0\}$. The second-period price can depend
on the realized first-period choice $x_1$. Independent Type-I extreme-value
shocks enter both alternatives in each period.

Before normalization the deterministic utilities are
\[
 V_{jt}^{\rm raw}=\beta+\alpha p_t+\gamma\mathbf1\{x_{t-1}=0\},
 \qquad V_{0t}^{\rm raw}=\gamma\mathbf1\{x_{t-1}=j\}.
\]
The payoff to switching applies in either direction. Subtracting the raw outside
utility from both alternatives gives normalized outside utility zero and
\[
 v_t(h)=\beta+\alpha p_t(h)+q(h)\gamma,\qquad
 q(j)=-1,\quad q(0)=+1.
\]
Normalization retains the outside switching payoff through the signed inside
utility difference; it does not delete that payoff. Negative $\gamma$ denotes
switching aversion and positive $\gamma$ denotes novelty preference.
\[
 s_t(h)=\frac{e^{v_t(h)}}{1+e^{v_t(h)}},\quad s_{0t}(h)=1-s_t(h),\quad
 \partial_p s_t=\alpha s_t(1-s_t),\quad
 (\partial_\gamma s_t)_p=q(h)s_t(1-s_t).
\]
A realized state $x_t$, a hypothetical switching indicator, and the probability
$s_t$ are distinct objects. Every derivative below holds the discrete history fixed.
```

Source lines 72–113:

```
\textbf{Assumptions.} $\alpha<0$, $0<\delta<1$, fixed marginal cost $c$, finite
parameters, and interior binary-logit shares. Prices are unrestricted real
numbers: introductory subsidies are allowed. Continuation values are conditional
on realized states and have no direct dependence on $p_1$. Type does not evolve.
The case $\delta=0$ is used only as a static counterfactual.

\begin{center}\begin{tabular}{ll}\toprule
Notation & Meaning\\\midrule
$q=q(x_0)$ & Signed initial-state coefficient\\
$s=s_1^*(x_0),\quad r_0=1-s$ & First-period equilibrium shares\\
$u=s_2^*(j),\quad v=s_2^*(0)$ & Inside shares in the two continuation states\\
$V_j=V_2(j),\quad V_0=V_2(0)$ & Optimized continuation profits\\
$\Delta V=V_j-V_0,\quad B=u+v$ & Continuation difference and sum of terminal shares\\
$\Pi^*$ & Expected discounted profit per initial consumer\\\bottomrule
\end{tabular}\end{center}

\section{Backward induction and optimality}
For each realized first-period choice $a\in\{j,0\}$, solve
```

```
\[
 V_2(a)=\max_{p_2(a)}s_2(a)[p_2(a)-c].
\]
Its first-order condition is
\[
 0=s_2+\alpha s_2(1-s_2)(p_2-c),\qquad
 p_2^*(a)=c-\frac1{\alpha[1-s_2^*(a)]}.
\]
This is an implicit equation: the share depends on the same price being solved.
Both $V_j$ and $V_0$ must be computed before period 1.

First-period profit integrates over both possible realized histories:
\begin{align*}
 \Pi(p_1;x_0)&=s_1(p_1-c)+\delta[s_1V_j+(1-s_1)V_0]\\
 &=\delta V_0+s_1[p_1-c+\delta\Delta V].
\end{align*}
Because the branch values do not depend directly on $p_1$,
\[
 \partial_{p_1}\{\delta[s_1V_j+(1-s_1)V_0]\}
 =\delta\alpha s_1(1-s_1)(V_j-V_0).
\]
Consequently,
\[
 0=s_1+\alpha s_1(1-s_1)[p_1-c+\delta\Delta V],\qquad
```

Source lines 114–155:

```
 p_1^*=c-\frac1{\alpha(1-s)}-\delta\Delta V.
\]
The static FOC component $c-1/[\alpha(1-s)]$ uses the share at the dynamic
optimum. It is not the counterfactual static optimum, which must be solved
separately with $\delta=0$.

\begin{proposition}[Unique optimal prices]
Under the stated assumptions each continuation price and the first-period price
has a unique finite global optimum, with no lower bound imposed on $p_1$.
\end{proposition}
\begin{proof}
Both price problems reduce, up to a constant, to maximizing
$s(p)(p-k)$ with $s(p)=\operatorname{logit}^{-1}(b+\alpha p)$.
Use $k=c$ in period 2 and $k=c-\delta\Delta V$ in period 1.
The sign of the profit derivative is the sign of
\[
 H(p)=1+e^{b+\alpha p}+\alpha(p-k),\quad
 H'(p)=\alpha(1+e^{b+\alpha p})<0.
\]
$H$ ranges from positive infinity to negative infinity, so its root is unique
and the objective increases before it and decreases afterward. The price may
lie below physical cost $c$ in period 1 when continuation incentives justify it.
\end{proof}

\section{Envelope derivatives and conditional comparative statics}
Differentiating each continuation value using the envelope theorem gives
\[
 V_2'(a)=q(a)s_2(1-s_2)(p_2^*-c)=-\frac{q(a)s_2}{\alpha}.
\]
Implicit differentiation of the terminal FOC gives the same expression for the
optimal price derivative:
\[
 p_2^{*\prime}(a)=-\frac{q(a)s_2}{\alpha},\quad
 V_j'=u/\alpha,\quad V_0'=-v/\alpha,\quad
 \Delta V'=(u+v)/\alpha.
```

```
\]
In particular, $p_2^{*\prime}(j)<0$ and $p_2^{*\prime}(0)>0$.
The total equilibrium terminal-share derivative is
$s_2'=q(a)s_2(1-s_2)^2$, including the optimal-price response.

For the first-period implicit pricing equation let
$F=p_1-c+1/[\alpha(1-s_1)]+\delta\Delta V$. At fixed history,
```

Source lines 156–197:

```
\[
 F_{p_1}=\frac1{r_0},\qquad
 F_\gamma=\frac{qs}{\alpha r_0}+\frac{\delta B}{\alpha},\qquad
 \boxed{p_1^{*\prime}=-\frac{qs+\delta r_0 B}{\alpha}}.
\]
The share therefore changes according to
\[
 s'=sr_0(q+\alpha p_1^{*\prime})=sr_0^2(q-\delta B).
\]
The first-period envelope theorem retains both continuation branches:
\begin{align*}
 \Pi^{*\prime}
 &=qsr_0[p_1^*-c+\delta\Delta V]+\delta[sV_j'+r_0V_0']\\
 &=\boxed{\frac{-qs+\delta(su-r_0v)}{\alpha}}.
\end{align*}
The term $\delta V_0'$ must not be dropped. The envelope partial derivative holds
the optimizing price fixed inside the partial derivative; it does not hold the
continuation value fixed as $\gamma$ varies.

For the analytical second price derivative define
\[
 u'=-u(1-u)^2,\quad v'=v(1-v)^2,\quad B'=u'+v'.
\]
Differentiating the first derivative and substituting $s'$ gives
\[
 \boxed{p_1^{*\prime\prime}
 =-\frac{sr_0^2(q-\delta B)^2+\delta r_0B'}{\alpha}}.
\]
This expression is used analytically in checks, rather than obtained from noisy
twice-differenced optimized prices. No second-derivative figure is included.

\begin{proposition}[Conditional signs]
For an initially outside consumer, $p_1^{*\prime}>0$ and $\Pi^{*\prime}>0$.
For an initially inside consumer,
\[
 p_1^{*\prime}>0\ \Longleftrightarrow\ \delta r_0(u+v)>s,\qquad
 \Pi^{*\prime}>0\ \Longleftrightarrow\ \delta r_0v>s(1+\delta u).
\]
Equalities give zero derivatives; reversed inequalities give negative derivatives.
\end{proposition}
\begin{proof}
For $q=1$, the price numerator $s+\delta r_0B$ is positive and $-1/\alpha>0$.
```

Source lines 198–239:

```
The profit numerator is $-s(1-\delta u)-\delta r_0v<0$ because $\delta u<1$;
dividing by $\alpha<0$ makes the derivative positive. Substituting $q=-1$
gives the two stated thresholds. These statements do not establish a common
monotonicity sign for both histories or a global sign for the price ratios.
\end{proof}

\section{Price ratios: three distinct conditioning statements}
```

```
The state-matched ratios are descriptive comparisons:
\[
 R_j=\frac{p_1^*(x_0=j)}{p_2^*(x_1=j)},\qquad
 R_0=\frac{p_1^*(x_0=0)}{p_2^*(x_1=0)}.
\]
They do not necessarily describe one realized consumer path. The acquisition
comparison is instead
\[
 R_{0\to j}=\frac{p_1^*(x_0=0)}{p_2^*(x_1=j)},
\]
conditional on acquisition in period 1. It is not an expectation over histories.
For each definition use its own numerator, denominator and state derivatives:
\[
 \boxed{R'=\frac{p_2^*p_1^{*\prime}-p_1^*p_2^{*\prime}}{(p_2^*)^2}}.
\]
Terminal prices exceed $c$; the benchmark $c=0$ therefore has strictly positive
ratio denominators. For other calibrations a ratio is only defined when its
denominator is nonzero. No expected ratio or ratio of expectations is substituted.

\section{Numerical benchmark and verification}
Every plotted point reoptimizes both terminal branches and period 1:
\[
 \beta=2,\qquad\alpha=-1,\qquad\delta=0.92,\qquad c=0,\qquad
 \gamma\in[-3,3]\ \text{on a 601-point grid}.
\]
The calibration is illustrative, not estimated. SymPy verifies raw-utility
normalization, both branch weights, the profit-derived FOCs, implicit price
derivatives and the envelope identities. Numerical checks compare all 1,202
benchmark state/grid solutions with the handoff's independent reference.
Central differences reoptimize the full model at nearby parameters; an
independent scalar profit optimizer checks optimality. Additional calibrations
check derivatives and an introductory-subsidy case. Numerical evidence is not
a proof of unqualified global comparative-static signs.

\subsection*{Selected optimized values}
```

Source lines 240–242:

```
\begin{center}\small\begin{tabular}{crrrrrr}\toprule
$x_0$ & $\gamma$ & $p_1^*$ & $p_2^*(j)$ & $p_2^*(0)$ & $\Pi^*$ & $\Pi^{*\prime}$\\\midrule
'''
```

Read this block:

- Line 30, `document`: Set this name to a multiline text template; its contents are data, not Python instructions.

Source lines 243–246:

```
for state in ['j','0']:
    for g in [-3,-1,0,1,3]:
        r=evaluate(g,state)
        document+=f"${state}$ & {g} & {r['p1']:.3f} & {r['p2j']:.3f} & {r['p20']:.3f} & {r['profit']:.3f} &
 ↪  {r['dprofit']:.3f}"+r'\\'+'\n'
```

Read this block:

- Line 243: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 244: for each entry yielded by `[-3, -1, 0, 1, 3]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 245, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.

- Line 246: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 247–250:

```
    document+=r'''\bottomrule\end{tabular}\end{center}
\clearpage
\input{sympy_equations.tex}
'''
```

Read this block:

- Line 247: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 251–264:

```
specs=[
    ('matched_price_ratio_derivative','State-matched price-ratio derivatives',
     r'$R_j=p_1^*(j)/p_2^*(j)$ and $R_0=p_1^*(0)/p_2^*(0)$. Each curve holds its stated histories fixed and uses
      ↪  the analytical quotient rule.'),
    ('induce_harvest_price_ratio_derivative','Induce-then-harvest price-ratio derivative',
     r'$R_{0\to j}=p_1^*(0)/p_2^*(j)$, conditional on a first-period acquisition. This is a path-specific
      ↪  comparison, not an ex-ante expectation.'),
    ('profit_by_gamma','Optimized profit levels',
     r'Expected discounted profit includes both realized continuation histories. The two curves condition on the
      ↪  initial realized state.'),
    ('gamma_profit_by_gamma','Optimized profit derivative',
     r'Total derivatives use the envelope theorem and both continuation-value derivatives. Their signs depend on
      ↪  the initial history and the stated thresholds.'),
    ('optimal_prices_by_gamma','Optimal prices in both periods',
     r'Left: period-1 prices for both initial states. Right: period-2 prices for both realized first-period states.
      ↪  Prices are unconstrained; all shares adjust.'),
    ('gamma_p1_by_gamma','Optimal period-1 price derivative',
     r'Analytical total derivatives, with both continuation branches reoptimized. Initial-state signs are
      ↪  interpreted using the proved conditional statements.'),
]
```

Read this block:

- Line 251, `specs`: Set this name to a list containing 6 displayed entries.

Source lines 265–270:

```
for name,title,caption in specs:
    if not (ROOT/'figures'/f'{name}.pdf').exists():
        raise FileNotFoundError(name)
    document+='\\clearpage\n\\section*{'+title+'}\n'
    document+='\\begin{figure}[htbp]\\centering\n\\includegraphics[width=\\linewidth]{figures/'+name+'.pdf}\n'
    document+='\\caption{'+caption+r' Benchmark: $\beta=2$, $\alpha=-1$, $\delta=0.92$, $c=0$;
 ↪  $\gamma\in[-3,3]$.}'+'\n\\end{figure}\n'
```

Read this block:

- Line 265: for each entry yielded by `specs`, bind/unpack it into `(name, title, caption)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 266: test `not (ROOT / 'figures' / f'{name}.pdf').exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 267: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 268: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 269: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 270: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `FileNotFoundError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 271–273:

```
271 document+='\\end{document}\n'
272 (ROOT/'comparative_statics_figures.tex').write_text(document)
273 print('Generated verified symbolic equations and appendix with six requested figures.')
```

Read this block:

- Line 271: update `document` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 272: call `(ROOT / 'comparative_statics_figures.tex').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 273: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

supply_code/model/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

There are no statements in this file.

supply_code/model/assumptions.py

This dictionary stores human-readable assumptions as strings. Python and SymPy do not enforce alpha<0 just because a dictionary says so. It serves documentation; the numerical evaluator performs explicit checks. Distinguish descriptive metadata from executable validation.

Module setup and direct-execution blocks

Source lines 1–3:

```
assumptions = {"alpha": "<0", "delta": "0<delta<1", "prices": "unconstrained real",
               "cost": "fixed", "shares": "strictly interior", "type": "fixed and observed",
               "history": "realized j or 0; both continuation states integrated"}
```

Read this block:

- Line 1, `assumptions`: Set this name to a dictionary with named entries `alpha`, `delta`, `prices`, `cost`, `shares`, `type`, `history`.

supply_code/model/numerical.py

Module-level tuples `_ARGS`, `_EXPRESSIONS`, and `_KEYS` establish the ordered bridge from symbolic algebra to named numerical derivative results. A leading underscore is a convention for an internal name, not access control. `sp.lambdify` executes once at import, avoiding repeated compilation of the same symbolic formulas for all 1,202 benchmark evaluations. The functions below reuse that callable. Positional order, rather than matching name spelling at runtime, connects its inputs.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Unconstrained backward induction; derivatives evaluated from SymPy."""
import numpy as np
import sympy as sp
from scipy.special import expit
from scipy.optimize import brentq
from model import symbols as S
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 4: from `scipy.special`, import `expit` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: from `scipy.optimize`, import `brentq` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: from `model`, import `symbols as S` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–14:

```
from model.utility import signed_state
from comparative_statics import canonical as C

PARAMETERS = dict(beta=2., alpha=-1., delta=.92, c=0.)
_KEYS = ("dp1", "d2p1", "dprofit", "matched_j", "matched_0", "induce_harvest", "dp2j", "dp20")
_ARGS = (S.alpha,S.beta,S.gamma,S.delta,S.c,S.q,S.sj1,S.u,S.v,S.p1,S.p2j,S.p20,S.Vj,S.V0)
_EXPRESSIONS = (C.dp1_dgamma,C.d2p1_dgamma2,C.dPi_dgamma,C.matched_j,C.matched_0,C.induce_harvest,
                C.price2_derivative(S.gamma,S.u,-1,S.p2j),C.price2_derivative(S.gamma,S.v,1,S.p20))
```

Read this block:

- Line 7: from `model.utility`, import `signed_state` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: from `comparative_statics`, import `canonical as C` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10, `PARAMETERS`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

- Line 11, `_KEYS`: Set this name to a tuple containing 8 displayed entries.
- Line 12, `_ARGS`: Set this name to a tuple containing 14 displayed entries.
- Line 13, `_EXPRESSIONS`: Set this name to a tuple containing 8 displayed entries.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 15–15:

```
15 _EVALUATE = sp.lambdify(_ARGS,_EXPRESSIONS,"numpy")
```

Read this block:

- Line 15, `_EVALUATE`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.lambdify`: Compile a symbolic expression into a numerical callable with an explicitly ordered argument list.

inside share

Original function: `inside_share`. Source begins at line 18.

This computes one numeric binary-logit probability. `signed_state` validates the state; `expit` evaluates logistic demand stably, then `float` turns a NumPy scalar into an ordinary Python number. Direct `exp(x)/(1+exp(x))` can overflow at large x. The defaults beta=2 and alpha=-1 support the benchmark, while callers can supply other calibrations. This function alone does not validate alpha's sign; full parameter-domain checks live in `evaluate`.

Definition and stated intent:

Source lines 18–18:

```
18 def inside_share(price,gamma,state,beta=2.,alpha=-1.):
```

`def` creates this callable; the parentheses name its inputs (`price`, `gamma`, `state`, `beta`, `alpha`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 19–19:

```
19 return float(expit(beta+signed_state(state)*gamma+alpha*price))
```

Read this block:

- Line 19: return the result of calling `float(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `signed_state`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

optimal price

Original function: `optimal_price`. Source begins at line 22.

This finds one optimal price for a utility intercept, negative alpha, and an effective cost. `f = lambda p: ...` defines the transformed scalar FOC. `clip` limits the exponential's argument to a representable range; it is a numerical safeguard rather than a theoretical cap on utility.

The initial lower endpoint is effective cost, where f is positive. `max` sets a reasonable initial width, and `while f(hi) > 0` doubles it until the bracket changes sign. `brentq` can then locate the zero to `xtol=1e-12`. A fixed interval is simpler but can miss a root under another calibration. Unconstrained generic minimization ignores the monotonic scalar structure and can need more evaluations. The maintained alpha<0 assumption establishes a unique root; the caller validates it.

Definition and stated intent:

Source lines 22–23:

```
22 def optimal_price(intercept,alpha,effective_cost):
23     # Multiplying the FOC by alpha gives a strictly decreasing function.
```

`def` creates this callable; the parentheses name its inputs (`intercept`, `alpha`, `effective_cost`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 24–27:

```
24 f = lambda p: alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p,-745,700))
25 lo = effective_cost
26 width = max(2.,2/abs(alpha))
27 hi = lo+width
```

Read this block:

- Line 24, `f`: Set this name to a one-expression callable, with input `p` and returned expression `alpha * (p - effective_cost) + 1 + np.exp(np.clip(in...`.
- Line 25, `lo`: Set this name to the object currently named `effective_cost`; plain assignment does not copy it.
- Line 26, `width`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 27, `hi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 28–30:

```
28 while f(hi) > 0:
29     width *= 2
30     hi = lo+width
```

Read this block:

- Line 28: repeat while `f(hi) > 0` remains true. Unlike for/range, this loop need not know the iteration count beforehand; its body must eventually change the condition.
- Line 29: update `width` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 30, `hi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `f`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 31–31:

```
31 return float(brentq(f,lo,hi,xtol=1e-12))
```

Read this block:

- Line 31: return the result of calling `float(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `brentq`: Find a scalar zero within a sign-changing bracket, rather than minimize the function itself.

evaluate

Original function: `evaluate`. Source begins at line 34.

This is backward induction expressed as ordinary assignments. It accepts gamma, initial history, and optional calibration numbers, then returns a dictionary. `np.isfinite([...]).all()` validates every numeric input at once. The terminal list comprehension solves two prices, unpacked into p2j and p20. Subsequent tuple assignments compute their shares and values; first-period effective cost then uses the value difference.

`_EVALUATE(...)` calls the numerical function built from symbolic derivatives at import time. Its argument order must match `_ARGS`; a positional swap can produce plausible but wrong derivatives. `result.update(dict(zip(_KEYS, map(float, values))))` converts and labels those outputs. State-inapplicable ratio derivatives become NaN intentionally. Final FOC residuals use original profit derivatives as a diagnostic independent of the root's transformed form. An external simultaneous optimizer would conceal the model's simpler sequential structure.

Definition and stated intent:

Source lines 34–34:

```
34 def evaluate(gamma,initial_state,beta=2.,alpha=-1.,delta=.92,c=0.):
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `initial_state`, `beta`, `alpha`, `delta`, `c`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 35–35:

```
35 q = signed_state(initial_state)
```

Read this block:

- Line 35, `q`: Set this name to the result of calling `signed_state(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `signed_state`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 36–37:

```
36 if not (np.isfinite([gamma,beta,alpha,delta,c]).all() and alpha<0 and 0<=delta<1):
37     raise ValueError("Require finite parameters, alpha<0, and 0<=delta<1 (delta=0 is the static counterfactual).")
```

Read this block:

- Line 36: test `not (np.isfinite([gamma, beta, alpha, delta, c]).all() and alpha < 0 and (0 <...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 37: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 38–43:

```
38 p2j,p20 = [optimal_price(beta+signed_state(state)*gamma,alpha,c) for state in ["j","0"]]
39 u,v = inside_share(p2j,gamma,"j",beta,alpha),inside_share(p20,gamma,"0",beta,alpha)
40 Vj,V0 = (p2j-c)*u,(p20-c)*v
41 DV = Vj-V0
42 p1 = optimal_price(beta+q*gamma,alpha,c-delta*DV)
43 s = inside_share(p1,gamma,initial_state,beta,alpha)
```

Read this block:

- Line 38, `(p2j, p20)`: Unpack a list computed by the displayed for-expression into these targets in order.
- Line 39, `(u, v)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 40, `(Vj, V0)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 41, `DV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 42, `p1`: Set this name to the result of calling `optimal_price(...)`; the arguments and preceding method chain are shown above.
- Line 43, `s`: Set this name to the result of calling `inside_share(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `optimal_price`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 44–51:

```
44 profit = s*(p1-c)+delta*(s*Vj+(1-s)*V0)
45 values = _EVALUATE(alpha,beta,gamma,delta,c,q,s,u,v,p1,p2j,p20,Vj,V0)
46 result = dict(gamma=float(gamma),initial_state=initial_state,p1=p1,p2j=p2j,p20=p20,s=s,u=u,v=v,
47               V2j=Vj,V20=V0,DeltaV=DV,profit=profit,
48               static_component=c-1/(alpha*(1-s)),continuation_adjustment=-delta*DV,
49               static_counterfactual=optimal_price(beta+q*gamma,alpha,c))
50 result.update(dict(zip(_KEYS,map(float,values))))
51 result['matched_j' if initial_state=='0' else 'matched_0'] = float('nan')
```

Read this block:

- Line 44, `profit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 45, `values`: Set this name to the result of calling `_EVALUATE(...)`; the arguments and preceding method chain are shown above.
- Line 46, `result`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 50: call `result.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51, `result['matched_j' if initial_state == '0' else 'matched_0']`: Store the result of calling `float(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `_EVALUATE`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `map`: Apply a callable to each iterable entry lazily, such as converting result scalars to Python float.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 52–53:

```
52 if initial_state=='j':
53     result['induce_harvest'] = float('nan')
```

Read this block:

- Line 52: test `initial_state == 'j'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 53, `result['induce_harvest']`: Store the result of calling `float(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 54–58:

```
54 result['d_matched_ratio'] = result['matched_j'] if initial_state=='j' else result['matched_0']
55 result['d_induce_ratio'] = result['induce_harvest'] if initial_state=='0' else float('nan')
56 result['foc1'] = s+alpha*s*(1-s)*(p1-c+delta*DV)
57 result['foc2j'] = u+alpha*u*(1-u)*(p2j-c)
58 result['foc20'] = v+alpha*v*(1-v)*(p20-c)
```

Read this block:

- Line 54, `result['d_matched_ratio']`: Store the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value in the selected entry/column.
- Line 55, `result['d_induce_ratio']`: Store the true-branch value if `initial_state == '0'` holds, otherwise the else-branch value in the selected entry/column.
- Line 56, `result['foc1']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 57, `result['foc2j']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 58, `result['foc20']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Source lines 59–59:

```
59 return result
```

Read this block:

- Line 59: return the object currently named `result`; plain assignment does not copy it. This ends the call.

solve

Original function: `solve`. Source begins at line 62.

This is a compact convenience interface. The bare `*` makes `initial_state` keyword-only; `**parameters` collects other keywords. Membership tests reject obsolete z1/z2 inputs. `parameters.pop("gamma", 0.)` retrieves and removes gamma, allowing the rest to be expanded into `evaluate` without passing gamma twice. The function returns seven values as a tuple. It is convenient for unpacking, while `evaluate` is preferable when you need named derivative or diagnostic fields.

Definition and stated intent:

Source lines 62–62:

```
62 def solve(*, initial_state="0", **parameters):
```

`def` creates this callable; the parentheses name its inputs (`initial_state`). A displayed `=value` supplies a default, and the colon begins its indented body.

`**parameters` collects additional keyword arguments in a dictionary.

Source lines 63–64:

```
63 if 'z1' in parameters or 'z2' in parameters:
64     raise ValueError("Old 0/1 histories are superseded: use initial_state='j' or '0'; both continuation branches
       ↪ are solved.")
```

Read this block:

- Line 63: test `'z1' in parameters or 'z2' in parameters`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 64: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 65–66:

```
65 gamma = parameters.pop('gamma',0.)
66 r = evaluate(gamma,initial_state,**parameters)
```

Read this block:

- Line 65, `gamma`: Set this name to the result of calling `parameters.pop(...)`; the arguments and preceding method chain are shown above.
- Line 66, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.pop`: Retrieve and remove a dictionary entry, using the supplied default if it is absent.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 67–67:

```
67 return r['p1'],r['p2j'],r['p20'],r['s'],r['u'],r['v'],r['profit']
```

Read this block:

- Line 67: return a tuple of 7 results. The caller can unpack them; return ends the call, unlike print.

supply_code/model/profit.py

These assignments construct symbolic current and continuation profit. `pi2` weights both optimized terminal values by first-period share and outside share. `Pi = pi1 + delta*pi2` creates total discounted profit. Capitalization matters: Python treats `Pi` and `pi1` as unrelated names. No optimizer runs in this file; it provides the expression that FOC modules and tests inspect.

Module setup and direct-execution blocks

Source lines 1–6:

```
from model.symbols import *
pi1 = sj1*(p1-c)
profit2 = sj2*(p2-c)
pi2 = sj1*Vj + (1-sj1)*V0
Pi = pi1 + delta*pi2
DeltaV = Vj-V0
```

Read this block:

- Line 1: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 2, `pi1`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 3, `profit2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 4, `pi2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 5, `Pi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 6, `DeltaV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

supply_code/model/shares.py

This module builds symbolic logit shares from normalized utility and defines derivative expressions. The star import makes shared symbols available directly without a module prefix, which shortens formulas but makes their origins harder for beginners to track. Expressions such as `alpha*sj1*(1-sj1)` encode the logistic price derivative using share placeholders. They are algebra, not evaluated probabilities for a dataset. The explicit-share expression and the share symbol are intentionally distinct objects.

Module setup and direct-execution blocks

Source lines 1–6:

```
import sympy as sp
from model.symbols import *
from model.utility import U1, U2
sj1_expr = 1/(1+sp.exp(-U1))
sj2_expr = 1/(1+sp.exp(-U2))
s01_expr, s02_expr = 1-sj1_expr, 1-sj2_expr
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: from `model.utility`, import `U1, U2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4, `sj1_expr`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 5, `sj2_expr`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 6, `(s01_expr, s02_expr)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 7–11:

```
dsj1_dp1, dsj2_dp2 = alpha*sj1*s01, alpha*sj2*s02
ds01_dp1, ds02_dp2 = -dsj1_dp1, -dsj2_dp2
dsj1_dgamma = q*sj1*s01
dsj2_dgamma_direct = q2*sj2*s02
dsj2_dutility = sj2*s02
```

Read this block:

- Line 7, `(dsj1_dp1, dsj2_dp2)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 8, `(ds01_dp1, ds02_dp2)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 9, `dsj1_dgamma`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 10, `dsj2_dgamma_direct`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 11, `dsj2_dutility`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

supply_code/model/symbols.py

`sp.symbols` constructs formal mathematical objects and tuple unpacking gives them Python names. The name displayed in equations can differ from the Python identifier, e.g. `p1` represents printed `p_1`. `real=True` records a symbolic domain; `positive=True` on shares does not also encode share below one. Economic restrictions such as alpha<0 remain assumptions used in interpretation and numeric validation. `s01 = 1-sj1` defines an expression, not another independent symbol. `z1,z2 = q,q2` aliases existing signed symbols for legacy code.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Symbols for a fixed type and signed realized histories."""
import sympy as sp
alpha, beta, gamma, delta, c = sp.symbols("alpha beta gamma delta c", real=True)
p1, p2, p2j, p20 = sp.symbols("p_1 p_2 p_{2j} p_{20}", real=True)
q, q2 = sp.symbols("q q_2", real=True)
sj1, sj2, u, v = sp.symbols("s s_2 u v", positive=True)
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 3, `(alpha, beta, gamma, delta, c)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 4, `(p1, p2, p2j, p20)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 5, `(q, q2)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 6, `(sj1, sj2, u, v)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.symbols`: Create SymPy symbols, which stand for algebraic variables rather than observed numbers.

Source lines 7–12:

```
7 s01, s02 = 1-sj1, 1-sj2
8 Vj, V0 = sp.symbols("V_{2j} V_{20}", real=True)
9 h0, h1 = sp.symbols("h_0 h_1")
10 p1_star_sym, p2_star_sym = sp.symbols("p_1^* p_2^*")
11 # Legacy names are signed coefficients now, never the former 0/1 indicators.
12 z1, z2 = q, q2
```

Read this block:

- Line 7, `(s01, s02)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 8, `(Vj, V0)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 9, `(h0, h1)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 10, `(p1_star_sym, p2_star_sym)`: Unpack the result of calling `sp.symbols(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 12, `(z1, z2)`: Unpack the tuple `(q, q2)` into these targets in order.

supply_code/model/utility.py

After defining its two helpers, module-level code calls `raw_utilities` for each period, unpacks inside/outside expressions, and subtracts outside utility. `sp.expand` distributes products to obtain the normalized linear index. This module combines function definitions with actual symbolic setup work at import time. It does not read private data or solve numeric prices.

Module setup and direct-execution blocks

Source lines 1–2:

```
1 import sympy as sp
2 from model.symbols import *
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import $*$ into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 17–20:

```
17 raw_inside1, raw_outside1 = raw_utilities(p1, q)
18 raw_inside2, raw_outside2 = raw_utilities(p2, q2)
19 U1 = sp.expand(raw_inside1 - raw_outside1)
20 U2 = sp.expand(raw_inside2 - raw_outside2)
```

Read this block:

- Line 17, `(raw_inside1, raw_outside1)`: Unpack the result of calling `raw_utilities(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 18, `(raw_inside2, raw_outside2)`: Unpack the result of calling `raw_utilities(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 19, `U1`: Set this name to the result of calling `sp.expand(...)`; the arguments and preceding method chain are shown above.
- Line 20, `U2`: Set this name to the result of calling `sp.expand(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `raw_utilities`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `.expand`: Distribute products and combine terms in a symbolic expression without solving its unknowns.

signed state

Original function: `signed_state`. Source begins at line 5.

This converts the two accepted string state names into integer signs. Two independent `if` blocks are fine because each successful branch returns at once. Any other value reaches `raise ValueError`. `signed_state("j")` returns -1; `signed_state("0")` returns +1; numeric `0` is not string `"0"` and is rejected. A dictionary lookup could shorten this to a mapping, but these explicit branches make both accepted values and the custom error easy for a novice to see.

Definition and stated intent:

Source lines 5–5:

```
5 def signed_state(state):
```

`def` creates this callable; the parentheses name its inputs (`state`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 6–7:

```
6 if state == "j":
7     return -1
```

Read this block:

- Line 6: test `state == 'j'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 7: return the negated or sign-adjusted expression `-1`. This ends the call.

Source lines 8–9:

```
8 if state == "0":
9     return 1
```

Read this block:

- Line 8: test `state == '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 9: return the literal value `1`. This ends the call.

Source lines 10–10:

```
10 raise ValueError("state must be 'j' or '0'")
```

Read this block:

- Line 10: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

raw utilities

Original function: `raw_utilities`. Source begins at line 13.

Input `price` may be a SymPy symbol and `signed_history` a sign or symbolic q. The return statement contains two comma-separated expressions, so it returns a tuple `(inside_utility, outside_utility)`. The combinations `(1+q)/2` and `(1-q)/2` act as complementary indicators when q is ±1. At q=-1 the first is zero and second one; at q=+1 they reverse.

Arithmetic with SymPy objects builds algebraic expressions rather than computing a numerical utility immediately. Returning both raw utilities preserves the outside switching payoff before normalization. Writing only an inside bonus would encode a different model. Module-level unpacking later names each result and subtracts the outside utility explicitly.

Definition and stated intent:

Source lines 13–13:

```
13 def raw_utilities(price, signed_history):
```

`def` creates this callable; the parentheses name its inputs (`price`, `signed_history`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 14–15:

```
14 return (beta + alpha*price + gamma*(1+signed_history)/2,
15         gamma*(1-signed_history)/2)
```

Read this block:

- Line 14: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

supply_code/pricing/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

There are no statements in this file.

supply_code/pricing/period1_foc.py

This module creates equivalent raw and collected forms of the dynamic first derivative. Factoring `(p1-c+delta*DeltaV)` displays the effective-margin meaning. `sp.Eq(expression, 0)` constructs a symbolic equation object; it does not solve that equation or assign zero to the expression. Keeping an equation for export beside an expression for differentiation avoids manually retyping formulas.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 import sympy as sp
2 from model.symbols import *
3 from model.profit import DeltaV
4 foc1_raw = sj1 + alpha*sj1*s01*(p1-c) + delta*alpha*sj1*s01*DeltaV
5 foc1_collected = sj1 + alpha*sj1*s01*(p1-c+delta*DeltaV)
6 foc1_eq = sp.Eq(foc1_collected, 0)
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import `*` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: from `model.profit`, import `DeltaV` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4, `foc1_raw`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 5, `foc1_collected`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 6, `foc1_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

supply_code/pricing/period1_price.py

`p1_star` is an implicit markup expression containing an equilibrium-share symbol and both continuation values. `sp.Eq` labels it with a formal starred price for display. It is not a numerical optimal price until those endogenous objects are solved consistently. Direct substitution of an arbitrary share would not solve the model.

Module setup and direct-execution blocks

Source lines 1–4:

```
1 import sympy as sp
2 from model.symbols import *
3 p1_star = c-1/(alpha*s01)-delta*(Vj-V0)
4 p1_star_eq = sp.Eq(p1_star_sym, p1_star)
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `p1_star`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 4, `p1_star_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

supply_code/pricing/period2_foc.py

The terminal FOC contains current share plus margin times its price derivative. There is no discount term because period 2 is terminal. `sp.Eq` wraps it as an equation for presentation; actual root solving happens in `model/numerical.py`. Separating the expression permits tests to compare it with a literal derivative of profit instead of trusting a displayed equation alone.

Module setup and direct-execution blocks

Source lines 1–4:

```
1 import sympy as sp
2 from model.symbols import *
3 foc2 = sj2 + alpha*sj2*s02*(p2-c)
4 foc2_eq = sp.Eq(foc2, 0)
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import ∗ into this module's namespace. Importing can execute the imported module's top-level setup.

- Line 3, `foc2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 4, `foc2_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

supply_code/pricing/period2_price.py

The markup expression is cost minus `1/(alpha*s02)`, positive above cost for negative alpha and interior shares. Parentheses make the whole product the denominator: `1/alpha*s02` would mean `(1/alpha)*s02`, a different formula. This is another implicit representation because outside share depends on price.

Module setup and direct-execution blocks

Source lines 1–4:

```
1 import sympy as sp
2 from model.symbols import *
3 p2_star = c-1/(alpha*s02)
4 p2_star_eq = sp.Eq(p2_star_sym, p2_star)
```

Read this block:

- Line 1: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 2: from `model.symbols`, import `*` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3, `p2_star`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 4, `p2_star_eq`: Set this name to the result of calling `sp.Eq(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.Eq`: Create a symbolic equality object; unlike assignment it does not bind its left side to its right side.

supply_code/run_all.py

`Path(__file__).resolve().parent` locates this script's directory regardless of the shell's current directory. A list of command-argument lists describes the three reproducibility steps. `[sys.executable, *command]` uses the current Python interpreter and expands each command's arguments without a shell. `cwd=ROOT` makes relative imports and outputs predictable; `check=True` raises if a step exits unsuccessfully, preventing later artifact generation after a failed check. Running a shell string with semicolons would be less portable and would require additional quoting/exit-status care.

Module setup and direct-execution blocks

Source lines 1–5:

```
1 """Reproducibility entry point: tests, requested figures, symbolic LaTeX."""
2 import subprocess
3 import sys
4 from pathlib import Path
5 ROOT=Path(__file__).resolve().parent
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `subprocess`. An as-name is a convenient local alias, not another library.
- Line 3: import `sys`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5, `ROOT`: Set this name to the attribute `Path(__file__).resolve().parent` of the object before the dot.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 6–11:

```
for command in [
    ['-m','unittest','-v','checks.test_symmetric'],
    ['make_comparative_static_figures.py'],
    ['make_latex_document.py'],
]:
    subprocess.run([sys.executable,*command],cwd=ROOT,check=True)
```

Read this block:

- Line 6: for each entry yielded by `[['-m', 'unittest', '-v', 'checks.test_symmetric'], ['make_com...`, bind/unpack it into `command` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 11: call `subprocess.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 12–12:

```
print('All checks passed; figures and LaTeX regenerated. Compile the TeX with latexmk.')
```

Read this block:

- Line 12: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

supply_code/structural_switching_reference.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–10:

```
"""Two-period symmetric switching model: solve, differentiate, verify, plot.

Run: python structural_switching_reference.py
Requires: numpy, scipy, matplotlib.  Outputs to ./outputs.
```

```
"""
from pathlib import Path
import csv
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 6: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 7: import `csv`. An as-name is a convenient local alias, not another library.
- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 10: from `scipy.optimize`, import `brentq` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 12–14:

```
BETA, ALPHA, DELTA, COST = 2.0, -1.0, 0.92, 0.0
GAMMAS = np.linspace(-3.0, 3.0, 601)
OUTPUT = Path('outputs')
```

Read this block:

- Line 12, `(BETA, ALPHA, DELTA, COST)`: Unpack a tuple containing 4 displayed entries into these targets in order.
- Line 13, `GAMMAS`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 14, `OUTPUT`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 130–131:

```
if __name__ == '__main__':
    main()
```

Read this block:

- Line 130: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 131: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

expit safe

Original function: `expit_safe`. Source begins at line 17.

This imports and returns SciPy's stable logistic transform. The local import occurs when the function is called; Python caches imported modules, so it is not reloading SciPy from scratch each time. A module-level import

would be more conventional here. The numerical advantage over a naive exponential ratio is avoiding overflow at extreme utility indices.

Definition and stated intent:

Source lines 17–17:

```
17 def expit_safe(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 18–18:

```
18 from scipy.special import expit
```

Read this block:

- Line 18: from `scipy.special`, import `expit` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 19–19:

```
19 return expit(x)
```

Read this block:

- Line 19: return the result of calling `expit(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

inside share

Original function: `inside_share`. Source begins at line 22.

This independent reference maps a state to a sign with nested conditional expressions, rejects invalid states, and evaluates logistic demand. The final `else None` makes invalid input detectable before multiplying it by gamma. Defaults capture the benchmark constants at function definition time. Unlike the production state helper, the sign logic is written independently to help cross-check the implementation rather than simply calling the same helper twice.

Definition and stated intent:

Source lines 22–23:

```
22 def inside_share(price, gamma, state, beta=BETA, alpha=ALPHA):
23     """state='j': existing customer, state='0': outside previously."""
```

`def` creates this callable; the parentheses name its inputs (`price`, `gamma`, `state`, `beta`, `alpha`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 24–24:

```
24 q = -1 if state == 'j' else +1 if state == '0' else None
```

Read this block:

- Line 24, `q`: Set this name to the true-branch value if `state == 'j'` holds, otherwise the else-branch value.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 25–26:

```
25 if q is None:
26     raise ValueError("state must be 'j' or '0'")
```

Read this block:

- Line 25: test `q is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 27–27:

```
27 return expit_safe(beta + q * gamma + alpha * price)
```

Read this block:

- Line 27: return the result of calling `expit_safe(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `expit_safe`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

p2 opt

Original function: `p2_opt`. Source begins at line 30.

This reference solves the terminal markup equation using a lambda and a fixed bracket [-30,40]. Its gamma/state inputs enter through the closed-over variables. The return is the `brentq` root. It is appropriate for the reference benchmark; the production solver's expanding bracket handles a wider calibration range without assuming the root remains in this interval. Separate implementations make agreement more informative than two wrappers around the same solver.

Definition and stated intent:

Source lines 30–31:

```
30 def p2_opt(gamma, state):
31     """Unique finite price solving p=c-1/[alpha*(1-s)] (interior)."""
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `state`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 32–32:

```
32 f = lambda p: p - COST + 1 / (ALPHA * (1 - inside_share(p, gamma, state)))
```

Read this block:

- Line 32, `f`: Set this name to a one-expression callable, with input `p` and returned expression `p - COST + 1 / (ALPHA * (1 - inside_share(p, gamma, ...`.

Syntax and calls used here:

- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 33–33:

```
33 return brentq(f, -30.0, 40.0, xtol=1e-13)
```

Read this block:

- Line 33: return the result of calling `brentq(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `brentq`: Find a scalar zero within a sign-changing bracket, rather than minimize the function itself.

evaluate

Original function: `evaluate`. Source begins at line 36.

This independently computes backward-induction prices, branch values, profit, and hand-written analytic gamma derivatives. Comma-separated assignments unpack small tuples; `r = 1-s` names the outside share. Conditional expressions choose the correct matched denominator, and acquisition ratios are NaN outside their applicable initial history. An `assert np.isclose(...)` checks the price decomposition internally. Production uses lambdified canonical formulas, whereas the reference writes formulas directly to provide an independent oracle. Its interface and benchmark constants are narrower than production's.

Definition and stated intent:

Source lines 36–36:

```
36 def evaluate(gamma, initial_state):
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `initial_state`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 37–42:

```
37 p2j, p20 = p2_opt(gamma, 'j'), p2_opt(gamma, '0')
38 u, v = inside_share(p2j, gamma, 'j'), inside_share(p20, gamma, '0')
39 Vj, V0 = (p2j-COST)*u, (p20-COST)*v
40 dV = Vj - V0
41 q = -1 if initial_state == 'j' else 1
42 f = lambda p: p-COST+1/(ALPHA*(1-inside_share(p, gamma, initial_state)))+DELTA*dV
```

Read this block:

- Line 37, `(p2j, p20)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 38, `(u, v)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 39, `(Vj, V0)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 40, `dV`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 41, `q`: Set this name to the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value.
- Line 42, `f`: Set this name to a one-expression callable, with input `p` and returned expression `p - COST + 1 / (ALPHA * (1 - inside_share(p, gamma, ...`.

Syntax and calls used here:

- `p2_opt`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 43–48:

```
p1 = brentq(f, -30.0, 40.0, xtol=1e-13)
s = inside_share(p1, gamma, initial_state)
r = 1-s
pi = s*(p1-COST)+DELTA*(s*Vj+r*V0)
static_piece = COST - 1/(ALPHA*r)
dynamic_piece = -DELTA*dV
```

Read this block:

- Line 43, `p1`: Set this name to the result of calling `brentq(...)`; the arguments and preceding method chain are shown above.
- Line 44, `s`: Set this name to the result of calling `inside_share(...)`; the arguments and preceding method chain are shown above.
- Line 45, `r`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 46, `pi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 47, `static_piece`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 48, `dynamic_piece`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `brentq`: Find a scalar zero within a sign-changing bracket, rather than minimize the function itself.

Source lines 50–56:

```
d_pi = (-q*s + DELTA*(s*u-r*v))/ALPHA
dp1 = -(q*s + DELTA*r*(u+v))/ALPHA
# ds/dgamma = s*r*(q + alpha*dp1) = s*r^2*(q - delta*(u+v)).
ds = s*r*r*(q-DELTA*(u+v))
dB = -u*(1-u)**2 + v*(1-v)**2
d2p1 = -(s*r*r*(q-DELTA*(u+v))**2 + DELTA*r*dB)/ALPHA
dp2j = -(-1)*u/ALPHA
```

Read this block:

- Line 50, `d_pi`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 51, `dp1`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 53, `ds`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 54, `dB`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 55, `d2p1`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 56, `dp2j`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 57–61:

```
57 dp20 = -(+1)*v/ALPHA
58 matched_p2 = p2j if initial_state == 'j' else p20
59 matched_dp2 = dp2j if initial_state == 'j' else dp20
60 d_matched_ratio = (dp1*matched_p2-p1*matched_dp2)/matched_p2**2
61 d_induce_ratio = (dp1*p2j - p1*dp2j)/p2j**2 if initial_state == '0' else np.nan
```

Read this block:

- Line 57, `dp20`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 58, `matched_p2`: Set this name to the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value.
- Line 59, `matched_dp2`: Set this name to the true-branch value if `initial_state == 'j'` holds, otherwise the else-branch value.
- Line 60, `d_matched_ratio`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 61, `d_induce_ratio`: Set this name to the true-branch value if `initial_state == '0'` holds, otherwise the else-branch value.

Source lines 62–62:

```
62 assert np.isclose(p1, static_piece+dynamic_piece, atol=1e-9)
```

Read this block:

- Line 62: require `np.isclose(p1, static_piece + dynamic_piece, atol=1e-09)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 63–67:

```
63 return dict(gamma=gamma, initial_state=initial_state, p1=p1, p2j=p2j,
64             p20=p20, profit=pi, V2j=Vj, V20=V0, DeltaV=dV,
65             static_component=static_piece, continuation_adjustment=dynamic_piece,
66             dp1=dp1, d2p1=d2p1, dprofit=d_pi,
67             d_matched_ratio=d_matched_ratio, d_induce_ratio=d_induce_ratio)
```

Read this block:

- Line 63: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

verify

Original function: `verify`. Source begins at line 70.

This loops over selected gamma values and both histories. For each it re-solves at gamma+h and gamma-h, then compares analytic derivatives with centered finite differences. The step h balances approximation and floating-point error; making it arbitrarily tiny does not ensure accuracy. `assert condition, details` raises AssertionError carrying a diagnostic tuple when a check fails. Ratios must be recomputed from reoptimized price levels, not differentiated holding terminal prices fixed. This is a numerical check, not a symbolic sign proof.

Definition and stated intent:

Source lines 70–70:

```
70 def verify():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 71–71:

```
71 h = 1e-4
```

Read this block:

- Line 71, `h`: Set this name to the literal value `0.0001`.

Source lines 72–86:

```
72 for g in [-2.4, -1.2, 0, 0.7, 2.6]:
73     for state in ['j', '0']:
74         result = evaluate(g, state)
75         plus, minus = evaluate(g+h, state), evaluate(g-h, state)
76         for value, key in [('dp1', 'p1'), ('dprofit', 'profit')]:
77             numerical = (plus[key]-minus[key])/(2*h)
78             assert np.isclose(result[value], numerical, atol=2e-7), (g,state,value,result[value],numerical)
79         numerical2 = (plus['dp1']-minus['dp1'])/(2*h)
80         assert np.isclose(result['d2p1'], numerical2, atol=2e-7), (g,state,'d2p1')
81         numerical_ratio = (plus['p1']/ (plus['p2j'] if state=='j' else plus['p20']) -
82                            minus['p1']/(minus['p2j'] if state=='j' else minus['p20']))/(2*h)
83         assert np.isclose(result['d_matched_ratio'], numerical_ratio, atol=2e-7)
84         if state == '0':
85             numerical_ih = (plus['p1']/plus['p2j'] - minus['p1']/minus['p2j'])/(2*h)
86             assert np.isclose(result['d_induce_ratio'], numerical_ih, atol=2e-7)
```

Read this block:

- Line 72: for each entry yielded by `[-2.4, -1.2, 0, 0.7, 2.6]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 73: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 74, `result`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 75, `(plus, minus)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 76: for each entry yielded by `[('dp1', 'p1'), ('dprofit', 'profit')]`, bind/unpack it into `(value, key)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 77, `numerical`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 78: require `np.isclose(result[value], numerical, atol=2e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 79, `numerical2`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 80: require `np.isclose(result['d2p1'], numerical2, atol=2e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 81, `numerical_ratio`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 83: require `np.isclose(result['d_matched_ratio'], numerical_ratio, atol=2e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 84: test `state == '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 85, `numerical_ih`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 86: require `np.isclose(result['d_induce_ratio'], numerical_ih, atol=2e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 87–87:

```
87 print('All analytical derivative checks passed.')
```

Read this block:

- Line 87: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

plot series

Original function: `plot_series`. Source begins at line 90.

This plots one result field for two initial histories and saves a raster image. `[r[key] for r in rows[state]]` extracts that field from every result dictionary. Separating the data key from axis label/filename lets one helper generate price, derivative, and ratio plots. The function expects rows already aligned to GAMMAS. Its outputs belong to the retained reference program, not the six-figure production path used by `run_all.py`.

Definition and stated intent:

Source lines 90–90:

```
90 def plot_series(rows, key, ylabel, filename, title):
```

`def` creates this callable; the parentheses name its inputs (`rows`, `key`, `ylabel`, `filename`, `title`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 91–91:

```
91 fig, ax = plt.subplots(figsize=(8.6, 5.2))
```

Read this block:

- Line 91, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 92–93:

```
92 for state,label in [('j',r'Initially inside ($x_0=j$)'),('0',r'Initially outside ($x_0=0$)')]:
93     ax.plot(GAMMAS, [r[key] for r in rows[state]], label=label, linewidth=2)
```

Read this block:

- Line 92: for each entry yielded by `[('j', 'Initially inside ($x_0=j$)'), ('0', 'Initially outside...`, bind/unpack it into `(state, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 93: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 94–99:

```
94 ax.axhline(0, color='gray', linewidth=.8)
95 ax.set(xlabel=r'$\gamma$', ylabel=ylabel, title=title)
96 ax.legend()
97 fig.tight_layout()
98 fig.savefig(OUTPUT/filename, dpi=180)
99 plt.close(fig)
```

Read this block:

- Line 94: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 96: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 97: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 98: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 102.

This first verifies the reference formulas, creates its `outputs` directory, solves benchmark rows, writes CSV, and saves several reference plots. `with ... open(..., newline="")` lets the CSV writer handle line endings correctly. Some lines contain semicolons separating multiple statements; Python executes them left-to-right just as if written

on separate lines. That compact style saves space but is less friendly to beginners. Production uses another entry point and another output folder; running this does not replace `run_all.py`.

Definition and stated intent:

Source lines 102–102:

```
102 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 103–105:

```
103 verify()
104 OUTPUT.mkdir(exist_ok=True)
105 rows = {state:[evaluate(float(g),state) for g in GAMMAS] for state in ['j','0']}
```

Read this block:

- Line 103: call `verify` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 104: call `OUTPUT.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105, `rows`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.

Syntax and calls used here:

- `verify`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 106–110:

```
106 with (OUTPUT/'model_results.csv').open('w', newline='') as file:
107     writer = csv.DictWriter(file, fieldnames=list(rows['j'][0]))
108     writer.writeheader()
109     for state in ['j','0']:
110         writer.writerows(rows[state])
```

Read this block:

- Line 106: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 107, `writer`: Set this name to the result of calling `csv.DictWriter(...)`; the arguments and preceding method chain are shown above.
- Line 108: call `writer.writeheader` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 109: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 110: call `writer.writerows` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `.DictWriter`: Create a CSV writer mapping dictionary keys to named columns in an explicit field order.

- `.writeheader`: Write the CSV's named column header before writing data rows.
- `.writerows`: Write a sequence of dictionaries/rows using the configured CSV schema.
- `list`: Materialize iterable entries as an ordered Python list.

Source lines 111–116:

```
111 plot_series(rows, 'p1', r'$p_1^*$', 'price1.png', 'Optimal period-1 prices')
112 plot_series(rows, 'dp1', r'$dp_1^*/d\gamma$', 'dp1.png', 'First derivative of period-1 price')
113 plot_series(rows, 'd2p1', r'$d^2p_1^*/d\gamma^2$', 'd2p1.png', 'Second derivative of period-1 price (analytic)')
114 plot_series(rows, 'dprofit', r'$d\Pi^*/d\gamma$', 'dprofit.png', 'Optimized profit derivative')
115 plot_series(rows, 'd_matched_ratio', r'$d(p_1/p_2)/d\gamma$', 'matched_price_ratio.png', 'State-matched
    ↪ price-ratio derivatives')
116 fig,ax=plt.subplots(figsize=(8.6,5.2))
```

Read this block:

- Line 111: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 112: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 113: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 114: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 115: call `plot_series` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 116, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `plot_series`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 117–120:

```
117 ax.plot(GAMMAS, [r['d_induce_ratio'] for r in rows['0']], linewidth=2)
118 ax.axhline(0,color='gray',linewidth=.8)
119 ax.set(xlabel=r'$\gamma$', ylabel=r'$d[p_1(0)/p_2(j)]/d\gamma$', title='Induce-then-harvest price-ratio
    ↪ derivative')
120 fig.tight_layout();fig.savefig(OUTPUT/'induce_harvest_ratio.png',dpi=180);plt.close(fig)
```

Read this block:

- Line 117: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 118: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 119: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 120: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 120: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 120: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.

- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.

Source lines 121–127:

```
121 for state in ['j','0']:
122     fig,ax=plt.subplots(figsize=(8.6,5.2))
123     for key,label in [('p1',r'$p_1^*$'),('static_component','Static FOC
        ↪ component'),('continuation_adjustment',r'$-\delta\Delta V_2$')]:
124         ax.plot(GAMMAS,[r[key] for r in rows[state]],label=label,linewidth=2)
125     ax.axhline(0,color='gray',linewidth=.8)
126     ax.set(xlabel=r'$\gamma$',ylabel='Price / component',title=f'First-period decomposition: initial state
    ↪ {state}')
127     ax.legend();fig.tight_layout();fig.savefig(OUTPUT/f'decomposition_{state}.png',dpi=180);plt.close(fig)
```

Read this block:

- Line 121: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 122, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 123: for each entry yielded by `[('p1', '$p_1^*$'), ('static_component', 'Static FOC component...`, bind/unpack it into `(key, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 124: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 125: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 126: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.legend`: Render labels previously attached to plotted series, identifying curves.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 128–128:

```
128 print('Saved outputs to', OUTPUT.resolve())
```

Read this block:

- Line 128: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

pricing_exercises: source walkthrough

pricing_exercises/run_empirical_targeting.py

Earlier combined-gamma berry-only pricing comparison. It does not accept the split fit. Use run_split_targeting.py or final_simple/demand/split_pricing.py for the current model. Its .95 revenue discount default differs from the new split exercise's .92.

Module setup and direct-execution blocks

Source lines 2–11:

```
"""Standalone empirical berry targeting, separate from demand estimation.

User-supplied paths only. No estimation is performed. --self-test uses only
generated synthetic inputs. See EMPIRICAL_TARGETING.md for definitions.
"""
from __future__ import annotations
import argparse
import hashlib
import json
import math
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `__future__`, import `annotations` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 9: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 10: import `json`. An as-name is a convenient local alias, not another library.
- Line 11: import `math`. An as-name is a convenient local alias, not another library.

Source lines 12–17:

```
import os
from pathlib import Path
import sys
import tempfile
import numpy as np
import pandas as pd
```

Read this block:

- Line 12: import `os`. An as-name is a convenient local alias, not another library.
- Line 13: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 14: import `sys`. An as-name is a convenient local alias, not another library.
- Line 15: import `tempfile`. An as-name is a convenient local alias, not another library.

- Line 16: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 17: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 19–19:

```
19 FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 19, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

Source lines 569–570:

```
569 if __name__ == "__main__":
570     main()
```

Read this block:

- Line 569: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 570: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

expit

Original function: `expit`. Source begins at line 21.

Evaluates the logistic probability with an exponential of a nonpositive magnitude to avoid overflow. Its input can be an array. This is a numerical transformation, not a demand-estimation call.

Definition and stated intent:

Source lines 21–21:

```
21 def expit(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 22–23:

```
22 x = np.asarray(x)
23 small = np.exp(-np.abs(x))
```

Read this block:

- Line 22, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 23, `small`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 24–24:

```
24 return np.where(x >= 0, 1/(1+small), small/(1+small))
```

Read this block:

- Line 24: return the result of calling `np.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

logsumexp

Original function: `logsumexp`. Source begins at line 26.

Computes log(sum(exp(x))) after subtracting the largest entry. The maximum is added back afterwards. axis selects the aggregation direction and keepdims retains singleton axes for broadcasting. This stabilizes likelihood normalization when utilities are large.

Definition and stated intent:

Source lines 26–26:

```
26 def logsumexp(x, axis=None, keepdims=False):
```

`def` creates this callable; the parentheses name its inputs (`x`, `axis`, `keepdims`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 27–29:

```
27 x = np.asarray(x)
28 maximum = np.max(x, axis=axis, keepdims=True)
29 result = maximum + np.log(np.exp(x-maximum).sum(axis=axis, keepdims=True))
```

Read this block:

- Line 27, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 28, `maximum`: Set this name to the result of calling `np.max(...)`; the arguments and preceding method chain are shown above.
- Line 29, `result`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 30–30:

```
30 return result if keepdims else np.squeeze(result, axis=axis)
```

Read this block:

- Line 30: return the true-branch value if `keepdims` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

plotting

Original function: `plotting`. Source begins at line 32.

Selects a noninteractive plotting backend and a writable temporary configuration directory, then returns pyplot. It does not read household data. Backend selection precedes importing pyplot so server runs do not require a display.

Definition and stated intent:

Source lines 32–32:

```
32 def plotting():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 33–36:

```
33 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
34 import matplotlib
35 matplotlib.use("Agg")
36 import matplotlib.pyplot as plt
```

Read this block:

- Line 33: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 34: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 35: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 37–37:

```
37 return plt
```

Read this block:

- Line 37: return the object currently named `plt`; plain assignment does not copy it. This ends the call.

persistent draws

Original function: `persistent_draws`. Source begins at line 39.

Generates seeded antithetic normal integration points: half the draws and their negatives. Split pricing uses three columns for log price sensitivity, novelty and own-stock habit; combined-gamma pricing uses two. These are fixed population integration points, not newly estimated coefficients or a bootstrap sample.

Definition and stated intent:

Source lines 39–39:

```
39 def persistent_draws(number, seed):
```

`def` creates this callable; the parentheses name its inputs (`number`, `seed`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 40–41:

```
40 if number < 2 or number % 2:
41     raise ValueError("Draw count must be even and >=2.")
```

Read this block:

- Line 40: test `number < 2 or number % 2`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 41: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 42–42:

```
42 half = np.random.default_rng(seed).standard_normal((number//2, 2))
```

Read this block:

- Line 42, `half`: Set this name to the result of calling `.standard_normal(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.standard_normal`: Draw independent standard-normal values in the requested shape, before coefficient transformations.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 43–43:

```
43 return np.vstack([half, -half])
```

Read this block:

- Line 43: return the result of calling `np.vstack(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.vstack`: Stack input arrays vertically, adding rows while preserving their common column layout.

posterior

Original function: `posterior`. Source begins at line 45.

Purpose. Recover household types from a saved fit and a matching weekly table. Each draw represents one persistent type. Utilities yield weekly count-weighted log probabilities; bincount adds all weeks within household before log-sum-exp integrates types. Normalizing those whole-history likelihoods gives household posterior weights.

For split pricing, alpha is positive price sensitivity, and the history array has columns b_N and b_S. weights @ history produces both posterior means. Sorting only the novelty means assigns bottom/top floor(N/4) reporting groups; both coefficients still enter forecasts. Posterior SDs and effective sample size describe type uncertainty. The returned NLL must match the saved fit before pricing. The earlier combined-gamma file instead returns a single gamma array.

Definition and stated intent:

Source lines 45–46:

```
45 def posterior(frame, parameters, draws, control=True):
46     """Household-integrated multinomial likelihood; types persist across weeks."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `parameters`, `draws`, `control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 47–52:

```
47 labels = np.sort(frame.household_code.unique())
48 hh = pd.Categorical(frame.household_code, categories=labels).codes
49 prices = frame[["price_"+f for f in FLAVORS]].to_numpy(float)
50 available = np.isfinite(prices) & (prices > 0)
51 counts = frame[FLAVORS+["outside"]].to_numpy(float)
52 xi = frame[["xi_"+f for f in FLAVORS]].to_numpy(float)
```

Read this block:

- Line 47, `labels`: Set this name to the result of calling `np.sort(...)`; the arguments and preceding method chain are shown above.
- Line 48, `hh`: Set this name to the attribute `pd.Categorical(frame.household_code, categories=labels).c...` of the object before the dot.
- Line 49, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 50, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 51, `counts`: Set this name to the result of calling `frame[FLAVORS + ['outside']].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 52, `xi`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 53–53:

```
53 cf = frame[["cf_"+f for f in FLAVORS]].to_numpy(float) if control else np.zeros_like(prices)
```

Read this block:

- Line 53, `cf`: Set this name to the true-branch value if `control` holds, otherwise the else-branch value.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 54–57:

```
54 if (not np.isfinite(counts).all() or (counts < 0).any()
55         or not np.isfinite(xi).all() or not np.isfinite(cf).all()
56         or ((counts[:, :3] > 0) & ~available).any()):
57     raise ValueError("Invalid counts, history, controls, or purchased-alternative coverage.")
```

Read this block:

- Line 54: test `not np.isfinite(counts).all() or (counts < 0).any() or (not np.isfinite(xi).a...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 57: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 58–61:

```
58 beta = parameters["inside_constant"] + np.array([0., parameters["beta_berry"], parameters["beta_plain"]])
59 alpha = np.exp(parameters["mu_log_minus_alpha"] + parameters["sd_log_minus_alpha"]*draws[:, 0])
60 gamma = parameters["mean_gamma"] + parameters["sd_gamma"]*draws[:, 1]
61 loglike = np.empty((len(labels), len(draws)))
```

Read this block:

- Line 58, `beta`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 59, `alpha`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 60, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 61, `loglike`: Set this name to the result of calling `np.empty(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 62–67:

```
62 for d, (a, g) in enumerate(zip(alpha, gamma)):
63     u = beta-a*np.nan_to_num(prices)+g*xi+parameters.get("price_control", 0.)*cf
64     u = np.column_stack([np.where(available, u, -np.inf), np.zeros(len(frame))])
65     logp = u-logsumexp(u, axis=1, keepdims=True)
66     ll = (counts*np.where(counts > 0, logp, 0.)).sum(axis=1)
67     loglike[:, d] = np.bincount(hh, weights=ll, minlength=len(labels))
```

Read this block:

- Line 62: for each entry yielded by `enumerate(zip(alpha, gamma))`, bind/unpack it into `(d, (a, g))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 63, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 64, `u`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 65, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 66, `ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.

- Line 67, `loglike[:, d]`: Store the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 68–74:

```
68 weights = np.exp(loglike-logsumexp(loglike, axis=1, keepdims=True))
69 nll = -float((logsumexp(loglike, axis=1)-np.log(len(draws))).sum())
70 means = weights @ gamma
71 # Exactly floor(N/4) per tail. Household ID breaks ties reproducibly.
72 order = np.argsort(means, kind="stable")
73 groups = np.full(len(labels), "middle", dtype=object)
74 n = len(labels)//4
```

Read this block:

- Line 68, `weights`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 69, `nll`: Set this name to the negated or sign-adjusted expression `-float((logsumexp(loglike, axis=1) - np.log(len(draws))).....`.
- Line 70, `means`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 72, `order`: Set this name to the result of calling `np.argsort(...)`; the arguments and preceding method chain are shown above.
- Line 73, `groups`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 74, `n`: Set this name to floor division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 75–76:

```
75 if n < 1:
76     raise ValueError("Need at least four households to form quartiles.")
```

Read this block:

- Line 75: test `n < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 76: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 77–83:

```
77 groups[order[:n]], groups[order[-n:]] = "low", "high"
78 population_q = np.quantile(gamma, [.25, .75])
79 table = pd.DataFrame(dict(household_code=labels, group=groups, gamma_mean=means,
80                           gamma_sd=np.sqrt(np.maximum(weights @ gamma**2-means**2, 0)),
81                           alpha_mean=weights @ alpha, posterior_ess=1/(weights**2).sum(axis=1),
82                           probability_gamma_below_draw_q25=weights @ (gamma <= population_q[0]),
83                           probability_gamma_above_draw_q75=weights @ (gamma >= population_q[1])))
```

Read this block:

- Line 77, `(groups[order[:n]], groups[order[-n:]])`: Unpack the tuple `('low', 'high')` into these targets in order.
- Line 78, `population_q`: Set this name to the result of calling `np.quantile(...)`; the arguments and preceding method chain are shown above.
- Line 79, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.quantile`: Find empirical quantiles, supplying percentile bootstrap endpoints rather than normal-Wald formulas.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 84–84:

```
84 return table, weights, alpha, gamma, beta, nll
```

Read this block:

- Line 84: return a tuple of 6 results. The caller can unpack them; return ends the call, unlike print.

compositions

Original function: `compositions`. Source begins at line 86.

Enumerates every nonnegative integer basket whose four counts sum to capacity: three flavors plus constructed nonpurchase. For capacity 7 there are 120 baskets. Enumerating all baskets preserves realized history transitions; substituting the mean basket would generally alter continuation values.

Definition and stated intent:

Source lines 86–87:

```
86 def compositions(capacity):
87     """All multinomial baskets of three inside flavors and outside."""
```

`def` creates this callable; the parentheses name its inputs (`capacity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 88–89:

```
88 return np.array([(a,b,c,capacity-a-b-c) for a in range(capacity+1)
89                  for b in range(capacity-a+1) for c in range(capacity-a-b+1)], dtype=int)
```

Read this block:

- Line 88: return the result of calling `np.array(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

forecast context

Original function: `forecast_context`. Source begins at line 91.

Relabels the first observed choice as t=0, adds its flavor shares once to its pre-choice stock, and uses that basket for next-week novelty. It carries the first menu/prices/controls forward, records the two subsequent pricing dates, and checks focal-menu eligibility. Later observed purchases affect type recovery but not this initial counterfactual history.

Definition and stated intent:

Source lines 91–96:

```
91 def forecast_context(frame, households):
92     """First observed choice establishes h0; price the next two consecutive weeks.
93 
94     Carry the initial observation's menu, posted prices and controls forward.
95     Type estimation still uses the complete observed sample.
96     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `households`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 97–103:

```
97 first = frame.sort_values(["household_code", "week"]).groupby("household_code", sort=True).head(1)
98 first = first.set_index("household_code").loc[households.household_code]
99 counts = first[FLAVORS].to_numpy(float)
100 totals = counts.sum(axis=1, keepdims=True)
101 shares = np.divide(counts, totals, out=np.zeros_like(counts), where=totals > 0)
102 # Saved stocks precede the observed choice; add its basket exactly once.
103 stocks = first[["stock_"+f for f in FLAVORS]].to_numpy(float)+shares
```

Read this block:

- Line 97, `first`: Set this name to the result of calling `.head(...)`; the arguments and preceding method chain are shown above.
- Line 98, `first`: Set this name to the entry/slice selected by `first.set_index('household_code').loc[households.househol...`; brackets select from the object to their left.
- Line 99, `counts`: Set this name to the result of calling `first[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 100, `totals`: Set this name to the result of calling `counts.sum(...)`; the arguments and preceding method chain are shown above.
- Line 101, `shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 103, `stocks`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 104–109:

```
104 novelty = shares.sum(axis=1, keepdims=True)-shares
105 prices = first[["price_"+f for f in FLAVORS]].to_numpy(float)
106 available = np.isfinite(prices) & (prices > 0)
107 eligible = available[:, 1]
108 households["berry_eligible"] = eligible
109 initial_weeks = first.week.to_numpy(dtype="datetime64[ns]")
```

Read this block:

- Line 104, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 105, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 106, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 107, `eligible`: Set this name to the entry/slice selected by `available[:, 1]`; brackets select from the object to their left.
- Line 108, `households['berry_eligible']`: Store the object currently named `eligible`; plain assignment does not copy it in the selected entry/column.
- Line 109, `initial_weeks`: Set this name to the result of calling `first.week.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 110–114:

```
110 households["initial_observed_week"] = initial_weeks
111 households["pricing_week1"] = initial_weeks+np.timedelta64(7, "D")
112 households["pricing_week2"] = initial_weeks+np.timedelta64(14, "D")
113 cf_cols = ["cf_"+f for f in FLAVORS]
114 cf = first[cf_cols].to_numpy(float) if all(c in first for c in cf_cols) else np.zeros_like(prices)
```

Read this block:

- Line 110, `households['initial_observed_week']`: Store the object currently named `initial_weeks`; plain assignment does not copy it in the selected entry/column.
- Line 111, `households['pricing_week1']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 112, `households['pricing_week2']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 113, `cf_cols`: Set this name to a list computed by the displayed for-expression.
- Line 114, `cf`: Set this name to the true-branch value if `all((c in first for c in cf_cols))` holds, otherwise the else-branch value.

Syntax and calls used here:

- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 115–116:

```
115 if not np.isfinite(stocks).all() or not np.isfinite(novelty).all():
116     raise ValueError("Nonfinite forecast history.")
```

Read this block:

- Line 115: test `not np.isfinite(stocks).all() or not np.isfinite(novelty).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 116: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

Source lines 117–118:

```
117 return dict(stocks=stocks, novelty=novelty, prices=np.where(available, prices, 0.),
118             available=available, cf=cf, eligible=eligible)
```

Read this block:

- Line 117: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

digest

Original function: `digest`. Source begins at line 120.

Streams file bytes through SHA-256 to identify the exact input or source used by a run. Hashes are provenance checks, not economic statistics. This function is called during an explicit empirical run, not by the documentation builder.

Definition and stated intent:

Source lines 120–120:

```
120 def digest(path):
```

`def` creates this callable; the parentheses name its inputs (`path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 121–121:

```
121 h = hashlib.sha256()
```

Read this block:

- Line 121, `h`: Set this name to the result of calling `hashlib.sha256(...)`; the arguments and preceding method chain are shown above.

Source lines 122–124:

```
122 with path.open("rb") as handle:
123     for chunk in iter(lambda: handle.read(1024*1024), b""):
124         h.update(chunk)
```

Read this block:

- Line 122: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 123: for each entry yielded by `iter(lambda: handle.read(1024 * 1024), b'')`, bind/unpack it into `chunk` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 124: call `h.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `iter`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 125–125:

```
125 return h.hexdigest()
```

Read this block:

- Line 125: return the result of calling `h.hexdigest(...)`; the arguments and preceding method chain are shown above. This ends the call.

targeted coupons

Original function: `targeted_coupons`. Source begins at line 128.

Research calculation. Solve terminal prices for every realized first basket, then choose the initial coupon using immediate revenue plus discounted optimized continuation revenue. In the readable version the outer loop visits one household; remaining axes are type, basket, coupon and revenue metric. Default known posterior mean types leave just one type; posterior-integrated mode keeps draws and updates their weights through the first-basket likelihood.

In split pricing, history_coefficient has two columns: history1=b_N$N1$+b_SS1, history2=b_N$N2$+b_SS2. Every basket updates own stock and other-flavor novelty. Both coefficients enter both dates. The solver's focal slot is coordinate one; run_households permutes the selected flavor into that coordinate. Rival quantities retain distinct identities for history updates.

Terminal coupons maximize revenue after integrating remaining unobserved type uncertainty, before averaging over realized baskets. The firm cannot condition on an unobserved true draw. Both focal and no-focal-purchase branches count. Bottom/top novelty groups get personalized offers; middle households remain at posted prices. No coupons supplies a feasible benchmark and optimized revenue must weakly dominate it. Coupon prices lie between zero and the posted price; they are not unrestricted theoretical prices.

Definition and stated intent:

Source lines 128–138:

```
128 def targeted_coupons(context, households, weights, alpha, gamma, beta, discounts,
129                      delta=1., capacity=7, habit_weight=.7, control_coefficient=0.,
130                      batch_size=24, type_treatment="posterior-mean", state_output=None):
131     """Observed h0, then p1(h0), realized h1, and terminal p2(h1).
132 
133     The observed initialization and two priced choices form three choice dates.
```

```
134     Individual backward induction; gamma quartiles are reporting groups only.
135
136     Baseline: posted prices in both weeks. Dynamic: choose first coupon accounting for optimized
137     state-contingent terminal revenue. The seller observes the full basket.
138     """
```

`def` creates this callable; the parentheses name its inputs (`context`, `households`, `weights`, `alpha`, `gamma`, `beta`, `discounts`, `delta`, `capacity`, `habit_weight`, `control_coefficient`, `batch_size`, `type_treatment`, `state_output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 139–144:

```
139 baskets = compositions(capacity)
140 inside = baskets[:, :3]
141 totals = inside.sum(axis=1, keepdims=True)
142 shares = np.divide(inside, totals, out=np.zeros_like(inside, dtype=float), where=totals > 0)
143 coeff = math.lgamma(capacity+1)-np.vectorize(math.lgamma)(baskets+1).sum(axis=1)
144 k, nstates = len(discounts), len(baskets)
```

Read this block:

- Line 139, `baskets`: Set this name to the result of calling `compositions(...)`; the arguments and preceding method chain are shown above.
- Line 140, `inside`: Set this name to the entry/slice selected by `baskets[:, :3]`; brackets select from the object to their left.
- Line 141, `totals`: Set this name to the result of calling `inside.sum(...)`; the arguments and preceding method chain are shown above.
- Line 142, `shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 143, `coeff`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 144, `(k, nstates)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 145–148:

```
145 rows, maximum_mass_error, wrote_states = [], 0., False
146 indices = np.flatnonzero(context["eligible"])
147 # Only the two tails receive targeting; middle households remain at posted prices.
148 targeted = households.group.isin(["low", "high"]).to_numpy()
```

Read this block:

- Line 145, `(rows, maximum_mass_error, wrote_states)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 146, `indices`: Set this name to the result of calling `np.flatnonzero(...)`; the arguments and preceding method chain are shown above.
- Line 148, `targeted`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.

Source lines 149–190:

```
for start in range(0, len(indices), batch_size):
    ids = indices[start:start+batch_size]
    h = len(ids)
    prices, availability = context["prices"][ids], context["available"][ids]
    stock, novelty, cf = context["stocks"][ids], context["novelty"][ids], context["cf"][ids]
    if type_treatment == "posterior-mean":
        # Base supply information assumption: an estimated fixed type is known.
        a, g = (weights[ids] @ alpha)[:, None], (weights[ids] @ gamma)[:, None]
        w = np.ones((h, 1))
    else:
        a = np.broadcast_to(alpha, (h, len(alpha)))
        g = np.broadcast_to(gamma, (h, len(gamma)))
        w = weights[ids]
    xi1 = (1-habit_weight)*novelty-habit_weight*stock
    xi2 = (1-habit_weight)*(shares.sum(axis=1, keepdims=True)-shares)[None, :, :] - habit_weight*(stock[:, None,
    :]+shares[None, :, :])
    u1 = beta[None, None, :]-a[..., None]*prices[:, None, :]+g[..., None]*xi1[:, None, :]+control_coefficient*cf[:,
    None, :]
    u2 = beta[None, None, None, :]-a[:, :, None, None]*prices[:, None, None, :]+g[:, :, None, None]*xi2[:, None, :,
    :]+control_coefficient*cf[:, None, None, :]
    u1 = np.where(availability[:, None, :], u1, -np.inf)
    u2 = np.where(availability[:, None, None, :], u2, -np.inf)
    terminal = []
    for discount in discounts:
        u = u2.copy()
        u[..., 1] += a[:, :, None]*prices[:, None, None, 1]*discount
        denominator = logsumexp(np.concatenate([u, np.zeros((*u.shape[:-1], 1))], axis=-1), axis=-1)
        quantity = capacity*np.exp(u[..., 1]-denominator)
        terminal.append(np.stack([quantity*prices[:, None, None, 1]*(1-discount), quantity,
                                  quantity*prices[:, None, None, 1]*discount], axis=-1))
    # Metrics: net revenue, berry units, coupon expenditure.
    first = np.zeros((h, k, 3))
    second = np.zeros((h, k, 3))
    stage0_second = None
    paths = []
    for i, discount in enumerate(discounts):
        u = u1.copy()
        u[..., 1] += a*prices[:, None, 1]*discount
        full = np.concatenate([u, np.zeros((*u.shape[:-1], 1))], axis=-1)
        logp = full-logsumexp(full, axis=-1, keepdims=True)
        terms = np.zeros((h, a.shape[1], nstates, 4))
        np.multiply(baskets[None, None, :, :], logp[:, :, None, :], out=terms,
                    where=baskets[None, None, :, :] > 0)
        probability = np.exp(coeff[None, None, :]+terms.sum(axis=-1))
        maximum_mass_error = max(maximum_mass_error, float(np.abs(probability.sum(axis=-1)-1).max()))
```

Source lines 191–232:

```
    joint = w[:, :, None]*probability
    state_probability = joint.sum(axis=1)
    quantity = capacity*(w*np.exp(logp[..., 1])).sum(axis=1)
    first[:, i] = np.stack([quantity*prices[:, 1]*(1-discount), quantity,
                            quantity*prices[:, 1]*discount], axis=-1)
    # Integrate types BEFORE choosing a terminal coupon: no foresight of
    # an unobserved true draw in the posterior-integrated sensitivity.
    metrics = np.stack([np.einsum("hds,hdsm->hsm", joint, candidate)
                        for candidate in terminal], axis=2)  # household,state,action,metric
    best = metrics[..., 0].argmax(axis=2)
    selected = np.take_along_axis(metrics, best[:, :, None, None], axis=2)[:, :, 0, :]
    # E_h1[p2(h1)*E(q2|h1,p2,type)] under this candidate p1(h0).
```

```
    # Optimize p2 separately in each state BEFORE summing its revenue.
    second[:, i] = selected.sum(axis=1)
    if i == 0:
        stage0_second = metrics[:, :, 0, :].sum(axis=1)
    # Storing these compact arrays allows exporting only chosen first actions.
    paths.append((best, selected, state_probability))
zero_choice = np.zeros(h, dtype=int)
dynamic_choice = (first[..., 0]+delta*second[..., 0]).argmax(axis=1)
dynamic_choice = np.where(targeted[ids], dynamic_choice, 0)
for name, choice in [("no_coupon", zero_choice), ("personalized_dynamic", dynamic_choice)]:
    for local, hh in enumerate(ids):
        action = int(choice[local])
        initial = first[local, action]
        is_target = targeted[hh] and name != "no_coupon"
        terminal_metrics = second[local, action] if is_target else stage0_second[local]
        best, selected, probability = paths[action]
        state_coupon = discounts[best[local]] if is_target else np.zeros(nstates)
        mass = probability[local]
        p1 = prices[local, 1]*(1-discounts[action])
        p2 = prices[local, 1]*(1-state_coupon)
        mean_p2 = float(mass @ p2)
        ratio = np.divide(p1, p2, out=np.full(nstates, np.nan), where=p2 > 0)
        positive_mass = mass > 1e-14
        expected_ratio = float(mass[positive_mass] @ ratio[positive_mass]) if
 ↪  np.isfinite(ratio[positive_mass]).all() else np.nan
        row = dict(household_code=households.iloc[hh].household_code,
                   group=households.iloc[hh].group, policy=name,
                   discount1=discounts[action], expected_discount2=float(mass @ state_coupon),
                   p1=p1, expected_p2=mean_p2,
                   ratio_of_expected_prices=p1/mean_p2 if mean_p2 > 0 else np.nan,
                   expected_price_ratio=expected_ratio,
```

Source lines 233–251:

```
                   revenue1=initial[0], revenue2=terminal_metrics[0],
                   discounted_revenue=initial[0]+delta*terminal_metrics[0],
                   berry_units1=initial[1], berry_units2=terminal_metrics[1],
                   coupon_expenditure=initial[2]+delta*terminal_metrics[2])
        for branch, label in [(baskets[:, 1] == 0, "no_berry"), (baskets[:, 1] > 0, "berry")]:
            branch_mass = mass[branch].sum()
            row[f"probability_{label}"] = branch_mass
            row[f"mean_p2_given_{label}"] = float(mass[branch] @ p2[branch]/branch_mass) if branch_mass > 0 else
 ↪  np.nan
        rows.append(row)
        if state_output is not None and is_target:
            detail = pd.DataFrame(dict(household_code=row["household_code"], group=row["group"], policy=name,
                                       state=np.arange(nstates), other=baskets[:, 0], berry=baskets[:, 1],
                                       plain=baskets[:, 2], outside=baskets[:, 3], probability=mass,
                                       discount1=discounts[action], discount2=state_coupon, p1=p1, p2=p2,
 ↪  price_ratio=ratio))
            detail.to_csv(state_output, mode="a" if wrote_states else "w", header=not wrote_states,
                          index=False, compression="gzip")
            wrote_states = True
if start % (batch_size*40) == 0 or start+batch_size >= len(indices):
    print(f"Targeted pricing: integrated {min(start+batch_size,len(indices))}/{len(indices)} eligible households",
     ↪  flush=True)
```

Read this block:

- Line 149: for each entry yielded by `range(0, len(indices), batch_size)`, bind/unpack it into `start` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 150, `ids`: Set this name to the entry/slice selected by `indices[start:start + batch_size]`; brackets

select from the object to their left.
- Line 151, `h`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 152, `(prices, availability)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 153, `(stock, novelty, cf)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 154: test `type_treatment == 'posterior-mean'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 156, `(a, g)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 157, `w`: Set this name to the result of calling `np.ones(...)`; the arguments and preceding method chain are shown above.
- Line 159, `a`: Set this name to the result of calling `np.broadcast_to(...)`; the arguments and preceding method chain are shown above.
- Line 160, `g`: Set this name to the result of calling `np.broadcast_to(...)`; the arguments and preceding method chain are shown above.
- Line 161, `w`: Set this name to the entry/slice selected by `weights[ids]`; brackets select from the object to their left.
- Line 162, `xi1`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 163, `xi2`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 164, `u1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 165, `u2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 166, `u1`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 167, `u2`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 168, `terminal`: Set this name to the list `[]`.
- Line 169: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 170, `u`: Set this name to the result of calling `u2.copy(...)`; the arguments and preceding method chain are shown above.
- Line 171: update `u[..., 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 172, `denominator`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 173, `quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 174: call `terminal.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 177, `first`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 178, `second`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 179, `stage0_second`: Set this name to the literal value `None`.
- Line 180, `paths`: Set this name to the list `[]`.
- Line 181: for each entry yielded by `enumerate(discounts)`, bind/unpack it into `(i, discount)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 182, `u`: Set this name to the result of calling `u1.copy(...)`; the arguments and preceding method chain are shown above.
- Line 183: update `u[..., 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 184, `full`: Set this name to the result of calling `np.concatenate(...)`; the arguments and preceding method chain are shown above.
- Line 185, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subex-

pressions are evaluated together.
- Line 186, `terms`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 187: call `np.multiply` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 189, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 190, `maximum_mass_error`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 191, `joint`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 192, `state_probability`: Set this name to the result of calling `joint.sum(...)`; the arguments and preceding method chain are shown above.
- Line 193, `quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 194, `first[:, i]`: Store the result of calling `np.stack(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 198, `metrics`: Set this name to the result of calling `np.stack(...)`; the arguments and preceding method chain are shown above.
- Line 200, `best`: Set this name to the result of calling `metrics[..., 0].argmax(...)`; the arguments and preceding method chain are shown above.
- Line 201, `selected`: Set this name to the entry/slice selected by `np.take_along_axis(metrics, best[:, :, None, None], axis=...`; brackets select from the object to their left.
- Line 204, `second[:, i]`: Store the result of calling `selected.sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 205: test `i == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 206, `stage0_second`: Set this name to the result of calling `metrics[:, :, 0, :].sum(...)`; the arguments and preceding method chain are shown above.
- Line 208: call `paths.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 209, `zero_choice`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 210, `dynamic_choice`: Set this name to the result of calling `.argmax(...)`; the arguments and preceding method chain are shown above.
- Line 211, `dynamic_choice`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 212: for each entry yielded by `[('no_coupon', zero_choice), ('personalized_dynamic', dynamic_...`, bind/unpack it into `(name, choice)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 213: for each entry yielded by `enumerate(ids)`, bind/unpack it into `(local, hh)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 214, `action`: Set this name to the result of calling `int(...)`; the arguments and preceding method chain are shown above.
- Line 215, `initial`: Set this name to the entry/slice selected by `first[local, action]`; brackets select from the object to their left.
- Line 216, `is_target`: Set this name to the Boolean condition `targeted[hh] and name != 'no_coupon'`; it can be a scalar or a row/array mask depending on its inputs.
- Line 217, `terminal_metrics`: Set this name to the true-branch value if `is_target` holds, otherwise the else-branch value.
- Line 218, `(best, selected, probability)`: Unpack the entry/slice selected by `paths[action]`; brackets select from the object to their left into these targets in order.
- Line 219, `state_coupon`: Set this name to the true-branch value if `is_target` holds, otherwise the else-branch value.
- Line 220, `mass`: Set this name to the entry/slice selected by `probability[local]`; brackets select from the object to their left.

- Line 221, `p1`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 222, `p2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 223, `mean_p2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 224, `ratio`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 225, `positive_mass`: Set this name to the Boolean condition `mass > 1e-14`; it can be a scalar or a row/array mask depending on its inputs.
- Line 226, `expected_ratio`: Set this name to the true-branch value if `np.isfinite(ratio[positive_mass]).all()` holds, otherwise the else-branch value.
- Line 227, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 237: for each entry yielded by `[(baskets[:, 1] == 0, 'no_berry'), (baskets[:, 1] > 0, 'berry')]`, bind/unpack it into `(branch, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 238, `branch_mass`: Set this name to the result of calling `mass[branch].sum(...)`; the arguments and preceding method chain are shown above.
- Line 239, `row[f'probability_{label}']`: Store the object currently named `branch_mass`; plain assignment does not copy it in the selected entry/column.
- Line 240, `row[f'mean_p2_given_{label}']`: Store the true-branch value if `branch_mass > 0` holds, otherwise the else-branch value in the selected entry/column.
- Line 241: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 242: test `state_output is not None and is_target`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 243, `detail`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 247: call `detail.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 249, `wrote_states`: Set this name to the literal value `True`.
- Line 250: test `start % (batch_size * 40) == 0 or start + batch_size >= len(indices)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 251: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.einsum`: Perform an explicitly labeled sum of products; rh,rhp->p sums draws/households and retains the parameter axis.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 252–253:

```
252 if maximum_mass_error > 1e-9:
253     raise AssertionError(f"Basket probability mass error: {maximum_mass_error}")
```

Read this block:

- Line 252: test `maximum_mass_error > 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 253: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 254–259:

```
254 table = pd.DataFrame(rows)
255 baseline = table[table.policy == "no_coupon"].set_index("household_code").discounted_revenue
256 table["baseline_revenue"] = table.household_code.map(baseline)
257 table["revenue_change"] = table.discounted_revenue-table.baseline_revenue
258 table["revenue_change_pct"] = 100*table.revenue_change/table.baseline_revenue
259 wide = table.pivot(index="household_code", columns="policy", values="discounted_revenue")
```

Read this block:

- Line 254, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 255, `baseline`: Set this name to the attribute `table[table.policy == 'no_coupon'].set_index('household_c...` of the object before the dot.
- Line 256, `table['baseline_revenue']`: Store the result of calling `table.household_code.map(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 257, `table['revenue_change']`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 258, `table['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 259, `wide`: Set this name to the result of calling `table.pivot(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.

Source lines 260–261:

```
260 if (wide.personalized_dynamic+1e-8 < wide.no_coupon).any():
261     raise AssertionError("Household dynamic policy does not dominate the feasible no-coupon baseline.")
```

Read this block:

- Line 260: test `(wide.personalized_dynamic + 1e-08 < wide.no_coupon).any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 261: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 262–262:

```
262 return table, maximum_mass_error
```

Read this block:

- Line 262: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

run households

Original function: `run_households`. Source begins at line 264.

Reads an explicitly selected fitted-results directory and its weekly table, checks the model, convergence, bounds, capacity and input schema, then recovers posterior types and reproduces the saved likelihood. For a split run, it permutes the selected focal flavor into the solver's berry coordinate, moving counts, prices, stocks, novelty, controls and intercepts consistently. Type recovery happens before permutation using the original fitted flavor labels.

It calls the household solver, renames public berry-coordinate columns to focal names, and writes policy/group/aggregate tables and figures. The manifest records how rival state coordinates map back to flavors. Swapping only prices or only counts would silently price the wrong product. The three runs are separate focal sellers with fixed rival prices, not a simultaneous equilibrium.

Definition and stated intent:

Source lines 264–266:

```
264 def run_households(args):
265     # This branch is entered only when the person running the script supplies
266     # BOTH paths. Validation supplies generated temporary fixtures only.
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 267–267:

```
267 fit = json.loads((args.fit_dir / "fit.json").read_text())
```

Read this block:

- Line 267, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.

Source lines 268–269:

```
268 if fit["model"] != "heterogeneous" or not fit.get("converged", False):
269     raise ValueError("This exercise requires a converged single-normal heterogeneous-gamma fit.")
```

Read this block:

- Line 268: test `fit['model'] != 'heterogeneous' or not fit.get('converged', False)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 269: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 270–270:

```
270 estimates = pd.read_csv(args.fit_dir / "point_estimates.csv")
```

Read this block:

- Line 270, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 271–272:

```
271 if "at_bound" in estimates and estimates.at_bound.astype(str).str.lower().eq("true").any():
272     raise ValueError("Fit has boundary parameters; review before running counterfactuals.")
```

Read this block:

- Line 271: test `'at_bound' in estimates and estimates.at_bound.astype(str).str.lower().eq('tr...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 272: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

Source lines 273–275:

```
273 parameters = dict(zip(estimates.parameter, estimates.estimate))
274 settings = fit["settings"]
275 capacity = settings["capacity"]
```

Read this block:

- Line 273, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 274, `settings`: Set this name to the entry/slice selected by `fit['settings']`; brackets select from the object to their left.

- Line 275, `capacity`: Set this name to the entry/slice selected by `settings['capacity']`; brackets select from the object to their left.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 276–277:

```
276 if not isinstance(capacity, int) or not 1 <= capacity <= 12:
277     raise ValueError("Exact basket enumeration supports integer capacity 1..12.")
```

Read this block:

- Line 276: test `not isinstance(capacity, int) or not 1 <= capacity <= 12`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 277: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 278–281:

```
278 frame = pd.read_csv(args.household_input, dtype={"household_code": str}, parse_dates=["week"])
279 required = ["household_code", "week", "outside", *FLAVORS]
280 required += [prefix+f for prefix in ["price_", "stock_", "novelty_", "xi_"] for f in FLAVORS]
281 control = not settings.get("no_price_control", False)
```

Read this block:

- Line 278, `frame`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 279, `required`: Set this name to a list containing 4 displayed entries.
- Line 280: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 281, `control`: Set this name to the negated or sign-adjusted expression `not settings.get('no_price_control', False)`.

Syntax and calls used here:

- A starred entry expands an iterable into separate elements, here within a list/tuple or call.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 282–283:

```
282 if control:
283     required += ["cf_"+f for f in FLAVORS]
```

Read this block:

- Line 282: test `control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 283: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 284–284:

```
284 missing = set(required)-set(frame)
```

Read this block:

- Line 284, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 285–286:

```
285 if missing:
286     raise ValueError(f"Household input missing columns: {sorted(missing)}")
```

Read this block:

- Line 285: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 286: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 287–288:

```
287 if frame.household_code.isna().any() or frame.week.isna().any() or frame.duplicated(["household_code",
    ↪ "week"]).any():
288     raise ValueError("Need unique household/week rows with nonmissing IDs/dates.")
```

Read this block:

- Line 287: test `frame.household_code.isna().any() or frame.week.isna().any() or frame.duplica...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 288: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.

Source lines 289–290:

```
289 if len(frame) != fit["weeks"] or frame.household_code.nunique() != fit["households"]:
290     raise ValueError("Input household/week counts do not match fit metadata.")
```

Read this block:

- Line 289: test `len(frame) != fit['weeks'] or frame.household_code.nunique() != fit['househol...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 290: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.

Source lines 291–291:

```
291 weight = settings.get("habit_weight", .7)
```

Read this block:

- Line 291, `weight`: Set this name to the result of calling `settings.get(...)`; the arguments and preceding method chain are shown above.

Source lines 292–295:

```
292 for f in FLAVORS:
293     expected = (1-weight)*frame["novelty_"+f]-weight*frame["stock_"+f]
294     if not np.allclose(expected, frame["xi_"+f], atol=1e-8):
295         raise ValueError("Saved Xi does not match the fit's history definition.")
```

Read this block:

- Line 292: for each entry yielded by `FLAVORS`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 293, `expected`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 294: test `not np.allclose(expected, frame['xi_' + f], atol=1e-08)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 295: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.

Source lines 296–297:

```
296 draws = persistent_draws(settings["draws"], settings["seed"])
297 households, weights, alpha, gamma, beta, nll = posterior(frame, parameters, draws, control)
```

Read this block:

- Line 296, `draws`: Set this name to the result of calling `persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 297, `(households, weights, alpha, gamma, beta, nll)`: Unpack the result of calling `posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `persistent_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `posterior`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 299–300:

```
299 if not np.isclose(nll, fit["nll"], rtol=1e-9, atol=1e-4):
300     raise ValueError(f"Likelihood mismatch: reconstructed {nll}, fit {fit['nll']}.")
```

Read this block:

- Line 299: test `not np.isclose(nll, fit['nll'], rtol=1e-09, atol=0.0001)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 300: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 301–302:

```
301 context = forecast_context(frame, households)
302 counts = [int(((households.group == g) & households.berry_eligible).sum()) for g in ["low", "middle", "high"]]
```

Read this block:

- Line 301, `context`: Set this name to the result of calling `forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 302, `counts`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `forecast_context`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 303–304:

```
303 if any(n == 0 for n in counts):
304     raise ValueError("Each gamma segment must have berry-eligible forecast households.")
```

Read this block:

- Line 303: test `any((n == 0 for n in counts))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 304: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 305–316:

```
305 table, mass_error = targeted_coupons(
306     context, households, weights, alpha, gamma, beta, args.discounts,
307     args.demand_delta, capacity, weight,
308     parameters.get("price_control", 0.) if control else 0., args.batch_size,
309     args.type_treatment,
310     args.output / "targeted_state_policies.csv.gz" if args.save_state_policies else None)
311 households.to_csv(args.output / "household_posterior_summary.csv", index=False)
312 table.to_csv(args.output / "targeted_household_results.csv", index=False)
313 totals = ["revenue1", "revenue2", "discounted_revenue", "baseline_revenue", "revenue_change",
314           "berry_units1", "berry_units2", "coupon_expenditure"]
315 aggregate = table.groupby("policy", sort=False)[totals].sum()
316 aggregate["revenue_change_pct"] = 100*aggregate.revenue_change/aggregate.baseline_revenue
```

Read this block:

- Line 305, `(table, mass_error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 311: call `households.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 312: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 313, `totals`: Set this name to a list containing 8 displayed entries.
- Line 315, `aggregate`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 316, `aggregate['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `targeted_coupons`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.

Source lines 317–323:

```
317 aggregate.to_csv(args.output / "targeted_revenue_aggregate.csv")
318 group = table.groupby(["policy", "group"], sort=False)[totals].sum()
319 group["revenue_change_pct"] = 100*group.revenue_change/group.baseline_revenue
320 means = table.groupby(["policy", "group"])[["p1", "expected_p2", "discount1", "expected_discount2",
321                                              "ratio_of_expected_prices", "expected_price_ratio"]].mean()
322 group = group.join(means)
323 group["eligible_households"] = table.groupby(["policy", "group"]).size()
```

Read this block:

- Line 317: call `aggregate.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 318, `group`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 319, `group['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 320, `means`: Set this name to the result of calling `.mean(...)`; the arguments and preceding method chain are shown above.
- Line 322, `group`: Set this name to the result of calling `group.join(...)`; the arguments and preceding method chain are shown above.
- Line 323, `group['eligible_households']`: Store the result of calling `.size(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.

Source lines 324–333:

```
324 group.to_csv(args.output / "targeted_revenue_by_gamma_group.csv")
325 households.groupby("group").agg(households=("household_code", "size"),
326                                  berry_eligible=("berry_eligible", "sum"),
327                                  gamma_mean=("gamma_mean", "mean"),
328                                  gamma_mean_min=("gamma_mean", "min"),
329                                  gamma_mean_max=("gamma_mean", "max"),
330                                  mean_posterior_sd=("gamma_sd", "mean"),
331                                  mean_alpha=("alpha_mean", "mean"),
332                                  median_posterior_ess=("posterior_ess", "median")).to_csv(args.output /
    ↪ "household_group_summary.csv")
333 make_targeting_figures(households, table, aggregate, args.output, delta=args.demand_delta)
```

Read this block:

- Line 324: call `group.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 325: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 333: call `make_targeting_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `make_targeting_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 334–344:

```
334 return dict(households=len(households), eligible_by_group=dict(zip(["low", "middle", "high"], counts)),
335             reconstructed_nll=nll, basket_mass_error=mass_error,
336             input_sha256=digest(args.household_input), fit_sha256=digest(args.fit_dir / "fit.json"),
337             parameter_sha256=digest(args.fit_dir / "point_estimates.csv"),
338             posterior_draws=len(draws), posterior_seed=settings["seed"],
339             capacity=capacity, habit_weight=weight, price_control=control,
340             initial_history="first_observed_basket", pricing_periods=2,
341             baskets=len(compositions(capacity)),
342             discount_grid=args.discounts.tolist(), demand_delta=args.demand_delta,
343             type_treatment=args.type_treatment,
344             aggregate=aggregate.reset_index().to_dict("records"))
```

Read this block:

- Line 334: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.

make targeting figures

Original function: `make_targeting_figures`. Source begins at line 347.

Draws household posterior switching types, discounted dynamic revenue by reporting group, and coupon patterns. Split figures use b_N labels and the selected focal flavor. Revenue is R1+delta*R2 with terminal revenues averaged after state-dependent optimization; mean prices times mean quantities would not give the same object.

Definition and stated intent:

Source lines 347–348:

```
347 def make_targeting_figures(households, table, aggregate, output, delta=.95):
348     """Plot household types and individually optimized berry discounts."""
```

`def` creates this callable; the parentheses name its inputs (`households`, `table`, `aggregate`, `output`, `delta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 349–354:

```
349 plt = plotting()
350 groups = ["low", "middle", "high"]
351 labels = ["Low gamma\n(bottom 25%)", "Middle gamma\n(middle 50%)", "High gamma\n(top 25%)"]
352 colors = ["#3575AD", "#999999", "#D17925"]
353 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
354 bins = np.histogram_bin_edges(households.gamma_mean, bins=30)
```

Read this block:

- Line 349, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.
- Line 350, `groups`: Set this name to the list `['low', 'middle', 'high']`.
- Line 351, `labels`: Set this name to the list `['Low gamma\n(bottom 25%)', 'Middle gamma\n(middle 50%)',...`.
- Line 352, `colors`: Set this name to the list `['#3575AD', '#999999', '#D17925']`.
- Line 353, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 354, `bins`: Set this name to the result of calling `np.histogram_bin_edges(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 355–357:

```
355 for g, label, color in zip(groups, labels, colors):
356     axes[0].hist(households.loc[households.group == g, "gamma_mean"],
357                  bins=bins, alpha=.65, label=label.replace("\n", " "), color=color)
```

Read this block:

- Line 355: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 356: call `axes[0].hist` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.hist`: Bin observed values, with optional weights controlling whether heights are counts, percentages, or another unit.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 358–365:

```
358 axes[0].set(title="Estimated consumer types", xlabel=r"Estimated household type (posterior mean $\gamma$)",
359             ylabel="Number of households")
360 axes[0].legend(fontsize=9)
361 revenue = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["revenue1",
    ↪  "revenue2"]].sum().reindex(groups)
362 x = np.arange(len(groups))
363 # Average over state-contingent terminal prices first, discount, then total.
364 total_expected_revenue = revenue.revenue1 + delta * revenue.revenue2
365 axes[1].bar(x, total_expected_revenue, width=.5, color=colors)
```

Read this block:

- Line 358: call `axes[0].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 360: call `axes[0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 361, `revenue`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 362, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.

- Line 364, `total_expected_revenue`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 365: call `axes[1].bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `len`: Count elements/rows rather than summing their numerical values.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.

Source lines 366–372:

```
366 axes[1].set_xticks(x, labels)
367 axes[1].set(title="Total Expected Revenue",
368            xlabel="Consumer type group", ylabel="Total Expected Revenue")
369 axes[1].set_ylim(bottom=0)
370 axes[1].grid(axis="y", alpha=.2)
371 axes[1].set_axisbelow(True)
372 axes[1].axhline(0, color="gray", linewidth=.6)
```

Read this block:

- Line 366: call `axes[1].set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 367: call `axes[1].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 369: call `axes[1].set_ylim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 370: call `axes[1].grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 371: call `axes[1].set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 372: call `axes[1].axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.grid`: Add visual grid lines without changing underlying numerical results.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.

Source lines 373–377:

```
373 fig.text(.5, .015,
374          f"First-period revenue + {delta:g} × expected terminal revenue. "
375          "Low/high types receive personalized berry offers; middle types receive no coupons.",
376          ha="center", fontsize=9)
377 fig.tight_layout(rect=(0, .05, 1, 1))
```

Read this block:

- Line 373: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 377: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 378–379:

```
378 for extension in ["png", "pdf"]:
379     fig.savefig(output / f"household_coupon_results.{extension}", dpi=180)
```

Read this block:

- Line 378: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 379: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 380–387:

```
380 plt.close(fig)
381
382 fig, ax = plt.subplots(figsize=(9, 5))
383 x = np.arange(3)
384 means = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["discount1",
    ↪  "expected_discount2"]].mean().reindex(groups)
385 ax.bar(x-.18, 100*means.discount1, width=.36, label=r"$p_1(h_0)$ offer", color=colors[0])
386 ax.bar(x+.18, 100*means.expected_discount2, width=.36,
387        label=r"Terminal $p_2(h_1)$ offer (expected over histories)", color=colors[2])
```

Read this block:

- Line 380: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 382, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 383, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 384, `means`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 385: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 386: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 388–394:

```
388 ax.set_xticks(x, labels)
389 ax.set(xlabel="Consumer type group", ylim=(0, 105),
390        ylabel="Mean optimal berry discount (% of posted price)")
391 ax.grid(axis="y", alpha=.2)
```

```
ax.set_axisbelow(True)
ax.legend(fontsize=8, loc="upper left")
fig.suptitle("Dynamic pricing: optimal discounts by consumer type")
```

Read this block:

- Line 388: call `ax.set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 389: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 391: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 392: call `ax.set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 393: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 394: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 395–397:

```
fig.text(.5, .015, "Household-weighted means among berry-eligible households. Middle group receives no coupons by policy design.",
         ha="center", fontsize=9)
fig.tight_layout(rect=(0, .05, 1, .94))
```

Read this block:

- Line 395: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 397: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 398–399:

```
for extension in ["png", "pdf"]:
    fig.savefig(output / f"discounts_by_consumer_type.{extension}", dpi=180)
```

Read this block:

- Line 398: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 399: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 400–404:

```
plt.close(fig)

dynamic = table.loc[table.policy == "personalized_dynamic"].merge(
    households[["household_code", "gamma_mean", "alpha_mean"]], on="household_code", validate="one_to_one")
fig, axes = plt.subplots(2, 2, figsize=(13, 9), sharey=True)
```

Read this block:

- Line 400: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 402, `dynamic`: Set this name to the result of calling `.merge(...)`; the arguments and preceding method chain are shown above.
- Line 404, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 405–417:

```
405 for column, (variable, xlabel) in enumerate([
406         ("gamma_mean", r"Estimated household type (posterior mean $\gamma$)"),
407         ("alpha_mean", r"Estimated price sensitivity (posterior mean $a$)")]):
408     for row, (discount, title) in enumerate([
409             ("discount1", r"Optimal discount at $p_1(h_0)$"),
410             ("expected_discount2", r"Expected terminal discount at $p_2(h_1)$")]):
411         ax = axes[row, column]
412         for g, label, color in zip(groups, labels, colors):
413             subset = dynamic.loc[dynamic.group == g]
414             ax.scatter(subset[variable], 100*subset[discount], s=12, alpha=.5,
415                        color=color, label=label.replace("\n", " "))
416         ax.set(xlabel=xlabel, ylabel="Berry discount (% of posted price)", title=title, ylim=(-2, 102))
417         ax.grid(alpha=.2)
```

Read this block:

- Line 405: for each entry yielded by `enumerate([('gamma_mean', 'Estimated household type (posterior...`, bind/unpack it into `(column, (variable, xlabel))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 408: for each entry yielded by `enumerate([('discount1', 'Optimal discount at $p_1(h_0)$'), ('...`, bind/unpack it into `(row, (discount, title))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 411, `ax`: Set this name to the entry/slice selected by `axes[row, column]`; brackets select from the object to their left.
- Line 412: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 413, `subset`: Set this name to the entry/slice selected by `dynamic.loc[dynamic.group == g]`; brackets select from the object to their left.
- Line 414: call `ax.scatter` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 416: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 417: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 418–422:

```
418 axes[0, 0].legend(fontsize=8)
419 fig.suptitle("Dynamic pricing: household discounts and estimated consumer characteristics")
420 fig.text(.5, .015, "Each point is one berry-eligible household. Terminal offers average over the household's
    ↪  realized h1 histories.",
421          ha="center", fontsize=9)
422 fig.tight_layout(rect=(0, .04, 1, .95))
```

Read this block:

- Line 418: call `axes[0, 0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 419: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 420: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 422: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 423–424:

```
423 for extension in ["png", "pdf"]:
424     fig.savefig(output / f"discounts_by_household_type.{extension}", dpi=180)
```

Read this block:

- Line 423: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 424: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 425–425:

```
425 plt.close(fig)
```

Read this block:

- Line 425: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

self test

Original function: `self_test`. Source begins at line 428.

Runs generated examples and assertions to check numerical calculations without launching the user-selected empirical exercise. Read the specific body to distinguish the CSV-blocked simple split checks from earlier complete export tests that reopen their own synthetic CSVs. No private input is needed.

Definition and stated intent:

Source lines 428–429:

```
428 def self_test():
429     """Generated synthetic households only; no user-supplied input is accessed."""
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 430–434:

```
430 rng = np.random.default_rng(41)
431 parameters = dict(inside_constant=.8, beta_berry=-.2, beta_plain=-.1,
432                   mu_log_minus_alpha=-.3, sd_log_minus_alpha=.4,
433                   mean_gamma=-.1, sd_gamma=.2)
434 rows = []
```

Read this block:

- Line 430, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 431, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 434, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 435–448:

```
435 for household in range(12):
436     stock, previous = np.zeros(3), np.zeros(3)
437     for t in range(3):
438         counts = rng.multinomial(2, [.25, .3, .2, .25])
439         novelty = previous.sum()-previous
440         row = dict(household_code=f"synthetic_{household:02}",
441                    week=pd.Timestamp("2020-01-04")+pd.Timedelta(weeks=t), outside=counts[3])
442         for j, flavor in enumerate(FLAVORS):
443             row.update({flavor: counts[j], "price_"+flavor: 1.+.1*j,
444                         "stock_"+flavor: stock[j], "novelty_"+flavor: novelty[j],
445                         "xi_"+flavor: .3*novelty[j]-.7*stock[j]})
446         rows.append(row)
447         previous = counts[:3]/max(counts[:3].sum(), 1)
448         stock += previous
```

Read this block:

- Line 435: for each entry yielded by `range(12)`, bind/unpack it into `household` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 436, `(stock, previous)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 437: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 438, `counts`: Set this name to the result of calling `rng.multinomial(...)`; the arguments and preceding method chain are shown above.
- Line 439, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 440, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 442: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 443: call `row.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 446: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 447, `previous`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 448: update `stock` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.multinomial`: Draw a vector of category counts summing to the specified total, given category probabilities.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 449–451:

```
449 frame = pd.DataFrame(rows)
450 draws = persistent_draws(8, 19)
451 households, weights, alpha, gamma, beta, nll = posterior(frame, parameters, draws, False)
```

Read this block:

- Line 449, `frame`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 450, `draws`: Set this name to the result of calling `persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 451, `(households, weights, alpha, gamma, beta, nll)`: Unpack the result of calling `posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `persistent_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `posterior`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 452–452:

```
452 assert np.allclose(weights.sum(axis=1), 1.)
```

Read this block:

- Line 452: require `np.allclose(weights.sum(axis=1), 1.0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.

Source lines 453–453:

```
453 scalar_ll = []
```

Read this block:

- Line 453, `scalar_ll`: Set this name to the list `[]`.

Source lines 454–459:

```
454 for a, g in zip(alpha, gamma):
455     value = 0.
456     for row in rows[:3]:
457         u = [beta[j]-a*row["price_"+f]+g*row["xi_"+f] for j, f in enumerate(FLAVORS)]+[0.]
458         value += np.array([row[f] for f in FLAVORS]+[row["outside"]]) @ (np.array(u)-logsumexp(u))
459     scalar_ll.append(value)
```

Read this block:

- Line 454: for each entry yielded by `zip(alpha, gamma)`, bind/unpack it into `(a, g)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 455, `value`: Set this name to the literal value `0.0`.
- Line 456: for each entry yielded by `rows[:3]`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 457, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 458: update `value` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

- Line 459: call `scalar_ll.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 460–460:

```
460 assert np.allclose(weights[0], np.exp(scalar_ll-logsumexp(scalar_ll)))
```

Read this block:

- Line 460: require `np.allclose(weights[0], np.exp(scalar_ll - logsumexp(scalar_ll)))` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 461–462:

```
461 context = forecast_context(frame, households)
462 discounts = np.array([0., .25, .5])
```

Read this block:

- Line 461, `context`: Set this name to the result of calling `forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 462, `discounts`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `forecast_context`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 463–470:

```
463 for treatment in ["posterior-mean", "posterior-integrated"]:
464     table, error = targeted_coupons(context, households, weights, alpha, gamma, beta, discounts,
465                                     capacity=2, batch_size=4, type_treatment=treatment)
466     assert error < 1e-12 and len(table) == 24
467     # Accounting and treating the middle segment as a holdout.
468     assert np.allclose(table.discounted_revenue+table.coupon_expenditure,
469                        1.1*(table.berry_units1+table.berry_units2))
470     assert np.allclose(table.loc[table.group == "middle", "revenue_change"], 0.)
```

Read this block:

- Line 463: for each entry yielded by `['posterior-mean', 'posterior-integrated']`, bind/unpack it into `treatment` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 464, `(table, error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.

- Line 466: require `error < 1e-12 and len(table) == 24` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 468: require `np.allclose(table.discounted_revenue + table.coupon_expenditure, 1.1 * (table.berr...` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 470: require `np.allclose(table.loc[table.group == 'middle', 'revenue_change'], 0.0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `targeted_coupons`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 471–472:

```
471 no_history, _ = targeted_coupons(context, households, weights, np.ones_like(alpha),
472                                  np.zeros_like(gamma), beta, discounts, capacity=2)
```

Read this block:

- Line 471, `(no_history, _)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 473–473:

```
473 assert np.allclose(no_history.revenue1, no_history.revenue2)
```

Read this block:

- Line 473: require `np.allclose(no_history.revenue1, no_history.revenue2)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 474–474:

```
474 assert np.allclose(no_history.discount1, no_history.expected_discount2)
```

Read this block:

- Line 474: require `np.allclose(no_history.discount1, no_history.expected_discount2)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 476–479:

```
476 swapped = households.copy()
477 swapped["group"] = swapped.group.replace({"low": "high", "high": "low"})
478 changed, _ = targeted_coupons(context, swapped, weights, alpha, gamma, beta, discounts, capacity=2)
479 original, _ = targeted_coupons(context, households, weights, alpha, gamma, beta, discounts, capacity=2)
```

Read this block:

- Line 476, `swapped`: Set this name to the result of calling `households.copy(...)`; the arguments and preceding method chain are shown above.
- Line 477, `swapped['group']`: Store the result of calling `swapped.group.replace(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

- Line 478, `(changed, _)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 479, `(original, _)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 480–480:

```
480 assert np.allclose(changed.discount1, original.discount1)
```

Read this block:

- Line 480: require `np.allclose(changed.discount1, original.discount1)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 481–481:

```
481 assert np.allclose(changed.discounted_revenue, original.discounted_revenue)
```

Read this block:

- Line 481: require `np.allclose(changed.discounted_revenue, original.discounted_revenue)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 482–482:

```
482 assert len(compositions(7)) == 120
```

Read this block:

- Line 482: require `len(compositions(7)) == 120` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 483–487:

```
483 restricted = {k: v.copy() for k,v in context.items()}
484 restricted["available"][:, 2] = False
485 restricted["prices"][:, 2] = 0.
486 _, error = targeted_coupons(restricted, households, weights, alpha, gamma, beta,
487                              discounts[:1], capacity=7, batch_size=4)
```

Read this block:

- Line 483, `restricted`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 484, `restricted['available'][:, 2]`: Store the literal value `False` in the selected entry/column.
- Line 485, `restricted['prices'][:, 2]`: Store the literal value `0.0` in the selected entry/column.
- Line 486, `(_, error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 488–488:

```
assert error < 1e-12
```

Read this block:

- Line 488: require `error < 1e-12` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Source lines 489–530:

```
with tempfile.TemporaryDirectory(prefix="targeting_synthetic_") as directory:
    root = Path(directory)
    synthetic_input = root / "synthetic_records.csv"
    frame.to_csv(synthetic_input, index=False)
    pd.DataFrame({"parameter": list(parameters), "estimate": list(parameters.values()), "at_bound":
    ↪ False}).to_csv(root / "point_estimates.csv", index=False)
    fit = dict(model="heterogeneous", converged=True, nll=nll, weeks=len(frame), households=12,
               settings=dict(capacity=2, draws=8, seed=19, no_price_control=True, habit_weight=.7))
    (root / "fit.json").write_text(json.dumps(fit))
    args = argparse.Namespace(fit_dir=root, household_input=synthetic_input, output=root,
                              discounts=discounts, demand_delta=1., batch_size=4,
                              type_treatment="posterior-mean", save_state_policies=True)
    report = run_households(args)
    assert report["households"] == 12
    states = pd.read_csv(root / "targeted_state_policies.csv.gz")
    assert np.allclose(states.groupby(["household_code", "policy"]).probability.sum(), 1.)
    # Independent exhaustive two-period calculation for the first targeted
    # household: all first actions and all terminal actions at each basket.
    household = int(np.flatnonzero(households.group.eq("low").to_numpy())[0])
    a, g = weights[household] @ alpha, weights[household] @ gamma
    basket_rows = compositions(2)
    values = []
    for d1 in discounts:
        prices = context["prices"][household].copy()
        prices[1] *= 1-d1
        xi = .3*context["novelty"][household]-.7*context["stocks"][household]
        u = np.r_[beta-a*prices+g*xi, 0.]
        prob = np.exp(u-logsumexp(u))
        value = 2*prob[1]*prices[1]
        continuation = 0.
        for basket in basket_rows:
            mass = math.factorial(2)/math.prod(math.factorial(int(n)) for n in basket)*np.prod(prob**basket)
            x = basket[:3]/max(basket[:3].sum(), 1)
            xi2 = .3*(x.sum()-x)-.7*(context["stocks"][household]+x)
            returns = []
            for d2 in discounts:
                second_prices = context["prices"][household].copy()
                second_prices[1] *= 1-d2
                u2 = np.r_[beta-a*second_prices+g*xi2, 0.]
                prob2 = np.exp(u2-logsumexp(u2))
                returns.append(2*prob2[1]*second_prices[1])
            continuation += mass*max(returns)
        values.append(value+continuation)
```

Source lines 531–533:

```
actual = pd.read_csv(root / "targeted_household_results.csv")
chosen = actual[(actual.household_code == households.iloc[household].household_code) &
↪ actual.policy.eq("personalized_dynamic")].iloc[0]
```

```
533 assert np.isclose(chosen.discounted_revenue, max(values))
```

Read this block:

- Line 489: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 490, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 491, `synthetic_input`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 492: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 493: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 494, `fit`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 496: call `(root / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 497, `args`: Set this name to the result of calling `argparse.Namespace(...)`; the arguments and preceding method chain are shown above.
- Line 500, `report`: Set this name to the result of calling `run_households(...)`; the arguments and preceding method chain are shown above.
- Line 501: require `report['households'] == 12` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 502, `states`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 503: require `np.allclose(states.groupby(['household_code', 'policy']).probability.sum(), 1.0)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 506, `household`: Set this name to the result of calling `int(...)`; the arguments and preceding method chain are shown above.
- Line 507, `(a, g)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 508, `basket_rows`: Set this name to the result of calling `compositions(...)`; the arguments and preceding method chain are shown above.
- Line 509, `values`: Set this name to the list `[]`.
- Line 510: for each entry yielded by `discounts`, bind/unpack it into `d1` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 511, `prices`: Set this name to the result of calling `context['prices'][household].copy(...)`; the arguments and preceding method chain are shown above.
- Line 512: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 513, `xi`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 514, `u`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + g * xi, 0.0]`; brackets select from the object to their left.
- Line 515, `prob`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 516, `value`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 517, `continuation`: Set this name to the literal value `0.0`.
- Line 518: for each entry yielded by `basket_rows`, bind/unpack it into `basket` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 519, `mass`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 520, `x`: Set this name to division of the displayed operands; parentheses control which subexpressions

are evaluated together.
- Line 521, `xi2`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 522, `returns`: Set this name to the list `[]`.
- Line 523: for each entry yielded by `discounts`, bind/unpack it into `d2` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 524, `second_prices`: Set this name to the result of calling `context['prices'][household].copy(...)`; the arguments and preceding method chain are shown above.
- Line 525: update `second_prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 526, `u2`: Set this name to the entry/slice selected by `np.r_[beta - a * second_prices + g * xi2, 0.0]`; brackets select from the object to their left.
- Line 527, `prob2`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 528: call `returns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 529: update `continuation` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 530: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 531, `actual`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 532, `chosen`: Set this name to the entry/slice selected by `actual[(actual.household_code == households.iloc[househol...`; brackets select from the object to their left.
- Line 533: require `np.isclose(chosen.discounted_revenue, max(values))` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `run_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `list`: Materialize iterable entries as an ordered Python list.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 534–534:

```
534 print("Synthetic targeting checks passed: posterior likelihood, per-household optimization, full-basket backward
    ↪  induction, reporting-group invariance, policy dominance, missing menus, exact basket mass, exports.")
```

Read this block:

- Line 534: call `print` with the displayed arguments. No assignment stores its return here; its documented

side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 537.

Parses explicit fit and weekly-input paths, validates coupon settings, and runs the earlier combined-gamma berry-only exercise. Its self-test uses generated temporary inputs. The new split fit requires the separate split entry point.

Definition and stated intent:

Source lines 537–537:

```
537 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 538–544:

```
538 parser = argparse.ArgumentParser(description=__doc__)
539 parser.add_argument("--household-input", type=Path, help="Explicit household input selected by you; no default.")
540 parser.add_argument("--fit-dir", type=Path, help="Explicit existing fitted-results directory; no default.")
541 parser.add_argument("--output", type=Path, default=Path(__file__).resolve().parent / "empirical_outputs")
542 parser.add_argument("--discounts", type=float, nargs="+", default=np.linspace(0., 1., 101),
543                     help="Candidate discounts; default is every 1%% from 0%% through 100%%.")
544 parser.add_argument("--demand-delta", type=float, default=.95)
```

Read this block:

- Line 538, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 539: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 540: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 541: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 542: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 544: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 545–550:

```
545 parser.add_argument("--batch-size", type=int, default=24)
546 parser.add_argument("--type-treatment", choices=["posterior-mean", "posterior-integrated"],
    ↪  default="posterior-mean",
547                     help="Default treats estimated household types as known, matching base supply; optional
                        ↪  posterior uncertainty sensitivity.")
548 parser.add_argument("--save-state-policies", action="store_true", help="Export every optimized household/basket
    ↪  terminal coupon to compressed CSV.")
549 parser.add_argument("--self-test", action="store_true", help="Generated synthetic inputs only.")
550 args = parser.parse_args()
```

Read this block:

- Line 545: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 546: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 548: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 549: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 550, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 551–553:

```
551 if args.self_test:
552     self_test()
553     return
```

Read this block:

- Line 551: test `args.self_test`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 552: call `self_test` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 553: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `self_test`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 554–555:

```
554 if args.household_input is None or args.fit_dir is None:
555     parser.error("Provide --household-input AND --fit-dir; there are no default empirical input paths.")
```

Read this block:

- Line 554: test `args.household_input is None or args.fit_dir is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 555: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 556–556:

```
556 args.discounts = np.unique(np.asarray(args.discounts, dtype=float))
```

Read this block:

- Line 556, `args.discounts`: Set this name to the result of calling `np.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 557–559:

```
557 if (not np.isfinite(args.discounts).all() or args.discounts[0] != 0. or args.discounts[-1] > 1.
558         or not 0 <= args.demand_delta <= 1 or args.batch_size < 1):
559     parser.error("Discounts must include zero and lie in [0,1]; delta in [0,1], batch size >=1.")
```

Read this block:

- Line 557: test `not np.isfinite(args.discounts).all() or args.discounts[0] != 0.0 or args.dis...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 559: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 560–566:

```
560 args.output.mkdir(parents=True, exist_ok=True)
561 report = run_households(args)
562 manifest = dict(settings={k: str(v) if isinstance(v, Path) else v.tolist() if isinstance(v,np.ndarray) else v for
    ↪  k,v in vars(args).items()},
563                 results=report, script_sha256=digest(Path(__file__).resolve()),
564                 versions=dict(python=sys.version, numpy=np.__version__, pandas=pd.__version__))
565 (args.output / "run_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
566 print(f"Standalone empirical targeting completed: {args.output}")
```

Read this block:

- Line 560: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 561, `report`: Set this name to the result of calling `run_households(...)`; the arguments and preceding method chain are shown above.
- Line 562, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 565: call `(args.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 566: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `run_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `isinstance`: Check a runtime type to select the appropriate conversion path.

- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

pricing_exercises/run_pricing_exercises.py

Compatibility launcher forwarding to the earlier supply or combined-gamma exercise. The current split workflow has a separate explicit entry point and is not selected by the old households branch.

Module setup and direct-execution blocks

Source lines 2–6:

```
2 """Compatibility launcher. Prefer the two explicit standalone entry points."""
3 import argparse
4 from pathlib import Path
5 import subprocess
6 import sys
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 3: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `subprocess`. An as-name is a convenient local alias, not another library.
- Line 6: import `sys`. An as-name is a convenient local alias, not another library.

Source lines 19–20:

```
19 if __name__ == "__main__":
20     main()
```

Read this block:

- Line 19: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 20: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

main

Original function: `main`. Source begins at line 9.

Forwards compatibility arguments to the standalone theoretical supply or earlier combined-gamma household script. The launcher does not estimate demand or select the new split workflow automatically. Use the explicit split-pricing entry point for the current exercise.

Definition and stated intent:

Source lines 9–9:

```
9 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 10–12:

```
10 parser = argparse.ArgumentParser(add_help=False)
11 parser.add_argument("--exercise", choices=["supply", "households", "both"], default="supply")
12 args, remaining = parser.parse_known_args()
```

Read this block:

- Line 10, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 11: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 12, `(args, remaining)`: Unpack the result of calling `parser.parse_known_args(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.

Source lines 13–14:

```
13 if args.exercise == "both":
14     raise SystemExit("Use run_supply_exercises.py and run_empirical_targeting.py separately; see README.md.")
```

Read this block:

- Line 13: test `args.exercise == 'both'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 14: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `SystemExit`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 15–16:

```
15 script = "run_supply_exercises.py" if args.exercise == "supply" else "run_empirical_targeting.py"
16 subprocess.run([sys.executable, str(Path(__file__).resolve().parent / script), *remaining], check=True)
```

Read this block:

- Line 15, `script`: Set this name to the true-branch value if `args.exercise == 'supply'` holds, otherwise the else-branch value.
- Line 16: call `subprocess.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.
- `str`: Convert to text for paths, messages, or serialization.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

pricing__exercises/run__split__targeting.py

Current original split-model empirical pricing implementation. It recovers alpha, novelty and own-stock types; ranks by novelty; retains both effects in demand; and runs all three focal flavors with fixed rivals. This file uses household batches; the readable final_simple version loops one household at a time with equivalent results.

Module setup and direct-execution blocks

Source lines 2–11:

```
2 """Split-history pricing for each focal flavor, separate from demand estimation.
3
4 User-supplied paths only. No estimation is performed. --self-test uses only
5 generated synthetic inputs. See SPLIT_TARGETING.md for definitions.
6 """
7 from __future__ import annotations
8 import argparse
9 import hashlib
10 import json
11 import math
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `__future__`, import `annotations` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 9: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 10: import `json`. An as-name is a convenient local alias, not another library.
- Line 11: import `math`. An as-name is a convenient local alias, not another library.

Source lines 12–17:

```
12 import os
13 from pathlib import Path
14 import sys
15 import tempfile
16 import numpy as np
17 import pandas as pd
```

Read this block:

- Line 12: import `os`. An as-name is a convenient local alias, not another library.
- Line 13: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 14: import `sys`. An as-name is a convenient local alias, not another library.
- Line 15: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 16: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 17: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 19–19:

```
19 FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 19, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

Source lines 502–503:

```
502 if __name__ == "__main__":
503     main()
```

Read this block:

- Line 502: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 503: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

expit

Original function: `expit`. Source begins at line 21.

Evaluates the logistic probability with an exponential of a nonpositive magnitude to avoid overflow. Its input can be an array. This is a numerical transformation, not a demand-estimation call.

Definition and stated intent:

Source lines 21–21:

```
21 def expit(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 22–23:

```
22 x = np.asarray(x)
23 small = np.exp(-np.abs(x))
```

Read this block:

- Line 22, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 23, `small`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 24–24:

```
24 return np.where(x >= 0, 1/(1+small), small/(1+small))
```

Read this block:

- Line 24: return the result of calling `np.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

logsumexp

Original function: `logsumexp`. Source begins at line 26.

Computes log(sum(exp(x))) after subtracting the largest entry. The maximum is added back afterwards. axis selects the aggregation direction and keepdims retains singleton axes for broadcasting. This stabilizes likelihood normalization when utilities are large.

Definition and stated intent:

Source lines 26–26:

```
26 def logsumexp(x, axis=None, keepdims=False):
```

`def` creates this callable; the parentheses name its inputs (`x`, `axis`, `keepdims`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 27–29:

```
27 x = np.asarray(x)
28 maximum = np.max(x, axis=axis, keepdims=True)
29 result = maximum + np.log(np.exp(x-maximum).sum(axis=axis, keepdims=True))
```

Read this block:

- Line 27, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 28, `maximum`: Set this name to the result of calling `np.max(...)`; the arguments and preceding method chain are shown above.
- Line 29, `result`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 30–30:

```
30 return result if keepdims else np.squeeze(result, axis=axis)
```

Read this block:

- Line 30: return the true-branch value if `keepdims` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

plotting

Original function: `plotting`. Source begins at line 32.

Selects a noninteractive plotting backend and a writable temporary configuration directory, then returns pyplot. It does not read household data. Backend selection precedes importing pyplot so server runs do not require a display.

Definition and stated intent:

Source lines 32–32:

```
32 def plotting():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 33–36:

```
33 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
34 import matplotlib
35 matplotlib.use("Agg")
36 import matplotlib.pyplot as plt
```

Read this block:

- Line 33: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 34: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 35: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 37–37:

```
37 return plt
```

Read this block:

- Line 37: return the object currently named `plt`; plain assignment does not copy it. This ends the call.

persistent draws

Original function: `persistent_draws`. Source begins at line 39.

Generates seeded antithetic normal integration points: half the draws and their negatives. Split pricing uses three columns for log price sensitivity, novelty and own-stock habit; combined-gamma pricing uses two. These are fixed population integration points, not newly estimated coefficients or a bootstrap sample.

Definition and stated intent:

Source lines 39–39:

```
39 def persistent_draws(number, seed):
```

`def` creates this callable; the parentheses name its inputs (`number`, `seed`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 40–41:

```
40 if number < 2 or number % 2:
41     raise ValueError("Draw count must be even and >=2.")
```

Read this block:

- Line 40: test `number < 2 or number % 2`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 41: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 42–42:

```
42 half = np.random.default_rng(seed).standard_normal((number//2, 3))
```

Read this block:

- Line 42, `half`: Set this name to the result of calling `.standard_normal(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.standard_normal`: Draw independent standard-normal values in the requested shape, before coefficient transformations.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 43–43:

```
43 return np.vstack([half, -half])
```

Read this block:

- Line 43: return the result of calling `np.vstack(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.vstack`: Stack input arrays vertically, adding rows while preserving their common column layout.

posterior

Original function: `posterior`. Source begins at line 45.

Purpose. Recover household types from a saved fit and a matching weekly table. Each draw represents one persistent type. Utilities yield weekly count-weighted log probabilities; bincount adds all weeks within household before log-sum-exp integrates types. Normalizing those whole-history likelihoods gives household posterior weights.

For split pricing, alpha is positive price sensitivity, and the history array has columns b_N and b_S. weights @ history produces both posterior means. Sorting only the novelty means assigns bottom/top floor(N/4) reporting groups; both coefficients still enter forecasts. Posterior SDs and effective sample size describe type uncertainty. The returned NLL must match the saved fit before pricing. The earlier combined-gamma file instead returns a single gamma array.

Definition and stated intent:

Source lines 45–46:

```
45 def posterior(frame, parameters, draws, control=True):
46     """Household-integrated multinomial likelihood; types persist across weeks."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `parameters`, `draws`, `control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 47–52:

```
47 labels = np.sort(frame.household_code.unique())
48 hh = pd.Categorical(frame.household_code, categories=labels).codes
49 prices = frame[["price_"+f for f in FLAVORS]].to_numpy(float)
50 available = np.isfinite(prices) & (prices > 0)
51 counts = frame[FLAVORS+["outside"]].to_numpy(float)
52 novelty = frame[["novelty_"+f for f in FLAVORS]].to_numpy(float)
```

Read this block:

- Line 47, `labels`: Set this name to the result of calling `np.sort(...)`; the arguments and preceding method chain are shown above.
- Line 48, `hh`: Set this name to the attribute `pd.Categorical(frame.household_code, categories=labels).c...` of the object before the dot.
- Line 49, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 50, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 51, `counts`: Set this name to the result of calling `frame[FLAVORS + ['outside']].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 52, `novelty`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 53–54:

```
53 stock = frame[["stock_"+f for f in FLAVORS]].to_numpy(float)
54 cf = frame[["cf_"+f for f in FLAVORS]].to_numpy(float) if control else np.zeros_like(prices)
```

Read this block:

- Line 53, `stock`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 54, `cf`: Set this name to the true-branch value if `control` holds, otherwise the else-branch value.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 55–58:

```
55 if (not np.isfinite(counts).all() or (counts < 0).any()
56         or not np.isfinite(novelty).all() or not np.isfinite(stock).all() or not np.isfinite(cf).all()
57         or ((counts[:, :3] > 0) & ~available).any()):
```

```
58     raise ValueError("Invalid counts, history, controls, or purchased-alternative coverage.")
```

Read this block:

- Line 55: test `not np.isfinite(counts).all() or (counts < 0).any() or (not np.isfinite(novel...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 58: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 59–63:

```
59 beta = parameters["inside_constant"] + np.array([0., parameters["beta_berry"], parameters["beta_plain"]])
60 alpha = np.exp(parameters["mu_log_minus_alpha"] + parameters["sd_log_minus_alpha"]*draws[:, 0])
61 switching = parameters["mean_novelty"] + parameters["sd_novelty"]*draws[:, 1]
62 habit = parameters["mean_own_stock"] + parameters["sd_own_stock"]*draws[:, 2]
63 loglike = np.empty((len(labels), len(draws)))
```

Read this block:

- Line 59, `beta`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 60, `alpha`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 61, `switching`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 62, `habit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 63, `loglike`: Set this name to the result of calling `np.empty(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 64–69:

```
64 for d, (a, g, b) in enumerate(zip(alpha, switching, habit)):
65     u = beta-a*np.nan_to_num(prices)+g*novelty+b*stock+parameters.get("price_control", 0.)*cf
66     u = np.column_stack([np.where(available, u, -np.inf), np.zeros(len(frame))])
67     logp = u-logsumexp(u, axis=1, keepdims=True)
68     ll = (counts*np.where(counts > 0, logp, 0.)).sum(axis=1)
69     loglike[:, d] = np.bincount(hh, weights=ll, minlength=len(labels))
```

Read this block:

- Line 64: for each entry yielded by `enumerate(zip(alpha, switching, habit))`, bind/unpack it into `(d, (a, g, b))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 65, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 66, `u`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.

- Line 67, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 68, `ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 69, `loglike[:, d]`: Store the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 70–76:

```
70 weights = np.exp(loglike-logsumexp(loglike, axis=1, keepdims=True))
71 nll = -float((logsumexp(loglike, axis=1)-np.log(len(draws))).sum())
72 means = weights @ switching
73 # Exactly floor(N/4) per tail. Household ID breaks ties reproducibly.
74 order = np.argsort(means, kind="stable")
75 groups = np.full(len(labels), "middle", dtype=object)
76 n = len(labels)//4
```

Read this block:

- Line 70, `weights`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 71, `nll`: Set this name to the negated or sign-adjusted expression `-float((logsumexp(loglike, axis=1) - np.log(len(draws))).....`.
- Line 72, `means`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 74, `order`: Set this name to the result of calling `np.argsort(...)`; the arguments and preceding method chain are shown above.
- Line 75, `groups`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 76, `n`: Set this name to floor division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 77–78:

```
77 if n < 1:
78     raise ValueError("Need at least four households to form quartiles.")
```

Read this block:

- Line 77: test `n < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 78: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 79–87:

```
79 groups[order[:n]], groups[order[-n:]] = "low", "high"
80 population_q = np.quantile(switching, [.25, .75])
81 table = pd.DataFrame(dict(household_code=labels, group=groups, switching_mean=means,
82                           switching_sd=np.sqrt(np.maximum(weights @ switching**2-means**2, 0)),
83                           alpha_mean=weights @ alpha, posterior_ess=1/(weights**2).sum(axis=1),
84                           probability_switching_below_draw_q25=weights @ (switching <= population_q[0]),
85                           probability_switching_above_draw_q75=weights @ (switching >= population_q[1])))
86 table["habit_mean"] = weights @ habit
87 table["habit_sd"] = np.sqrt(np.maximum(weights @ habit**2 - table.habit_mean.to_numpy()**2, 0))
```

Read this block:

- Line 79, `(groups[order[:n]], groups[order[-n:]])`: Unpack the tuple `('low', 'high')` into these targets in order.
- Line 80, `population_q`: Set this name to the result of calling `np.quantile(...)`; the arguments and preceding method chain are shown above.
- Line 81, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 86, `table['habit_mean']`: Store matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 87, `table['habit_sd']`: Store the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `.quantile`: Find empirical quantiles, supplying percentile bootstrap endpoints rather than normal-Wald formulas.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 88–88:

```
88 return table, weights, alpha, np.column_stack([switching, habit]), beta, nll
```

Read this block:

- Line 88: return a tuple of 6 results. The caller can unpack them; return ends the call, unlike print.

compositions

Original function: `compositions`. Source begins at line 90.

Enumerates every nonnegative integer basket whose four counts sum to capacity: three flavors plus constructed nonpurchase. For capacity 7 there are 120 baskets. Enumerating all baskets preserves realized history transitions; substituting the mean basket would generally alter continuation values.

Definition and stated intent:

Source lines 90–91:

```
90 def compositions(capacity):
91     """All multinomial baskets of three inside flavors and outside."""
```

`def` creates this callable; the parentheses name its inputs (`capacity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 92–93:

```
92 return np.array([(a,b,c,capacity-a-b-c) for a in range(capacity+1)
93                  for b in range(capacity-a+1) for c in range(capacity-a-b+1)], dtype=int)
```

Read this block:

- Line 92: return the result of calling `np.array(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

forecast context

Original function: `forecast_context`. Source begins at line 95.

Relabels the first observed choice as t=0, adds its flavor shares once to its pre-choice stock, and uses that basket for next-week novelty. It carries the first menu/prices/controls forward, records the two subsequent pricing dates, and checks focal-menu eligibility. Later observed purchases affect type recovery but not this initial counterfactual history.

Definition and stated intent:

Source lines 95–100:

```
95 def forecast_context(frame, households):
96     """First observed choice establishes h0; price the next two consecutive weeks.
97
98     Carry the initial observation's menu, posted prices and controls forward.
99     Type estimation still uses the complete observed sample.
100    """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `households`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 101–107:

```
101 first = frame.sort_values(["household_code", "week"]).groupby("household_code", sort=True).head(1)
102 first = first.set_index("household_code").loc[households.household_code]
103 counts = first[FLAVORS].to_numpy(float)
104 totals = counts.sum(axis=1, keepdims=True)
105 shares = np.divide(counts, totals, out=np.zeros_like(counts), where=totals > 0)
106 # Saved stocks precede the observed choice; add its basket exactly once.
107 stocks = first[["stock_"+f for f in FLAVORS]].to_numpy(float)+shares
```

Read this block:

- Line 101, `first`: Set this name to the result of calling `.head(...)`; the arguments and preceding method chain are shown above.
- Line 102, `first`: Set this name to the entry/slice selected by `first.set_index('household_code').loc[households.househol...`; brackets select from the object to their left.

- Line 103, `counts`: Set this name to the result of calling `first[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 104, `totals`: Set this name to the result of calling `counts.sum(...)`; the arguments and preceding method chain are shown above.
- Line 105, `shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 107, `stocks`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 108–113:

```
108 novelty = shares.sum(axis=1, keepdims=True)-shares
109 prices = first[["price_"+f for f in FLAVORS]].to_numpy(float)
110 available = np.isfinite(prices) & (prices > 0)
111 eligible = available[:, 1]
112 households["berry_eligible"] = eligible
113 initial_weeks = first.week.to_numpy(dtype="datetime64[ns]")
```

Read this block:

- Line 108, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 109, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 110, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 111, `eligible`: Set this name to the entry/slice selected by `available[:, 1]`; brackets select from the object to their left.
- Line 112, `households['berry_eligible']`: Store the object currently named `eligible`; plain assignment does not copy it in the selected entry/column.
- Line 113, `initial_weeks`: Set this name to the result of calling `first.week.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 114–118:

```
114 households["initial_observed_week"] = initial_weeks
115 households["pricing_week1"] = initial_weeks+np.timedelta64(7, "D")
116 households["pricing_week2"] = initial_weeks+np.timedelta64(14, "D")
117 cf_cols = ["cf_"+f for f in FLAVORS]
118 cf = first[cf_cols].to_numpy(float) if all(c in first for c in cf_cols) else np.zeros_like(prices)
```

Read this block:

- Line 114, `households['initial_observed_week']`: Store the object currently named `initial_weeks`; plain assignment does not copy it in the selected entry/column.
- Line 115, `households['pricing_week1']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 116, `households['pricing_week2']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 117, `cf_cols`: Set this name to a list computed by the displayed for-expression.
- Line 118, `cf`: Set this name to the true-branch value if `all((c in first for c in cf_cols))` holds, otherwise the else-branch value.

Syntax and calls used here:

- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 119–120:

```
119 if not np.isfinite(stocks).all() or not np.isfinite(novelty).all():
120     raise ValueError("Nonfinite forecast history.")
```

Read this block:

- Line 119: test `not np.isfinite(stocks).all() or not np.isfinite(novelty).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 120: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

Source lines 121–122:

```
121 return dict(stocks=stocks, novelty=novelty, prices=np.where(available, prices, 0.),
122             available=available, cf=cf, eligible=eligible)
```

Read this block:

- Line 121: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

digest

Original function: `digest`. Source begins at line 124.

Streams file bytes through SHA-256 to identify the exact input or source used by a run. Hashes are provenance checks, not economic statistics. This function is called during an explicit empirical run, not by the documentation builder.

Definition and stated intent:

Source lines 124–124:

```
124 def digest(path):
```

`def` creates this callable; the parentheses name its inputs (`path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 125–125:

```
125 h = hashlib.sha256()
```

Read this block:

- Line 125, `h`: Set this name to the result of calling `hashlib.sha256(...)`; the arguments and preceding method chain are shown above.

Source lines 126–128:

```
126 with path.open("rb") as handle:
127     for chunk in iter(lambda: handle.read(1024*1024), b""):
128         h.update(chunk)
```

Read this block:

- Line 126: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 127: for each entry yielded by `iter(lambda: handle.read(1024 * 1024), b'')`, bind/unpack it into `chunk` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 128: call `h.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.
- `iter`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 129–129:

```
129 return h.hexdigest()
```

Read this block:

- Line 129: return the result of calling `h.hexdigest(...)`; the arguments and preceding method chain are shown above. This ends the call.

targeted coupons

Original function: `targeted_coupons`. Source begins at line 132.

Research calculation. Solve terminal prices for every realized first basket, then choose the initial coupon using immediate revenue plus discounted optimized continuation revenue. In the readable version the outer loop visits one household; remaining axes are type, basket, coupon and revenue metric. Default known posterior mean types leave just one type; posterior-integrated mode keeps draws and updates their weights through the first-basket likelihood.

In split pricing, history_coefficient has two columns: history1=b_N$N1$+b_SS1, history2=b_N$N2$+b_SS2. Every basket updates own stock and other-flavor novelty. Both coefficients enter both dates. The solver's focal slot is coordinate one; run_households permutes the selected flavor into that coordinate. Rival quantities retain distinct identities for history updates.

Terminal coupons maximize revenue after integrating remaining unobserved type uncertainty, before averaging over realized baskets. The firm cannot condition on an unobserved true draw. Both focal and no-focal-purchase branches count. Bottom/top novelty groups get personalized offers; middle households remain at posted prices.

No coupons supplies a feasible benchmark and optimized revenue must weakly dominate it. Coupon prices lie between zero and the posted price; they are not unrestricted theoretical prices.

Definition and stated intent:

Source lines 132–142:

```
132 def targeted_coupons(context, households, weights, alpha, switching, beta, discounts,
133                     delta=1., capacity=7, control_coefficient=0.,
134                     batch_size=24, type_treatment="posterior-mean", state_output=None):
135     """Observed h0, then p1(h0), realized h1, and terminal p2(h1).
136 
137     The observed initialization and two priced choices form three choice dates.
138     Individual backward induction; switching quartiles are reporting groups only.
139 
140     Baseline: posted prices in both weeks. Dynamic: choose first coupon accounting for optimized
141     state-contingent terminal revenue. The seller observes the full basket.
142     """
```

`def` creates this callable; the parentheses name its inputs (`context`, `households`, `weights`, `alpha`, `switching`, `beta`, `discounts`, `delta`, `capacity`, `control_coefficient`, `batch_size`, `type_treatment`, `state_output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 143–148:

```
143 baskets = compositions(capacity)
144 inside = baskets[:, :3]
145 totals = inside.sum(axis=1, keepdims=True)
146 shares = np.divide(inside, totals, out=np.zeros_like(inside, dtype=float), where=totals > 0)
147 coeff = math.lgamma(capacity+1)-np.vectorize(math.lgamma)(baskets+1).sum(axis=1)
148 k, nstates = len(discounts), len(baskets)
```

Read this block:

- Line 143, `baskets`: Set this name to the result of calling `compositions(...)`; the arguments and preceding method chain are shown above.
- Line 144, `inside`: Set this name to the entry/slice selected by `baskets[:, :3]`; brackets select from the object to their left.
- Line 145, `totals`: Set this name to the result of calling `inside.sum(...)`; the arguments and preceding method chain are shown above.
- Line 146, `shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 147, `coeff`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 148, `(k, nstates)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 149–152:

```
149 rows, maximum_mass_error, wrote_states = [], 0., False
150 indices = np.flatnonzero(context["eligible"])
151 # Only the two tails receive targeting; middle households remain at posted prices.
152 targeted = households.group.isin(["low", "high"]).to_numpy()
```

Read this block:

- Line 149, `(rows, maximum_mass_error, wrote_states)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 150, `indices`: Set this name to the result of calling `np.flatnonzero(...)`; the arguments and preceding method chain are shown above.
- Line 152, `targeted`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.

Source lines 153–194:

```
for start in range(0, len(indices), batch_size):
    ids = indices[start:start+batch_size]
    h = len(ids)
    prices, availability = context["prices"][ids], context["available"][ids]
    stock, novelty, cf = context["stocks"][ids], context["novelty"][ids], context["cf"][ids]
    if type_treatment == "posterior-mean":
        # Base supply information assumption: an estimated fixed type is known.
        a, g = (weights[ids] @ alpha)[:, None], (weights[ids] @ switching)[:, None]
        w = np.ones((h, 1))
    else:
        a = np.broadcast_to(alpha, (h, len(alpha)))
        g = np.broadcast_to(switching, (h, *switching.shape))
        w = weights[ids]
    # g[...,0] is switching/novelty; g[...,1] is signed own-stock habit.
    history1 = g[..., 0, None]*novelty[:, None, :] + g[..., 1, None]*stock[:, None, :]
    novelty2 = shares.sum(axis=1, keepdims=True)-shares
    stock2 = stock[:, None, :]+shares[None, :, :]
    history2 = (g[..., 0, None, None]*novelty2[None, None, :, :]
                +g[..., 1, None, None]*stock2[:, None, :, :])
    u1 = beta[None, None, :]-a[..., None]*prices[:, None, :]+history1+control_coefficient*cf[:, None, :]
    u2 = beta[None, None, None, :]-a[:, :, None, None]*prices[:, None, None, :]+history2+control_coefficient*cf[:,
↪   None, None, :]
    u1 = np.where(availability[:, None, :], u1, -np.inf)
    u2 = np.where(availability[:, None, None, :], u2, -np.inf)
    terminal = []
    for discount in discounts:
        u = u2.copy()
        u[..., 1] += a[:, :, None]*prices[:, None, None, 1]*discount
        denominator = logsumexp(np.concatenate([u, np.zeros((*u.shape[:-1], 1))], axis=-1), axis=-1)
        quantity = capacity*np.exp(u[..., 1]-denominator)
        terminal.append(np.stack([quantity*prices[:, None, None, 1]*(1-discount), quantity,
                                  quantity*prices[:, None, None, 1]*discount], axis=-1))
    # Metrics: net revenue, berry units, coupon expenditure.
    first = np.zeros((h, k, 3))
    second = np.zeros((h, k, 3))
    stage0_second = None
    paths = []
    for i, discount in enumerate(discounts):
        u = u1.copy()
        u[..., 1] += a*prices[:, None, 1]*discount
        full = np.concatenate([u, np.zeros((*u.shape[:-1], 1))], axis=-1)
        logp = full-logsumexp(full, axis=-1, keepdims=True)
        terms = np.zeros((h, a.shape[1], nstates, 4))
```

Source lines 195–236:

```
    np.multiply(baskets[None, None, :, :], logp[:, :, None, :], out=terms,
                where=baskets[None, None, :, :] > 0)
    probability = np.exp(coeff[None, None, :]+terms.sum(axis=-1))
```

```
    maximum_mass_error = max(maximum_mass_error, float(np.abs(probability.sum(axis=-1)-1).max()))
    joint = w[:, :, None]*probability
    state_probability = joint.sum(axis=1)
    quantity = capacity*(w*np.exp(logp[..., 1])).sum(axis=1)
    first[:, i] = np.stack([quantity*prices[:, 1]*(1-discount), quantity,
                            quantity*prices[:, 1]*discount], axis=-1)
    # Integrate types BEFORE choosing a terminal coupon: no foresight of
    # an unobserved true draw in the posterior-integrated sensitivity.
    metrics = np.stack([np.einsum("hds,hdsm->hsm", joint, candidate)
                        for candidate in terminal], axis=2)  # household,state,action,metric
    best = metrics[..., 0].argmax(axis=2)
    selected = np.take_along_axis(metrics, best[:, :, None, None], axis=2)[:, :, 0, :]
    # E_h1[p2(h1)*E(q2|h1,p2,type)] under this candidate p1(h0).
    # Optimize p2 separately in each state BEFORE summing its revenue.
    second[:, i] = selected.sum(axis=1)
    if i == 0:
        stage0_second = metrics[:, :, 0, :].sum(axis=1)
    # Storing these compact arrays allows exporting only chosen first actions.
    paths.append((best, selected, state_probability))
zero_choice = np.zeros(h, dtype=int)
dynamic_choice = (first[..., 0]+delta*second[..., 0]).argmax(axis=1)
dynamic_choice = np.where(targeted[ids], dynamic_choice, 0)
for name, choice in [("no_coupon", zero_choice), ("personalized_dynamic", dynamic_choice)]:
    for local, hh in enumerate(ids):
        action = int(choice[local])
        initial = first[local, action]
        is_target = targeted[hh] and name != "no_coupon"
        terminal_metrics = second[local, action] if is_target else stage0_second[local]
        best, selected, probability = paths[action]
        state_coupon = discounts[best[local]] if is_target else np.zeros(nstates)
        mass = probability[local]
        p1 = prices[local, 1]*(1-discounts[action])
        p2 = prices[local, 1]*(1-state_coupon)
        mean_p2 = float(mass @ p2)
        ratio = np.divide(p1, p2, out=np.full(nstates, np.nan), where=p2 > 0)
        positive_mass = mass > 1e-14
        expected_ratio = float(mass[positive_mass] @ ratio[positive_mass]) if
    ↪ np.isfinite(ratio[positive_mass]).all() else np.nan
        row = dict(household_code=households.iloc[hh].household_code,
                   group=households.iloc[hh].group, policy=name,
```

Source lines 237–259:

```
                   discount1=discounts[action], expected_discount2=float(mass @ state_coupon),
                   p1=p1, expected_p2=mean_p2,
                   ratio_of_expected_prices=p1/mean_p2 if mean_p2 > 0 else np.nan,
                   expected_price_ratio=expected_ratio,
                   revenue1=initial[0], revenue2=terminal_metrics[0],
                   discounted_revenue=initial[0]+delta*terminal_metrics[0],
                   berry_units1=initial[1], berry_units2=terminal_metrics[1],
                   coupon_expenditure=initial[2]+delta*terminal_metrics[2])
        for branch, label in [(baskets[:, 1] == 0, "no_berry"), (baskets[:, 1] > 0, "berry")]:
            branch_mass = mass[branch].sum()
            row[f"probability_{label}"] = branch_mass
            row[f"mean_p2_given_{label}"] = float(mass[branch] @ p2[branch]/branch_mass) if branch_mass > 0 else
    ↪ np.nan
        rows.append(row)
        if state_output is not None and is_target:
            detail = pd.DataFrame(dict(household_code=row["household_code"], group=row["group"], policy=name,
                                       state=np.arange(nstates), rival1_units=baskets[:, 0], focal_units=baskets[:, 1],
                                           rival2_units=baskets[:, 2], outside=baskets[:, 3], probability=mass,
                                           discount1=discounts[action], discount2=state_coupon, p1=p1, p2=p2,
    ↪ price_ratio=ratio))
```

```
            detail.to_csv(state_output, mode="a" if wrote_states else "w", header=not wrote_states,
                          index=False, compression="gzip")
            wrote_states = True
if start % (batch_size*40) == 0 or start+batch_size >= len(indices):
    print(f"Targeted pricing: integrated {min(start+batch_size,len(indices))}/{len(indices)} eligible households",
     ↪  flush=True)
```

Read this block:

- Line 153: for each entry yielded by `range(0, len(indices), batch_size)`, bind/unpack it into `start` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 154, `ids`: Set this name to the entry/slice selected by `indices[start:start + batch_size]`; brackets select from the object to their left.
- Line 155, `h`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 156, `(prices, availability)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 157, `(stock, novelty, cf)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 158: test `type_treatment == 'posterior-mean'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 160, `(a, g)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 161, `w`: Set this name to the result of calling `np.ones(...)`; the arguments and preceding method chain are shown above.
- Line 163, `a`: Set this name to the result of calling `np.broadcast_to(...)`; the arguments and preceding method chain are shown above.
- Line 164, `g`: Set this name to the result of calling `np.broadcast_to(...)`; the arguments and preceding method chain are shown above.
- Line 165, `w`: Set this name to the entry/slice selected by `weights[ids]`; brackets select from the object to their left.
- Line 167, `history1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 168, `novelty2`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 169, `stock2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 170, `history2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 172, `u1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 173, `u2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 174, `u1`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 175, `u2`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 176, `terminal`: Set this name to the list `[]`.
- Line 177: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 178, `u`: Set this name to the result of calling `u2.copy(...)`; the arguments and preceding method chain are shown above.
- Line 179: update `u[..., 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 180, `denominator`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 181, `quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 182: call `terminal.append` with the displayed arguments. No assignment stores its return here; its

documented side effect or assertion is what matters.
- Line 185, `first`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 186, `second`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 187, `stage0_second`: Set this name to the literal value `None`.
- Line 188, `paths`: Set this name to the list `[]`.
- Line 189: for each entry yielded by `enumerate(discounts)`, bind/unpack it into `(i, discount)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 190, `u`: Set this name to the result of calling `u1.copy(...)`; the arguments and preceding method chain are shown above.
- Line 191: update `u[..., 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 192, `full`: Set this name to the result of calling `np.concatenate(...)`; the arguments and preceding method chain are shown above.
- Line 193, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 194, `terms`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 195: call `np.multiply` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 197, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 198, `maximum_mass_error`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 199, `joint`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 200, `state_probability`: Set this name to the result of calling `joint.sum(...)`; the arguments and preceding method chain are shown above.
- Line 201, `quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 202, `first[:, i]`: Store the result of calling `np.stack(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 206, `metrics`: Set this name to the result of calling `np.stack(...)`; the arguments and preceding method chain are shown above.
- Line 208, `best`: Set this name to the result of calling `metrics[..., 0].argmax(...)`; the arguments and preceding method chain are shown above.
- Line 209, `selected`: Set this name to the entry/slice selected by `np.take_along_axis(metrics, best[:, :, None, None], axis=...)`; brackets select from the object to their left.
- Line 212, `second[:, i]`: Store the result of calling `selected.sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 213: test `i == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 214, `stage0_second`: Set this name to the result of calling `metrics[:, :, 0, :].sum(...)`; the arguments and preceding method chain are shown above.
- Line 216: call `paths.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 217, `zero_choice`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 218, `dynamic_choice`: Set this name to the result of calling `.argmax(...)`; the arguments and preceding method chain are shown above.
- Line 219, `dynamic_choice`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 220: for each entry yielded by `[('no_coupon', zero_choice), ('personalized_dynamic', dynamic_...`, bind/unpack it into `(name, choice)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 221: for each entry yielded by `enumerate(ids)`, bind/unpack it into `(local, hh)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 222, `action`: Set this name to the result of calling `int(...)`; the arguments and preceding method chain are shown above.
- Line 223, `initial`: Set this name to the entry/slice selected by `first[local, action]`; brackets select from the object to their left.
- Line 224, `is_target`: Set this name to the Boolean condition `targeted[hh] and name != 'no_coupon'`; it can be a scalar or a row/array mask depending on its inputs.
- Line 225, `terminal_metrics`: Set this name to the true-branch value if `is_target` holds, otherwise the else-branch value.
- Line 226, `(best, selected, probability)`: Unpack the entry/slice selected by `paths[action]`; brackets select from the object to their left into these targets in order.
- Line 227, `state_coupon`: Set this name to the true-branch value if `is_target` holds, otherwise the else-branch value.
- Line 228, `mass`: Set this name to the entry/slice selected by `probability[local]`; brackets select from the object to their left.
- Line 229, `p1`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 230, `p2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 231, `mean_p2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 232, `ratio`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 233, `positive_mass`: Set this name to the Boolean condition `mass > 1e-14`; it can be a scalar or a row/array mask depending on its inputs.
- Line 234, `expected_ratio`: Set this name to the true-branch value if `np.isfinite(ratio[positive_mass]).all()` holds, otherwise the else-branch value.
- Line 235, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 245: for each entry yielded by `[(baskets[:, 1] == 0, 'no_berry'), (baskets[:, 1] > 0, 'berry')]`, bind/unpack it into `(branch, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 246, `branch_mass`: Set this name to the result of calling `mass[branch].sum(...)`; the arguments and preceding method chain are shown above.
- Line 247, `row[f'probability_{label}']`: Store the object currently named `branch_mass`; plain assignment does not copy it in the selected entry/column.
- Line 248, `row[f'mean_p2_given_{label}']`: Store the true-branch value if `branch_mass > 0` holds, otherwise the else-branch value in the selected entry/column.
- Line 249: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 250: test `state_output is not None and is_target`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 251, `detail`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 255: call `detail.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 257, `wrote_states`: Set this name to the literal value `True`.
- Line 258: test `start % (batch_size * 40) == 0 or start + batch_size >= len(indices)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 259: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere;

np.where instead takes explicit true/false branches.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.einsum`: Perform an explicitly labeled sum of products; rh,rhp->p sums draws/households and retains the parameter axis.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 260–261:

```
260 if maximum_mass_error > 1e-9:
261     raise AssertionError(f"Basket probability mass error: {maximum_mass_error}")
```

Read this block:

- Line 260: test `maximum_mass_error > 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 261: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 262–267:

```
262 table = pd.DataFrame(rows)
263 baseline = table[table.policy == "no_coupon"].set_index("household_code").discounted_revenue
264 table["baseline_revenue"] = table.household_code.map(baseline)
265 table["revenue_change"] = table.discounted_revenue-table.baseline_revenue
```

```
266 table["revenue_change_pct"] = 100*table.revenue_change/table.baseline_revenue
267 wide = table.pivot(index="household_code", columns="policy", values="discounted_revenue")
```

Read this block:

- Line 262, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 263, `baseline`: Set this name to the attribute `table[table.policy == 'no_coupon'].set_index('household_c...` of the object before the dot.
- Line 264, `table['baseline_revenue']`: Store the result of calling `table.household_code.map(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 265, `table['revenue_change']`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 266, `table['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 267, `wide`: Set this name to the result of calling `table.pivot(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.

Source lines 268–269:

```
268 if (wide.personalized_dynamic+1e-8 < wide.no_coupon).any():
269     raise AssertionError("Household dynamic policy does not dominate the feasible no-coupon baseline.")
```

Read this block:

- Line 268: test `(wide.personalized_dynamic + 1e-08 < wide.no_coupon).any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 269: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 270–270:

```
270 return table, maximum_mass_error
```

Read this block:

- Line 270: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

run households

Original function: `run_households`. Source begins at line 272.

Reads an explicitly selected fitted-results directory and its weekly table, checks the model, convergence, bounds, capacity and input schema, then recovers posterior types and reproduces the saved likelihood. For a split run, it permutes the selected focal flavor into the solver's berry coordinate, moving counts, prices, stocks, novelty, controls and intercepts consistently. Type recovery happens before permutation using the original fitted flavor labels.

It calls the household solver, renames public berry-coordinate columns to focal names, and writes policy/group/aggregate tables and figures. The manifest records how rival state coordinates map back to flavors. Swapping only prices or only counts would silently price the wrong product. The three runs are separate focal sellers with fixed rival prices, not a simultaneous equilibrium.

Definition and stated intent:

Source lines 272–274:

```
272 def run_households(args):
273     # This branch is entered only when the person running the script supplies
274     # BOTH paths. Validation supplies generated temporary fixtures only.
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 275–275:

```
275 fit = json.loads((args.fit_dir / "fit.json").read_text())
```

Read this block:

- Line 275, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.

Source lines 276–277:

```
276 if fit["model"] != "heterogeneous_history" or not fit.get("converged", False):
277     raise ValueError("This exercise requires a converged split heterogeneous-history fit.")
```

Read this block:

- Line 276: test `fit['model'] != 'heterogeneous_history' or not fit.get('converged', False)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 277: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 278–278:

```
278 estimates = pd.read_csv(args.fit_dir / "point_estimates.csv")
```

Read this block:

- Line 278, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 279–280:

```
279 if "at_bound" in estimates and estimates.at_bound.astype(str).str.lower().eq("true").any():
280     raise ValueError("Fit has boundary parameters; review before running counterfactuals.")
```

Read this block:

- Line 279: test `'at_bound' in estimates and estimates.at_bound.astype(str).str.lower().eq('tr...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 280: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

Source lines 281–283:

```
281 parameters = dict(zip(estimates.parameter, estimates.estimate))
282 settings = fit["settings"]
283 capacity = settings["capacity"]
```

Read this block:

- Line 281, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 282, `settings`: Set this name to the entry/slice selected by `fit['settings']`; brackets select from the object to their left.
- Line 283, `capacity`: Set this name to the entry/slice selected by `settings['capacity']`; brackets select from the object to their left.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 284–285:

```
284 if not isinstance(capacity, int) or not 1 <= capacity <= 12:
285     raise ValueError("Exact basket enumeration supports integer capacity 1..12.")
```

Read this block:

- Line 284: test `not isinstance(capacity, int) or not 1 <= capacity <= 12`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 285: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 286–289:

```
286 frame = pd.read_csv(args.household_input, dtype={"household_code": str}, parse_dates=["week"])
287 required = ["household_code", "week", "outside", *FLAVORS]
288 required += [prefix+f for prefix in ["price_", "stock_", "novelty_"] for f in FLAVORS]
289 control = not settings.get("no_price_control", False)
```

Read this block:

- Line 286, `frame`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 287, `required`: Set this name to a list containing 4 displayed entries.
- Line 288: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 289, `control`: Set this name to the negated or sign-adjusted expression `not settings.get('no_price_control', False)`.

Syntax and calls used here:

- A starred entry expands an iterable into separate elements, here within a list/tuple or call.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 290–291:

```
290 if control:
291     required += ["cf_"+f for f in FLAVORS]
```

Read this block:

- Line 290: test `control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 291: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 292–292:

```
292 missing = set(required)-set(frame)
```

Read this block:

- Line 292, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 293–294:

```
293 if missing:
294     raise ValueError(f"Household input missing columns: {sorted(missing)}")
```

Read this block:

- Line 293: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 294: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 295–296:

```
295 if frame.household_code.isna().any() or frame.week.isna().any() or frame.duplicated(["household_code",
    ↪  "week"]).any():
296     raise ValueError("Need unique household/week rows with nonmissing IDs/dates.")
```

Read this block:

- Line 295: test `frame.household_code.isna().any() or frame.week.isna().any() or frame.duplica...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 296: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.

Source lines 297–298:

```
297 if len(frame) != fit["weeks"] or frame.household_code.nunique() != fit["households"]:
298     raise ValueError("Input household/week counts do not match fit metadata.")
```

Read this block:

- Line 297: test `len(frame) != fit['weeks'] or frame.household_code.nunique() != fit['househol...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 298: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.

Source lines 299–300:

```
draws = persistent_draws(settings["draws"], settings["seed"])
households, weights, alpha, switching, beta, nll = posterior(frame, parameters, draws, control)
```

Read this block:

- Line 299, `draws`: Set this name to the result of calling `persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 300, `(households, weights, alpha, switching, beta, nll)`: Unpack the result of calling `posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `persistent_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `posterior`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 302–303:

```
if not np.isclose(nll, fit["nll"], rtol=1e-9, atol=1e-4):
    raise ValueError(f"Likelihood mismatch: reconstructed {nll}, fit {fit['nll']}.")
```

Read this block:

- Line 302: test `not np.isclose(nll, fit['nll'], rtol=1e-09, atol=0.0001)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 303: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 304–304:

```
focal = args.focal_flavor
```

Read this block:

- Line 304, `focal`: Set this name to the attribute `args.focal_flavor` of the object before the dot.

Source lines 305–313:

```
if focal != "berry":
    mapping = {"berry": focal, focal: "berry"}
    original = frame.copy()
    original_beta = beta.copy()
    for destination, source in mapping.items():
        for prefix in ["", "price_", "stock_", "novelty_", "cf_"]:
            if prefix+source in original:
                frame[prefix+destination] = original[prefix+source]
        beta[FLAVORS.index(destination)] = original_beta[FLAVORS.index(source)]
```

Read this block:

- Line 305: test `focal != 'berry'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 306, `mapping`: Set this name to a dictionary with named entries `berry`.
- Line 307, `original`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 308, `original_beta`: Set this name to the result of calling `beta.copy(...)`; the arguments and preceding method chain are shown above.
- Line 309: for each entry yielded by `mapping.items()`, bind/unpack it into `(destination, source)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 310: for each entry yielded by `['', 'price_', 'stock_', 'novelty_', 'cf_']`, bind/unpack it into `prefix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 311: test `prefix + source in original`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 312, `frame[prefix + destination]`: Store the entry/slice selected by `original[prefix + source]`; brackets select from the object to their left in the selected entry/column.
- Line 313, `beta[FLAVORS.index(destination)]`: Store the entry/slice selected by `original_beta[FLAVORS.index(source)]`; brackets select from the object to their left in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 314–315:

```
314 context = forecast_context(frame, households)
315 counts = [int(((households.group == g) & households.berry_eligible).sum()) for g in ["low", "middle", "high"]]
```

Read this block:

- Line 314, `context`: Set this name to the result of calling `forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 315, `counts`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `forecast_context`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 316–317:

```
316 if any(n == 0 for n in counts):
317     raise ValueError("Each switching segment must have focal-eligible forecast households.")
```

Read this block:

- Line 316: test `any((n == 0 for n in counts))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 317: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 318–328:

```
318 table, mass_error = targeted_coupons(
319     context, households, weights, alpha, switching, beta, args.discounts,
320     args.demand_delta, capacity,
321     parameters.get("price_control", 0.) if control else 0., args.batch_size,
322     args.type_treatment,
323     args.output / "targeted_state_policies.csv.gz" if args.save_state_policies else None)
324 households.rename(columns={"berry_eligible": "focal_eligible"}, inplace=True)
325 table.rename(columns={c: c.replace("berry", "focal") for c in table if "berry" in c}, inplace=True)
326 table["focal_flavor"] = focal
327 households.to_csv(args.output / "household_posterior_summary.csv", index=False)
328 table.to_csv(args.output / "targeted_household_results.csv", index=False)
```

Read this block:

- Line 318, `(table, mass_error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 324: call `households.rename` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 325: call `table.rename` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 326, `table['focal_flavor']`: Store the object currently named `focal`; plain assignment does not copy it in the selected entry/column.
- Line 327: call `households.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 328: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `targeted_coupons`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

Source lines 329–335:

```
329 totals = ["revenue1", "revenue2", "discounted_revenue", "baseline_revenue", "revenue_change",
330           "focal_units1", "focal_units2", "coupon_expenditure"]
331 aggregate = table.groupby("policy", sort=False)[totals].sum()
332 aggregate["revenue_change_pct"] = 100*aggregate.revenue_change/aggregate.baseline_revenue
333 aggregate.to_csv(args.output / "targeted_revenue_aggregate.csv")
334 group = table.groupby(["policy", "group"], sort=False)[totals].sum()
335 group["revenue_change_pct"] = 100*group.revenue_change/group.baseline_revenue
```

Read this block:

- Line 329, `totals`: Set this name to a list containing 8 displayed entries.
- Line 331, `aggregate`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 332, `aggregate['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 333: call `aggregate.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 334, `group`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.

- Line 335, `group['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.

Source lines 336–350:

```
means = table.groupby(["policy", "group"])[["p1", "expected_p2", "discount1", "expected_discount2",
                                            "ratio_of_expected_prices", "expected_price_ratio"]].mean()
group = group.join(means)
group["eligible_households"] = table.groupby(["policy", "group"]).size()
group.to_csv(args.output / "targeted_revenue_by_switching_group.csv")
households.groupby("group").agg(households=("household_code", "size"),
                                focal_eligible=("focal_eligible", "sum"),
                                switching_mean=("switching_mean", "mean"),
                                switching_mean_min=("switching_mean", "min"),
                                switching_mean_max=("switching_mean", "max"),
                                mean_posterior_sd=("switching_sd", "mean"),
                                habit_mean=("habit_mean", "mean"),
                                mean_habit_posterior_sd=("habit_sd", "mean"),
                                mean_alpha=("alpha_mean", "mean"),
                                median_posterior_ess=("posterior_ess", "median")).to_csv(args.output /
 ↪  "household_group_summary.csv")
```

Read this block:

- Line 336, `means`: Set this name to the result of calling `.mean(...)`; the arguments and preceding method chain are shown above.
- Line 338, `group`: Set this name to the result of calling `group.join(...)`; the arguments and preceding method chain are shown above.
- Line 339, `group['eligible_households']`: Store the result of calling `.size(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 340: call `group.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 341: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.

Source lines 351–352:

```
make_targeting_figures(households, table, aggregate, args.output, delta=args.demand_delta,
                       focal_flavor=focal)
```

Read this block:

- Line 351: call `make_targeting_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_targeting_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 353–365:

```
return dict(focal_flavor=focal, state_coordinates={"focal_units": focal,
            "rival1_units": "berry" if focal == "other" else "other",
```

```
            "rival2_units": "berry" if focal == "plain" else "plain"}, households=len(households),
             ↪  eligible_by_group=dict(zip(["low", "middle", "high"], counts)),
            reconstructed_nll=nll, basket_mass_error=mass_error,
            input_sha256=digest(args.household_input), fit_sha256=digest(args.fit_dir / "fit.json"),
            parameter_sha256=digest(args.fit_dir / "point_estimates.csv"),
            posterior_draws=len(draws), posterior_seed=settings["seed"],
            capacity=capacity, history_model="b_N*N + b_S*S", price_control=control,
            initial_history="first_observed_basket", pricing_periods=2,
            baskets=len(compositions(capacity)),
            discount_grid=args.discounts.tolist(), demand_delta=args.demand_delta,
            type_treatment=args.type_treatment,
            aggregate=aggregate.reset_index().to_dict("records"))
```

Read this block:

- Line 353: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.

make targeting figures

Original function: `make_targeting_figures`. Source begins at line 368.

Draws household posterior switching types, discounted dynamic revenue by reporting group, and coupon patterns. Split figures use b_N labels and the selected focal flavor. Revenue is R1+delta*R2 with terminal revenues averaged after state-dependent optimization; mean prices times mean quantities would not give the same object.

Definition and stated intent:

Source lines 368–369:

```
def make_targeting_figures(households, table, aggregate, output, delta=.95, focal_flavor="berry"):
    """Plot household types and individually optimized berry discounts."""
```

`def` creates this callable; the parentheses name its inputs (`households`, `table`, `aggregate`, `output`, `delta`, `focal_flavor`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 370–375:

```
plt = plotting()
groups = ["low", "middle", "high"]
labels = ["Low switching\n(bottom 25%)", "Middle switching\n(middle 50%)", "High switching\n(top 25%)"]
colors = ["#3575AD", "#999999", "#D17925"]
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
bins = np.histogram_bin_edges(households.switching_mean, bins=30)
```

Read this block:

- Line 370, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.

- Line 371, `groups`: Set this name to the list `['low', 'middle', 'high']`.
- Line 372, `labels`: Set this name to the list `['Low switching\n(bottom 25%)', 'Middle switching\n(middl...`.
- Line 373, `colors`: Set this name to the list `['#3575AD', '#999999', '#D17925']`.
- Line 374, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 375, `bins`: Set this name to the result of calling `np.histogram_bin_edges(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 376–378:

```
376 for g, label, color in zip(groups, labels, colors):
377     axes[0].hist(households.loc[households.group == g, "switching_mean"],
378                  bins=bins, alpha=.65, label=label.replace("\n", " "), color=color)
```

Read this block:

- Line 376: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 377: call `axes[0].hist` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.hist`: Bin observed values, with optional weights controlling whether heights are counts, percentages, or another unit.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 379–386:

```
379 axes[0].set(title="Estimated consumer types", xlabel=r"Estimated household type (posterior mean $b_N$)",
380             ylabel="Number of households")
381 axes[0].legend(fontsize=9)
382 revenue = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["revenue1",
    ↪  "revenue2"]].sum().reindex(groups)
383 x = np.arange(len(groups))
384 # Average over state-contingent terminal prices first, discount, then total.
385 total_expected_revenue = revenue.revenue1 + delta * revenue.revenue2
386 axes[1].bar(x, total_expected_revenue, width=.5, color=colors)
```

Read this block:

- Line 379: call `axes[0].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 381: call `axes[0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 382, `revenue`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 383, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 385, `total_expected_revenue`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 386: call `axes[1].bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `len`: Count elements/rows rather than summing their numerical values.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.

Source lines 387–393:

```
387 axes[1].set_xticks(x, labels)
388 axes[1].set(title="Total Expected Revenue",
389             xlabel="Consumer type group", ylabel="Total Expected Revenue")
390 axes[1].set_ylim(bottom=0)
391 axes[1].grid(axis="y", alpha=.2)
392 axes[1].set_axisbelow(True)
393 axes[1].axhline(0, color="gray", linewidth=.6)
```

Read this block:

- Line 387: call `axes[1].set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 388: call `axes[1].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 390: call `axes[1].set_ylim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 391: call `axes[1].grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 392: call `axes[1].set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 393: call `axes[1].axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.grid`: Add visual grid lines without changing underlying numerical results.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.

Source lines 394–398:

```
394 fig.text(.5, .015,
395          f"First-period revenue + {delta:g} × expected terminal revenue. "
396          f"Low/high types receive personalized {focal_flavor} offers; middle types receive no coupons.",
397          ha="center", fontsize=9)
398 fig.tight_layout(rect=(0, .05, 1, 1))
```

Read this block:

- Line 394: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 398: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.text`: Place a text label on the axes/Figure at the supplied coordinates.

- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 399–400:

```
399 for extension in ["png", "pdf"]:
400     fig.savefig(output / f"household_coupon_results.{extension}", dpi=180)
```

Read this block:

- Line 399: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 400: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 401–408:

```
401 plt.close(fig)
402
403 fig, ax = plt.subplots(figsize=(9, 5))
404 x = np.arange(3)
405 means = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["discount1",
    ↪  "expected_discount2"]].mean().reindex(groups)
406 ax.bar(x-.18, 100*means.discount1, width=.36, label=r"$p_1(h_0)$ offer", color=colors[0])
407 ax.bar(x+.18, 100*means.expected_discount2, width=.36,
408        label=r"Terminal $p_2(h_1)$ offer (expected over histories)", color=colors[2])
```

Read this block:

- Line 401: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 403, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 404, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 405, `means`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 406: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 407: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 409–415:

```
409 ax.set_xticks(x, labels)
410 ax.set(xlabel="Consumer type group", ylim=(0, 105),
411        ylabel=f"Mean optimal {focal_flavor} discount (% of posted price)")
412 ax.grid(axis="y", alpha=.2)
413 ax.set_axisbelow(True)
414 ax.legend(fontsize=8, loc="upper left")
415 fig.suptitle("Dynamic pricing: optimal discounts by consumer type")
```

Read this block:

- Line 409: call `ax.set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 410: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 412: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 413: call `ax.set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 414: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 415: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 416–418:

```
416 fig.text(.5, .015, f"Household-weighted means among {focal_flavor}-eligible households. Middle group receives no
    ↪  coupons by policy design.",
417         ha="center", fontsize=9)
418 fig.tight_layout(rect=(0, .05, 1, .94))
```

Read this block:

- Line 416: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 418: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 419–420:

```
419 for extension in ["png", "pdf"]:
420     fig.savefig(output / f"discounts_by_consumer_type.{extension}", dpi=180)
```

Read this block:

- Line 419: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 420: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 421–425:

```
421 plt.close(fig)
422
423 dynamic = table.loc[table.policy == "personalized_dynamic"].merge(
424     households[["household_code", "switching_mean", "alpha_mean"]], on="household_code", validate="one_to_one")
425 fig, axes = plt.subplots(2, 2, figsize=(13, 9), sharey=True)
```

Read this block:

- Line 421: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 423, `dynamic`: Set this name to the result of calling `.merge(...)`; the arguments and preceding method chain are shown above.
- Line 425, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 426–438:

```
426 for column, (variable, xlabel) in enumerate([
427         ("switching_mean", r"Estimated household type (posterior mean $b_N$)"),
428         ("alpha_mean", r"Estimated price sensitivity (posterior mean $a$)")]):
429     for row, (discount, title) in enumerate([
430             ("discount1", r"Optimal discount at $p_1(h_0)$"),
431             ("expected_discount2", r"Expected terminal discount at $p_2(h_1)$")]):
432         ax = axes[row, column]
433         for g, label, color in zip(groups, labels, colors):
434             subset = dynamic.loc[dynamic.group == g]
435             ax.scatter(subset[variable], 100*subset[discount], s=12, alpha=.5,
436                        color=color, label=label.replace("\n", " "))
437         ax.set(xlabel=xlabel, ylabel=f"{focal_flavor.title()} discount (% of posted price)", title=title, ylim=(-2,
    ↪  102))
438         ax.grid(alpha=.2)
```

Read this block:

- Line 426: for each entry yielded by `enumerate([('switching_mean', 'Estimated household type (poste...`, bind/unpack it into `(column, (variable, xlabel))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 429: for each entry yielded by `enumerate([('discount1', 'Optimal discount at $p_1(h_0)$'), ('...`, bind/unpack it into `(row, (discount, title))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 432, `ax`: Set this name to the entry/slice selected by `axes[row, column]`; brackets select from the object to their left.
- Line 433: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 434, `subset`: Set this name to the entry/slice selected by `dynamic.loc[dynamic.group == g]`; brackets select from the object to their left.
- Line 435: call `ax.scatter` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 437: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 438: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 439–443:

```
439 axes[0, 0].legend(fontsize=8)
440 fig.suptitle("Dynamic pricing: household discounts and estimated consumer characteristics")
441 fig.text(.5, .015, f"Each point is one {focal_flavor}-eligible household. Terminal offers average over the
    ↪  household's realized h1 histories.",
442         ha="center", fontsize=9)
443 fig.tight_layout(rect=(0, .04, 1, .95))
```

Read this block:

- Line 439: call `axes[0, 0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 440: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 441: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 443: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 444–445:

```
444 for extension in ["png", "pdf"]:
445     fig.savefig(output / f"discounts_by_household_type.{extension}", dpi=180)
```

Read this block:

- Line 444: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 445: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 446–446:

```
446 plt.close(fig)
```

Read this block:

- Line 446: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

self test

Original function: `self_test`. Source begins at line 449.

Runs generated examples and assertions to check numerical calculations without launching the user-selected empirical exercise. Read the specific body to distinguish the CSV-blocked simple split checks from earlier complete export tests that reopen their own synthetic CSVs. No private input is needed.

Definition and stated intent:

Source lines 449–449:

```
449 def self_test():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 450–451:

```
450 import unittest
451 suite = unittest.defaultTestLoader.discover(str(Path(__file__).parent), pattern="test_split_targeting.py")
```

Read this block:

- Line 450: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 451, `suite`: Set this name to the result of calling `unittest.defaultTestLoader.discover(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 452–453:

```
452 if not unittest.TextTestRunner(verbosity=2).run(suite).wasSuccessful():
453     raise RuntimeError("Split targeting checks failed")
```

Read this block:

- Line 452: test `not unittest.TextTestRunner(verbosity=2).run(suite).wasSuccessful()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 453: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.
- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 456.

Parses explicit empirical inputs and loops over the selected focal-flavor cases, writing per-flavor outputs and a combined comparison. Defaults select all three flavors and delta=.92. The self-test branch uses generated inputs.

Definition and stated intent:

Source lines 456–456:

```
456 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 457–463:

```
457 parser = argparse.ArgumentParser(description=__doc__)
458 parser.add_argument("--household-input", type=Path, help="Explicit household input selected by you; no default.")
459 parser.add_argument("--fit-dir", type=Path, help="Explicit existing fitted-results directory; no default.")
460 parser.add_argument("--output", type=Path, default=Path(__file__).resolve().parent / "split_empirical_outputs")
461 parser.add_argument("--discounts", type=float, nargs="+", default=np.linspace(0., 1., 101),
462                     help="Candidate discounts; default is every 1%% from 0%% through 100%%.")
463 parser.add_argument("--demand-delta", type=float, default=.92)
```

Read this block:

- Line 457, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 458: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 459: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 460: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 461: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 463: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 464–470:

```
464 parser.add_argument("--batch-size", type=int, default=24)
465 parser.add_argument("--type-treatment", choices=["posterior-mean", "posterior-integrated"],
    ↪  default="posterior-mean",
466                     help="Default treats estimated household types as known, matching base supply; optional
                        ↪  posterior uncertainty sensitivity.")
467 parser.add_argument("--save-state-policies", action="store_true", help="Export every optimized household/basket
    ↪  terminal coupon to compressed CSV.")
468 parser.add_argument("--focal-flavor", choices=["all", *FLAVORS], default="all")
469 parser.add_argument("--self-test", action="store_true", help="Generated synthetic inputs only.")
470 args = parser.parse_args()
```

Read this block:

- Line 464: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 465: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 467: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 468: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 469: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 470, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- A starred entry expands an iterable into separate elements, here within a list/tuple or call.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 471–473:

```
471 if args.self_test:
472     self_test()
473     return
```

Read this block:

- Line 471: test `args.self_test`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 472: call `self_test` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 473: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `self_test`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 474–475:

```
474 if args.household_input is None or args.fit_dir is None:
475     parser.error("Provide --household-input AND --fit-dir; there are no default empirical input paths.")
```

Read this block:

- Line 474: test `args.household_input is None or args.fit_dir is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 475: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 476–476:

```
476 args.discounts = np.unique(np.asarray(args.discounts, dtype=float))
```

Read this block:

- Line 476, `args.discounts`: Set this name to the result of calling `np.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 477–479:

```
477 if (not np.isfinite(args.discounts).all() or args.discounts[0] != 0. or args.discounts[-1] > 1.
478         or not 0 <= args.demand_delta <= 1 or args.batch_size < 1):
479     parser.error("Discounts must include zero and lie in [0,1]; delta in [0,1], batch size >=1.")
```

Read this block:

- Line 477: test `not np.isfinite(args.discounts).all() or args.discounts[0] != 0.0 or args.dis...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 479: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 480–483:

```
480 args.output.mkdir(parents=True, exist_ok=True)
481 output_root = args.output
482 selected = FLAVORS if args.focal_flavor == "all" else [args.focal_flavor]
483 reports, comparisons = {}, []
```

Read this block:

- Line 480: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 481, `output_root`: Set this name to the attribute `args.output` of the object before the dot.
- Line 482, `selected`: Set this name to the true-branch value if `args.focal_flavor == 'all'` holds, otherwise the else-branch value.
- Line 483, `(reports, comparisons)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 484–490:

```
484 for focal in selected:
485     args.focal_flavor = focal
486     args.output = output_root / focal
487     args.output.mkdir(parents=True, exist_ok=True)
488     reports[focal] = run_households(args)
489     for row in reports[focal]["aggregate"]:
490         comparisons.append(dict(focal_flavor=focal, **row))
```

Read this block:

- Line 484: for each entry yielded by `selected`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 485, `args.focal_flavor`: Set this name to the object currently named `focal`; plain assignment does not copy it.
- Line 486, `args.output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 487: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 488, `reports[focal]`: Store the result of calling `run_households(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 489: for each entry yielded by `reports[focal]['aggregate']`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 490: call `comparisons.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `run_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 491–498:

```
491 args.output = output_root
492 args.focal_flavor = "all" if len(selected) == 3 else selected[0]
493 pd.DataFrame(comparisons).to_csv(output_root / "focal_flavor_comparison.csv", index=False)
494 report = reports
495 manifest = dict(settings={k: str(v) if isinstance(v, Path) else v.tolist() if isinstance(v,np.ndarray) else v for
    ↪  k,v in vars(args).items()},
496                 results=report, script_sha256=digest(Path(__file__).resolve()),
497                 versions=dict(python=sys.version, numpy=np.__version__, pandas=pd.__version__))
498 (args.output / "run_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
```

Read this block:

- Line 491, `args.output`: Set this name to the object currently named `output_root`; plain assignment does not copy it.
- Line 492, `args.focal_flavor`: Set this name to the true-branch value if `len(selected) == 3` holds, otherwise the else-branch value.
- Line 493: call `pd.DataFrame(comparisons).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 494, `report`: Set this name to the object currently named `reports`; plain assignment does not copy it.
- Line 495, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 498: call `(args.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.

Source lines 499–499:

```
499 print(f"Standalone empirical targeting completed: {args.output}")
```

Read this block:

- Line 499: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

pricing__exercises/run__supply__exercises.py

All three theoretical information cases: known initial history, hidden initial history, and advance commitment to two prices. The binary symmetric switching model is a mechanism benchmark; it is not the cumulative-stock split demand likelihood. Both continuation branches and unrestricted theoretical prices are retained.

Module setup and direct-execution blocks

Source lines 2–11:

```
2  """Run all three theoretical supply cases, prices, ratios and profit, in one file.
3
4  No empirical input is read. Dependencies: numpy, pandas, matplotlib.
5  python3 run_supply_exercises.py [--self-test] [--output DIRECTORY]
6  """
7  from __future__ import annotations
8  import argparse
9  import hashlib
10 import json
11 import math
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `__future__`, import `annotations` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 9: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 10: import `json`. An as-name is a convenient local alias, not another library.
- Line 11: import `math`. An as-name is a convenient local alias, not another library.

Source lines 12–17:

```
12 import os
13 from pathlib import Path
14 import sys
15 import tempfile
```

```
16 import numpy as np
17 import pandas as pd
```

Read this block:

- Line 12: import `os`. An as-name is a convenient local alias, not another library.
- Line 13: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 14: import `sys`. An as-name is a convenient local alias, not another library.
- Line 15: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 16: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 17: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 19–19:

```
19 FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 19, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

Source lines 162–163:

```
162 CASES = ["known_state", "hidden_initial_state", "ex_ante_commitment"]
163 LABELS = dict(zip(CASES, ["Known state (base)", "Hidden initial state", "Ex ante commitment"]))
```

Read this block:

- Line 162, `CASES`: Set this name to the list `['known_state', 'hidden_initial_state', 'ex_ante_commitme...`.
- Line 163, `LABELS`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 343–344:

```
343 if __name__ == "__main__":
344     main()
```

Read this block:

- Line 343: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 344: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

expit

Original function: `expit`. Source begins at line 21.

Evaluates the logistic probability with an exponential of a nonpositive magnitude to avoid overflow. Its input can be an array. This is a numerical transformation, not a demand-estimation call.

Definition and stated intent:

Source lines 21–21:

```
21 def expit(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 22–23:

```
22 x = np.asarray(x)
23 small = np.exp(-np.abs(x))
```

Read this block:

- Line 22, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 23, `small`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 24–24:

```
24 return np.where(x >= 0, 1/(1+small), small/(1+small))
```

Read this block:

- Line 24: return the result of calling `np.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

golden maximum

Original function: `golden_maximum`. Source begins at line 26.

Refines a bracketed scalar maximizer by golden-section interval reduction, reusing previous function values. This is a numerical search, not a proof that a multimodal objective has only one peak.

Definition and stated intent:

Source lines 26–26:

```
26 def golden_maximum(function, lo, hi):
```

`def` creates this callable; the parentheses name its inputs (`function`, `lo`, `hi`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 27–29:

```
27 ratio = (math.sqrt(5)-1)/2
28 left, right = hi-ratio*(hi-lo), lo+ratio*(hi-lo)
29 fl, fr = function(left), function(right)
```

Read this block:

- Line 27, `ratio`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 28, `(left, right)`: Unpack a tuple containing 2 displayed entries into these targets in order.

- Line 29, `(fl, fr)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `function`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 30–40:

```
for _ in range(100):
    if hi-lo <= 1e-10*(1+abs(left)+abs(right)):
        break
    if fl > fr:
        hi, right, fr = right, left, fl
        left = hi-ratio*(hi-lo)
        fl = function(left)
    else:
        lo, left, fl = left, right, fr
        right = lo+ratio*(hi-lo)
        fr = function(right)
```

Read this block:

- Line 30: for each entry yielded by `range(100)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 31: test `hi - lo <= 1e-10 * (1 + abs(left) + abs(right))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 32: exit the nearest enclosing loop immediately, rather than merely skipping one iteration.
- Line 33: test `fl > fr`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 34, `(hi, right, fr)`: Unpack the tuple `(right, left, fl)` into these targets in order.
- Line 35, `left`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 36, `fl`: Set this name to the result of calling `function(...)`; the arguments and preceding method chain are shown above.
- Line 38, `(lo, left, fl)`: Unpack the tuple `(left, right, fr)` into these targets in order.
- Line 39, `right`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 40, `fr`: Set this name to the result of calling `function(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 41–41:

```
return (left, fl) if fl > fr else (right, fr)
```

Read this block:

- Line 41: return the true-branch value if `fl > fr` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

scalar maximum

Original function: `scalar_maximum`. Source begins at line 43.

Evaluates a scalar objective on a grid, refines every detected local peak and includes endpoints, then keeps the best candidate. Hidden-state mixtures can be multimodal, so assuming a single global peak would be unsafe.

Definition and stated intent:

Source lines 43–44:

```
43 def scalar_maximum(function, bounds, points=161):
44     """Search every grid-local maximum, including boundaries (mixtures may multimode)."""
```

`def` creates this callable; the parentheses name its inputs (`function`, `bounds`, `points`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 45–45:

```
45 grid = np.linspace(*bounds, points)
```

Read this block:

- Line 45, `grid`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 47–52:

```
47 try:
48     values = np.asarray(function(grid))
49     if values.shape != grid.shape:
50         raise ValueError("Scalar objective")
51 except (ValueError, TypeError):
52     values = np.array([function(x) for x in grid])
```

Read this block:

- Line 47: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 48, `values`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 49: test `values.shape != grid.shape`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 50: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 51: catch `(ValueError, TypeError)` from the preceding try; the optional as-name holds the error object for logging.
- Line 52, `values`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `function`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 53–53:

```
53 candidates = [(values[0], grid[0]), (values[-1], grid[-1])]
```

Read this block:

- Line 53, `candidates`: Set this name to a list containing 2 displayed entries.

Source lines 54–57:

```
54 for i in range(1, points - 1):
55     if values[i] >= max(values[i - 1], values[i + 1]) and values[i] > min(values[i - 1], values[i + 1]):
56         price, value = golden_maximum(function, grid[i - 1], grid[i + 1])
57         candidates.append((value, price))
```

Read this block:

- Line 54: for each entry yielded by `range(1, points - 1)`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 55: test `values[i] >= max(values[i - 1], values[i + 1]) and values[i] > min(values[i -...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 56, `(price, value)`: Unpack the result of calling `golden_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 57: call `candidates.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `golden_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.

Source lines 58–58:

```
58 value, price = max(candidates)
```

Read this block:

- Line 58, `(value, price)`: Unpack the result of calling `max(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 59–59:

```
59 return float(price), float(value)
```

Read this block:

- Line 59: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.

static price

Original function: `static_price`. Source begins at line 61.

Solves the transformed binary-logit price first-order condition by bracketing and bisection for signed alpha<0. The effective cost can incorporate continuation value, allowing introductory subsidies; do not impose the coupon floor from empirical pricing on this theoretical solver.

Definition and stated intent:

Source lines 61–62:

```
61 def static_price(intercept, alpha, effective_cost=0.):
62     """Signed alpha<0; monotone transformed FOC, including introductory subsidies."""
```

`def` creates this callable; the parentheses name its inputs (`intercept`, `alpha`, `effective_cost`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 63–64:

```
63 def foc(p):
64     return alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p, -745, 700))
```

Read this block:

- Line 63: define nested callable `foc`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 65–65:

```
65 lo, hi = effective_cost, effective_cost+max(2., 2/abs(alpha))
```

Read this block:

- Line 65, `(lo, hi)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.

Source lines 66–67:

```
66 while foc(hi) > 0:
67     hi = lo+2*(hi-lo)
```

Read this block:

- Line 66: repeat while `foc(hi) > 0` remains true. Unlike for/range, this loop need not know the iteration count beforehand; its body must eventually change the condition.
- Line 67, `hi`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `foc`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 68–73:

```
68 for _ in range(100):
69     mid = (lo+hi)/2
70     if foc(mid) > 0:
71         lo = mid
72     else:
73         hi = mid
```

Read this block:

- Line 68: for each entry yielded by `range(100)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 69, `mid`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 70: test `foc(mid) > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 71, `lo`: Set this name to the object currently named `mid`; plain assignment does not copy it.
- Line 73, `hi`: Set this name to the object currently named `mid`; plain assignment does not copy it.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Source lines 74–74:

```
74 return (lo+hi)/2
```

Read this block:

- Line 74: return division of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

static price.foc

Original function: `static_price.foc`. Source begins at line 63.

This nested function returns the transformed price first-order-condition residual for a trial scalar. Captured slope, intercept and effective cost remain fixed during the root search. A zero is a candidate optimal price under the maintained monotonicity argument.

Definition and stated intent:

Source lines 63–63:

```
63 def foc(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 64–64:

```
64 return alpha*(p-effective_cost)+1+np.exp(np.clip(intercept+alpha*p, -745, 700))
```

Read this block:

- Line 64: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point range.

supply solution

Original function: `supply_solution`. Source begins at line 76.

Solves the known-state baseline, hidden-initial-state case, and commitment case for one fixed type and prior. Both terminal histories enter expected profits. The hidden-state case pools initial histories within a known type; it does not pool different gamma types. The commitment case chooses two numbers before the first purchase and cannot adapt p2 to h1.

Definition and stated intent:

Source lines 76–77:

```
76 def supply_solution(gamma, prior=.5, beta=2., alpha=-1., delta=.92, cost=0.):
77     """Known type; hidden h0, then either observe h1 or commit to p1,p2."""
```

`def` creates this callable; the parentheses name its inputs (`gamma`, `prior`, `beta`, `alpha`, `delta`, `cost`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 78–80:

```
78 if not (np.isfinite([gamma, prior, beta, alpha, delta, cost]).all()
79         and alpha < 0 and 0 <= prior <= 1 and 0 <= delta <= 1):
80     raise ValueError("Need finite inputs, alpha<0, prior/delta in [0,1].")
```

Read this block:

- Line 78: test `not (np.isfinite([gamma, prior, beta, alpha, delta, cost]).all() and alpha < ...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 80: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 82–87:

```
82 intercepts = beta + np.array([-gamma, gamma])
83 weights = np.array([prior, 1 - prior])
84 p2 = np.array([static_price(x, alpha, cost) for x in intercepts])
85 s2 = expit(intercepts + alpha * p2)
86 W2 = (p2 - cost) * s2
87 adjustment = delta * (W2[0] - W2[1])
```

Read this block:

- Line 82, `intercepts`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 83, `weights`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 84, `p2`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 85, `s2`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 86, `W2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 87, `adjustment`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `static_price`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 89–90:

```
89 def share1(p):
90     return expit(intercepts + alpha * np.asarray(p)[..., None]) @ weights
```

Read this block:

- Line 89: define nested callable `share1`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 92–94:

```
92 def adaptive_value(p):
93     s = share1(p)
94     return s * (p - cost) + delta * (s * W2[0] + (1 - s) * W2[1])
```

Read this block:

- Line 92: define nested callable `adaptive_value`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 96–100:

```
96 scale = (20 + abs(beta) + abs(gamma) + abs(alpha * cost)) / abs(alpha)
97 bounds = (cost - scale, cost + scale)
98 pooled_p1, adaptive = scalar_maximum(adaptive_value, bounds)
99 # Rechecking a wider domain catches a binding numerical search boundary.
100 wide_p1, wide_value = scalar_maximum(adaptive_value, (cost - 2*scale, cost + 2*scale), 321)
```

Read this block:

- Line 96, `scale`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 97, `bounds`: Set this name to a tuple containing 2 displayed entries.
- Line 98, `(pooled_p1, adaptive)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 100, `(wide_p1, wide_value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 101–102:

```
101 if wide_value > adaptive + 1e-8:
102     pooled_p1, adaptive = wide_p1, wide_value
```

Read this block:

- Line 101: test `wide_value > adaptive + 1e-08`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 102, `(pooled_p1, adaptive)`: Unpack the tuple `(wide_p1, wide_value)` into these targets in order.

Source lines 103–104:

```
103 known_p1 = np.array([static_price(x, alpha, cost - adjustment) for x in intercepts])
104 known_values = []
```

Read this block:

- Line 103, `known_p1`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 104, `known_values`: Set this name to the list `[]`.

Source lines 105–107:

```
105 for x, price in zip(intercepts, known_p1):
106     s = expit(x + alpha * price)
107     known_values.append(s*(price-cost) + delta*(s*W2[0]+(1-s)*W2[1]))
```

Read this block:

- Line 105: for each entry yielded by `zip(intercepts, known_p1)`, bind/unpack it into `(x, price)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 106, `s`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 107: call `known_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 109–116:

```
109 def committed_profile(second, return_price=False):
110     terminal = expit(intercepts + alpha * second)
111     Wj, W0 = terminal*(second-cost)
112     def objective(first):
113         s = share1(first)
114         return s*(first-cost)+delta*(s*Wj+(1-s)*W0)
115     price, value = scalar_maximum(objective, (cost-2*scale, cost+2*scale))
116     return (price, value) if return_price else value
```

Read this block:

- Line 109: define nested callable `committed_profile`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 120–131:

```
120 if delta == 0:
121     cp1, cp2, committed = pooled_p1, static_price(beta, alpha, cost), adaptive
122 else:
123     cp2, committed = scalar_maximum(committed_profile, (cost, cost+scale))
124     cp1, committed = committed_profile(cp2, True)
125     # Independent wider and denser profile scan.
126     check_p2, check_value = scalar_maximum(committed_profile, (cost, cost+2*scale), 321)
127     if check_value > committed+1e-9:
128         cp2 = check_p2
129         cp1, committed = committed_profile(cp2, True)
130     if min(cp2-cost, cost+2*scale-cp2) < scale*1e-5:
131         raise RuntimeError("Committed optimum is near a search bound; inspect calibration.")
```

Read this block:

- Line 120: test `delta == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 121, `(cp1, cp2, committed)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 123, `(cp2, committed)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 124, `(cp1, committed)`: Unpack the result of calling `committed_profile(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 126, `(check_p2, check_value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 127: test `check_value > committed + 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 128, `cp2`: Set this name to the object currently named `check_p2`; plain assignment does not copy it.
- Line 129, `(cp1, committed)`: Unpack the result of calling `committed_profile(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 130: test `min(cp2 - cost, cost + 2 * scale - cp2) < scale * 1e-05`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 131: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `committed_profile`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 132–132:

```
132 known = float(weights @ known_values)
```

Read this block:

- Line 132, `known`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 133–134:

```
133 if not known + 1e-7 >= adaptive >= committed - 1e-7:
134     raise AssertionError("Information-value ordering failed.")
```

Read this block:

- Line 133: test `not known + 1e-07 >= adaptive >= committed - 1e-07`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 134: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 135–141:

```
135 s = share1(pooled_p1)
136 derivative = float(weights @ (alpha*expit(intercepts+alpha*pooled_p1)
137                              * expit(-intercepts-alpha*pooled_p1)))
138 committed_first_share = share1(cp1)
139 committed_first_derivative = float(weights @ (alpha*expit(intercepts+alpha*cp1)*expit(-intercepts-alpha*cp1)))
140 terminal = expit(intercepts+alpha*cp2)
141 terminal_weights = np.array([committed_first_share, 1-committed_first_share])
```

Read this block:

- Line 135, `s`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.
- Line 136, `derivative`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 138, `committed_first_share`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.
- Line 139, `committed_first_derivative`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 140, `terminal`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.

- Line 141, `terminal_weights`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `share1`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 142–143:

```
142 committed_foc1 =
    ↪  committed_first_share+committed_first_derivative*(cp1-cost+delta*(cp2-cost)*(terminal[0]-terminal[1]))
143 committed_foc2 = float(terminal_weights @ (terminal+alpha*(cp2-cost)*terminal*(1-terminal))) if delta > 0 else 0.
```

Read this block:

- Line 142, `committed_foc1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 143, `committed_foc2`: Set this name to the true-branch value if `delta > 0` holds, otherwise the else-branch value.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 144–152:

```
144 return dict(gamma=gamma, prior_inside=prior, beta=beta, alpha=alpha, delta=delta, cost=cost,
145             known_p1_inside=known_p1[0], known_p1_outside=known_p1[1],
146             known_value=known, hidden_p1=pooled_p1, adaptive_p2_inside=p2[0],
147             adaptive_p2_outside=p2[1], hidden_value=adaptive,
148             committed_p1=cp1, committed_p2=cp2, committed_value=committed,
149             initial_information_value=known-adaptive,
150             second_period_flexibility_value=adaptive-committed,
151             committed_foc1=committed_foc1, committed_foc2=committed_foc2,
152             hidden_foc=s+derivative*(pooled_p1-cost+adjustment))
```

Read this block:

- Line 144: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

supply solution.share1

Original function: `supply_solution.share1`. Source begins at line 89.

Returns first-period binary-logit demand, or its initial-state mixture, at a trial price. The function closes over the current theoretical type and prior. Initial history is integrated as probabilities, not averaged as utilities.

Definition and stated intent:

Source lines 89–89:

```
89 def share1(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 90–90:

```
90 return expit(intercepts + alpha * np.asarray(p)[..., None]) @ weights
```

Read this block:

- Line 90: return matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

supply solution.adaptive value

Original function: `supply_solution.adaptive_value`. Source begins at line 92.

Evaluates current profit plus discounted, state-contingent optimized terminal values. The first choice changes the probabilities of both continuation histories, which is why its derivative contains their value difference.

Definition and stated intent:

Source lines 92–92:

```
92 def adaptive_value(p):
```

`def` creates this callable; the parentheses name its inputs (`p`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 93–93:

```
93 s = share1(p)
```

Read this block:

- Line 93, `s`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `share1`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 94–94:

```
94 return s * (p - cost) + delta * (s * W2[0] + (1 - s) * W2[1])
```

Read this block:

- Line 94: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

supply solution.committed profile

Original function: `supply_solution.committed_profile`. Source begins at line 109.

For a trial first price, solves the best single committed terminal price and returns its value/profile. This inner optimization must respect the information restriction: p2 is common across realized h1.

Definition and stated intent:

Source lines 109–109:

```
109 def committed_profile(second, return_price=False):
```

`def` creates this callable; the parentheses name its inputs (`second`, `return_price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 110–111:

```
110 terminal = expit(intercepts + alpha * second)
111 Wj, W0 = terminal*(second-cost)
```

Read this block:

- Line 110, `terminal`: Set this name to the result of calling `expit(...)`; the arguments and preceding method chain are shown above.
- Line 111, `(Wj, W0)`: Unpack multiplication of the displayed operands; parentheses control which subexpressions are evaluated together into these targets in order.

Syntax and calls used here:

- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.

Source lines 112–114:

```
112 def objective(first):
113     s = share1(first)
114     return s*(first-cost)+delta*(s*Wj+(1-s)*W0)
```

Read this block:

- Line 112: define nested callable `objective`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 115–115:

```
115 price, value = scalar_maximum(objective, (cost-2*scale, cost+2*scale))
```

Read this block:

- Line 115, `(price, value)`: Unpack the result of calling `scalar_maximum(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `scalar_maximum`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 116–116:

```
116 return (price, value) if return_price else value
```

Read this block:

- Line 116: return the true-branch value if `return_price` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

supply solution.committed profile.objective

Original function: `supply_solution.committed_profile.objective`. Source begins at line 112.

Defines the scalar objective for the surrounding search using captured data and parameters. Inspect the displayed expression to identify whether it returns a profit or its negative; minimizers and maximizers require opposite conventions.

Definition and stated intent:

Source lines 112–112:

```
112 def objective(first):
```

`def` creates this callable; the parentheses name its inputs (`first`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 113–113:

```
113 s = share1(first)
```

Read this block:

- Line 113, `s`: Set this name to the result of calling `share1(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `share1`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 114–114:

```
114 return s*(first-cost)+delta*(s*Wj+(1-s)*W0)
```

Read this block:

- Line 114: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

plotting

Original function: `plotting`. Source begins at line 154.

Selects a noninteractive plotting backend and a writable temporary configuration directory, then returns pyplot. It does not read household data. Backend selection precedes importing pyplot so server runs do not require a display.

Definition and stated intent:

Source lines 154–154:

```
154 def plotting():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 155–158:

```
155 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
156 import matplotlib
157 matplotlib.use("Agg")
158 import matplotlib.pyplot as plt
```

Read this block:

- Line 155: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 156: import `matplotlib`. An as-name is a convenient local alias, not another library.

- Line 157: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 158: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 159–159:

```
159 return plt
```

Read this block:

- Line 159: return the object currently named `plt`; plain assignment does not copy it. This ends the call.

outcome tables

Original function: `outcome_tables`. Source begins at line 166.

Exports history-specific prices, quantities and profits plus properly probability-weighted ex ante outcomes. Conditional realized-history values and expected profits answer different questions; averaging conditional profits without their path probabilities would be wrong.

Definition and stated intent:

Source lines 166–167:

```
166 def outcome_tables(result):
167     """All four realized histories per case, plus correctly weighted ex ante objects."""
```

`def` creates this callable; the parentheses name its inputs (`result`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 168–169:

```
168 g, beta, alpha, delta, cost, pi = [result[k] for k in ["gamma", "beta", "alpha", "delta", "cost", "prior_inside"]]
169 paths, summary = [], []
```

Read this block:

- Line 168, `(g, beta, alpha, delta, cost, pi)`: Unpack a list computed by the displayed for-expression into these targets in order.
- Line 169, `(paths, summary)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 170–208:

```
170 for case in CASES:
171     case_paths = []
172     for h0, weight in [("j", pi), ("0", 1-pi)]:
173         if case == "known_state":
174             p1 = result["known_p1_inside" if h0 == "j" else "known_p1_outside"]
175         else:
176             p1 = result["hidden_p1" if case == "hidden_initial_state" else "committed_p1"]
177         s1 = float(expit(beta+(-1 if h0 == "j" else 1)*g+alpha*p1))
178         conditional_value = 0.
179         local = []
180         for h1, transition in [("j", s1), ("0", 1-s1)]:
```

```
            p2 = result["committed_p2"] if case == "ex_ante_commitment" else result["adaptive_p2_inside" if h1 == "j" else "adaptive_p2_outside"]
            s2 = float(expit(beta+(-1 if h1 == "j" else 1)*g+alpha*p2))
            # Realize the first choice; integrate the terminal taste shock.
            conditional_profit = (p1-cost)*(h1 == "j")+delta*(p2-cost)*s2
            conditional_value += transition*conditional_profit
            row = dict(gamma=g, prior_inside=pi, case=case, h0=h0, h1=h1,
                       initial_probability=weight, transition_probability=transition,
                       path_probability=weight*transition, p1=p1, p2=p2,
                       price_ratio=p1/p2 if abs(p2) > 1e-12 else np.nan,
                       first_purchase_probability=s1, second_purchase_probability=s2,
                       conditional_profit=conditional_profit)
            local.append(row)
        for row in local:
            row["optimal_profit_given_h0"] = conditional_value
        case_paths.extend(local)
    probability = np.array([r["path_probability"] for r in case_paths])
    if not np.isclose(probability.sum(), 1.):
        raise AssertionError("Path probabilities do not sum to one.")
    mean_p1, mean_p2, profit = [float(probability @ np.array([r[k] for r in case_paths]))
                                for k in ["p1", "p2", "conditional_profit"]]
    expected_ratio = float(probability @ np.array([r["price_ratio"] for r in case_paths]))
    expected_key = {"known_state": "known_value", "hidden_initial_state": "hidden_value", "ex_ante_commitment": "committed_value"}[case]
    if not np.isclose(profit, result[expected_key], atol=1e-8):
        raise AssertionError("Path profits do not reproduce optimized objective.")
    summary.append(dict(gamma=g, prior_inside=pi, case=case, expected_p1=mean_p1,
                        expected_p2=mean_p2, ratio_of_expected_prices=mean_p1/mean_p2 if abs(mean_p2) > 1e-12 else np.nan,
                        expected_price_ratio=expected_ratio, optimal_profit=profit))
    paths.extend(case_paths)
```

Read this block:

- Line 170: for each entry yielded by `CASES`, bind/unpack it into `case` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 171, `case_paths`: Set this name to the list `[]`.
- Line 172: for each entry yielded by `[('j', pi), ('0', 1 - pi)]`, bind/unpack it into `(h0, weight)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 173: test `case == 'known_state'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 174, `p1`: Set this name to the entry/slice selected by `result['known_p1_inside' if h0 == 'j' else 'known_p1_outs...`; brackets select from the object to their left.
- Line 176, `p1`: Set this name to the entry/slice selected by `result['hidden_p1' if case == 'hidden_initial_state' else...`; brackets select from the object to their left.
- Line 177, `s1`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 178, `conditional_value`: Set this name to the literal value `0.0`.
- Line 179, `local`: Set this name to the list `[]`.
- Line 180: for each entry yielded by `[('j', s1), ('0', 1 - s1)]`, bind/unpack it into `(h1, transition)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 181, `p2`: Set this name to the true-branch value if `case == 'ex_ante_commitment'` holds, otherwise the else-branch value.
- Line 182, `s2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 184, `conditional_profit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 185: update `conditional_value` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

- Line 186, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 192: call `local.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 193: for each entry yielded by `local`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 194, `row['optimal_profit_given_h0']`: Store the object currently named `conditional_value`; plain assignment does not copy it in the selected entry/column.
- Line 195: call `case_paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 196, `probability`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 197: test `not np.isclose(probability.sum(), 1.0)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 198: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 199, `(mean_p1, mean_p2, profit)`: Unpack a list computed by the displayed for-expression into these targets in order.
- Line 201, `expected_ratio`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 202, `expected_key`: Set this name to the entry/slice selected by `{'known_state': 'known_value', 'hidden_initial_state': 'h...`; brackets select from the object to their left.
- Line 203: test `not np.isclose(profit, result[expected_key], atol=1e-08)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 204: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 205: call `summary.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 208: call `paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `expit`: Evaluate the logistic function stably, avoiding overflow from a naive exp(x)/(1+exp(x)) formula.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 209–209:

```
209 return paths, summary
```

Read this block:

- Line 209: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

make figures

Original function: `make_figures`. Source begins at line 212.

Turns already computed supply solution tables into case-comparison figures; it does not add another solver or change the information assumptions. Plot labels distinguish state-contingent policies from committed prices.

Definition and stated intent:

Source lines 212–212:

```
212 def make_figures(paths, summary, output):
```

`def` creates this callable; the parentheses name its inputs (`paths`, `summary`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 213–213:

```
213 plt = plotting()
```

Read this block:

- Line 213, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 214–255:

```
214 for prior in summary.prior_inside.unique():
215     stem = f"prior_{prior:g}".replace(".", "p")
216     subset = paths[paths.prior_inside == prior]
217     fig, axes = plt.subplots(3, 3, figsize=(16, 12))
218     for column, case in enumerate(CASES):
219         data = subset[subset.case == case]
220         if case == "known_state":
221             for h0 in ["0", "j"]:
222                 initial = data[(data.h0 == h0) & (data.h1 == "j")]
223                 axes[0, column].plot(initial.gamma, initial.p1, label=f"p1(h0={h0})")
224         else:
225             initial = data[(data.h0 == "0") & (data.h1 == "j")]
226             axes[0, column].plot(initial.gamma, initial.p1, label="p1")
227         if case == "ex_ante_commitment":
228             axes[0, column].plot(initial.gamma, initial.p2, label="p2")
229         else:
230             for h1 in ["0", "j"]:
231                 terminal = data[(data.h0 == "0") & (data.h1 == h1)]
232                 axes[0, column].plot(terminal.gamma, terminal.p2, "--", label=f"p2(h1={h1})")
233         for (h0, h1), history in data.groupby(["h0", "h1"]):
234             # Duplicate ratios in restricted cases need only one curve.
235             if case != "known_state" and h0 != "0":
236                 continue
237             if case == "ex_ante_commitment" and h1 != "j":
238                 continue
239             axes[1, column].plot(history.gamma, history.price_ratio, label=f"h0={h0}, h1={h1}")
240         for h0 in ["0", "j"]:
241             initial = data[(data.h0 == h0) & (data.h1 == "j")]
242             axes[2, column].plot(initial.gamma, initial.optimal_profit_given_h0, label=f"E[profit|h0={h0}]")
243         expected = summary[(summary.prior_inside == prior) & (summary.case == case)]
244         axes[2, column].plot(expected.gamma, expected.optimal_profit, "k--", label="Ex ante E[profit]")
245         axes[0, column].set_title(LABELS[case])
246     for row, ylabel in enumerate(["Optimal prices", "Path-specific p1 / p2", "Optimal discounted profit"]):
```

```
247         for ax in axes[row]:
248             ax.set(xlabel="gamma", ylabel=ylabel)
249             ax.axhline(0, color="gray", linewidth=.5)
250             ax.legend(fontsize=7)
251     fig.suptitle(f"Supply cases: Pr(h0=j)={prior:g}")
252     fig.tight_layout()
253     fig.savefig(output / f"supply_cases_{stem}.png", dpi=180)
254     fig.savefig(output / f"supply_cases_{stem}.pdf")
255     plt.close(fig)
```

Source lines 256–280:

```
256
257 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
258 for case, group in summary[summary.prior_inside == prior].groupby("case", sort=False):
259     axes[0].plot(group.gamma, group.optimal_profit, label=LABELS[case])
260     axes[1].plot(group.gamma, group.ratio_of_expected_prices, label=LABELS[case])
261     axes[2].plot(group.gamma, group.expected_price_ratio, label=LABELS[case])
262 for ax, title in zip(axes, ["Optimal expected profit", "E[p1] / E[p2]", "E[p1 / p2]"]):
263     ax.set(xlabel="gamma", title=title)
264     ax.legend(fontsize=7)
265 fig.tight_layout()
266 fig.savefig(output / f"supply_comparison_{stem}.png", dpi=180)
267 fig.savefig(output / f"supply_comparison_{stem}.pdf")
268 plt.close(fig)
269
270 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
271 for ax, case in zip(axes, CASES):
272     data = subset[subset.case == case]
273     for (h0, h1), history in data.groupby(["h0", "h1"]):
274         ax.plot(history.gamma, history.conditional_profit, label=f"h0={h0}, h1={h1}")
275     ax.set(xlabel="gamma", ylabel="Path-conditional discounted profit", title=LABELS[case])
276     ax.legend(fontsize=7)
277 fig.tight_layout()
278 fig.savefig(output / f"supply_history_profit_{stem}.png", dpi=180)
279 fig.savefig(output / f"supply_history_profit_{stem}.pdf")
280 plt.close(fig)
```

Read this block:

- Line 214: for each entry yielded by `summary.prior_inside.unique()`, bind/unpack it into `prior` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 215, `stem`: Set this name to the result of calling `f'prior_{prior:g}'.replace(...)`; the arguments and preceding method chain are shown above.
- Line 216, `subset`: Set this name to the entry/slice selected by `paths[paths.prior_inside == prior]`; brackets select from the object to their left.
- Line 217, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 218: for each entry yielded by `enumerate(CASES)`, bind/unpack it into `(column, case)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 219, `data`: Set this name to the entry/slice selected by `subset[subset.case == case]`; brackets select from the object to their left.
- Line 220: test `case == 'known_state'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 221: for each entry yielded by `['0', 'j']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 222, `initial`: Set this name to the entry/slice selected by `data[(data.h0 == h0) & (data.h1 == 'j')]`; brackets select from the object to their left.
- Line 223: call `axes[0, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 225, `initial`: Set this name to the entry/slice selected by `data[(data.h0 == '0') & (data.h1 == 'j')]`; brackets select from the object to their left.
- Line 226: call `axes[0, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 227: test `case == 'ex_ante_commitment'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 228: call `axes[0, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 230: for each entry yielded by `['0', 'j']`, bind/unpack it into `h1` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 231, `terminal`: Set this name to the entry/slice selected by `data[(data.h0 == '0') & (data.h1 == h1)]`; brackets select from the object to their left.
- Line 232: call `axes[0, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 233: for each entry yielded by `data.groupby(['h0', 'h1'])`, bind/unpack it into `((h0, h1), history)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 235: test `case != 'known_state' and h0 != '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 236: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 237: test `case == 'ex_ante_commitment' and h1 != 'j'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 238: skip the rest of this iteration and continue the nearest enclosing loop.
- Line 239: call `axes[1, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 240: for each entry yielded by `['0', 'j']`, bind/unpack it into `h0` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 241, `initial`: Set this name to the entry/slice selected by `data[(data.h0 == h0) & (data.h1 == 'j')]`; brackets select from the object to their left.
- Line 242: call `axes[2, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 243, `expected`: Set this name to the entry/slice selected by `summary[(summary.prior_inside == prior) & (summary.case =...`; brackets select from the object to their left.
- Line 244: call `axes[2, column].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 245: call `axes[0, column].set_title` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 246: for each entry yielded by `enumerate(['Optimal prices', 'Path-specific p1 / p2', 'Optimal...`, bind/unpack it into `(row, ylabel)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 247: for each entry yielded by `axes[row]`, bind/unpack it into `ax` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 248: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 249: call `ax.axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 250: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 251: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 252: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 253: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 254: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 255: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented

side effect or assertion is what matters.
- Line 257, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 258: for each entry yielded by `summary[summary.prior_inside == prior].groupby('case', sort=Fa...`, bind/unpack it into `(case, group)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 259: call `axes[0].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 260: call `axes[1].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 261: call `axes[2].plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 262: for each entry yielded by `zip(axes, ['Optimal expected profit', 'E[p1] / E[p2]', 'E[p1 /...`, bind/unpack it into `(ax, title)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 263: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 264: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 265: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 266: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 267: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 268: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 270, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 271: for each entry yielded by `zip(axes, CASES)`, bind/unpack it into `(ax, case)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 272, `data`: Set this name to the entry/slice selected by `subset[subset.case == case]`; brackets select from the object to their left.
- Line 273: for each entry yielded by `data.groupby(['h0', 'h1'])`, bind/unpack it into `((h0, h1), history)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 274: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 275: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 276: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 277: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 278: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 279: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 280: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

self test

Original function: `self_test`. Source begins at line 283.

Runs generated examples and assertions to check numerical calculations without launching the user-selected empirical exercise. Read the specific body to distinguish the CSV-blocked simple split checks from earlier complete export tests that reopen their own synthetic CSVs. No private input is needed.

Definition and stated intent:

Source lines 283–283:

```
283 def self_test():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 284–295:

```
284 for gamma, prior, delta in [(0., .5, .92), (-2., .25, .92), (1.5, .75, .92),
285                             (-1., 0., .92), (1., 1., .92), (1., .5, 0.)]:
286     solution = supply_solution(gamma, prior, delta=delta)
287     paths, summary = outcome_tables(solution)
288     assert len(paths) == 12 and len(summary) == 3
289     for key in ["hidden_foc", "committed_foc1", "committed_foc2"]:
290         assert abs(solution[key]) < 1e-6
291     if gamma == 0:
292         assert np.ptp([row["optimal_profit"] for row in summary]) < 1e-7
293         assert all(np.isclose(row["price_ratio"], 1., atol=1e-6) for row in paths)
294     if prior in (0., 1.):
295         assert np.isclose(solution["known_value"], solution["hidden_value"], atol=1e-7)
```

Read this block:

- Line 284: for each entry yielded by `[(0.0, 0.5, 0.92), (-2.0, 0.25, 0.92), (1.5, 0.75, 0.92), (-1....`, bind/unpack it into `(gamma, prior, delta)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 286, `solution`: Set this name to the result of calling `supply_solution(...)`; the arguments and preceding method chain are shown above.
- Line 287, `(paths, summary)`: Unpack the result of calling `outcome_tables(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 288: require `len(paths) == 12 and len(summary) == 3` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError

checks are preferable for essential input validation.
- Line 289: for each entry yielded by `['hidden_foc', 'committed_foc1', 'committed_foc2']`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 290: require `abs(solution[key]) < 1e-06` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 291: test `gamma == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 292: require `np.ptp([row['optimal_profit'] for row in summary]) < 1e-07` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 293: require `all((np.isclose(row['price_ratio'], 1.0, atol=1e-06) for row in paths))` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.
- Line 294: test `prior in (0.0, 1.0)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 295: require `np.isclose(solution['known_value'], solution['hidden_value'], atol=1e-07)` to be true or raise AssertionError. Assertions document checks, but Python can disable plain assert statements in optimized mode; explicit ValueError checks are preferable for essential input validation.

Syntax and calls used here:

- `supply_solution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `outcome_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.
- `len`: Count elements/rows rather than summing their numerical values.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 296–296:

```
296 print("Supply checks passed: all cases, four histories, probabilities, profit reconstruction, FOCs, degenerate
    ↪  priors, gamma=0, delta=0.")
```

Read this block:

- Line 296: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

main

Original function: `main`. Source begins at line 299.

Parses the theoretical gamma grid, initial-history priors, cost/discount settings and output directory, then solves all three information cases and writes solution/outcome tables and figures. The self-test branch uses theoretical calibrations. No household sample or fitted empirical distribution is needed.

Definition and stated intent:

Source lines 299–299:

```
299 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 300–305:

```
300 parser = argparse.ArgumentParser(description=__doc__)
301 parser.add_argument("--output", type=Path, default=Path(__file__).resolve().parent / "supply_outputs")
302 parser.add_argument("--gamma-min", type=float, default=-3.)
303 parser.add_argument("--gamma-max", type=float, default=3.)
304 parser.add_argument("--gamma-points", type=int, default=61)
305 parser.add_argument("--priors", type=float, nargs="+", default=[.25, .5, .75])
```

Read this block:

- Line 300, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 301: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 302: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 303: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 304: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 305: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 306–311:

```
306 parser.add_argument("--beta", type=float, default=2.)
307 parser.add_argument("--alpha", type=float, default=-1., help="Signed price coefficient, negative.")
308 parser.add_argument("--delta", type=float, default=.92)
309 parser.add_argument("--cost", type=float, default=0.)
310 parser.add_argument("--self-test", action="store_true")
311 args = parser.parse_args()
```

Read this block:

- Line 306: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 307: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 308: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 309: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 310: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 311, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 312–314:

```
312 if args.self_test:
313     self_test()
314     return
```

Read this block:

- Line 312: test `args.self_test`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 313: call `self_test` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 314: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `self_test`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 315–316:

```
315 if args.gamma_points < 2 or args.gamma_max <= args.gamma_min:
316     parser.error("Need >=2 gamma points and gamma-max > gamma-min.")
```

Read this block:

- Line 315: test `args.gamma_points < 2 or args.gamma_max <= args.gamma_min`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 316: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 317–318:

```
317 args.output.mkdir(parents=True, exist_ok=True)
318 solutions, paths, summaries = [], [], []
```

Read this block:

- Line 317: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 318, `(solutions, paths, summaries)`: Unpack a tuple containing 3 displayed entries into these targets in order.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.

Source lines 319–327:

```
319 for prior in args.priors:
320     for i, gamma in enumerate(np.linspace(args.gamma_min, args.gamma_max, args.gamma_points)):
321         result = supply_solution(gamma, prior, args.beta, args.alpha, args.delta, args.cost)
322         history_rows, summary_rows = outcome_tables(result)
323         solutions.append(result)
324         paths.extend(history_rows)
325         summaries.extend(summary_rows)
326         if i % 10 == 0 or i == args.gamma_points-1:
```

```
327             print(f"Prior {prior:g}: solved {i+1}/{args.gamma_points} gamma values", flush=True)
```

Read this block:

- Line 319: for each entry yielded by `args.priors`, bind/unpack it into `prior` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 320: for each entry yielded by `enumerate(np.linspace(args.gamma_min, args.gamma_max, args.gam...`, bind/unpack it into `(i, gamma)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 321, `result`: Set this name to the result of calling `supply_solution(...)`; the arguments and preceding method chain are shown above.
- Line 322, `(history_rows, summary_rows)`: Unpack the result of calling `outcome_tables(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 323: call `solutions.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 324: call `paths.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 325: call `summaries.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 326: test `i % 10 == 0 or i == args.gamma_points - 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 327: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `supply_solution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `outcome_tables`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 328–332:

```
328 raw, histories, expected = pd.DataFrame(solutions), pd.DataFrame(paths), pd.DataFrame(summaries)
329 raw.to_csv(args.output / "supply_solutions.csv", index=False)
330 histories.to_csv(args.output / "supply_history_outcomes.csv", index=False)
331 expected.to_csv(args.output / "supply_expected_outcomes.csv", index=False)
332 residual = float(raw[["hidden_foc", "committed_foc1", "committed_foc2"]].abs().to_numpy().max())
```

Read this block:

- Line 328, `(raw, histories, expected)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 329: call `raw.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 330: call `histories.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 331: call `expected.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 332, `residual`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 333–334:

```
333 if residual > 1e-6:
334     raise AssertionError(f"FOC residual too large: {residual}")
```

Read this block:

- Line 333: test `residual > 1e-06`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 334: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 335–340:

```
335 make_figures(histories, expected, args.output)
336 manifest = dict(settings={k: str(v) if isinstance(v, Path) else v for k,v in vars(args).items()},
337                 maximum_foc_residual=residual, calibrations=len(raw), cases=CASES,
338                 script_sha256=hashlib.sha256(Path(__file__).read_bytes()).hexdigest())
339 (args.output / "run_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
340 print(f"All three supply cases completed. CSVs and PNG/PDF figures: {args.output}")
```

Read this block:

- Line 335: call `make_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 336, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 339: call `(args.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 340: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `len`: Count elements/rows rather than summing their numerical values.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.

Current readable demand workflow (final_simple)

../final_simple/demand/estimation.py

Readable demand estimator: editable settings, numbered sections, and visible run_estimation steps. Current defaults are heterogeneous_history and capacity 7. Section 3 contains split utility and persistent-type likelihood; section 6 exports population novelty/habit marginals. All supported earlier models remain available. No research functions are imported from final_draft.

Module setup and direct-execution blocks

Source lines 1–11:

```
"""Demand estimation. Edit Parameters, then run python3 demand/estimation.py.

Read from top to bottom: inputs -> weekly data -> histories -> probabilities ->
likelihood -> optimization -> standard errors -> saved results.
No calculations are imported from final_draft. All seven model choices remain.
"""
from pathlib import Path
from statistics import NormalDist
from types import SimpleNamespace
import argparse
import json
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: from `statistics`, import `NormalDist` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 9: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 11: import `json`. An as-name is a convenient local alias, not another library.

Source lines 12–18:

```
import os
import sys
import tempfile
from pprint import pformat

import numpy as np
import pandas as pd
```

Read this block:

- Line 12: import `os`. An as-name is a convenient local alias, not another library.
- Line 13: import `sys`. An as-name is a convenient local alias, not another library.
- Line 14: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 15: from `pprint`, import `pformat` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 17: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 18: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 21–27:

```
21 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "final_simple_matplotlib"))
22
23 import time
24 import polars as pl
25 from scipy.optimize import minimize
26 from scipy.special import logsumexp
27 from scipy.stats import t as student_t
```

Read this block:

- Line 21: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 23: import `time`. An as-name is a convenient local alias, not another library.
- Line 24: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 25: from `scipy.optimize`, import `minimize` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 26: from `scipy.special`, import `logsumexp` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 27: from `scipy.stats`, import `t as student_t` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 28–28:

```
28 from scipy.stats import norm
```

Read this block:

- Line 28: from `scipy.stats`, import `norm` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 35–39:

```
35 DATA_DIR = Path("/scratch/dtm63837/Kilts_Panel/Data")
36 sample = DATA_DIR / "summary_outputs/yogurt_estimation_sample.parquet"
37 flavors = DATA_DIR / "summary_outputs/estimation_flavor_reference.csv"
38 products_extra = None                # Optional flavor metadata for older inputs.
39 output = Path(__file__).resolve().parents[1] / "results" / "estimation_split"
```

Read this block:

- Line 35, `DATA_DIR`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 36, `sample`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 37, `flavors`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 38, `products_extra`: Set this name to the literal value `None`.
- Line 39, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.

Source lines 44–49:

```
44 model = "heterogeneous_history"
45 first_year = 2006
46 last_year = 2015
47 max_households = None                # Use 100 for a development sample.
48 capacity = 7                         # Pricing setup; use 0 for conditional flavor choice.
49 no_price_control = False             # False means include the IV residual control.
```

Read this block:

- Line 44, `model`: Set this name to the literal text `'heterogeneous_history'`.
- Line 45, `first_year`: Set this name to the literal value `2006`.
- Line 46, `last_year`: Set this name to the literal value `2015`.
- Line 47, `max_households`: Set this name to the literal value `None`.
- Line 48, `capacity`: Set this name to the literal value `7`.
- Line 49, `no_price_control`: Set this name to the literal value `False`.

Source lines 50–55:

```
50 household_dma_column = "dma_cd"
51 habit_weight = 0.7                   # Xi = (1-lambda)*novelty + lambda*own stock.
52 draws = 128                          # Even number of fixed integration draws.
53 starts = 3                           # Number of optimizer starting points.
54 maxiter = 600
55 seed = 306
```

Read this block:

- Line 50, `household_dma_column`: Set this name to the literal text `'dma_cd'`.
- Line 51, `habit_weight`: Set this name to the literal value `0.7`.
- Line 52, `draws`: Set this name to the literal value `128`.
- Line 53, `starts`: Set this name to the literal value `3`.
- Line 54, `maxiter`: Set this name to the literal value `600`.
- Line 55, `seed`: Set this name to the literal value `306`.

Source lines 56–61:

```
56 se_method = "hessian"               # hessian, bootstrap, or none.
57 bootstrap_reps = 49
58 bootstrap_starts = 2
59
60 FLAVORS = ["other", "berry", "plain"] # This column order is used in all arrays.
61 FLAVOR_FILE = flavors
```

Read this block:

- Line 56, `se_method`: Set this name to the literal text `'hessian'`.
- Line 57, `bootstrap_reps`: Set this name to the literal value `49`.
- Line 58, `bootstrap_starts`: Set this name to the literal value `2`.
- Line 60, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.
- Line 61, `FLAVOR_FILE`: Set this name to the object currently named `flavors`; plain assignment does not copy it.

Source lines 1254–1280:

```
1254 if __name__ == "__main__":
1255     # Collect the ordinary Python variables above into one settings record.
1256     # settings.draws means exactly the variable draws from the Parameters section.
1257     settings = SimpleNamespace(
1258         sample=sample,
1259         flavors=flavors,
1260         products_extra=products_extra,
1261         output=output,
1262         model=model,
1263         first_year=first_year,
1264         last_year=last_year,
1265         max_households=max_households,
1266         capacity=capacity,
1267         no_price_control=no_price_control,
1268         household_dma_column=household_dma_column,
1269         se_method=se_method,
1270         bootstrap_reps=bootstrap_reps,
1271         bootstrap_starts=bootstrap_starts,
1272         habit_weight=habit_weight,
1273         draws=draws,
1274         starts=starts,
1275         maxiter=maxiter,
1276         seed=seed,
1277     )
1278     if len(sys.argv) > 1:
1279         settings = read_command_line(settings)
1280     run_estimation(settings)
```

Read this block:

- Line 1254: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1257, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 1278: test `len(sys.argv) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1279, `settings`: Set this name to the result of calling `read_command_line(...)`; the arguments and preceding method chain are shown above.
- Line 1280: call `run_estimation` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `run_estimation`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `read_command_line`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

check dma

Original function: `check_dma`. Source begins at line 70.

This converts a pandas Series to integer DMA IDs, rejecting missing, nonnumeric, nonpositive, and fractional entries. `pd.to_numeric(errors="coerce")` turns bad strings into missing numbers so one Boolean mask can find all invalid rows. `codes == np.floor(codes)` checks integrality without rounding fractional inputs into a valid ID. Only after validation does `astype("int64")` convert the dtype. `description` makes an error identify the consumer

or retail source. Quietly replacing an invalid DMA with zero would contaminate own-market exclusion.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 70–71:

```
70 def check_dma(values, description):
71     """Normalize Nielsen's numeric DMA identifiers across panel and RMS files."""
```

`def` creates this callable; the parentheses name its inputs (`values`, `description`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 72–73:

```
72 codes = pd.to_numeric(values, errors="coerce")
73 valid = codes.notna() & (codes > 0) & (codes == np.floor(codes))
```

Read this block:

- Line 72, `codes`: Set this name to the result of calling `pd.to_numeric(...)`; the arguments and preceding method chain are shown above.
- Line 73, `valid`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.floor`: Take the greatest integer no larger than each value; equality with it tests whether a positive code is integral.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 74–75:

```
74 if not valid.all():
75     raise ValueError(f"Missing/invalid {description} DMA codes: {(~valid).sum():,}")
```

Read this block:

- Line 74: test `not valid.all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 75: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 76–76:

```
76 return codes.astype("int64")
```

Read this block:

- Line 76: return the result of calling `codes.astype(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

other dma instruments

Original function: `other_dma_instruments`. Source begins at line 79.

This retained helper accepts three pandas tables and returns weekly rows with UPC-matched flavor IVs. It is **not** the pooled instrument construction used by current `est.py`. It first averages retail-DMA prices outside the consumer's DMA per UPC/week, then averages across the exposure store's UPC basket.

Renaming the own-DMA table's columns makes its keys match the merge. `present` is a Boolean Series; converting it to integers subtracts one from the DMA count only when that own-market quote exists. The grouped `count` ignores missing values whereas `size` counts all rows. Comparing them detects an incompletely instrumented basket and marks the whole flavor IV missing, rather than quietly averaging only a subset. Several joins replace a much slower nested lookup loop.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 79–85:

```
79 def other_dma_instruments(weekly, exposure_upcs, dma_prices):
80     """Match each exposure-store UPC to its same-week price in OTHER DMAs.
81 
82     dma_prices has ONE mean package price per UPC/week/retail DMA. First average
83     these means over all DMAs except the CONSUMER's DMA, then average over the
84     exposure store's UPCs in a flavor. This matches the local price's UPC basket.
85     """
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `exposure_upcs`, `dma_prices`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 86–93:

```
86 dma_prices = dma_prices.copy()
87 dma_prices["dma_code"] = check_dma(dma_prices["dma_code"], "retail")
88 pk = ["upc", "week"]
89 totals = (dma_prices.groupby(pk)["dma_price"].agg(["sum", "count"])
90           .rename(columns={"sum": "sum_dma", "count": "n_dma"}).reset_index())
91 exposure = weekly[["store_code_uc", "week", "dma_cd"]].drop_duplicates()
92 exposure = exposure.merge(exposure_upcs, on=["store_code_uc", "week"],
93                           how="left", validate="many_to_many")
```

Read this block:

- Line 86, `dma_prices`: Set this name to the result of calling `dma_prices.copy(...)`; the arguments and preceding method chain are shown above.
- Line 87, `dma_prices['dma_code']`: Store the result of calling `check_dma(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 88, `pk`: Set this name to the list `['upc', 'week']`.
- Line 89, `totals`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 91, `exposure`: Set this name to the result of calling `.drop_duplicates(...)`; the arguments and preceding method chain are shown above.
- Line 92, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `check_dma`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.
- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 94–100:

```
94 exposure = exposure.merge(totals, on=pk, how="left", validate="many_to_one")
95 own = dma_prices.rename(columns={"dma_code": "dma_cd", "dma_price": "own_dma_price"})
96 exposure = exposure.merge(own, on=pk + ["dma_cd"], how="left", validate="many_to_one")
97 present = exposure["own_dma_price"].notna()
98 denominator = exposure["n_dma"] - present.astype(int)
99 exposure["iv"] = ((exposure["sum_dma"] - exposure["own_dma_price"].fillna(0))
100                  / denominator.where(denominator > 0))
```

Read this block:

- Line 94, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.
- Line 95, `own`: Set this name to the result of calling `dma_prices.rename(...)`; the arguments and preceding method chain are shown above.
- Line 96, `exposure`: Set this name to the result of calling `exposure.merge(...)`; the arguments and preceding method chain are shown above.
- Line 97, `present`: Set this name to the result of calling `exposure['own_dma_price'].notna(...)`; the arguments and preceding method chain are shown above.
- Line 98, `denominator`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 99, `exposure['iv']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 102–106:

```
102 keys = ["store_code_uc", "week", "dma_cd", "flavor"]
103 grouped = exposure.groupby(keys)["iv"].agg(["mean", "count", "size"])
104 grouped.loc[grouped["count"] != grouped["size"], "mean"] = np.nan
105 result = (grouped["mean"].unstack("flavor").reindex(columns=FLAVORS)
106           .rename(columns={f: "iv_" + f for f in FLAVORS}).reset_index())
```

Read this block:

- Line 102, `keys`: Set this name to the list `['store_code_uc', 'week', 'dma_cd', 'flavor']`.
- Line 103, `grouped`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 104, `grouped.loc[grouped['count'] != grouped['size'], 'mean']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 105, `result`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.unstack`: Move an index level into columns, such as turning long flavor counts into wide weekly outcomes.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 107–108:

```
107 return weekly.merge(result, on=["store_code_uc", "week", "dma_cd"], how="left",
108                     validate="many_to_one")
```

Read this block:

- Line 107: return the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above. This ends the call.

absorb effects

Original function: `absorb_effects`. Source begins at line 113.

Input `values` is a numerical matrix; `groups` lists row-aligned group labels such as week and DMA. Output is a copy with group means removed. Factorization converts arbitrary labels to dense integers. `np.add.at(means, code, residual)` accumulates rows into group sums even when indices repeat; ordinary fancy-index `means[code] += residual` may not accumulate repeats as intended.

`counts[:, None]` makes group counts a column so every variable is divided by its group size. The function cycles week and DMA demeaning until the biggest absolute change falls below tolerance, then returns. A maximum iteration limit raises on failure. A huge explicit dummy matrix is a valid alternative used in tests, but its memory grows with the number of fixed-effect groups. One pass is insufficient when group cells are unbalanced.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 113–118:

```
113 def absorb_effects(values, groups, tolerance=1e-10, maxiter=2000):
114     """Remove week and DMA means without a huge matrix of dummy variables.
115 
116     Repeated demeaning is needed because DMA/week cells are unbalanced.
117     Each household-week exposure has equal weight in the first stage.
118     """
```

`def` creates this callable; the parentheses name its inputs (`values`, `groups`, `tolerance`, `maxiter`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 119–120:

```
119 residual = np.asarray(values, dtype=float).copy()
120 codes = [pd.factorize(group, sort=True)[0] for group in groups]
```

Read this block:

- Line 119, `residual`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.
- Line 120, `codes`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 121–130:

```
121 for _ in range(maxiter):
122     previous = residual.copy()
123     for code in codes:
124         counts = np.bincount(code)
125         means = np.zeros((len(counts), residual.shape[1]))
126         np.add.at(means, code, residual)
127         means /= counts[:, None]
128         residual -= means[code]
129     if np.max(np.abs(residual - previous)) < tolerance:
130         return residual
```

Read this block:

- Line 121: for each entry yielded by `range(maxiter)`, bind/unpack it into `_` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 122, `previous`: Set this name to the result of calling `residual.copy(...)`; the arguments and preceding method chain are shown above.
- Line 123: for each entry yielded by `codes`, bind/unpack it into `code` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 124, `counts`: Set this name to the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above.
- Line 125, `means`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 126: call `np.add.at` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 127: update `means` using `/=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 128: update `residual` using `-=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 129: test `np.max(np.abs(residual - previous)) < tolerance`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 130: return the object currently named `residual`; plain assignment does not copy it. This ends the call.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.add.at`: Accumulate in place at possibly repeated indices; this avoids the lost-repeat behavior of buffered fancy-index updates.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 131–131:

```
131 raise ValueError("First-stage fixed-effect demeaning did not converge.")
```

Read this block:

- Line 131: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

add price controls

Original function: `add_price_controls`. Source begins at line 136.

This returns `(augmented_frame, diagnostic_table)`. For each flavor it selects covered prices, absorbs week/DMA effects jointly from price, instrument, and Xi, then partials Xi out of y and z. `within[:, 0]`, `[:, 1]`, `[:, 2:]` split those roles by known column order. `lstsq(...)[0]` keeps only the coefficient vector from the least-squares result tuple.

Dot products compute the IV slope after partialling. A small residual instrument sum of squares raises instead of dividing by an almost zero denominator. `np.linalg.lstsq` tolerates redundant controls more gracefully than explicitly inverting their cross-product matrix. The residual is assigned only to covered rows; unavailable alternatives retain zero placeholders. Cluster score sums, partial R-squared, and residual SD describe first-stage relevance and fit. They do not prove instrument validity. Reusing this function in bootstrap replications keeps the two-stage estimator consistent across samples.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 136–141:

```
136 def add_price_controls(frame):
137     """OLS separately by flavor: price ~ IV + all three Xi + week FE + DMA FE.
138 
139     Refit this function in every household bootstrap sample. Return residuals
140     in DOLLARS (no arbitrary rescaling) for the choice-model control function.
141     """
```

`def` creates this callable; the parentheses name its inputs (`frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 142–143:

```
142 result = frame.copy()
143 reports = []
```

Read this block:

- Line 142, `result`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 143, `reports`: Set this name to the list `[]`.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.

Source lines 144–183:

```
144 for flavor in FLAVORS:
145     columns = ["price_" + flavor, "iv_" + flavor] + ["xi_" + f for f in FLAVORS]
146     covered = np.isfinite(result["price_" + flavor]) & (result["price_" + flavor] > 0)
147     sample = result.loc[covered]
148     if sample.empty:
149         raise ValueError(f"No covered prices for the {flavor} first stage.")
150     hh, households = pd.factorize(sample["household_code"], sort=True)
151     values = sample[columns].to_numpy(float)
152     if not np.isfinite(values).all():
153         raise ValueError(f"Nonfinite first-stage inputs for {flavor}.")
154     within = absorb_effects(values, [sample["week"], sample["dma_cd"]])
155     y, z, controls = within[:, 0], within[:, 1], within[:, 2:]
156     # Partial out Xi before estimating the excluded instrument's coefficient.
157     # This tolerates redundant Xi columns, but NOT a redundant instrument.
158     z_partial = z - controls @ np.linalg.lstsq(controls, z, rcond=None)[0]
159     y_partial = y - controls @ np.linalg.lstsq(controls, y, rcond=None)[0]
160     z_ss = z_partial @ z_partial
161     if z_ss <= 1e-12 * max(1., values[:, 1] @ values[:, 1]):
162         raise ValueError(f"No excluded-instrument variation after controls for {flavor}.")
163     pi = (z_partial @ y_partial) / z_ss
164     control_beta = np.linalg.lstsq(controls, y - pi*z, rcond=None)[0]
165     residual = y - pi*z - controls @ control_beta
166     if np.std(residual) < 1e-10:
167         raise ValueError(f"First-stage residual for {flavor} is essentially zero.")
168     result["cf_" + flavor] = 0.
169     result.loc[covered, "cf_" + flavor] = residual
170     # Household-clustered partial-IV t/F diagnostic (not a weak-IV-robust test).
171     scores = np.bincount(hh, weights=z_partial * residual, minlength=len(households))
172     pi_se = np.sqrt((scores @ scores) / z_ss**2)
173     if len(households) > 1:
174         pi_se *= np.sqrt(len(households) / (len(households) - 1))
175     f_stat = float((pi / pi_se)**2) if pi_se > 0 else np.nan
176     restricted_ss = y_partial @ y_partial
177     partial_r2 = 1 - (residual @ residual) / restricted_ss if restricted_ss > 0 else np.nan
178     reports.append(dict(flavor=flavor, iv_coefficient=float(pi),
179                         iv_cluster_se=float(pi_se), iv_cluster_F=f_stat,
180                         iv_pvalue=float(2*student_t.sf(np.sqrt(f_stat), len(households)-1))
181                         if len(households) > 1 else np.nan,
182                         partial_R2=float(partial_r2), residual_sd=float(np.std(residual)),
183                         observations=len(sample), households=len(households)))
```

Read this block:

- Line 144: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 145, `columns`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 146, `covered`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 147, `sample`: Set this name to the entry/slice selected by `result.loc[covered]`; brackets select from the object to their left.
- Line 148: test `sample.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 149: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 150, `(hh, households)`: Unpack the result of calling `pd.factorize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 151, `values`: Set this name to the result of calling `sample[columns].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 152: test `not np.isfinite(values).all()`. Execute the indented branch only when it is true; an else/elif

branch handles its alternative.
- Line 153: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 154, `within`: Set this name to the result of calling `absorb_effects(...)`; the arguments and preceding method chain are shown above.
- Line 155, `(y, z, controls)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 158, `z_partial`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 159, `y_partial`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 160, `z_ss`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 161: test `z_ss <= 1e-12 * max(1.0, values[:, 1] @ values[:, 1])`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 162: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 163, `pi`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 164, `control_beta`: Set this name to the entry/slice selected by `np.linalg.lstsq(controls, y - pi * z, rcond=None)[0]`; brackets select from the object to their left.
- Line 165, `residual`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 166: test `np.std(residual) < 1e-10`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 167: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 168, `result['cf_' + flavor]`: Store the literal value `0.0` in the selected entry/column.
- Line 169, `result.loc[covered, 'cf_' + flavor]`: Store the object currently named `residual`; plain assignment does not copy it in the selected entry/column.
- Line 171, `scores`: Set this name to the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above.
- Line 172, `pi_se`: Set this name to the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above.
- Line 173: test `len(households) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 174: update `pi_se` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 175, `f_stat`: Set this name to the true-branch value if `pi_se > 0` holds, otherwise the else-branch value.
- Line 176, `restricted_ss`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 177, `partial_r2`: Set this name to the true-branch value if `restricted_ss > 0` holds, otherwise the else-branch value.
- Line 178: call `reports.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `absorb_effects`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.lstsq`: Solve least squares without explicitly inverting a cross-product matrix; [0] keeps its coefficient vector.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `len`: Count elements/rows rather than summing their numerical values.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.sf`: Evaluate survival probability above a threshold, useful for a two-sided t diagnostic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 184–184:

```
184 return result, pd.DataFrame(reports)
```

Read this block:

- Line 184: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

normalize upcs

Original function: `normalize_upcs`. Source begins at line 192.

This is the pandas normalization helper also used in estimation: strip whitespace, recognize numeric-looking identifiers, remove `.0` and leading zeros, and retain nonnumeric strings. Its argument is a Series and its return is a Series. The `where` condition retains original text on nonnumeric rows. The reason for string processing rather than floating-point conversion is identifier precision. See the estimation function's worked examples; identical naming here does not mean Python automatically shares their definitions across modules.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 192–193:

```
192 def normalize_upcs(values):
193     """Canonical numeric UPC keys without floating-point conversion."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 194–196:

```
194 text = values.astype("string").str.strip()
195 numeric = text.str.fullmatch(r"[0-9]+(?:\.0+)?", na=False)
196 canonical = text.str.replace(r"\.0+$", "", regex=True).str.lstrip("0")
```

Read this block:

- Line 194, `text`: Set this name to the result of calling `.strip(...)`; the arguments and preceding method chain are shown above.

- Line 195, `numeric`: Set this name to the result of calling `text.str.fullmatch(...)`; the arguments and preceding method chain are shown above.
- Line 196, `canonical`: Set this name to the result of calling `.lstrip(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip`: Remove surrounding string whitespace; it does not remove internal spaces.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.fullmatch`: Require the entire string to match a regex; contains would allow a partial match inside unrelated text.
- `.lstrip`: Remove any leading occurrences of the specified character set, such as zeros in numeric UPC text.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 197–197:

```
197 return text.where(~numeric, canonical.replace("", "0"))
```

Read this block:

- Line 197: return the result of calling `text.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

normalized upc expr

Original function: `normalized_upc_expr`. Source begins at line 199.

This constructs the Polars-expression version of UPC normalization for a lazy pipeline. It returns a recipe named `upc`, not a table. Nested conditions replace an all-zero numeric identifier with `"0"`, preserving nonnumeric text. The function avoids repeating a long expression at every join boundary. Using matching canonical keys on both sides prevents `"001"` and `"1"` from mistakenly appearing unmatched.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 199–199:

```
199 def normalized_upc_expr():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 200–202:

```
200 text = pl.col("upc").cast(pl.String).str.strip_chars()
201 numeric = text.str.contains(r"^[0-9]+(?:\.0+)?$")
202 canonical = text.str.replace(r"\.0+$", "").str.strip_chars_start("0")
```

Read this block:

- Line 200, `text`: Set this name to the result of calling `.strip_chars(...)`; the arguments and preceding method chain are shown above.
- Line 201, `numeric`: Set this name to the result of calling `text.str.contains(...)`; the arguments and preceding method chain are shown above.

- Line 202, `canonical`: Set this name to the result of calling `.strip_chars_start(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.strip_chars_start`: Trim characters from the start of each Polars string; it does not remove matching characters inside it.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 203–208:

```
203 return (
204     pl.when(numeric)
205     .then(pl.when(canonical == "").then(pl.lit("0")).otherwise(canonical))
206     .otherwise(text)
207     .alias("upc")
208 )
```

Read this block:

- Line 203: return the result of calling `.alias(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.

read flavors

Original function: `read_flavors`. Source begins at line 210.

Input `path` is a CSV path; optional `products_extra` is a Parquet path. Output is a two-column pandas table with one row per UPC. This function has an early return: if the CSV already contains resolved `upc,flavor`, validation succeeds and the rest of the recovery/classification code is never executed.

`{"upc", "flavor"}.issubset(frame.columns)` asks whether both required labels exist. `missing = required - set(frame.columns)` is set subtraction. Validation uses `.any()` for "at least one invalid row" and `.all()` for "every label is allowed." `if products_extra is not None` distinguishes an omitted optional argument from a supplied path that happens not to exist.

The metadata path scans only useful columns and anti-joins existing reference UPCs. It appends absent UPCs, rather than overwriting existing classifications. `[... for c in columns if c != "upc"]` builds one aggregation expression per non-key field. `np.select` picks the first matching condition, so plain wins when both plain and berry text occur. A generic substring loop could be easier to write at first, but would make precedence and vectorized missing-value handling less explicit. `drop_duplicates(..., keep="first")` is a reconciliation rule; it does not prove the retained label is correct.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 210–211:

```
210 def read_flavors(path, products_extra=None):
211     """Use the same plain-first classification as data_code/summ_stats.py."""
```

`def` creates this callable; the parentheses name its inputs (`path`, `products_extra`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 212–212:

```
212 frame = pd.read_csv(path, dtype={"upc": str})
```

Read this block:

- Line 212, `frame`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 213–227:

```
213 if {"upc", "flavor"}.issubset(frame.columns):
214     # Consume the exact mapping saved by summ_stats; no recovery or relabeling.
215     frame["upc"] = normalize_upcs(frame["upc"])
216     if (
217         frame["upc"].isna().any()
218         or frame["upc"].eq("").any()
219         or frame["upc"].duplicated().any()
220     ):
221         raise ValueError(f"{path}: saved mapping needs unique, nonmissing UPCs.")
222     if not frame["flavor"].isin(FLAVORS).all():
223         raise ValueError(f"{path}: saved mapping must use other, berry, plain.")
224     print(
225         f"Loaded {len(frame):,} resolved UPC flavor labels from {path}.", flush=True
226     )
227     return frame[["upc", "flavor"]]
```

Read this block:

- Line 213: test `{'upc', 'flavor'}.issubset(frame.columns)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 215, `frame['upc']`: Store the result of calling `normalize_upcs(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 216: test `frame['upc'].isna().any() or frame['upc'].eq('').any() or frame['upc'].duplic...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 221: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 222: test `not frame['flavor'].isin(FLAVORS).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 223: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 224: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 227: return the entry/slice selected by `frame[['upc', 'flavor']]`; brackets select from the object to their left. This ends the call.

Syntax and calls used here:

- `normalize_upcs`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 228–228:

```
228 missing = {"upc", "flavor_descr"} - set(frame.columns)
```

Read this block:

- Line 228, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 229–230:

```
229 if missing:
230     raise ValueError(f"{path}: missing columns {sorted(missing)}")
```

Read this block:

- Line 229: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 230: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.

Source lines 231–231:

```
231 frame["upc"] = normalize_upcs(frame["upc"])
```

Read this block:

- Line 231, `frame['upc']`: Store the result of calling `normalize_upcs(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 232–233:

```
232 if frame["upc"].isna().any() or frame["upc"].eq("").any():
233     raise ValueError(f"{path}: flavor reference contains missing UPCs.")
```

Read this block:

- Line 232: test `frame['upc'].isna().any() or frame['upc'].eq('').any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 233: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 234–275:

```
if products_extra is not None:
    products_extra = Path(products_extra)
    if products_extra.exists():
        products = pl.scan_parquet(products_extra)
        names = products.collect_schema().names()
        if not {"upc", "flavor_descr"}.issubset(names):
            raise ValueError(
                f"{products_extra}: needs upc and flavor_descr columns."
            )
        columns = [
            c
            for c in ["upc", "flavor_descr", "product_descr", "panel_year"]
            if c in names
        ]
        products = (
            products.select(columns)
            .with_columns(normalized_upc_expr())
            .join(
                pl.DataFrame(
                    {"upc": frame["upc"].dropna().unique().tolist()}
                ).lazy(),
                on="upc",
                how="anti",
            )
        )
        products = products.filter(
            pl.col("upc").is_not_null()
            & pl.col("flavor_descr").is_not_null()
            & ~pl.col("flavor_descr")
            .cast(pl.String)
            .str.strip_chars()
            .str.to_lowercase()
            .is_in(["", "na", "n/a", "nan", "null", "none"])
        )
        if "panel_year" in names:
            products = products.group_by("upc").agg(
                [
                    pl.col(c)
                    .sort_by("panel_year", nulls_last=True)
                    .first()
                    .alias(c)
                    for c in columns
```

Source lines 276–293:

```
                    if c != "upc"
                ]
            )
    else:
        products = products.unique(
            subset="upc", keep="first", maintain_order=True
        )
    recovered = products.collect(engine="streaming").to_pandas()
    frame = pd.concat([frame, recovered], ignore_index=True)
    print(
        f"Recovered flavor labels for {len(recovered):,} additional UPCs from {products_extra}.",
        flush=True,
    )
else:
    print(
        f"Flavor recovery source not found: {products_extra}; using CSV reference only.",
        flush=True,
```

```
293     )
```

Read this block:

- Line 234: test `products_extra is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 235, `products_extra`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 236: test `products_extra.exists()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 237, `products`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 238, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.
- Line 239: test `not {'upc', 'flavor_descr'}.issubset(names)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 240: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 243, `columns`: Set this name to a list computed by the displayed for-expression, retaining only entries passing its if filter.
- Line 248, `products`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 259, `products`: Set this name to the result of calling `products.filter(...)`; the arguments and preceding method chain are shown above.
- Line 268: test `'panel_year' in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 269, `products`: Set this name to the result of calling `.agg(...)`; the arguments and preceding method chain are shown above.
- Line 280, `products`: Set this name to the result of calling `products.unique(...)`; the arguments and preceding method chain are shown above.
- Line 283, `recovered`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 284, `frame`: Set this name to the result of calling `pd.concat(...)`; the arguments and preceding method chain are shown above.
- Line 285: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 290: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough

gives the input/output contract.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.to_lowercase`: Build a lowercase Polars string expression for reproducible label comparisons.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.sort_by`: Order values within an expression/group by another column before selecting a first value.
- `.strip_chars`: Trim surrounding characters/whitespace in a Polars string expression.
- `.dropna`: Remove pandas missing observations before the following calculation.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 294–294:

```
294 text = frame["flavor_descr"].fillna("")
```

Read this block:

- Line 294, `text`: Set this name to the result of calling `frame['flavor_descr'].fillna(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.

Source lines 295–296:

```
295 if "product_descr" in frame.columns:
296     text = text + " " + frame["product_descr"].fillna("")
```

Read this block:

- Line 295: test `'product_descr' in frame.columns`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 296, `text`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 297–307:

```
297 text = text.str.lower()
298 frame["flavor"] = np.select(
299     [
300         text.str.contains("plain|unflavored"),
301         text.str.contains("berry|straw|blue|rasp|black|cran|cherry|wildberry"),
302     ],
303     ["plain", "berry"],
304     default="other",
305 )
306 # Match summ_stats: retain missing labels and use the first CSV row per UPC.
307 conflicts = (frame.groupby("upc")["flavor"].nunique() > 1).sum()
```

Read this block:

- Line 297, `text`: Set this name to the result of calling `text.str.lower(...)`; the arguments and preceding method chain are shown above.
- Line 298, `frame['flavor']`: Store the result of calling `np.select(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 307, `conflicts`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.contains`: Test whether text contains the supplied regex pattern; anchors can make this an entire-string test.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.

Source lines 308–313:

```
308 if conflicts:
309     print(
310         f"{conflicts:,} UPCs have conflicting flavor classifications; "
311         "keeping the first flavor in CSV row order.",
312         flush=True,
313     )
```

Read this block:

- Line 308: test `conflicts`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 309: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 314–314:

```
314 return frame[["upc", "flavor"]].drop_duplicates(subset="upc", keep="first")
```

Read this block:

- Line 314: return the result of calling `.drop_duplicates(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.

week dates

Original function: `week_dates`. Source begins at line 316.

`pd.to_datetime` interprets strings with an explicit `%Y%m%d` format: four-digit year, two-digit month, two-digit day. The regex removes a trailing `.0` from integer dates that were stored as float-like text. `errors="raise"` rejects a malformed date instead of silently creating a missing date.

For example, `20100109` should mean January 9, 2010. Passing that integer directly to a general timestamp constructor can instead mean nanoseconds from an epoch. Explicit parsing prevents that mistake. This helper exists in the module, but the current main preparation path parses purchase-date strings with Polars directly; a function's presence does not mean the entry point uses it.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 316–317:

```
316 def week_dates(values):
317     """Parse YYYYMMDD safely (integer dates are NOT nanoseconds)."""
```

`def` creates this callable; the parentheses name its inputs (`values`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 318–322:

```
318 return pd.to_datetime(
319     values.astype(str).str.replace(r"\.0$", "", regex=True),
320     format="%Y%m%d",
321     errors="raise",
322 )
```

Read this block:

- Line 318: return the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

prepare data

Original function: `prepare_data`. Source begins at line 326.

This is the data assembly function. `args` is an argparse Namespace: a small object whose attributes hold command-line settings. It returns a pandas table with one row per usable household-week. Its work crosses a deliberate boundary: large input filtering and quote aggregation use lazy Polars, while the smaller household choice table becomes pandas for subsequent array calculations.

Read it in phases. First it scans and filters years, resolves labels, chooses the price field, and projects columns. `getattr(args, name, default)` provides compatibility when a manually constructed Namespace lacks an optional attribute. `next((c for c in candidates if c in names), None)` chooses the first supported price field. It is a small loop written as a generator, not a special price API.

Next it collects selected households, checks quantities, sums counts, and chooses an exposure store. Sorting by descending quantity followed by ascending store ID then keeping the first household-week row implements the tie-break exactly. `groupby(...).sum().unstack(...)` turns long flavor counts into three columns; `reindex` fixes column order even if one flavor is absent from an intermediate table. Merge `validate` arguments catch unintended duplicate keys.

The quote section continues to use full `base`, not the household-limited `selected`. It deduplicates quotes and checks conflicts before constructing menus. The DMA section computes sums and counts and subtracts own-DMA totals. This algebra avoids looping over every possible outside DMA for every household. The same pooled IV is copied into all three flavor-named IV columns.

Finally it constructs histories **before** menu/IV likelihood exclusions, builds Boolean availability masks, filters unusable rows, reapplies the two-week rule, and adds outside counts. `complete &= iv_complete` combines masks in place; it is not a comparison. `transform("nunique")` produces a row-aligned count so the household restriction can

be applied without a second merge. Moving `add_stocks` below these exclusions is a plausible-looking alternative that changes the information used to build history.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 326–331:

```
326 def prepare_data(settings):
327     """Use only the merged purchase table, including its existing prices.
328 
329     The IV pools all flavors: the same-week average observed price outside the
330     consumer's DMA. Repeated purchases do not duplicate store/UPC price quotes.
331     """
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 332–333:

```
332 base = pl.scan_parquet(settings.sample)
333 names = base.collect_schema().names()
```

Read this block:

- Line 332, `base`: Set this name to the result of calling `pl.scan_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 333, `names`: Set this name to the result of calling `.names(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.scan_parquet`: Build a lazy Parquet read plan; rows are not yet materialized in a Python table.
- `.names`: Retrieve schema column names as a sequence, useful for checking optional fields.
- `.collect_schema`: Inspect column names/types of a lazy plan without collecting its entire result table.

Source lines 334–335:

```
334 if "household_size" not in names:
335     raise ValueError("Estimation requires household_size to restrict purchases to single-person households.")
```

Read this block:

- Line 334: test `'household_size' not in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 335: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 336–336:

```
336 base = base.filter(pl.col("panel_year").is_between(settings.first_year, settings.last_year))
```

Read this block:

- Line 336, `base`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.is_between`: Build an inclusive interval check by default, such as selecting the requested panel-year range.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.

Source lines 337–342:

```
337 if "flavor" not in names:
338     products_extra = getattr(settings, "products_extra", None) or DATA_DIR / "RMS/combined/products_extra.parquet"
339     flavor = read_flavors(settings.flavors, products_extra)
340     base = base.with_columns(normalized_upc_expr()).join(pl.from_pandas(flavor).lazy(), on="upc", how="left")
341 else:
342     base = base.with_columns(normalized_upc_expr())
```

Read this block:

- Line 337: test `'flavor' not in names`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 338, `products_extra`: Set this name to the Boolean condition `getattr(settings, 'products_extra', None) or DATA_DIR / 'RMS/c...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 339, `flavor`: Set this name to the result of calling `read_flavors(...)`; the arguments and preceding method chain are shown above.
- Line 340, `base`: Set this name to the result of calling `.join(...)`; the arguments and preceding method chain are shown above.
- Line 342, `base`: Set this name to the result of calling `base.with_columns(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `read_flavors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `getattr`: Retrieve an attribute by its string name, optionally supplying a default if it is absent.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `normalized_upc_expr`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.from_pandas`: Convert a pandas object into a Polars table; calling lazy afterward makes a query plan from it.

Source lines 343–343:

```
343 price_column = next((c for c in ["price", "price_rms"] if c in names), None)
```

Read this block:

- Line 343, `price_column`: Set this name to the result of calling `next(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `next`: Take the first item from an iterator, or the supplied default when no item matches.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 344–345:

```
344 if price_column is None:
345     raise ValueError("Merged purchase table needs a price or price_rms column.")
```

Read this block:

- Line 344: test `price_column is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 345: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 346–348:

```
346 consumer_dma = getattr(settings, "household_dma_column", "dma_cd")
347 columns = ["household_code", "household_size", "panel_year", "purchase_date", "trip_code_uc", "upc",
348            "store_code_uc", "quantity", "flavor", pl.col(price_column).cast(pl.Float64).alias("price")]
```

Read this block:

- Line 346, `consumer_dma`: Set this name to the result of calling `getattr(...)`; the arguments and preceding method chain are shown above.
- Line 347, `columns`: Set this name to a list containing 10 displayed entries.

Syntax and calls used here:

- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.

Source lines 349–350:

```
349 if not settings.no_price_control:
350     columns.append(pl.col(consumer_dma).alias("dma_cd"))
```

Read this block:

- Line 349: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 350: call `columns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 351–358:

```
351 base = base.select(columns).with_columns(
352     pl.col("purchase_date").cast(pl.String).str.slice(0, 10).str.to_date())
353 base = base.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
354 base = base.with_columns((pl.col("purchase_date") + pl.duration(
355     days=(6 - pl.col("purchase_date").dt.weekday()) % 7)).alias("week"))
356 # Keep the full merged table for prices/IVs, even in a household-limited run.
357 # Restrict demand histories before weekly aggregation; retain all quotes for menus/IVs.
358 selected = base.filter(pl.col("household_size").cast(pl.Float64, strict=False) == 1)
```

Read this block:

- Line 351, `base`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 353, `base`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.
- Line 354, `base`: Set this name to the result of calling `base.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 358, `selected`: Set this name to the result of calling `base.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- `.slice`: Take a substring starting at a position for a given length, here often the first ten date characters.
- `.is_finite`: Build the Polars finite-number test used to reject missing/infinite quotes.

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- `.duration`: Build a date/time duration from the named unit, allowing a different day offset per row.
- `.weekday`: Extract a Polars weekday number, Monday=1 through Sunday=7, for the Saturday offset calculation.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 359–362:

```
359 if settings.max_households:
360     ids = (selected.select("household_code").unique().sort("household_code")
361            .limit(settings.max_households).collect(engine="streaming")["household_code"])
362     selected = selected.filter(pl.col("household_code").is_in(ids.implode()))
```

Read this block:

- Line 359: test `settings.max_households`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 360, `ids`: Set this name to the entry/slice selected by `selected.select('household_code').unique().sort('househol...`; brackets select from the object to their left.
- Line 362, `selected`: Set this name to the result of calling `selected.filter(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.is_in`: Build a Polars membership expression, often used for eligible household IDs or allowed labels.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

Source lines 363–364:

```
363 print(f"Loading merged purchase table: {settings.sample}", flush=True)
364 lines = selected.collect(engine="streaming").to_pandas()
```

Read this block:

- Line 363: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 364, `lines`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.

Source lines 365–366:

```
365 if lines.empty:
366     raise ValueError("No purchase lines in selected sample.")
```

Read this block:

- Line 365: test `lines.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 366: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 367–368:

```
367 if lines["flavor"].isna().any() or not lines["flavor"].isin(FLAVORS).all():
368     raise ValueError("Merged purchase table has missing or invalid flavor labels.")
```

Read this block:

- Line 367: test `lines['flavor'].isna().any() or not lines['flavor'].isin(FLAVORS).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 368: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.

Source lines 369–369:

```
369 q = lines["quantity"].to_numpy(float)
```

Read this block:

- Line 369, `q`: Set this name to the result of calling `lines['quantity'].to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.

Source lines 370–371:

```
370 if not np.isfinite(q).all() or (q <= 0).any() or not np.allclose(q, np.round(q)):
371     raise ValueError("Quantity must be positive integer package counts.")
```

Read this block:

- Line 370: test `not np.isfinite(q).all() or (q <= 0).any() or (not np.allclose(q, np.round(q)))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 371: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.
- `.round`: Round numeric values for a check/display rule; rounding quantities is compared to the original before acceptance.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 372–378:

```
372 lines["week"] = pd.to_datetime(lines["week"])
373 keys = ["household_code", "week"]
374 weekly = (lines.groupby(keys + ["flavor"])["quantity"].sum().unstack("flavor", fill_value=0)
375           .reindex(columns=FLAVORS, fill_value=0).reset_index())
376 stores = lines.groupby(keys + ["store_code_uc"], as_index=False)["quantity"].sum()
377 stores = stores.sort_values(keys + ["quantity", "store_code_uc"], ascending=[True, True, False, True])
378 stores = stores.drop_duplicates(keys)[keys + ["store_code_uc"]]
```

Read this block:

- Line 372, `lines['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

- Line 373, `keys`: Set this name to the list `['household_code', 'week']`.
- Line 374, `weekly`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 376, `stores`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 377, `stores`: Set this name to the result of calling `stores.sort_values(...)`; the arguments and preceding method chain are shown above.
- Line 378, `stores`: Set this name to the entry/slice selected by `stores.drop_duplicates(keys)[keys + ['store_code_uc']]`; brackets select from the object to their left.

Syntax and calls used here:

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.
- `.unstack`: Move an index level into columns, such as turning long flavor counts into wide weekly outcomes.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.drop_duplicates`: Retain one row per selected key; keep='first' makes earlier sorting/order economically consequential.

Source lines 379–379:

```
379 weekly = weekly.merge(stores, on=keys, validate="one_to_one")
```

Read this block:

- Line 379, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 380–384:

```
380 if not settings.no_price_control:
381     lines["dma_cd"] = check_dma(lines["dma_cd"], "consumer")
382     if (lines.groupby(keys)["dma_cd"].nunique() > 1).any():
383         raise ValueError("Multiple consumer DMAs within a household-week.")
384     weekly = weekly.merge(lines[keys + ["dma_cd"]].drop_duplicates(), on=keys, validate="one_to_one")
```

Read this block:

- Line 380: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 381, `lines['dma_cd']`: Store the result of calling `check_dma(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 382: test `(lines.groupby(keys)['dma_cd'].nunique() > 1).any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 383: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 384, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `check_dma`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.

Source lines 386–387:

```
386 quote_keys = ["store_code_uc", "week", "upc"]
387 price_columns = quote_keys + ["flavor", "price"]
```

Read this block:

- Line 386, `quote_keys`: Set this name to the list `['store_code_uc', 'week', 'upc']`.
- Line 387, `price_columns`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 388–389:

```
388 if not settings.no_price_control:
389     price_columns.append("dma_cd")
```

Read this block:

- Line 388: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 389: call `price_columns.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 390–396:

```
390 quotes = (base.filter(pl.col("price").is_finite() & (pl.col("price") > 0))
391           .select(price_columns).unique())
392 # A store/week/UPC should have one price and one flavor. Consumer DMA may
393 # differ when consumers from different markets visit the same store.
394 local = quotes.select(quote_keys + ["flavor", "price"]).unique()
395 conflicting = (local.group_by(quote_keys).len().filter(pl.col("len") > 1)
396                 .limit(1).collect(engine="streaming"))
```

Read this block:

- Line 390, `quotes`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 394, `local`: Set this name to the result of calling `.unique(...)`; the arguments and preceding method chain are shown above.
- Line 395, `conflicting`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.

Source lines 397–398:

```
397 if len(conflicting):
398     raise ValueError("Conflicting merged prices or flavors for the same store/week/UPC.")
```

Read this block:

- Line 397: test `len(conflicting)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 398: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 399–405:

```
menus = (local.group_by(["store_code_uc", "week", "flavor"])
         .agg(pl.col("price").mean()).collect(engine="streaming").to_pandas())
menus["week"] = pd.to_datetime(menus["week"])
menus = (menus.pivot(index=["store_code_uc", "week"], columns="flavor", values="price")
         .reindex(columns=FLAVORS).rename(columns={f: "price_" + f for f in FLAVORS}).reset_index())
weekly = weekly.merge(menus, on=["store_code_uc", "week"], how="left", validate="many_to_one")
print("Using merged prices directly; no separate RMS data are loaded.", flush=True)
```

Read this block:

- Line 399, `menus`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 401, `menus['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 402, `menus`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 404, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 405: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.

Source lines 406–430:

```
if not settings.no_price_control:
    invalid_dma = quotes.select(
        ((pl.col("dma_cd").cast(pl.Float64, strict=False).is_null())
         | (pl.col("dma_cd").cast(pl.Float64, strict=False) <= 0)
         | (pl.col("dma_cd").cast(pl.Float64, strict=False) != pl.col("dma_cd").cast(pl.Float64,
          ↪  strict=False).floor()))
        .fill_null(True).any()).collect(engine="streaming").item()
    if invalid_dma:
        raise ValueError("Merged price records contain invalid dma_cd values for the other-DMA IV.")
    # Pool across ALL flavors. Deduplicate quote repeats within each DMA.
    by_dma = (quotes.with_columns(pl.col("dma_cd").cast(pl.Float64).cast(pl.Int64))
              .group_by(["week", "dma_cd"]).agg(
                  pl.col("price").sum().alias("dma_sum"), pl.len().alias("dma_count"))
              .collect(engine="streaming").to_pandas())
    by_dma["week"] = pd.to_datetime(by_dma["week"])
    totals = (by_dma.groupby("week", as_index=False)[["dma_sum", "dma_count"]].sum()
              .rename(columns={"dma_sum": "total_sum", "dma_count": "total_count"}))
    weekly = weekly.merge(totals, on="week", how="left", validate="many_to_one")
    weekly = weekly.merge(by_dma, on=["week", "dma_cd"], how="left", validate="many_to_one")
    denominator = weekly["total_count"] - weekly["dma_count"].fillna(0)
    weekly["iv_overall"] = ((weekly["total_sum"] - weekly["dma_sum"].fillna(0))
                            / denominator.where(denominator > 0))
    for f in FLAVORS:
        weekly["iv_" + f] = weekly["iv_overall"]
    weekly = weekly.drop(columns=["total_sum", "total_count", "dma_sum", "dma_count"])
    print("IV: overall same-week mean price outside consumer dma_cd, pooled across flavors.", flush=True)
```

Read this block:

- Line 406: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 407, `invalid_dma`: Set this name to the result of calling `.item(...)`; the arguments and preceding method chain are shown above.
- Line 412: test `invalid_dma`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 413: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 415, `by_dma`: Set this name to the result of calling `.to_pandas(...)`; the arguments and preceding method chain are shown above.
- Line 419, `by_dma['week']`: Store the result of calling `pd.to_datetime(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 420, `totals`: Set this name to the result of calling `.rename(...)`; the arguments and preceding method chain are shown above.
- Line 422, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 423, `weekly`: Set this name to the result of calling `weekly.merge(...)`; the arguments and preceding method chain are shown above.
- Line 424, `denominator`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 425, `weekly['iv_overall']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 427: for each entry yielded by `FLAVORS`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 428, `weekly['iv_' + f]`: Store the entry/slice selected by `weekly['iv_overall']`; brackets select from the object to their left in the selected entry/column.
- Line 429, `weekly`: Set this name to the result of calling `weekly.drop(...)`; the arguments and preceding method chain are shown above.
- Line 430: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.
- `.fillna`: Replace pandas missing entries with the provided fallback; it can change downstream denominators or validity checks.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.fill_null`: Replace Polars nulls with a chosen value/expression; null and floating-point NaN are distinct representations.
- `.is_null`: Create a Polars null-value mask, useful when validating joins and conversions.
- `.floor`: Take the greatest integer no larger than each value; equality with it tests whether a positive code is integral.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.

Source lines 431–435:

```
431 weekly = add_stocks(weekly.sort_values(keys).reset_index(drop=True), settings.habit_weight)
432 price_cols = ["price_" + f for f in FLAVORS]
433 available = np.isfinite(weekly[price_cols].to_numpy(float)) & (weekly[price_cols].to_numpy(float) > 0)
434 counts = weekly[FLAVORS].to_numpy(float)
435 complete = available.any(axis=1) & ((counts == 0) | available).all(axis=1)
```

Read this block:

- Line 431, `weekly`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding

method chain are shown above.
- Line 432, `price_cols`: Set this name to a list computed by the displayed for-expression.
- Line 433, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 434, `counts`: Set this name to the result of calling `weekly[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 435, `complete`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 436–437:

```
436 for j, f in enumerate(FLAVORS):
437     weekly["available_" + f] = available[:, j]
```

Read this block:

- Line 436: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, f)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 437, `weekly['available_' + f]`: Store the entry/slice selected by `available[:, j]`; brackets select from the object to their left in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 438–438:

```
438 print(f"Weeks excluded for purchased flavors without a covered price: {(~complete).sum():,}", flush=True)
```

Read this block:

- Line 438: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 439–443:

```
439 if not settings.no_price_control:
440     iv_cols = ["iv_" + f for f in FLAVORS]
441     iv_complete = np.isfinite(weekly[iv_cols]).all(axis=1)
442     print(f"Additional weeks excluded for missing other-DMA IVs: {(complete & ~iv_complete).sum():,}", flush=True)
443     complete &= iv_complete
```

Read this block:

- Line 439: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 440, `iv_cols`: Set this name to a list computed by the displayed for-expression.
- Line 441, `iv_complete`: Set this name to the result of calling `.all(...)`; the arguments and preceding method chain are shown above.
- Line 442: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 443: update `complete` using `&=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 444–448:

```
444 weekly = weekly.loc[complete].copy()
445 repeated = weekly.groupby("household_code")["week"].transform("nunique") >= 2
446 print(f"Households excluded for fewer than two usable weeks: {weekly.loc[~repeated,
    ↪  'household_code'].nunique():,}", flush=True)
447 weekly = weekly.loc[repeated].reset_index(drop=True)
448 weekly["outside"] = np.maximum(settings.capacity - weekly[FLAVORS].sum(axis=1), 0)
```

Read this block:

- Line 444, `weekly`: Set this name to the result of calling `weekly.loc[complete].copy(...)`; the arguments and preceding method chain are shown above.
- Line 445, `repeated`: Set this name to the Boolean condition `weekly.groupby('household_code')['week'].transform('nunique') ...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 446: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 447, `weekly`: Set this name to the result of calling `weekly.loc[repeated].reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 448, `weekly['outside']`: Store the result of calling `np.maximum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.transform`: Compute a pandas group result broadcast back to original rows, unlike a reduced one-row-per-group aggregation.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.

Source lines 449–450:

```
449 if weekly.empty:
450     raise ValueError("No households with at least two usable purchase weeks.")
```

Read this block:

- Line 449: test `weekly.empty`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 450: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 451–451:

```
451 return weekly
```

Read this block:

- Line 451: return the object currently named `weekly`; plain assignment does not copy it. This ends the call.

add stocks

Original function: `add_stocks`. Source begins at line 455.

Input is a weekly pandas table; output is a sorted copy with stock, novelty, and Xi columns. The outer loop resets history for each household. `for _, group` ignores the household label but keeps its subtable. The inner `iterrows()` loop supplies a row index and a Series containing that row's values.

`stocks[index] = stock` stores pre-choice history into one output row. The seven-day condition tests genuine adjacent calendar weeks. Then counts are normalized into shares, stock is updated, and the current date becomes the previous date. A gap resets recent novelty by leaving its preallocated zeros, but does not reset cumulative stock.

A loop is reasonable here because state at the next observation depends on the preceding update; vectorization requires careful shifted cumulative sums and calendar-gap checks.

The final `enumerate` loop writes one named column per flavor. `stocks[:, j]` extracts all rows of the jth flavor. With `habit_weight=.7`, current Xi uses .3 times novelty minus .7 times own stock. The separately preallocated `stocks` matrix means historical output rows do not change when `stock` is reassigned. Try a one-household, two-week example before reading the likelihood.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 455–466:

```
455 def add_stocks(weekly, habit_weight=0.7):
456     """All past observed occasions, delta=1; no rolling window or decay.
457 
458     S_jt = sum_{s<t} share_js
459     N_jt = sum_{k != j} share_k,t-1  (immediately preceding calendar week)
460     Xi_jt = (1-habit_weight)*N_jt - habit_weight*S_jt
461 
462     Positive gamma rewards recent other-flavor consumption and penalizes own
463     fatigue. Negative gamma represents habit. A purchase occasion is an observed
464     household purchase week. Current purchases enter history only AFTER Xi is
465     recorded. The first observed occasion starts with zero history.
466     """
```

`def` creates this callable; the parentheses name its inputs (`weekly`, `habit_weight`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 467–471:

```
467 result = (
468     weekly.sort_values(["household_code", "week"]).reset_index(drop=True).copy()
469 )
470 stocks = np.zeros((len(result), 3))
471 novelty = np.zeros_like(stocks)
```

Read this block:

- Line 467, `result`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.
- Line 470, `stocks`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 471, `novelty`: Set this name to the result of calling `np.zeros_like(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 472–483:

```
472 for _, group in result.groupby("household_code", sort=False):
473     stock = np.zeros(3)
474     previous_share = np.zeros(3)
```

```
475     previous_week = None
476     for index, row in group.iterrows():
477         stocks[index] = stock
478         if previous_week is not None and (row["week"] - previous_week).days == 7:
479             novelty[index] = previous_share.sum() - previous_share
480         counts = row[FLAVORS].to_numpy(float)
481         previous_share = counts / counts.sum()
482         stock = stock + previous_share
483         previous_week = row["week"]
```

Read this block:

- Line 472: for each entry yielded by `result.groupby('household_code', sort=False)`, bind/unpack it into `(_, group)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 473, `stock`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 474, `previous_share`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 475, `previous_week`: Set this name to the literal value `None`.
- Line 476: for each entry yielded by `group.iterrows()`, bind/unpack it into `(index, row)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 477, `stocks[index]`: Store the object currently named `stock`; plain assignment does not copy it in the selected entry/column.
- Line 478: test `previous_week is not None and (row['week'] - previous_week).days == 7`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 479, `novelty[index]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 480, `counts`: Set this name to the result of calling `row[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 481, `previous_share`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 482, `stock`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 483, `previous_week`: Set this name to the entry/slice selected by `row['week']`; brackets select from the object to their left.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 484–489:

```
484 for j, name in enumerate(FLAVORS):
485     result["stock_" + name] = stocks[:, j]
486     result["novelty_" + name] = novelty[:, j]
487     result["xi_" + name] = (1 - habit_weight) * novelty[
488         :, j
489     ] - habit_weight * stocks[:, j]
```

Read this block:

- Line 484: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 485, `result['stock_' + name]`: Store the entry/slice selected by `stocks[:, j]`; brackets select from the object to their left in the selected entry/column.

- Line 486, `result['novelty_' + name]`: Store the entry/slice selected by `novelty[:, j]`; brackets select from the object to their left in the selected entry/column.
- Line 487, `result['xi_' + name]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 490–490:

```
490 return result
```

Read this block:

- Line 490: return the object currently named `result`; plain assignment does not copy it. This ends the call.

make arrays

Original function: `make_arrays`. Source begins at line 494.

This converts a labeled table into a dictionary of NumPy arrays. `pd.factorize` returns both dense integer household codes and their original labels: household IDs `[10,10,30]` become codes `[0,0,1]`. Dense codes let `bincount` sum efficiently. The expression `FLAVORS + (["outside"] if capacity else [])` builds either three or four outcome columns without changing the inside-price array's three columns.

The function validates counts/Xi, checks menu consistency, and inserts zero placeholders for missing prices. The separate availability mask ensures those zeros are not interpreted as offered free goods. `all(c in frame for c in cf_cols)` asks whether every residual column exists. If controls are absent, `zeros_like` supplies the right-shaped residual array. The returned dictionary deliberately keeps flags such as `outside` and `control` beside arrays so the likelihood can use one interface for several models. Passing raw tables directly into every optimization step would repeatedly perform label lookup and conversion.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 494–495:

```
494 def make_arrays(frame, capacity):
495     """Use household codes to sum log likelihoods before integrating types."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `capacity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 496–502:

```
496 hh, labels = pd.factorize(frame["household_code"], sort=True)
497 counts = frame[FLAVORS + (["outside"] if capacity else [])].to_numpy(float)
498 prices = frame[["price_" + f for f in FLAVORS]].to_numpy(float)
499 xi = frame[["xi_" + f for f in FLAVORS]].to_numpy(float)
500 history = {key: frame[[prefix + f for f in FLAVORS]].to_numpy(float)
501            for key, prefix in [("novelty", "novelty_"), ("stock", "stock_")]
502            if all(prefix + f in frame for f in FLAVORS)}
```

Read this block:

- Line 496, `(hh, labels)`: Unpack the result of calling `pd.factorize(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 497, `counts`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 498, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

- Line 499, `xi`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 500, `history`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.

Syntax and calls used here:

- `.factorize`: Encode labels as dense integer positions and return the original-label lookup alongside those positions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 503–504:

```
503 if any(not np.isfinite(value).all() for value in history.values()):
504     raise ValueError("Nonfinite separate history inputs.")
```

Read this block:

- Line 503: test `any((not np.isfinite(value).all() for value in history.values()))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 504: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 505–505:

```
505 available = np.isfinite(prices) & (prices > 0)
```

Read this block:

- Line 505, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 506–507:

```
506 if not all(np.isfinite(x).all() for x in [counts, xi]):
507     raise ValueError("Nonfinite estimation inputs.")
```

Read this block:

- Line 506: test `not all((np.isfinite(x).all() for x in [counts, xi]))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 507: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 508–509:

```
if ((counts[:, :3] > 0) & ~available).any() or not available.any(axis=1).all():
    raise ValueError("Purchased flavors must have covered prices and a nonempty choice set.")
```

Read this block:

- Line 508: test `((counts[:, :3] > 0) & ~available).any() or not available.any(axis=1).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 509: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 511–514:

```
prices = np.where(available, prices, 0.)
cf_cols = ["cf_" + f for f in FLAVORS]
control = all(c in frame for c in cf_cols)
cf = frame[cf_cols].to_numpy(float) if control else np.zeros_like(prices)
```

Read this block:

- Line 511, `prices`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 512, `cf_cols`: Set this name to a list computed by the displayed for-expression.
- Line 513, `control`: Set this name to the result of calling `all(...)`; the arguments and preceding method chain are shown above.
- Line 514, `cf`: Set this name to the true-branch value if `control` holds, otherwise the else-branch value.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.

Source lines 515–516:

```
if not np.isfinite(cf).all():
    raise ValueError("Nonfinite price-control residuals.")
```

Read this block:

- Line 515: test `not np.isfinite(cf).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 516: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 517–529:

```
return dict(
    counts=counts,
    prices=prices,
    available=available,
    xi=xi,
    **history,
    cf=cf,
    control=control,
    hh=hh,
    labels=labels,
    n_households=len(labels),
    outside=bool(capacity),
```

```
)
```

Read this block:

- Line 517: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `len`: Count elements/rows rather than summing their numerical values.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.

make draws

Original function: `make_draws`. Source begins at line 539.

Inputs are an even draw count, an integer seed, and an optional model name. Output has shape `(number,3)` for heterogeneous_history and `(number,2)` otherwise. `number % 2` is the remainder after division by two: zero for even numbers, one for odd. `number // 2` is integer division. The generator draws half the rows, then `vstack([z, -z])` appends their negatives. With two draws, an initial row `[.4,-1.2]` gives a second row `[-.4,1.2]`.

The seed makes repeated calls reproducible, and a local Generator avoids changing NumPy's global random state. Reusing these fixed antithetic nodes reduces noise when comparing candidate parameter vectors. Drawing afresh inside each objective call would make the optimizer chase random changes as well as parameter effects. More draws still cost more computation.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Split-model update. heterogeneous_history uses D-by-3 draws with alpha, novelty and own-stock coordinates; other models keep D-by-2 draws. The draw dimension and seed must match the saved fit when reconstructing likelihoods.

Definition and stated intent:

Source lines 539–540:

```
def make_draws(number, seed, model=None):
    """Fixed antithetic normal draws, reused throughout optimization."""
```

`def` creates this callable; the parentheses name its inputs (`number`, `seed`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 541–542:

```
if number < 2 or number % 2:
    raise ValueError("Use an even number of draws, at least two.")
```

Read this block:

- Line 541: test `number < 2 or number % 2`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 542: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 543–543:

```
543 z = np.random.default_rng(seed).standard_normal((number // 2, 3 if model == "heterogeneous_history" else 2))
```

Read this block:

- Line 543, `z`: Set this name to the result of calling `.standard_normal(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.standard_normal`: Draw independent standard-normal values in the requested shape, before coefficient transformations.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- The // operator is floor division, used here for integer counts or calendar-year extraction.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 544–544:

```
544 return np.vstack([z, -z])
```

Read this block:

- Line 544: return the result of calling `np.vstack(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.vstack`: Stack input arrays vertically, adding rows while preserving their common column layout.

parameter names

Original function: `parameter_names`. Source begins at line 548.

This returns the ordered labels defining the parameter vector. `names += [...]` extends a list; it does not numerically add its string entries. Membership tests such as `model in ("mixed_naive", "heterogeneous")` let related specifications share branches. The default `control=False` omits a residual coefficient unless the caller explicitly enables it.

For naive conditional demand without controls, the returned labels are two betas and mean log-minus-alpha. Heterogeneous demand adds mean gamma and both SDs; the optional outside constant is prepended and price control appended. Keeping this logic in one helper is safer than independently remembering vector positions in optimization, utility, and output. The order is part of the API: zipping a reordered list with old coefficients silently changes their meaning.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Split layouts. separate_history adds mean_novelty and mean_own_stock plus random price sensitivity; heterogeneous_history also adds sd_novelty and sd_own_stock. This helper is also used when checking the saved likelihood during weekly-export regeneration.

Definition and stated intent:

Source lines 548–548:

```
548 def parameter_names(model, outside, control=False):
```

`def` creates this callable; the parentheses name its inputs (`model`, `outside`, `control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 549–550:

```
549 names = (["inside_constant"] if outside else []) + ["beta_berry", "beta_plain"]
550 names += ["mu_log_minus_alpha"]
```

Read this block:

- Line 549, `names`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 550: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 551–552:

```
551 if model in ("homogeneous", "heterogeneous"):
552     names += ["mean_gamma"]
```

Read this block:

- Line 551: test `model in ('homogeneous', 'heterogeneous')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 552: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 553–554:

```
553 if model in ("mixed_naive", "heterogeneous"):
554     names += ["sd_log_minus_alpha"]
```

Read this block:

- Line 553: test `model in ('mixed_naive', 'heterogeneous')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 554: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 555–556:

```
555 if model == "heterogeneous":
556     names += ["sd_gamma"]
```

Read this block:

- Line 555: test `model == 'heterogeneous'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 556: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 557–559:

```
557 if model == "mixture":
558     names += ["mean_gamma", "sd_log_minus_alpha", "sd_gamma",
559               "gamma_mean_gap", "sd_gamma_2", "mixture_weight"]
```

Read this block:

- Line 557: test `model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 558: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 560–561:

```
560 if model in ("separate_history", "heterogeneous_history"):
561     names += ["mean_novelty", "mean_own_stock", "sd_log_minus_alpha"]
```

Read this block:

- Line 560: test `model in ('separate_history', 'heterogeneous_history')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 561: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 562–563:

```
562 if model == "heterogeneous_history":
563     names += ["sd_novelty", "sd_own_stock"]
```

Read this block:

- Line 562: test `model == 'heterogeneous_history'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 563: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 564–565:

```
564 if control:
565     names += ["price_control"]
```

Read this block:

- Line 564: test `control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 565: update `names` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 566–566:

```
566 return names
```

Read this block:

- Line 566: return the object currently named `names`; plain assignment does not copy it. This ends the call.

mixture components

Original function: `mixture_components`. Source begins at line 570.

Unpacks a parameter vector into two ordered normal-gamma components and a common price distribution. Adding a nonnegative gap orders component means and prevents label switching; it does not force either component to have positive gamma.

Definition and stated intent:

Source lines 570–571:

```
570 def mixture_components(theta, data):
571     """Ordered component means prevent label switching; alpha is shared."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 572–577:

```
572 names = parameter_names("mixture", data["outside"], data["control"])
573 p = dict(zip(names, theta))
574 normal_names = parameter_names("heterogeneous", data["outside"], data["control"])
575 first = np.array([p[n] for n in normal_names])
576 second = first.copy()
577 second[normal_names.index("mean_gamma")] += p["gamma_mean_gap"]
```

Read this block:

- Line 572, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 573, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 574, `normal_names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 575, `first`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 576, `second`: Set this name to the result of calling `first.copy(...)`; the arguments and preceding method chain are shown above.
- Line 577: update `second[normal_names.index('mean_gamma')]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.

Source lines 578–578:

```
578 second[normal_names.index("sd_gamma")] = p["sd_gamma_2"]
```

Read this block:

- Line 578, `second[normal_names.index('sd_gamma')]`: Store the entry/slice selected by `p['sd_gamma_2']`; brackets select from the object to their left in the selected entry/column.

Source lines 579–579:

```
579 return names, normal_names, first, second, p["mixture_weight"]
```

Read this block:

- Line 579: return a tuple of 5 results. The caller can unpack them; return ends the call, unlike print.

mixture loglike score

Original function: `mixture_loglike_score`. Source begins at line 582.

Combines complete household likelihoods across persistent mixture components, using stable log-sum-exp. Mixing each week separately would change the model by allowing household membership to change over time. The score combines component derivatives with posterior membership weights.

Definition and stated intent:

Source lines 582–583:

```
582 def mixture_loglike_score(theta, data, draws, score=False):
583     """Mix whole household histories, never individual weeks."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `score`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 584–584:

```
584 names, normal_names, first, second, weight = mixture_components(theta, data)
```

Read this block:

- Line 584, `(names, normal_names, first, second, weight)`: Unpack the result of calling `mixture_components(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `mixture_components`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 585–590:

```
585 if score:
586     ll1, g1 = demand_objective_gradient(first, data, draws, "heterogeneous", household=True)
587     ll2, g2 = demand_objective_gradient(second, data, draws, "heterogeneous", household=True)
588 else:
589     ll1 = household_loglike(first, data, draws, "heterogeneous")
590     ll2 = household_loglike(second, data, draws, "heterogeneous")
```

Read this block:

- Line 585: test `score`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 586, `(ll1, g1)`: Unpack the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 587, `(ll2, g2)`: Unpack the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 589, `ll1`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 590, `ll2`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 591–592:

```
591 terms = np.array([np.log(weight) + ll1, np.log1p(-weight) + ll2])
592 ll = logsumexp(terms, axis=0)
```

Read this block:

- Line 591, `terms`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 592, `ll`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.

Source lines 593–594:

```
593 if not score:
594     return ll
```

Read this block:

- Line 593: test `not score`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 594: return the object currently named `ll`; plain assignment does not copy it. This ends the call.

Source lines 595–596:

```
595 responsibility = np.exp(terms - ll)
596 gradient = np.zeros((data["n_households"], len(names)))
```

Read this block:

- Line 595, `responsibility`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 596, `gradient`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 597–602:

```
597 for k, name in enumerate(normal_names):
598     if name == "sd_gamma":
599         gradient[:, names.index(name)] = responsibility[0] * g1[:, k]
600         gradient[:, names.index("sd_gamma_2")] = responsibility[1] * g2[:, k]
601     else:
602         gradient[:, names.index(name)] = responsibility[0] * g1[:, k] + responsibility[1] * g2[:, k]
```

Read this block:

- Line 597: for each entry yielded by `enumerate(normal_names)`, bind/unpack it into `(k, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 598: test `name == 'sd_gamma'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 599, `gradient[:, names.index(name)]`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 600, `gradient[:, names.index('sd_gamma_2')]`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 602, `gradient[:, names.index(name)]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 603–604:

```
603 gradient[:, names.index("gamma_mean_gap")] = responsibility[1] * g2[:, normal_names.index("mean_gamma")]
604 gradient[:, names.index("mixture_weight")] = responsibility[0] / weight - responsibility[1] / (1 - weight)
```

Read this block:

- Line 603, `gradient[:, names.index('gamma_mean_gap')]`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 604, `gradient[:, names.index('mixture_weight')]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Source lines 605–605:

```
605 return -ll.mean(), -gradient.mean(axis=0)
```

Read this block:

- Line 605: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

history utility

Original function: `history_utility`. Source begins at line 609.

Chooses the history formula from the model name. Split models use b_NN+b_SS directly, with distinct draw coordinates for the two random coefficients. Combined-gamma models use gamma*Xi. The habit-weight parameter belongs to Xi and does not multiply the independently estimated split coefficients.

Definition and stated intent:

Source lines 609–609:

```
609 def history_utility(p, data, z, model):
```

`def` creates this callable; the parentheses name its inputs (`p`, `data`, `z`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 610–617:

```
610 if model in ("separate_history", "heterogeneous_history"):
611     novelty = p["mean_novelty"] + p.get("sd_novelty", 0.) * z[1]
612     own_stock = p["mean_own_stock"]
613     if model == "heterogeneous_history":
614         if len(z) < 3:
615             raise ValueError("heterogeneous_history requires three-dimensional draws.")
616         own_stock += p["sd_own_stock"] * z[2]
617     return novelty * data["novelty"] + own_stock * data["stock"]
```

Read this block:

- Line 610: test `model in ('separate_history', 'heterogeneous_history')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 611, `novelty`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 612, `own_stock`: Set this name to the entry/slice selected by `p['mean_own_stock']`; brackets select from the object to their left.
- Line 613: test `model == 'heterogeneous_history'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 614: test `len(z) < 3`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 615: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 616: update `own_stock` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 617: return addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `len`: Count elements/rows rather than summing their numerical values.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 618–618:

```
618 gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
```

Read this block:

- Line 618, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 619–619:

```
619 return gamma * data["xi"]
```

Read this block:

- Line 619: return multiplication of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

household loglike

Original function: `household_loglike`. Source begins at line 623.

Input `theta` is a one-dimensional parameter array, `data` is the dictionary from `make_arrays`, and `draws` is a draw-by-two matrix for combined models or draw-by-three for split heterogeneity. The return is one log likelihood per household, not a scalar. `dict(zip(...))` attaches coefficient names so the rest of the function need not remember their numeric positions.

For each latent-type draw, negative exponential constructs alpha and an affine normal transformation constructs gamma. Broadcasting makes beta and these scalar coefficients operate across every row/flavor. Unavailable utility is set to minus infinity; an optional zero outside column is appended. `logprob = utility - logsumexp(..., axis=1, keepdims=True)` normalizes each row. The retained column dimension makes subtraction broadcast across that row.

The count-weighted row sum uses `where(counts > 0, logprob, 0)` to avoid zero times minus infinity. `bincount` adds all weeks of the same household. Only after this aggregation does the final `logsumexp` integrate draws. This order implements persistent preferences. Averaging weekly probabilities first would be simpler computationally but would estimate a different preference process.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Current split utility. history_utility supplies b_NN+b_S$S, with separate means and normal draw coordinates. Gamma*Xi is retained only for combined models. In a mixture model, the separate component helper mixes complete household likelihoods. Neither split coefficient receives the old fixed lambda weight.

Definition and stated intent:

Source lines 623–629:

```
def household_loglike(theta, data, draws, model):
    """L_i = mean_r exp(sum_t sum_j count_itj * log P_itj,r).

    Gamma is normal and common across flavors; alpha = -exp(mu + sd*z). Beta never varies by draw.
    Process one draw at a time to avoid a weeks x draws x products array.
    The multinomial combinatorial constant is omitted (independent of theta).
    """
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `model`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 630–631:

```
if model == "mixture":
    return mixture_loglike_score(theta, data, draws)
```

Read this block:

- Line 630: test `model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 631: return the result of calling `mixture_loglike_score(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `mixture_loglike_score`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 632–638:

```
p = dict(zip(parameter_names(model, data["outside"], data["control"]), theta))
beta = np.array([0.0, p["beta_berry"], p["beta_plain"]])
beta += p.get("inside_constant", 0.0)
loglikes = []
active_draws = (
    draws if model in ("mixed_naive", "heterogeneous", "separate_history", "heterogeneous_history") else
    np.zeros((1, 2))
)
```

Read this block:

- Line 632, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 633, `beta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 634: update `beta` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 635, `loglikes`: Set this name to the list `[]`.
- Line 636, `active_draws`: Set this name to the true-branch value if `model in ('mixed_naive', 'heterogeneous', 'sepa...` holds, otherwise the else-branch value.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 639–657:

```
639 for z in active_draws:
640     alpha = -np.exp(
641         p["mu_log_minus_alpha"] + p.get("sd_log_minus_alpha", 0.0) * z[0]
642     )
643     gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
644     utility = (
645         beta
646         + alpha * data["prices"]
647         + history_utility(p, data, z, model)
648         + p.get("price_control", 0.0) * data["cf"]
649     )
650     utility = np.where(data["available"], utility, -np.inf)
651     if data["outside"]:
652         utility = np.column_stack([utility, np.zeros(len(utility))])
653     logprob = utility - logsumexp(utility, axis=1, keepdims=True)
654     row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
655     loglikes.append(
656         np.bincount(data["hh"], weights=row_ll, minlength=data["n_households"])
657     )
```

Read this block:

- Line 639: for each entry yielded by `active_draws`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 640, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(p['mu_log_minus_alpha'] + p.get('sd_log_minus_alp...`.
- Line 643, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 644, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 650, `utility`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 651: test `data['outside']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 652, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 653, `logprob`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 654, `row_ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 655: call `loglikes.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.

- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `history_utility`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 658–658:

```
658 return logsumexp(loglikes, axis=0) - np.log(len(active_draws))
```

Read this block:

- Line 658: return subtraction of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

demand objective gradient

Original function: `demand_objective_gradient`. Source begins at line 662.

This returns a tuple `(mean_negative_log_likelihood, gradient)`, which SciPy requests when `jac=True`. The utility and likelihood steps mirror `household_loglike`; the additional work computes derivatives in the same draw loop. `total[:, None]` expands a weekly count vector to a column so it can multiply the matrix of choice probabilities. `residual` is observed count minus predicted count for the three inside alternatives.

The `columns` dictionary holds one weekly score vector per coefficient name. For example, the berry intercept takes column 1 of residual, while the alpha location score uses the price-weighted residual multiplied by alpha itself because differentiating `-exp(mu + sd*z)` contributes alpha. Each score is aggregated to households, and `column_stack` places parameters on a final axis.

`np.einsum("rh,rhp->p", weights, scores)` means multiply posterior weight at draw r/household h by its parameter-p score, then sum r and h, retaining p. A nested loop over r, h, p would express the same sum more verbosely and slowly. The minus sign and household-count division match minimization of average NLL. This is an analytic derivative of the finite-draw objective, not automatic differentiation or a finite-difference estimate.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 662–663:

```
662 def demand_objective_gradient(theta, data, draws, model, household=False):
663     """Exact simulated likelihood score, weighted by household type posteriors."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `data`, `draws`, `model`, `household`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 664–665:

```
664 if model == "mixture":
665     return mixture_loglike_score(theta, data, draws, score=True)
```

Read this block:

- Line 664: test `model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 665: return the result of calling `mixture_loglike_score(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `mixture_loglike_score`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 666–670:

```
666 names = parameter_names(model, data["outside"], data["control"])
667 p = dict(zip(names, theta))
668 active = draws if model in ("mixed_naive", "heterogeneous", "separate_history", "heterogeneous_history") else
    ↪  np.zeros((1, 2))
669 likelihoods, scores = [], []
670 total = data["counts"].sum(axis=1)
```

Read this block:

- Line 666, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 667, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 668, `active`: Set this name to the true-branch value if `model in ('mixed_naive', 'heterogeneous', 'sepa...` holds, otherwise the else-branch value.
- Line 669, `(likelihoods, scores)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 670, `total`: Set this name to the result of calling `data['counts'].sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 671–697:

```
671 for z in active:
672     alpha = -np.exp(p["mu_log_minus_alpha"] + p.get("sd_log_minus_alpha", 0.) * z[0])
673     gamma = p.get("mean_gamma", 0.) + p.get("sd_gamma", 0.) * z[1]
674     utility = (np.array([0., p["beta_berry"], p["beta_plain"]]) + p.get("inside_constant", 0.)
675                + alpha * data["prices"] + history_utility(p, data, z, model) + p.get("price_control", 0.) *
                    ↪  data["cf"])
676     utility = np.where(data["available"], utility, -np.inf)
677     if data["outside"]:
678         utility = np.column_stack([utility, np.zeros(len(utility))])
679     logprob = utility - logsumexp(utility, axis=1, keepdims=True)
680     row_ll = (data["counts"] * np.where(data["counts"] > 0, logprob, 0.)).sum(axis=1)
681     likelihoods.append(np.bincount(data["hh"], weights=row_ll, minlength=data["n_households"]))
682     residual = data["counts"][:, :3] - total[:, None] * np.exp(logprob[:, :3])
683     price_score = (residual * data["prices"]).sum(axis=1) * alpha
684     columns = {"inside_constant": residual.sum(axis=1), "beta_berry": residual[:, 1],
685                "beta_plain": residual[:, 2], "mu_log_minus_alpha": price_score,
686                "sd_log_minus_alpha": price_score * z[0],
687                "price_control": (residual * data["cf"]).sum(axis=1)}
688     columns["mean_gamma"] = (residual * data["xi"]).sum(axis=1)
689     columns["sd_gamma"] = columns["mean_gamma"] * z[1]
690     if model in ("separate_history", "heterogeneous_history"):
691         columns["mean_novelty"] = (residual * data["novelty"]).sum(axis=1)
692         columns["mean_own_stock"] = (residual * data["stock"]).sum(axis=1)
693         columns["sd_novelty"] = columns["mean_novelty"] * z[1]
```

```
694         if model == "heterogeneous_history":
695             columns["sd_own_stock"] = columns["mean_own_stock"] * z[2]
696     scores.append(np.column_stack([np.bincount(data["hh"], weights=columns[n],
697                                                minlength=data["n_households"]) for n in names]))
```

Read this block:

- Line 671: for each entry yielded by `active`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 672, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(p['mu_log_minus_alpha'] + p.get('sd_log_minus_alp...`.
- Line 673, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 674, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 676, `utility`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 677: test `data['outside']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 678, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 679, `logprob`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 680, `row_ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 681: call `likelihoods.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 682, `residual`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 683, `price_score`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 684, `columns`: Set this name to a dictionary with named entries `inside_constant`, `beta_berry`, `beta_plain`, `mu_log_minus_alpha`, `sd_log_minus_alpha`, `price_control`.
- Line 688, `columns['mean_gamma']`: Store the result of calling `(residual * data['xi']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 689, `columns['sd_gamma']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 690: test `model in ('separate_history', 'heterogeneous_history')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 691, `columns['mean_novelty']`: Store the result of calling `(residual * data['novelty']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 692, `columns['mean_own_stock']`: Store the result of calling `(residual * data['stock']).sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 693, `columns['sd_novelty']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 694: test `model == 'heterogeneous_history'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 695, `columns['sd_own_stock']`: Store multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 696: call `scores.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `history_utility`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `len`: Count elements/rows rather than summing their numerical values.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 698–704:

```
698 likelihoods = np.asarray(likelihoods)
699 normalizer = logsumexp(likelihoods, axis=0)
700 weights = np.exp(likelihoods - normalizer)
701 # weights is draws-by-households; scores is draws-by-households-by-parameters.
702 # Weight each draw's score, then sum over the draw axis.
703 weighted_scores = weights[:, :, None] * np.asarray(scores)
704 household_scores = weighted_scores.sum(axis=0)
```

Read this block:

- Line 698, `likelihoods`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 699, `normalizer`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 700, `weights`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 703, `weighted_scores`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 704, `household_scores`: Set this name to the result of calling `weighted_scores.sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 705–706:

```
705 if household:
706     return normalizer - np.log(len(active)), household_scores
```

Read this block:

- Line 705: test `household`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 706: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

Source lines 707–707:

```
707 gradient = -household_scores.mean(axis=0)
```

Read this block:

- Line 707, `gradient`: Set this name to the negated or sign-adjusted expression `-household_scores.mean(axis=0)`.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 708–708:

```
708 return -(normalizer - np.log(len(active))).mean(), gradient
```

Read this block:

- Line 708: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

optimize demand

Original function: `optimize_demand`. Source begins at line 718.

This sets up and runs multiple bounded local searches and returns the selected fit, every starting solution, and the bounds. Initial SDs are .3 because zero can be stationary under symmetric integration. Bounds are pairs `(lower, upper)` in the same order as parameter names. The optional initial vector is copied before perturbation, preventing caller-owned coefficients from being changed.

Each start uses `minimize(..., method="L-BFGS-B", jac=True)`. The callback reports progress every ten iterations, while completed starts are saved to JSON. `np.clip` keeps perturbed starts feasible. `min(results, key=lambda r: r.fun)` uses each result's loss as its comparison key; it selects the smallest loss, not necessarily a converged fit. An alternative is to select only converged results, but that is not the rule this code implements. Multistart search probes local solutions and does not establish a global optimum.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 718–718:

```
718 def optimize_demand(data, settings, initial=None, number_starts=None, quiet=False):
```

`def` creates this callable; the parentheses name its inputs (`data`, `settings`, `initial`, `number_starts`, `quiet`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 719–722:

```
719 draws = make_draws(settings.draws, settings.seed, settings.model)
720 names = parameter_names(settings.model, data["outside"], data["control"])
721 start = np.zeros(len(names))
722 bounds = []
```

Read this block:

- Line 719, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 720, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 721, `start`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 722, `bounds`: Set this name to the list `[]`.

Syntax and calls used here:

- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 723–736:

```
723 for k, name in enumerate(names):
724     if name.startswith("sd_"):
725         start[k] = 0.3  # Zero SD is a stationary point: start away from zero.
726         bounds.append((0.0, 3.0))
727     elif name == "gamma_mean_gap":
728         start[k] = 0.4
729         bounds.append((0.0, 40.0))
730     elif name == "mixture_weight":
731         start[k] = 0.5
732         bounds.append((0.001, 0.999))
733     elif name == "mu_log_minus_alpha":
734         bounds.append((-6.0, 3.0))
735     else:
736         bounds.append((-20.0, 20.0))
```

Read this block:

- Line 723: for each entry yielded by `enumerate(names)`, bind/unpack it into `(k, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 724: test `name.startswith('sd_')`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 725, `start[k]`: Store the literal value `0.3` in the selected entry/column.
- Line 726: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 727: test `name == 'gamma_mean_gap'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 728, `start[k]`: Store the literal value `0.4` in the selected entry/column.
- Line 729: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 730: test `name == 'mixture_weight'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 731, `start[k]`: Store the literal value `0.5` in the selected entry/column.
- Line 732: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 733: test `name == 'mu_log_minus_alpha'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 734: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 736: call `bounds.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 737–738:

```
737 if settings.model == "mixture":
738     start[names.index("mean_gamma")] = -0.2
```

Read this block:

- Line 737: test `settings.model == 'mixture'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.

- Line 738, `start[names.index('mean_gamma')]`: Store the negated or sign-adjusted expression `-0.2` in the selected entry/column.

Source lines 739–740:

```
if initial is not None:
    start = np.asarray(initial).copy()
```

Read this block:

- Line 739: test `initial is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 740, `start`: Set this name to the result of calling `.copy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 741–742:

```
rng = np.random.default_rng(settings.seed + 1)
results = []
```

Read this block:

- Line 741, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 742, `results`: Set this name to the list `[]`.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.

Source lines 744–745:

```
def objective(theta):
    return demand_objective_gradient(theta, data, draws, settings.model)
```

Read this block:

- Line 744: define nested callable `objective`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 747–781:

```
for s in range(settings.starts if number_starts is None else number_starts):
    initial = start.copy()
    if s:
        initial += rng.normal(0, 0.5, len(start))
        initial = np.clip(initial, np.array(bounds)[:, 0], np.array(bounds)[:, 1])
    began = time.monotonic()
    iterations = [0]

    def progress(theta):
        iterations[0] += 1
        if not quiet and iterations[0] % 10 == 0:
            print(f"Start {s + 1}: iteration {iterations[0]}, elapsed {time.monotonic() - began:.0f}s", flush=True)

    fit = minimize(
        objective,
        initial,
```

```
763         method="L-BFGS-B",
764         jac=True,
765         callback=progress,
766         bounds=bounds,
767         options={"maxiter": settings.maxiter, "ftol": 1e-10, "gtol": 1e-6},
768     )
769     results.append(fit)
770     if not quiet:
771         settings.output.mkdir(parents=True, exist_ok=True)
772         (settings.output / "starts_checkpoint.json").write_text(json.dumps([
773             dict(theta=r.x.tolist(), nll=float(r.fun * data["n_households"]),
774                  converged=bool(r.success), iterations=int(r.nit), evaluations=int(r.nfev))
775             for r in results], indent=2))
776     if not quiet:
777         print(
778             f"Start {s + 1}: NLL={fit.fun * data['n_households']:.4f}; "
779             f"converged={fit.success}",
780             flush=True,
781         )
```

Read this block:

- Line 747: for each entry yielded by `range(settings.starts if number_starts is None else number_sta...`, bind/unpack it into `s` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 748, `initial`: Set this name to the result of calling `start.copy(...)`; the arguments and preceding method chain are shown above.
- Line 749: test `s`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 750: update `initial` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 751, `initial`: Set this name to the result of calling `np.clip(...)`; the arguments and preceding method chain are shown above.
- Line 752, `began`: Set this name to the result of calling `time.monotonic(...)`; the arguments and preceding method chain are shown above.
- Line 753, `iterations`: Set this name to the list `[0]`.
- Line 755: define nested callable `progress`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 760, `fit`: Set this name to the result of calling `minimize(...)`; the arguments and preceding method chain are shown above.
- Line 769: call `results.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 770: test `not quiet`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 771: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 772: call `(settings.output / 'starts_checkpoint.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 776: test `not quiet`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 777: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.
- `minimize`: Run iterative numerical minimization; jac=True expects the objective to return value plus gradient.
- `.normal`: Draw normal random values with the supplied location/scale, used for starts or synthetic data.
- `.clip`: Cap values at lower/upper limits; here it keeps starts feasible or exponentials within floating-point

range.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 783–783:

```
783 best = min(results, key=lambda r: r.fun)
```

Read this block:

- Line 783, `best`: Set this name to the result of calling `min(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 784–784:

```
784 return best, results, bounds
```

Read this block:

- Line 784: return a tuple of 3 results. The caller can unpack them; return ends the call, unlike print.

optimize demand.objective

Original function: `optimize_demand.objective`. Source begins at line 744.

Defines the scalar objective for the surrounding search using captured data and parameters. Inspect the displayed expression to identify whether it returns a profit or its negative; minimizers and maximizers require opposite conventions.

Definition and stated intent:

Source lines 744–744:

```
744 def objective(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 745–745:

```
745 return demand_objective_gradient(theta, data, draws, settings.model)
```

Read this block:

- Line 745: return the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

optimize demand.progress

Original function: `optimize_demand.progress`. Source begins at line 755.

SciPy calls this nested callback after iterations, passing the current vector. The body increments `iterations[0]`, checks a remainder modulo ten, and prints elapsed time unless quiet mode is active. The callback does not return an objective value and does not choose the next parameter vector.

The enclosing variable is a one-element list so the nested function can mutate its element without rebinding the outer name. An ordinary integer counter with assignment would require `nonlocal iterations`. That alternative is often clearer; the existing list technique works because objects can be mutated through a captured reference. `time.monotonic` measures elapsed duration without wall-clock adjustments changing the result.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 755–755:

```
755 def progress(theta):
```

`def` creates this callable; the parentheses name its inputs (`theta`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 756–756:

```
756 iterations[0] += 1
```

Read this block:

- Line 756: update `iterations[0]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 757–758:

```
757 if not quiet and iterations[0] % 10 == 0:
758     print(f"Start {s + 1}: iteration {iterations[0]}, elapsed {time.monotonic() - began:.0f}s", flush=True)
```

Read this block:

- Line 757: test `not quiet and iterations[0] % 10 == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 758: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.monotonic`: Read a monotonic clock for elapsed durations rather than a wall-clock date/time.

- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

resample households

Original function: `resample_households`. Source begins at line 794.

Input is the weekly frame and a NumPy random Generator. Output is a concatenated bootstrap sample with the same number of household draws as the original. The comprehension stores whole household subtables. `rng.integers(0, H, size=H)` draws indices with replacement; its upper endpoint H is excluded.

`enumerate(selected)` supplies a fresh ID and selected old-group position. `.copy()` is essential before changing that ID so the original history is not modified. If selection is `[2,2,0]`, two copies of old household 2 get new IDs 0 and 1, keeping their likelihoods separate. Concatenating with `ignore_index=True` gives clean row labels. Resampling individual weeks would break persistent household preferences and within-household dependence.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 794–799:

```
794 def resample_households(frame, rng):
795     """Keep whole histories; repeated households get distinct bootstrap IDs.
796 
797     Stocks/Xi stay attached to the original histories. Rebuilding stocks after
798     dropping incomplete price weeks would incorrectly erase observed purchases.
799     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `rng`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 800–802:

```
800 groups = [group for _, group in frame.groupby("household_code", sort=True)]
801 selected = rng.integers(0, len(groups), size=len(groups))
802 samples = []
```

Read this block:

- Line 800, `groups`: Set this name to a list computed by the displayed for-expression.
- Line 801, `selected`: Set this name to the result of calling `rng.integers(...)`; the arguments and preceding method chain are shown above.
- Line 802, `samples`: Set this name to the list `[]`.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.integers`: Draw integer positions with replacement by default; the high endpoint is excluded.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 803–806:

```
803 for new_id, old_id in enumerate(selected):
804     group = groups[old_id].copy()
```

```
805     group["household_code"] = new_id
806     samples.append(group)
```

Read this block:

- Line 803: for each entry yielded by `enumerate(selected)`, bind/unpack it into `(new_id, old_id)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 804, `group`: Set this name to the result of calling `groups[old_id].copy(...)`; the arguments and preceding method chain are shown above.
- Line 805, `group['household_code']`: Store the object currently named `new_id`; plain assignment does not copy it in the selected entry/column.
- Line 806: call `samples.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

Source lines 807–807:

```
807 return pd.concat(samples, ignore_index=True)
```

Read this block:

- Line 807: return the result of calling `pd.concat(...)`; the arguments and preceding method chain are shown above. This ends the call.

bootstrap standard errors

Original function: `bootstrap_standard_errors`. Source begins at line 812.

The `estimate` argument is a callable, not an estimate array. Each repetition resamples households, refits enabled price controls, calls `estimate(sample, initial)`, and records its result. `record.update` adds named fields to an existing dictionary. Accepted vectors go to `values`; all attempts go to `records`, so failure does not disappear from the audit trail.

The `try/except` catches expected data/linear-algebra problems. Every attempt checkpoints the records as CSV. After the loop, the threshold requires at least 20 usable fits and at least 80% success. Returning `(None, report)` explicitly signals inference was withheld. Otherwise, `std(ddof=1)` uses sample rather than population SD, `quantile` forms empirical percentile intervals, and `np.cov(..., rowvar=False)` treats columns as parameters. Analytic Hessian SEs would be cheaper but would require handling the estimated first-stage residuals and household dependence; this implementation uses a two-stage cluster bootstrap.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 812–818:

```
812 def bootstrap_standard_errors(frame, settings, estimate, initial, names):
813     """Refit price controls AND demand in every household-cluster replication.
814 
815     estimate(frame, initial) returns a scipy optimizer result. All replications
816     use the same integration draws as the main fit. Retail instruments stay
817     fixed: this is inference conditional on the observed retail price universe.
818     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `settings`, `estimate`, `initial`, `names`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 819–821:

```
819 rng = np.random.default_rng(settings.seed + 10000)
820 records = []
821 values = []
```

Read this block:

- Line 819, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 820, `records`: Set this name to the list `[]`.
- Line 821, `values`: Set this name to the list `[]`.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.

Source lines 822–841:

```
822 for replication in range(settings.bootstrap_reps):
823     record = {"replication": replication + 1, "converged": False}
824     try:
825         sample = resample_households(frame, rng)
826         if not settings.no_price_control:
827             sample, _ = add_price_controls(sample)
828         fit = estimate(sample, initial)
829         record.update(converged=bool(fit.success), objective=float(fit.fun),
830                       message=str(fit.message))
831         record.update(dict(zip(names, fit.x)))
832         if fit.success and np.isfinite(fit.x).all() and np.isfinite(fit.fun):
833             values.append(fit.x)
834     except (ValueError, np.linalg.LinAlgError) as error:
835         # An unidentifiable resample is recorded, never silently accepted.
836         record["message"] = str(error)
837     records.append(record)
838     # Checkpoint after each replication in case a long cluster job ends.
839     pd.DataFrame(records).to_csv(settings.output / "bootstrap_draws.csv", index=False)
840     print(f"Bootstrap {replication+1}/{settings.bootstrap_reps}: "
841           f"{len(values)} usable fits", flush=True)
```

Read this block:

- Line 822: for each entry yielded by `range(settings.bootstrap_reps)`, bind/unpack it into `replication` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 823, `record`: Set this name to a dictionary with named entries `replication`, `converged`.
- Line 824: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 825, `sample`: Set this name to the result of calling `resample_households(...)`; the arguments and preceding method chain are shown above.
- Line 826: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 827, `(sample, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 828, `fit`: Set this name to the result of calling `estimate(...)`; the arguments and preceding method chain are shown above.
- Line 829: call `record.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 831: call `record.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 832: test `fit.success and np.isfinite(fit.x).all() and np.isfinite(fit.fun)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 833: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 834: catch `(ValueError, np.linalg.LinAlgError)` from the preceding try; the optional as-name holds the error object for logging.
- Line 836, `record['message']`: Store the result of calling `str(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 837: call `records.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 839: call `pd.DataFrame(records).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 840: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `resample_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `estimate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `str`: Convert to text for paths, messages, or serialization.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `len`: Count elements/rows rather than summing their numerical values.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 842–846:

```
842 valid = len(values)
843 report = dict(method=("household bootstrap" if settings.no_price_control
844                       else "household bootstrap, refitting both stages"),
845               requested=settings.bootstrap_reps, usable=valid,
846               conditional_on="retail instruments and fixed integration draws")
```

Read this block:

- Line 842, `valid`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 843, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another

expression.

Source lines 848–850:

```
if valid < max(20, int(np.ceil(.8 * settings.bootstrap_reps))):
    report["status"] = "insufficient converged replications; no SEs reported"
    return None, report
```

Read this block:

- Line 848: test `valid < max(20, int(np.ceil(0.8 * settings.bootstrap_reps)))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 849, `report['status']`: Store the literal text `'insufficient converged replications; no SEs reported'` in the selected entry/column.
- Line 850: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.ceil`: Round upward, so the minimum usable-bootstrap threshold meets the requested success fraction.

Source lines 851–858:

```
estimates = np.asarray(values)
table = pd.DataFrame({"parameter": names,
                      "std_error": estimates.std(axis=0, ddof=1),
                      "ci_lower": np.quantile(estimates, .025, axis=0),
                      "ci_upper": np.quantile(estimates, .975, axis=0)})
covariance = pd.DataFrame(np.cov(estimates, rowvar=False), index=names, columns=names)
covariance.to_csv(settings.output / "covariance.csv")
report["status"] = "computed; percentile 95% intervals"
```

Read this block:

- Line 851, `estimates`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 852, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 856, `covariance`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 857: call `covariance.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 858, `report['status']`: Store the literal text `'computed; percentile 95% intervals'` in the selected entry/column.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `.quantile`: Find empirical quantiles, supplying percentile bootstrap endpoints rather than normal-Wald formulas.
- `.cov`: Compute covariance; rowvar=False means columns represent parameters rather than treating rows as variables.

Source lines 859–859:

```
return table, report
```

Read this block:

- Line 859: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

observed covariance

Original function: `observed_covariance`. Source begins at line 863.

Central-differences the analytic mean-household NLL gradient to form observed information, checks bounds and eigenvalues, and divides its inverse by the household count. Missing household scaling would give incorrect standard errors. Nonpositive or ill-conditioned information is rejected instead of inverted blindly.

Definition and stated intent:

Source lines 863–864:

```
863 def observed_covariance(theta, gradient, households, bounds):
864     """Differentiate the mean negative-log-likelihood gradient, then scale by I."""
```

`def` creates this callable; the parentheses name its inputs (`theta`, `gradient`, `households`, `bounds`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 865–867:

```
865 theta = np.asarray(theta, dtype=float)
866 steps = np.cbrt(np.finfo(float).eps) * np.maximum(1., np.abs(theta))
867 lower, upper = np.asarray(bounds, dtype=float).T
```

Read this block:

- Line 865, `theta`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 866, `steps`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 867, `(lower, upper)`: Unpack the attribute `np.asarray(bounds, dtype=float).T` of the object before the dot into these targets in order.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 868–869:

```
868 if np.any(np.minimum(theta - lower, upper - theta) <= np.maximum(steps, 1e-4)):
869     raise ValueError("parameter at or near bounds; normal inference withheld")
```

Read this block:

- Line 868: test `np.any(np.minimum(theta - lower, upper - theta) <= np.maximum(steps, 0.0001))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 869: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 870–870:

```
870 hessian = np.empty((len(theta), len(theta)))
```

Read this block:

- Line 870, `hessian`: Set this name to the result of calling `np.empty(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 871–874:

```
871 for j, step in enumerate(steps):
872     delta = np.zeros(len(theta))
873     delta[j] = step
874     hessian[:, j] = (gradient(theta + delta) - gradient(theta - delta)) / (2 * step)
```

Read this block:

- Line 871: for each entry yielded by `enumerate(steps)`, bind/unpack it into `(j, step)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 872, `delta`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 873, `delta[j]`: Store the object currently named `step`; plain assignment does not copy it in the selected entry/column.
- Line 874, `hessian[:, j]`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 875–875:

```
875 hessian = (hessian + hessian.T) / 2
```

Read this block:

- Line 875, `hessian`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 876–877:

```
876 if not np.isfinite(hessian).all():
877     raise ValueError("nonfinite observed information; SEs withheld")
```

Read this block:

- Line 876: test `not np.isfinite(hessian).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 877: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 878–878:

```
878 eigenvalues = np.linalg.eigvalsh(hessian)
```

Read this block:

- Line 878, `eigenvalues`: Set this name to the result of calling `np.linalg.eigvalsh(...)`; the arguments and preceding method chain are shown above.

Source lines 879–880:

```
if eigenvalues[0] <= max(0., eigenvalues[-1] * 1e-10):
    raise ValueError("observed information not positive definite or ill-conditioned; SEs withheld")
```

Read this block:

- Line 879: test `eigenvalues[0] <= max(0.0, eigenvalues[-1] * 1e-10)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 880: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.

Source lines 881–881:

```
return np.linalg.solve(hessian, np.eye(len(theta))) / households
```

Read this block:

- Line 881: return division of the displayed operands; parentheses control which subexpressions are evaluated together. This ends the call.

hessian standard errors

Original function: `hessian_standard_errors`. Source begins at line 886.

Converts an accepted observed-information covariance into parameter SEs and normal intervals, writes the covariance, and returns inference metadata. These are conditional model-based SEs; they do not include price-first-stage uncertainty or provide a robust sandwich covariance.

Definition and stated intent:

Source lines 886–886:

```
def hessian_standard_errors(theta, gradient, households, bounds, names, output):
```

`def` creates this callable; the parentheses name its inputs (`theta`, `gradient`, `households`, `bounds`, `names`, `output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 887–889:

```
report = dict(method="observed-information Hessian",
              conditional_on="estimated price controls, retail instruments, and fixed integration draws",
              first_stage_uncertainty="not included", robust=False)
```

Read this block:

- Line 887, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 890–894:

```
try:
    covariance = observed_covariance(theta, gradient, households, bounds)
except ValueError as error:
    report["status"] = str(error)
    return None, report
```

Read this block:

- Line 890: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 891, `covariance`: Set this name to the result of calling `observed_covariance(...)`; the arguments and preceding method chain are shown above.
- Line 892: catch `ValueError` from the preceding try; the optional as-name holds the error object for logging.
- Line 893, `report['status']`: Store the result of calling `str(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 894: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- `observed_covariance`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 895–900:

```
895 pd.DataFrame(covariance, index=names, columns=names).to_csv(output / "covariance.csv")
896 se = np.sqrt(np.diag(covariance))
897 critical = norm.ppf(.975)
898 table = pd.DataFrame(dict(parameter=names, std_error=se,
899                           ci_lower=theta - critical * se, ci_upper=theta + critical * se))
900 report["status"] = "computed; normal 95% intervals; first-stage uncertainty omitted"
```

Read this block:

- Line 895: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 896, `se`: Set this name to the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above.
- Line 897, `critical`: Set this name to the result of calling `norm.ppf(...)`; the arguments and preceding method chain are shown above.
- Line 898, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 900, `report['status']`: Store the literal text `'computed; normal 95% intervals; first-stage uncertainty omitted'` in the selected entry/column.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

Source lines 901–901:

```
901 return table, report
```

Read this block:

- Line 901: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

implied wtp

Original function: `implied_wtp`. Source begins at line 911.

Computes population moments of coefficient/positive price sensitivity. Independence permits multiplying the coefficient moments by inverse-lognormal moments. Averaging these household ratios differs from dividing a mean coefficient by mean price sensitivity. Split history rows use their own coefficients without lambda weights.

Definition and stated intent:

Source lines 911–912:

```
911 def implied_wtp(estimates, habit_weight=.7):
912     """Return population moments, not sampling standard errors, in price units."""
```

`def` creates this callable; the parentheses name its inputs (`estimates`, `habit_weight`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 913–917:

```
913 p = dict(zip(estimates.parameter, estimates.estimate))
914 mu, sigma = p['mu_log_minus_alpha'], p.get('sd_log_minus_alpha', 0.)
915 inverse_mean = np.exp(-mu + sigma**2 / 2)
916 inverse_second = np.exp(-2 * mu + 2 * sigma**2)
917 rows = []
```

Read this block:

- Line 913, `p`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 914, `(mu, sigma)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 915, `inverse_mean`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 916, `inverse_second`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 917, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 919–923:

```
919 def add(term, mean, second, interpretation):
920     wtp_mean = mean * inverse_mean
921     variance = max(0., second * inverse_second - wtp_mean**2)
922     rows.append(dict(term=term, mean_wtp=wtp_mean, population_sd=np.sqrt(variance),
923                      interpretation=interpretation))
```

Read this block:

- Line 919: define nested callable `add`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 925–929:

```
925 for name in ['inside_constant', 'beta_berry', 'beta_plain']:
926     if name in p:
927         interpretation = ('Other flavor vs outside, at zero history and control residual'
928                           if name == 'inside_constant' else 'Flavor premium relative to other')
929         add(name, p[name], p[name]**2, interpretation)
```

Read this block:

- Line 925: for each entry yielded by `['inside_constant', 'beta_berry', 'beta_plain']`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 926: test `name in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 927, `interpretation`: Set this name to the true-branch value if `name == 'inside_constant'` holds, otherwise the else-branch value.
- Line 929: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `add`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 930–944:

```
930 if 'mean_gamma' in p:
931     mean, sd = p['mean_gamma'], p.get('sd_gamma', 0.)
932     second = mean**2 + sd**2
933     if 'mixture_weight' in p:
934         weight = p['mixture_weight']
935         mean2 = mean + p['gamma_mean_gap']
936         second2 = mean2**2 + p['sd_gamma_2']**2
937         add('gamma_component_1', mean, second, 'Per unit Xi, conditional on component 1')
938         add('gamma_component_2', mean2, second2, 'Per unit Xi, conditional on component 2')
939         mean, second = weight * mean + (1 - weight) * mean2, weight * second + (1 - weight) * second2
940     add('gamma', mean, second, 'Per unit increase in history index Xi')
941     add('gamma_novelty', (1 - habit_weight) * mean, (1 - habit_weight)**2 * second,
942         'Per unit novelty N, holding own-flavor stock fixed')
943     add('gamma_own_stock', -habit_weight * mean, habit_weight**2 * second,
944         'Per unit own-flavor stock S, holding novelty fixed')
```

Read this block:

- Line 930: test `'mean_gamma' in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 931, `(mean, sd)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 932, `second`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 933: test `'mixture_weight' in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 934, `weight`: Set this name to the entry/slice selected by `p['mixture_weight']`; brackets select from the object to their left.
- Line 935, `mean2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 936, `second2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 937: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 938: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 939, `(mean, second)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 940: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 941: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 943: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 945–949:

```
945 for term in ["novelty", "own_stock"]:
946     if "mean_" + term in p:
947         mean, sd = p["mean_" + term], p.get("sd_" + term, 0.)
948         add(term, mean, mean**2 + sd**2,
949             "Per unit " + term + ", holding the other history measure fixed")
```

Read this block:

- Line 945: for each entry yielded by `['novelty', 'own_stock']`, bind/unpack it into `term` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 946: test `'mean_' + term in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 947, `(mean, sd)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 948: call `add` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 950–950:

```
950 return pd.DataFrame(rows)
```

Read this block:

- Line 950: return the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above. This ends the call.

implied wtp.add

Original function: `implied_wtp.add`. Source begins at line 919.

This nested helper adds one named WTP row from coefficient first and second moments. It uses variables captured from the enclosing WTP call, converts coefficient moments into ratio moments, and appends to the enclosing output list. Its side effect is a new row; it does not refit coefficients.

Definition and stated intent:

Source lines 919–919:

```
919 def add(term, mean, second, interpretation):
```

`def` creates this callable; the parentheses name its inputs (`term`, `mean`, `second`, `interpretation`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 920–923:

```
920 wtp_mean = mean * inverse_mean
921 variance = max(0., second * inverse_second - wtp_mean**2)
922 rows.append(dict(term=term, mean_wtp=wtp_mean, population_sd=np.sqrt(variance),
923                  interpretation=interpretation))
```

Read this block:

- Line 920, `wtp_mean`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 921, `variance`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 922: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

plot gamma distribution

Original function: `plot_gamma_distribution`. Source begins at line 952.

Input is an estimate table, output directory, and convergence flag. This is a side-effect function: it saves a figure and implicitly returns `None`. `set_index("parameter")["estimate"]` creates a name-indexed Series for coefficient lookup. The early bare `return` skips naive models with no gamma.

For positive SD, `linspace` makes a smooth horizontal grid, `norm.pdf` computes density, and `fill_between(..., where=x < 0)` shades the negative region. For zero SD a continuous density would divide by zero or misrepresent a point mass, so the `else` branch draws a vertical line. `norm.cdf(0, ...)` computes the implied population habit share. `Agg` supports file rendering without a GUI; `plt.close(fig)` releases the figure. This is a distribution of latent types, not an uncertainty interval for the mean.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 952–953:

```
952 def plot_gamma_distribution(table, output, converged=True):
953     """Plot the fitted population distribution, not uncertainty in its mean."""
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 954–954:

```
954 estimates = table.set_index("parameter")["estimate"]
```

Read this block:

- Line 954, `estimates`: Set this name to the entry/slice selected by `table.set_index('parameter')['estimate']`; brackets select from the object to their left.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

Source lines 955–956:

```
955 if "mean_gamma" not in estimates:
956     return  # Naive models have no gamma parameter.
```

Read this block:

- Line 955: test `'mean_gamma' not in estimates`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 956: bare return ends this call immediately with None. Later statements in the function are skipped.

Source lines 957–963:

```
957 import matplotlib
958 matplotlib.use("Agg")
959 import matplotlib.pyplot as plt
960 from scipy.stats import norm
961
962 mean = float(estimates["mean_gamma"])
963 sd = float(estimates.get("sd_gamma", 0.))
```

Read this block:

- Line 957: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 958: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 959: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 960: from `scipy.stats`, import `norm` into this module's namespace. Importing can execute the imported module's top-level setup.

- Line 962, `mean`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 963, `sd`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 964–964:

```
964 fig, ax = plt.subplots(figsize=(7, 5))
```

Read this block:

- Line 964, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 965–1000:

```
965 if "mixture_weight" in estimates:
966     weight = float(estimates["mixture_weight"])
967     mean2 = mean + float(estimates["gamma_mean_gap"])
968     sd2 = float(estimates["sd_gamma_2"])
969     x = np.linspace(min(mean - 4 * sd, mean2 - 4 * sd2) - .01,
970                     max(mean + 4 * sd, mean2 + 4 * sd2) + .01, 1001)
971     density = np.zeros_like(x)
972     negative = 0.
973     for mu, sigma, share in [(mean, sd, weight), (mean2, sd2, 1 - weight)]:
974         if sigma > 0:
975             component = share * norm.pdf(x, loc=mu, scale=sigma)
976             density += component
977             ax.plot(x, component, linestyle=":", label=f"Component (weight {share:.1%})")
978             negative += share * norm.cdf(0, loc=mu, scale=sigma)
979         else:
980             ax.axvline(mu, linestyle=":", label=f"Point mass (weight {share:.1%})")
981             negative += share * (mu < 0)
982     ax.plot(x, density, color="steelblue", label="Mixture continuous density")
983     ax.set_ylabel("Population density")
984     ax.legend()
985     description = f"Component means = {mean:.4g}, {mean2:.4g}; P(gamma < 0) = {negative:.1%}"
986 elif sd > 0:
987     x = np.linspace(mean - 4 * sd, mean + 4 * sd, 801)
988     density = norm.pdf(x, loc=mean, scale=sd)
989     ax.plot(x, density, color="steelblue")
990     ax.fill_between(x, density, where=x < 0, color="steelblue", alpha=.25)
991     ax.set_ylabel("Population density")
992     negative = norm.cdf(0, loc=mean, scale=sd)
993     description = f"Mean = {mean:.4g}; SD = {sd:.4g}; P(gamma < 0) = {negative:.1%}"
994 else:
995     ax.axvline(mean, color="steelblue", linewidth=2, label="Point mass (SD = 0)")
996     ax.set_xlim(mean - 1, mean + 1)
997     ax.set_yticks([])
998     ax.set_ylabel("Point mass")
999     ax.legend()
1000    description = f"All households: gamma = {mean:.4g} (SD = 0)"
```

Read this block:

- Line 965: test `'mixture_weight' in estimates`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 966, `weight`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 967, `mean2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 968, `sd2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 969, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 971, `density`: Set this name to the result of calling `np.zeros_like(...)`; the arguments and preceding method chain are shown above.
- Line 972, `negative`: Set this name to the literal value `0.0`.
- Line 973: for each entry yielded by `[(mean, sd, weight), (mean2, sd2, 1 - weight)]`, bind/unpack it into `(mu, sigma, share)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 974: test `sigma > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 975, `component`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 976: update `density` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 977: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 978: update `negative` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 980: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 981: update `negative` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 982: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 983: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 984: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 985, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 986: test `sd > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 987, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 988, `density`: Set this name to the result of calling `norm.pdf(...)`; the arguments and preceding method chain are shown above.
- Line 989: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 990: call `ax.fill_between` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 991: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 992, `negative`: Set this name to the result of calling `norm.cdf(...)`; the arguments and preceding method chain are shown above.
- Line 993, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.
- Line 995: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its docu-

mented side effect or assertion is what matters.
- Line 996: call `ax.set_xlim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 997: call `ax.set_yticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 998: call `ax.set_ylabel` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 999: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1000, `description`: Set this name to formatted text, substituting the brace-enclosed values into the f-string.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.set_ylabel`: Set the vertical-axis label, whose unit should match the plotted values.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.pdf`: Evaluate a probability density (here a normal distribution), not a PDF document file.
- `.fill_between`: Shade the area between curves/zero, optionally only where a Boolean condition is true.
- `.cdf`: Evaluate cumulative probability below a threshold, used for the population share with negative gamma.
- `.axvline`: Add a vertical reference line, such as zero gamma or a point-mass type.
- `.set_xlim`: Set the visible x-axis limits; this changes presentation rather than the calculated results.
- `.set_yticks`: Choose which vertical tick marks to show, including an empty list for a point-mass display.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 1001–1002:

```
1001 ax.axvline(0, color="gray", linestyle="--", linewidth=1)
1002 title = "Fitted population gamma distribution"
```

Read this block:

- Line 1001: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1002, `title`: Set this name to the literal text `'Fitted population gamma distribution'`.

Source lines 1003–1004:

```
1003 if not converged or table["at_bound"].any():
1004     title += " (provisional fit)"
```

Read this block:

- Line 1003: test `not converged or table['at_bound'].any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1004: update `title` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 1005–1009:

```
ax.set(xlabel="Gamma (common across flavors)", title=title)
fig.text(.5, .02, description, ha="center")
fig.tight_layout(rect=(0, .06, 1, 1))
fig.savefig(output / "gamma_distribution.pdf")
plt.close(fig)
```

Read this block:

- Line 1005: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1006: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1007: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1008: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1009: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.
- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.
- `.close`: Release a Figure or file resource, according to the receiving object.

export history distributions

Original function: `export_history_distributions`. Source begins at line 1011.

Inputs and outputs. The coefficient table supplies mean_novelty, mean_own_stock and their SDs; output is a directory. For each marginal, the function writes a summary of mean, SD, sign share and quantiles, a grid of densities/CDFs, and PDF/PNG plots. These describe population heterogeneity, not confidence intervals or household posterior means.

`NormalDist(mean, sd)` represents a fitted normal distribution; `.cdf(0)` gives its share below zero and `.inv_cdf(.25)` gives its lower quartile. A zero SD is handled as a point mass instead of dividing by zero. A negative own-stock coefficient means satiation, while a positive one means reinforcement. Convergence/boundary status marks provisional exports. This calculation recovers the already estimated distribution without optimizing demand again.

Definition and stated intent:

Source lines 1011–1011:

```
def export_history_distributions(table, output, converged=True):
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1012–1012:

```
p = table.set_index('parameter').estimate
```

Read this block:

- Line 1012, `p`: Set this name to the attribute `table.set_index('parameter').estimate` of the object before the dot.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

Source lines 1013–1014:

```
1013 if 'mean_novelty' not in p:
1014     return
```

Read this block:

- Line 1013: test `'mean_novelty' not in p`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1014: bare return ends this call immediately with None. Later statements in the function are skipped.

Source lines 1015–1020:

```
1015 output = Path(output)
1016 output.mkdir(parents=True, exist_ok=True)
1017 provisional = not converged or table.get('at_bound', pd.Series(False)).astype(str).str.lower().eq('true').any()
1018 summaries, curves = [], []
1019 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
1020 import matplotlib
```

Read this block:

- Line 1015, `output`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 1016: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1017, `provisional`: Set this name to the Boolean condition `not converged or table.get('at_bound', pd.Series(False)).astyp...`; it can be a scalar or a row/array mask depending on its inputs.
- Line 1018, `(summaries, curves)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 1019: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1020: import `matplotlib`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `str`: Convert to text for paths, messages, or serialization.

Source lines 1021–1023:

```
1021 matplotlib.use('Agg')
1022 import matplotlib.pyplot as plt
1023 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

Read this block:

- Line 1021: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1022: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.
- Line 1023, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 1024–1044:

```
1024 for ax, term, label in zip(axes, ['novelty', 'own_stock'], ['Switching / novelty', 'Own-stock habit']):
1025     mean, sd = float(p['mean_'+term]), float(p.get('sd_'+term, 0.))
1026     if not np.isfinite([mean, sd]).all() or sd < 0:
1027         raise ValueError('Invalid distribution parameters')
1028     normal = NormalDist(mean, sd) if sd > 0 else None
1029     row = dict(coefficient=term, mean=mean, sd=sd, point_mass=sd == 0,
1030                share_below_zero=normal.cdf(0) if normal else float(mean < 0), provisional=bool(provisional))
1031     for q in [.01, .05, .25, .5, .75, .95, .99]:
1032         row[f'q{int(100*q):02d}'] = normal.inv_cdf(q) if normal else mean
1033     summaries.append(row)
1034     if normal:
1035         x = np.linspace(mean-4*sd, mean+4*sd, 801)
1036         density = np.exp(-.5*((x-mean)/sd)**2)/(sd*np.sqrt(2*np.pi))
1037         ax.plot(x, density)
1038         curves.extend(dict(coefficient=term, value=float(v), density=float(d), cdf=normal.cdf(float(v)),
     ↪ point_mass=False)
1039                       for v, d in zip(x, density))
1040     else:
1041         ax.axvline(mean, label='Point mass')
1042         curves.append(dict(coefficient=term, value=mean, density=np.nan, cdf=1., point_mass=True))
1043     ax.axvline(0, color='gray', ls='--')
1044     ax.set(title=f'{label}\nMean={mean:.4g}; SD={sd:.4g}', xlabel='Utility coefficient', ylabel='Population
     ↪ density')
```

Read this block:

- Line 1024: for each entry yielded by `zip(axes, ['novelty', 'own_stock'], ['Switching / novelty', 'O...`, bind/unpack it into `(ax, term, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 1025, `(mean, sd)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 1026: test `not np.isfinite([mean, sd]).all() or sd < 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1027: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 1028, `normal`: Set this name to the true-branch value if `sd > 0` holds, otherwise the else-branch value.
- Line 1029, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 1031: for each entry yielded by `[0.01, 0.05, 0.25, 0.5, 0.75, 0.95, 0.99]`, bind/unpack it into `q` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 1032, `row[f'q{int(100 * q):02d}']`: Store the true-branch value if `normal` holds, otherwise the else-branch value in the selected entry/column.
- Line 1033: call `summaries.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1034: test `normal`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1035, `x`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 1036, `density`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 1037: call `ax.plot` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1038: call `curves.extend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1041: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its docu-

mented side effect or assertion is what matters.
- Line 1042: call `curves.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1043: call `ax.axvline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1044: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.axvline`: Add a vertical reference line, such as zero gamma or a point-mass type.
- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `NormalDist`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.plot`: Draw a line from supplied x/y values on the receiving plotting axes.
- `.extend`: Append every element from an iterable, unlike append which would add that iterable as one item.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.cdf`: Evaluate cumulative probability below a threshold, used for the population share with negative gamma.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 1045–1048:

```
1045 pd.DataFrame(summaries).to_csv(output/'history_distribution_summary.csv', index=False)
1046 pd.DataFrame(curves).to_csv(output/'history_distribution_grid.csv', index=False)
1047 fig.suptitle('Fitted population history distributions'+(' (provisional)' if provisional else ''))
1048 fig.tight_layout()
```

Read this block:

- Line 1045: call `pd.DataFrame(summaries).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1046: call `pd.DataFrame(curves).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1047: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1048: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 1049–1050:

```
1049 for extension in ['pdf', 'png']:
1050     fig.savefig(output/f'history_distributions.{extension}')
```

Read this block:

- Line 1049: for each entry yielded by `['pdf', 'png']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 1050: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 1051–1051:

```
1051 plt.close(fig)
```

Read this block:

- Line 1051: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.

plot history distributions

Original function: `plot_history_distributions`. Source begins at line 1054.

Delegates to the population-distribution export routine immediately after a split fit. Its side effects are tables and plots, not new coefficient estimates. The readable estimator keeps the full export calculation in the same file; the original estimator imports that helper.

Definition and stated intent:

Source lines 1054–1054:

```
1054 def plot_history_distributions(table, output, converged=True):
```

`def` creates this callable; the parentheses name its inputs (`table`, `output`, `converged`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1055–1055:

```
1055 export_history_distributions(table, output, converged)
```

Read this block:

- Line 1055: call `export_history_distributions` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `export_history_distributions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

write run notes

Original function: `write_run_notes`. Source begins at line 1062.

Writes a readable text rendering of the same automatic run information saved as JSON. pformat lays out nested records for reading; this is an output record, not an alternative settings file.

Definition and stated intent:

Source lines 1062–1063:

```
1062 def write_run_notes(output, information):
1063     """Write the same run information in plain text; you never edit a JSON file."""
```

`def` creates this callable; the parentheses name its inputs (`output`, `information`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1064–1069:

```
1064 output.mkdir(parents=True, exist_ok=True)
1065 text = "RUN INFORMATION\n\n"
1066 text += "This is an output record, not a settings file.\n"
1067 text += "Change the Python parameters at the top of the script to change the run.\n\n"
1068 text += pformat(information, sort_dicts=False, width=90)
1069 (output / "run_notes.txt").write_text(text + "\n")
```

Read this block:

- Line 1064: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1065, `text`: Set this name to the literal text `'RUN INFORMATION\n\n'`.
- Line 1066: update `text` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 1067: update `text` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 1068: update `text` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 1069: call `(output / 'run_notes.txt').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `pformat`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

fit model

Original function: `fit_model`. Source begins at line 1074.

This coordinates point fitting, reporting, optional inference, and saved files. It turns `best.x` into a labeled table. The boundary list comprehension pairs each coefficient with its bounds and tests distance below `1e-4`; flags do not automatically change the estimate. Saving point estimates before bootstrap protects the useful main result if a long inference job is interrupted.

The nested `estimate` callback supplies demand fitting to the generic bootstrap. The bootstrap runs only after a converged main fit and with the original weekly frame available. Missing inference columns are explicitly filled with NaN so an absent SE cannot be mistaken for zero. `vars(args).items()` enumerates settings; Path objects

become strings so JSON serialization can encode them. The returned `best` is still a SciPy result object, while CSV/JSON provide portable outputs.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Distribution outputs. After the main split fit, both history marginals are exported as parameter/quantile summaries, density/CDF grids and PDF/PNG plots. Their SDs are population variation; SEs in estimates.csv describe sampling uncertainty. The saved fit can regenerate these plots without optimization.

Definition and stated intent:

Source lines 1074–1075:

```
1074 def fit_model(data, settings, frame=None):
1075     """Estimate demand, compute requested inference, and save readable outputs."""
```

`def` creates this callable; the parentheses name its inputs (`data`, `settings`, `frame`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1076–1083:

```
1076 best, results, bounds = optimize_demand(data, settings)
1077 names = parameter_names(settings.model, data["outside"], data["control"])
1078 table = pd.DataFrame({"parameter": names, "estimate": best.x})
1079 table["at_bound"] = [
1080     min(abs(v - lo), abs(v - hi)) < 1e-4 for v, (lo, hi) in zip(best.x, bounds)
1081 ]
1082 settings.output.mkdir(parents=True, exist_ok=True)
1083 table.to_csv(settings.output / "point_estimates.csv", index=False)
```

Read this block:

- Line 1076, `(best, results, bounds)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 1077, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 1078, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 1079, `table['at_bound']`: Store a list computed by the displayed for-expression in the selected entry/column.
- Line 1082: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1083: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `min`: Select the smallest entry; key= can select by objective value rather than object identity.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

Source lines 1084–1089:

```
plot_gamma_distribution(table, settings.output, converged=best.success)
plot_history_distributions(table, settings.output, converged=best.success)
wtp = implied_wtp(table, getattr(settings, "habit_weight", .7))
wtp["provisional_fit"] = not best.success or bool(table["at_bound"].any())
wtp.to_csv(settings.output / "implied_wtp.csv", index=False)
inference = {"method": settings.se_method, "status": "disabled"}
```

Read this block:

- Line 1084: call `plot_gamma_distribution` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1085: call `plot_history_distributions` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1086, `wtp`: Set this name to the result of calling `implied_wtp(...)`; the arguments and preceding method chain are shown above.
- Line 1087, `wtp['provisional_fit']`: Store the Boolean condition `not best.success or bool(table['at_bound'].any())`; it can be a scalar or a row/array mask depending on its inputs in the selected entry/column.
- Line 1088: call `wtp.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1089, `inference`: Set this name to a dictionary with named entries `method`, `status`.

Syntax and calls used here:

- `plot_gamma_distribution`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `plot_history_distributions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `implied_wtp`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `getattr`: Retrieve an attribute by its string name, optionally supplying a default if it is absent.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 1090–1120:

```
if settings.se_method == "bootstrap":
    if not best.success:
        inference["status"] = "main fit not converged; bootstrap not run"
    elif frame is None:
        raise ValueError("Bootstrap requires the weekly input frame.")
    else:

        def estimate(sample, initial):
            bootstrap_data = make_arrays(sample, settings.capacity)
            fitted, _, _ = optimize_demand(
                bootstrap_data, settings, initial, settings.bootstrap_starts, quiet=True
            )
            return fitted

        standard_errors, inference = bootstrap_standard_errors(
            frame, settings, estimate, best.x, names
        )
        if standard_errors is not None:
            table = table.merge(
                standard_errors, on="parameter", validate="one_to_one"
            )
elif settings.se_method == "hessian":
    if not best.success:
        inference["status"] = "main fit not converged; Hessian inference not run"
    else:
```

```
1115        draws = make_draws(settings.draws, settings.seed, settings.model)
1116        gradient = lambda theta: demand_objective_gradient(theta, data, draws, settings.model)[1]
1117        standard_errors, inference = hessian_standard_errors(
1118            best.x, gradient, data["n_households"], bounds, names, settings.output)
1119        if standard_errors is not None:
1120            table = table.merge(standard_errors, on="parameter", validate="one_to_one")
```

Read this block:

- Line 1090: test `settings.se_method == 'bootstrap'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1091: test `not best.success`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1092, `inference['status']`: Store the literal text `'main fit not converged; bootstrap not run'` in the selected entry/column.
- Line 1093: test `frame is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1094: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.
- Line 1097: define nested callable `estimate`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 1104, `(standard_errors, inference)`: Unpack the result of calling `bootstrap_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 1107: test `standard_errors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1108, `table`: Set this name to the result of calling `table.merge(...)`; the arguments and preceding method chain are shown above.
- Line 1111: test `settings.se_method == 'hessian'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1112: test `not best.success`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1113, `inference['status']`: Store the literal text `'main fit not converged; Hessian inference not run'` in the selected entry/column.
- Line 1115, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 1116, `gradient`: Set this name to a one-expression callable, with input `theta` and returned expression `demand_objective_gradient(theta, data, draws, settin...`.
- Line 1117, `(standard_errors, inference)`: Unpack the result of calling `hessian_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 1119: test `standard_errors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1120, `table`: Set this name to the result of calling `table.merge(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `bootstrap_standard_errors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `hessian_standard_errors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 1122–1124:

```
1122 for column in ["std_error", "ci_lower", "ci_upper"]:
1123     if column not in table:
1124         table[column] = np.nan
```

Read this block:

- Line 1122: for each entry yielded by `['std_error', 'ci_lower', 'ci_upper']`, bind/unpack it into `column` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 1123: test `column not in table`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1124, `table[column]`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.

Source lines 1125–1125:

```
1125 table.to_csv(settings.output / "estimates.csv", index=False)
```

Read this block:

- Line 1125: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 1126–1152:

```
1126 summary = dict(
1127     model=settings.model,
1128     price_control=data["control"],
1129     history_specification="separate" if settings.model in ("separate_history", "heterogeneous_history") else
     ↪ "combined_or_none",
1130     sample_restriction="household_size == 1 on each purchase record",
1131     households=data["n_households"],
1132     weeks=len(data["hh"]),
1133     capacity=settings.capacity,
1134     inference=inference,
1135     converged=bool(best.success),
1136     message=str(best.message),
1137     nll=float(best.fun * data["n_households"]),
1138     parameters=len(names),
1139     aic=float(2 * best.fun * data["n_households"] + 2 * len(names)),
1140     bic=float(2 * best.fun * data["n_households"] + np.log(data["n_households"]) * len(names)),
1141     settings={
1142         k: str(v) if isinstance(v, Path) else v for k, v in vars(settings).items()
1143     },
1144     starts=[
1145         dict(
1146             nll=float(r.fun * data["n_households"]),
1147             converged=bool(r.success),
1148             theta=r.x.tolist(),
1149         )
1150         for r in results
1151     ],
1152 )
```

Read this block:

- Line 1126, `summary`: Set this name to the result of calling `dict(...)`; the arguments and preceding method

chain are shown above.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `len`: Count elements/rows rather than summing their numerical values.
- `str`: Convert to text for paths, messages, or serialization.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 1153–1156:

```
1153 (settings.output / "fit.json").write_text(json.dumps(summary, indent=2))
1154 write_run_notes(settings.output, summary)
1155 print("Inference: " + inference["status"], flush=True)
1156 print(table.to_string(index=False))
```

Read this block:

- Line 1153: call `(settings.output / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1154: call `write_run_notes` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1155: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1156: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `write_run_notes`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 1157–1158:

```
1157 if not best.success or table["at_bound"].any():
1158     print("Provisional fit: inspect convergence/bounds before interpretation.")
```

Read this block:

- Line 1157: test `not best.success or table['at_bound'].any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1158: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 1159–1159:

```
1159 return best
```

Read this block:

- Line 1159: return the object currently named `best`; plain assignment does not copy it. This ends the call.

fit model.estimate

Original function: `fit_model.estimate`. Source begins at line 1097.

This nested fake estimator returns an `OptimizeResult` with coefficients equal to resampled residual SDs. It deliberately depends on the recomputed controls, so frozen controls would change the bootstrap distribution. It declares success and a harmless objective because the test targets bootstrap orchestration, not numerical optimization. Mocking the demand fit makes this check faster and isolates whether first stages are rerun and empirical SEs calculated correctly.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 1097–1097:

```
1097 def estimate(sample, initial):
```

`def` creates this callable; the parentheses name its inputs (`sample`, `initial`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1098–1101:

```
1098 bootstrap_data = make_arrays(sample, settings.capacity)
1099 fitted, _, _ = optimize_demand(
1100     bootstrap_data, settings, initial, settings.bootstrap_starts, quiet=True
1101 )
```

Read this block:

- Line 1098, `bootstrap_data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 1099, `(fitted, _, _)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 1102–1102:

```
1102 return fitted
```

Read this block:

- Line 1102: return the object currently named `fitted`; plain assignment does not copy it. This ends the call.

read command line

Original function: `read_command_line`. Source begins at line 1167.

Creates optional command-line overrides, then sets parser defaults from the ordinary Python settings record. With no flags, the top-of-file parameters govern the run. Parsing arguments itself does not estimate demand or rebuild household inputs.

Definition and stated intent:

Source lines 1167–1168:

```
1167 def read_command_line(settings):
1168     """Optional server flags. Normal use: edit the parameters at the top."""
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1169–1181:

```
1169 parser = argparse.ArgumentParser(description=__doc__)
1170 parser.add_argument(
1171     "--sample",
1172     type=Path,
1173     default=DATA_DIR / "summary_outputs/yogurt_estimation_sample.parquet",
1174 )
1175 parser.add_argument("--flavors", type=Path, default=FLAVOR_FILE)
1176 parser.add_argument(
1177     "--products-extra",
1178     type=Path,
1179     help="Annual flavor metadata; defaults to RMS/combined/products_extra.parquet beside the RMS data directory.",
1180 )
1181 parser.add_argument("--output", type=Path, default=Path("estimation_output"))
```

Read this block:

- Line 1169, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 1170: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1175: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1176: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1181: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 1182–1195:

```
1182 parser.add_argument(
1183     "--model",
1184     choices=["naive", "homogeneous", "mixed_naive", "heterogeneous", "mixture", "separate_history",
     ↪ "heterogeneous_history"],
1185     default=model,
1186 )
1187 parser.add_argument("--first-year", type=int, default=2006)
1188 parser.add_argument("--last-year", type=int, default=2015)
1189 parser.add_argument("--max-households", type=int)
1190 parser.add_argument(
1191     "--capacity",
1192     type=int,
1193     default=0,
1194     help="0: conditional flavor demand; 7: outside=max(7-Q,0)",
1195 )
```

Read this block:

- Line 1182: call `parser.add_argument` with the displayed arguments. No assignment stores its return here;

its documented side effect or assertion is what matters.
- Line 1187: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1188: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1189: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1190: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 1196–1208:

```
parser.add_argument(
    "--no-price-control",
    action="store_true",
    default=no_price_control,
    help="Disable the price control using the overall other-DMA mean-price IV",
)
parser.add_argument("--household-dma-column", default="dma_cd")
parser.add_argument(
    "--se-method", choices=["hessian", "bootstrap", "none"], default="hessian"
)
parser.add_argument("--bootstrap-reps", type=int, default=49)
parser.add_argument("--bootstrap-starts", type=int, default=2)
parser.add_argument("--habit-weight", type=float, default=0.7)
```

Read this block:

- Line 1196: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1202: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1203: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1206: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1207: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1208: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 1209–1213:

```
parser.add_argument("--draws", type=int, default=128)
parser.add_argument("--starts", type=int, default=3)
parser.add_argument("--maxiter", type=int, default=600)
parser.add_argument("--seed", type=int, default=306)
parser.set_defaults(**vars(settings))
```

Read this block:

- Line 1209: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1210: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1211: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1212: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1213: call `parser.set_defaults` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.

Source lines 1214–1214:

```
1214 return parser.parse_args()
```

Read this block:

- Line 1214: return the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

run estimation

Original function: `run_estimation`. Source begins at line 1223.

Runs the preparation, optional price first stages, exact weekly-table export, and demand fit/inference in that order. Current simple defaults select heterogeneous_history and capacity 7. Reconstructing a deleted weekly table with regeneration is a separate path that skips optimization.

Definition and stated intent:

Source lines 1223–1224:

```
1223 def run_estimation(settings):
1224     """The actual research steps, in the order they run."""
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 1225–1226:

```
1225 if not 0 <= settings.habit_weight <= 1 or settings.capacity < 0 or settings.starts < 1:
1226     raise ValueError("Check habit weight, capacity, and starts.")
```

Read this block:

- Line 1225: test `not 0 <= settings.habit_weight <= 1 or settings.capacity < 0 or settings.star...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1226: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 1227–1230:

```
1227 if settings.first_year > settings.last_year or (
1228     settings.max_households is not None and settings.max_households < 1
1229 ):
1230     raise ValueError("Check years and household limit.")
```

Read this block:

- Line 1227: test `settings.first_year > settings.last_year or (settings.max_households is not N...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1230: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 1231–1232:

```
if settings.draws < 2 or settings.draws % 2 or settings.maxiter < 1:
    raise ValueError("Use an even draw count >= 2 and maxiter >= 1.")
```

Read this block:

- Line 1231: test `settings.draws < 2 or settings.draws % 2 or settings.maxiter < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1232: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 1233–1236:

```
if settings.se_method == "bootstrap" and (
    settings.bootstrap_reps < 20 or settings.bootstrap_starts < 1
):
    raise ValueError("Bootstrap requires at least 20 replications and one start.")
```

Read this block:

- Line 1233: test `settings.se_method == 'bootstrap' and (settings.bootstrap_reps < 20 or settin...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1236: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 1238–1238:

```
frame = prepare_data(settings)
```

Read this block:

- Line 1238, `frame`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `prepare_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 1241–1245:

```
if not settings.no_price_control:
    frame, first_stage = add_price_controls(frame)
    settings.output.mkdir(parents=True, exist_ok=True)
    first_stage.to_csv(settings.output / "first_stage.csv", index=False)
    print(first_stage.to_string(index=False), flush=True)
```

Read this block:

- Line 1241: test `not settings.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 1242, `(frame, first_stage)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 1243: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1244: call `first_stage.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1245: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 1246–1252:

```
1246 settings.output.mkdir(parents=True, exist_ok=True)
1247 # Step 3: save the exact table that will enter the likelihood.
1248 frame.to_csv(settings.output / "weekly_inputs.csv", index=False)
1249
1250 # Step 4: form numeric arrays, estimate demand, compute inference, save results.
1251 data = make_arrays(frame, settings.capacity)
1252 fit_model(data, settings, frame)
```

Read this block:

- Line 1246: call `settings.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1248: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 1251, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 1252: call `fit_model` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `fit_model`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/demand/split_pricing.py

Recommended current pricing reading path. The parameter section selects all three focal flavors; sections 1–2 recover persistent types and initial history; section 3 visibly solves one household at a time; section 4 permutes/validates each focal case; run_pricing writes separate folders and a combined comparison. It retains habit even though reporting groups rank novelty alone.

Module setup and direct-execution blocks

Source lines 2–11:

```
2 """Split-history pricing for each focal flavor, separate from demand estimation.
3
4 User-supplied paths only. No estimation is performed. --self-test uses only
5 generated synthetic inputs. See README.md for definitions.
6 """
7 from __future__ import annotations
8 import argparse
9 import hashlib
10 import json
11 import math
```

Read this block:

- Line 2: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 7: from `__future__`, import `annotations` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 9: import `hashlib`. An as-name is a convenient local alias, not another library.
- Line 10: import `json`. An as-name is a convenient local alias, not another library.
- Line 11: import `math`. An as-name is a convenient local alias, not another library.

Source lines 12–17:

```
import os
from pathlib import Path
import sys
from types import SimpleNamespace
from pprint import pformat
import tempfile
```

Read this block:

- Line 12: import `os`. An as-name is a convenient local alias, not another library.
- Line 13: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 14: import `sys`. An as-name is a convenient local alias, not another library.
- Line 15: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: from `pprint`, import `pformat` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 17: import `tempfile`. An as-name is a convenient local alias, not another library.

Source lines 18–19:

```
import numpy as np
import pandas as pd
```

Read this block:

- Line 18: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 19: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 24–29:

```
household_input = None
fit_dir = None
output = Path(__file__).resolve().parents[1] / "results" / "split_pricing"
focal_flavor = "all"                  # all, other, berry, or plain
discounts = np.linspace(0., 1., 101)  # Candidate focal-product coupons.
demand_delta = .92                    # Same default as theoretical supply.
```

Read this block:

- Line 24, `household_input`: Set this name to the literal value `None`.
- Line 25, `fit_dir`: Set this name to the literal value `None`.
- Line 26, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 27, `focal_flavor`: Set this name to the literal text `'all'`.
- Line 28, `discounts`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 29, `demand_delta`: Set this name to the literal value `0.92`.

Syntax and calls used here:

- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 30–33:

```
30 batch_size = 24                      # Progress interval in the household loop.
31 type_treatment = "posterior-mean"    # Or posterior-integrated.
32 save_state_policies = False
33 run_self_tests = False
```

Read this block:

- Line 30, `batch_size`: Set this name to the literal value `24`.
- Line 31, `type_treatment`: Set this name to the literal text `'posterior-mean'`.
- Line 32, `save_state_policies`: Set this name to the literal value `False`.
- Line 33, `run_self_tests`: Set this name to the literal value `False`.

Source lines 37–37:

```
37 FLAVORS = ["other", "berry", "plain"]
```

Read this block:

- Line 37, `FLAVORS`: Set this name to the list `['other', 'berry', 'plain']`.

Source lines 591–598:

```
591 if __name__ == "__main__":
592     settings = SimpleNamespace(household_input=household_input, fit_dir=fit_dir,
593         output=output, focal_flavor=focal_flavor, discounts=discounts,
594         demand_delta=demand_delta, batch_size=batch_size, type_treatment=type_treatment,
595         save_state_policies=save_state_policies, self_test=run_self_tests)
596     if len(sys.argv) > 1:
597         settings = read_command_line(settings)
598     run_pricing(settings)
```

Read this block:

- Line 591: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 592, `settings`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 596: test `len(sys.argv) > 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 597, `settings`: Set this name to the result of calling `read_command_line(...)`; the arguments and preceding method chain are shown above.
- Line 598: call `run_pricing` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `run_pricing`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `read_command_line`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

expit

Original function: `expit`. Source begins at line 39.

Evaluates the logistic probability with an exponential of a nonpositive magnitude to avoid overflow. Its input can be an array. This is a numerical transformation, not a demand-estimation call.

Definition and stated intent:

Source lines 39–39:

```
39 def expit(x):
```

`def` creates this callable; the parentheses name its inputs (`x`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 40–41:

```
40 x = np.asarray(x)
41 small = np.exp(-np.abs(x))
```

Read this block:

- Line 40, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 41, `small`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.

Source lines 42–42:

```
42 return np.where(x >= 0, 1/(1+small), small/(1+small))
```

Read this block:

- Line 42: return the result of calling `np.where(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

logsumexp

Original function: `logsumexp`. Source begins at line 44.

Computes log(sum(exp(x))) after subtracting the largest entry. The maximum is added back afterwards. axis selects the aggregation direction and keepdims retains singleton axes for broadcasting. This stabilizes likelihood normalization when utilities are large.

Definition and stated intent:

Source lines 44–44:

```
44 def logsumexp(x, axis=None, keepdims=False):
```

`def` creates this callable; the parentheses name its inputs (`x`, `axis`, `keepdims`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 45–47:

```
45 x = np.asarray(x)
46 maximum = np.max(x, axis=axis, keepdims=True)
47 result = maximum + np.log(np.exp(x-maximum).sum(axis=axis, keepdims=True))
```

Read this block:

- Line 45, `x`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 46, `maximum`: Set this name to the result of calling `np.max(...)`; the arguments and preceding method chain are shown above.
- Line 47, `result`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

Source lines 48–48:

```
48 return result if keepdims else np.squeeze(result, axis=axis)
```

Read this block:

- Line 48: return the true-branch value if `keepdims` holds, otherwise the else-branch value. This ends the call.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

plotting

Original function: `plotting`. Source begins at line 50.

Selects a noninteractive plotting backend and a writable temporary configuration directory, then returns pyplot. It does not read household data. Backend selection precedes importing pyplot so server runs do not require a display.

Definition and stated intent:

Source lines 50–50:

```
50 def plotting():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 51–54:

```
51 os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "pricing_exercises_mpl"))
52 import matplotlib
53 matplotlib.use("Agg")
54 import matplotlib.pyplot as plt
```

Read this block:

- Line 51: call `os.environ.setdefault` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 52: import `matplotlib`. An as-name is a convenient local alias, not another library.
- Line 53: call `matplotlib.use` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 54: import `matplotlib.pyplot as plt`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.use`: Select a Matplotlib backend; Agg renders to files without opening a GUI window.

Source lines 55–55:

```
55 return plt
```

Read this block:

- Line 55: return the object currently named `plt`; plain assignment does not copy it. This ends the call.

persistent draws

Original function: `persistent_draws`. Source begins at line 57.

Generates seeded antithetic normal integration points: half the draws and their negatives. Split pricing uses three columns for log price sensitivity, novelty and own-stock habit; combined-gamma pricing uses two. These are fixed population integration points, not newly estimated coefficients or a bootstrap sample.

Definition and stated intent:

Source lines 57–57:

```
57 def persistent_draws(number, seed):
```

`def` creates this callable; the parentheses name its inputs (`number`, `seed`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 58–59:

```
58 if number < 2 or number % 2:
59     raise ValueError("Draw count must be even and >=2.")
```

Read this block:

- Line 58: test `number < 2 or number % 2`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 59: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 60–60:

```
60 half = np.random.default_rng(seed).standard_normal((number//2, 3))
```

Read this block:

- Line 60, `half`: Set this name to the result of calling `.standard_normal(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.standard_normal`: Draw independent standard-normal values in the requested shape, before coefficient transformations.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 61–61:

```
61 return np.vstack([half, -half])
```

Read this block:

- Line 61: return the result of calling `np.vstack(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.vstack`: Stack input arrays vertically, adding rows while preserving their common column layout.

posterior

Original function: `posterior`. Source begins at line 63.

Purpose. Recover household types from a saved fit and a matching weekly table. Each draw represents one persistent type. Utilities yield weekly count-weighted log probabilities; bincount adds all weeks within household before log-sum-exp integrates types. Normalizing those whole-history likelihoods gives household posterior weights.

For split pricing, alpha is positive price sensitivity, and the history array has columns b_N and b_S. weights @ history produces both posterior means. Sorting only the novelty means assigns bottom/top floor(N/4) reporting groups; both coefficients still enter forecasts. Posterior SDs and effective sample size describe type uncertainty. The returned NLL must match the saved fit before pricing. The earlier combined-gamma file instead returns a single gamma array.

Definition and stated intent:

Source lines 63–64:

```
63 def posterior(frame, parameters, draws, control=True):
64     """Household-integrated multinomial likelihood; types persist across weeks."""
```

`def` creates this callable; the parentheses name its inputs (`frame`, `parameters`, `draws`, `control`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 65–70:

```
65 labels = np.sort(frame.household_code.unique())
66 hh = pd.Categorical(frame.household_code, categories=labels).codes
67 prices = frame[["price_"+f for f in FLAVORS]].to_numpy(float)
68 available = np.isfinite(prices) & (prices > 0)
69 counts = frame[FLAVORS+["outside"]].to_numpy(float)
70 novelty = frame[["novelty_"+f for f in FLAVORS]].to_numpy(float)
```

Read this block:

- Line 65, `labels`: Set this name to the result of calling `np.sort(...)`; the arguments and preceding method chain are shown above.
- Line 66, `hh`: Set this name to the attribute `pd.Categorical(frame.household_code, categories=labels).c...` of the object before the dot.
- Line 67, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 68, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 69, `counts`: Set this name to the result of calling `frame[FLAVORS + ['outside']].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 70, `novelty`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 71–72:

```
71 stock = frame[["stock_"+f for f in FLAVORS]].to_numpy(float)
72 cf = frame[["cf_"+f for f in FLAVORS]].to_numpy(float) if control else np.zeros_like(prices)
```

Read this block:

- Line 71, `stock`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 72, `cf`: Set this name to the true-branch value if `control` holds, otherwise the else-branch value.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 73–76:

```
73 if (not np.isfinite(counts).all() or (counts < 0).any()
74         or not np.isfinite(novelty).all() or not np.isfinite(stock).all() or not np.isfinite(cf).all()
75         or ((counts[:, :3] > 0) & ~available).any()):
76     raise ValueError("Invalid counts, history, controls, or purchased-alternative coverage.")
```

Read this block:

- Line 73: test `not np.isfinite(counts).all() or (counts < 0).any() or (not np.isfinite(novel...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 76: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 77–81:

```
77 beta = parameters["inside_constant"] + np.array([0., parameters["beta_berry"], parameters["beta_plain"]])
78 alpha = np.exp(parameters["mu_log_minus_alpha"] + parameters["sd_log_minus_alpha"]*draws[:, 0])
79 switching = parameters["mean_novelty"] + parameters["sd_novelty"]*draws[:, 1]
80 habit = parameters["mean_own_stock"] + parameters["sd_own_stock"]*draws[:, 2]
81 loglike = np.empty((len(labels), len(draws)))
```

Read this block:

- Line 77, `beta`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 78, `alpha`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 79, `switching`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 80, `habit`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 81, `loglike`: Set this name to the result of calling `np.empty(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `len`: Count elements/rows rather than summing their numerical values.

Source lines 82–87:

```
82 for d, (a, g, b) in enumerate(zip(alpha, switching, habit)):
83     u = beta-a*np.nan_to_num(prices)+g*novelty+b*stock+parameters.get("price_control", 0.)*cf
84     u = np.column_stack([np.where(available, u, -np.inf), np.zeros(len(frame))])
85     logp = u-logsumexp(u, axis=1, keepdims=True)
86     ll = (counts*np.where(counts > 0, logp, 0.)).sum(axis=1)
87     loglike[:, d] = np.bincount(hh, weights=ll, minlength=len(labels))
```

Read this block:

- Line 82: for each entry yielded by `enumerate(zip(alpha, switching, habit))`, bind/unpack it into `(d, (a, g, b))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 83, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 84, `u`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 85, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 86, `ll`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 87, `loglike[:, d]`: Store the result of calling `np.bincount(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.bincount`: Accumulate frequencies or weighted sums by dense nonnegative integer labels; repeated household codes are added together.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 88–94:

```
weights = np.exp(loglike-logsumexp(loglike, axis=1, keepdims=True))
nll = -float((logsumexp(loglike, axis=1)-np.log(len(draws))).sum())
means = weights @ switching
# Exactly floor(N/4) per tail. Household ID breaks ties reproducibly.
order = np.argsort(means, kind="stable")
groups = np.full(len(labels), "middle", dtype=object)
n = len(labels)//4
```

Read this block:

- Line 88, `weights`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 89, `nll`: Set this name to the negated or sign-adjusted expression `-float((logsumexp(loglike, axis=1) - np.log(len(draws))).....`.
- Line 90, `means`: Set this name to matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 92, `order`: Set this name to the result of calling `np.argsort(...)`; the arguments and preceding method chain are shown above.
- Line 93, `groups`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 94, `n`: Set this name to floor division of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- The // operator is floor division, used here for integer counts or calendar-year extraction.

Source lines 95–96:

```
if n < 1:
    raise ValueError("Need at least four households to form quartiles.")
```

Read this block:

- Line 95: test `n < 1`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 96: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Source lines 97–105:

```
groups[order[:n]], groups[order[-n:]] = "low", "high"
population_q = np.quantile(switching, [.25, .75])
table = pd.DataFrame(dict(household_code=labels, group=groups, switching_mean=means,
                          switching_sd=np.sqrt(np.maximum(weights @ switching**2-means**2, 0)),
                          alpha_mean=weights @ alpha, posterior_ess=1/(weights**2).sum(axis=1),
                          probability_switching_below_draw_q25=weights @ (switching <= population_q[0]),
                          probability_switching_above_draw_q75=weights @ (switching >= population_q[1])))
table["habit_mean"] = weights @ habit
table["habit_sd"] = np.sqrt(np.maximum(weights @ habit**2 - table.habit_mean.to_numpy()**2, 0))
```

Read this block:

- Line 97, `(groups[order[:n]], groups[order[-n:]])`: Unpack the tuple `('low', 'high')` into these targets in order.
- Line 98, `population_q`: Set this name to the result of calling `np.quantile(...)`; the arguments and preceding method chain are shown above.

- Line 99, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 104, `table['habit_mean']`: Store matrix multiplication of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 105, `table['habit_sd']`: Store the result of calling `np.sqrt(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Syntax and calls used here:

- `.quantile`: Find empirical quantiles, supplying percentile bootstrap endpoints rather than normal-Wald formulas.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.maximum`: Take an elementwise maximum, e.g. max(capacity minus packages, zero) independently each week.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 106–106:

```
106 return table, weights, alpha, np.column_stack([switching, habit]), beta, nll
```

Read this block:

- Line 106: return a tuple of 6 results. The caller can unpack them; return ends the call, unlike print.

compositions

Original function: `compositions`. Source begins at line 110.

Enumerates every nonnegative integer basket whose four counts sum to capacity: three flavors plus constructed nonpurchase. For capacity 7 there are 120 baskets. Enumerating all baskets preserves realized history transitions; substituting the mean basket would generally alter continuation values.

Definition and stated intent:

Source lines 110–111:

```
110 def compositions(capacity):
111     """All multinomial baskets of three inside flavors and outside."""
```

`def` creates this callable; the parentheses name its inputs (`capacity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 112–113:

```
112 return np.array([(a,b,c,capacity-a-b-c) for a in range(capacity+1)
113                  for b in range(capacity-a+1) for c in range(capacity-a-b+1)], dtype=int)
```

Read this block:

- Line 112: return the result of calling `np.array(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

forecast context

Original function: `forecast_context`. Source begins at line 115.

Relabels the first observed choice as t=0, adds its flavor shares once to its pre-choice stock, and uses that basket for next-week novelty. It carries the first menu/prices/controls forward, records the two subsequent pricing dates, and checks focal-menu eligibility. Later observed purchases affect type recovery but not this initial counterfactual history.

Definition and stated intent:

Source lines 115–120:

```
115 def forecast_context(frame, households):
116     """First observed choice establishes h0; price the next two consecutive weeks.
117 
118     Carry the initial observation's menu, posted prices and controls forward.
119     Type estimation still uses the complete observed sample.
120     """
```

`def` creates this callable; the parentheses name its inputs (`frame`, `households`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 121–127:

```
121 first = frame.sort_values(["household_code", "week"]).groupby("household_code", sort=True).head(1)
122 first = first.set_index("household_code").loc[households.household_code]
123 counts = first[FLAVORS].to_numpy(float)
124 totals = counts.sum(axis=1, keepdims=True)
125 shares = np.divide(counts, totals, out=np.zeros_like(counts), where=totals > 0)
126 # Saved stocks precede the observed choice; add its basket exactly once.
127 stocks = first[["stock_"+f for f in FLAVORS]].to_numpy(float)+shares
```

Read this block:

- Line 121, `first`: Set this name to the result of calling `.head(...)`; the arguments and preceding method chain are shown above.
- Line 122, `first`: Set this name to the entry/slice selected by `first.set_index('household_code').loc[households.househol...`; brackets select from the object to their left.
- Line 123, `counts`: Set this name to the result of calling `first[FLAVORS].to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 124, `totals`: Set this name to the result of calling `counts.sum(...)`; the arguments and preceding method chain are shown above.
- Line 125, `shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 127, `stocks`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 128–133:

```
128 novelty = shares.sum(axis=1, keepdims=True)-shares
129 prices = first[["price_"+f for f in FLAVORS]].to_numpy(float)
130 available = np.isfinite(prices) & (prices > 0)
```

```
eligible = available[:, 1]
households["berry_eligible"] = eligible
initial_weeks = first.week.to_numpy(dtype="datetime64[ns]")
```

Read this block:

- Line 128, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 129, `prices`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 130, `available`: Set this name to elementwise AND of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 131, `eligible`: Set this name to the entry/slice selected by `available[:, 1]`; brackets select from the object to their left.
- Line 132, `households['berry_eligible']`: Store the object currently named `eligible`; plain assignment does not copy it in the selected entry/column.
- Line 133, `initial_weeks`: Set this name to the result of calling `first.week.to_numpy(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 134–138:

```
households["initial_observed_week"] = initial_weeks
households["pricing_week1"] = initial_weeks+np.timedelta64(7, "D")
households["pricing_week2"] = initial_weeks+np.timedelta64(14, "D")
cf_cols = ["cf_"+f for f in FLAVORS]
cf = first[cf_cols].to_numpy(float) if all(c in first for c in cf_cols) else np.zeros_like(prices)
```

Read this block:

- Line 134, `households['initial_observed_week']`: Store the object currently named `initial_weeks`; plain assignment does not copy it in the selected entry/column.
- Line 135, `households['pricing_week1']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 136, `households['pricing_week2']`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 137, `cf_cols`: Set this name to a list computed by the displayed for-expression.
- Line 138, `cf`: Set this name to the true-branch value if `all((c in first for c in cf_cols))` holds, otherwise the else-branch value.

Syntax and calls used here:

- `all`: Require every generated condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 139–140:

```
if not np.isfinite(stocks).all() or not np.isfinite(novelty).all():
    raise ValueError("Nonfinite forecast history.")
```

Read this block:

- Line 139: test `not np.isfinite(stocks).all() or not np.isfinite(novelty).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 140: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

Source lines 141–142:

```
141 return dict(stocks=stocks, novelty=novelty, prices=np.where(available, prices, 0.),
142             available=available, cf=cf, eligible=eligible)
```

Read this block:

- Line 141: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.

digest

Original function: `digest`. Source begins at line 144.

Streams file bytes through SHA-256 to identify the exact input or source used by a run. Hashes are provenance checks, not economic statistics. This function is called during an explicit empirical run, not by the documentation builder.

Definition and stated intent:

Source lines 144–144:

```
144 def digest(path):
```

`def` creates this callable; the parentheses name its inputs (`path`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 145–145:

```
145 h = hashlib.sha256()
```

Read this block:

- Line 145, `h`: Set this name to the result of calling `hashlib.sha256(...)`; the arguments and preceding method chain are shown above.

Source lines 146–148:

```
146 with path.open("rb") as handle:
147     for chunk in iter(lambda: handle.read(1024*1024), b""):
148         h.update(chunk)
```

Read this block:

- Line 146: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 147: for each entry yielded by `iter(lambda: handle.read(1024 * 1024), b'')`, bind/unpack it into `chunk` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 148: call `h.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.open`: Open a file in the requested mode; using a with block ensures it closes on exit.

- `iter`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 149–149:

```
149 return h.hexdigest()
```

Read this block:

- Line 149: return the result of calling `h.hexdigest(...)`; the arguments and preceding method chain are shown above. This ends the call.

targeted coupons

Original function: `targeted_coupons`. Source begins at line 154.

Research calculation. Solve terminal prices for every realized first basket, then choose the initial coupon using immediate revenue plus discounted optimized continuation revenue. In the readable version the outer loop visits one household; remaining axes are type, basket, coupon and revenue metric. Default known posterior mean types leave just one type; posterior-integrated mode keeps draws and updates their weights through the first-basket likelihood.

In split pricing, history_coefficient has two columns: history1=b_N$N1$+b_SS1, history2=b_N$N2$+b_SS2. Every basket updates own stock and other-flavor novelty. Both coefficients enter both dates. The solver's focal slot is coordinate one; run_households permutes the selected flavor into that coordinate. Rival quantities retain distinct identities for history updates.

Terminal coupons maximize revenue after integrating remaining unobserved type uncertainty, before averaging over realized baskets. The firm cannot condition on an unobserved true draw. Both focal and no-focal-purchase branches count. Bottom/top novelty groups get personalized offers; middle households remain at posted prices. No coupons supplies a feasible benchmark and optimized revenue must weakly dominate it. Coupon prices lie between zero and the posted price; they are not unrestricted theoretical prices.

Definition and stated intent:

Source lines 154–163:

```
154 def targeted_coupons(context, households, weights, alpha, switching, beta, discounts,
155                      delta=1., capacity=7, control_coefficient=0.,
156                      batch_size=24, type_treatment="posterior-mean", state_output=None):
157     """Choose household-specific p1(h0) and terminal p2(h1) by backward induction.
158 
159     Work through ONE household at a time. This makes type-specific pricing
160     explicit and avoids the old extra household dimension in every array.
161     For this household, arrays run over possible types, baskets, and coupons.
162     batch_size only controls how frequently progress is reported.
163     """
```

`def` creates this callable; the parentheses name its inputs (`context`, `households`, `weights`, `alpha`, `switching`, `beta`, `discounts`, `delta`, `capacity`, `control_coefficient`, `batch_size`, `type_treatment`, `state_output`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 164–171:

```
164 baskets = compositions(capacity)
165 number_states = len(baskets)
166 number_coupons = len(discounts)
167 state_numbers = np.arange(number_states)
168 
169 # A basket records [other, berry, plain, outside]. History uses inside shares.
```

```
170 inside_counts = baskets[:, :3]
171 inside_total = inside_counts.sum(axis=1, keepdims=True)
```

Read this block:

- Line 164, `baskets`: Set this name to the result of calling `compositions(...)`; the arguments and preceding method chain are shown above.
- Line 165, `number_states`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 166, `number_coupons`: Set this name to the result of calling `len(...)`; the arguments and preceding method chain are shown above.
- Line 167, `state_numbers`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 170, `inside_counts`: Set this name to the entry/slice selected by `baskets[:, :3]`; brackets select from the object to their left.
- Line 171, `inside_total`: Set this name to the result of calling `inside_counts.sum(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `len`: Count elements/rows rather than summing their numerical values.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 172–182:

```
172 basket_shares = np.divide(
173     inside_counts, inside_total,
174     out=np.zeros_like(inside_counts, dtype=float), where=inside_total > 0)
175 basket_novelty = basket_shares.sum(axis=1, keepdims=True) - basket_shares
176 log_multinomial_coefficient = (
177     math.lgamma(capacity + 1)
178     - np.vectorize(math.lgamma)(baskets + 1).sum(axis=1))
179
180 eligible_households = np.flatnonzero(context["eligible"])
181 targeted = households.group.isin(["low", "high"]).to_numpy()
182 rows = []
```

Read this block:

- Line 172, `basket_shares`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 175, `basket_novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 176, `log_multinomial_coefficient`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 180, `eligible_households`: Set this name to the result of calling `np.flatnonzero(...)`; the arguments and preceding method chain are shown above.
- Line 181, `targeted`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 182, `rows`: Set this name to the list `[]`.

Syntax and calls used here:

- `.zeros_like`: Create zeros matching another array's shape/dtype instead of manually repeating dimensions.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.isin`: Test each pandas entry's membership in an allowed sequence, producing a Boolean Series.

Source lines 183–184:

```
maximum_mass_error = 0.
wrote_states = False
```

Read this block:

- Line 183, `maximum_mass_error`: Set this name to the literal value `0.0`.
- Line 184, `wrote_states`: Set this name to the literal value `False`.

Source lines 186–227:

```
for number_done, household in enumerate(eligible_households, start=1):
    prices = context["prices"][household]
    available = context["available"][household]
    stock = context["stocks"][household]
    novelty = context["novelty"][household]
    control = context["cf"][household]

    # Default: the firm knows estimated price, switching, and habit coefficients.
    # Sensitivity: retain the full probability distribution of possible types.
    if type_treatment == "posterior-mean":
        price_sensitivity = np.array([weights[household] @ alpha])
        history_coefficient = (weights[household] @ switching)[None, :]
        type_weights = np.ones(1)
    else:
        price_sensitivity = alpha
        history_coefficient = switching
        type_weights = weights[household]

    # Columns of history_coefficient are b_N (switching) and b_S (habit).
    b_n = history_coefficient[:, 0]
    b_s = history_coefficient[:, 1]
    history1 = b_n[:, None]*novelty + b_s[:, None]*stock
    history2 = (b_n[:, None, None]*basket_novelty
                + b_s[:, None, None]*(stock + basket_shares))
    utility1 = (beta - price_sensitivity[:, None]*prices
                + history1 + control_coefficient*control)
    utility2 = (beta - price_sensitivity[:, None, None]*prices
                + history2 + control_coefficient*control)
    utility1 = np.where(available, utility1, -np.inf)
    utility2 = np.where(available, utility2, -np.inf)

    # STEP A: terminal outcomes for every candidate coupon and every h1.
    # Metrics are [revenue, berry units, coupon expenditure].
    terminal = []
    for discount in discounts:
        utility = utility2.copy()
        utility[:, :, 1] += price_sensitivity[:, None] * prices[1] * discount
        outside = np.zeros((*utility.shape[:-1], 1))
        all_utilities = np.concatenate([utility, outside], axis=-1)
        log_denominator = logsumexp(all_utilities, axis=-1)
        berry_quantity = capacity * np.exp(utility[:, :, 1] - log_denominator)
        offered_price = prices[1] * (1 - discount)
```

Source lines 228–269:

```
    terminal.append(np.stack([
        berry_quantity * offered_price,
        berry_quantity,
        berry_quantity * prices[1] * discount], axis=-1))
terminal = np.asarray(terminal)

```

```
first = np.zeros((number_coupons, 3))
second = np.zeros((number_coupons, 3))
paths = []

# STEP B: try each p1(h0), calculate its state probabilities, and then
# select the best p2 for each state. Optimize BEFORE averaging revenue.
for action, discount in enumerate(discounts):
    utility = utility1.copy()
    utility[:, 1] += price_sensitivity * prices[1] * discount
    all_utilities = np.column_stack([utility, np.zeros(len(type_weights))])
    log_probability = all_utilities - logsumexp(all_utilities, axis=1, keepdims=True)
    terms = np.zeros((len(type_weights), number_states, 4))
    np.multiply(baskets[None, :, :], log_probability[:, None, :],
                out=terms, where=baskets[None, :, :] > 0)
    basket_probability = np.exp(log_multinomial_coefficient + terms.sum(axis=-1))
    error = float(np.abs(basket_probability.sum(axis=1) - 1).max())
    maximum_mass_error = max(maximum_mass_error, error)

    # Joint(type,h1) = initial type weight x Pr(h1 | type,p1).
    joint = type_weights[:, None] * basket_probability
    state_probability = joint.sum(axis=0)
    berry_quantity = capacity * (type_weights * np.exp(log_probability[:, 1])).sum()
    first[action] = [berry_quantity * prices[1] * (1 - discount),
                     berry_quantity,
                     berry_quantity * prices[1] * discount]

    # In the uncertainty sensitivity, integrate possible types before
    # choosing a terminal coupon: the firm cannot observe the true draw.
    # Result dimensions: terminal coupon x basket h1 x metric.
    weighted_terminal = (terminal * joint[None, :, :, None]).sum(axis=1)
    best_terminal_action = weighted_terminal[:, :, 0].argmax(axis=0)
    selected = weighted_terminal[best_terminal_action, state_numbers]
    second[action] = selected.sum(axis=0)
    if action == 0:
        no_coupon_second = weighted_terminal[0].sum(axis=0)
    paths.append((best_terminal_action, state_probability))
```

Source lines 270–311:

```

# STEP C: choose the first coupon by total expected discounted revenue.
dynamic_action = int((first[:, 0] + delta * second[:, 0]).argmax())
if not targeted[household]:
    dynamic_action = 0

# Keep posted prices as a benchmark. Only dynamic pricing is optimized.
for policy, action in [("no_coupon", 0), ("personalized_dynamic", dynamic_action)]:
    use_coupons = targeted[household] and policy == "personalized_dynamic"
    initial = first[action]
    best_terminal_action, state_probability = paths[action]
    if use_coupons:
        terminal_metrics = second[action]
        terminal_coupon = discounts[best_terminal_action]
    else:
        terminal_metrics = no_coupon_second
        terminal_coupon = np.zeros(number_states)

    p1 = prices[1] * (1 - discounts[action])
    p2 = prices[1] * (1 - terminal_coupon)
    mean_p2 = float(state_probability @ p2)
    ratio = np.divide(p1, p2, out=np.full(number_states, np.nan), where=p2 > 0)
    positive_mass = state_probability > 1e-14
```

```
    expected_ratio = np.nan
    if np.isfinite(ratio[positive_mass]).all():
        expected_ratio = float(state_probability[positive_mass] @ ratio[positive_mass])

    row = dict(
        household_code=households.iloc[household].household_code,
        group=households.iloc[household].group, policy=policy,
        discount1=discounts[action],
        expected_discount2=float(state_probability @ terminal_coupon),
        p1=p1, expected_p2=mean_p2,
        ratio_of_expected_prices=p1 / mean_p2 if mean_p2 > 0 else np.nan,
        expected_price_ratio=expected_ratio,
        revenue1=initial[0], revenue2=terminal_metrics[0],
        discounted_revenue=initial[0] + delta * terminal_metrics[0],
        berry_units1=initial[1], berry_units2=terminal_metrics[1],
        coupon_expenditure=initial[2] + delta * terminal_metrics[2])
    for branch, label in [(baskets[:, 1] == 0, "no_berry"), (baskets[:, 1] > 0, "berry")]:
        branch_mass = state_probability[branch].sum()
        row["probability_" + label] = branch_mass
```

Source lines 312–330:

```
        row["mean_p2_given_" + label] = np.nan
        if branch_mass > 0:
            row["mean_p2_given_" + label] = float(
                state_probability[branch] @ p2[branch] / branch_mass)
    rows.append(row)

    if state_output is not None and use_coupons:
        detail = pd.DataFrame(dict(
            household_code=row["household_code"], group=row["group"], policy=policy,
            state=state_numbers, rival1_units=baskets[:, 0], focal_units=baskets[:, 1],
            rival2_units=baskets[:, 2], outside=baskets[:, 3], probability=state_probability,
            discount1=discounts[action], discount2=terminal_coupon, p1=p1, p2=p2,
            price_ratio=ratio))
        detail.to_csv(state_output, mode="a" if wrote_states else "w",
                      header=not wrote_states, index=False, compression="gzip")
        wrote_states = True

if number_done % (batch_size * 40) == 0 or number_done == len(eligible_households):
    print(f"Targeted pricing: integrated {number_done}/{len(eligible_households)} eligible households", flush=True)
```

Read this block:

- Line 186: for each entry yielded by `enumerate(eligible_households, start=1)`, bind/unpack it into `(number_done, household)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 187, `prices`: Set this name to the entry/slice selected by `context['prices'][household]`; brackets select from the object to their left.
- Line 188, `available`: Set this name to the entry/slice selected by `context['available'][household]`; brackets select from the object to their left.
- Line 189, `stock`: Set this name to the entry/slice selected by `context['stocks'][household]`; brackets select from the object to their left.
- Line 190, `novelty`: Set this name to the entry/slice selected by `context['novelty'][household]`; brackets select from the object to their left.
- Line 191, `control`: Set this name to the entry/slice selected by `context['cf'][household]`; brackets select from the object to their left.
- Line 195: test `type_treatment == 'posterior-mean'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 196, `price_sensitivity`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

- Line 197, `history_coefficient`: Set this name to the entry/slice selected by `(weights[household] @ switching)[None, :]`; brackets select from the object to their left.
- Line 198, `type_weights`: Set this name to the result of calling `np.ones(...)`; the arguments and preceding method chain are shown above.
- Line 200, `price_sensitivity`: Set this name to the object currently named `alpha`; plain assignment does not copy it.
- Line 201, `history_coefficient`: Set this name to the object currently named `switching`; plain assignment does not copy it.
- Line 202, `type_weights`: Set this name to the entry/slice selected by `weights[household]`; brackets select from the object to their left.
- Line 205, `b_n`: Set this name to the entry/slice selected by `history_coefficient[:, 0]`; brackets select from the object to their left.
- Line 206, `b_s`: Set this name to the entry/slice selected by `history_coefficient[:, 1]`; brackets select from the object to their left.
- Line 207, `history1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 208, `history2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 210, `utility1`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 212, `utility2`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 214, `utility1`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 215, `utility2`: Set this name to the result of calling `np.where(...)`; the arguments and preceding method chain are shown above.
- Line 219, `terminal`: Set this name to the list `[]`.
- Line 220: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 221, `utility`: Set this name to the result of calling `utility2.copy(...)`; the arguments and preceding method chain are shown above.
- Line 222: update `utility[:, :, 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 223, `outside`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 224, `all_utilities`: Set this name to the result of calling `np.concatenate(...)`; the arguments and preceding method chain are shown above.
- Line 225, `log_denominator`: Set this name to the result of calling `logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 226, `berry_quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 227, `offered_price`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 228: call `terminal.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 232, `terminal`: Set this name to the result of calling `np.asarray(...)`; the arguments and preceding method chain are shown above.
- Line 234, `first`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 235, `second`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 236, `paths`: Set this name to the list `[]`.
- Line 240: for each entry yielded by `enumerate(discounts)`, bind/unpack it into `(action, discount)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 241, `utility`: Set this name to the result of calling `utility1.copy(...)`; the arguments and preceding method chain are shown above.

- Line 242: update `utility[:, 1]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 243, `all_utilities`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 244, `log_probability`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 245, `terms`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 246: call `np.multiply` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 248, `basket_probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 249, `error`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 250, `maximum_mass_error`: Set this name to the result of calling `max(...)`; the arguments and preceding method chain are shown above.
- Line 253, `joint`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 254, `state_probability`: Set this name to the result of calling `joint.sum(...)`; the arguments and preceding method chain are shown above.
- Line 255, `berry_quantity`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 256, `first[action]`: Store a list containing 3 displayed entries in the selected entry/column.
- Line 263, `weighted_terminal`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 264, `best_terminal_action`: Set this name to the result of calling `weighted_terminal[:, :, 0].argmax(...)`; the arguments and preceding method chain are shown above.
- Line 265, `selected`: Set this name to the entry/slice selected by `weighted_terminal[best_terminal_action, state_numbers]`; brackets select from the object to their left.
- Line 266, `second[action]`: Store the result of calling `selected.sum(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 267: test `action == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 268, `no_coupon_second`: Set this name to the result of calling `weighted_terminal[0].sum(...)`; the arguments and preceding method chain are shown above.
- Line 269: call `paths.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 272, `dynamic_action`: Set this name to the result of calling `int(...)`; the arguments and preceding method chain are shown above.
- Line 273: test `not targeted[household]`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 274, `dynamic_action`: Set this name to the literal value `0`.
- Line 277: for each entry yielded by `[('no_coupon', 0), ('personalized_dynamic', dynamic_action)]`, bind/unpack it into `(policy, action)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 278, `use_coupons`: Set this name to the Boolean condition `targeted[household] and policy == 'personalized_dynamic'`; it can be a scalar or a row/array mask depending on its inputs.
- Line 279, `initial`: Set this name to the entry/slice selected by `first[action]`; brackets select from the object to their left.
- Line 280, `(best_terminal_action, state_probability)`: Unpack the entry/slice selected by `paths[action]`; brackets select from the object to their left into these targets in order.
- Line 281: test `use_coupons`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 282, `terminal_metrics`: Set this name to the entry/slice selected by `second[action]`; brackets select from the object to their left.
- Line 283, `terminal_coupon`: Set this name to the entry/slice selected by `discounts[best_terminal_action]`;

brackets select from the object to their left.
- Line 285, `terminal_metrics`: Set this name to the object currently named `no_coupon_second`; plain assignment does not copy it.
- Line 286, `terminal_coupon`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 288, `p1`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 289, `p2`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 290, `mean_p2`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 291, `ratio`: Set this name to the result of calling `np.divide(...)`; the arguments and preceding method chain are shown above.
- Line 292, `positive_mass`: Set this name to the Boolean condition `state_probability > 1e-14`; it can be a scalar or a row/array mask depending on its inputs.
- Line 293, `expected_ratio`: Set this name to the attribute `np.nan` of the object before the dot.
- Line 294: test `np.isfinite(ratio[positive_mass]).all()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 295, `expected_ratio`: Set this name to the result of calling `float(...)`; the arguments and preceding method chain are shown above.
- Line 297, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 309: for each entry yielded by `[(baskets[:, 1] == 0, 'no_berry'), (baskets[:, 1] > 0, 'berry')]`, bind/unpack it into `(branch, label)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 310, `branch_mass`: Set this name to the result of calling `state_probability[branch].sum(...)`; the arguments and preceding method chain are shown above.
- Line 311, `row['probability_' + label]`: Store the object currently named `branch_mass`; plain assignment does not copy it in the selected entry/column.
- Line 312, `row['mean_p2_given_' + label]`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 313: test `branch_mass > 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 314, `row['mean_p2_given_' + label]`: Store the result of calling `float(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 316: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 318: test `state_output is not None and use_coupons`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 319, `detail`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 325: call `detail.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 327, `wrote_states`: Set this name to the literal value `True`.
- Line 329: test `number_done % (batch_size * 40) == 0 or number_done == len(eligible_households)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 330: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.where`: For pandas receivers keep original values where the condition is true and use the fallback elsewhere; np.where instead takes explicit true/false branches.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.abs`: Take absolute magnitude, as needed for tolerance tests and bound distances.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 332–333:

```
332 if maximum_mass_error > 1e-9:
333     raise AssertionError(f"Basket probability mass error: {maximum_mass_error}")
```

Read this block:

- Line 332: test `maximum_mass_error > 1e-09`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 333: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `AssertionError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 334–339:

```
334 table = pd.DataFrame(rows)
335 baseline = table.loc[table.policy == "no_coupon"].set_index("household_code").discounted_revenue
336 table["baseline_revenue"] = table.household_code.map(baseline)
337 table["revenue_change"] = table.discounted_revenue - table.baseline_revenue
338 table["revenue_change_pct"] = 100 * table.revenue_change / table.baseline_revenue
339 comparison = table.pivot(index="household_code", columns="policy", values="discounted_revenue")
```

Read this block:

- Line 334, `table`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 335, `baseline`: Set this name to the attribute `table.loc[table.policy == 'no_coupon'].set_index('househo...` of the object before the dot.

- Line 336, `table['baseline_revenue']`: Store the result of calling `table.household_code.map(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 337, `table['revenue_change']`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 338, `table['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 339, `comparison`: Set this name to the result of calling `table.pivot(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.pivot`: Reshape long labels into matrix columns; index/columns/values specify their roles.

Source lines 340–341:

```
340 if (comparison.personalized_dynamic + 1e-8 < comparison.no_coupon).any():
341     raise AssertionError("Dynamic pricing does not dominate the feasible no-coupon benchmark.")
```

Read this block:

- Line 340: test `(comparison.personalized_dynamic + 1e-08 < comparison.no_coupon).any()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 341: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 342–342:

```
342 return table, maximum_mass_error
```

Read this block:

- Line 342: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

run households

Original function: `run_households`. Source begins at line 349.

Reads an explicitly selected fitted-results directory and its weekly table, checks the model, convergence, bounds, capacity and input schema, then recovers posterior types and reproduces the saved likelihood. For a split run, it permutes the selected focal flavor into the solver's berry coordinate, moving counts, prices, stocks, novelty, controls and intercepts consistently. Type recovery happens before permutation using the original fitted flavor labels.

It calls the household solver, renames public berry-coordinate columns to focal names, and writes policy/group/aggregate tables and figures. The manifest records how rival state coordinates map back to flavors. Swapping only prices or only counts would silently price the wrong product. The three runs are separate focal sellers with fixed rival prices, not a simultaneous equilibrium.

Definition and stated intent:

Source lines 349–351:

```
349 def run_households(args):
350     # This branch is entered only when the person running the script supplies
351     # BOTH paths. Validation supplies generated temporary fixtures only.
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 352–352:

```
352 fit = json.loads((args.fit_dir / "fit.json").read_text())
```

Read this block:

- Line 352, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.

Source lines 353–354:

```
353 if fit["model"] != "heterogeneous_history" or not fit.get("converged", False):
354     raise ValueError("This exercise requires a converged split heterogeneous-history fit.")
```

Read this block:

- Line 353: test `fit['model'] != 'heterogeneous_history' or not fit.get('converged', False)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 354: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 355–355:

```
355 estimates = pd.read_csv(args.fit_dir / "point_estimates.csv")
```

Read this block:

- Line 355, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.

Source lines 356–357:

```
356 if "at_bound" in estimates and estimates.at_bound.astype(str).str.lower().eq("true").any():
357     raise ValueError("Fit has boundary parameters; review before running counterfactuals.")
```

Read this block:

- Line 356: test `'at_bound' in estimates and estimates.at_bound.astype(str).str.lower().eq('tr...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 357: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.
- `.lower`: Convert pandas/Python text to lowercase so comparisons need not list every capitalization.
- `.astype`: Convert the pandas/NumPy object's dtype; this changes representation, not merely its displayed label.

Source lines 358–360:

```
358 parameters = dict(zip(estimates.parameter, estimates.estimate))
359 settings = fit["settings"]
360 capacity = settings["capacity"]
```

Read this block:

- Line 358, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 359, `settings`: Set this name to the entry/slice selected by `fit['settings']`; brackets select from the object to their left.
- Line 360, `capacity`: Set this name to the entry/slice selected by `settings['capacity']`; brackets select from the object to their left.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.

Source lines 361–362:

```
361 if not isinstance(capacity, int) or not 1 <= capacity <= 12:
362     raise ValueError("Exact basket enumeration supports integer capacity 1..12.")
```

Read this block:

- Line 361: test `not isinstance(capacity, int) or not 1 <= capacity <= 12`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 362: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `isinstance`: Check a runtime type to select the appropriate conversion path.

Source lines 363–366:

```
363 frame = pd.read_csv(args.household_input, dtype={"household_code": str}, parse_dates=["week"])
364 required = ["household_code", "week", "outside", *FLAVORS]
365 required += [prefix+f for prefix in ["price_", "stock_", "novelty_"] for f in FLAVORS]
366 control = not settings.get("no_price_control", False)
```

Read this block:

- Line 363, `frame`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 364, `required`: Set this name to a list containing 4 displayed entries.
- Line 365: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 366, `control`: Set this name to the negated or sign-adjusted expression `not settings.get('no_price_control', False)`.

Syntax and calls used here:

- A starred entry expands an iterable into separate elements, here within a list/tuple or call.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Source lines 367–368:

```
367 if control:
368     required += ["cf_"+f for f in FLAVORS]
```

Read this block:

- Line 367: test `control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 368: update `required` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.

Source lines 369–369:

```
369 missing = set(required)-set(frame)
```

Read this block:

- Line 369, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 370–371:

```
370 if missing:
371     raise ValueError(f"Household input missing columns: {sorted(missing)}")
```

Read this block:

- Line 370: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 371: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 372–373:

```
372 if frame.household_code.isna().any() or frame.week.isna().any() or frame.duplicated(["household_code",
    ↪ "week"]).any():
373     raise ValueError("Need unique household/week rows with nonmissing IDs/dates.")
```

Read this block:

- Line 372: test `frame.household_code.isna().any() or frame.week.isna().any() or frame.duplica...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 373: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.

Source lines 374–375:

```
374 if len(frame) != fit["weeks"] or frame.household_code.nunique() != fit["households"]:
375     raise ValueError("Input household/week counts do not match fit metadata.")
```

Read this block:

- Line 374: test `len(frame) != fit['weeks'] or frame.household_code.nunique() != fit['househol...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 375: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.

Source lines 376–377:

```
draws = persistent_draws(settings["draws"], settings["seed"])
households, weights, alpha, switching, beta, nll = posterior(frame, parameters, draws, control)
```

Read this block:

- Line 376, `draws`: Set this name to the result of calling `persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 377, `(households, weights, alpha, switching, beta, nll)`: Unpack the result of calling `posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `persistent_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `posterior`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 379–380:

```
if not np.isclose(nll, fit["nll"], rtol=1e-9, atol=1e-4):
    raise ValueError(f"Likelihood mismatch: reconstructed {nll}, fit {fit['nll']}.")
```

Read this block:

- Line 379: test `not np.isclose(nll, fit['nll'], rtol=1e-09, atol=0.0001)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 380: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 381–381:

```
focal = args.focal_flavor
```

Read this block:

- Line 381, `focal`: Set this name to the attribute `args.focal_flavor` of the object before the dot.

Source lines 382–390:

```
if focal != "berry":
    mapping = {"berry": focal, focal: "berry"}
    original = frame.copy()
    original_beta = beta.copy()
    for destination, source in mapping.items():
        for prefix in ["", "price_", "stock_", "novelty_", "cf_"]:
            if prefix+source in original:
                frame[prefix+destination] = original[prefix+source]
        beta[FLAVORS.index(destination)] = original_beta[FLAVORS.index(source)]
```

Read this block:

- Line 382: test `focal != 'berry'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 383, `mapping`: Set this name to a dictionary with named entries `berry`.
- Line 384, `original`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 385, `original_beta`: Set this name to the result of calling `beta.copy(...)`; the arguments and preceding method chain are shown above.

- Line 386: for each entry yielded by `mapping.items()`, bind/unpack it into `(destination, source)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 387: for each entry yielded by `['', 'price_', 'stock_', 'novelty_', 'cf_']`, bind/unpack it into `prefix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 388: test `prefix + source in original`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 389, `frame[prefix + destination]`: Store the entry/slice selected by `original[prefix + source]`; brackets select from the object to their left in the selected entry/column.
- Line 390, `beta[FLAVORS.index(destination)]`: Store the entry/slice selected by `original_beta[FLAVORS.index(source)]`; brackets select from the object to their left in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.

Source lines 391–392:

```
391 context = forecast_context(frame, households)
392 counts = [int(((households.group == g) & households.berry_eligible).sum()) for g in ["low", "middle", "high"]]
```

Read this block:

- Line 391, `context`: Set this name to the result of calling `forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 392, `counts`: Set this name to a list computed by the displayed for-expression.

Syntax and calls used here:

- `forecast_context`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.

Source lines 393–394:

```
393 if any(n == 0 for n in counts):
394     raise ValueError("Each switching segment must have focal-eligible forecast households.")
```

Read this block:

- Line 393: test `any((n == 0 for n in counts))`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 394: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 395–405:

```
395 table, mass_error = targeted_coupons(
396     context, households, weights, alpha, switching, beta, args.discounts,
397     args.demand_delta, capacity,
398     parameters.get("price_control", 0.) if control else 0., args.batch_size,
399     args.type_treatment,
400     args.output / "targeted_state_policies.csv.gz" if args.save_state_policies else None)
401 households.rename(columns={"berry_eligible": "focal_eligible"}, inplace=True)
402 table.rename(columns={c: c.replace("berry", "focal") for c in table if "berry" in c}, inplace=True)
```

```
table["focal_flavor"] = focal
households.to_csv(args.output / "household_posterior_summary.csv", index=False)
table.to_csv(args.output / "targeted_household_results.csv", index=False)
```

Read this block:

- Line 395, `(table, mass_error)`: Unpack the result of calling `targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 401: call `households.rename` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 402: call `table.rename` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 403, `table['focal_flavor']`: Store the object currently named `focal`; plain assignment does not copy it in the selected entry/column.
- Line 404: call `households.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 405: call `table.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `targeted_coupons`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `.rename`: Change column labels (or apply a renaming function); this does not change their underlying values.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.

Source lines 406–412:

```
totals = ["revenue1", "revenue2", "discounted_revenue", "baseline_revenue", "revenue_change",
          "focal_units1", "focal_units2", "coupon_expenditure"]
aggregate = table.groupby("policy", sort=False)[totals].sum()
aggregate["revenue_change_pct"] = 100*aggregate.revenue_change/aggregate.baseline_revenue
aggregate.to_csv(args.output / "targeted_revenue_aggregate.csv")
group = table.groupby(["policy", "group"], sort=False)[totals].sum()
group["revenue_change_pct"] = 100*group.revenue_change/group.baseline_revenue
```

Read this block:

- Line 406, `totals`: Set this name to a list containing 8 displayed entries.
- Line 408, `aggregate`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 409, `aggregate['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 410: call `aggregate.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 411, `group`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 412, `group['revenue_change_pct']`: Store division of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.

Source lines 413–427:

```
means = table.groupby(["policy", "group"])[["p1", "expected_p2", "discount1", "expected_discount2",
                                            "ratio_of_expected_prices", "expected_price_ratio"]].mean()
group = group.join(means)
group["eligible_households"] = table.groupby(["policy", "group"]).size()
group.to_csv(args.output / "targeted_revenue_by_switching_group.csv")
households.groupby("group").agg(households=("household_code", "size"),
                                focal_eligible=("focal_eligible", "sum"),
                                switching_mean=("switching_mean", "mean"),
                                switching_mean_min=("switching_mean", "min"),
                                switching_mean_max=("switching_mean", "max"),
                                mean_posterior_sd=("switching_sd", "mean"),
                                habit_mean=("habit_mean", "mean"),
                                mean_habit_posterior_sd=("habit_sd", "mean"),
                                mean_alpha=("alpha_mean", "mean"),
                                median_posterior_ess=("posterior_ess", "median")).to_csv(args.output /
 ↪  "household_group_summary.csv")
```

Read this block:

- Line 413, `means`: Set this name to the result of calling `.mean(...)`; the arguments and preceding method chain are shown above.
- Line 415, `group`: Set this name to the result of calling `group.join(...)`; the arguments and preceding method chain are shown above.
- Line 416, `group['eligible_households']`: Store the result of calling `.size(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 417: call `group.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 418: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.join`: Join table/query rows on keys; left, inner, semi, and anti joins have different retention meanings.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.

Source lines 428–429:

```
make_targeting_figures(households, table, aggregate, args.output, delta=args.demand_delta,
                       focal_flavor=focal)
```

Read this block:

- Line 428: call `make_targeting_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_targeting_figures`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 430–442:

```
return dict(focal_flavor=focal, state_coordinates={"focal_units": focal,
            "rival1_units": "berry" if focal == "other" else "other",
            "rival2_units": "berry" if focal == "plain" else "plain"}, households=len(households),
             ↪  eligible_by_group=dict(zip(["low", "middle", "high"], counts)),
            reconstructed_nll=nll, basket_mass_error=mass_error,
            input_sha256=digest(args.household_input), fit_sha256=digest(args.fit_dir / "fit.json"),
            parameter_sha256=digest(args.fit_dir / "point_estimates.csv"),
            posterior_draws=len(draws), posterior_seed=settings["seed"],
            capacity=capacity, history_model="b_N*N + b_S*S", price_control=control,
            initial_history="first_observed_basket", pricing_periods=2,
```

```
            baskets=len(compositions(capacity)),
            discount_grid=args.discounts.tolist(), demand_delta=args.demand_delta,
            type_treatment=args.type_treatment,
            aggregate=aggregate.reset_index().to_dict("records"))
```

Read this block:

- Line 430: return the result of calling `dict(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.
- `compositions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.

make targeting figures

Original function: `make_targeting_figures`. Source begins at line 447.

Draws household posterior switching types, discounted dynamic revenue by reporting group, and coupon patterns. Split figures use b_N labels and the selected focal flavor. Revenue is R1+delta*R2 with terminal revenues averaged after state-dependent optimization; mean prices times mean quantities would not give the same object.

Definition and stated intent:

Source lines 447–448:

```
def make_targeting_figures(households, table, aggregate, output, delta=.95, focal_flavor="berry"):
    """Plot household types and individually optimized berry discounts."""
```

`def` creates this callable; the parentheses name its inputs (`households`, `table`, `aggregate`, `output`, `delta`, `focal_flavor`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 449–454:

```
plt = plotting()
groups = ["low", "middle", "high"]
labels = ["Low switching\n(bottom 25%)", "Middle switching\n(middle 50%)", "High switching\n(top 25%)"]
colors = ["#3575AD", "#999999", "#D17925"]
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
bins = np.histogram_bin_edges(households.switching_mean, bins=30)
```

Read this block:

- Line 449, `plt`: Set this name to the result of calling `plotting(...)`; the arguments and preceding method chain are shown above.
- Line 450, `groups`: Set this name to the list `['low', 'middle', 'high']`.
- Line 451, `labels`: Set this name to the list `['Low switching\n(bottom 25%)', 'Middle switching\n(middl...`.
- Line 452, `colors`: Set this name to the list `['#3575AD', '#999999', '#D17925']`.
- Line 453, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 454, `bins`: Set this name to the result of calling `np.histogram_bin_edges(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `plotting`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.subplots`: Create a Figure plus one or more Axes; the Axes hold plots and the Figure holds page-level layout.

Source lines 455–457:

```
455 for g, label, color in zip(groups, labels, colors):
456     axes[0].hist(households.loc[households.group == g, "switching_mean"],
457                  bins=bins, alpha=.65, label=label.replace("\n", " "), color=color)
```

Read this block:

- Line 455: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 456: call `axes[0].hist` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.hist`: Bin observed values, with optional weights controlling whether heights are counts, percentages, or another unit.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 458–465:

```
458 axes[0].set(title="Estimated consumer types", xlabel=r"Estimated household type (posterior mean $b_N$)",
459             ylabel="Number of households")
460 axes[0].legend(fontsize=9)
461 revenue = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["revenue1",
    ↪  "revenue2"]].sum().reindex(groups)
462 x = np.arange(len(groups))
463 # Average over state-contingent terminal prices first, discount, then total.
464 total_expected_revenue = revenue.revenue1 + delta * revenue.revenue2
465 axes[1].bar(x, total_expected_revenue, width=.5, color=colors)
```

Read this block:

- Line 458: call `axes[0].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 460: call `axes[0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 461, `revenue`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 462, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 464, `total_expected_revenue`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 465: call `axes[1].bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set`: Set multiple plot properties using named arguments rather than separate setter calls.
- `.legend`: Render labels previously attached to plotted series, identifying curves.
- `.reindex`: Align to an explicitly requested row/column order, adding missing labels with a chosen fill value if provided.

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `len`: Count elements/rows rather than summing their numerical values.
- `.bar`: Draw bar heights from supplied categories and values; it does not compute the underlying shares.

Source lines 466–472:

```
466 axes[1].set_xticks(x, labels)
467 axes[1].set(title="Total Expected Revenue",
468            xlabel="Consumer type group", ylabel="Total Expected Revenue")
469 axes[1].set_ylim(bottom=0)
470 axes[1].grid(axis="y", alpha=.2)
471 axes[1].set_axisbelow(True)
472 axes[1].axhline(0, color="gray", linewidth=.6)
```

Read this block:

- Line 466: call `axes[1].set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 467: call `axes[1].set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 469: call `axes[1].set_ylim` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 470: call `axes[1].grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 471: call `axes[1].set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 472: call `axes[1].axhline` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.grid`: Add visual grid lines without changing underlying numerical results.
- `.axhline`: Add a horizontal reference line, commonly zero for a derivative plot.

Source lines 473–477:

```
473 fig.text(.5, .015,
474          f"First-period revenue + {delta:g} × expected terminal revenue. "
475          f"Low/high types receive personalized {focal_flavor} offers; middle types receive no coupons.",
476          ha="center", fontsize=9)
477 fig.tight_layout(rect=(0, .05, 1, 1))
```

Read this block:

- Line 473: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 477: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.text`: Place a text label on the axes/Figure at the supplied coordinates.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.
- `.tight_layout`: Adjust plot spacing to reduce overlap between labels and content.

Source lines 478–479:

```
478 for extension in ["png", "pdf"]:
479     fig.savefig(output / f"household_coupon_results.{extension}", dpi=180)
```

Read this block:

- Line 478: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 479: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.savefig`: Write the Figure to a file whose extension selects PDF/PNG; this is a filesystem side effect.

Source lines 480–487:

```
480 plt.close(fig)
481
482 fig, ax = plt.subplots(figsize=(9, 5))
483 x = np.arange(3)
484 means = table.loc[table.policy == "personalized_dynamic"].groupby("group")[["discount1",
    ↪  "expected_discount2"]].mean().reindex(groups)
485 ax.bar(x-.18, 100*means.discount1, width=.36, label=r"$p_1(h_0)$ offer", color=colors[0])
486 ax.bar(x+.18, 100*means.expected_discount2, width=.36,
487        label=r"Terminal $p_2(h_1)$ offer (expected over histories)", color=colors[2])
```

Read this block:

- Line 480: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 482, `(fig, ax)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 483, `x`: Set this name to the result of calling `np.arange(...)`; the arguments and preceding method chain are shown above.
- Line 484, `means`: Set this name to the result of calling `.reindex(...)`; the arguments and preceding method chain are shown above.
- Line 485: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 486: call `ax.bar` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.close`: Release a Figure or file resource, according to the receiving object.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.

Source lines 488–494:

```
488 ax.set_xticks(x, labels)
489 ax.set(xlabel="Consumer type group", ylim=(0, 105),
490        ylabel=f"Mean optimal {focal_flavor} discount (% of posted price)")
491 ax.grid(axis="y", alpha=.2)
492 ax.set_axisbelow(True)
493 ax.legend(fontsize=8, loc="upper left")
494 fig.suptitle("Dynamic pricing: optimal discounts by consumer type")
```

Read this block:

- Line 488: call `ax.set_xticks` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 489: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 491: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 492: call `ax.set_axisbelow` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 493: call `ax.legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 494: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 495–497:

```
fig.text(.5, .015, f"Household-weighted means among {focal_flavor}-eligible households. Middle group receives no coupons by policy design.",
         ha="center", fontsize=9)
fig.tight_layout(rect=(0, .05, 1, .94))
```

Read this block:

- Line 495: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 497: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 498–499:

```
for extension in ["png", "pdf"]:
    fig.savefig(output / f"discounts_by_consumer_type.{extension}", dpi=180)
```

Read this block:

- Line 498: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 499: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 500–504:

```
plt.close(fig)

dynamic = table.loc[table.policy == "personalized_dynamic"].merge(
    households[["household_code", "switching_mean", "alpha_mean"]], on="household_code", validate="one_to_one")
fig, axes = plt.subplots(2, 2, figsize=(13, 9), sharey=True)
```

Read this block:

- Line 500: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 502, `dynamic`: Set this name to the result of calling `.merge(...)`; the arguments and preceding method chain are shown above.
- Line 504, `(fig, axes)`: Unpack the result of calling `plt.subplots(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `.merge`: Join pandas tables through matching keys; how controls retention and validate checks intended key cardinality.

Source lines 505–517:

```
for column, (variable, xlabel) in enumerate([
        ("switching_mean", r"Estimated household type (posterior mean $b_N$)"),
        ("alpha_mean", r"Estimated price sensitivity (posterior mean $a$)")]):
    for row, (discount, title) in enumerate([
            ("discount1", r"Optimal discount at $p_1(h_0)$"),
```

```
            ("expected_discount2", r"Expected terminal discount at $p_2(h_1)$")]):
        ax = axes[row, column]
        for g, label, color in zip(groups, labels, colors):
            subset = dynamic.loc[dynamic.group == g]
            ax.scatter(subset[variable], 100*subset[discount], s=12, alpha=.5,
                       color=color, label=label.replace("\n", " "))
        ax.set(xlabel=xlabel, ylabel=f"{focal_flavor.title()} discount (% of posted price)", title=title, ylim=(-2,
    102))
        ax.grid(alpha=.2)
```

Read this block:

- Line 505: for each entry yielded by `enumerate([('switching_mean', 'Estimated household type (poste...`, bind/unpack it into `(column, (variable, xlabel))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 508: for each entry yielded by `enumerate([('discount1', 'Optimal discount at $p_1(h_0)$'), ('...`, bind/unpack it into `(row, (discount, title))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 511, `ax`: Set this name to the entry/slice selected by `axes[row, column]`; brackets select from the object to their left.
- Line 512: for each entry yielded by `zip(groups, labels, colors)`, bind/unpack it into `(g, label, color)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 513, `subset`: Set this name to the entry/slice selected by `dynamic.loc[dynamic.group == g]`; brackets select from the object to their left.
- Line 514: call `ax.scatter` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 516: call `ax.set` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 517: call `ax.grid` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 518–522:

```
axes[0, 0].legend(fontsize=8)
fig.suptitle("Dynamic pricing: household discounts and estimated consumer characteristics")
fig.text(.5, .015, f"Each point is one {focal_flavor}-eligible household. Terminal offers average over the
    household's realized h1 histories.",
         ha="center", fontsize=9)
fig.tight_layout(rect=(0, .04, 1, .95))
```

Read this block:

- Line 518: call `axes[0, 0].legend` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 519: call `fig.suptitle` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 520: call `fig.text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 522: call `fig.tight_layout` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 523–524:

```
for extension in ["png", "pdf"]:
    fig.savefig(output / f"discounts_by_household_type.{extension}", dpi=180)
```

Read this block:

- Line 523: for each entry yielded by `['png', 'pdf']`, bind/unpack it into `extension` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 524: call `fig.savefig` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 525–525:

```
525 plt.close(fig)
```

Read this block:

- Line 525: call `plt.close` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

self test

Original function: `self_test`. Source begins at line 528.

Runs generated examples and assertions to check numerical calculations without launching the user-selected empirical exercise. Read the specific body to distinguish the CSV-blocked simple split checks from earlier complete export tests that reopen their own synthetic CSVs. No private input is needed.

Definition and stated intent:

Source lines 528–528:

```
528 def self_test():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 529–531:

```
529 import unittest
530 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
531 suite = unittest.defaultTestLoader.loadTestsFromName("checks.test_split_pricing")
```

Read this block:

- Line 529: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 530: call `sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 531, `suite`: Set this name to the result of calling `.loadTestsFromName(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 532–533:

```
532 if not unittest.TextTestRunner(verbosity=2).run(suite).wasSuccessful():
533     raise RuntimeError("Split pricing checks failed")
```

Read this block:

- Line 532: test `not unittest.TextTestRunner(verbosity=2).run(suite).wasSuccessful()`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 533: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `RuntimeError`: Construct an exception describing why this operation cannot proceed normally.
- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

read command line

Original function: `read_command_line`. Source begins at line 538.

Creates optional command-line overrides, then sets parser defaults from the ordinary Python settings record. With no flags, the top-of-file parameters govern the run. Parsing arguments itself does not estimate demand or rebuild household inputs.

Definition and stated intent:

Source lines 538–538:

```
538 def read_command_line(settings):
```

`def` creates this callable; the parentheses name its inputs (`settings`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 539–545:

```
539 parser = argparse.ArgumentParser(description=__doc__)
540 parser.add_argument("--household-input", type=Path, help="Explicit household input selected by you; no default.")
541 parser.add_argument("--fit-dir", type=Path, help="Explicit existing fitted-results directory; no default.")
542 parser.add_argument("--output", type=Path, default=Path(__file__).resolve().parent / "split_empirical_outputs")
543 parser.add_argument("--discounts", type=float, nargs="+", default=np.linspace(0., 1., 101),
544                     help="Candidate discounts; default is every 1%% from 0%% through 100%%.")
545 parser.add_argument("--demand-delta", type=float, default=.92)
```

Read this block:

- Line 539, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 540: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 541: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 542: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 543: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 545: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 546–552:

```
546 parser.add_argument("--batch-size", type=int, default=24)
547 parser.add_argument("--type-treatment", choices=["posterior-mean", "posterior-integrated"],
    ↪  default="posterior-mean",
```

```
548                     help="Default treats estimated household types as known, matching base supply; optional
                         ↪  posterior uncertainty sensitivity.")
549 parser.add_argument("--save-state-policies", action="store_true", help="Export every optimized household/basket
     ↪  terminal coupon to compressed CSV.")
550 parser.add_argument("--focal-flavor", choices=["all", *FLAVORS], default="all")
551 parser.add_argument("--self-test", action="store_true", help="Generated synthetic inputs only.")
552 parser.set_defaults(**vars(settings))
```

Read this block:

- Line 546: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 547: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 549: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 550: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 551: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 552: call `parser.set_defaults` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- A starred entry expands an iterable into separate elements, here within a list/tuple or call.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.

Source lines 553–553:

```
553 return parser.parse_args()
```

Read this block:

- Line 553: return the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

run pricing

Original function: `run_pricing`. Source begins at line 558.

Validates explicit paths and coupon settings, loops through all selected focal flavors, and writes separate policy folders plus a root comparison table, manifest and plain-text notes. all selects three independent exercises; neither results nor competitor responses are shared as an equilibrium. Its default discount factor is .92.

Definition and stated intent:

Source lines 558–558:

```
558 def run_pricing(args):
```

`def` creates this callable; the parentheses name its inputs (`args`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 559–561:

```
559 if args.self_test:
560     self_test()
561     return
```

Read this block:

- Line 559: test `args.self_test`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 560: call `self_test` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 561: bare return ends this call immediately with None. Later statements in the function are skipped.

Syntax and calls used here:

- `self_test`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 562–563:

```
562 if args.household_input is None or args.fit_dir is None:
563     raise ValueError("Provide --household-input AND --fit-dir; there are no default empirical input paths.")
```

Read this block:

- Line 562: test `args.household_input is None or args.fit_dir is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 563: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.

Source lines 564–564:

```
564 args.discounts = np.unique(np.asarray(args.discounts, dtype=float))
```

Read this block:

- Line 564, `args.discounts`: Set this name to the result of calling `np.unique(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.asarray`: View/convert data as an array without demanding a copy when the representation already fits.

Source lines 565–567:

```
565 if (not np.isfinite(args.discounts).all() or args.discounts[0] != 0. or args.discounts[-1] > 1.
566         or not 0 <= args.demand_delta <= 1 or args.batch_size < 1):
567     raise ValueError("Discounts must include zero and lie in [0,1]; delta in [0,1], batch size >=1.")
```

Read this block:

- Line 565: test `not np.isfinite(args.discounts).all() or args.discounts[0] != 0.0 or args.dis...`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 567: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Source lines 568–571:

```
568 args.output.mkdir(parents=True, exist_ok=True)
569 output_root = args.output
570 selected = FLAVORS if args.focal_flavor == "all" else [args.focal_flavor]
```

```
571 reports, comparisons = {}, []
```

Read this block:

- Line 568: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 569, `output_root`: Set this name to the attribute `args.output` of the object before the dot.
- Line 570, `selected`: Set this name to the true-branch value if `args.focal_flavor == 'all'` holds, otherwise the else-branch value.
- Line 571, `(reports, comparisons)`: Unpack a tuple containing 2 displayed entries into these targets in order.

Syntax and calls used here:

- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 572–578:

```
572 for focal in selected:
573     args.focal_flavor = focal
574     args.output = output_root / focal
575     args.output.mkdir(parents=True, exist_ok=True)
576     reports[focal] = run_households(args)
577     for row in reports[focal]["aggregate"]:
578         comparisons.append(dict(focal_flavor=focal, **row))
```

Read this block:

- Line 572: for each entry yielded by `selected`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 573, `args.focal_flavor`: Set this name to the object currently named `focal`; plain assignment does not copy it.
- Line 574, `args.output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 575: call `args.output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 576, `reports[focal]`: Store the result of calling `run_households(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 577: for each entry yielded by `reports[focal]['aggregate']`, bind/unpack it into `row` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 578: call `comparisons.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `run_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 579–586:

```
579 args.output = output_root
580 args.focal_flavor = "all" if len(selected) == 3 else selected[0]
581 pd.DataFrame(comparisons).to_csv(output_root / "focal_flavor_comparison.csv", index=False)
582 report = reports
583 manifest = dict(settings={k: str(v) if isinstance(v, Path) else v.tolist() if isinstance(v,np.ndarray) else v for
    ↪  k,v in vars(args).items()},
584                 results=report, script_sha256=digest(Path(__file__).resolve()),
585                 versions=dict(python=sys.version, numpy=np.__version__, pandas=pd.__version__))
586 (args.output / "run_manifest.json").write_text(json.dumps(manifest, indent=2)+"\n")
```

Read this block:

- Line 579, `args.output`: Set this name to the object currently named `output_root`; plain assignment does not copy it.
- Line 580, `args.focal_flavor`: Set this name to the true-branch value if `len(selected) == 3` holds, otherwise the else-branch value.
- Line 581: call `pd.DataFrame(comparisons).to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 582, `report`: Set this name to the object currently named `reports`; plain assignment does not copy it.
- Line 583, `manifest`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 586: call `(args.output / 'run_manifest.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `digest`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `isinstance`: Check a runtime type to select the appropriate conversion path.
- `str`: Convert to text for paths, messages, or serialization.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `vars`: Expose an object's attribute dictionary, here the parsed settings saved with the result.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.

Source lines 587–588:

```
587 (args.output / "run_notes.txt").write_text(pformat(manifest, sort_dicts=False, width=90)+"\n")
588 print(f"Standalone empirical targeting completed: {args.output}")
```

Read this block:

- Line 587: call `(args.output / 'run_notes.txt').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 588: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `pformat`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

../final_simple/demand/regenerate_weekly_inputs.py

Readable server reconstruction entry point. It imports preparation/likelihood functions from the local estimation module, avoiding a second demand implementation. Edit paths only on the server. Consistency failures stop before replacing the export; successful regeneration leaves fitted demand coefficients unchanged.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Regenerate weekly inputs on the server without refitting saved demand parameters."""
import argparse
import json
from pathlib import Path
import tempfile
from types import SimpleNamespace
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 3: import `json`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 6: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 8–9:

```
import numpy as np
import pandas as pd
```

Read this block:

- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 11–14:

```
try:
    from .estimation import prepare_data, make_arrays, make_draws, parameter_names, household_loglike,
    ↪ add_price_controls
except ImportError:
    from estimation import prepare_data, make_arrays, make_draws, parameter_names, household_loglike,
    ↪ add_price_controls
```

Read this block:

- Line 11: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 12: from `estimation`, import `prepare_data, make_arrays, make_draws, parameter_names, household_loglike, add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 13: catch `ImportError` from the preceding try; the optional as-name holds the error object for logging.
- Line 14: from `estimation`, import `prepare_data, make_arrays, make_draws, parameter_names, household_loglike, add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 17–19:

```
fit_dir = None
sample = None     # Optional override if the saved prepared-sample path moved.
flavors = None    # Optional override if the saved flavor-reference path moved.
```

Read this block:

- Line 17, `fit_dir`: Set this name to the literal value `None`.

- Line 18, `sample`: Set this name to the literal value `None`.
- Line 19, `flavors`: Set this name to the literal value `None`.

Source lines 90–91:

```
90 if __name__ == '__main__':
91     main()
```

Read this block:

- Line 90: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 91: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

regenerate

Original function: `regenerate`. Source begins at line 24.

Job and contract. Rebuild the deleted weekly_inputs.csv on the server using the existing fit directory, optionally overriding moved sample/reference paths. The function reads saved settings, calls the same sample/history/menu preparation, and reconstructs price controls. It never calls the demand optimizer or replaces fitted parameters and inference.

`SimpleNamespace(**settings)` turns dictionary keys into ordinary attributes. Converting saved path strings back to Path objects restores the input contract. The reconstructed table must match household/week counts and the saved integrated likelihood, using the exact parameter order, draw count and seed. A mismatch raises before publishing a CSV.

`NamedTemporaryFile` creates a private intermediate beside the destination; `temporary.replace(target)` publishes only a verified complete export. The finally block cleans up an unfinished intermediate. The JSON regeneration record stores diagnostics, not household rows. Run this entry point on the server; reading the textbook does not execute it.

Definition and stated intent:

Source lines 24–24:

```
24 def regenerate(fit_dir, sample=None, flavors=None):
```

`def` creates this callable; the parentheses name its inputs (`fit_dir`, `sample`, `flavors`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 25–30:

```
25 fit_dir = Path(fit_dir)
26 fit = json.loads((fit_dir / 'fit.json').read_text())
27 settings = dict(fit['settings'])
28 required = ['sample', 'first_year', 'last_year', 'max_households', 'capacity',
29             'no_price_control', 'household_dma_column', 'habit_weight', 'draws', 'seed']
30 missing = set(required)-settings.keys()
```

Read this block:

- Line 25, `fit_dir`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 26, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.

- Line 27, `settings`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 28, `required`: Set this name to a list containing 10 displayed entries.
- Line 30, `missing`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.

Source lines 31–32:

```
31 if missing:
32     raise ValueError(f'Saved fit lacks reconstruction settings: {sorted(missing)}')
```

Read this block:

- Line 31: test `missing`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 32: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `ValueError`: Construct an exception describing why this operation cannot proceed normally.
- `sorted`: Return entries in deterministic sorted order, with optional key and reverse direction.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Source lines 33–34:

```
33 settings['model'] = fit['model']
34 settings['output'] = fit_dir
```

Read this block:

- Line 33, `settings['model']`: Store the entry/slice selected by `fit['model']`; brackets select from the object to their left in the selected entry/column.
- Line 34, `settings['output']`: Store the object currently named `fit_dir`; plain assignment does not copy it in the selected entry/column.

Source lines 35–39:

```
35 for key in ['sample', 'flavors', 'products_extra']:
36     if settings.get(key) is not None:
37         settings[key] = Path(settings[key])
38     else:
39         settings[key] = None
```

Read this block:

- Line 35: for each entry yielded by `['sample', 'flavors', 'products_extra']`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 36: test `settings.get(key) is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 37, `settings[key]`: Store the result of calling `Path(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 39, `settings[key]`: Store the literal value `None` in the selected entry/column.

Syntax and calls used here:

- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 40–41:

```
40 if sample is not None:
41     settings['sample'] = Path(sample)
```

Read this block:

- Line 40: test `sample is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 41, `settings['sample']`: Store the result of calling `Path(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 42–43:

```
42 if flavors is not None:
43     settings['flavors'] = Path(flavors)
```

Read this block:

- Line 42: test `flavors is not None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 43, `settings['flavors']`: Store the result of calling `Path(...)`; the arguments and preceding method chain are shown above in the selected entry/column.

Source lines 44–45:

```
44 args = SimpleNamespace(**settings)
45 frame = prepare_data(args)
```

Read this block:

- Line 44, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 45, `frame`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `prepare_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 46–48:

```
46 if not args.no_price_control:
47     # Reconstruct the original first stage; preserve saved inference artifacts.
48     frame, _ = add_price_controls(frame)
```

Read this block:

- Line 46: test `not args.no_price_control`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 48, `(frame, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 49–49:

```
49 data = make_arrays(frame, args.capacity)
```

Read this block:

- Line 49, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 50–51:

```
50 if len(frame) != fit['weeks'] or data['n_households'] != fit['households']:
51     raise ValueError('Regenerated household/week totals do not match the saved fit; no CSV written.')
```

Read this block:

- Line 50: test `len(frame) != fit['weeks'] or data['n_households'] != fit['households']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 51: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `len`: Count elements/rows rather than summing their numerical values.

Source lines 52–57:

```
52 estimates = pd.read_csv(fit_dir / 'point_estimates.csv')
53 parameters = estimates.set_index('parameter').estimate
54 names = parameter_names(args.model, data['outside'], data['control'])
55 theta = np.array([parameters[name] for name in names])
56 draws = make_draws(args.draws, args.seed, args.model)
57 nll = -float(household_loglike(theta, data, draws, args.model).sum())
```

Read this block:

- Line 52, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 53, `parameters`: Set this name to the attribute `estimates.set_index('parameter').estimate` of the object before the dot.
- Line 54, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 55, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 56, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 57, `nll`: Set this name to the negated or sign-adjusted expression `-float(household_loglike(theta, data, draws, args.model)....`.

Syntax and calls used here:

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 58–59:

```
58 if not np.isfinite(nll) or not np.isclose(nll, fit['nll'], rtol=1e-9, atol=1e-4):
59     raise ValueError(f'Regenerated likelihood {nll} differs from saved {fit["nll"]}; no CSV written.')
```

Read this block:

- Line 58: test `not np.isfinite(nll) or not np.isclose(nll, fit['nll'], rtol=1e-09, atol=0.0001)`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 59: raise the displayed exception, stopping normal flow because continuing would produce misleading or unusable results. A surrounding matching except may handle it.

Syntax and calls used here:

- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.isclose`: Create approximate-equality flags rather than requiring exact floating-point representation.

Source lines 60–60:

```
60 target = fit_dir / 'weekly_inputs.csv'
```

Read this block:

- Line 60, `target`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 62–63:

```
62 with tempfile.NamedTemporaryFile(dir=fit_dir, prefix='.weekly_inputs_', suffix='.csv', delete=False) as handle:
63     temporary = Path(handle.name)
```

Read this block:

- Line 62: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 63, `temporary`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.

Source lines 64–68:

```
64 try:
65     frame.to_csv(temporary, index=False)
66     temporary.replace(target)
67 finally:
68     temporary.unlink(missing_ok=True)
```

Read this block:

- Line 64: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 65: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 66: call `temporary.replace` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 68: call `temporary.unlink` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.replace`: Replace matched values/text; pandas values, regex string namespaces, and Python strings have different replacement semantics.

Source lines 69–73:

```
69 report = dict(households=data['n_households'], weeks=len(frame), reconstructed_nll=nll,
70               saved_nll=fit['nll'], model=args.model, sample=str(args.sample),
71               demand_refitted=False, price_control_refitted=not args.no_price_control)
72 (fit_dir / 'weekly_inputs_regeneration.json').write_text(json.dumps(report, indent=2)+'\n')
73 print(f'Regenerated {target}; household/week totals and saved likelihood verified.', flush=True)
```

Read this block:

- Line 69, `report`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 72: call `.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 73: call `print` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `print`: Display/log a value; this differs from returning it to another calculation. flush=True sends progress promptly.

Source lines 74–74:

```
74 return report
```

Read this block:

- Line 74: return the object currently named `report`; plain assignment does not copy it. This ends the call.

main

Original function: `main`. Source begins at line 79.

Parses a fit directory and optional moved input paths, then calls server-side reconstruction. Missing required paths stop before processing purchases. No optimizer is invoked.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 79–79:

```
79 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 80–84:

```
80 parser = argparse.ArgumentParser(description=__doc__)
81 parser.add_argument('--fit-dir', type=Path, default=fit_dir)
82 parser.add_argument('--sample', type=Path, default=sample, help='Override saved prepared-sample path if moved.')
83 parser.add_argument('--flavors', type=Path, default=flavors, help='Override saved flavor-reference path if moved.')
84 args = parser.parse_args()
```

Read this block:

- Line 80, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 81: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 82: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 83: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 84, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 85–86:

```
85 if args.fit_dir is None:
86     parser.error('Set fit_dir above or supply --fit-dir on the server.')
```

Read this block:

- Line 85: test `args.fit_dir is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 86: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 87–87:

```
87 regenerate(args.fit_dir, args.sample, args.flavors)
```

Read this block:

- Line 87: call `regenerate` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `regenerate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/demand/history_distributions.py

Small saved-fit distribution entry point using the export calculation in the readable estimator. Edit fit_dir or use –fit-dir. It exports population density/CDF grids, quantiles/sign shares and plots without reading a weekly household table.

Module setup and direct-execution blocks

Source lines 1–5:

```
"""Export distributions from an existing split fit; edit Parameters or use flags."""
from pathlib import Path
import argparse
import json
import pandas as pd
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 3: import `argparse`. An as-name is a convenient local alias, not another library.
- Line 4: import `json`. An as-name is a convenient local alias, not another library.
- Line 5: import `pandas as pd`. An as-name is a convenient local alias, not another library.

Source lines 6–9:

```
try:
    from .estimation import export_history_distributions
except ImportError:
    from estimation import export_history_distributions
```

Read this block:

- Line 6: try the indented operation; matching except blocks handle specified exceptions. Unhandled error types still propagate instead of being silently treated as success.
- Line 7: from `estimation`, import `export_history_distributions` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: catch `ImportError` from the preceding try; the optional as-name holds the error object for logging.
- Line 9: from `estimation`, import `export_history_distributions` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 12–13:

```
fit_dir = None
output = None  # None writes alongside the saved fit.
```

Read this block:

- Line 12, `fit_dir`: Set this name to the literal value `None`.
- Line 13, `output`: Set this name to the literal value `None`.

Source lines 29–30:

```
if __name__ == '__main__':
    main()
```

Read this block:

- Line 29: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 30: call `main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `main`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

main

Original function: `main`. Source begins at line 16.

Reads saved fit metadata and coefficient estimates, verifies a split model, and exports population marginals. It does not read household histories or re-estimate demand.

In this readable version the settings record comes from editable Python variables at the top of the file; the algorithm is implemented here without importing final_draft.

Definition and stated intent:

Source lines 16–16:

```
16 def main():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 17–20:

```
17 parser = argparse.ArgumentParser(description=__doc__)
18 parser.add_argument('--fit-dir', type=Path, default=fit_dir)
19 parser.add_argument('--output', type=Path, default=output)
20 args = parser.parse_args()
```

Read this block:

- Line 17, `parser`: Set this name to the result of calling `argparse.ArgumentParser(...)`; the arguments and preceding method chain are shown above.
- Line 18: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 19: call `parser.add_argument` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 20, `args`: Set this name to the result of calling `parser.parse_args(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.ArgumentParser`: Create the command-line parser object that will register and validate options.
- `.add_argument`: Register a command-line option and its type/default/choices/flag behavior before parsing.
- `.parse_args`: Read command-line arguments into a Namespace of named attributes.

Source lines 21–22:

```
21 if args.fit_dir is None:
22     parser.error('Set fit_dir above or supply --fit-dir.')
```

Read this block:

- Line 21: test `args.fit_dir is None`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 22: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.error`: Report invalid command-line input and exit through argparse; it is not a recoverable data-cleaning step here.

Source lines 23–23:

```
23 fit = json.loads((args.fit_dir/'fit.json').read_text())
```

Read this block:

- Line 23, `fit`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.

Source lines 24–25:

```
if fit['model'] not in ['separate_history', 'heterogeneous_history']:
    parser.error('Requires a split-history fit')
```

Read this block:

- Line 24: test `fit['model'] not in ['separate_history', 'heterogeneous_history']`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 25: call `parser.error` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 26–27:

```
export_history_distributions(pd.read_csv(args.fit_dir/'point_estimates.csv'),
                             args.output or args.fit_dir, fit.get('converged', False))
```

Read this block:

- Line 26: call `export_history_distributions` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `export_history_distributions`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

Tests and verification: worked calling examples

data_code/test_summ_stats_memory.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Synthetic checks for sparse summaries and exact results across batch sizes."""
import contextlib
import io
from pathlib import Path
import tempfile
from types import SimpleNamespace
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `contextlib`. An as-name is a convenient local alias, not another library.
- Line 3: import `io`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 6: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–12:

```
import unittest

import pandas as pd
import polars as pl

import summ_stats as summary
```

Read this block:

- Line 7: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 10: import `polars as pl`. An as-name is a convenient local alias, not another library.
- Line 12: import `summ_stats as summary`. An as-name is a convenient local alias, not another library.

Source lines 15–15:

```
15 class MemorySummaryChecks(unittest.TestCase):
```

Read this block:

- Line 15: define class `MemorySummaryChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 141–142:

```
141 if __name__ == "__main__":
142     unittest.main()
```

Read this block:

- Line 141: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 142: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

MemorySummaryChecks.test single person records precede repeat week rule

Original function: `MemorySummaryChecks.test_single_person_records_precede_repeat_week_rule`. Source begins at line 16.

Checks record-level size-one filtering occurs before requiring repeated weeks; otherwise a household can qualify using excluded purchases. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 16–16:

```
16 def test_single_person_records_precede_repeat_week_rule(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 17–25:

```
17 source = pl.DataFrame({
18     "household_code": [1, 1, 2, 2, 3, 3, 4, 4],
19     "purchase_date": ["2010-01-02", "2010-01-09"] * 4,
20     "household_size": ["1", "2", "1", "1", None, "1", "bad", "2"],
21 }).lazy().with_columns(pl.col("purchase_date").str.to_date())
22 restricted = summary.single_person_purchases(source)
23 self.assertEqual(restricted.collect().height, 4)
24 repeated = summary.repeated_yogurt_households(restricted).collect()
25 self.assertEqual(repeated["household_code"].unique().to_list(), [2])
```

Read this block:

- Line 17, `source`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 22, `restricted`: Set this name to the result of calling `summary.single_person_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 23: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 24, `repeated`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 25: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.

Source lines 26–27:

```
26 with self.assertRaisesRegex(ValueError, "require household_size"):
27     summary.single_person_purchases(source.drop("household_size"))
```

Read this block:

- Line 26: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 27: call `summary.single_person_purchases` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

MemorySummaryChecks.test batches preserve global statistics

Original function: `MemorySummaryChecks.test_batches_preserve_global_statistics`. Source begins at line 29.

What this test establishes. Runs a small complete summary pipeline with one and three partitions, sorts outputs, and compares their actual contents. It verifies partitioning changes execution/storage rather than statistical results and preserves original metadata in the prepared sample.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 29–29:

```
29 def test_batches_preserve_global_statistics(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 30–71:

```
30 with tempfile.TemporaryDirectory() as directory, contextlib.redirect_stdout(io.StringIO()):
31     root = Path(directory)
32     source = pl.DataFrame({
33         "household_code": [1, 1, 1, 1, 2, 2, 3], "panel_year": [2010] * 7,
34         "purchase_date": ["2010-01-02", "2010-01-09", "2010-01-09", "2010-01-23", "2010-01-02", "2010-01-09",
            ↪  "2010-01-02"],
35         "trip_code_uc": [1, 2, 2, 3, 4, 5, 6], "upc": [1, 1, 2, 3, 2, 3, 1],
```

```
        "quantity": [2, 1, 1, 3, 1, 2, 1], "race": [1, 1, 1, 1, 2, 2, 3],
        "hispanic_origin": [2, 2, 2, 2, 1, 1, 2], "household_income": [5, 5, 5, 5, 8, 8, 9],
        "female_head_age": [3, 3, 3, 3, 0, 0, 4], "male_head_age": [4, 4, 4, 4, 2, 2, 5],
        "household_size": [1] * 7, "store_code_uc": [100] * 7, "price": [1.] * 7,
        "original_metadata": ["preserved"] * 7,
    })
    for name in ["merged_panel", "yogurt_estimation_sample"]:
        source.write_parquet(root / f"{name}.parquet")
    reference = root / "reference.csv"
    reference.write_text("upc,flavor_descr,module\n001,vanilla,3612\n002,strawberry,3612\n003,plain,3612\n
↪   004,unused,3612\n")
    for batches in [1, 3]:
        out, work = root / f"out{batches}", root / f"work{batches}"
        out.mkdir()
        work.mkdir()
        args = SimpleNamespace(data_dir=root, flavors=reference, products_extra=root / "absent",
                               first_year=2006, last_year=2015, household_batches=batches, no_figures=True)
        summary.run(args, out, work)
    for name, keys in [
        ("full_panel_households", ["household_code"]), ("matched_yogurt_households", ["household_code"]),
        ("yogurt_household_flavors", ["household_code"]), ("yogurt_dynamics", ["metric"]),
        ("full_panel_overview", ["metric"]), ("matched_yogurt_overview", ["metric"]),
        ("yogurt_flavor_shares", ["flavor"]), ("full_panel_year_coverage", ["panel_year"]),
    ]:
        left = pd.read_csv(root / "out1" / f"{name}.csv").sort_values(keys).reset_index(drop=True)
        right = pd.read_csv(root / "out3" / f"{name}.csv").sort_values(keys).reset_index(drop=True)
        pd.testing.assert_frame_equal(left, right)
    prepared = pl.read_parquet(root / "out3/yogurt_estimation_sample.parquet")
    self.assertEqual(prepared.height, 6)
    self.assertTrue(set(source.columns).issubset(prepared.columns))
    self.assertEqual(set(prepared["household_code"].to_list()), {1, 2})
    self.assertEqual(prepared["original_metadata"].unique().to_list(), ["preserved"])
    self.assertEqual(set(prepared["upc"].to_list()), {"1", "2", "3"})
    self.assertEqual(prepared["flavor"].null_count(), 0)
    hh = pd.read_csv(root / "out3/yogurt_household_flavors.csv").set_index("household_code")
    self.assertEqual(set(hh.index), {1, 2})
    self.assertAlmostEqual(hh.loc[1, "flavor_HHI"], 19 / 49)
```

Source lines 72–97:

```
self.assertAlmostEqual(hh.loc[2, "flavor_HHI"], 5 / 9)
metrics = pd.read_csv(root / "out3/yogurt_dynamics.csv").set_index("metric")["value"]
self.assertEqual(metrics["Eligible adjacent single-flavor week pairs"], 1)
self.assertEqual(metrics["Switch rate in eligible pairs (%)"], 100)
self.assertEqual(metrics["Median gap between purchase weeks"], 1)
# Price filtering precedes the two-week household rule.
unpriced = source.with_columns(pl.when(
    ((pl.col("household_code") == 2) & (pl.col("purchase_date") == "2010-01-09"))
    | ((pl.col("household_code") == 1) & (pl.col("purchase_date") == "2010-01-09") & (pl.col("upc") == 1))
    ).then(None).otherwise(pl.col("price")).alias("price"))
unpriced.write_parquet(root / "yogurt_estimation_sample.parquet")
out, work = root / "filtered", root / "filtered_work"
out.mkdir()
work.mkdir()
args.household_batches = 2
summary.run(args, out, work)
filtered = pl.read_parquet(out / "yogurt_estimation_sample.parquet")
self.assertEqual(filtered.height, 3)
self.assertEqual(filtered["household_code"].unique().to_list(), [1])
self.assertTrue(filtered["price"].is_not_null().all())
self.assertEqual(pl.read_csv(out / "yogurt_price_coverage.csv")["purchase_lines_without_price_coverage"][0], 2)
```

```
# Sparse quantity output has one row per observed flavor/week, not
# one column for every flavor in the full reference.
quantities = pl.read_csv(root / "out3/yogurt_weekly_flavor_quantities.csv")
self.assertEqual(quantities.height, 6)
self.assertEqual(set(quantities.columns), {"household_code", "week", "flavor", "quantity"})
```

Read this block:

- Line 30: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 31, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 32, `source`: Set this name to the result of calling `pl.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 42: for each entry yielded by `['merged_panel', 'yogurt_estimation_sample']`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 43: call `source.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 44, `reference`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 45: call `reference.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 46: for each entry yielded by `[1, 3]`, bind/unpack it into `batches` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 47, `(out, work)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 48: call `out.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49: call `work.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 50, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 52: call `summary.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 53: for each entry yielded by `[('full_panel_households', ['household_code']), ('matched_yogu...`, bind/unpack it into `(name, keys)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 59, `left`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 60, `right`: Set this name to the result of calling `.reset_index(...)`; the arguments and preceding method chain are shown above.
- Line 61: call `pd.testing.assert_frame_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 62, `prepared`: Set this name to the result of calling `pl.read_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 63: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 66: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 67: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 68: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 69, `hh`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 70: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 71: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 72: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 73, `metrics`: Set this name to the entry/slice selected by `pd.read_csv(root / 'out3/yogurt_dynamics.csv').set_index(...`; brackets select from the object to their left.
- Line 74: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 75: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 76: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 78, `unpriced`: Set this name to the result of calling `source.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 82: call `unpriced.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 83, `(out, work)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 84: call `out.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 85: call `work.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 86, `args.household_batches`: Set this name to the literal value `2`.
- Line 87: call `summary.run` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88, `filtered`: Set this name to the result of calling `pl.read_parquet(...)`; the arguments and preceding method chain are shown above.
- Line 89: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 92: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95, `quantities`: Set this name to the result of calling `pl.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 96: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 97: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.redirect_stdout`: Temporarily capture printed output, keeping tests quiet without changing the tested computation.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.read_parquet`: Read Parquet into an eager table, rather than retaining only a deferred plan.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.write_parquet`: Persist an eager table as Parquet, often used for small synthetic fixtures.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `.run`: For subprocess.run, execute an argument-list command; check=True raises if its exit status is nonzero.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.StringIO`: Create an in-memory text buffer instead of writing captured output to a real file.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.assert_frame_equal`: Compare pandas tables including values, labels, and dtype conventions rather than only row counts.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.sort_values`: Order pandas rows by values; ascending can supply one direction for each listed sort key.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.is_not_null`: Create a Polars nonnull mask; additional range checks can still be needed.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The | operator accepts either elementwise condition; Python's scalar or is generally unsuitable for an entire table column.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

MemorySummaryChecks.test demographics match previous fieldwise sort

Original function: `MemorySummaryChecks.test_demographics_match_previous_fieldwise_sort`. Source begins at line 99.

What this test establishes. Compares the optimized shared demographic-history sort to separate field-by-field selection on a synthetic example. This checks that a memory/runtime improvement preserves earliest-nonmissing semantics.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 99–101:

```
99 def test_demographics_match_previous_fieldwise_sort(self):
100     # Unsorted histories, missing fields, conflicting same-date values and
101     # repeated purchase lines must preserve the earliest nonmissing values.
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 102–116:

```
source = pl.DataFrame({
    "household_code": [1, 2, 1, 1, 1, 2],
    "panel_year": [2011, 2010, 2010, 2010, 2010, 2011],
    "purchase_date": ["2011-01-01", "2010-01-02", "2010-01-02", "2010-01-02", "2010-01-02", "2011-01-01"],
    "trip_code_uc": [1, 2, 3, 3, 3, 4], "upc": [1, 2, 3, 3, 3, 4],
    "quantity": [1.] * 6,
}).with_columns(pl.col("purchase_date").str.to_date())
fields = ["race", "income", "hispanic_origin", "household_size", "head_age",
          "female_head_age", "male_head_age", "dma_cd"]
source = source.with_columns([pl.Series(c, ["9", None, None, "2", "3", "7"]) for c in fields])
expected = source.lazy().group_by("household_code").agg([
    pl.col(c).sort_by(["panel_year", "purchase_date"]).drop_nulls().first().alias(c)
    for c in fields
]).collect().sort("household_code")
actual = summary.household_summary(source.lazy()).collect().sort("household_code")
```

Read this block:

- Line 102, `source`: Set this name to the result of calling `.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 109, `fields`: Set this name to a list containing 8 displayed entries.
- Line 111, `source`: Set this name to the result of calling `source.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 112, `expected`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 116, `actual`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.to_date`: Parse text into calendar-date values before weekday extraction and date arithmetic.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.agg`: Evaluate aggregations within the current groups; the selected expressions define output columns.
- `.group_by`: Partition Polars rows by key expressions before evaluating per-group aggregations.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.lazy`: Wrap an eager Polars table as a lazy query so later operations can be planned together.
- `.first`: Select the first value in the current expression/group; preceding ordering and null filtering determine which one.
- `.drop_nulls`: Remove null values from the Polars expression before selecting a first observed value.
- `.sort_by`: Order values within an expression/group by another column before selecting a first value.

Source lines 117–118:

```
self.assertEqual(actual.select(["household_code"] + fields).to_dicts(), expected.to_dicts())
self.assertEqual(actual["purchase_lines"].to_list(), [4, 2])
```

Read this block:

- Line 117: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 118: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.

- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.

Source lines 119–126:

```
119 with tempfile.TemporaryDirectory() as directory:
120     parts = summary.partition_purchases(source.lazy(), Path(directory), "check", 16)
121     restored = pl.concat(parts).collect()
122     self.assertEqual(restored.height, source.height)
123     counts = restored.group_by("household_code").len().sort("household_code")
124     self.assertEqual(counts["len"].to_list(), [4, 2])
125     batched = pl.concat([summary.household_summary(part) for part in parts]).collect().sort("household_code")
126     self.assertEqual(batched.to_dicts(), actual.to_dicts())
```

Read this block:

- Line 119: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 120, `parts`: Set this name to the result of calling `summary.partition_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 121, `restored`: Set this name to the result of calling `.collect(...)`; the arguments and preceding method chain are shown above.
- Line 122: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 123, `counts`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 124: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 125, `batched`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 126: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

MemorySummaryChecks.test recovery preserves existing labels and uses earliest year

Original function: `MemorySummaryChecks.test_recovery_preserves_existing_labels_and_uses_earliest_year`. Source begins at line 128.

What this test establishes. Constructs reference and annual metadata cases to check that existing labels are retained and absent UPCs take earliest usable metadata. It verifies a substantive recovery precedence rule.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 128–128:

```
128 def test_recovery_preserves_existing_labels_and_uses_earliest_year(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 129–138:

```
129 with tempfile.TemporaryDirectory() as directory, contextlib.redirect_stdout(io.StringIO()):
130     root = Path(directory)
131     ref, extra, output = root / "ref.csv", root / "extra.parquet", root / "result.csv"
132     ref.write_text("upc,flavor_descr\n001,vanilla\n002,NA\n")
133     pl.DataFrame({"upc": [1, 2, 3, 3, 4], "flavor_descr": ["plain", "berry", "cherry", "strawberry", None],
134                   "panel_year": [2006, 2006, 2007, 2006, 2006]}).write_parquet(extra)
135     summary.recover_flavor_reference(ref, extra, output)
136     result = summary.flavor_reference(output, detailed=True).sort("upc")
137     self.assertEqual(result.to_dict(as_series=False),
138                      {"upc": ["1", "2", "3"], "flavor": ["vanilla", "unknown", "strawberry"]})
```

Read this block:

- Line 129: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 130, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 131, `(ref, extra, output)`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 132: call `ref.write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133: call `.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 135: call `summary.recover_flavor_reference` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 136, `result`: Set this name to the result of calling `.sort(...)`; the arguments and preceding method chain are shown above.
- Line 137: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.redirect_stdout`: Temporarily capture printed output, keeping tests quiet without changing the tested computation.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.write_parquet`: Persist an eager table as Parquet, often used for small synthetic fixtures.
- `.sort`: Order values/rows; sorting before first-row selection determines which record wins a tie.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.StringIO`: Create an in-memory text buffer instead of writing captured output to a real file.
- `.to_dict`: Convert table contents to a Python dictionary; output orientation/options determine its nested layout.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est__code/test__est.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Small checks that need no Nielsen data: python3 -m unittest -v test_est.py."""
import unittest
import tempfile
from pathlib import Path
from types import SimpleNamespace
import polars as pl
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 7–10:

```
import numpy as np
import pandas as pd
from scipy.special import logsumexp
from est import read_flavors, prepare_data, fit_model, add_stocks, make_arrays, household_loglike, make_draws,
    parameter_names, demand_objective_gradient
```

Read this block:

- Line 7: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 8: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 9: from `scipy.special`, import `logsumexp` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: from `est`, import `read_flavors, prepare_data, fit_model, add_stocks, make_arrays, household_loglike, make_draws, parameter_names, demand_objective_gradient` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 13–13:

```
class EstimationChecks(unittest.TestCase):
```

Read this block:

- Line 13: define class `EstimationChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 270–271:

```
if __name__ == "__main__":
    unittest.main()
```

Read this block:

- Line 270: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 271: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

EstimationChecks.frame

Original function: `EstimationChecks.frame`. Source begins at line 14.

This fixture method returns a tiny three-week pandas table. It includes a two-week gap, mixed counts, all three prices, and outside counts so history, menu, and likelihood tests can reuse the same explicit example. `self.frame()` returns a fresh table each time, letting one test modify it without contaminating another. A shared global mutable table would make results depend on test order.

Definition and stated intent:

Source lines 14–14:

```
14 def frame(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 15–20:

```
15 return pd.DataFrame({
16     "household_code": [10, 10, 10],
17     "week": pd.to_datetime(["2010-01-02", "2010-01-09", "2010-01-23"]),
18     "other": [2., 0., 1.], "berry": [0., 2., 1.], "plain": [0., 0., 0.],
19     "price_other": [1., 1.2, 1.1], "price_berry": [1.2, 1., 1.3],
20     "price_plain": [1.5, 1.4, 1.3], "outside": [5., 5., 5.]})
```

Read this block:

- Line 15: return the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.

EstimationChecks.test history excludes current counts and accumulates all occasions

Original function: `EstimationChecks.test_history_excludes_current_counts_and_accumulates_all_occasions`. Source begins at line 22.

What this test establishes. Checks pre-choice timing: changing a current basket must not rewrite the stock already available for that choice. It compares explicit stock vectors, not just output dimensions.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 22–22:

```
22 def test_history_excludes_current_counts_and_accumulates_all_occasions(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 23–28:

```
23 frame = add_stocks(self.frame())
24 stock = frame[["stock_other", "stock_berry", "stock_plain"]]
25 np.testing.assert_allclose(stock.iloc[0], [0., 0., 0.])
```

```
26 np.testing.assert_allclose(stock.iloc[1], [1., 0., 0.])
27 np.testing.assert_allclose(stock.iloc[2], [1., 1., 0.])
28 changed = self.frame()
```

Read this block:

- Line 23, `frame`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 24, `stock`: Set this name to the entry/slice selected by `frame[['stock_other', 'stock_berry', 'stock_plain']]`; brackets select from the object to their left.
- Line 25: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 26: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 27: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 28, `changed`: Set this name to the result of calling `self.frame(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 29–31:

```
29 changed.loc[2, "plain"] = 100
30 np.testing.assert_allclose(add_stocks(changed).filter(like="stock_"),
31                            frame.filter(like="stock_"))
```

Read this block:

- Line 29, `changed.loc[2, 'plain']`: Store the literal value `100` in the selected entry/column.
- Line 30: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test xi sign and unlimited history

Original function: `EstimationChecks.test_xi_sign_and_unlimited_history`. Source begins at line 33.

What this test establishes. Checks numerical Xi signs and persistence across long gaps. The explicit -.7/.3 expectations distinguish own-stock penalties from recent-other rewards.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 33–33:

```
def test_xi_sign_and_unlimited_history(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 34–40:

```
frame = add_stocks(self.frame(), .7)
self.assertAlmostEqual(frame.loc[1, "xi_other"], -.7)
self.assertAlmostEqual(frame.loc[1, "xi_berry"], .3)
# Time gaps do not change occasion-time histories when delta=1.
old = self.frame()
old.loc[2, "week"] = pd.Timestamp("2011-04-03")
old = add_stocks(old)
```

Read this block:

- Line 34, `frame`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 35: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38, `old`: Set this name to the result of calling `self.frame(...)`; the arguments and preceding method chain are shown above.
- Line 39, `old.loc[2, 'week']`: Store the result of calling `pd.Timestamp(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 40, `old`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Source lines 41–43:

```
np.testing.assert_allclose(old.filter(like="xi_"), frame.filter(like="xi_"))
self.assertEqual(frame.loc[2, "novelty_other"], 0.)
self.assertAlmostEqual(frame.loc[2, "xi_other"], -.7)
```

Read this block:

- Line 41: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test parquet pipeline and fit

Original function: `EstimationChecks.test_parquet_pipeline_and_fit`. Source begins at line 45.

What this test establishes. Builds a tiny real Parquet input and runs preparation plus a deliberately short fit. It checks all-store counts, exposure-store menus, missing alternatives, exported files, and own-DMA exclusion. Its low iteration limit is a smoke check, not proof of statistical convergence.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 45–45:

```
45 def test_parquet_pipeline_and_fit(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 46–87:

```
46 with tempfile.TemporaryDirectory() as temp:
47     root = Path(temp)
48     rows = []
49     def append(household, week, upc, store, dma, price, quantity=1):
50         rows.append(dict(household_code=household, household_size=1 if household in [10, 40] else 2,
    ↪  panel_year=2010,
51                          purchase_date=str(pd.to_datetime(str(week), format="%Y%m%d").date()),
52                          trip_code_uc=len(rows) + 1, upc=str(upc), store_code_uc=store,
53                          quantity=quantity, price=float(price), dma_cd=dma,
54                          flavor={1: "other", 2: "berry", 3: "plain"}[upc]))
55     append(10, 20100102, 1, 100, 1, 3, 2)
56     append(10, 20100102, 2, 200, 1, 4)
57     append(10, 20100109, 2, 100, 1, 4, 2)
58     for week in [20100102, 20100109]:
59         for upc in [1, 2, 3]:
60             append(20, week, upc, 200, 2, 2 + upc)
61             append(30, week, upc, 300, 3, 6 + upc)
62             append(40, week, upc, 100, 1, 2 + upc)
63     # Repeat a cheap quote: it must not overweight the overall IV.
64     append(20, 20100102, 1, 200, 2, 3)
65     sample = root / "prepared.parquet"
66     original = pl.DataFrame(rows)
67     original.write_parquet(sample)
68     args = SimpleNamespace(
69         sample=sample, flavors=root / "not_needed.csv", rms=root / "does_not_exist.parquet",
70         first_year=2010, last_year=2010, max_households=1, habit_weight=.7,
71         capacity=0, no_price_control=True, household_dma_column="dma_cd", se_method="none",
72         model="heterogeneous", draws=8, starts=1, maxiter=5, seed=306, output=root / "output")
73     # The limit is applied after restricting demand to single-person records.
74     mixed = original.with_columns(pl.when(pl.col("household_code") == 10)
75         .then(2).otherwise(pl.col("household_size")).alias("household_size"))
76     mixed.write_parquet(sample)
77     self.assertEqual(prepare_data(args).household_code.unique().tolist(), [40])
78     original.drop("household_size").write_parquet(sample)
79     with self.assertRaisesRegex(ValueError, "requires household_size"):
80         prepare_data(args)
81     original.with_columns(pl.lit(None).alias("household_size")).write_parquet(sample)
82     with self.assertRaisesRegex(ValueError, "No purchase lines"):
83         prepare_data(args)
```

```
    original.write_parquet(sample)
    frame = prepare_data(args)
    self.assertEqual(len(frame), 2)
    self.assertEqual(frame.household_code.unique().tolist(), [10])
```

Source lines 88–119:

```
self.assertEqual(frame.loc[0, "other"], 2)
self.assertEqual(frame.loc[0, "berry"], 1)
self.assertEqual(frame.loc[0, "store_code_uc"], 100)
self.assertEqual(frame.loc[0, "price_other"], 3)
np.testing.assert_allclose(frame.filter(like="stock_").iloc[0], 0)
fit = fit_model(make_arrays(frame, 0), args)
self.assertTrue(np.isfinite(fit.fun))
self.assertTrue((args.output / "fit.json").exists())
self.assertTrue((args.output / "gamma_distribution.pdf").exists())
incomplete = original.filter(~((pl.col("purchase_date") == "2010-01-02")
                               & (pl.col("store_code_uc") == 100) & (pl.col("upc") == "3")))
incomplete.write_parquet(sample)
partial = prepare_data(args)
self.assertEqual(len(partial), 2)
self.assertFalse(partial.loc[0, "available_plain"])
self.assertTrue(np.isnan(partial.loc[0, "price_plain"]))
original.write_parquet(sample)
args.no_price_control = False
controlled = prepare_data(args)
# Pool DMA 2's quotes 3,4,5 and DMA 3's 7,8,9; mean = 6.
for name in ["iv_overall", "iv_other", "iv_berry", "iv_plain"]:
    np.testing.assert_allclose(controlled[name], 6.)
# Changing every own-DMA quote cannot change the instrument.
changed = original.with_columns(pl.when(pl.col("dma_cd") == 1)
                                .then(pl.col("price") + 100).otherwise(pl.col("price")).alias("price"))
changed.write_parquet(sample)
# Store 200 is shared across DMAs in this fixture. Update both copies
# of its quoted price to keep local store/week/UPC prices consistent.
changed = changed.with_columns(pl.when(pl.col("store_code_uc") == 200)
                               .then(pl.col("upc").cast(pl.Float64) + 2).otherwise(pl.col("price")).alias("price"))
changed.write_parquet(sample)
np.testing.assert_allclose(prepare_data(args)["iv_overall"], 6.)
```

Read this block:

- Line 46: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 47, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 48, `rows`: Set this name to the list `[]`.
- Line 49: define nested callable `append`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 55: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 57: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 58: for each entry yielded by `[20100102, 20100109]`, bind/unpack it into `week` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 59: for each entry yielded by `[1, 2, 3]`, bind/unpack it into `upc` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 60: call `append` with the displayed arguments. No assignment stores its return here; its documented

side effect or assertion is what matters.
- Line 61: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 62: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: call `append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 65, `sample`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 66, `original`: Set this name to the result of calling `pl.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 67: call `original.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 68, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 74, `mixed`: Set this name to the result of calling `original.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 76: call `mixed.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 77: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 78: call `original.drop('household_size').write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 79: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 80: call `prepare_data` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 81: call `.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 82: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 83: call `prepare_data` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 84: call `original.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 85, `frame`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.
- Line 86: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 87: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 88: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 89: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 92: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 93, `fit`: Set this name to the result of calling `fit_model(...)`; the arguments and preceding method chain are shown above.
- Line 94: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its

documented side effect or assertion is what matters.
- Line 96: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 97, `incomplete`: Set this name to the result of calling `original.filter(...)`; the arguments and preceding method chain are shown above.
- Line 99: call `incomplete.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 100, `partial`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.
- Line 101: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 102: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 103: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 104: call `original.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105, `args.no_price_control`: Set this name to the literal value `False`.
- Line 106, `controlled`: Set this name to the result of calling `prepare_data(...)`; the arguments and preceding method chain are shown above.
- Line 108: for each entry yielded by `['iv_overall', 'iv_other', 'iv_berry', 'iv_plain']`, bind/unpack it into `name` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 109: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 111, `changed`: Set this name to the result of calling `original.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 113: call `changed.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 116, `changed`: Set this name to the result of calling `changed.with_columns(...)`; the arguments and preceding method chain are shown above.
- Line 118: call `changed.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 119: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `append`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.write_parquet`: Persist an eager table as Parquet, often used for small synthetic fixtures.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `.with_columns`: Add or replace named columns while keeping the other columns in the table.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `prepare_data`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `fit_model`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.alias`: Name a Polars expression's result; this often determines which column with_columns replaces.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.
- `len`: Count elements/rows rather than summing their numerical values.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.isnan`: Identify floating-point NaNs; NaN must not be tested with ordinary equality.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.drop`: Remove the specified columns/labels; inspect the receiver because list and table methods have different APIs.
- `.otherwise`: Supply the fallback expression for rows not selected by the preceding condition.
- `.unique`: Retain distinct rows/values; subset and keep determine the keys and duplicate-resolution rule.
- `str`: Convert to text for paths, messages, or serialization.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.then`: Supply the value/expression selected where the preceding Polars condition is true.
- `.lit`: Build a literal-valued Polars expression, distinguishing constant text/numbers from column names.
- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- `.when`: Start a Polars row-wise conditional expression, completed by then and otherwise.
- `.cast`: Request a dtype conversion; strict=False can turn invalid conversions into nulls for later validation.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test parquet pipeline and fit.append

Original function: `EstimationChecks.test_parquet_pipeline_and_fit.append`. Source begins at line 49.

This nested fixture builder appends one purchase dictionary to an enclosing `rows` list. Positional inputs describe household/week/UPC/store/DMA/price; quantity defaults to one. A dictionary lookup turns UPC 1/2/3 into the three flavor names. `len(rows)+1` assigns a fresh trip ID at append time. The helper reduces repetitive fixture text while keeping test prices and households explicit. It mutates the captured list and returns None; callers need not assign its result.

Definition and stated intent:

Source lines 49–49:

```
49 def append(household, week, upc, store, dma, price, quantity=1):
```

`def` creates this callable; the parentheses name its inputs (`household`, `week`, `upc`, `store`, `dma`, `price`, `quantity`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 50–54:

```
50 rows.append(dict(household_code=household, household_size=1 if household in [10, 40] else 2, panel_year=2010,
51                  purchase_date=str(pd.to_datetime(str(week), format="%Y%m%d").date()),
52                  trip_code_uc=len(rows) + 1, upc=str(upc), store_code_uc=store,
53                  quantity=quantity, price=float(price), dma_cd=dma,
54                  flavor={1: "other", 2: "berry", 3: "plain"}[upc]))
```

Read this block:

- Line 50: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `str`: Convert to text for paths, messages, or serialization.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `len`: Count elements/rows rather than summing their numerical values.
- `.to_datetime`: Parse/convert dates; an explicit format avoids interpreting numeric YYYYMMDD values as timestamp offsets.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test unavailable flavor is excluded from logit denominator

Original function: `EstimationChecks.test_unavailable_flavor_is_excluded_from_logit_denominator`. Source begins at line 121.

What this test establishes. Removes plain prices and independently computes the remaining two-flavor likelihood. It also requires a ValueError if an unavailable flavor is actually purchased. This catches treating a missing-price placeholder as an available free good.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 121–121:

```
121 def test_unavailable_flavor_is_excluded_from_logit_denominator(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 122–127:

```
122 frame = self.frame()
123 frame["price_plain"] = np.nan
124 data = make_arrays(add_stocks(frame), 0)
125 theta = np.array([.1, -.2, -.4])
126 actual = household_loglike(theta, data, make_draws(8, 306), "naive")[0]
127 utility = np.array([0., .1]) - np.exp(-.4) * frame[["price_other", "price_berry"]].to_numpy()
```

Read this block:

- Line 122, `frame`: Set this name to the result of calling `self.frame(...)`; the arguments and preceding method chain are shown above.
- Line 123, `frame['price_plain']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 124, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

- Line 125, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 126, `actual`: Set this name to the entry/slice selected by `household_loglike(theta, data, make_draws(8, 306), 'naive...`; brackets select from the object to their left.
- Line 127, `utility`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.

Source lines 128–131:

```
expected = (frame[["other", "berry"]].to_numpy() * (utility - logsumexp(utility, axis=1, keepdims=True))).sum()
self.assertAlmostEqual(actual, expected)
self.assertFalse(data["available"][:, 2].any())
frame.loc[0, "plain"] = 1
```

Read this block:

- Line 128, `expected`: Set this name to the result of calling `.sum(...)`; the arguments and preceding method chain are shown above.
- Line 129: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 130: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 131, `frame.loc[0, 'plain']`: Store the literal value `1` in the selected entry/column.

Syntax and calls used here:

- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.

Source lines 132–133:

```
with self.assertRaisesRegex(ValueError, "Purchased flavors"):
    make_arrays(add_stocks(frame), 0)
```

Read this block:

- Line 132: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 133: call `make_arrays` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test persistent household integration

Original function: `EstimationChecks.test_persistent_household_integration`. Source begins at line 135.

What this test establishes. Independently computes a draw's full household history before averaging draws. This catches a subtle wrong model that integrates each week separately.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 135–135:

```
135 def test_persistent_household_integration(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 136–142:

```
136 data = make_arrays(add_stocks(self.frame()), 7)
137 draws = make_draws(8, 306)
138 names = parameter_names("heterogeneous", True)
139 theta = np.array([.4 if n.startswith("sd_") else .2 for n in names])
140 actual = household_loglike(theta, data, draws, "heterogeneous")[0]
141 # Independently calculate each fixed type's full household history.
142 by_draw = []
```

Read this block:

- Line 136, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 137, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 138, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 139, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 140, `actual`: Set this name to the entry/slice selected by `household_loglike(theta, data, draws, 'heterogeneous')[0]`; brackets select from the object to their left.
- Line 142, `by_draw`: Set this name to the list `[]`.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 143–149:

```
143 for z in draws:
144     alpha = -np.exp(.2 + .4*z[0])
145     gamma = .2 + .4*z[1]
146     utility = np.array([.2, .4, .4]) + alpha*data["prices"] + gamma*data["xi"]
147     utility = np.column_stack([utility, np.zeros(3)])
148     logp = utility - logsumexp(utility, axis=1, keepdims=True)
149     by_draw.append(np.sum(data["counts"]*logp))
```

Read this block:

- Line 143: for each entry yielded by `draws`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 144, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(0.2 + 0.4 * z[0])`.
- Line 145, `gamma`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 146, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 147, `utility`: Set this name to the result of calling `np.column_stack(...)`; the arguments and preceding method chain are shown above.
- Line 148, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 149: call `by_draw.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.column_stack`: Place one-dimensional vectors beside each other as columns of a matrix.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

Source lines 150–150:

```
150 self.assertAlmostEqual(actual, logsumexp(by_draw)-np.log(8))
```

Read this block:

- Line 150: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test separate history nests combined index

Original function: `EstimationChecks.test_separate_history_nests_combined_index`. Source begins at line 152.

Sets b_N=(1-lambda)*gamma and b_S=-lambda*gamma and verifies the split likelihood nests the old combined index under that restriction. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 152–152:

```
152 def test_separate_history_nests_combined_index(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 153–153:

```
153 data = make_arrays(add_stocks(self.frame(), .7), 7)
```

Read this block:

- Line 153, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 154–167:

```
154 for control in [False, True]:
155     data["control"] = control
156     data["cf"] = np.full_like(data["prices"], .1)
157     old_names = parameter_names("heterogeneous", True, control)
158     values = dict.fromkeys(old_names, .2)
159     values["sd_gamma"] = 0.
160     values["mean_gamma"] = -.4
161     separate = dict(values, mean_novelty=.3*values["mean_gamma"],
162                     mean_own_stock=-.7*values["mean_gamma"])
163     draws = make_draws(8, 306)
164     old = household_loglike([values[n] for n in old_names], data, draws, "heterogeneous")
165     new = household_loglike([separate[n] for n in parameter_names("separate_history", True, control)],
166                             data, draws, "separate_history")
167     np.testing.assert_allclose(old, new, atol=1e-12)
```

Read this block:

- Line 154: for each entry yielded by `[False, True]`, bind/unpack it into `control` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 155, `data['control']`: Store the object currently named `control`; plain assignment does not copy it in the selected entry/column.
- Line 156, `data['cf']`: Store the result of calling `np.full_like(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 157, `old_names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.

- Line 158, `values`: Set this name to the result of calling `dict.fromkeys(...)`; the arguments and preceding method chain are shown above.
- Line 159, `values['sd_gamma']`: Store the literal value `0.0` in the selected entry/column.
- Line 160, `values['mean_gamma']`: Store the negated or sign-adjusted expression `-0.4` in the selected entry/column.
- Line 161, `separate`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 163, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 164, `old`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 165, `new`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 167: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test no control ignores residuals but keeps price

Original function: `EstimationChecks.test_no_control_ignores_residuals_but_keeps_price`. Source begins at line 169.

Checks disabling the control removes residual utility while retaining price sensitivity. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 169–169:

```
169 def test_no_control_ignores_residuals_but_keeps_price(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 170–170:

```
170 data = make_arrays(add_stocks(self.frame()), 7)
```

Read this block:

- Line 170, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 171–179:

```
171 for model in ["heterogeneous", "separate_history", "heterogeneous_history"]:
172     names = parameter_names(model, True, False)
173     self.assertNotIn("price_control", names)
174     self.assertIn("mu_log_minus_alpha", names)
175     theta = np.full(len(names), .2)
176     draws = make_draws(8, 306, model)
177     before = household_loglike(theta, data, draws, model)
178     np.testing.assert_allclose(before, household_loglike(theta, dict(data, cf=data["cf"]+100), draws, model))
179     self.assertFalse(np.allclose(before, household_loglike(theta, dict(data, prices=data["prices"]+1), draws,
        ↪  model)))
```

Read this block:

- Line 171: for each entry yielded by `['heterogeneous', 'separate_history', 'heterogeneous_history']`, bind/unpack it into `model` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 172, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 173: call `self.assertNotIn` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 174: call `self.assertIn` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 175, `theta`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 176, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 177, `before`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 178: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 179: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `len`: Count elements/rows rather than summing their numerical values.
- `.allclose`: Check approximate numerical equality with tolerances, appropriate for floating-point arrays.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test analytic gradient

Original function: `EstimationChecks.test_analytic_gradient`. Source begins at line 181.

What this test establishes. Perturbs each coefficient and compares centered finite-difference objective slopes with the supplied analytic gradient. This checks derivative values rather than just whether optimization returns a result.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 181–181:

```
181 def test_analytic_gradient(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 182–182:

```
182 from scipy.optimize._numdiff import approx_derivative
```

Read this block:

- Line 182: from `scipy.optimize._numdiff`, import `approx_derivative` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 183–193:

```
183 for capacity in [0, 7]:
184     frame = add_stocks(self.frame())
185     frame["cf_other"], frame["cf_berry"], frame["cf_plain"] = .1, -.2, .3
186     data = make_arrays(frame, capacity)
187     for model in ["naive", "homogeneous", "mixed_naive", "heterogeneous", "mixture", "separate_history",
        ↪ "heterogeneous_history"]:
188         theta = np.full(len(parameter_names(model, bool(capacity), True)), .2)
189         draws = make_draws(8, 306, model)
190         value, gradient = demand_objective_gradient(theta, data, draws, model)
191         objective = lambda t: -household_loglike(t, data, draws, model).mean()
192         self.assertAlmostEqual(value, objective(theta))
193         np.testing.assert_allclose(gradient, approx_derivative(objective, theta).ravel(), rtol=1e-5, atol=1e-7)
```

Read this block:

- Line 183: for each entry yielded by `[0, 7]`, bind/unpack it into `capacity` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 184, `frame`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 185, `(frame['cf_other'], frame['cf_berry'], frame['cf_plain'])`: Unpack a tuple containing 3 displayed entries into these targets in order.
- Line 186, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 187: for each entry yielded by `['naive', 'homogeneous', 'mixed_naive', 'heterogeneous', 'mixt...`, bind/unpack it into `model` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 188, `theta`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.

- Line 189, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 190, `(value, gradient)`: Unpack the result of calling `demand_objective_gradient(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 191, `objective`: Set this name to a one-expression callable, with input `t` and returned expression `-household_loglike(t, data, draws, model).mean()`.
- Line 192: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 193: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `demand_objective_gradient`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `len`: Count elements/rows rather than summing their numerical values.
- `objective`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `bool`: Convert scalar truthiness to an explicit Boolean flag.
- `approx_derivative`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test mixture combines household likelihoods and nests normal

Original function: `EstimationChecks.test_mixture_combines_household_likelihoods_and_nests_normal`. Source begins at line 195.

Verifies component mixing happens after whole-household likelihood construction and coincident components recover the single-normal model. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 195–195:

```
195 def test_mixture_combines_household_likelihoods_and_nests_normal(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 196–201:

```
from est import mixture_components
frame = add_stocks(self.frame())
other = frame.copy()
other["household_code"] = 20
other["xi_other"] += .5
data = make_arrays(pd.concat([frame, other], ignore_index=True), 7)
```

Read this block:

- Line 196: from `est`, import `mixture_components` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 197, `frame`: Set this name to the result of calling `add_stocks(...)`; the arguments and preceding method chain are shown above.
- Line 198, `other`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 199, `other['household_code']`: Store the literal value `20` in the selected entry/column.
- Line 200: update `other['xi_other']` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 201, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 202–207:

```
draws = make_draws(16, 306)
names = parameter_names("mixture", True)
theta = np.full(len(names), .2)
theta[names.index("mixture_weight")] = .35
_, _, first, second, weight = mixture_components(theta, data)
l1 = household_loglike(first, data, draws, "heterogeneous")
```

Read this block:

- Line 202, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 203, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 204, `theta`: Set this name to the result of calling `np.full(...)`; the arguments and preceding method chain are shown above.
- Line 205, `theta[names.index('mixture_weight')]`: Store the literal value `0.35` in the selected entry/column.
- Line 206, `(_, _, first, second, weight)`: Unpack the result of calling `mixture_components(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 207, `l1`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `.full`: Allocate an array filled with one value, such as equal histogram weights.
- `len`: Count elements/rows rather than summing their numerical values.
- `mixture_components`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 208–212:

```
l2 = household_loglike(second, data, draws, "heterogeneous")
expected = np.logaddexp(np.log(weight) + l1, np.log1p(-weight) + l2)
np.testing.assert_allclose(household_loglike(theta, data, draws, "mixture"), expected)
theta[names.index("gamma_mean_gap")] = 0.
np.testing.assert_allclose(household_loglike(theta, data, draws, "mixture"), l1)
```

Read this block:

- Line 208, `l2`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 209, `expected`: Set this name to the result of calling `np.logaddexp(...)`; the arguments and preceding method chain are shown above.
- Line 210: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 211, `theta[names.index('gamma_mean_gap')]`: Store the literal value `0.0` in the selected entry/column.
- Line 212: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test mixture fit outputs

Original function: `EstimationChecks.test_mixture_fit_outputs`. Source begins at line 214.

Checks a tiny fitted mixture writes consistent parameter and inference metadata; its empirical content is entirely generated. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 214–214:

```
def test_mixture_fit_outputs(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 215–216:

```
import json
data = make_arrays(add_stocks(self.frame()), 7)
```

Read this block:

- Line 215: import `json`. An as-name is a convenient local alias, not another library.
- Line 216, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 217–226:

```
217 with tempfile.TemporaryDirectory() as directory:
218     args = SimpleNamespace(model="mixture", draws=8, seed=306, starts=2,
219                            maxiter=3, output=Path(directory), se_method="none", capacity=7)
220     fit_model(data, args)
221     estimates = pd.read_csv(args.output / "estimates.csv")
222     self.assertIn("mixture_weight", estimates.parameter.tolist())
223     self.assertTrue((args.output / "gamma_distribution.pdf").exists())
224     report = json.loads((args.output / "fit.json").read_text())
225     self.assertEqual(report["parameters"], len(estimates))
226     self.assertAlmostEqual(report["aic"], 2 * report["nll"] + 2 * len(estimates))
```

Read this block:

- Line 217: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 218, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 220: call `fit_model` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 221, `estimates`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 222: call `self.assertIn` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 223: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 224, `report`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 225: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 226: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `fit_model`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.tolist`: Convert a NumPy/pandas sequence to ordinary Python list values, useful for JSON output or comparisons.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `len`: Count elements/rows rather than summing their numerical values.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing,

or another documented side effect.

EstimationChecks.test four model specifications

Original function: `EstimationChecks.test_four_model_specifications`. Source begins at line 228.

What this test establishes. Checks parameter layouts/likelihood behavior for the four supported specifications. The labels define vector meaning; silently changed ordering would corrupt estimates even when vector lengths still match.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 228–228:

```
228 def test_four_model_specifications(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 229–240:

```
229 for model, gamma, alpha_sd, gamma_sd in [
230     ("naive", False, False, False),
231     ("homogeneous", True, False, False),
232     ("mixed_naive", False, True, False),
233     ("heterogeneous", True, True, True),
234 ]:
235     with self.subTest(model=model):
236         names = parameter_names(model, False)
237         self.assertEqual("mean_gamma" in names, gamma)
238         self.assertEqual("sd_log_minus_alpha" in names, alpha_sd)
239         self.assertEqual("sd_gamma" in names, gamma_sd)
240         self.assertFalse(any("sd_beta" in n for n in names))
```

Read this block:

- Line 229: for each entry yielded by `[('naive', False, False, False), ('homogeneous', True, False, ...`, bind/unpack it into `(model, gamma, alpha_sd, gamma_sd)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 235: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 236, `names`: Set this name to the result of calling `parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 237: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 238: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 239: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 240: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `parameter_names`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `any`: Require at least one condition to be true; this reduces a sequence to one Boolean.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test mixed naive integrates only alpha

Original function: `EstimationChecks.test_mixed_naive_integrates_only_alpha`. Source begins at line 242.

What this test establishes. Checks that mixed-naive choice probabilities vary with alpha draws while excluding a gamma effect. It independently constructs the corresponding integrated likelihood.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 242–242:

```
242 def test_mixed_naive_integrates_only_alpha(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 243–247:

```
243 data = make_arrays(add_stocks(self.frame()), 0)
244 draws = make_draws(8, 306)
245 theta = np.array([.1, -.2, -.4, .6])
246 actual = household_loglike(theta, data, draws, "mixed_naive")
247 independent = []
```

Read this block:

- Line 243, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 244, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 245, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 246, `actual`: Set this name to the result of calling `household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 247, `independent`: Set this name to the list `[]`.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 248–252:

```
248 for z in draws:
249     alpha = -np.exp(-.4 + .6*z[0])
250     utility = np.array([0., .1, -.2]) + alpha*data["prices"]
251     logp = utility - logsumexp(utility, axis=1, keepdims=True)
252     independent.append(np.sum(data["counts"]*logp))
```

Read this block:

- Line 248: for each entry yielded by `draws`, bind/unpack it into `z` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 249, `alpha`: Set this name to the negated or sign-adjusted expression `-np.exp(-0.4 + 0.6 * z[0])`.
- Line 250, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 251, `logp`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 252: call `independent.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `logsumexp`: Compute log(sum(exp(values))) stably; axis selects alternatives, draws, or another reduction dimension.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 253–258:

```
253 np.testing.assert_allclose(actual, [logsumexp(independent)-np.log(len(draws))])
254 changed = dict(data, xi=data["xi"] + 100.)
255 np.testing.assert_allclose(actual, household_loglike(theta, changed, draws, "mixed_naive"))
256 np.testing.assert_allclose(
257     household_loglike(theta[:3], data, draws, "naive"),
258     household_loglike(np.r_[theta[:3], 0.], data, draws, "mixed_naive"))
```

Read this block:

- Line 253: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 254, `changed`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 255: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 256: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `len`: Count elements/rows rather than summing their numerical values.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EstimationChecks.test zero heterogeneity nests homogeneous model

Original function: `EstimationChecks.test_zero_heterogeneity_nests_homogeneous_model`. Source begins at line 260.

What this test establishes. Sets random-coefficient SDs to zero and compares against the common-coefficient model. A nesting identity is a strong check on parameter transforms and integration.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 260–260:

```
260 def test_zero_heterogeneity_nests_homogeneous_model(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 261–267:

```
261 data = make_arrays(add_stocks(self.frame()), 0)
262 fixed = np.array([.1, -.2, -.4, .3])
263 random = np.r_[fixed, np.zeros(2)]
264 draws = make_draws(8, 306)
265 np.testing.assert_allclose(
266     household_loglike(fixed, data, draws, "homogeneous"),
267     household_loglike(random, data, draws, "heterogeneous"))
```

Read this block:

- Line 261, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 262, `fixed`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 263, `random`: Set this name to the entry/slice selected by `np.r_[fixed, np.zeros(2)]`; brackets select from the object to their left.
- Line 264, `draws`: Set this name to the result of calling `make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 265: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_stocks`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `make_draws`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `household_loglike`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est__code/test__hessian__se.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Check information scaling, correlation, and invalid-inference handling."""
2 import tempfile
3 import unittest
4 from pathlib import Path
5 import numpy as np
6 from hessian_se import observed_covariance, hessian_standard_errors
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 3: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 6: from `hessian_se`, import `observed_covariance, hessian_standard_errors` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 9–9:

```
9 class HessianChecks(unittest.TestCase):
```

Read this block:

- Line 9: define class `HessianChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

HessianChecks.test correlated information and household scaling

Original function: `HessianChecks.test_correlated_information_and_household_scaling`. Source begins at line 10.

Uses known quadratic curvature to check inversion and the number-of-households scaling in Hessian covariance. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 10–10:

```
10 def test_correlated_information_and_household_scaling(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 11–15:

```
information = np.array([[4., 1.], [1., 2.]])
theta = np.array([.2, -.1])
gradient = lambda x: information @ (x - theta)
covariance = observed_covariance(theta, gradient, 100, [(-5, 5)] * 2)
np.testing.assert_allclose(covariance, np.linalg.inv(100 * information), rtol=1e-9)
```

Read this block:

- Line 11, `information`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 12, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 13, `gradient`: Set this name to a one-expression callable, with input `x` and returned expression `information @ (x - theta)`.
- Line 14, `covariance`: Set this name to the result of calling `observed_covariance(...)`; the arguments and preceding method chain are shown above.
- Line 15: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- `observed_covariance`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 16–23:

```
with tempfile.TemporaryDirectory() as tmp:
    table, report = hessian_standard_errors(theta, gradient, 100,
        [(-5, 5)] * 2, ['a', 'b'], Path(tmp))
    np.testing.assert_allclose(table.std_error, np.sqrt(np.diag(covariance)))
    self.assertTrue((table.ci_lower < theta).all())
    self.assertTrue((table.ci_upper > theta).all())
    self.assertTrue((Path(tmp) / 'covariance.csv').exists())
    self.assertEqual(report['first_stage_uncertainty'], 'not included')
```

Read this block:

- Line 16: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 17, `(table, report)`: Unpack the result of calling `hessian_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 19: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 20: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 21: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 22: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 23: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `hessian_standard_errors`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

HessianChecks.test boundary and indefinite information withheld

Original function: `HessianChecks.test_boundary_and_indefinite_information_withheld`. Source begins at line 25.

Confirms boundary or invalid-information cases withhold inference instead of reporting misleading SEs. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 25–25:

```
25 def test_boundary_and_indefinite_information_withheld(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 26–27:

```
26 with self.assertRaisesRegex(ValueError, 'bounds'):
27     observed_covariance(np.array([0.]), lambda x: x, 10, [(0, 3)])
```

Read this block:

- Line 26: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 27: call `observed_covariance` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.
- `observed_covariance`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- lambda introduces a short callable. Its parameter is left of the colon; its one returned expression is right of it.

Source lines 28–29:

```
28 with self.assertRaisesRegex(ValueError, 'positive definite'):
29     observed_covariance(np.array([1.]), lambda x: -x, 10, [(-3, 3)])
```

Read this block:

- Line 28: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.

- Line 29: call `observed_covariance` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 30–32:

```
with self.assertRaisesRegex(ValueError, 'ill-conditioned'):
    observed_covariance(np.array([1., 1.]), lambda x: np.array([x[0], 0.]),
                        10, [(-3, 3)] * 2)
```

Read this block:

- Line 30: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 31: call `observed_covariance` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est_code/test_price_control.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Checks for DMA exclusion, first-stage OLS, and two-stage inference."""
import contextlib
import io
import tempfile
import unittest
from pathlib import Path
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `contextlib`. An as-name is a convenient local alias, not another library.
- Line 3: import `io`. An as-name is a convenient local alias, not another library.
- Line 4: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 5: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 6: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–14:

```
from types import SimpleNamespace

import numpy as np
import pandas as pd
from scipy.optimize import OptimizeResult
from unittest.mock import patch

from price_control import other_dma_instruments, absorb_effects, add_price_controls
```

Read this block:

- Line 7: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.

- Line 9: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 10: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 11: from `scipy.optimize`, import `OptimizeResult` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: from `unittest.mock`, import `patch` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 14: from `price_control`, import `other_dma_instruments, absorb_effects, add_price_controls` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 15–16:

```
15 from bootstrap_se import resample_households, bootstrap_standard_errors
16 from est import FLAVORS, make_arrays, optimize_demand
```

Read this block:

- Line 15: from `bootstrap_se`, import `resample_households, bootstrap_standard_errors` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 16: from `est`, import `FLAVORS, make_arrays, optimize_demand` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 43–43:

```
43 class PriceControlChecks(unittest.TestCase):
```

Read this block:

- Line 43: define class `PriceControlChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 136–137:

```
136 if __name__ == "__main__":
137     unittest.main()
```

Read this block:

- Line 136: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 137: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

synthetic panel

Original function: `synthetic_panel`. Source begins at line 19.

This constructs a reproducible simulated panel with 60 households and eight periods, first-stage prices, residuals, utilities, and multinomial package counts. `repeat` holds each household ID for eight rows; `tile` repeats the eight-date sequence for all households. Stable softmax subtracts each row's largest utility before exponentiating. `rng.multinomial(8,p)` samples an eight-package vector whose entries sum to eight. This gives the tests known structure without private data; it is a test fixture, not estimated reality.

Definition and stated intent:

Source lines 19–19:

```
19 def synthetic_panel():
```

`def` creates this callable; the parentheses name its inputs (). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 20–25:

```
rng = np.random.default_rng(120)
n_households, periods = 60, 8
size = n_households * periods
frame = pd.DataFrame({"household_code": np.repeat(np.arange(n_households), periods),
                      "week": np.tile(pd.date_range("2010-01-02", periods=periods, freq="7D"), n_households),
                      "dma_cd": np.repeat(np.arange(n_households) % 4 + 1, periods)})
```

Read this block:

- Line 20, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 21, `(n_households, periods)`: Unpack the tuple `(60, 8)` into these targets in order.
- Line 22, `size`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 23, `frame`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- `.repeat`: Repeat each element consecutively, e.g. each household ID across all its periods.
- `.tile`: Repeat a complete sequence, e.g. the period-date sequence for every household.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 26–30:

```
for flavor in FLAVORS:
    frame["xi_" + flavor] = rng.normal(size=size)
    frame["iv_" + flavor] = rng.uniform(.5, 2.5, size=size)
    frame["price_" + flavor] = (1 + .8*frame["iv_" + flavor]
                                + .1*frame["xi_" + flavor] + rng.normal(0, .2, size))
```

Read this block:

- Line 26: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 27, `frame['xi_' + flavor]`: Store the result of calling `rng.normal(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 28, `frame['iv_' + flavor]`: Store the result of calling `rng.uniform(...)`; the arguments and preceding method chain are shown above in the selected entry/column.
- Line 29, `frame['price_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Syntax and calls used here:

- `.normal`: Draw normal random values with the supplied location/scale, used for starts or synthetic data.
- `.uniform`: Draw values from a continuous uniform interval for controlled synthetic inputs.

Source lines 31–37:

```
controlled, _ = add_price_controls(frame)
utility = (np.array([0., .4, -.3])
           - controlled[["price_"+f for f in FLAVORS]].to_numpy()
           + .8*controlled[["cf_"+f for f in FLAVORS]].to_numpy())
probs = np.exp(utility - utility.max(axis=1, keepdims=True))
probs /= probs.sum(axis=1, keepdims=True)
counts = np.array([rng.multinomial(8, p) for p in probs])
```

Read this block:

- Line 31, `(controlled, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 32, `utility`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 35, `probs`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 36: update `probs` using `/=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 37, `counts`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.max`: Take the largest value, or a per-group/axis maximum when used on a table expression.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.multinomial`: Draw a vector of category counts summing to the specified total, given category probabilities.

Source lines 38–39:

```
38 for j, flavor in enumerate(FLAVORS):
39     frame[flavor] = counts[:, j]
```

Read this block:

- Line 38: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 39, `frame[flavor]`: Store the entry/slice selected by `counts[:, j]`; brackets select from the object to their left in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.

Source lines 40–40:

```
40 return frame
```

Read this block:

- Line 40: return the object currently named `frame`; plain assignment does not copy it. This ends the call.

PriceControlChecks.test excludes consumer dma even when exposure store elsewhere

Original function: `PriceControlChecks.test_excludes_consumer_dma_even_when_exposure_store_elsewhere`. Source begins at line 44.

What this test establishes. Exercises the retained UPC-matched IV helper with explicit tiny prices. It excludes the consumer DMA rather than assuming exposure store and consumer are in the same market, and checks incomplete baskets become missing.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results,

and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 44–44:

```
44 def test_excludes_consumer_dma_even_when_exposure_store_elsewhere(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 45–53:

```
45 week = pd.Timestamp("2010-01-02")
46 weekly = pd.DataFrame({"store_code_uc": [99, 99, 99], "week": [week]*3,
47                        "dma_cd": [1, 2, 4]})
48 exposure = pd.DataFrame({"store_code_uc": [99, 99], "week": [week]*2,
49                          "upc": ["a", "b"], "flavor": ["berry", "berry"]})
50 prices = pd.DataFrame({"upc": ["a"]*3+["b"]*3, "week": [week]*6,
51                        "dma_code": [1, 2, 3]*2, "dma_price": [1., 3., 5., 2., 4., 6.]})
52 result = other_dma_instruments(weekly, exposure, prices)
53 np.testing.assert_allclose(result["iv_berry"], [4.5, 3.5, 3.5])
```

Read this block:

- Line 45, `week`: Set this name to the result of calling `pd.Timestamp(...)`; the arguments and preceding method chain are shown above.
- Line 46, `weekly`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 48, `exposure`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 50, `prices`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 52, `result`: Set this name to the result of calling `other_dma_instruments(...)`; the arguments and preceding method chain are shown above.
- Line 53: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `other_dma_instruments`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 55–57:

```
55 missing = prices.loc[~((prices["upc"] == "b") & (prices["dma_code"] != 1))]
56 result = other_dma_instruments(weekly, exposure, missing)
57 self.assertTrue(np.isnan(result.loc[0, "iv_berry"]))
```

Read this block:

- Line 55, `missing`: Set this name to the entry/slice selected by `prices.loc[~((prices['upc'] == 'b') & (prices['dma_code']...`; brackets select from the object to their left.
- Line 56, `result`: Set this name to the result of calling `other_dma_instruments(...)`; the arguments and preceding method chain are shown above.
- Line 57: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.isnan`: Identify floating-point NaNs; NaN must not be tested with ordinary equality.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

PriceControlChecks.test absorption matches explicit dummy ols

Original function: `PriceControlChecks.test_absorption_matches_explicit_dummy_ols`. Source begins at line 59.

What this test establishes. Compares repeated demeaning to a small regression on explicit week/DMA dummy columns. The dummy matrix is feasible in a fixture and supplies a transparent independent alternative to the production memory-saving algorithm.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 59–59:

```
59 def test_absorption_matches_explicit_dummy_ols(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 60–66:

```
60 rng = np.random.default_rng(20)
61 weeks = pd.Series([1, 1, 2, 2, 2, 3, 3, 4, 4, 4])
62 dma = pd.Series([1, 2, 1, 2, 3, 1, 3, 1, 2, 3])
63 x = rng.normal(size=(len(weeks), 3))
64 dummies = pd.concat([pd.get_dummies(weeks, dtype=float),
65                      pd.get_dummies(dma, dtype=float)], axis=1).to_numpy()
66 expected = x - dummies @ np.linalg.lstsq(dummies, x, rcond=None)[0]
```

Read this block:

- Line 60, `rng`: Set this name to the result of calling `np.random.default_rng(...)`; the arguments and preceding method chain are shown above.
- Line 61, `weeks`: Set this name to the result of calling `pd.Series(...)`; the arguments and preceding method chain are shown above.
- Line 62, `dma`: Set this name to the result of calling `pd.Series(...)`; the arguments and preceding method chain are shown above.
- Line 63, `x`: Set this name to the result of calling `rng.normal(...)`; the arguments and preceding method chain are shown above.
- Line 64, `dummies`: Set this name to the result of calling `.to_numpy(...)`; the arguments and preceding method chain are shown above.
- Line 66, `expected`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- `.normal`: Draw normal random values with the supplied location/scale, used for starts or synthetic data.
- `len`: Count elements/rows rather than summing their numerical values.

- `.to_numpy`: Convert table/Series values to a numerical array for subsequent array arithmetic.
- `.get_dummies`: Construct indicator columns for categories, used as a small explicit fixed-effect benchmark in tests.
- `.lstsq`: Solve least squares without explicitly inverting a cross-product matrix; [0] keeps its coefficient vector.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 67–67:

```
67 np.testing.assert_allclose(absorb_effects(x, [weeks, dma]), expected, atol=1e-8)
```

Read this block:

- Line 67: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `absorb_effects`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

PriceControlChecks.test first stage residuals orthogonal and in utility

Original function: `PriceControlChecks.test_first_stage_residuals_orthogonal_and_in_utility`. Source begins at line 69.

What this test establishes. Checks positive first-stage relevance in controlled data, residual orthogonality, zero group means, and that the demand arrays actually contain the computed controls.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 69–69:

```
69 def test_first_stage_residuals_orthogonal_and_in_utility(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 70–73:

```
70 frame = synthetic_panel()
71 controlled, report = add_price_controls(frame)
72 self.assertTrue((report["iv_coefficient"] > .7).all())
73 self.assertTrue((report["iv_cluster_F"] > 10).all())
```

Read this block:

- Line 70, `frame`: Set this name to the result of calling `synthetic_panel(...)`; the arguments and preceding method chain are shown above.
- Line 71, `(controlled, report)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.

- Line 72: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 73: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

Source lines 74–77:

```
74 for flavor in FLAVORS:
75     residual = controlled["cf_" + flavor]
76     self.assertAlmostEqual(float(residual @ frame["iv_" + flavor]), 0., places=7)
77     np.testing.assert_allclose(residual.groupby(frame["week"]).mean(), 0., atol=1e-8)
```

Read this block:

- Line 74: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 75, `residual`: Set this name to the entry/slice selected by `controlled['cf_' + flavor]`; brackets select from the object to their left.
- Line 76: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 77: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.mean`: Compute an arithmetic mean over the selected group/axis, not automatically a weighted mean.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 78–80:

```
78 data = make_arrays(controlled, 0)
79 self.assertTrue(data["control"])
80 np.testing.assert_allclose(data["cf"], controlled[["cf_"+f for f in FLAVORS]])
```

Read this block:

- Line 78, `data`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 79: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 80: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

PriceControlChecks.test first stage uses only covered flavor prices

Original function: `PriceControlChecks.test_first_stage_uses_only_covered_flavor_prices`. Source begins at line 82.

What this test establishes. Moves purchases away from unavailable alternatives, removes their prices, and checks each first stage uses only its covered rows. It also verifies zero residual placeholders remain finite and masked.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 82–82:

```
82 def test_first_stage_uses_only_covered_flavor_prices(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 83–83:

```
83 frame = synthetic_panel()
```

Read this block:

- Line 83, `frame`: Set this name to the result of calling `synthetic_panel(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 84–89:

```
84 for j, flavor in enumerate(FLAVORS):
85     missing = np.arange(len(frame)) % 4 == j
86     next_flavor = FLAVORS[(j + 1) % 3]
87     frame.loc[missing, next_flavor] += frame.loc[missing, flavor]
88     frame.loc[missing, flavor] = 0
89     frame.loc[missing, "price_" + flavor] = np.nan
```

Read this block:

- Line 84: for each entry yielded by `enumerate(FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 85, `missing`: Set this name to the Boolean condition `np.arange(len(frame)) % 4 == j`; it can be a scalar or a row/array mask depending on its inputs.
- Line 86, `next_flavor`: Set this name to the entry/slice selected by `FLAVORS[(j + 1) % 3]`; brackets select from the object to their left.

- Line 87: update `frame.loc[missing, next_flavor]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 88, `frame.loc[missing, flavor]`: Store the literal value `0` in the selected entry/column.
- Line 89, `frame.loc[missing, 'price_' + flavor]`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.

Syntax and calls used here:

- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.arange`: Create evenly stepped values with an excluded stop, commonly row positions 0 through n-1.
- `len`: Count elements/rows rather than summing their numerical values.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 90–90:

```
90 controlled, report = add_price_controls(frame)
```

Read this block:

- Line 90, `(controlled, report)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 91–96:

```
91 for flavor in FLAVORS:
92     available = frame["price_" + flavor].notna()
93     self.assertEqual(report.set_index("flavor").loc[flavor, "observations"], available.sum())
94     self.assertTrue((controlled.loc[~available, "cf_" + flavor] == 0).all())
95     self.assertAlmostEqual(float(controlled.loc[available, "cf_" + flavor]
96                                  @ frame.loc[available, "iv_" + flavor]), 0., places=7)
```

Read this block:

- Line 91: for each entry yielded by `FLAVORS`, bind/unpack it into `flavor` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 92, `available`: Set this name to the result of calling `frame['price_' + flavor].notna(...)`; the arguments and preceding method chain are shown above.
- Line 93: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 94: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 95: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.notna`: Create a pandas mask marking nonmissing values; it does not establish numeric positivity or finiteness.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.

- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ~ operator reverses a Boolean mask here, changing true entries to false and vice versa.

Source lines 97–99:

```
97 arrays = make_arrays(controlled, 0)
98 self.assertFalse(arrays["available"].all())
99 self.assertTrue(np.isfinite(arrays["cf"]).all())
```

Read this block:

- Line 97, `arrays`: Set this name to the result of calling `make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 98: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 99: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

PriceControlChecks.test resampling preserves histories and relabels duplicates

Original function: `PriceControlChecks.test_resampling_preserves_histories_and_relabels_duplicates`. Source begins at line 101.

What this test establishes. Resamples a 60-household/eight-period fixture and verifies there are 60 newly labeled histories, each with eight rows and no repeated new-household/week key. Sampling whole groups is compared with the behavior a row bootstrap would break.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 101–101:

```
101 def test_resampling_preserves_histories_and_relabels_duplicates(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 102–106:

```
102 frame = synthetic_panel()
103 sampled = resample_households(frame, np.random.default_rng(1))
104 self.assertEqual(sampled["household_code"].nunique(), 60)
105 np.testing.assert_array_equal(sampled.groupby("household_code").size(), 8)
106 self.assertFalse(sampled.duplicated(["household_code", "week"]).any())
```

Read this block:

- Line 102, `frame`: Set this name to the result of calling `synthetic_panel(...)`; the arguments and preceding method chain are shown above.
- Line 103, `sampled`: Set this name to the result of calling `resample_households(...)`; the arguments and preceding method chain are shown above.
- Line 104: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 105: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 106: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `resample_households`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.default_rng`: Create a local seeded random Generator, avoiding dependence on a shared global random stream.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.nunique`: Count distinct pandas values, rather than counting repeated purchase rows as separate identities.
- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.
- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.any`: Reduce Boolean values to whether at least one is true; an axis can request a result for every row/column.
- `.duplicated`: Flag repeated keys/rows so validation can detect ambiguity instead of resolving it silently.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

PriceControlChecks.test bootstrap refits first stage and reports empirical sd

Original function: `PriceControlChecks.test_bootstrap_refits_first_stage_and_reports_empirical_sd`. Source begins at line 108.

What this test establishes. Wraps the first-stage callable with a mock to count 20 refits, supplies a cheap residual-dependent fake estimator, and compares output SEs with SDs calculated from saved bootstrap draws. This isolates two-stage orchestration from optimization.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 108–108:

```
108 def test_bootstrap_refits_first_stage_and_reports_empirical_sd(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 109–109:

```
109 frame = synthetic_panel()
```

Read this block:

- Line 109, `frame`: Set this name to the result of calling `synthetic_panel(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 110–125:

```
110 with tempfile.TemporaryDirectory() as directory:
111     args = SimpleNamespace(seed=306, bootstrap_reps=20, no_price_control=False,
112                            output=Path(directory))
113     def estimate(sample, initial):
114         # Explicitly depend on the recomputed residual, so frozen residuals
115         # would produce a different bootstrap distribution.
116         theta = np.array([sample["cf_other"].std(), sample["cf_berry"].std()])
117         return OptimizeResult(x=theta, fun=1., success=True, message="test fit")
118     with patch("bootstrap_se.add_price_controls", wraps=add_price_controls) as first_stage:
119         with contextlib.redirect_stdout(io.StringIO()):
120             table, report = bootstrap_standard_errors(frame, args, estimate, np.zeros(2), ["a", "b"])
121     self.assertEqual(first_stage.call_count, 20)
122     self.assertEqual(report["usable"], 20)
123     draws = pd.read_csv(Path(directory) / "bootstrap_draws.csv")
124     np.testing.assert_allclose(table["std_error"], draws[["a", "b"]].std(ddof=1))
125     self.assertTrue((table["std_error"] > 0).all())
```

Read this block:

- Line 110: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 111, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 113: define nested callable `estimate`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 118: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 119: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 120, `(table, report)`: Unpack the result of calling `bootstrap_standard_errors(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 121: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 122: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 123, `draws`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 124: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 125: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.

- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `OptimizeResult`: Construct a SciPy-style result object; test fixtures can provide one without a genuine optimizer run.
- `patch`: Temporarily replace/wrap a callable in a test so its invocations can be observed.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.redirect_stdout`: Temporarily capture printed output, keeping tests quiet without changing the tested computation.
- `bootstrap_standard_errors`: Call this project helper/callable with the displayed arguments; its own walk-through gives the input/output contract.
- `.StringIO`: Create an in-memory text buffer instead of writing captured output to a real file.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.

PriceControlChecks.test bootstrap refits first stage and reports empirical sd.estimate

Original function: `PriceControlChecks.test_bootstrap_refits_first_stage_and_reports_empirical_sd.estimate`. Source begins at line 113.

This nested fake estimator returns an `OptimizeResult` with coefficients equal to resampled residual SDs. It deliberately depends on the recomputed controls, so frozen controls would change the bootstrap distribution. It declares success and a harmless objective because the test targets bootstrap orchestration, not numerical optimization. Mocking the demand fit makes this check faster and isolates whether first stages are rerun and empirical SEs calculated correctly.

Definition and stated intent:

Source lines 113–115:

```
113 def estimate(sample, initial):
114     # Explicitly depend on the recomputed residual, so frozen residuals
115     # would produce a different bootstrap distribution.
```

`def` creates this callable; the parentheses name its inputs (`sample`, `initial`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 116–116:

```
116 theta = np.array([sample["cf_other"].std(), sample["cf_berry"].std()])
```

Read this block:

- Line 116, `theta`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.std`: Compute standard deviation; ddof=1 requests sample SD rather than dividing squared deviations by the full count.

Source lines 117–117:

```
117 return OptimizeResult(x=theta, fun=1., success=True, message="test fit")
```

Read this block:

- Line 117: return the result of calling `OptimizeResult(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `OptimizeResult`: Construct a SciPy-style result object; test fixtures can provide one without a genuine optimizer run.

PriceControlChecks.test controlled demand optimizer runs

Original function: `PriceControlChecks.test_controlled_demand_optimizer_runs`. Source begins at line 127.

What this test establishes. Fits a small controlled naive demand model and checks convergence, a four-coefficient vector, and finite objective. This is a controlled optimizer smoke test, not an assertion about private-data fit quality.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 127–127:

```
127 def test_controlled_demand_optimizer_runs(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 128–133:

```
128 controlled, _ = add_price_controls(synthetic_panel())
129 args = SimpleNamespace(model="naive", draws=8, seed=306, starts=2, maxiter=150)
130 fit, _, _ = optimize_demand(make_arrays(controlled, 0), args, quiet=True)
131 self.assertTrue(fit.success)
132 self.assertEqual(len(fit.x), 4)  # Two betas, mean log(-alpha), price control.
133 self.assertTrue(np.isfinite(fit.fun))
```

Read this block:

- Line 128, `(controlled, _)`: Unpack the result of calling `add_price_controls(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 129, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 130, `(fit, _, _)`: Unpack the result of calling `optimize_demand(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 131: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 132: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `add_price_controls`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `synthetic_panel`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.

- `optimize_demand`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `make_arrays`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `len`: Count elements/rows rather than summing their numerical values.
- `.isfinite`: Test for finite numbers, rejecting both NaN and positive/negative infinity.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est_code/test_regenerate_weekly_inputs.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Recovery control flow tests with generated fixtures only."""
2 import json
3 from pathlib import Path
4 import tempfile
5 import unittest
6 from unittest.mock import patch
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `json`. An as-name is a convenient local alias, not another library.
- Line 3: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 5: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 6: from `unittest.mock`, import `patch` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 8–10:

```
8 import numpy as np
9 import pandas as pd
10 import regenerate_weekly_inputs as recovery
```

Read this block:

- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 10: import `regenerate_weekly_inputs as recovery`. An as-name is a convenient local alias, not another library.

Source lines 13–13:

```
13 class RecoveryTests(unittest.TestCase):
```

Read this block:

- Line 13: define class `RecoveryTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 53–54:

```
53 if __name__ == '__main__':
54     unittest.main()
```

Read this block:

- Line 53: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 54: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

RecoveryTests.check recovery

Original function: `RecoveryTests.check_recovery`. Source begins at line 14.

Builds temporary synthetic fit records and mocks data preparation/CSV loading. It checks successful publication or a likelihood mismatch, and verifies saved fitted parameters are preserved. The mock is a controlled substitute, not evidence that an actual server dataset has already matched.

Definition and stated intent:

Source lines 14–14:

```
14 def check_recovery(self, mismatch):
```

`def` creates this callable; the parentheses name its inputs (`self`, `mismatch`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 15–44:

```
15 with tempfile.TemporaryDirectory() as temporary:
16     directory = Path(temporary)
17     fit = dict(model='heterogeneous_history', households=1, weeks=2, nll=3., settings=dict(
18         sample='/server/synthetic.parquet', first_year=2006, last_year=2015,
19         max_households=None, capacity=7, no_price_control=False,
20         household_dma_column='dma_cd', habit_weight=.7, draws=4, seed=306))
21     (directory/'fit.json').write_text(json.dumps(fit))
22     names = recovery.parameter_names('heterogeneous_history', True, True)
23     pd.DataFrame(dict(parameter=names, estimate=np.ones(len(names)))).to_csv(directory/'point_estimates.csv',
    ↪  index=False)
24     before = {p.name: p.read_bytes() for p in directory.iterdir()}
25     frame = pd.DataFrame({'synthetic_row': [1, 2]})
26     data = dict(n_households=1, outside=True, control=True)
27     with patch.object(recovery, 'prepare_data', return_value=frame) as prepare, \
28             patch.object(recovery, 'add_price_controls', return_value=(frame, None)) as controls, \
29             patch.object(recovery, 'make_arrays', return_value=data), \
30             patch.object(recovery, 'household_loglike', return_value=np.array([-4. if mismatch else -3.])):
31         if mismatch:
32             (directory/'weekly_inputs.csv').write_text('existing export')
33             with self.assertRaisesRegex(ValueError, 'likelihood'):
34                 recovery.regenerate(directory)
35             self.assertEqual((directory/'weekly_inputs.csv').read_text(), 'existing export')
36             self.assertFalse((directory/'weekly_inputs_regeneration.json').exists())
37         else:
38             report = recovery.regenerate(directory)
39             self.assertFalse(report['demand_refitted'])
40             self.assertEqual(len(pd.read_csv(directory/'weekly_inputs.csv')), 2)
41         controls.assert_called_once()
42         self.assertEqual(prepare.call_args.args[0].sample, Path('/server/synthetic.parquet'))
43     for name, content in before.items():
```

```
44          self.assertEqual((directory/name).read_bytes(), content)
```

Read this block:

- Line 15: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 16, `directory`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 17, `fit`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 21: call `(directory / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 22, `names`: Set this name to the result of calling `recovery.parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 23: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 24, `before`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 25, `frame`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 26, `data`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 27: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 31: test `mismatch`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 32: call `(directory / 'weekly_inputs.csv').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 34: call `recovery.regenerate` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 35: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38, `report`: Set this name to the result of calling `recovery.regenerate(...)`; the arguments and preceding method chain are shown above.
- Line 39: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 40: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 41: call `controls.assert_called_once` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: for each entry yielded by `before.items()`, bind/unpack it into `(name, content)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 44: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.

- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `len`: Count elements/rows rather than summing their numerical values.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

RecoveryTests.test recovery preserves parameters

Original function: `RecoveryTests.test_recovery_preserves_parameters`. Source begins at line 46.

Exercises successful weekly-export regeneration with mocks and confirms fit/parameter records remain byte-for-byte unchanged. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 46–46:

```
46 def test_recovery_preserves_parameters(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 47–47:

```
47 self.check_recovery(False)
```

Read this block:

- Line 47: call `self.check_recovery` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

RecoveryTests.test mismatch preserves existing export

Original function: `RecoveryTests.test_mismatch_preserves_existing_export`. Source begins at line 49.

Forces a reconstructed-likelihood mismatch and checks no replacement weekly export or success record is published. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 49–49:

```
49 def test_mismatch_preserves_existing_export(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 50–50:

```
50 self.check_recovery(True)
```

Read this block:

- Line 50: call `self.check_recovery` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est__code/test__summary__stats.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
1 """Checks for the sibling summary script, using synthetic purchases only."""
2 import importlib.util
3 import os
4 from pathlib import Path
5 import tempfile
6 import unittest
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `importlib.util`. An as-name is a convenient local alias, not another library.
- Line 3: import `os`. An as-name is a convenient local alias, not another library.
- Line 4: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 5: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 6: import `unittest`. An as-name is a convenient local alias, not another library.

Source lines 8–10:

```
8 import numpy as np
9 import pandas as pd
10 import polars as pl
```

Read this block:

- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 10: import `polars as pl`. An as-name is a convenient local alias, not another library.

Source lines 13–13:

```
13 class SummaryChecks(unittest.TestCase):
```

Read this block:

- Line 13: define class `SummaryChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 61–62:

```
61 if __name__ == "__main__":
62     unittest.main()
```

Read this block:

- Line 61: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 62: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

SummaryChecks.test demographics and mixed baskets

Original function: `SummaryChecks.test_demographics_and_mixed_baskets`. Source begins at line 14.

What this test establishes. Uses tiny tables to check household demographic selection, missing categories, and mixed-basket summary behavior. Explicit source values make it possible to verify denominators by hand.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 14–15:

```
14 def test_demographics_and_mixed_baskets(self):
15     # The override lets a prepared replacement be tested before installation.
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 16–19:

```
16 script = Path(os.environ.get("SUMMARY_SCRIPT", Path(__file__).parent / "../data_code/summ_stats.py"))
17 spec = importlib.util.spec_from_file_location("summary", script)
18 summary = importlib.util.module_from_spec(spec)
19 spec.loader.exec_module(summary)
```

Read this block:

- Line 16, `script`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 17, `spec`: Set this name to the result of calling `importlib.util.spec_from_file_location(...)`; the arguments and preceding method chain are shown above.
- Line 18, `summary`: Set this name to the result of calling `importlib.util.module_from_spec(...)`; the arguments and preceding method chain are shown above.
- Line 19: call `spec.loader.exec_module` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.get`: Retrieve a dictionary/Series entry with a fallback for an absent key; direct bracket lookup would require it.

Source lines 20–58:

```
20 with tempfile.TemporaryDirectory() as directory:
21     root = Path(directory)
22     source = pl.DataFrame({
23         "household_code": [1, 1, 1, 1, 2], "panel_year": [2010]*5,
```

```
        "purchase_date": ["2010-01-02", "2010-01-02", "2010-01-09", "2010-01-23", "2010-01-03"],
        "trip_code_uc": [1, 1, 2, 3, 4], "upc": [1, 2, 2, 3, 1], "quantity": [2, 1, 1, 3, 1],
        "race": [1, 1, 1, 1, 2], "hispanic_origin": [2, 2, 2, 2, 1],
        "household_income": [5, 5, 5, 5, 8], "female_head_age": [3, 3, 3, 3, 0],
        "male_head_age": [4, 4, 4, 4, 2], "household_size": [2]*5})
    source.write_parquet(root / "sample.parquet")
    scan = summary.scan_purchases(root / "sample.parquet", 2006, 2015)
    households =
↪   summary.household_summary(scan).collect(engine="streaming").to_pandas().set_index("household_code")
    self.assertEqual(len(households), 2)
    self.assertEqual(households.loc[1, "trips"], 3)
    self.assertEqual(households.loc[2, "race"], "Hispanic")
    self.assertEqual(households.loc[2, "race_original"], "2")
    summary.demographic_tables(households.reset_index(), root, "test")
    race = pd.read_csv(root / "test_race.csv")
    np.testing.assert_allclose(race["percent_all_households"], 50.)
    reference = pl.DataFrame({"upc": ["1", "2", "3"], "flavor": ["other", "berry", "plain"]})
    weekly, _ = summary.yogurt_weeks(scan, reference)
    self.assertEqual(pd.Timestamp(weekly.filter(pl.col("household_code") ==
↪    2).select("week").collect(engine="streaming").item()), pd.Timestamp("2010-01-09"))
    repeated = summary.repeated_yogurt_households(scan)
    self.assertEqual(summary.household_summary(repeated).collect(engine="streaming")["household_code"].to_list(),
↪    [1])
    # Several purchases in a single week still do not qualify.
    one_week = scan.filter(pl.col("purchase_date") <= pl.date(2010, 1, 3))
    self.assertEqual(summary.repeated_yogurt_households(one_week).collect().height, 0)
    shares, transitions = summary.flavor_summaries(weekly, root)
    summary.make_hhi_figures(root)
    metrics = pd.read_csv(root / "yogurt_household_flavors.csv").set_index("household_code")
    self.assertAlmostEqual(metrics.loc[1, "flavor_HHI"], 17 / 49)
    self.assertEqual(metrics.loc[2, "flavor_HHI"], 1.)
    for kind in ["histogram", "cdf"]:
        self.assertTrue((root / f"flavor_hhi_{kind}.pdf").exists())
    baskets = pd.read_csv(root / "yogurt_weekly_baskets.csv")
    self.assertEqual(baskets.loc[(baskets.household_code == 1) & (baskets.week == "2010-01-02"),
↪    "flavors_in_basket"].iloc[0], 2)
    self.assertTrue(transitions.isna().all().all())
    self.assertAlmostEqual(shares.percent_packages.sum(), 100.)
    self.assertTrue((root / "test_income.tex").exists())
```

Read this block:

- Line 20: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 21, `root`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 22, `source`: Set this name to the result of calling `pl.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 29: call `source.write_parquet` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 30, `scan`: Set this name to the result of calling `summary.scan_purchases(...)`; the arguments and preceding method chain are shown above.
- Line 31, `households`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 32: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 34: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 35: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `summary.demographic_tables` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37, `race`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 38: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `reference`: Set this name to the result of calling `pl.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 40, `(weekly, _)`: Unpack the result of calling `summary.yogurt_weeks(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 41: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42, `repeated`: Set this name to the result of calling `summary.repeated_yogurt_households(...)`; the arguments and preceding method chain are shown above.
- Line 43: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 45, `one_week`: Set this name to the result of calling `scan.filter(...)`; the arguments and preceding method chain are shown above.
- Line 46: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 47, `(shares, transitions)`: Unpack the result of calling `summary.flavor_summaries(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 48: call `summary.make_hhi_figures` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 49, `metrics`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 50: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 51: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 52: for each entry yielded by `['histogram', 'cdf']`, bind/unpack it into `kind` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 53: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 54, `baskets`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 55: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 56: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 57: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 58: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `.write_parquet`: Persist an eager table as Parquet, often used for small synthetic fixtures.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

- `.filter`: Retain rows whose Boolean condition is true; missing-condition behavior depends on the table library.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `len`: Count elements/rows rather than summing their numerical values.
- `.reset_index`: Turn the index back into ordinary row numbering; drop=True discards the old index rather than saving it as a column.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `.to_pandas`: Convert a Polars table to pandas, crossing from query/table preparation to eager labeled analysis.
- `.item`: Extract a scalar from a one-element array/one-cell result rather than passing a table wrapper onward.
- `.col`: Build a Polars column-expression recipe; the string identifies a column rather than a literal value.
- `.collect`: Execute a lazy Polars query and materialize its resulting table; this can use substantial memory.
- `.isna`: Create a pandas mask marking missing values; compare masks rather than using equality to NaN.
- `.select`: Choose output columns or evaluate selected expressions; unselected columns do not automatically remain.
- The & operator combines Boolean masks element by element; parenthesize the separate comparisons.
- The f-prefixed string inserts expressions in braces; format suffixes control display without changing stored coefficients.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

est__code/test__wtp.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–4:

```
1 import unittest
2 import numpy as np
3 import pandas as pd
4 from wtp import implied_wtp
```

Read this block:

- Line 1: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 2: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 3: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 4: from `wtp`, import `implied_wtp` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–7:

```
7 class WTPChecks(unittest.TestCase):
```

Read this block:

- Line 7: define class `WTPChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 50–51:

```
50 if __name__ == '__main__':
51     unittest.main()
```

Read this block:

- Line 50: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 51: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

WTPChecks.table

Original function: `WTPChecks.table`. Source begins at line 8.

Builds a small generated coefficient table with the parameter/estimate schema expected by the WTP routines; it performs no file read.

Definition and stated intent:

Source lines 8–8:

```
8 def table(self, **values):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

`**values` collects additional keyword arguments in a dictionary.

Source lines 9–9:

```
9 return pd.DataFrame(dict(parameter=list(values), estimate=list(values.values())))
```

Read this block:

- Line 9: return the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above. This ends the call.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `list`: Materialize iterable entries as an ordered Python list.

WTPChecks.test fixed alpha and signed history

Original function: `WTPChecks.test_fixed_alpha_and_signed_history`. Source begins at line 11.

Checks WTP conversion against an elementary fixed-price-sensitivity calculation, preserving the signs of history effects. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 11–11:

```
11 def test_fixed_alpha_and_signed_history(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 12–18:

```
12 result = implied_wtp(self.table(mu_log_minus_alpha=np.log(2), beta_berry=-4.,
13                                  mean_gamma=-2.), .7).set_index('term')
14 self.assertAlmostEqual(result.loc['beta_berry', 'mean_wtp'], -2.)
15 self.assertAlmostEqual(result.loc['gamma', 'mean_wtp'], -1.)
```

```
16 self.assertAlmostEqual(result.loc['gamma_novelty', 'mean_wtp'], -.3)
17 self.assertAlmostEqual(result.loc['gamma_own_stock', 'mean_wtp'], .7)
18 np.testing.assert_allclose(result.population_sd, 0., atol=1e-7)
```

Read this block:

- Line 12, `result`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 14: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 15: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 16: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 17: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 18: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `implied_wtp`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

WTPChecks.test mixture moments include between component variation

Original function: `WTPChecks.test_mixture_moments_include_between_component_variation`. Source begins at line 20.

Checks mixture WTP variance includes differences between component means as well as within-component variances. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 20–20:

```
20 def test_mixture_moments_include_between_component_variation(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 21–29:

```
21 result = implied_wtp(self.table(mu_log_minus_alpha=0., sd_log_minus_alpha=.4,
22     mean_gamma=-1., sd_gamma=.2, gamma_mean_gap=3., sd_gamma_2=.3,
23     mixture_weight=.25)).set_index('term')
24 mean = .25 * -1 + .75 * 2
25 second = .25 * (1 + .2**2) + .75 * (4 + .3**2)
26 expected = mean * np.exp(.4**2/2)
27 self.assertAlmostEqual(result.loc['gamma', 'mean_wtp'], expected)
28 self.assertAlmostEqual(result.loc['gamma', 'population_sd'],
29                        np.sqrt(second * np.exp(2*.4**2) - expected**2))
```

Read this block:

- Line 21, `result`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 24, `mean`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 25, `second`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 26, `expected`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 27: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 28: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `implied_wtp`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Source lines 30–32:

```
30 self.assertAlmostEqual(result.loc['gamma', 'mean_wtp'],
31     .25 * result.loc['gamma_component_1', 'mean_wtp'] +
32     .75 * result.loc['gamma_component_2', 'mean_wtp'])
```

Read this block:

- Line 30: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

WTPChecks.test separate history wtp has no fixed weight

Original function: `WTPChecks.test_separate_history_wtp_has_no_fixed_weight`. Source begins at line 34.

Verifies independently estimated novelty and stock coefficients are converted to WTP without applying the old lambda weights. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 34–34:

```
34 def test_separate_history_wtp_has_no_fixed_weight(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 35–41:

```
35 table = self.table(mu_log_minus_alpha=np.log(2), mean_novelty=.6,
36                    mean_own_stock=-.8, sd_own_stock=.4)
37 result = implied_wtp(table, .1).set_index("term")
38 self.assertAlmostEqual(result.loc["novelty", "mean_wtp"], .3)
39 self.assertAlmostEqual(result.loc["own_stock", "mean_wtp"], -.4)
```

```
40 self.assertAlmostEqual(result.loc["own_stock", "population_sd"], .2)
41 pd.testing.assert_frame_equal(implied_wtp(table, .1), implied_wtp(table, .9))
```

Read this block:

- Line 35, `table`: Set this name to the result of calling `self.table(...)`; the arguments and preceding method chain are shown above.
- Line 37, `result`: Set this name to the result of calling `.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 38: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 40: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 41: call `pd.testing.assert_frame_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `implied_wtp`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.assert_frame_equal`: Compare pandas tables including values, labels, and dtype conventions rather than only row counts.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

WTPChecks.test beta averages ratios

Original function: `WTPChecks.test_beta_averages_ratios`. Source begins at line 43.

Checks beta WTP averages inverse price sensitivities rather than using the inverse of their mean. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 43–43:

```
43 def test_beta_averages_ratios(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 44–47:

```
44 result = implied_wtp(self.table(mu_log_minus_alpha=.2, sd_log_minus_alpha=.5,
45                                  beta_plain=2.)).iloc[0]
46 self.assertAlmostEqual(result.mean_wtp, 2 * np.exp(-.2 + .5**2/2))
47 self.assertGreater(result.population_sd, 0.)
```

Read this block:

- Line 44, `result`: Set this name to the entry/slice selected by `implied_wtp(self.table(mu_log_minus_alpha=0.2, sd_log_min...`; brackets select from the object to their left.
- Line 46: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 47: call `self.assertGreater` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `implied_wtp`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

supply_code/checks/___init___.py

This empty file marks an importable package directory. It defines no functions and performs no data calculations. Python can support some namespace packages without it, but an explicit marker makes the intended package structure clear.

Module setup and direct-execution blocks

There are no statements in this file.

supply_code/checks/check_alpha.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 """Compatibility entry point for the canonical symmetric-switching checks."""
2 import unittest
3 from checks.test_symmetric import SymbolicChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply__code/checks/check__beta.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply__code/checks/check__delta.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply__code/checks/check__focs.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply__code/checks/check__gamma.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.

- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply__code/checks/check__numerical.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
1 """Compatibility entry point for the canonical symmetric-switching checks."""
2 import unittest
3 from checks.test_symmetric import NumericalChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `NumericalChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
4 if __name__ == "__main__":
5     unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply__code/checks/check__pricing__derivatives.py

This is a compatibility entry point for the canonical unittest checks. Importing a TestCase class makes its methods discoverable to unittest; the guarded unittest.main() runs the selected checks when invoked directly. It does not define another derivative model.

Module setup and direct-execution blocks

Source lines 1–3:

```
"""Compatibility entry point for the canonical symmetric-switching checks."""
import unittest
from checks.test_symmetric import SymbolicChecks
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: from `checks.test_symmetric`, import `SymbolicChecks` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 4–5:

```
if __name__ == "__main__":
    unittest.main()
```

Read this block:

- Line 4: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 5: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

supply_code/checks/test_symmetric.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Symbolic identities and independent reoptimized finite differences."""
import unittest
import numpy as np
import sympy as sp
from scipy.optimize import minimize_scalar
from model.symbols import *
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 3: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 4: import `sympy as sp`. An as-name is a convenient local alias, not another library.
- Line 5: from `scipy.optimize`, import `minimize_scalar` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 6: from `model.symbols`, import $*$ into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 7–12:

```
from model.utility import raw_utilities, U1, U2
from model.shares import sj1_expr, sj2_expr
from model.profit import Pi
from pricing.period1_foc import foc1_collected
from pricing.period2_foc import foc2
```

```
12 from comparative_statics import canonical as C
```

Read this block:

- Line 7: from `model.utility`, import `raw_utilities, U1, U2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 8: from `model.shares`, import `sj1_expr, sj2_expr` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 9: from `model.profit`, import `Pi` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 10: from `pricing.period1_foc`, import `foc1_collected` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 11: from `pricing.period2_foc`, import `foc2` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 12: from `comparative_statics`, import `canonical as C` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 13–14:

```
13 from model.numerical import evaluate, inside_share
14 import structural_switching_reference as reference
```

Read this block:

- Line 13: from `model.numerical`, import `evaluate, inside_share` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 14: import `structural_switching_reference as reference`. An as-name is a convenient local alias, not another library.

Source lines 17–17:

```
17 class SymbolicChecks(unittest.TestCase):
```

Read this block:

- Line 17: define class `SymbolicChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 57–57:

```
57 class NumericalChecks(unittest.TestCase):
```

Read this block:

- Line 57: define class `NumericalChecks` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 134–135:

```
134 if __name__=='__main__':
135     unittest.main()
```

Read this block:

- Line 134: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 135: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

SymbolicChecks.equal

Original function: `SymbolicChecks.equal`. Source begins at line 18.

This helper asserts symbolic equality by simplifying the difference to zero. `self` supplies the current unittest instance, so `self.assertEqual` records a normal test failure instead of only printing a warning. Structural `left == right` may fail for algebraically equivalent expressions written in different forms. Simplifying their difference is a stronger algebraic comparison for these expressions, though symbolic simplification is not a universal theorem prover.

Definition and stated intent:

Source lines 18–18:

```
18 def equal(self,left,right):
```

`def` creates this callable; the parentheses name its inputs (`self`, `left`, `right`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 19–19:

```
19 self.assertEqual(sp.simplify(left-right),0)
```

Read this block:

- Line 19: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.simplify`: Try algebraic simplification; it does not numerically optimize a price or establish every economic sign.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SymbolicChecks.test raw normalization and both states

Original function: `SymbolicChecks.test_raw_normalization_and_both_states`. Source begins at line 21.

What this test establishes. Evaluates both signed histories and symbolically verifies subtracting raw outside utility yields beta+alpha*price*+*q*gamma. This catches accidentally deleting outside switching utility.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 21–21:

```
21 def test_raw_normalization_and_both_states(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 22–24:

```
22 for history in [-1,1]:
23     a,b = raw_utilities(p1,history)
24     self.equal(a-b,beta+alpha*p1+history*gamma)
```

Read this block:

- Line 22: for each entry yielded by `[-1, 1]`, bind/unpack it into `history` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 23, `(a, b)`: Unpack the result of calling `raw_utilities(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 24: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `raw_utilities`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 25–25:

```
25 self.equal(U1,beta+alpha*p1+q*gamma)
```

Read this block:

- Line 25: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SymbolicChecks.test focs from actual profit

Original function: `SymbolicChecks.test_focs_from_actual_profit`. Source begins at line 27.

What this test establishes. Differentiates literal symbolic profits and compares them to stored FOCs, including both continuation weights. It checks the objective behind the equations rather than merely rearranging a copied FOC.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 27–27:

```
27 def test_focs_from_actual_profit(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 28–33:

```
28 literal = Pi.subs(sj1,sj1_expr)
29 expected = foc1_collected.subs(sj1,sj1_expr)
30 self.equal(sp.diff(literal,p1),expected)
31 self.equal(sp.diff((p2-c)*sj2_expr,p2),foc2.subs(sj2,sj2_expr))
32 self.equal(sp.diff(literal,Vj),delta*sj1_expr)
33 self.equal(sp.diff(literal,V0),delta*(1-sj1_expr))
```

Read this block:

- Line 28, `literal`: Set this name to the result of calling `Pi.subs(...)`; the arguments and preceding method chain are shown above.
- Line 29, `expected`: Set this name to the result of calling `foc1_collected.subs(...)`; the arguments and preceding method chain are shown above.
- Line 30: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 31: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 32: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.subs`: Substitute symbols/expressions in symbolic algebra; this does not by itself solve an equilibrium.
- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Source lines 34–34:

```
34 self.equal(sp.diff(Vj,p1),0)
```

Read this block:

- Line 34: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SymbolicChecks.test envelope and implicit gamma derivatives

Original function: `SymbolicChecks.test_envelope_and_implicit_gamma_derivatives`. Source begins at line 36.

What this test establishes. Verifies explicit gamma price/profit/curvature identities and total derivatives of the implicit FOC. Algebraic comparison catches missing chain-rule and continuation terms.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 36–36:

```
36 def test_envelope_and_implicit_gamma_derivatives(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 37–43:

```
37 self.equal(C.dp1_dgamma,-(q*sj1+delta*(1-sj1)*(u+v))/alpha)
38 self.equal(C.dPi_dgamma,(-q*sj1+delta*(sj1*u-(1-sj1)*v))/alpha)
39 expected2 = -(sj1*(1-sj1)**2*(q-delta*(u+v))**2
40              +delta*(1-sj1)*(-u*(1-u)**2+v*(1-v)**2))/alpha
41 self.equal(C.d2p1_dgamma2,expected2)
42 self.equal(C.ds_dgamma,sj1*(1-sj1)**2*(q-delta*(u+v)))
43 self.equal(C.dVj_dgamma,u/alpha)
```

Read this block:

- Line 37: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `expected2`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 41: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 44–49:

```
44 self.equal(C.dV0_dgamma,-v/alpha)
45 # Differentiate implicit pricing equations with endogenous shares.
46 F = p1-c+1/(alpha*(1-sj1))+delta*(Vj-V0)
47 self.equal(C.total_derivative(F,gamma),0)
48 self.equal(sp.diff(p2-c+1/(alpha*(1-sj2)),p2)
49           +sp.diff(p2-c+1/(alpha*(1-sj2)),sj2)*alpha*sj2*(1-sj2),1/(1-sj2))
```

Read this block:

- Line 44: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 46, `F`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 47: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 48: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.diff`: For SymPy differentiate algebra; for a Polars/pandas date/value expression calculate successive differences. The receiver decides which operation this name means.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SymbolicChecks.test all parameter implicit derivatives

Original function: `SymbolicChecks.test_all_parameter_implicit_derivatives`. Source begins at line 51.

What this test establishes. Loops through alpha, beta, delta, and cost and verifies that differentiating the equilibrium pricing identity along each solution gives zero. This extends checks beyond the headline gamma plots.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 51–51:

```
51 def test_all_parameter_implicit_derivatives(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 52–52:

```
52 F = p1-c+1/(alpha*(1-sj1))+delta*(Vj-V0)
```

Read this block:

- Line 52, `F`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.

Source lines 53–54:

```
53 for theta in [alpha,beta,delta,c]:
54     self.equal(C.total_derivative(F,theta),0)
```

Read this block:

- Line 53: for each entry yielded by `[alpha, beta, delta, c]`, bind/unpack it into `theta` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 54: call `self.equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

NumericalChecks.test reference on entire benchmark grid

Original function: `NumericalChecks.test_reference_on_entire_benchmark_grid`. Source begins at line 58.

What this test establishes. Solves both histories at all 601 gamma grid points and compares named numerical outputs against the independently coded reference. It also checks original FOC residual magnitudes.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 58–58:

```
58 def test_reference_on_entire_benchmark_grid(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 59–64:

```
59 for g in np.linspace(-3,3,601):
60     for state in ['j','0']:
61         ours, original = evaluate(float(g),state),reference.evaluate(float(g),state)
62         for key in ['p1','p2j','p20','profit','dp1','d2p1','dprofit','d_matched_ratio']:
63             self.assertAlmostEqual(ours[key],original[key],places=9)
64         self.assertLess(max(abs(ours[k]) for k in ['foc1','foc2j','foc20']),1e-10)
```

Read this block:

- Line 59: for each entry yielded by `np.linspace(-3, 3, 601)`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 60: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 61, `(ours, original)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 62: for each entry yielded by `['p1', 'p2j', 'p20', 'profit', 'dp1', 'd2p1', 'dprofit', 'd_ma...`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 63: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.
- `.assertLess`: Fail unless the first value is strictly smaller than the second.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `abs`: Take magnitude, removing a sign before a distance/tolerance comparison.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

NumericalChecks.test independent reoptimization and derivatives

Original function: `NumericalChecks.test_independent_reoptimization_and_derivatives`. Source begins at line 66.

What this test establishes. At several calibrations compares analytic slopes and curvature with finite differences of fully reoptimized solutions, ratio derivatives with recomputed ratios, and p1 with a separate direct profit optimizer. Different computational routes can catch shared-expression mistakes.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 66–66:

```
66 def test_independent_reoptimization_and_derivatives(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 67–67:

```
67 h=1e-4
```

Read this block:

- Line 67, `h`: Set this name to the literal value `0.0001`.

Source lines 68–91:

```
68 for parameters in [dict(beta=2,alpha=-1,delta=.92,c=0),
69                    dict(beta=-1,alpha=-2,delta=.7,c=1),
70                    dict(beta=1.5,alpha=-.7,delta=.4,c=.3)]:
71     for g in [-2.4,-1.2,0,.7,2.6]:
72         for state in ['j','0']:
73             r=evaluate(g,state,**parameters)
74             plus,minus=evaluate(g+h,state,**parameters),evaluate(g-h,state,**parameters)
75             for derivative,level in [('dp1','p1'),('dprofit','profit'),('dp2j','p2j'),('dp20','p20')]:
76                 self.assertAlmostEqual(r[derivative],(plus[level]-minus[level])/(2*h),delta=3e-7)
```

```
            self.assertAlmostEqual(r['d2p1'],(plus['dp1']-minus['dp1'])/(2*h),delta=3e-7)
            self.assertAlmostEqual(r['d2p1'],(plus['p1']-2*r['p1']+minus['p1'])/h**2,delta=3e-5)
            denominator='p2j' if state=='j' else 'p20'
            dr=(plus['p1']/plus[denominator]-minus['p1']/minus[denominator])/(2*h)
            self.assertAlmostEqual(r['d_matched_ratio'],dr,delta=3e-7)
            if state=='0':
                self.assertAlmostEqual(r['d_induce_ratio'],(plus['p1']/plus['p2j']-minus['p1']/minus['p2j'])/(2*
                 ↪  h),delta=3e-7)
            self.assertAlmostEqual(r['p1'],r['static_component']+r['continuation_adjustment'],places=10)
            # Independent optimizer of the complete two-branch profit.
            def negative_profit(price):
                share=inside_share(price,g,state,parameters['beta'],parameters['alpha'])
                return -(share*(price-parameters['c'])+parameters['delta']*(share*r['V2j']+(1-share)*r['V20']))
            optimum=minimize_scalar(negative_profit,bounds=(r['p1']-10,r['p1']+10),method='bounded')
            self.assertAlmostEqual(optimum.x,r['p1'],delta=1e-5)
            self.assertLess(negative_profit(r['p1']),(negative_profit(r['p1']+h)+negative_profit(r['p1']-h))/2)
```

Read this block:

- Line 68: for each entry yielded by `[dict(beta=2, alpha=-1, delta=0.92, c=0), dict(beta=-1, alpha=...`, bind/unpack it into `parameters` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 71: for each entry yielded by `[-2.4, -1.2, 0, 0.7, 2.6]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 72: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 73, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 74, `(plus, minus)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 75: for each entry yielded by `[('dp1', 'p1'), ('dprofit', 'profit'), ('dp2j', 'p2j'), ('dp20...`, bind/unpack it into `(derivative, level)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 76: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 77: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 78: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 79, `denominator`: Set this name to the true-branch value if `state == 'j'` holds, otherwise the else-branch value.
- Line 80, `dr`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 81: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 82: test `state == '0'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 83: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 84: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 86: define nested callable `negative_profit`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.
- Line 89, `optimum`: Set this name to the result of calling `minimize_scalar(...)`; the arguments and preceding method chain are shown above.
- Line 90: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its

documented side effect or assertion is what matters.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `minimize_scalar`: Numerically minimize a one-variable objective, used as an independent check of price optimality.
- `.assertLess`: Fail unless the first value is strictly smaller than the second.
- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `negative_profit`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

NumericalChecks.test independent reoptimization and derivatives.negative profit

Original function: `NumericalChecks.test_independent_reoptimization_and_derivatives.negative_profit`. Source begins at line 86.

This nested objective accepts a candidate first-period price, computes its logit share, and returns the negative of complete two-branch discounted profit. It captures current gamma, state, calibration, and solved continuation values from the enclosing test loop. `minimize_scalar` minimizes, so negating profit turns maximization into minimization. Using a fresh direct objective here checks the production root against another computational route.

Definition and stated intent:

Source lines 86–86:

```
86 def negative_profit(price):
```

`def` creates this callable; the parentheses name its inputs (`price`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 87–87:

```
87 share=inside_share(price,g,state,parameters['beta'],parameters['alpha'])
```

Read this block:

- Line 87, `share`: Set this name to the result of calling `inside_share(...)`; the arguments and preceding method chain are shown above.

Syntax and calls used here:

- `inside_share`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

Source lines 88–88:

```
88 return -(share*(price-parameters['c'])+parameters['delta']*(share*r['V2j']+(1-share)*r['V20']))
```

Read this block:

- Line 88: return the negated or sign-adjusted expression `-(share * (price - parameters['c']) + parameters['delta']...`. This ends the call.

NumericalChecks.test all parameter and mixed derivatives numerically

Original function: `NumericalChecks.test_all_parameter_and_mixed_derivatives_numerically`. Source begins at line 93.

What this test establishes. Lambdifies general parameter derivatives, perturbs alpha/beta/delta/c separately, and compares finite differences. It also checks a beta-gamma mixed derivative using reoptimized results.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 93–93:

```
93 def test_all_parameter_and_mixed_derivatives_numerically(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 94–95:

```
94 parameters=dict(beta=2.,alpha=-1.,delta=.92,c=0.)
95 arguments=(alpha,beta,gamma,delta,c,q,sj1,u,v,p1,p2j,p20,Vj,V0)
```

Read this block:

- Line 94, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 95, `arguments`: Set this name to a tuple containing 14 displayed entries.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.

Source lines 96–98:

```
96 def arguments_at(result):
97     return (-1.,2.,result['gamma'],.92,0.,-1 if result['initial_state']=='j' else 1,
98
 ↪  result['s'],result['u'],result['v'],result['p1'],result['p2j'],result['p20'],result['V2j'],result['V20'])
```

Read this block:

- Line 96: define nested callable `arguments_at`. Its body runs when it is called, and it can read enclosing local names. It has its own dedicated walkthrough below.

Source lines 99–112:

```
99 for theta,name in [(alpha,'alpha'),(beta,'beta'),(delta,'delta'),(c,'c')]:
100     expression=C.price1_derivative(theta)
101     terminal=C.price2_derivative(theta,u,-1,p2j)
102     evaluator=sp.lambdify(arguments,[expression,terminal],'numpy')
103     for state in ['j','0']:
104         for g in [-1.,1.]:
105             result=evaluate(g,state,**parameters)
106             h=1e-5
107             plus_parameters=dict(parameters);plus_parameters[name]+=h
108             minus_parameters=dict(parameters);minus_parameters[name]-=h
109             plus=evaluate(g,state,**plus_parameters);minus=evaluate(g,state,**minus_parameters)
110             analytical=evaluator(*arguments_at(result))
111             self.assertAlmostEqual(float(analytical[0]),(plus['p1']-minus['p1'])/(2*h),delta=2e-6)
112             self.assertAlmostEqual(float(analytical[1]),(plus['p2j']-minus['p2j'])/(2*h),delta=2e-6)
```

Read this block:

- Line 99: for each entry yielded by `[(alpha, 'alpha'), (beta, 'beta'), (delta, 'delta'), (c, 'c')]`, bind/unpack it into `(theta, name)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 100, `expression`: Set this name to the result of calling `C.price1_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 101, `terminal`: Set this name to the result of calling `C.price2_derivative(...)`; the arguments and preceding method chain are shown above.
- Line 102, `evaluator`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.
- Line 103: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 104: for each entry yielded by `[-1.0, 1.0]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 105, `result`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 106, `h`: Set this name to the literal value `1e-05`.
- Line 107, `plus_parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 107: update `plus_parameters[name]` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 108, `minus_parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 108: update `minus_parameters[name]` using `-=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 109, `plus`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 109, `minus`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 110, `analytical`: Set this name to the result of calling `evaluator(...)`; the arguments and preceding method chain are shown above.
- Line 111: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 112: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.lambdify`: Compile a symbolic expression into a numerical callable with an explicitly ordered argument list.
- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `evaluator`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `float`: Convert a scalar to an ordinary Python floating-point value, useful for portable outputs.
- `arguments_at`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 113–114:

```
113 mixed=sp.lambdify(arguments,C.total_derivative(C.price1_derivative(beta),gamma),'numpy')
114 slope=sp.lambdify(arguments,C.price1_derivative(beta),'numpy')
```

Read this block:

- Line 113, `mixed`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.
- Line 114, `slope`: Set this name to the result of calling `sp.lambdify(...)`; the arguments and preceding method chain are shown above.

Source lines 115–119:

```
115 for state in ['j','0']:
116     g=.7;h=1e-4
117     actual=mixed(*arguments_at(evaluate(g,state)))
118     expected=(slope(*arguments_at(evaluate(g+h,state)))-slope(*arguments_at(evaluate(g-h,state))))/(2*h)
119     self.assertAlmostEqual(float(actual),float(expected),delta=2e-6)
```

Read this block:

- Line 115: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 116, `g`: Set this name to the literal value `0.7`.
- Line 116, `h`: Set this name to the literal value `0.0001`.
- Line 117, `actual`: Set this name to the result of calling `mixed(...)`; the arguments and preceding method chain are shown above.
- Line 118, `expected`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 119: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `mixed`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `slope`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.

NumericalChecks.test all parameter and mixed derivatives numerically.arguments at

Original function: `NumericalChecks.test_all_parameter_and_mixed_derivatives_numerically.arguments_at`. Source begins at line 96.

This nested adapter turns a result dictionary into the exact tuple expected by lambdified symbolic derivatives. The q sign uses initial history, and branch prices/shares/values retain their order. Its benchmark constants deliberately match the enclosing test calibration. A tuple is appropriate because the numerical evaluator uses positional arguments; a dictionary would require explicit ordered extraction anyway. Swapping u/v could make a wrong formula look correct at the symmetric gamma=0 point, so tests also use nonzero gamma.

Definition and stated intent:

Source lines 96–96:

```
96 def arguments_at(result):
```

`def` creates this callable; the parentheses name its inputs (`result`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 97–98:

```
97 return (-1.,2.,result['gamma'],.92,0.,-1 if result['initial_state']=='j' else 1,
98         result['s'],result['u'],result['v'],result['p1'],result['p2j'],result['p20'],result['V2j'],result['V20'])
```

Read this block:

- Line 97: return a tuple of 14 results. The caller can unpack them; return ends the call, unlike print.

Syntax and calls used here:

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

NumericalChecks.test zero switching static case and symmetry

Original function: `NumericalChecks.test_zero_switching_static_case_and_symmetry`. Source begins at line 121.

What this test establishes. Checks gamma=0 branch equality, delta=0 equality with the true static counterfactual, terminal history/sign symmetry, and an admissible negative introductory price. These limit cases catch state and price-floor regressions.

Reading its syntax. This method is called by unittest on a TestCase instance. The first parameter self provides the assertion methods. Setup assignments create known inputs, the helper call produces actual results, and assertion lines compare them with expectations. The method normally returns None; a failed assertion raises and marks the test as failed. The following line notes spell out its particular operations and comparisons. Printing results instead would require manual inspection and would not automatically fail a regression check.

Definition and stated intent:

Source lines 121–121:

```
121 def test_zero_switching_static_case_and_symmetry(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 122–127:

```
122 for state in ['j','0']:
123     r=evaluate(0,state)
124     self.assertAlmostEqual(r['p1'],r['p2j'])
125     self.assertAlmostEqual(r['p2j'],r['p20'])
126     static=evaluate(2,state,delta=0)
127     self.assertAlmostEqual(static['p1'],static['static_counterfactual'])
```

Read this block:

- Line 122: for each entry yielded by `['j', '0']`, bind/unpack it into `state` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 123, `r`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 124: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 125: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 126, `static`: Set this name to the result of calling `evaluate(...)`; the arguments and preceding method chain are shown above.
- Line 127: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `evaluate`: Call this project helper/callable with the displayed arguments; its own walkthrough gives the input/output contract.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Source lines 128–129:

```
128 for g in [-3,-1,1,3]:
129     self.assertAlmostEqual(evaluate(g,'j')['p2j'],evaluate(-g,'0')['p20'])
```

Read this block:

- Line 128: for each entry yielded by `[-3, -1, 1, 3]`, bind/unpack it into `g` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 129: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Source lines 131–131:

```
131 self.assertLess(evaluate(-8,'0')['p1'],0)
```

Read this block:

- Line 131: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assertLess`: Fail unless the first value is strictly smaller than the second.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

pricing_exercises/test_empirical_timing.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–8:

```
1 """Synthetic timing and backward-induction checks; no CSV input or output."""
2 import math
3 import unittest
4
5 import numpy as np
6 import pandas as pd
7
8 from pricing_exercises import run_empirical_targeting as pricing
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `math`. An as-name is a convenient local alias, not another library.
- Line 3: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 5: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 6: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 8: from `pricing_exercises`, import `run_empirical_targeting as pricing` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 11–11:

```
11 class EmpiricalTimingTests(unittest.TestCase):
```

Read this block:

- Line 11: define class `EmpiricalTimingTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 95–96:

```
95 if __name__ == "__main__":
96     unittest.main()
```

Read this block:

- Line 95: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 96: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

EmpiricalTimingTests.test first observation becomes t0

Original function: `EmpiricalTimingTests.test_first_observation_becomes_t0`. Source begins at line 12.

Checks the observed first basket initializes h0 once and future observed purchases do not leak into counterfactual initial states. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 12–12:

```
12 def test_first_observation_becomes_t0(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 13–13:

```
13 rows = []
```

Read this block:

- Line 13, `rows`: Set this name to the list `[]`.

Source lines 14–27:

```
14 for household, initial in [("a", [1., 3., 0.]), ("b", [0., 0., 0.])]:
15     for t in range(3):
16         counts = initial if t == 0 else [0., 0., 7.]
17         row = dict(household_code=household,
18                    week=pd.date_range("2020-01-04", periods=3, freq="7D")[t])
19         for j, flavor in enumerate(pricing.FLAVORS):
20             row[flavor] = counts[j]
21             row["stock_"+flavor] = 0. if t == 0 else 10.+t
22             row["price_"+flavor] = 1.+j+t
23             row["cf_"+flavor] = .1*j+t
24         # Eligibility comes from t0, even when berry is available later.
25         if household == "b" and t == 0:
26             row["price_berry"] = np.nan
27         rows.append(row)
```

Read this block:

- Line 14: for each entry yielded by `[('a', [1.0, 3.0, 0.0]), ('b', [0.0, 0.0, 0.0])]`, bind/unpack it into `(household, initial)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 15: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 16, `counts`: Set this name to the true-branch value if `t == 0` holds, otherwise the else-branch value.
- Line 17, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

- Line 19: for each entry yielded by `enumerate(pricing.FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 20, `row[flavor]`: Store the entry/slice selected by `counts[j]`; brackets select from the object to their left in the selected entry/column.
- Line 21, `row['stock_' + flavor]`: Store the true-branch value if `t == 0` holds, otherwise the else-branch value in the selected entry/column.
- Line 22, `row['price_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 23, `row['cf_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 25: test `household == 'b' and t == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26, `row['price_berry']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 27: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 28–33:

```
28 frame = pd.DataFrame(rows).sample(frac=1., random_state=7)
29 households = pd.DataFrame({"household_code": ["b", "a"]})
30 context = pricing.forecast_context(frame, households)
31 np.testing.assert_allclose(context["stocks"], [[0., 0., 0.], [.25, .75, 0.]])
32 np.testing.assert_allclose(context["novelty"], [[0., 0., 0.], [.75, .25, 1.]])
33 np.testing.assert_allclose(context["prices"], [[1., 0., 3.], [1., 2., 3.]])
```

Read this block:

- Line 28, `frame`: Set this name to the result of calling `.sample(...)`; the arguments and preceding method chain are shown above.
- Line 29, `households`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 30, `context`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 31: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 32: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 34–39:

```
34 np.testing.assert_allclose(context["cf"], [[0., .1, .2], [0., .1, .2]])
35 np.testing.assert_array_equal(context["eligible"], [False, True])
36 self.assertTrue(households.initial_observed_week.eq(pd.Timestamp("2020-01-04")).all())
37 self.assertTrue(households.pricing_week1.eq(pd.Timestamp("2020-01-11")).all())
```

```
self.assertTrue(households.pricing_week2.eq(pd.Timestamp("2020-01-18")).all())
later = frame.week > pd.Timestamp("2020-01-04")
```

Read this block:

- Line 34: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 35: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `later`: Set this name to the Boolean condition `frame.week > pd.Timestamp('2020-01-04')`; it can be a scalar or a row/array mask depending on its inputs.

Syntax and calls used here:

- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.

Source lines 40–41:

```
changed = frame.copy()
changed.loc[later, [*pricing.FLAVORS, *["stock_"+f for f in pricing.FLAVORS]]] = 999.
```

Read this block:

- Line 40, `changed`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 41, `changed.loc[later, [*pricing.FLAVORS, *['stock_' + f for f in ...]`: Store the literal value `999.0` in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 42–43:

```
for prefix in ["price_", "cf_"]:
    changed.loc[later, [prefix+f for f in pricing.FLAVORS]] = 123.
```

Read this block:

- Line 42: for each entry yielded by `['price_', 'cf_']`, bind/unpack it into `prefix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 43, `changed.loc[later, [prefix + f for f in pricing.FLAVORS]]`: Store the literal value `123.0` in the selected entry/column.

Source lines 44–44:

```
repeated = pricing.forecast_context(changed, households.copy())
```

Read this block:

- Line 44, `repeated`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.

Source lines 45–46:

```
45 for key in context:
46     np.testing.assert_array_equal(context[key], repeated[key])
```

Read this block:

- Line 45: for each entry yielded by `context`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 46: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EmpiricalTimingTests.test p1 h0 then terminal p2 h1 matches scalar enumeration

Original function: `EmpiricalTimingTests.test_p1_h0_then_terminal_p2_h1_matches_scalar_enumeration`. Source begins at line 48.

Independently enumerates basket probabilities and terminal actions, then compares the dynamic optimum and first coupon with the vectorized calculation. Split checks choose b_N and b_S outside the old fixed-lambda restriction. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 48–49:

```
48 def test_p1_h0_then_terminal_p2_h1_matches_scalar_enumeration(self):
49     # h0 is an observed berry-only basket: stock=1 and novelty for rivals=1.
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 50–61:

```
50 households = pd.DataFrame({"household_code": ["synthetic"], "group": ["low"]})
51 context = dict(stocks=np.array([[0., 1., 0.]]),
52                novelty=np.array([[1., 0., 1.]]),
53                prices=np.array([[2., 2., 2.]]),
54                available=np.ones((1, 3), dtype=bool),
55                cf=np.zeros((1, 3)), eligible=np.ones(1, dtype=bool))
56 discounts = np.linspace(0., 1., 101)
57 a, g, capacity, delta = 1., -.8, 2, .92
58 beta = np.ones(3)*.5
59 table, error = pricing.targeted_coupons(
60     context, households, np.ones((1, 1)), np.array([a]), np.array([g]),
61     beta, discounts, capacity=capacity, delta=delta)
```

Read this block:

- Line 50, `households`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 51, `context`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 56, `discounts`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.

- Line 57, `(a, g, capacity, delta)`: Unpack a tuple containing 4 displayed entries into these targets in order.
- Line 58, `beta`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 59, `(table, error)`: Unpack the result of calling `pricing.targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 62–64:

```
62 self.assertLess(error, 1e-12)
63 baskets = pricing.compositions(capacity)
64 terminal_values = []
```

Read this block:

- Line 62: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 63, `baskets`: Set this name to the result of calling `pricing.compositions(...)`; the arguments and preceding method chain are shown above.
- Line 64, `terminal_values`: Set this name to the list `[]`.

Syntax and calls used here:

- `.assertLess`: Fail unless the first value is strictly smaller than the second.

Source lines 65–75:

```
65 for basket in baskets:
66     x = basket[:3]/max(basket[:3].sum(), 1)
67     xi = .3*(x.sum()-x)-.7*(context["stocks"][0]+x)
68     values = []
69     for discount in discounts:
70         prices = context["prices"][0].copy()
71         prices[1] *= 1-discount
72         utilities = np.r_[beta-a*prices+g*xi, 0.]
73         probability = np.exp(utilities-pricing.logsumexp(utilities))
74         values.append(capacity*prices[1]*probability[1])
75     terminal_values.append(max(values))
```

Read this block:

- Line 65: for each entry yielded by `baskets`, bind/unpack it into `basket` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 66, `x`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 67, `xi`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 68, `values`: Set this name to the list `[]`.
- Line 69: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 70, `prices`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 71: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under `+=`; strings concatenate under `+=`. Read the receiving object's type.
- Line 72, `utilities`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + g * xi, 0.0]`; brackets select from the object to their left.

- Line 73, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 74: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 75: call `terminal_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 76–76:

```
76 total_values = []
```

Read this block:

- Line 76, `total_values`: Set this name to the list `[]`.

Source lines 77–87:

```
77 for discount in discounts:
78     prices = context["prices"][0].copy()
79     prices[1] *= 1-discount
80     xi = .3*context["novelty"][0]-.7*context["stocks"][0]
81     utilities = np.r_[beta-a*prices+g*xi, 0.]
82     probability = np.exp(utilities-pricing.logsumexp(utilities))
83     first = capacity*prices[1]*probability[1]
84     mass = np.array([
85         math.factorial(capacity)/math.prod(math.factorial(int(n)) for n in basket)
86         *np.prod(probability**basket) for basket in baskets])
87     total_values.append(first+delta*(mass @ terminal_values))
```

Read this block:

- Line 77: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 78, `prices`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 79: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under `+=`; strings concatenate under `+=`. Read the receiving object's type.
- Line 80, `xi`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 81, `utilities`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + g * xi, 0.0]`; brackets select from the object to their left.
- Line 82, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 83, `first`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 84, `mass`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 87: call `total_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 88–92:

```
88 policies = table.set_index("policy")
89 self.assertEqual(set(policies.index), {"no_coupon", "personalized_dynamic"})
90 dynamic = policies.loc["personalized_dynamic"]
91 self.assertEqual(dynamic.discount1, discounts[np.argmax(total_values)])
92 self.assertAlmostEqual(dynamic.discounted_revenue, max(total_values), places=12)
```

Read this block:

- Line 88, `policies`: Set this name to the result of calling `table.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 89: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 90, `dynamic`: Set this name to the entry/slice selected by `policies.loc['personalized_dynamic']`; brackets select from the object to their left.
- Line 91: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 92: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

pricing_exercises/test_split_targeting.py

Module-level imports and constants set up the functions below. Definitions are registered when imported, while function bodies wait until called. The **main** guard, if present, selects direct execution. Each function has its own input/output and syntax discussion below.

Module setup and direct-execution blocks

Source lines 1–8:

```
1 """Synthetic timing and backward-induction checks; no CSV input or output."""
2 import math
3 import unittest
4
5 import numpy as np
6 import pandas as pd
7
8 import run_split_targeting as pricing
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `math`. An as-name is a convenient local alias, not another library.
- Line 3: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 5: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 6: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 8: import `run_split_targeting as pricing`. An as-name is a convenient local alias, not another library.

Source lines 11–11:

```
11 class EmpiricalTimingTests(unittest.TestCase):
```

Read this block:

- Line 11: define class `EmpiricalTimingTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 97–97:

```
97 class SplitPosteriorTests(unittest.TestCase):
```

Read this block:

- Line 97: define class `SplitPosteriorTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 203–204:

```
203 if __name__ == "__main__":
204     unittest.main()
```

Read this block:

- Line 203: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 204: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

EmpiricalTimingTests.test first observation becomes t0

Original function: `EmpiricalTimingTests.test_first_observation_becomes_t0`. Source begins at line 12.

Checks the observed first basket initializes h0 once and future observed purchases do not leak into counterfactual initial states. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 12–12:

```
12 def test_first_observation_becomes_t0(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 13–13:

```
13 rows = []
```

Read this block:

- Line 13, `rows`: Set this name to the list `[]`.

Source lines 14–27:

```
14 for household, initial in [("a", [1., 3., 0.]), ("b", [0., 0., 0.])]:
15     for t in range(3):
16         counts = initial if t == 0 else [0., 0., 7.]
17         row = dict(household_code=household,
18                    week=pd.date_range("2020-01-04", periods=3, freq="7D")[t])
19         for j, flavor in enumerate(pricing.FLAVORS):
20             row[flavor] = counts[j]
21             row["stock_"+flavor] = 0. if t == 0 else 10.+t
22             row["price_"+flavor] = 1.+j+t
23             row["cf_"+flavor] = .1*j+t
24         # Eligibility comes from t0, even when berry is available later.
25         if household == "b" and t == 0:
26             row["price_berry"] = np.nan
27         rows.append(row)
```

Read this block:

- Line 14: for each entry yielded by `[('a', [1.0, 3.0, 0.0]), ('b', [0.0, 0.0, 0.0])]`, bind/unpack it into `(household, initial)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 15: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 16, `counts`: Set this name to the true-branch value if `t == 0` holds, otherwise the else-branch value.
- Line 17, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 19: for each entry yielded by `enumerate(pricing.FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 20, `row[flavor]`: Store the entry/slice selected by `counts[j]`; brackets select from the object to their left in the selected entry/column.
- Line 21, `row['stock_' + flavor]`: Store the true-branch value if `t == 0` holds, otherwise the else-branch value in the selected entry/column.
- Line 22, `row['price_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 23, `row['cf_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 25: test `household == 'b' and t == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26, `row['price_berry']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 27: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 28–33:

```
28 frame = pd.DataFrame(rows).sample(frac=1., random_state=7)
29 households = pd.DataFrame({"household_code": ["b", "a"]})
30 context = pricing.forecast_context(frame, households)
31 np.testing.assert_allclose(context["stocks"], [[0., 0., 0.], [.25, .75, 0.]])
32 np.testing.assert_allclose(context["novelty"], [[0., 0., 0.], [.75, .25, 1.]])
33 np.testing.assert_allclose(context["prices"], [[1., 0., 3.], [1., 2., 3.]])
```

Read this block:

- Line 28, `frame`: Set this name to the result of calling `.sample(...)`; the arguments and preceding method chain are shown above.
- Line 29, `households`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 30, `context`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 31: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 32: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 34–39:

```
np.testing.assert_allclose(context["cf"], [[0., .1, .2], [0., .1, .2]])
np.testing.assert_array_equal(context["eligible"], [False, True])
self.assertTrue(households.initial_observed_week.eq(pd.Timestamp("2020-01-04")).all())
self.assertTrue(households.pricing_week1.eq(pd.Timestamp("2020-01-11")).all())
self.assertTrue(households.pricing_week2.eq(pd.Timestamp("2020-01-18")).all())
later = frame.week > pd.Timestamp("2020-01-04")
```

Read this block:

- Line 34: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 35: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `later`: Set this name to the Boolean condition `frame.week > pd.Timestamp('2020-01-04')`; it can be a scalar or a row/array mask depending on its inputs.

Syntax and calls used here:

- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.

Source lines 40–41:

```
changed = frame.copy()
changed.loc[later, [*pricing.FLAVORS, *["stock_"+f for f in pricing.FLAVORS]]] = 999.
```

Read this block:

- Line 40, `changed`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.

- Line 41, `changed.loc[later, [*pricing.FLAVORS, *['stock_' + f for f in ...]`: Store the literal value `999.0` in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 42–43:

```
42 for prefix in ["price_", "cf_"]:
43     changed.loc[later, [prefix+f for f in pricing.FLAVORS]] = 123.
```

Read this block:

- Line 42: for each entry yielded by `['price_', 'cf_']`, bind/unpack it into `prefix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 43, `changed.loc[later, [prefix + f for f in pricing.FLAVORS]]`: Store the literal value `123.0` in the selected entry/column.

Source lines 44–44:

```
44 repeated = pricing.forecast_context(changed, households.copy())
```

Read this block:

- Line 44, `repeated`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.

Source lines 45–46:

```
45 for key in context:
46     np.testing.assert_array_equal(context[key], repeated[key])
```

Read this block:

- Line 45: for each entry yielded by `context`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 46: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EmpiricalTimingTests.test p1 h0 then terminal p2 h1 matches scalar enumeration

Original function: `EmpiricalTimingTests.test_p1_h0_then_terminal_p2_h1_matches_scalar_enumeration`. Source begins at line 48.

Independently enumerates basket probabilities and terminal actions, then compares the dynamic optimum and first coupon with the vectorized calculation. Split checks choose b_N and b_S outside the old fixed-lambda restriction. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 48–49:

```
48 def test_p1_h0_then_terminal_p2_h1_matches_scalar_enumeration(self):
49     # h0 is an observed berry-only basket: stock=1 and novelty for rivals=1.
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 50–59:

```
50 households = pd.DataFrame({"household_code": ["synthetic"], "group": ["low"]})
51 context = dict(stocks=np.array([[0., 1., 0.]]),
52                novelty=np.array([[1., 0., 1.]]),
53                prices=np.array([[2., 2., 2.]]),
54                available=np.ones((1, 3), dtype=bool),
55                cf=np.zeros((1, 3)), eligible=np.ones(1, dtype=bool))
56 discounts = np.linspace(0., 1., 101)
57 a, g, capacity, delta = 1., -.8, 2, .92
58 b_n, b_s = -.6, .35
59 beta = np.ones(3)*.5
```

Read this block:

- Line 50, `households`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 51, `context`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 56, `discounts`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 57, `(a, g, capacity, delta)`: Unpack a tuple containing 4 displayed entries into these targets in order.
- Line 58, `(b_n, b_s)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 59, `beta`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 60–65:

```
60 table, error = pricing.targeted_coupons(
61     context, households, np.ones((1, 1)), np.array([a]), np.array([[b_n, b_s]]),
62     beta, discounts, capacity=capacity, delta=delta)
63 self.assertLess(error, 1e-12)
64 baskets = pricing.compositions(capacity)
65 terminal_values = []
```

Read this block:

- Line 60, `(table, error)`: Unpack the result of calling `pricing.targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 63: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64, `baskets`: Set this name to the result of calling `pricing.compositions(...)`; the arguments and preceding method chain are shown above.
- Line 65, `terminal_values`: Set this name to the list `[]`.

Syntax and calls used here:

- `.assertLess`: Fail unless the first value is strictly smaller than the second.

Source lines 66–76:

```
66 for basket in baskets:
67     x = basket[:3]/max(basket[:3].sum(), 1)
```

```
    history = b_n*(x.sum()-x)+b_s*(context["stocks"][0]+x)
    values = []
    for discount in discounts:
        prices = context["prices"][0].copy()
        prices[1] *= 1-discount
        utilities = np.r_[beta-a*prices+history, 0.]
        probability = np.exp(utilities-pricing.logsumexp(utilities))
        values.append(capacity*prices[1]*probability[1])
    terminal_values.append(max(values))
```

Read this block:

- Line 66: for each entry yielded by `baskets`, bind/unpack it into `basket` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 67, `x`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 68, `history`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 69, `values`: Set this name to the list `[]`.
- Line 70: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 71, `prices`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 72: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under `+=`; strings concatenate under `+=`. Read the receiving object's type.
- Line 73, `utilities`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + history, 0.0]`; brackets select from the object to their left.
- Line 74, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 75: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 76: call `terminal_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 77–77:

```
total_values = []
```

Read this block:

- Line 77, `total_values`: Set this name to the list `[]`.

Source lines 78–88:

```
for discount in discounts:
    prices = context["prices"][0].copy()
    prices[1] *= 1-discount
    history = b_n*context["novelty"][0]+b_s*context["stocks"][0]
    utilities = np.r_[beta-a*prices+history, 0.]
    probability = np.exp(utilities-pricing.logsumexp(utilities))
```

```
    first = capacity*prices[1]*probability[1]
    mass = np.array([
        math.factorial(capacity)/math.prod(math.factorial(int(n)) for n in basket)
        *np.prod(probability**basket) for basket in baskets])
    total_values.append(first+delta*(mass @ terminal_values))
```

Read this block:

- Line 78: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 79, `prices`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 80: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 81, `history`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 82, `utilities`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + history, 0.0]`; brackets select from the object to their left.
- Line 83, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 84, `first`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 85, `mass`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 88: call `total_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 89–93:

```
policies = table.set_index("policy")
self.assertEqual(set(policies.index), {"no_coupon", "personalized_dynamic"})
dynamic = policies.loc["personalized_dynamic"]
self.assertEqual(dynamic.discount1, discounts[np.argmax(total_values)])
self.assertAlmostEqual(dynamic.discounted_revenue, max(total_values), places=12)
```

Read this block:

- Line 89, `policies`: Set this name to the result of calling `table.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 90: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91, `dynamic`: Set this name to the entry/slice selected by `policies.loc['personalized_dynamic']`; brackets select from the object to their left.
- Line 92: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 93: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SplitPosteriorTests.fixture

Original function: `SplitPosteriorTests.fixture`. Source begins at line 98.

Constructs tiny, explicitly generated household histories and coefficient dictionaries for checks. These values are inputs chosen to expose errors, not estimates from the private sample.

Definition and stated intent:

Source lines 98–98:

```
98 def fixture(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 99–99:

```
99 rows = []
```

Read this block:

- Line 99, `rows`: Set this name to the list `[]`.

Source lines 100–112:

```
100 for i in range(8):
101     stock = np.zeros(3)
102     previous = np.zeros(3)
103     for t in range(3):
104         counts = np.eye(3)[(i+t) % 3]
105         novelty = previous.sum()-previous
106         row = dict(household_code=str(i), week=pd.Timestamp('2020-01-04')+pd.Timedelta(weeks=t), outside=0.)
107         for j, f in enumerate(pricing.FLAVORS):
108             row.update({f: counts[j], 'price_'+f: 1.+j/2,
109                         'novelty_'+f: novelty[j], 'stock_'+f: stock[j]})
110         rows.append(row)
111         stock += counts
112         previous = counts
```

Read this block:

- Line 100: for each entry yielded by `range(8)`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 101, `stock`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 102, `previous`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 103: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 104, `counts`: Set this name to the entry/slice selected by `np.eye(3)[(i + t) % 3]`; brackets select from the object to their left.
- Line 105, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.

- Line 106, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 107: for each entry yielded by `enumerate(pricing.FLAVORS)`, bind/unpack it into `(j, f)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 108: call `row.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 110: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 111: update `stock` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 112, `previous`: Set this name to the object currently named `counts`; plain assignment does not copy it.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `str`: Convert to text for paths, messages, or serialization.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 113–115:

```
113 parameters = dict(inside_constant=.5, beta_berry=.2, beta_plain=-.1,
114                   mu_log_minus_alpha=-.3, sd_log_minus_alpha=.4,
115                   mean_novelty=-.2, sd_novelty=.7, mean_own_stock=.1, sd_own_stock=.3)
```

Read this block:

- Line 113, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Source lines 116–116:

```
116 return pd.DataFrame(rows), parameters
```

Read this block:

- Line 116: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

SplitPosteriorTests.test posterior matches scalar and estimator

Original function: `SplitPosteriorTests.test_posterior_matches_scalar_and_estimator`. Source begins at line 118.

Compares persistent split-draw likelihoods and posterior weights against a scalar calculation and the demand estimator, and verifies exact binary aggregation by an outside inclusive value. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 118–118:

```
118 def test_posterior_matches_scalar_and_estimator(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 119–122:

```
frame, p = self.fixture()
draws = pricing.persistent_draws(8, 306)
households, weights, alpha, history, beta, nll = pricing.posterior(frame, p, draws, False)
scalar = []
```

Read this block:

- Line 119, `(frame, p)`: Unpack the result of calling `self.fixture(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 120, `draws`: Set this name to the result of calling `pricing.persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 121, `(households, weights, alpha, history, beta, nll)`: Unpack the result of calling `pricing.posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 122, `scalar`: Set this name to the list `[]`.

Source lines 123–133:

```
for _, block in frame.groupby('household_code', sort=True):
    logs = []
    for a, (b_n, b_s) in zip(alpha, history):
        ll = 0.
        for _, row in block.iterrows():
            u = [beta[j]-a*row['price_'+f]+b_n*row['novelty_'+f]+b_s*row['stock_'+f]
                 for j, f in enumerate(pricing.FLAVORS)]+[0.]
            probs = np.exp(u-pricing.logsumexp(u))
            ll += sum(row[f]*np.log(probs[j]) for j, f in enumerate(pricing.FLAVORS+['outside']))
        logs.append(ll)
    scalar.append(logs)
```

Read this block:

- Line 123: for each entry yielded by `frame.groupby('household_code', sort=True)`, bind/unpack it into `(_, block)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 124, `logs`: Set this name to the list `[]`.
- Line 125: for each entry yielded by `zip(alpha, history)`, bind/unpack it into `(a, (b_n, b_s))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 126, `ll`: Set this name to the literal value `0.0`.
- Line 127: for each entry yielded by `block.iterrows()`, bind/unpack it into `(_, row)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 128, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 130, `probs`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 131: update `ll` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 132: call `logs.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133: call `scalar.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.

- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 134–139:

```
scalar = np.array(scalar)
expected = np.exp(scalar-pricing.logsumexp(scalar, axis=1, keepdims=True))
np.testing.assert_allclose(weights, expected, atol=1e-13)
self.assertAlmostEqual(nll, -sum(pricing.logsumexp(row)-np.log(len(draws)) for row in scalar), places=12)
np.testing.assert_allclose(households.habit_mean, weights @ history[:, 1])
import sys
```

Read this block:

- Line 134, `scalar`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 135, `expected`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 136: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 137: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 138: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 139: import `sys`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `len`: Count elements/rows rather than summing their numerical values.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 140–142:

```
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1]/"est_code"))
import est
```

Read this block:

- Line 140: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 141: call `sys.path.insert` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 142: import `est`. An as-name is a convenient local alias, not another library.

Syntax and calls used here:

- `str`: Convert to text for paths, messages, or serialization.
- `.resolve`: Normalize a path to an absolute resolved location before locating siblings or comparing paths.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.

Source lines 143–144:

```
for f in pricing.FLAVORS:
    frame["xi_"+f] = .3*frame["novelty_"+f]-.7*frame["stock_"+f]
```

Read this block:

- Line 143: for each entry yielded by `pricing.FLAVORS`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 144, `frame['xi_' + f]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Source lines 145–152:

```
data = est.make_arrays(frame, 1)
names = est.parameter_names("heterogeneous_history", True, False)
estimator_draws = est.make_draws(8, 306, "heterogeneous_history")
np.testing.assert_array_equal(draws, estimator_draws)
expected_ll = est.household_loglike(np.array([p[n] for n in names]), data,
                                    estimator_draws, "heterogeneous_history")
np.testing.assert_allclose(pricing.logsumexp(scalar, axis=1)-np.log(len(draws)),
                           expected_ll, atol=1e-12)
```

Read this block:

- Line 145, `data`: Set this name to the result of calling `est.make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 146, `names`: Set this name to the result of calling `est.parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 147, `estimator_draws`: Set this name to the result of calling `est.make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 148: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 149, `expected_ll`: Set this name to the result of calling `est.household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 151: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.

Source lines 155–156:

```
u = np.array([.3, -.2, .7, 0.])
probs = np.exp(u-pricing.logsumexp(u))
```

Read this block:

- Line 155, `u`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 156, `probs`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Source lines 157–159:

```
for focal in range(3):
    inclusive = pricing.logsumexp(np.delete(u, focal))
    self.assertAlmostEqual(pricing.expit(u[focal]-inclusive), probs[focal])
```

Read this block:

- Line 157: for each entry yielded by `range(3)`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 158, `inclusive`: Set this name to the result of calling `pricing.logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 159: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SplitPosteriorTests.test all focal exports and integrated types

Original function: `SplitPosteriorTests.test_all_focal_exports_and_integrated_types`. Source begins at line 161.

Runs all focal flavors under both type treatments with generated temporary inputs and verifies policy dominance, likelihood guards, outputs and the all-flavor entry point. This complete export check reads only its own synthetic CSVs. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 161–161:

```
161 def test_all_focal_exports_and_integrated_types(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 162–167:

```
162 import tempfile
163 import json
164 from pathlib import Path
165 from types import SimpleNamespace
166 frame, p = self.fixture()
167 _, _, _, _, _, nll = pricing.posterior(frame, p, pricing.persistent_draws(8, 306), False)
```

Read this block:

- Line 162: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 163: import `json`. An as-name is a convenient local alias, not another library.
- Line 164: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 165: from `types`, import `SimpleNamespace` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 166, `(frame, p)`: Unpack the result of calling `self.fixture(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 167, `(_, _, _, _, _, nll)`: Unpack the result of calling `pricing.posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 168–199:

```
168 with tempfile.TemporaryDirectory() as directory:
169     directory = Path(directory)
170     frame.to_csv(directory/'weekly_inputs.csv', index=False)
171     pd.DataFrame(dict(parameter=list(p), estimate=list(p.values()),
    ↪   at_bound=False)).to_csv(directory/'point_estimates.csv', index=False)
172     (directory/'fit.json').write_text(json.dumps(dict(model='heterogeneous_history', converged=True,
173         nll=nll, weeks=len(frame), households=8,
174         settings=dict(capacity=1, no_price_control=True, draws=8, seed=306))))
```

```
    for focal in pricing.FLAVORS:
        for treatment in ['posterior-mean', 'posterior-integrated']:
            output = directory/focal/treatment
            output.mkdir(parents=True)
            args = SimpleNamespace(fit_dir=directory, household_input=directory/'weekly_inputs.csv',
                output=output, focal_flavor=focal, discounts=np.array([0., .5, 1.]),
                demand_delta=.95, batch_size=3, type_treatment=treatment, save_state_policies=True)
            report = pricing.run_households(args)
            self.assertLess(report['basket_mass_error'], 1e-12)
            result = pd.read_csv(output/'targeted_household_results.csv')
            self.assertTrue((result.revenue_change >= -1e-10).all())
            self.assertEqual(set(result.focal_flavor), {focal})
            self.assertIn('focal_units1', result)
    from unittest.mock import patch
    import sys
    output = directory/'all_focal_cli'
    with patch.object(sys, 'argv', ['run_split_targeting.py', '--household-input',
            str(directory/'weekly_inputs.csv'), '--fit-dir', str(directory),
            '--output', str(output), '--discounts', '0', '.5', '1']), \
            patch.object(pricing, 'make_targeting_figures'):
        pricing.main()
    comparison = pd.read_csv(output/'focal_flavor_comparison.csv')
    self.assertEqual(set(comparison.focal_flavor), set(pricing.FLAVORS))
    manifest = json.loads((output/'run_manifest.json').read_text())
    self.assertEqual(set(manifest['results']), set(pricing.FLAVORS))
```

Read this block:

- Line 168: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 169, `directory`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 170: call `frame.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 171: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 172: call `(directory / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 175: for each entry yielded by `pricing.FLAVORS`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 176: for each entry yielded by `['posterior-mean', 'posterior-integrated']`, bind/unpack it into `treatment` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 177, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 178: call `output.mkdir` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 179, `args`: Set this name to the result of calling `SimpleNamespace(...)`; the arguments and preceding method chain are shown above.
- Line 182, `report`: Set this name to the result of calling `pricing.run_households(...)`; the arguments and preceding method chain are shown above.
- Line 183: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 184, `result`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 185: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 186: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

- Line 187: call `self.assertIn` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 188: from `unittest.mock`, import `patch` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 189: import `sys`. An as-name is a convenient local alias, not another library.
- Line 190, `output`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 191: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 195: call `pricing.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 196, `comparison`: Set this name to the result of calling `pd.read_csv(...)`; the arguments and preceding method chain are shown above.
- Line 197: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 198, `manifest`: Set this name to the result of calling `json.loads(...)`; the arguments and preceding method chain are shown above.
- Line 199: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.read_csv`: Read a CSV into an eager table; unlike a lazy scan, the requested data are loaded now.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.mkdir`: Create a directory; parents=True creates ancestors and exist_ok=True accepts an existing directory.
- `SimpleNamespace`: Construct a lightweight object with named attributes, useful for args-like synthetic test settings.
- `.assertLess`: Fail unless the first value is strictly smaller than the second.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.
- `str`: Convert to text for paths, messages, or serialization.
- `len`: Count elements/rows rather than summing their numerical values.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `list`: Materialize iterable entries as an ordered Python list.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/checks/test_split_pricing.py

Generated-array checks of split posterior likelihood, exact binary aggregation, timing, independent-coefficient backward induction, and all focal cases under both type treatments. The final_simple check runner blocks CSV readers.

Module setup and direct-execution blocks

Source lines 1–8:

```
"""Synthetic timing and backward-induction checks; no CSV input or output."""
import math
import unittest

import numpy as np
import pandas as pd

from demand import split_pricing as pricing
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `math`. An as-name is a convenient local alias, not another library.
- Line 3: import `unittest`. An as-name is a convenient local alias, not another library.
- Line 5: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 6: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 8: from `demand`, import `split_pricing as pricing` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 11–11:

```
class EmpiricalTimingTests(unittest.TestCase):
```

Read this block:

- Line 11: define class `EmpiricalTimingTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 97–97:

```
class SplitPosteriorTests(unittest.TestCase):
```

Read this block:

- Line 97: define class `SplitPosteriorTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 178–179:

```
if __name__ == "__main__":
    unittest.main()
```

Read this block:

- Line 178: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 179: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

EmpiricalTimingTests.test first observation becomes t0

Original function: `EmpiricalTimingTests.test_first_observation_becomes_t0`. Source begins at line 12.

Checks the observed first basket initializes h0 once and future observed purchases do not leak into counterfactual initial states. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 12–12:

```
def test_first_observation_becomes_t0(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 13–13:

```
13 rows = []
```

Read this block:

- Line 13, `rows`: Set this name to the list `[]`.

Source lines 14–27:

```
14 for household, initial in [("a", [1., 3., 0.]), ("b", [0., 0., 0.])]:
15     for t in range(3):
16         counts = initial if t == 0 else [0., 0., 7.]
17         row = dict(household_code=household,
18                    week=pd.date_range("2020-01-04", periods=3, freq="7D")[t])
19         for j, flavor in enumerate(pricing.FLAVORS):
20             row[flavor] = counts[j]
21             row["stock_"+flavor] = 0. if t == 0 else 10.+t
22             row["price_"+flavor] = 1.+j+t
23             row["cf_"+flavor] = .1*j+t
24         # Eligibility comes from t0, even when berry is available later.
25         if household == "b" and t == 0:
26             row["price_berry"] = np.nan
27         rows.append(row)
```

Read this block:

- Line 14: for each entry yielded by `[('a', [1.0, 3.0, 0.0]), ('b', [0.0, 0.0, 0.0])]`, bind/unpack it into `(household, initial)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 15: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 16, `counts`: Set this name to the true-branch value if `t == 0` holds, otherwise the else-branch value.
- Line 17, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 19: for each entry yielded by `enumerate(pricing.FLAVORS)`, bind/unpack it into `(j, flavor)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 20, `row[flavor]`: Store the entry/slice selected by `counts[j]`; brackets select from the object to their left in the selected entry/column.
- Line 21, `row['stock_' + flavor]`: Store the true-branch value if `t == 0` holds, otherwise the else-branch value in the selected entry/column.
- Line 22, `row['price_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 23, `row['cf_' + flavor]`: Store addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.
- Line 25: test `household == 'b' and t == 0`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 26, `row['price_berry']`: Store the attribute `np.nan` of the object before the dot in the selected entry/column.
- Line 27: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.

- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Source lines 28–33:

```
frame = pd.DataFrame(rows).sample(frac=1., random_state=7)
households = pd.DataFrame({"household_code": ["b", "a"]})
context = pricing.forecast_context(frame, households)
np.testing.assert_allclose(context["stocks"], [[0., 0., 0.], [.25, .75, 0.]])
np.testing.assert_allclose(context["novelty"], [[0., 0., 0.], [.75, .25, 1.]])
np.testing.assert_allclose(context["prices"], [[1., 0., 3.], [1., 2., 3.]])
```

Read this block:

- Line 28, `frame`: Set this name to the result of calling `.sample(...)`; the arguments and preceding method chain are shown above.
- Line 29, `households`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 30, `context`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 31: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 32: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 33: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.

Source lines 34–39:

```
np.testing.assert_allclose(context["cf"], [[0., .1, .2], [0., .1, .2]])
np.testing.assert_array_equal(context["eligible"], [False, True])
self.assertTrue(households.initial_observed_week.eq(pd.Timestamp("2020-01-04")).all())
self.assertTrue(households.pricing_week1.eq(pd.Timestamp("2020-01-11")).all())
self.assertTrue(households.pricing_week2.eq(pd.Timestamp("2020-01-18")).all())
later = frame.week > pd.Timestamp("2020-01-04")
```

Read this block:

- Line 34: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 35: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 38: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `later`: Set this name to the Boolean condition `frame.week > pd.Timestamp('2020-01-04')`; it can be a scalar or a row/array mask depending on its inputs.

Syntax and calls used here:

- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

- `.eq`: Compare elementwise for equality, the method equivalent of == for these pandas values.

Source lines 40–41:

```
changed = frame.copy()
changed.loc[later, [*pricing.FLAVORS, *["stock_"+f for f in pricing.FLAVORS]]] = 999.
```

Read this block:

- Line 40, `changed`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 41, `changed.loc[later, [*pricing.FLAVORS, *['stock_' + f for f in ...]`: Store the literal value `999.0` in the selected entry/column.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- A starred entry expands an iterable into separate elements, here within a list/tuple or call.

Source lines 42–43:

```
for prefix in ["price_", "cf_"]:
    changed.loc[later, [prefix+f for f in pricing.FLAVORS]] = 123.
```

Read this block:

- Line 42: for each entry yielded by `['price_', 'cf_']`, bind/unpack it into `prefix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 43, `changed.loc[later, [prefix + f for f in pricing.FLAVORS]]`: Store the literal value `123.0` in the selected entry/column.

Source lines 44–44:

```
repeated = pricing.forecast_context(changed, households.copy())
```

Read this block:

- Line 44, `repeated`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.

Source lines 45–46:

```
for key in context:
    np.testing.assert_array_equal(context[key], repeated[key])
```

Read this block:

- Line 45: for each entry yielded by `context`, bind/unpack it into `key` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 46: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

EmpiricalTimingTests.test p1 h0 then terminal p2 h1 matches scalar enumeration

Original function: `EmpiricalTimingTests.test_p1_h0_then_terminal_p2_h1_matches_scalar_enumeration`. Source begins at line 48.

Independently enumerates basket probabilities and terminal actions, then compares the dynamic optimum and first coupon with the vectorized calculation. Split checks choose b_N and b_S outside the old fixed-lambda

restriction. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 48–49:

```
48 def test_p1_h0_then_terminal_p2_h1_matches_scalar_enumeration(self):
49     # h0 is an observed berry-only basket: stock=1 and novelty for rivals=1.
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 50–59:

```
50 households = pd.DataFrame({"household_code": ["synthetic"], "group": ["low"]})
51 context = dict(stocks=np.array([[0., 1., 0.]]),
52                novelty=np.array([[1., 0., 1.]]),
53                prices=np.array([[2., 2., 2.]]),
54                available=np.ones((1, 3), dtype=bool),
55                cf=np.zeros((1, 3)), eligible=np.ones(1, dtype=bool))
56 discounts = np.linspace(0., 1., 101)
57 a, g, capacity, delta = 1., -.8, 2, .92
58 b_n, b_s = -.6, .35
59 beta = np.ones(3)*.5
```

Read this block:

- Line 50, `households`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 51, `context`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 56, `discounts`: Set this name to the result of calling `np.linspace(...)`; the arguments and preceding method chain are shown above.
- Line 57, `(a, g, capacity, delta)`: Unpack a tuple containing 4 displayed entries into these targets in order.
- Line 58, `(b_n, b_s)`: Unpack a tuple containing 2 displayed entries into these targets in order.
- Line 59, `beta`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.

Syntax and calls used here:

- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `.linspace`: Create the requested number of evenly spaced points, normally including both endpoints.

Source lines 60–65:

```
60 table, error = pricing.targeted_coupons(
61     context, households, np.ones((1, 1)), np.array([a]), np.array([[b_n, b_s]]),
62     beta, discounts, capacity=capacity, delta=delta)
63 self.assertLess(error, 1e-12)
64 baskets = pricing.compositions(capacity)
65 terminal_values = []
```

Read this block:

- Line 60, `(table, error)`: Unpack the result of calling `pricing.targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 63: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 64, `baskets`: Set this name to the result of calling `pricing.compositions(...)`; the arguments and preceding method chain are shown above.

- Line 65, `terminal_values`: Set this name to the list `[]`.

Syntax and calls used here:

- `.assertLess`: Fail unless the first value is strictly smaller than the second.

Source lines 66–76:

```
66 for basket in baskets:
67     x = basket[:3]/max(basket[:3].sum(), 1)
68     history = b_n*(x.sum()-x)+b_s*(context["stocks"][0]+x)
69     values = []
70     for discount in discounts:
71         prices = context["prices"][0].copy()
72         prices[1] *= 1-discount
73         utilities = np.r_[beta-a*prices+history, 0.]
74         probability = np.exp(utilities-pricing.logsumexp(utilities))
75         values.append(capacity*prices[1]*probability[1])
76     terminal_values.append(max(values))
```

Read this block:

- Line 66: for each entry yielded by `baskets`, bind/unpack it into `basket` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 67, `x`: Set this name to division of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 68, `history`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 69, `values`: Set this name to the list `[]`.
- Line 70: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 71, `prices`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 72: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under `+=`; strings concatenate under `+=`. Read the receiving object's type.
- Line 73, `utilities`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + history, 0.0]`; brackets select from the object to their left.
- Line 74, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 75: call `values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 76: call `terminal_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `max`: Select the largest entry, used for widths, tolerances, and diagnostic residuals.
- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.

Source lines 77–77:

```
77 total_values = []
```

Read this block:

- Line 77, `total_values`: Set this name to the list `[]`.

Source lines 78–88:

```
for discount in discounts:
    prices = context["prices"][0].copy()
    prices[1] *= 1-discount
    history = b_n*context["novelty"][0]+b_s*context["stocks"][0]
    utilities = np.r_[beta-a*prices+history, 0.]
    probability = np.exp(utilities-pricing.logsumexp(utilities))
    first = capacity*prices[1]*probability[1]
    mass = np.array([
        math.factorial(capacity)/math.prod(math.factorial(int(n)) for n in basket)
        *np.prod(probability**basket) for basket in baskets])
    total_values.append(first+delta*(mass @ terminal_values))
```

Read this block:

- Line 78: for each entry yielded by `discounts`, bind/unpack it into `discount` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 79, `prices`: Set this name to the result of calling `context['prices'][0].copy(...)`; the arguments and preceding method chain are shown above.
- Line 80: update `prices[1]` using `*=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 81, `history`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 82, `utilities`: Set this name to the entry/slice selected by `np.r_[beta - a * prices + history, 0.0]`; brackets select from the object to their left.
- Line 83, `probability`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 84, `first`: Set this name to multiplication of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 85, `mass`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 88: call `total_values.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `int`: Convert to an integer; it does not by itself validate that a source value was already integral.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.
- The ** operator raises to a power. Python's ^ operator would mean bitwise XOR instead.

Source lines 89–93:

```
policies = table.set_index("policy")
self.assertEqual(set(policies.index), {"no_coupon", "personalized_dynamic"})
dynamic = policies.loc["personalized_dynamic"]
self.assertEqual(dynamic.discount1, discounts[np.argmax(total_values)])
self.assertAlmostEqual(dynamic.discounted_revenue, max(total_values), places=12)
```

Read this block:

- Line 89, `policies`: Set this name to the result of calling `table.set_index(...)`; the arguments and preceding method chain are shown above.
- Line 90: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 91, `dynamic`: Set this name to the entry/slice selected by `policies.loc['personalized_dynamic']`; brackets select from the object to their left.

- Line 92: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 93: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.set_index`: Use one or more columns as pandas row labels, making named lookup convenient.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `set`: Build a collection of distinct values, used for column membership and missing-field checks.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SplitPosteriorTests.fixture

Original function: `SplitPosteriorTests.fixture`. Source begins at line 98.

Constructs tiny, explicitly generated household histories and coefficient dictionaries for checks. These values are inputs chosen to expose errors, not estimates from the private sample.

Definition and stated intent:

Source lines 98–98:

```
98 def fixture(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 99–99:

```
99 rows = []
```

Read this block:

- Line 99, `rows`: Set this name to the list `[]`.

Source lines 100–112:

```
100 for i in range(8):
101     stock = np.zeros(3)
102     previous = np.zeros(3)
103     for t in range(3):
104         counts = np.eye(3)[(i+t) % 3]
105         novelty = previous.sum()-previous
106         row = dict(household_code=str(i), week=pd.Timestamp('2020-01-04')+pd.Timedelta(weeks=t), outside=0.)
107         for j, f in enumerate(pricing.FLAVORS):
108             row.update({f: counts[j], 'price_'+f: 1.+j/2,
109                         'novelty_'+f: novelty[j], 'stock_'+f: stock[j]})
110         rows.append(row)
111         stock += counts
112         previous = counts
```

Read this block:

- Line 100: for each entry yielded by `range(8)`, bind/unpack it into `i` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 101, `stock`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.
- Line 102, `previous`: Set this name to the result of calling `np.zeros(...)`; the arguments and preceding method chain are shown above.

- Line 103: for each entry yielded by `range(3)`, bind/unpack it into `t` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 104, `counts`: Set this name to the entry/slice selected by `np.eye(3)[(i + t) % 3]`; brackets select from the object to their left.
- Line 105, `novelty`: Set this name to subtraction of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 106, `row`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 107: for each entry yielded by `enumerate(pricing.FLAVORS)`, bind/unpack it into `(j, f)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 108: call `row.update` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 110: call `rows.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 111: update `stock` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 112, `previous`: Set this name to the object currently named `counts`; plain assignment does not copy it.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.
- `.zeros`: Allocate zero-filled numerical storage of the requested shape before accumulation.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.sum`: Sum values along the specified axis or within each group; check the receiver and axis to identify the unit.
- `.update`: Add/replace dictionary entries in place; unrelated entries remain available.
- `str`: Convert to text for paths, messages, or serialization.
- The % operator takes a remainder. It is used for parity, repeating schedules, calendar offsets, or hash buckets.

Source lines 113–115:

```
113 parameters = dict(inside_constant=.5, beta_berry=.2, beta_plain=-.1,
114                   mu_log_minus_alpha=-.3, sd_log_minus_alpha=.4,
115                   mean_novelty=-.2, sd_novelty=.7, mean_own_stock=.1, sd_own_stock=.3)
```

Read this block:

- Line 113, `parameters`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.

Source lines 116–116:

```
116 return pd.DataFrame(rows), parameters
```

Read this block:

- Line 116: return a tuple of 2 results. The caller can unpack them; return ends the call, unlike print.

SplitPosteriorTests.test posterior matches scalar and estimator

Original function: `SplitPosteriorTests.test_posterior_matches_scalar_and_estimator`. Source begins at line 118.

Compares persistent split-draw likelihoods and posterior weights against a scalar calculation and the demand estimator, and verifies exact binary aggregation by an outside inclusive value. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 118–118:

```
def test_posterior_matches_scalar_and_estimator(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 119–122:

```
frame, p = self.fixture()
draws = pricing.persistent_draws(8, 306)
households, weights, alpha, history, beta, nll = pricing.posterior(frame, p, draws, False)
scalar = []
```

Read this block:

- Line 119, `(frame, p)`: Unpack the result of calling `self.fixture(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 120, `draws`: Set this name to the result of calling `pricing.persistent_draws(...)`; the arguments and preceding method chain are shown above.
- Line 121, `(households, weights, alpha, history, beta, nll)`: Unpack the result of calling `pricing.posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 122, `scalar`: Set this name to the list `[]`.

Source lines 123–133:

```
for _, block in frame.groupby('household_code', sort=True):
    logs = []
    for a, (b_n, b_s) in zip(alpha, history):
        ll = 0.
        for _, row in block.iterrows():
            u = [beta[j]-a*row['price_'+f]+b_n*row['novelty_'+f]+b_s*row['stock_'+f]
                 for j, f in enumerate(pricing.FLAVORS)]+[0.]
            probs = np.exp(u-pricing.logsumexp(u))
            ll += sum(row[f]*np.log(probs[j]) for j, f in enumerate(pricing.FLAVORS+['outside']))
        logs.append(ll)
    scalar.append(logs)
```

Read this block:

- Line 123: for each entry yielded by `frame.groupby('household_code', sort=True)`, bind/unpack it into `(_, block)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 124, `logs`: Set this name to the list `[]`.
- Line 125: for each entry yielded by `zip(alpha, history)`, bind/unpack it into `(a, (b_n, b_s))` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 126, `ll`: Set this name to the literal value `0.0`.
- Line 127: for each entry yielded by `block.iterrows()`, bind/unpack it into `(_, row)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 128, `u`: Set this name to addition/concatenation of the displayed operands; parentheses control which subexpressions are evaluated together.
- Line 130, `probs`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 131: update `ll` using `+=`. Numeric arrays update elementwise; lists extend under +=; strings concatenate under +=. Read the receiving object's type.
- Line 132: call `logs.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 133: call `scalar.append` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.groupby`: Partition pandas rows by key values; the following aggregation/transform specifies what each group produces.
- `zip`: Pair entries from sequences in order; by default iteration stops at the shortest input.
- `.append`: Add one item to the end of a Python list; it mutates the list and normally returns None.
- `.exp`: Apply the exponential, elementwise for NumPy and symbolically for SymPy; it is not multiplication by a constant.
- `sum`: Accumulate a sequence, including generated chain-rule terms or counts.
- `enumerate`: Yield position/value pairs, allowing an index and label to be named together.
- `.log`: Take the natural logarithm, used here for log likelihoods and integration normalization.
- The bracketed for-expression is a list comprehension: one computed entry per loop item, optionally filtered by its final if.
- The parenthesized for-expression yields items as requested; it does not first allocate a complete list.

Source lines 134–139:

```
134 scalar = np.array(scalar)
135 expected = np.exp(scalar-pricing.logsumexp(scalar, axis=1, keepdims=True))
136 np.testing.assert_allclose(weights, expected, atol=1e-13)
137 self.assertAlmostEqual(nll, -sum(pricing.logsumexp(row)-np.log(len(draws)) for row in scalar), places=12)
138 np.testing.assert_allclose(households.habit_mean, weights @ history[:, 1])
139 from demand import estimation as est
```

Read this block:

- Line 134, `scalar`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 135, `expected`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.
- Line 136: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 137: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 138: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 139: from `demand`, import `estimation as est` into this module's namespace. Importing can execute the imported module's top-level setup.

Syntax and calls used here:

- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.assert_allclose`: Compare arrays numerically with tolerances rather than requiring bit-for-bit floating-point equality.
- `.assertAlmostEqual`: Fail unless two numbers agree to the requested decimal places or absolute delta.
- `len`: Count elements/rows rather than summing their numerical values.
- The @ operator is matrix multiplication/dot product. It sums matching products; * would multiply elementwise.

Source lines 140–141:

```
140 for f in pricing.FLAVORS:
141     frame["xi_"+f] = .3*frame["novelty_"+f]-.7*frame["stock_"+f]
```

Read this block:

- Line 140: for each entry yielded by `pricing.FLAVORS`, bind/unpack it into `f` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 141, `frame['xi_' + f]`: Store subtraction of the displayed operands; parentheses control which subexpressions are evaluated together in the selected entry/column.

Source lines 142–149:

```
142 data = est.make_arrays(frame, 1)
143 names = est.parameter_names("heterogeneous_history", True, False)
144 estimator_draws = est.make_draws(8, 306, "heterogeneous_history")
145 np.testing.assert_array_equal(draws, estimator_draws)
146 expected_ll = est.household_loglike(np.array([p[n] for n in names]), data,
147                                     estimator_draws, "heterogeneous_history")
148 np.testing.assert_allclose(pricing.logsumexp(scalar, axis=1)-np.log(len(draws)),
149                            expected_ll, atol=1e-12)
```

Read this block:

- Line 142, `data`: Set this name to the result of calling `est.make_arrays(...)`; the arguments and preceding method chain are shown above.
- Line 143, `names`: Set this name to the result of calling `est.parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 144, `estimator_draws`: Set this name to the result of calling `est.make_draws(...)`; the arguments and preceding method chain are shown above.
- Line 145: call `np.testing.assert_array_equal` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 146, `expected_ll`: Set this name to the result of calling `est.household_loglike(...)`; the arguments and preceding method chain are shown above.
- Line 148: call `np.testing.assert_allclose` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.assert_array_equal`: Check that corresponding array entries agree exactly, appropriate for integer identities/counts.

Source lines 151–152:

```
151 u = np.array([.3, -.2, .7, 0.])
152 probs = np.exp(u-pricing.logsumexp(u))
```

Read this block:

- Line 151, `u`: Set this name to the result of calling `np.array(...)`; the arguments and preceding method chain are shown above.
- Line 152, `probs`: Set this name to the result of calling `np.exp(...)`; the arguments and preceding method chain are shown above.

Source lines 153–155:

```
153 for focal in range(3):
154     inclusive = pricing.logsumexp(np.delete(u, focal))
155     self.assertAlmostEqual(pricing.expit(u[focal]-inclusive), probs[focal])
```

Read this block:

- Line 153: for each entry yielded by `range(3)`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 154, `inclusive`: Set this name to the result of calling `pricing.logsumexp(...)`; the arguments and preceding method chain are shown above.
- Line 155: call `self.assertAlmostEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `range`: Create a loop range whose stop is excluded; range(3) visits 0, 1, 2.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

SplitPosteriorTests.test three focal cases both type treatments

Original function: `SplitPosteriorTests.test_three_focal_cases_both_type_treatments`. Source begins at line 157.

Checks all focal permutations under known posterior means and persistent posterior uncertainty. Generated arrays preserve probability mass and dynamic revenue weakly dominates no coupons; the simple runner blocks all CSV reads. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 157–157:

```
157 def test_three_focal_cases_both_type_treatments(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 158–160:

```
158 frame, parameters = self.fixture()
159 households, weights, alpha, history, beta, _ = pricing.posterior(
160     frame, parameters, pricing.persistent_draws(8, 306), False)
```

Read this block:

- Line 158, `(frame, parameters)`: Unpack the result of calling `self.fixture(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 159, `(households, weights, alpha, history, beta, _)`: Unpack the result of calling `pricing.posterior(...)`; the arguments and preceding method chain are shown above into these targets in order.

Source lines 161–175:

```
161 for focal in pricing.FLAVORS:
162     forecast = frame.copy()
163     intercepts = beta.copy()
164     if focal != 'berry':
165         for destination, source in {'berry': focal, focal: 'berry'}.items():
166             for prefix in ['', 'price_', 'stock_', 'novelty_']:
167                 forecast[prefix+destination] = frame[prefix+source]
168             intercepts[pricing.FLAVORS.index(destination)] = beta[pricing.FLAVORS.index(source)]
169     context = pricing.forecast_context(forecast, households.copy())
170     for treatment in ['posterior-mean', 'posterior-integrated']:
171         table, error = pricing.targeted_coupons(context, households, weights, alpha,
172             history, intercepts, np.array([0., .5, 1.]), capacity=2,
173             delta=.92, type_treatment=treatment)
174         self.assertLess(error, 1e-12)
175         self.assertTrue((table.revenue_change >= -1e-10).all())
```

Read this block:

- Line 161: for each entry yielded by `pricing.FLAVORS`, bind/unpack it into `focal` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 162, `forecast`: Set this name to the result of calling `frame.copy(...)`; the arguments and preceding method chain are shown above.
- Line 163, `intercepts`: Set this name to the result of calling `beta.copy(...)`; the arguments and preceding method chain are shown above.
- Line 164: test `focal != 'berry'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 165: for each entry yielded by `{'berry': focal, focal: 'berry'}.items()`, bind/unpack it into `(destination, source)` and run the indented body. Changes inside an iteration can feed subsequent iterations.

- Line 166: for each entry yielded by `['', 'price_', 'stock_', 'novelty_']`, bind/unpack it into `prefix` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 167, `forecast[prefix + destination]`: Store the entry/slice selected by `frame[prefix + source]`; brackets select from the object to their left in the selected entry/column.
- Line 168, `intercepts[pricing.FLAVORS.index(destination)]`: Store the entry/slice selected by `beta[pricing.FLAVORS.index(source)]`; brackets select from the object to their left in the selected entry/column.
- Line 169, `context`: Set this name to the result of calling `pricing.forecast_context(...)`; the arguments and preceding method chain are shown above.
- Line 170: for each entry yielded by `['posterior-mean', 'posterior-integrated']`, bind/unpack it into `treatment` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 171, `(table, error)`: Unpack the result of calling `pricing.targeted_coupons(...)`; the arguments and preceding method chain are shown above into these targets in order.
- Line 174: call `self.assertLess` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 175: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.copy`: Make an independent container before changing it; plain assignment would give the original object another name.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.assertLess`: Fail unless the first value is strictly smaller than the second.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.all`: Reduce Boolean values to whether every value is true; with axis=1 the result has one entry per row.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

../final_simple/checks/test_regenerate_weekly_inputs.py

Mocked regeneration checks. Synthetic records supply saved settings and parameters; mocked preparation/CSV loading allows the checks to run while CSV readers are blocked. Success preserves fitted artifacts and a likelihood mismatch preserves any old export.

Module setup and direct-execution blocks

Source lines 1–6:

```
"""Recovery control flow tests with generated fixtures only."""
import json
from pathlib import Path
import tempfile
import unittest
from unittest.mock import patch
```

Read this block:

- Line 1: this string is documentation when placed first in the body. It describes behavior but does not itself perform the stated calculation.
- Line 2: import `json`. An as-name is a convenient local alias, not another library.
- Line 3: from `pathlib`, import `Path` into this module's namespace. Importing can execute the imported module's top-level setup.
- Line 4: import `tempfile`. An as-name is a convenient local alias, not another library.
- Line 5: import `unittest`. An as-name is a convenient local alias, not another library.

- Line 6: from `unittest.mock`, import `patch` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 8–10:

```
8 import numpy as np
9 import pandas as pd
10 from demand import regenerate_weekly_inputs as recovery
```

Read this block:

- Line 8: import `numpy as np`. An as-name is a convenient local alias, not another library.
- Line 9: import `pandas as pd`. An as-name is a convenient local alias, not another library.
- Line 10: from `demand`, import `regenerate_weekly_inputs as recovery` into this module's namespace. Importing can execute the imported module's top-level setup.

Source lines 13–13:

```
13 class RecoveryTests(unittest.TestCase):
```

Read this block:

- Line 13: define class `RecoveryTests` and its methods. TestCase inheritance provides the self.assert… checking methods.

Source lines 54–55:

```
54 if __name__ == '__main__':
55     unittest.main()
```

Read this block:

- Line 54: test `__name__ == '__main__'`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 55: call `unittest.main` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

RecoveryTests.check recovery

Original function: `RecoveryTests.check_recovery`. Source begins at line 14.

Builds temporary synthetic fit records and mocks data preparation/CSV loading. It checks successful publication or a likelihood mismatch, and verifies saved fitted parameters are preserved. The mock is a controlled substitute, not evidence that an actual server dataset has already matched.

Definition and stated intent:

Source lines 14–14:

```
14 def check_recovery(self, mismatch):
```

`def` creates this callable; the parentheses name its inputs (`self`, `mismatch`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 15–45:

```
15 with tempfile.TemporaryDirectory() as temporary:
16     directory = Path(temporary)
17     fit = dict(model='heterogeneous_history', households=1, weeks=2, nll=3., settings=dict(
18         sample='/server/synthetic.parquet', first_year=2006, last_year=2015,
19         max_households=None, capacity=7, no_price_control=False,
20         household_dma_column='dma_cd', habit_weight=.7, draws=4, seed=306))
21     (directory/'fit.json').write_text(json.dumps(fit))
22     names = recovery.parameter_names('heterogeneous_history', True, True)
23     pd.DataFrame(dict(parameter=names, estimate=np.ones(len(names)))).to_csv(directory/'point_estimates.csv',
    ↪  index=False)
```

```
    before = {p.name: p.read_bytes() for p in directory.iterdir()}
    frame = pd.DataFrame({'synthetic_row': [1, 2]})
    data = dict(n_households=1, outside=True, control=True)
    with patch.object(recovery, 'prepare_data', return_value=frame) as prepare, \
            patch.object(recovery, 'add_price_controls', return_value=(frame, None)) as controls, \
            patch.object(recovery, 'make_arrays', return_value=data), \
            patch.object(recovery, 'household_loglike', return_value=np.array([-4. if mismatch else -3.])), \
            patch.object(recovery.pd, 'read_csv', return_value=pd.DataFrame(dict(parameter=names,
    estimate=np.ones(len(names))))):
        if mismatch:
            (directory/'weekly_inputs.csv').write_text('existing export')
            with self.assertRaisesRegex(ValueError, 'likelihood'):
                recovery.regenerate(directory)
            self.assertEqual((directory/'weekly_inputs.csv').read_text(), 'existing export')
            self.assertFalse((directory/'weekly_inputs_regeneration.json').exists())
        else:
            report = recovery.regenerate(directory)
            self.assertFalse(report['demand_refitted'])
            self.assertTrue((directory/'weekly_inputs.csv').exists())
        controls.assert_called_once()
        self.assertEqual(prepare.call_args.args[0].sample, Path('/server/synthetic.parquet'))
    for name, content in before.items():
        self.assertEqual((directory/name).read_bytes(), content)
```

Read this block:

- Line 15: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 16, `directory`: Set this name to the result of calling `Path(...)`; the arguments and preceding method chain are shown above.
- Line 17, `fit`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 21: call `(directory / 'fit.json').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 22, `names`: Set this name to the result of calling `recovery.parameter_names(...)`; the arguments and preceding method chain are shown above.
- Line 23: call `.to_csv` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 24, `before`: Set this name to a dictionary computed one key/value pair at a time by the comprehension.
- Line 25, `frame`: Set this name to the result of calling `pd.DataFrame(...)`; the arguments and preceding method chain are shown above.
- Line 26, `data`: Set this name to the result of calling `dict(...)`; the arguments and preceding method chain are shown above.
- Line 27: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 32: test `mismatch`. Execute the indented branch only when it is true; an else/elif branch handles its alternative.
- Line 33: call `(directory / 'weekly_inputs.csv').write_text` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 34: enter the displayed context manager(s). An as-name receives the temporary resource; leaving the block restores/closes/cleans it according to that manager, including ordinary exception exits.
- Line 35: call `recovery.regenerate` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 36: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 37: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 39, `report`: Set this name to the result of calling `recovery.regenerate(...)`; the arguments and

preceding method chain are shown above.
- Line 40: call `self.assertFalse` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 41: call `self.assertTrue` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 42: call `controls.assert_called_once` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 43: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.
- Line 44: for each entry yielded by `before.items()`, bind/unpack it into `(name, content)` and run the indented body. Changes inside an iteration can feed subsequent iterations.
- Line 45: call `self.assertEqual` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Syntax and calls used here:

- `.TemporaryDirectory`: Create temporary working storage whose context manager cleans it up when the block exits.
- `Path`: Construct a pathlib filesystem-path object, enabling / joining and path methods.
- `dict`: Build a key/value mapping, often by pairing parameter names with numeric coefficients.
- `.write_text`: Write a string into a text file, replacing an existing file at that path.
- `.to_csv`: Serialize a pandas table as CSV; index=False avoids writing row labels as an extra column.
- `.items`: Iterate through dictionary key/value pairs together, allowing two-name unpacking in a loop.
- `.dumps`: Serialize Python dictionaries/lists/scalars into a JSON string, with optional readable indentation.
- `.assertEqual`: Fail this unittest method unless actual and expected objects are equal.
- `.assertFalse`: Fail unless the supplied condition is false, checking an expected absence/invalidity.
- `.assertTrue`: Fail unless the supplied condition is true; array conditions normally need any/all first.
- `.array`: Create a NumPy array; nested equal-length lists form rows and columns.
- `.assertRaisesRegex`: Expect a particular exception and matching error text inside the with block; no exception is a test failure.
- `.exists`: Test whether a filesystem path exists; this is not a schema/content validity check.
- `len`: Count elements/rows rather than summing their numerical values.
- The brace-enclosed key:value for-expression builds a dictionary, rather than an ordinary list.
- The expression a if condition else b chooses one value; unlike a full if statement it fits inside another expression.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

RecoveryTests.test recovery preserves parameters

Original function: `RecoveryTests.test_recovery_preserves_parameters`. Source begins at line 47.

Exercises successful weekly-export regeneration with mocks and confirms fit/parameter records remain byte-for-byte unchanged. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 47–47:

```
47 def test_recovery_preserves_parameters(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 48–48:

```
48 self.check_recovery(False)
```

Read this block:

- Line 48: call `self.check_recovery` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

RecoveryTests.test mismatch preserves existing export

Original function: `RecoveryTests.test_mismatch_preserves_existing_export`. Source begins at line 50.

Forces a reconstructed-likelihood mismatch and checks no replacement weekly export or success record is published. The assertions fail automatically when the stated property does not hold; passing synthetic checks does not establish empirical fit quality.

Definition and stated intent:

Source lines 50–50:

```
50 def test_mismatch_preserves_existing_export(self):
```

`def` creates this callable; the parentheses name its inputs (`self`). A displayed `=value` supplies a default, and the colon begins its indented body.

Source lines 51–51:

```
51 self.check_recovery(True)
```

Read this block:

- Line 51: call `self.check_recovery` with the displayed arguments. No assignment stores its return here; its documented side effect or assertion is what matters.

Output reminder: reaching the end returns `None`. The useful result here is assertions, printing, plotting, writing, or another documented side effect.

Coverage, rebuilding, and next steps

This edition includes 94 Python files, 243 named functions/methods (including nested helpers), and 8,519 source lines. Every AST statement start is accounted for in the annotated listings. Empty files and top-level scripts are included. Lambda expressions are explained at their use sites rather than counted as named functions. Large raw LaTeX strings are reproduced as data; their mathematical content is discussed in the separate economic guide.

The build reads Python files as text and parses their syntax; it never imports or executes the private-data processing scripts. The companion JSON manifest records source hashes, function names/lines, and statement coverage. If source changes, rebuild the book so printed line references match the new snapshot. A newly added function requires a teaching note before the build succeeds.

Rebuild from the repository's Code directory:

```
python3 final_draft/build_python101.py
```

Use `--markdown-only` to regenerate the readable source/coverage manifest without invoking Pandoc/XeLaTeX. The source lessons are in `final_draft/python101/`. The generated Markdown is `final_draft/PYTHON101_WALKTHROUGH.md`.

Keep this book beside the separate `complete_code_guides.pdf`: use that guide for model derivations and this one for what the corresponding Python actually says. Try changing tiny test fixtures or tracing values on paper before editing the large server pipelines.